Python for computational Problem Solving · Unit_01 · Unit 1: Introduction

DRAFT — UE26CS151A · Unit_01 — Unit 1: Introduction · Complete Unit Master

Source-complete searchable master · transcript, slide text, page visuals and archive references

Hugging Face PESU_Materials source tree, official units.json catalogue, transcript sidecars, slide text/PDF/PPT sources and the selected First_Year_8enterprise archive; see the provenance datasets in the release ZIP. Generated 18 October 2026.

· How to use this pack

Search this single unit master by lesson, timestamp, slide text or source page. Read the transcript in timestamp order, then compare extracted slide text with its page-faithful source figure. Formula candidates and archive question mappings are explicitly flagged for review. Topics include the applicable parts: overview, key terms, formulas, figures, flowcharts, important points, worked examples, past-paper questions and a recap. IDs are shared with the separate documents: E = formula, G = graph, F = flowchart, W = worked example. Each ID is numbered by topic, so E4.2 is the second formula of topic 4.

Source text	Verbatim or OCR-derived text, kept separate from editorial notes.
Candidate	Automated topic/source association requiring manual confirmation.
Low ASR confidence	Transcript segment requires audio verification.
DRAFT	Human review remains incomplete.

Contents

1. 1 · UE26CS151A · Unit_01: Unit 1: Introduction
2. 2 · Introduction
3. 3 · Man Cabbage Goat Wolf Problem
4. 4 · Process of Computation Problem Solving - Analysis, Design, Implementation, Testing
5. 5 · Process of Computation Problem Solving - Analysis, Design, Implementation, Testing
6. 6 · Computer Hardware and software
7. 7 · Computer hardware
8. 8 · Program Structure
9. 9 · Program structure
10. 10 · Program structure
11. 11 · print function
12. 12 · Variable video 1
13. 13 · variable and id function
14. 14 · input function
15. 15 · input function
16. 16 · Arithmetic, membership and relational operators
17. 17 · Bitwise operators
18. 18 · logical operators and short circuit evaluation
19. 19 · operators and expressions
20. 20 · sequential, selection control
21. 21 · iterative control structure
22. 22 · String Operations
23. 23 · Source slide deck: UE23CS151A PCPS Unit 3 Sowmya Shree Sindhu Class 83 84 85 Modules
24. 24 · Source slide deck: UE24CS151A PCPS Unit 1 SindhuRPai Class 6 Intro to Programming Languages

25. 25 · Source slide deck: UE24CS151A PCPS Unit 1 SindhuRPai Class 7 Syntax semantics translation models
26. 26 · Source slide deck: UE24CS151A PCPS Unit 1 SindhuRPai Class 8 Installation and execution
27. 27 · Source slide deck: IDLE
28. 28 · Source slide deck: UE24CS151A PCPS Unit 1 SindhuRPai Class 10 variable id type
29. 29 · Source slide deck: Operator Polymorphism
30. 30 · Source slide deck: Operator precedence and associativity
31. 31 · Source slide deck: UE24CS151A PCPS Unit 2 SindhuRPai Class 1 General intro to data structures in python
32. 32 · Source slide deck: UE24CS151A PCPS Unit 2 SindhuRPai Class 2 lists
33. 33 · Historical question-source pages · unit mapping candidates
34. Appendix A · Symbol glossary
35. Appendix B · Sources, coverage and gaps
36. Appendix C · Revision and retrieval practice

M1

Unit_01 · Unit 1: Introduction

1 UE26CS151A · Unit_01: Unit 1: Introduction

This unit master organizes source material for Python for computational Problem Solving, Unit_01, officially titled Unit 1: Introduction. It includes 21 video transcripts and 20 source slide decks, with page-level text and rendered images where available. This draft preserves source wording and page order; it does not claim a complete human subject review or full audiovisual review. EV0000001

SHORT-NOTE RECAP

Official catalogue entry: UE26CS151A, Unit_01 (Unit 1: Introduction). Bucket folder present: yes. Videos: 21; slide decks: 20.

EV0000001

2 Introduction

This lesson is indexed from the source video titled Introduction. Its complete timestamped transcript is preserved below in segment order. The original video path and content hash are recorded in datasets/lecture-corpus.json. Transcript text is source wording; it has not been silently corrected or rewritten. A candidate slide deck is grouped here according to the current matching score; the exact path and page locators are listed in the source evidence. EV0000002 · EV0000003

Graphs and diagrams

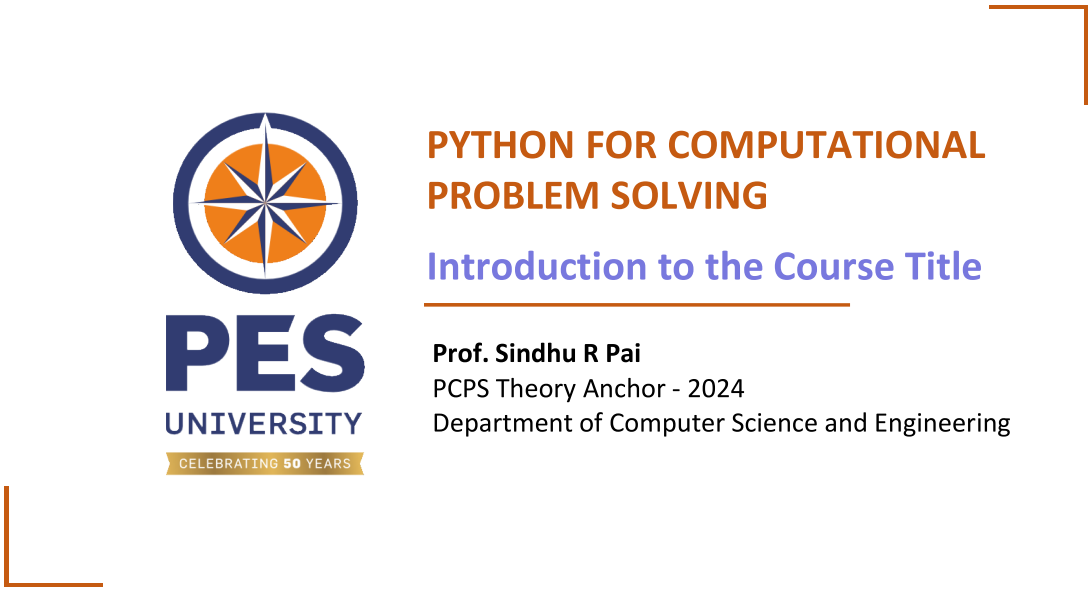


Figure G000031. UE24CS151A PCPS Unit 1 SindhuRPai Class 1 2 Intro to title · source slide page 1.

EV0000421

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

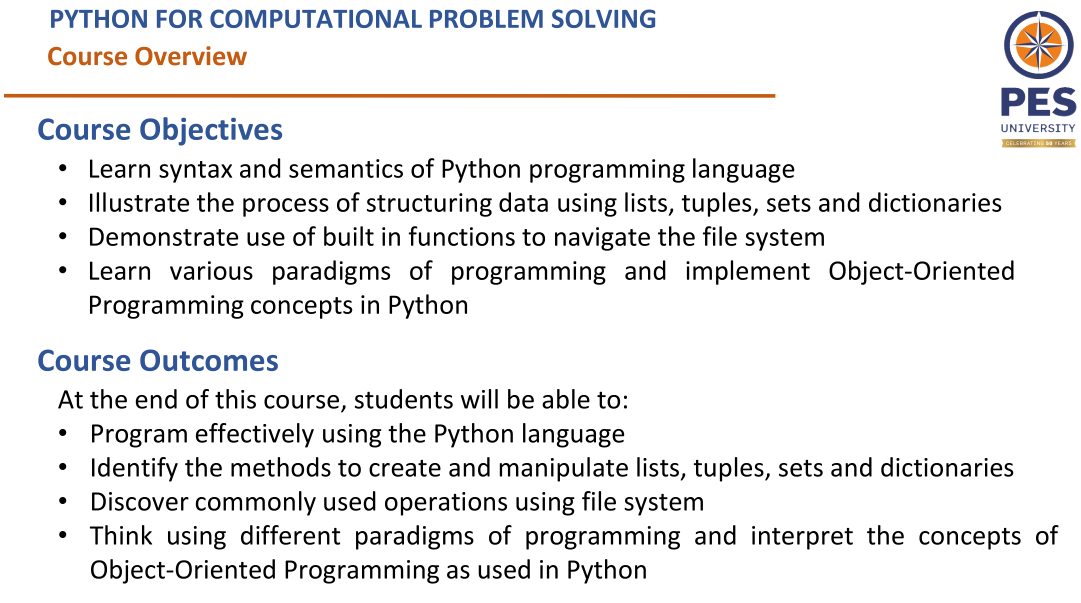


Figure G000032. UE24CS151A PCPS Unit 1 SindhuRPai Class 1 2 Intro to title · source slide page 2.

EV0000424

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

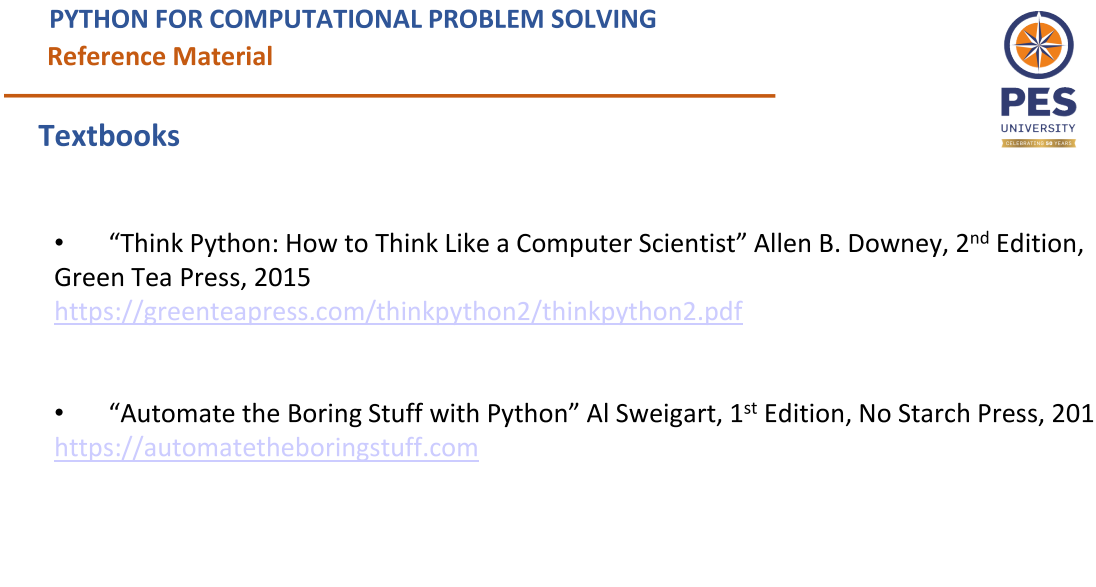


Figure G000033. UE24CS151A PCPS Unit 1 SindhuRPai Class 1 2 Intro to title · source slide page 3.

EV0000427

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

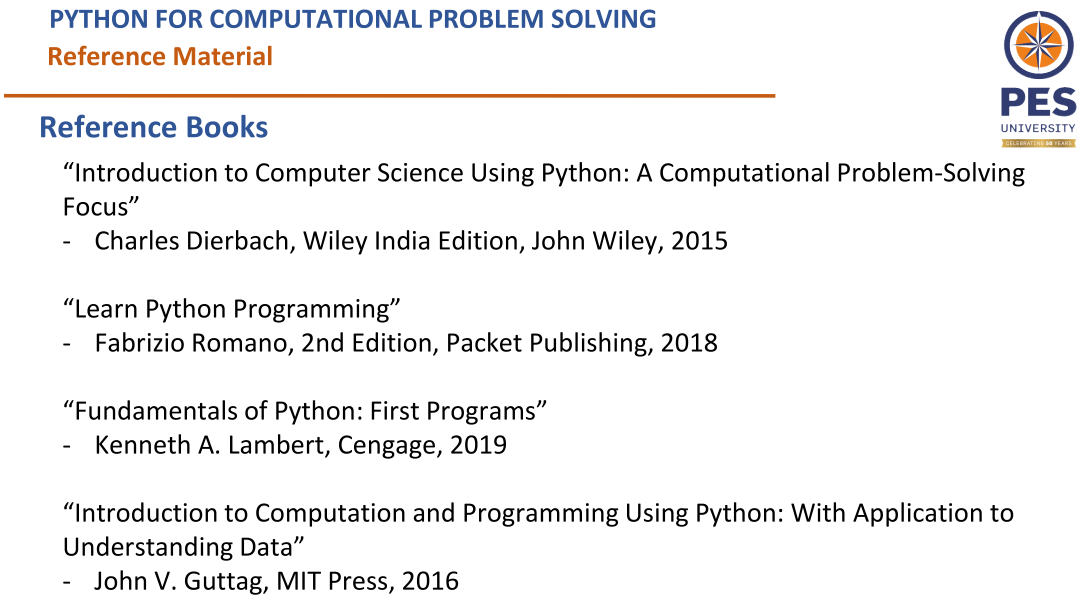


Figure G000034. UE24CS151A PCPS Unit 1 SindhuRPai Class 1 2 Intro to title · source slide page 4.

EV0000430

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING
Introduction

PES UNIVERSITY

What does a computer do?

Fundamentally performs calculations, remembers results

What types of calculations can a computer do?

- Basic built-in operations like simple arithmetic and logic
- More complex operations derived from the built-in operations

5

Figure G000035. UE24CS151A PCPS Unit 1 SindhuRPai Class 1 2 Intro to title · source slide page 5.

EV0000433

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

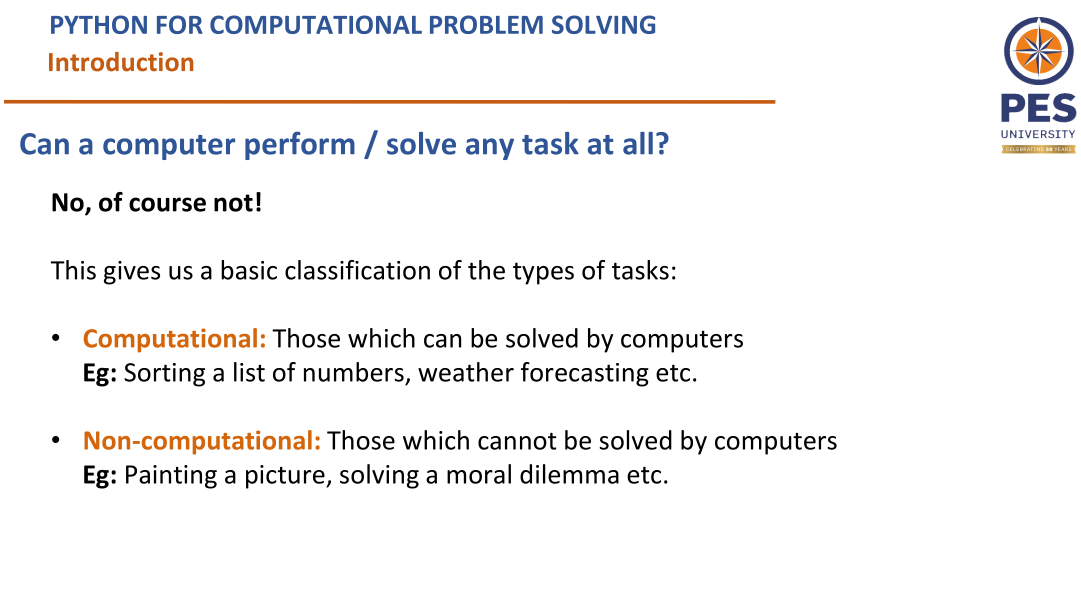


Figure G000036. UE24CS151A PCPS Unit 1 SindhuRPai Class 1 2 Intro to title · source slide page 6.

EV0000436

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

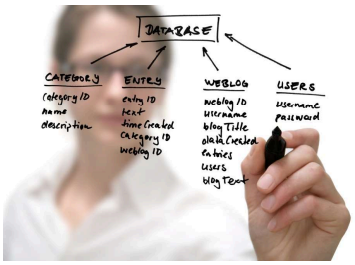
Computational Problem

PES UNIVERSITY

So what exactly makes a problem computational?

Two main aspects to all computational problems:

- **A representation** that captures all the relevant aspects of the problem.
- **An algorithm** that solves the problem by use of the representation. A sequence of unambiguous instructions for solving a problem, i.e.,for obtaining a required output for any legitimate input in a finite amount of time. The word "algorithm" is derived from the name of a ninth century Arab mathematician, Al-Khwarizmi.



7

Figure G000037. UE24CS151A PCPS Unit 1 SindhuRPai Class 1 2 Intro to title · source slide page 7.

EV0000439

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

MCGW Problem

PES UNIVERSITY

Problem Statement: Man, Cabbage, Goat, Wolf Problem

A man lives on the east side of a river. He wishes to bring a cabbage, a goat, and a wolf to a village on the west side of the river to sell. However, **his boat is only big enough to hold himself, and either the cabbage, goat, or wolf**. In addition, the man **cannot leave the goat alone with the cabbage** because the goat will eat the cabbage, and he **cannot leave the wolf alone with the goat** because the wolf will eat the goat. How does the man solve his problem?

8

Figure G000038. UE24CS151A PCPS Unit 1 SindhuRPai Class 1 2 Intro to title · source slide page 8.

EV0000442

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

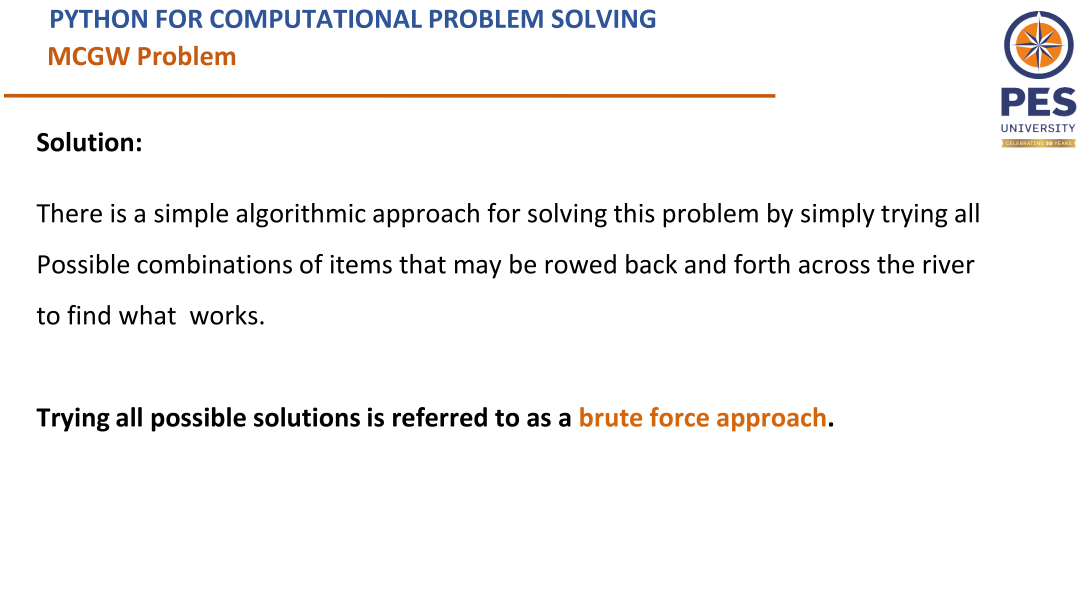


Figure G000039. UE24CS151A PCPS Unit 1 SindhuRPai Class 1 2 Intro to title · source slide page 9.

EV0000445

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

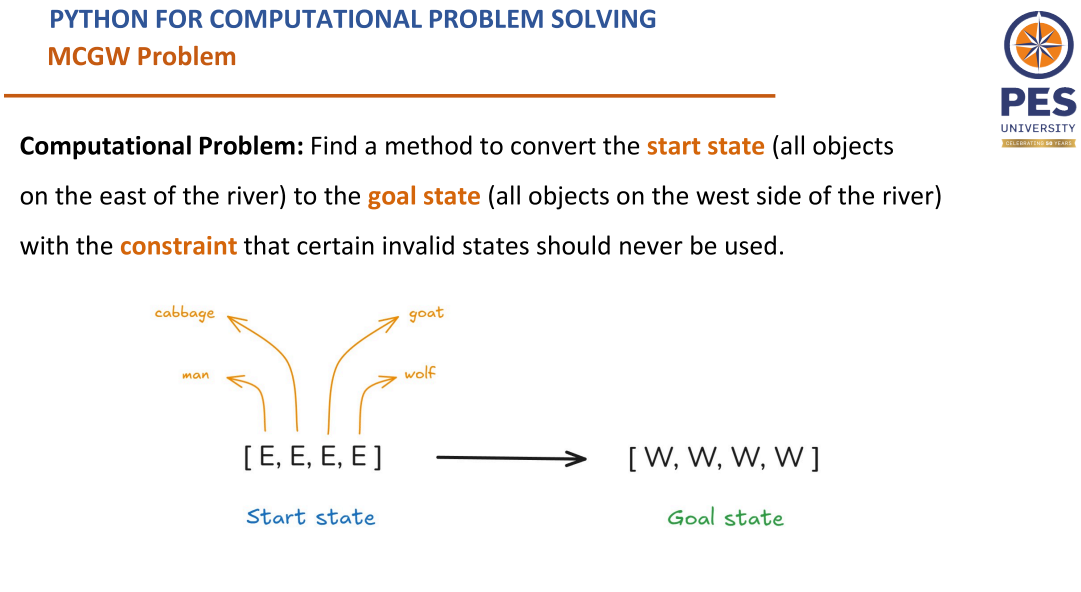


Figure G000040. UE24CS151A PCPS Unit 1 SindhuRPai Class 1 2 Intro to title · source slide page 10.

EV0000448

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

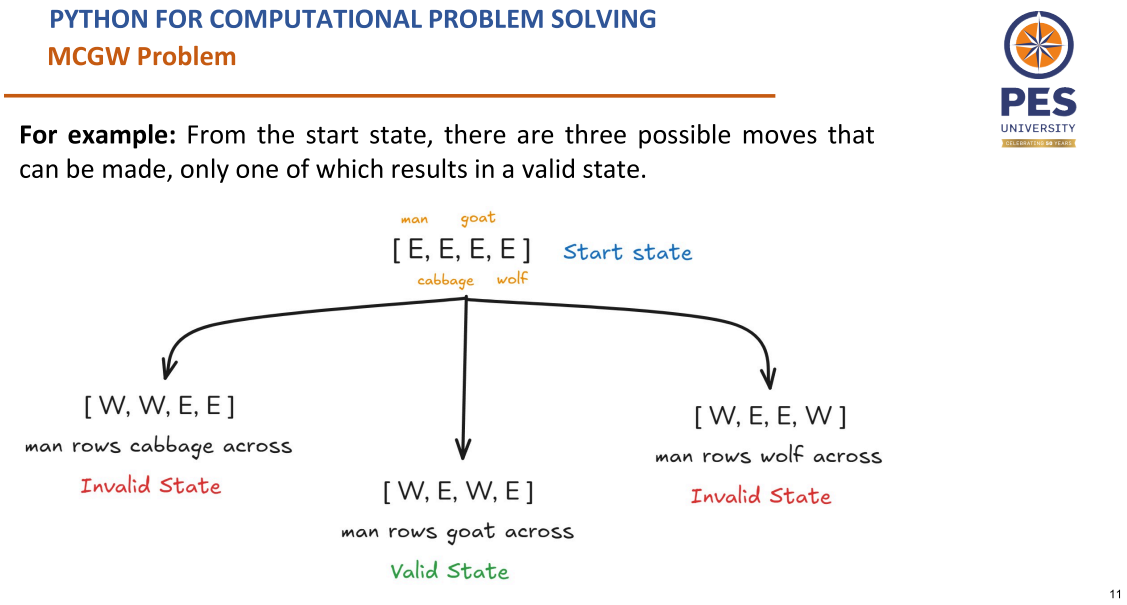


Figure G000041. UE24CS151A PCPS Unit 1 SindhuRPai Class 1 2 Intro to title · source slide page 11.

EV0000451

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

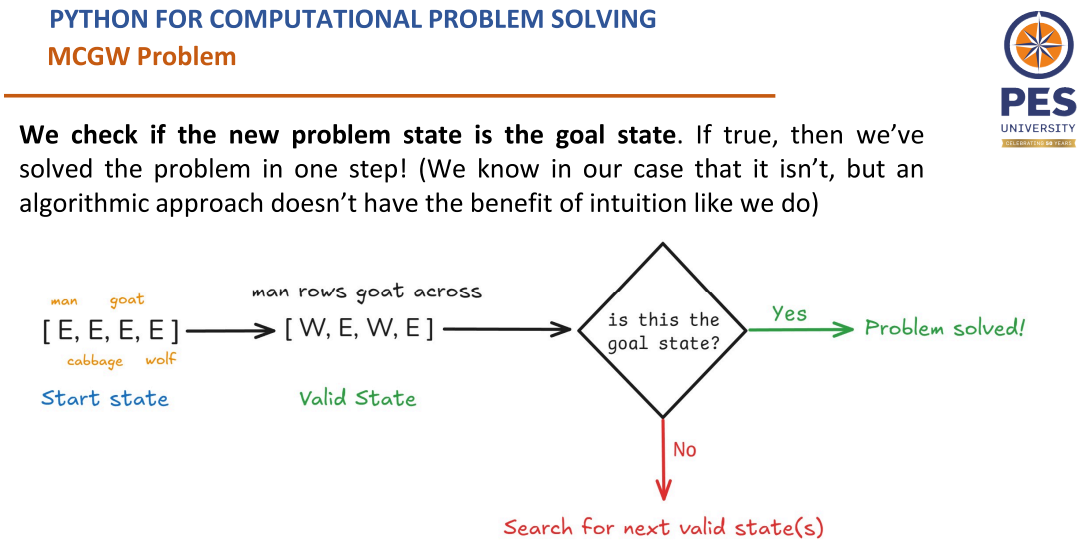


Figure G000042. UE24CS151A PCPS Unit 1 SindhuRPai Class 1 2 Intro to title · source slide page 12.

EV0000454

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

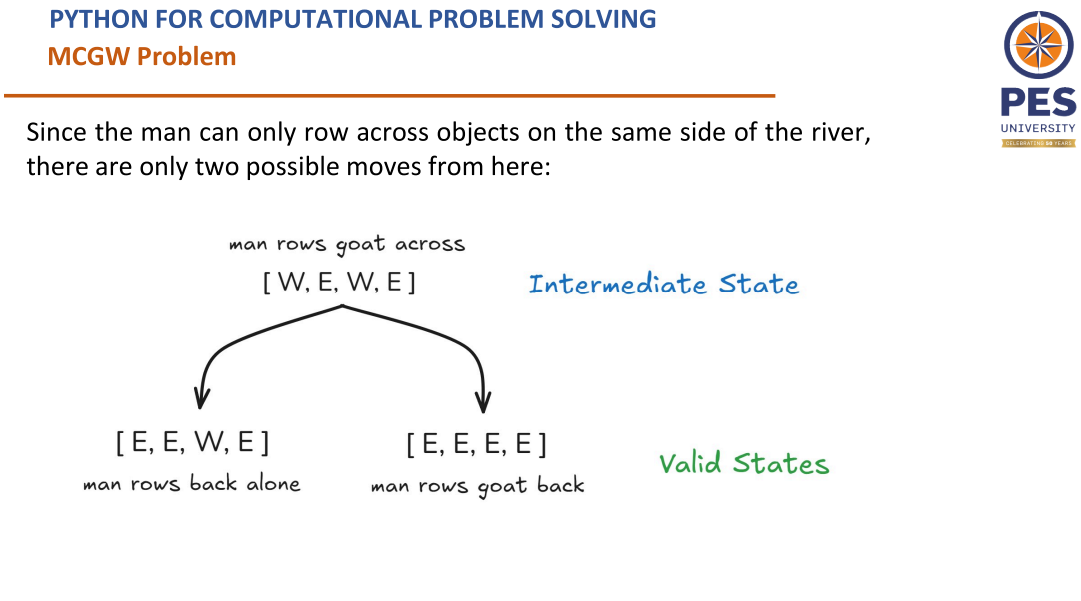


Figure G000043. UE24CS151A PCPS Unit 1 SindhuRPai Class 1 2 Intro to title · source slide page 13.

EV0000457

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

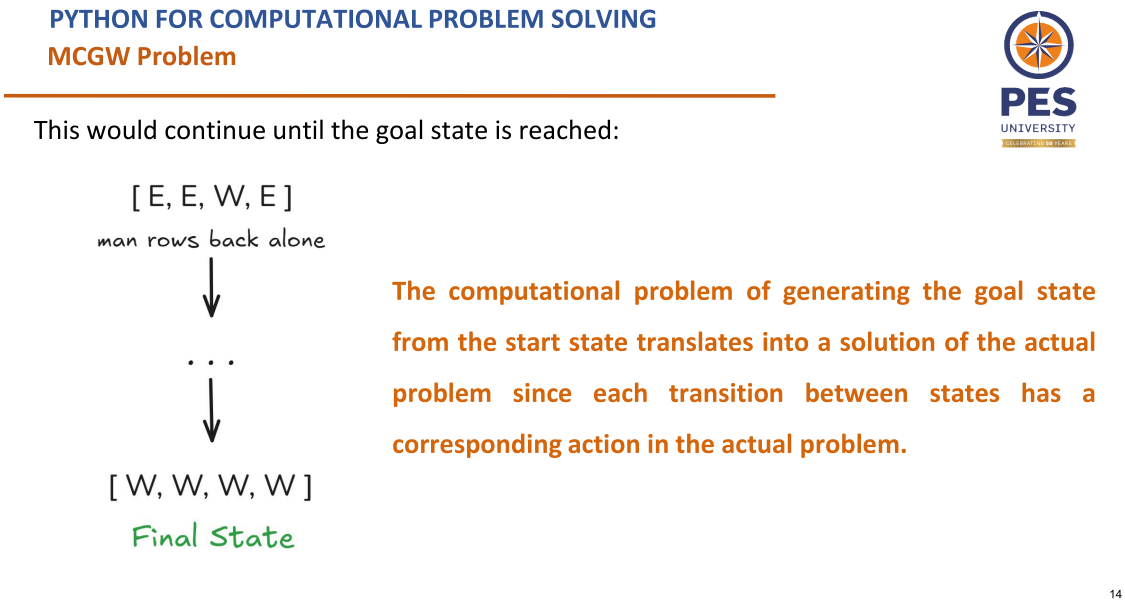


Figure G000044. UE24CS151A PCPS Unit 1 SindhuRPai Class 1 2 Intro to title · source slide page 14.

EV0000460

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

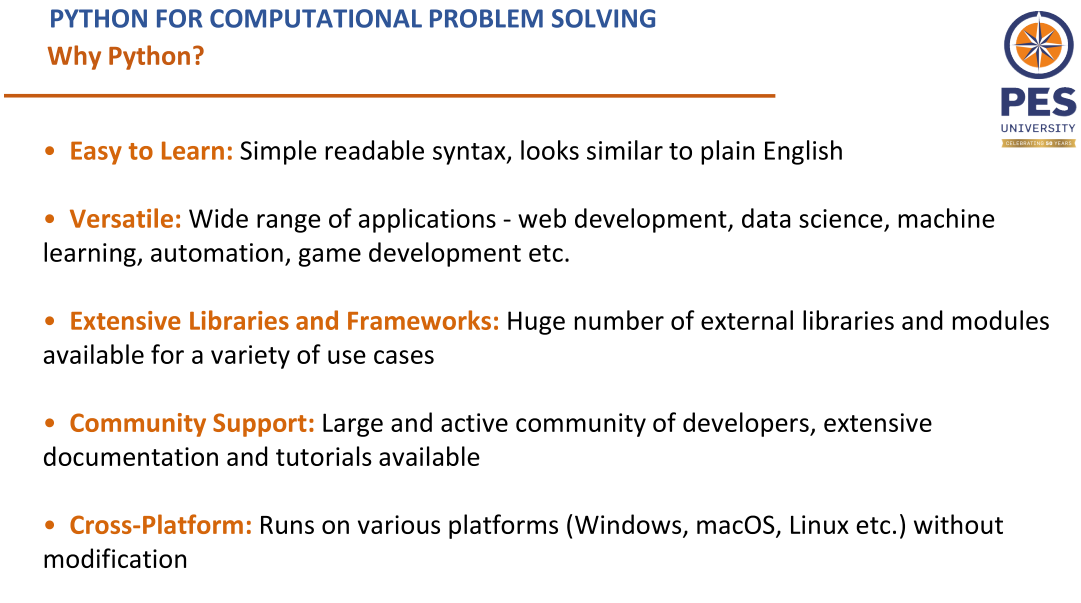


Figure G000045. UE24CS151A PCPS Unit 1 SindhuRPai Class 1 2 Intro to title · source slide page 15.

EV0000463

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

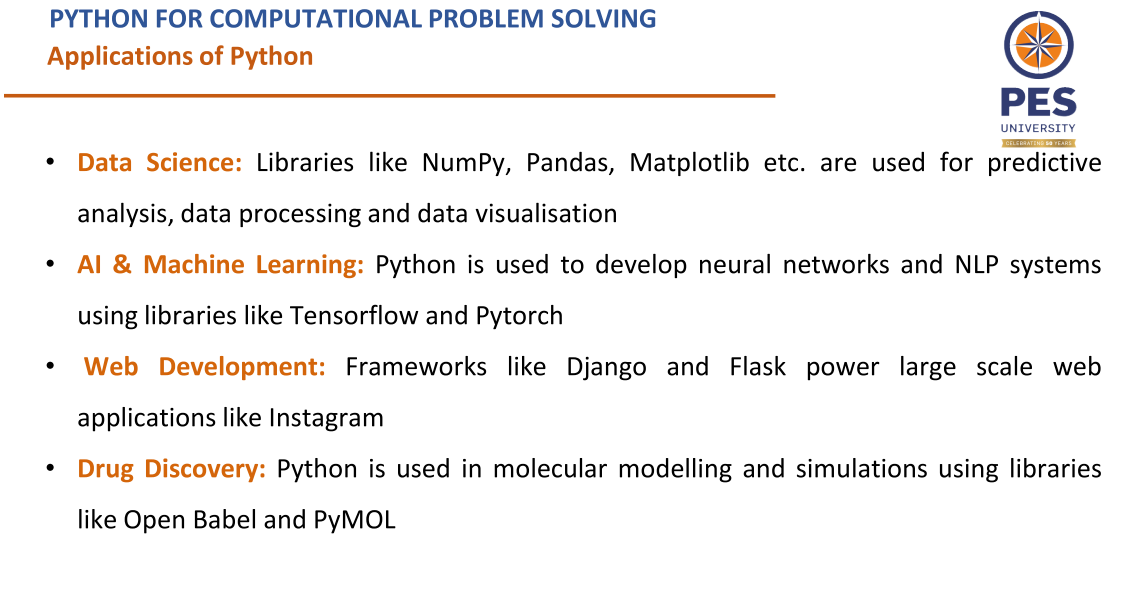


Figure G000046. UE24CS151A PCPS Unit 1 SindhuRPai Class 1 2 Intro to title · source slide page 16.

EV0000466

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

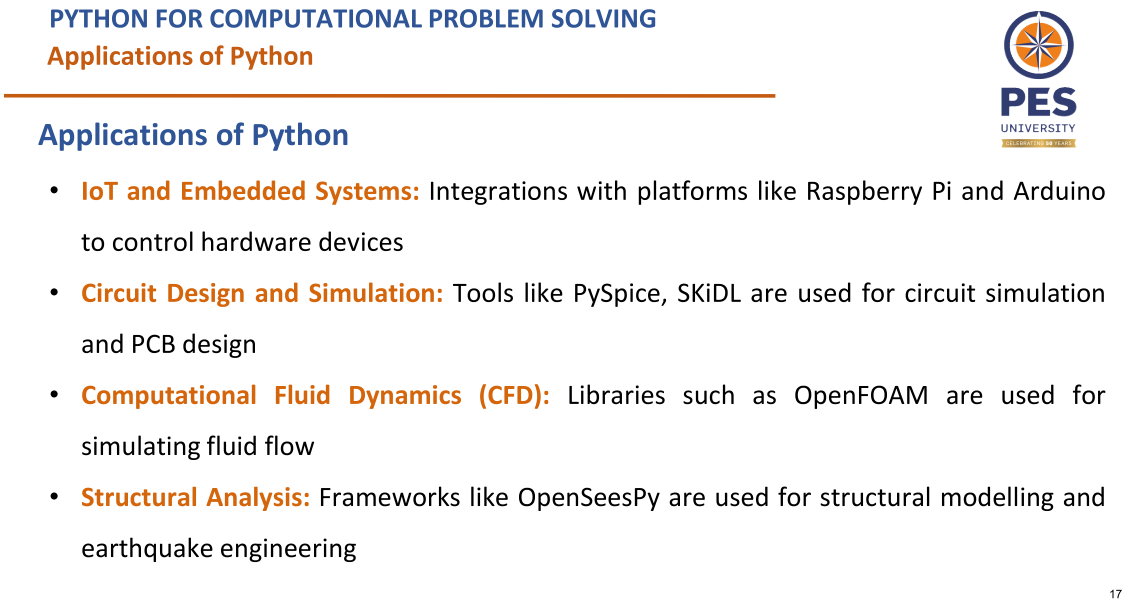


Figure G000047. UE24CS151A PCPS Unit 1 SindhuRPai Class 1 2 Intro to title · source slide page 17.

EV0000469

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

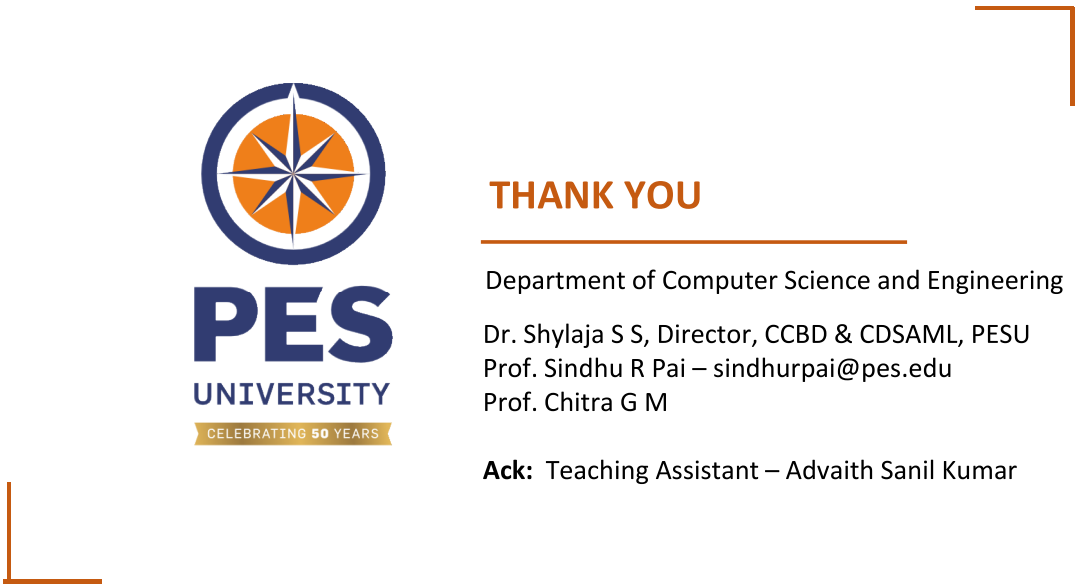


Figure G000048. UE24CS151A PCPS Unit 1 SindhuRPai Class 1 2 Intro to title · source slide page 18.

EV0000472

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Important points and why

1. Source slide page 1 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Introduction to the Course Title Prof. Sindhu R Pai PCPS Theory Anchor - 2024 Department of Computer Science and Engineering EV0000420 · EV0000421 · EV0000422
2. Source slide page 2 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Course Overview Course Objectives • Learn syntax and semantics of Python programming language • Illustrate the process of structuring data using lists, tuples, sets and dictionaries • Demonstrate use of built in functions to navigate the file system • Learn various paradigms of programming and implement Object-Oriented Programming concepts in Python Course Outcomes At the end of this course, students will be able to: • Program effectively using the Python language • Identify the methods to create and manipulate lists, tuples, sets and dictionaries • Discover commonly used operations using file system • Think using different paradigms of programming and interpret the concepts of Object-Oriented Programming as used in Python 2 EV0000423 · EV0000424 · EV0000425

3. Source slide page 3 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Reference Material Textbooks • "Think Python: How to Think Like a Computer Scientist" Allen B. Downey, 2nd Edition, Green Tea Press, 2015 https://greenteapress.com/thinkpython2/thinkpython2.pdf • "Automate the Boring Stuff with Python" Al Sweigart, 1st Edition, No Starch Press, 2015 https://automatetheboringstuff.com 3 EV0000426 · EV0000427 · EV0000428
4. Source slide page 4 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Reference Material Reference Books "Introduction to Computer Science Using Python: A Computational Problem-Solving Focus" - Charles Dierbach, Wiley India Edition, John Wiley, 2015 "Learn Python Programming" - Fabrizio Romano, 2nd Edition, Packet Publishing, 2018 "Fundamentals of Python: First Programs" - Kenneth A. Lambert, Cengage, 2019 "Introduction to Computation and Programming Using Python: With Application to Understanding Data" - John V. Guttag, MIT Press, 2016 4 EV0000429 · EV0000430 · EV0000431
5. Source slide page 5 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Introduction What does a computer do? Fundamentally performs calculations, remembers results What types of calculations can a computer do? • Basic built-in operations like simple arithmetic and logic • More complex operations derived from the built-in operations 5 EV0000432 · EV0000433 · EV0000434
6. Source slide page 6 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Introduction Can a computer perform / solve any task at all? No, of course not! This gives us a basic classification of the types of tasks: • Computational: Those which can be solved by computers Eg: Sorting a list of numbers, weather forecasting etc. • Non-computational: Those which cannot be solved by computers Eg: Painting a picture, solving a moral dilemma etc. 6 EV0000435 · EV0000436 · EV0000437
7. Source slide page 7 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Computational Problem So what exactly makes a problem computational? Two main aspects to all computational problems: • A representation that captures all the relevant aspectsaspects ofof thethe problemproblem. • An algorithm that solves the problem by use of the representation. A sequence of unambiguous instructions for solving a problem, i.e.,for obtaining a required output for any legitimate input in a finite amount of time. The word "algorithm" is derived from the name of a ninth century Arab mathematician, Al-Khwarizmi. 7 EV0000438 · EV0000439 · EV0000440
8. Source slide page 8 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING MCGW Problem Problem Statement: Man, Cabbage, Goat, Wolf Problem A man lives on the east side of a river. He wishes to bring a cabbage, a goat, and a wolf to a village on the west side of the river to sell. However, his boat is only big enough to hold himself, and either the cabbage, goat, or wolf. In addition, the man cannot leave the goat alone with the cabbage because the goat will eat the cabbage, and he cannot leave the wolf alone with the goat because the wolf will eat the goat. How does the man solve his problem? 8 EV0000441 · EV0000442 · EV0000443
9. Source slide page 9 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING MCGW Problem Solution: There is a simple algorithmic approach for solving this problem by simply trying all Possible combinations of items that may be rowed back and forth across the river to find what works. Trying all possible solutions is referred to as a brute force approach. 9 EV0000444 · EV0000445 · EV0000446

10. Source slide page 10 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING MCGW Problem Computational Problem: Find a method to convert the start state (all objects on the east of the river) to the goal state (all objects on the west side of the river) with the constraint that certain invalid states should never be used. 10 EV0000447 · EV0000448 · EV0000449
11. Source slide page 11 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING MCGW Problem For example: From the start state, there are three possible moves that can be made, only one of which results in a valid state. 11 EV0000450 · EV0000451 · EV0000452
12. Source slide page 12 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING MCGW Problem We check if the new problem state is the goal state. If true, then we've solved the problem in one step! (We know in our case that it isn't, but an algorithmic approach doesn't have the benefit of intuition like we do) 12 EV0000453 · EV0000454 · EV0000455
13. Source slide page 13 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING MCGW Problem Since the man can only row across objects on the same side of the river, there are only two possible moves from here: 13 EV0000456 · EV0000457 · EV0000458
14. Source slide page 14 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING MCGW Problem This would continue until the goal state is reached: TheThe computationalcomputational problemproblem ofof generatinggenerating thethe goalgoal statestate from the start state translates into a solution of the actual problem since each transition between states has a corresponding action in the actual problem. 14 EV0000459 · EV0000460 · EV0000461
15. Source slide page 15 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Why Python? • Easy to Learn: Simple readable syntax, looks similar to plain English • Versatile: Wide range of applications - web development, data science, machine learning, automation, game development etc. • Extensive Libraries and Frameworks: Huge number of external libraries and modules available for a variety of use cases • Community Support: Large and active community of developers, extensive documentation and tutorials available • Cross-Platform: Runs on various platforms (Windows, macOS, Linux etc.) without modification 15 EV0000462 · EV0000463 · EV0000464
16. Source slide page 16 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Applications of Python • Data Science: Libraries like NumPy, Pandas, Matplotlib etc. are used for predictive analysis, data processing and data visualisation • AI & Machine Learning: Python is used to develop neural networks and NLP systems using libraries like Tensorflow and Pytorch • Web Development: Frameworks like Django and Flask power large scale web applications like Instagram • Drug Discovery: Python is used in molecular modelling and simulations using libraries like Open Babel and PyMOL 16 EV0000465 · EV0000466 · EV0000467
17. Source slide page 17 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Applications of Python Applications of Python • IoT and Embedded Systems: Integrations with platforms like Raspberry Pi and Arduino to control hardware devices •• CircuitCircuit DesignDesign andand Simulation:Simulation: ToolsTools likelike PySpicePySpice, SKiDLSKiDL areare usedused forfor circuitcircuit simulationsimulation and PCB design • Computational Fluid Dynamics (CFD): Libraries such as OpenFOAM are used for simulating fluid flow • Structural Analysis: Frameworks like OpenSeesPy are used for structural modelling and earthquake engineering 17 EV0000468 · EV0000469 · EV0000470

18. Source slide page 18 (extracted text): THANK YOU Departmentp of Computerp Science and Engineeringg g Dr. Shylaja S S, Director, CCBD & CDSAML, PESU Prof. Sindhu R Pai – sindhurpai@pes.edu Prof. Chitra G M Ack: Teaching Assistant – Advaith Sanil Kumar EV0000471 · EV0000472 · EV0000473

SHORT-NOTE RECAP

Transcript block 1 (source interval 000000-000299 seconds): [00:00:00.600] Hello everyone, I'm Chitra GM from the Department of Computer Science and Engineering, PES University. [00:00:07.230] The subject that I'm going to be handling is Python for Computational Problem Solving for the first year. [00:00:13.400] As mentioned in the title itself, Computational, we are going to be dealing with the different issues revolving around computational thinking and programming. [00:00:24.960] So what are we going to be doing in this course? What is the takeaway at the end of the course for you? [00:00:30.230] Certainly, we are going to be teaching you programming and in a more specific language called as Python. [00:00:37.460] More importantly, we want you to be more proficient to think computationally, to think algorithmically, at the same time to think the way computer scientists think. [00:00:49.840] What does that mean? It means that given a problem or a challenge in hand, how will you make your computer to solve it for you? [00:01:00.690] How will you define the different stages that are involved in order to solve a problem in such a way that you will not be doing, instead your computer is going to be doing it for you? [00:01:11.590] That's the notion of computational thinking and algorithmic thinking. [00:01:19.300] In order to make our computer to solve a problem for us, in your coming weeks, we are going to be dealing with the following topics. [00:01:29.040] It starts from control structures, which includes your looping concepts, followed by data structures, which includes list, string, tuple, dictionary, sets, and so on. [00:01:42.420] We'll also be playing around files. We will try to work with file manipulation operations. [00:01:48.500] Following with the functions where you can make use of the built-in functions and you can also create your own set of functions and then followed by the object-oriented programming. [00:02:00.800] So, if you want to make your computer to work for you or to solve a problem for you, what does a computer do? [00:02:08.970] Here's my next question. Question seems a dumb question. Definitely a valid question for us. [00:02:18.710] Computers are so amazing, it can do all sorts of work for you. You name any, it can help you out to do it. You can use computer in order to play around, play music, play games. You can ask your computer to search for certain things in case of your World Wide Web. You name anything, it will do it for you. But fundamentally, it can perform only two different things. It can do only two different things for us. [00:02:47.620] One is called as calculation, the other one is, it can remember the results of the calculations. [00:02:54.250] So, what type of calculations a computer can do? [00:02:58.690] Definitely, every computer comes with a built-in operations. [00:03:03.220] Typically, arithmetic. [00:03:05.030] Hope you all know, arithmetic includes addition, subtraction, multiplication, and so on. [00:03:09.450] And followed by, it can also do some simple logical operations. [00:03:14.880] Is that enough? No. [00:03:17.910] So, you can also create many more functions that are required for you. [00:03:22.970] You can create your own set of operations and make your computer to solve a particular task for you. [00:03:31.920] So, when we are talking about the task, the next immediate question would be, [00:03:37.420] can a computer perform any task that exists in this particular world? [00:03:42.090] Can it solve everything? [00:03:44.280] The obvious answer is going to be no. [00:03:47.110] The reason is limitation. [00:03:49.460] There are

limitations. To name a few, limitations could be in terms of your non-availability of the resources that are required for computing or it may be in terms of non-availability of the data required in order to perform certain tasks and so on. [00:04:06.540] Thus, the task or a job that a computer can do can either be classified as a computational or it could be a non-computational. [00:04:23.000] Where the problems can be solved. Non-computational means the problems that cannot be solved. In this semester, we will be dealing with the computational way of solving the problems. In order to solve a problem computationally, we need two different things. One is called as representation and the other one is called as algorithm. So let's look into representation and algorithm in depth. [00:04:53.720] Representation is nothing but capturing all the relevant aspects of the problem. [00:04:58.620] In order to understand it better, let's take up an example.

EV0000002 · EV0000003 · EV0000004 · EV0000005

Transcript block 2 (source interval 000300-000599 seconds): [00:05:04.780] The example is as follows. [00:05:08.620] Assume that you have been allocated a task wherein you have to travel from one city to another city. [00:05:17.280] The mode of transportation that is given to you is flight. [00:05:22.470] In such case, if you want to travel from one city to another city, [00:05:27.160] What is very important for you to know is what is the departure time of the flight? What is the landing time of the flight? What is the cost that is involved to travel from one city to another city? What is the distance between one city to another city and so on? At this point, I am not interested in knowing what is the color of the flight that you will be traveling? Who is the pilot? [00:05:57.620] How many number of flight attendees are there? And so on. Thus, representation means capture all the relevant aspects, leaving out the ones that are irrelevant for us. [00:06:10.730] Let us go and understand what is an algorithm. Let me remind you, you would have come across with the different definitions for your algorithm in your school. [00:06:31.350] Definition that exists, but we would say it as incomplete. [00:06:35.610] So from now onwards, we will be using the standard definition in order to describe an algorithm. [00:06:42.340] So an algorithm is a sequence of unambiguous instructions for solving a problem, that is, for obtaining a required output for any legitimate input in a finite amount of time. [00:06:53.940] The word algorithm is derived by a mathematician called as algorithm. [00:07:01.110] What we will do is, we will take up an example to understand how a problem can be solved. [00:07:14.960] The example that we are going to be looking are basically called as a river crossing problems. [00:07:20.460] One such river crossing problems that we want to deal in today's session is called as man-cabbage-goat-wolf problem, MCGW problem. [00:07:32.420] Let me give you a brief description of the problem and then we will try to solve it. [00:07:40.120] Description goes as follows. [00:07:43.000] A man lives on the east side of a river. [00:07:47.900] He wishes to bring cabbage, goat and oolf to a village on the west side of a river to sell. [00:07:54.660] Which means a man is standing at the one end of the river. [00:08:00.110] And he is carrying three different items with him. The three different items are cabbage, goat, and wolf. [00:08:07.460] His main aim is to travel from one end to the another end so that he can sell all the three different items. [00:08:16.300] In order to go from one end to the another end, he has been given a boat. [00:08:24.970] But there are certain constraints that exist. [00:08:27.730] The constraint is, boat can hold man. Along with a man, it can hold only one item. An item could be cabbage, goat or wolf. In order to these, let's add few more constraints. The constraints are, man cannot leave goat and cabbage together. The reason is, goat will eat the cabbage. At the same time, [00:08:56.980] Goat and Ulf cannot be there together because goat will be eaten by Ulf. [00:09:03.920] So how does a man solve this problem? [00:09:08.890] There are

different solutions that exist for this particular problem. [00:09:13.270] The simplest solution would be try with different possibilities. [00:09:17.840] If you are solving a problem by trying with the different possibilities, [00:09:22.020] then this approach is called as your brute force approach. [00:09:27.250] So, what we will try to do is, we will try to take up this particular problem and solve it by hand, come back and then discuss the actual solution that is there. [00:09:38.060] Hope you are all ready. Let's go solve this problem and then we will come back and continue our discussion in this mode.

EV0000006 · EV0000007 · EV0000008 · EV0000009

Full transcript: 1333 normalized words, 67 timestamped segments, 585.173333 seconds duration. Low-confidence segments flagged: 0.

EV0000002 · EV0000003 · EV0000004 · EV0000005

3 Man Cabbage Goat Wolf Problem

This lesson is indexed from the source video titled Man Cabbage Goat Wolf Problem. Its complete timestamped transcript is preserved below in segment order. The original video path and content hash are recorded in datasets/lecture-corpus.json. Transcript text is source wording; it has not been silently corrected or rewritten. A candidate slide deck is grouped here according to the current matching score; the exact path and page locators are listed in the source evidence. EV0000010 · EV0000011

SHORT-NOTE RECAP

Transcript block 1 (source interval 000000-000299 seconds): [00:00:01.840] Let's continue solving the problem. These problems are called as River Crossing Problems. [00:00:08.000] So, there is a river in between. Man along with those three items, cabbage, goat, sorry, this is man. [00:00:19.990] Man, cabbage, goat and wolf are towards the one end of the river. Their intention is to come to the other end. [00:00:28.790] M, C, G, T. [00:00:31.380] So, there are two different ends. So, I will mention one end as east end. The other was east side. You can also mention the other end as west side. I will represent east side as 0 and west side as 1. As mentioned earlier, he has already been given a boat to traverse. Let us give him a row so that he can go forward and come back. [00:00:57.170] Fine. Let me repeat the constraints that are there. A boat can carry man. Along with him, he can carry only one item. Item could be cabbage, goat, and wolf or wolf, any one of them. [00:01:12.470] Fine. So now, we are trying to find out the different combinations that are there. Question arises is, what are those different combinations that are there? [00:01:24.260] So, in order to find out the total number of combinations, if you can see along with the man, there are three different items. So, totally including man, there are four different items with us. [00:01:48.560] So, the total number of combinations that exist are 2 to the power of 4, which is going to be totally 16. [00:01:58.780] Fine. [00:01:58.960] So, I will mention starting from 0 to 15, but there are totally 16 different combinations that exist for us. [00:02:09.080] Fine. [00:02:09.830] Now, we have 0 to 15, in between I have 1, 2, 3 and so on. [00:02:17.140] Fine. [00:02:17.600] These combinations, I will also call them as states. [00:02:21.430] These combinations I will also call them as states. So how many different states are there for us? There are totally 16 different states for us starting from 0. So let me mention the states here. Let me write the states here. 0, 1, 2, 3, 4, 5, 6,

7. [00:02:43.600] 8, 9, 10, 11, 12, 13, 14, and 15. Hopefully it is visible for you. So now what I will do is these states as there are 4 different inputs let's represent each state with 4 different values. Let me say 4 different inputs, 4 different inputs, 4 different inputs, 4 different inputs, 5 different inputs, 5 different inputs, 5

still a valid one. So we'll go ahead with the next one. It's also a valid state. 12 is invalid for us. 13 valid, 14 valid and 15 also valid. Now we have how many valid states? [00:07:52.150] Valid states are 0, 1, 2, 4, 5, 10, 11, 13, 14, and 15. Valid. [00:08:03.330] Invalid. Invalid states for us are, the remaining ones are invalid for us, 3, 6, 7, 8, 9, 12. [00:08:28.400] 6. So there are totally 16 different combinations were there out of which only 10 are correct. The rest 6 does not satisfy the requirement. So what next? Next is to whatever we have done we will represent them in terms of the states as already mentioned. So let me go ahead and then start doing. [00:08:55.640] As we all know our start state is zeroth state because all of them are towards the east end. So this is my starting point, start point. What is my ending point? My ending point is where all of them will reach to the other end and this value gives me. So at the end I should reach to your stage number 15. So now when we are doing this please remember and take care of the valid states. [00:09:26.920] Right? We are going to be using only the valid states in order to represent. Right? Because invalid states do not satisfy the constraint, so we are going to be leaving them out. Okay? So let's start. Now, all of them are towards east side. They have to come to the west side, which means to start with, man will come to the west side, which becomes one. [00:09:57.040] At the same time, he can carry any one item. So which item will he carry? He cannot leave cabbage and goat together. He cannot leave goat and wolf together. Fine. So that's the reason why he is going to carry goat with him, leaving cabbage and wolf to the other side itself. In the same side where they are now.

EV0000014 · EV0000015 · EV0000016 · EV0000017

Transcript block 3 (source interval 000600-000899 seconds): [00:10:22.770] This value is basically 10 for us. Okay. And check whether 10 is in the valid state. Yes. 10 is in the valid state. So now from 0, I will navigate to state number 10. Let's move ahead. Now, man and goats are in the west side. The other two are in the east side. [00:10:48.400] What man has to do now in order to get the item? He has to go to the other side which means from 1 he will go to east side which is 0. Goat is already in the west side so let's not touch anything with this. Fine. Now what he can do? What is the other option that exists? The other option that basically exists for us is sorry. [00:11:16.080] Yes. Only man is coming here to the east end. Fine. Goat is already to the west side itself. While going back to the east side, is man carrying any items with him? No, he is not carrying any items. So these values will be 00 itself. Fine. Which means I will get a transition which is 2. [00:11:44.470] So now, man is in the east side, goat is in the west side, the cabbage and wolf are in the east side. [00:11:53.590] What next? Man has to come to the other end, goat is already present. Now, whom man will carry? [00:12:03.670] He can carry either a cabbage or he can carry wolf also. Both the options are valid. [00:12:08.810] So at this point, at this point, there are two different possibilities that exist for us. [00:12:15.010] What are those two different possibilities? The two different possibilities are, he can carry wolf with him, leaving cabbage on the other side itself. [00:12:26.760] So which corresponds to a value called as 11. So now from 2, I will navigate to 11. [00:12:36.240] If he carries from this state, he can also carry Cabbage with him. [00:12:42.810] If he is carrying Cabbage, then it is going to be 1, 1, 1 and 0 which corresponds to me for 14. [00:12:54.220] From 14, we will be going to stage number 4 or state number 4. [00:13:01.220] From state number 11 to state number 1. [00:13:05.430] Either you are in state number 4 or you are in state number 1, I would be navigating to state number 13. [00:13:16.600] I request all the students to just trace it out so that you can understand it in a better way. [00:13:23.180] From 13, I will be going to state number 5, from 5 to state number 15, which marks the end of all the states. [00:13:39.280] Out of which only a few of the states have been used and rest of the states have not been used.

[00:13:45.660] Which means we are considering only the relevant aspects that are required, leaving the outs that are not relevant to us. [00:13:56.180] There are few things that are not relevant to us. What are they? They are, what is the size of the boat? What is the color of the boat? Whether man is tall? What is the name of the man? What is the color of the goat? And so on. So, we have considered only the things that are required and ones that are not required have been left out. Hopefully, you have understood how to solve a problem computationally. Thank you.

EV0000018 · EV0000019 · EV0000020 · EV0000021

Full transcript: 1999 normalized words, 66 timestamped segments, 867.84 seconds duration. Low-confidence segments flagged: 0.

EV0000010 · EV0000011 · EV0000012 · EV0000013

4 Process of Computation Problem Solving - Analysis, Design, Implementation, Testing

This lesson is indexed from the source video titled Process of Computation Problem Solving - Analysis, Design, Implementation, Testing. Its complete timestamped transcript is preserved below in segment order. The original video path and content hash are recorded in datasets/lecture-corpus.json. Transcript text is source wording; it has not been silently corrected or rewritten. A candidate slide deck is grouped here according to the current matching score; the exact path and page locators are listed in the source evidence. EV0000022 · EV0000023

Graphs and diagrams

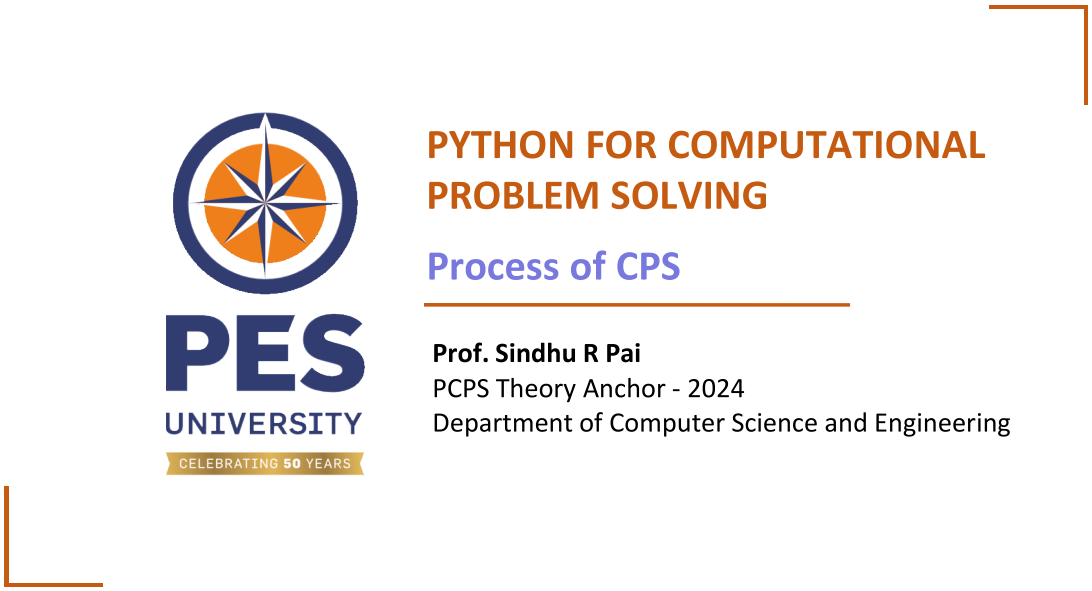


Figure G000076. UE24CS151A PCPS Unit 1 SindhuRPai Class 3 Process of CPS · source slide page 1.

EV0000556

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Process of Computation Problem Solving

- This does not simply involve the act of computer programming.
- It is a process, with programming being only one of the steps.
- Before a program is written, a design for the program must be developed.
- And before a design can be developed, the problem to be solved must be well understood.
- Once written, the program must be thoroughly tested.

Figure G000077. UE24CS151A PCPS Unit 1 SindhuRPai Class 3 Process of CPS · source slide page 2.

EV0000559

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

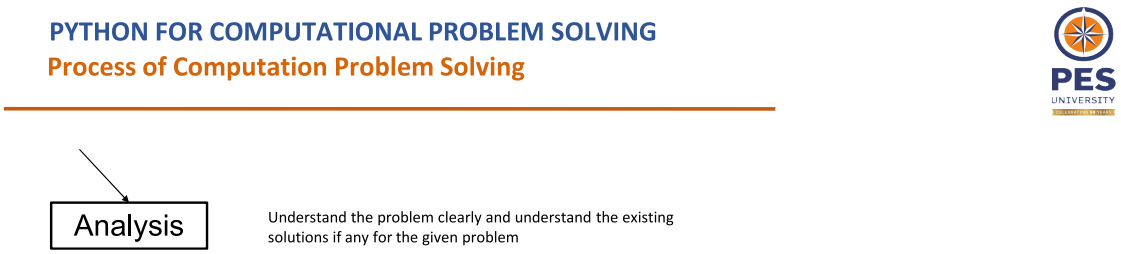


Figure G000078. UE24CS151A PCPS Unit 1 SindhuRPai Class 3 Process of CPS · source slide page 3.

EV0000562

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

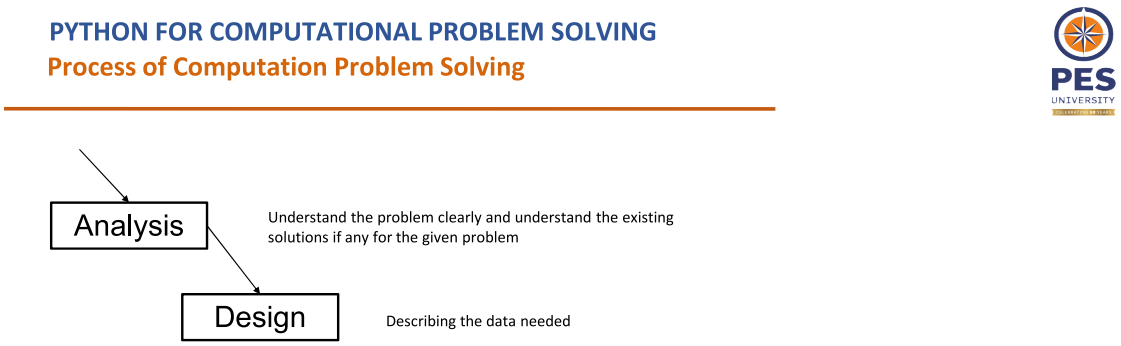


Figure G000079. UE24CS151A PCPS Unit 1 SindhuRPai Class 3 Process of CPS · source slide page 4.

EV0000565

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

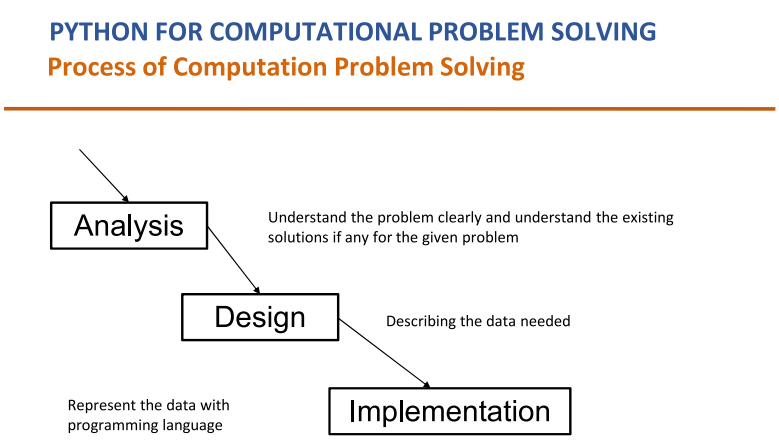


Figure G000080. UE24CS151A PCPS Unit 1 SindhuRPai Class 3 Process of CPS · source slide page 5.

EV0000568

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

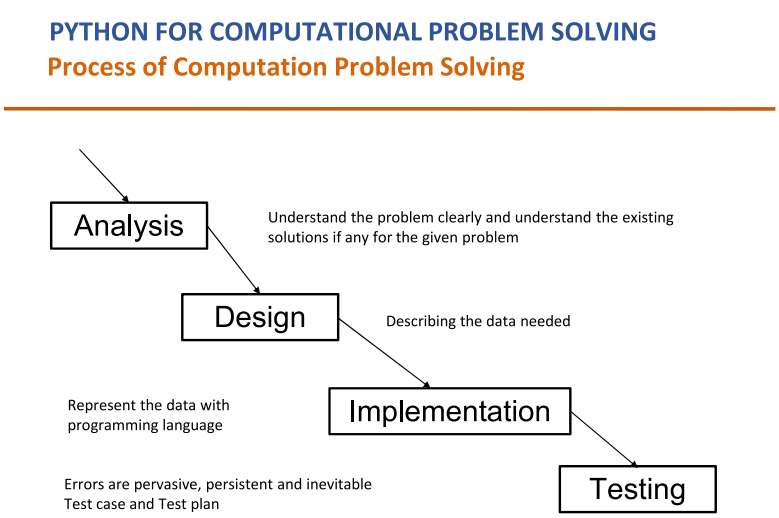


Figure G000081. UE24CS151A PCPS Unit 1 SindhuRPai Class 3 Process of CPS · source slide page 6.

EV0000571

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Process of Computation Problem Solving

Problem Analysis:

- Must understand the fundamental computational issues involved.

- Example:
  For MCGW problem, can use brute-force approach of trying all of the possible rowing actions that may be taken.

Figure G000082. UE24CS151A PCPS Unit 1 SindhuRPai Class 3 Process of CPS · source slide page 7.

EV0000574

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Process of Computation Problem Solving

PES UNIVERSITY

Knowing what constitutes a solution.

For some problems, there is only one solution. For others, there may be a number (or infinite number) of solutions. Thus, a problem may be stated as finding,

- **A solution**
- **An approximate solution**
- **A best solution**
- **All solutions**

Figure G000083. UE24CS151A PCPS Unit 1 SindhuRPai Class 3 Process of CPS · source slide page 8.

EV0000577

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

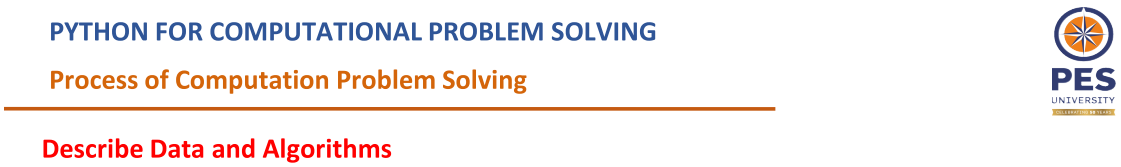


Figure G000084. UE24CS151A PCPS Unit 1 SindhuRPai Class 3 Process of CPS · source slide page 9.

EV0000580

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Process of Computation Problem Solving

PES UNIVERSITY

Program Implementation

- Design decisions provide general details of the data representation and the algorithmic approaches for solving a problem.
- The details, however, do not specify which programming language to use, or how to implement the program.
- Since we are programming in Python, the implementation needs to be expressed in a syntactically correct and appropriate way, using the instructions and features available in Python.

Figure G000085. UE24CS151A PCPS Unit 1 SindhuRPai Class 3 Process of CPS · source slide page 10.

EV0000583

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Process of Computation Problem Solving

PES UNIVERSITY

Program Testing

Writing computer programs is difficult and challenging. As a result, **programming errors are pervasive, persistent and inevitable**.

Given this fact, **software testing is a crucial part of software development**. Testing is done incrementally as a program is being developed, when the program is complete, and when the program needs to be updated.

Figure G000086. UE24CS151A PCPS Unit 1 SindhuRPai Class 3 Process of CPS · source slide page 11.

EV0000586

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

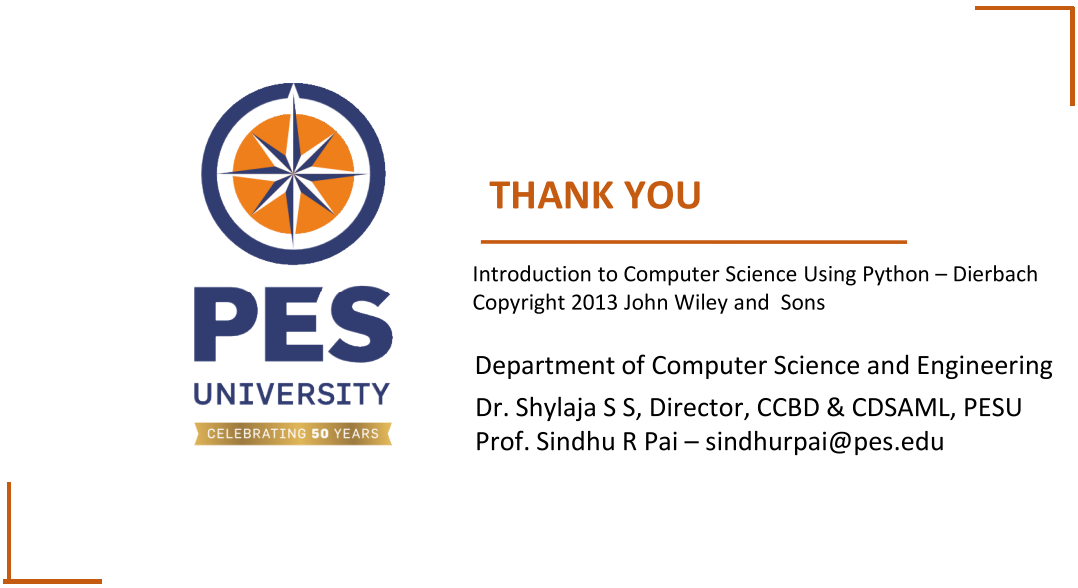


Figure G000087. UE24CS151A PCPS Unit 1 SindhuRPai Class 3 Process of CPS · source slide page 12.

EV0000589

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Important points and why

1. Source slide page 1 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Process of CPS Prof. Sindhu R Pai PCPS Theory Anchor - 2024 Department of Computer Science and Engineering EV0000555 · EV0000556 · EV0000557
2. Source slide page 2 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Process of Computation Problem Solving • This does not simply involve the act of computer programming. • It is a process, with programming being only one of the steps. • Before a program is written, a design for the program must be developed. • And before a design can be developed, the problem to be solved must be well understood. • Once written, the program must be thoroughly tested. EV0000558 · EV0000559 · EV0000560
3. Source slide page 3 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Process of Computation Problem Solving Analysis Understand the problem clearly and understand the existing solutions if any for the given problem EV0000561 · EV0000562 · EV0000563

4. Source slide page 4 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Process of Computation Problem Solving Analysis Understand the problem clearly and understand the existing solutions if any for the given problem Design Describing the data needed EV0000564 · EV0000565 · EV0000566
5. Source slide page 5 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Process of Computation Problem Solving Analysis Understand the problem clearly and understand the existing solutions if any for the given problem Design Describing the data needed Represent the data with Implementation programming language EV0000567 · EV0000568 · EV0000569
6. Source slide page 6 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Process of Computation Problem Solving Analysis Understand the problem clearly and understand the existing solutions if any for the given problem Design Describing the data needed Represent the data with Implementation programming language Errors are pervasive, persistent and inevitable Testing Test case and Test plan EV0000570 · EV0000571 · EV0000572
7. Source slide page 7 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Process of Computation Problem Solving Problem Analysis: • Must understand the fundamental computational issues involved. • Example: ForFor MCGWMCGW problemproblem, cancan useuse brutebrute-forceforce approachapproach ofof tryingtrying allall ofof thethe possiblepossible rowing actions that may be taken. EV0000573 · EV0000574 · EV0000575
8. Source slide page 8 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Process of Computation Problem Solving Knowing what constitutes a solution. For some problems, there is only one solution. For others, there may be a number (or infinite number) of solutions. Thus, a problem may be stated as finding, • A solution • An approximate solution • A best solution • All solutions EV0000576 · EV0000577 · EV0000578
9. Source slide page 9 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Process of Computation Problem Solving Describe Data and Algorithms • For the MCGW problem, need to store the current state of the problem. • When solving a computational problem, either suitable existing algorithms may be found, or new algorithms must be developed. EV0000579 · EV0000580 · EV0000581
10. Source slide page 10 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Process of Computation Problem Solving Program Implementation • Design decisions provide general details of the data representation and the algorithmic approaches for solving a problem. • The details, however, do not specify which programming language to use, or how to implement the program. • Since we are programming in Python, the implementation needs to be expressed in a syntactically correct and appropriate way, using the instructions and features available in Python. EV0000582 · EV0000583 · EV0000584
11. Source slide page 11 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Process of Computation Problem Solving Program Testing Writing computer programs is difficult and challenging. As a result, programming errors are pervasive, persistent and inevitable. Given this fact, software testing is a crucial part of software development. Testing is done incrementally as a program is being developed, when the program is complete, and when the program needs to be updated. EV0000585 · EV0000586 · EV0000587

12. Source slide page 12 (extracted text): THANK YOU Introduction to Computer Science Using Python – Dierbach Copyright 2013 John Wiley and Sons Department of Computer Science and Engineering Dr. Shylaja S S, Director, CCBD & CDSAML, PESU Prof. Sindhu R Pai – sindhurpai@pes.edu EV0000588 · EV0000589 · EV0000590

SHORT-NOTE RECAP

Transcript block 1 (source interval 000000-000299 seconds): [00:00:00.310] Hello everyone. Welcome back to the lecture series on Python for Computational Problem Solving. [00:00:06.310] I am Neeta Anjieker from the Department of Computer Science and Engineering. [00:00:11.600] In the previous video, we discussed what a computer hardware is and how information is stored internally or how it is represented in the computer hardware. [00:00:23.540] In this lecture, we will look at what computer software is, what are the syntax and semantics and program translation that we would use to interact with the computer hardware. [00:00:38.450] So what is a computer software? It is a set of program instructions and related data that can be executed by the computer. We need a computer software to tell the computer how exactly it should work. [00:00:56.210] So it can be categorized into system software designed to provide a platform for your computer hardware and application software which performs a specific task like text editing or say playing games and so on. [00:01:14.710] So let's look at this representation. You have the computer hardware on top of which you need an operating system to provide a platform on which the application software can reside. [00:01:27.710] And as users, we would interact with the computer through various applications. So what exactly is an operating system? [00:01:38.390] It is a software that manages and interacts with the hardware components of the computer. [00:01:44.720] So, the operating system determines how to handle the memory, the main memory, the secondary memory, and how the CPU should function, or which application gets access to the CPU. [00:01:58.220] An operating system is intrinsic to the operation of a computer, and that is why it is called the system software. [00:02:07.320] It acts like a middleman by telling the application software whether it can run and how much of the storage it's available for use and what it does is it provides a user interface and a look and feel. [00:02:24.020] So, when you use computers based on a Windows operating system, Mac operating system or Ubuntu, you will see that each of them have a different interface. They look and feel different. The way we access the files or the way we access the applications are different because of the operating system and the interface it provides to that particular computer hardware. [00:02:51.990] So it determines how we interact with the computer. [00:02:56.560] So let's look at this diagram here where I have the computer hardware and the system software houses the application software. [00:03:05.660] Now you know that when you're using your Android phone even, you can... [00:03:10.550] Install applications that work only on Android. If you are using an Apple phone, you can install applications that only work on iOS. So similarly, we have application softwares that are specific to certain applications. [00:03:25.880] operating systems. So you can see here operating system takes care of file management, assembly, debugging, utilities and compilers and the application software are those which perform a specific task. So when I want to write a word document, I will use a specific application software like a word processing software. [00:03:48.760] If I want to play games, I have to install that game on my operating system in order to play the desired game. [00:03:56.760] So this is how we have application software through which I communicate with the system software and in turn access the resources of the computer hardware. [00:04:10.870] Now that we know how it works, how exactly are we communicating with

the hardware through the application and system software? [00:04:19.860] So, when I am communicating with you, I have chosen English as a means of communication. [00:04:26.360] Now, you are able to understand what I am saying because I am following the rules of the English language and that is why you understand it. [00:04:36.530] So, you have syntax, semantics and a translation that takes place even when we are communicating with each other. [00:04:45.530] So, let me explain that in detail. [00:04:49.200] Now, programming languages are also called languages, right? But they are artificial languages because we are using that to communicate with a computer. And natural languages are what we use to communicate with each other, such as English. So, syntax and semantics are concepts that apply to all languages, whether they are natural or artificial.

EV0000022 · EV0000023 · EV0000024 · EV0000025

Transcript block 2 (source interval 000300-000599 seconds): [00:05:17.140] So, what exactly is syntax? Syntax is basically a set of rules that define the placement of the characters to form a valid sequence of those characters. So, in English, syntax would be the spellings of certain words that we use. So, syntax, English, [00:05:39.860] For English include the letters of the alphabet, punctuation, and properly spelt words. [00:05:46.360] You would notice that I just made an error when I was talking. [00:05:50.780] So, when I say, hello there, how are you? [00:05:54.680] You understand that sentence because it is a set of properly spelled words with properly used punctuations. [00:06:04.680] You know that this is a question because I have used the question mark as a punctuation. Whereas, if I would write the same thing, hello there, how are you? [00:06:23.970] is not how we pronounce the word how according to English language. So although it sounds right, it is not syntactically correct. So syntax tells you the rules of how to create these words and sentences. So what exactly is semantics? Semantics will tell you what these collection of words mean. [00:06:49.810] So it's the meaning associated with the collection of syntactically correct sequence of characters. [00:06:57.950] So look at the following sentence. [00:07:00.340] Hello, are you how? [00:07:02.340] That is syntactically correct because if you look at it, I have used the question mark at the end of the sentence and each of these words are spelled correctly. [00:07:13.300] But does it make any sense? Is there meaning associated with it? No. Because this is now a syntactically correct sentence but semantically incorrect. Because I can't make out what it is. Right? So, every language has its own syntax and semantics. So, similarly, in programming also, when we're using or learning the Python programming language, [00:07:38.430] We will see what are the syntax and semantics we have to use when we write programs. [00:07:44.510] So, let's look at an example. [00:07:49.750] In English, how is syntactically incorrect, but in Chinese, however, how is a valid word which means good. [00:07:58.750] So you can see here, the syntax for English is how and has no meaning, but in Mandarin, how means good. [00:08:06.750] And you can also see the syntax here, although the language is different, what I am trying to tell you is the pronunciation of the word has different semantics. [00:08:18.350] for different languages. And that is what we would have to keep in mind when we write programs as well. [00:08:25.980] So, when we communicate with the computer through a programming language, we know the CPU can only interpret machine code, which is your zeros and ones. [00:08:39.630] So you can see here, all these zeros and ones is what the CPU understands. But can I write? [00:08:46.900] Instructions as a collection of 0 and 1. [00:08:50.030] It's going to be extremely difficult and I can make errors. Right? So writing programs in low level is very tedious and error prone because I may not understand what I've written at the end of it and it's so difficult to keep a track of each bit and

bytes that I have used. And that is why we would use high level languages like Python, which is closer to our understanding. [00:09:19.030] So since instructions are not in machine code we need a translator program. So for example if I start talking in a different language that you do not understand you would need a translator to tell you what I am trying to communicate. So similarly when I am writing a program in a high level language that the CPU does not understand I need a translator program. [00:09:46.740] There are two types of translators. One is the compiler, which translates the entire code into machine code to be executed by the CPU, and the interpreter, which executes or interprets instructions one at a time.

EV0000026 · EV0000027 · EV0000028 · EV0000029

Transcript block 3 (source interval 000600-000899 seconds): [00:10:04.470] So how does it work? When I use a compiler as a translator, you can see here I have written a small program. Don't worry about the syntax or what exactly these mean. [00:10:15.730] We will explore that when we write programs later on. [00:10:19.230] So I've tried to add two numbers and find the sum. [00:10:22.930] So the compiler would take this entire program, translate it, check the syntax semantics, convert it to machine code. [00:10:31.950] And then it will give you the result. And this is how the compiler would work. Whereas an interpreter immediately execute instructions as they are entered. And this is referred to as an interactive mode. So when you give the program, the interpreter runs on top of the CPU where each and every line of code is interpreted, translated and executed immediately. And you see the results. [00:11:00.670] one line at a time. So this is how your computer software works and how we can interact with the computer hardware. I hope you have understood all of this and we will meet in the next video to understand what programming language we are going to use and how exactly we would start writing your first computer program. Thank you.

EV0000030 · EV0000031 · EV0000032 · EV0000033

Full transcript: 1534 normalized words, 56 timestamped segments, 684.373333 seconds duration. Low-confidence segments flagged: 0.

EV0000022 · EV0000023 · EV0000024 · EV0000025

5 Process of Computation Problem Solving - Analysis, Design, Implementation, Testing

This lesson is indexed from the source video titled Process of Computation Problem Solving - Analysis, Design, Implementation, Testing. Its complete timestamped transcript is preserved below in segment order. The original video path and content hash are recorded in datasets/lecture-corpus.json. Transcript text is source wording; it has not been silently corrected or rewritten. A candidate slide deck is grouped here according to the current matching score; the exact path and page locators are listed in the source evidence. EV0000034 · EV0000035

SHORT-NOTE RECAP

Transcript block 1 (source interval 000000-000299 seconds): [00:00:00.850] Hello all. Welcome back. Python for Computational Problem Solving. So what did we do in our previous class? We looked into digital computers. What is computer made up of? It is made up of hardware. [00:00:16.280]

Followed by, it also requires a software. So, how does the communication between the user and your hardware takes place? It is with the help of the operating system. [00:00:25.780] We also looked into syntax, semantics and the required program translation models. So, what are we going to be doing in today's class? What is the agenda for today's class? [00:00:37.280] So, in today's class, I will be dealing with the process of Computation Problem Solving. [00:00:43.250] Followed by Introduction to Python Programming Language and we will also write our first Python program. [00:00:50.580] Hope you are all ready. Let's get started. [00:00:53.580] To start with, assume that you've been given a problem to solve. In the sense, I'm asking you people to write a program. [00:01:07.670] At this point, I will definitely assume that each one of you already know either any of the programming language or you already know Python. [00:01:17.170] So, in such case, the moment the problem is given to you, will you start directly coding a problem? Will you start with your computer and start coding? Usually, the answer is no. [00:01:31.410] The reason is once a problem is given coding is just a part of there are different things that has to happen whenever we want to solve a problem computationally. [00:01:44.070] So what are we looking at? We are going to be looking at the different steps that are involved while I have to solve a problem using computers. [00:02:00.020] Computational Problem Solving Approach. So let's get started directly. The different steps that are involved are, the first step is called as analysis, second one is called as a design, third one is called as your implementation, and fourth one is testing. So we will try to understand each one of these steps. [00:02:26.840] With a particular example. The example that I am going to be taking is the one that we have already solved in our previous session which is our MCGW problem. So hopefully we will get into this. The first step is called as problem analysis. It is also called as an analysis step. Now given a particular problem MCGW, what in all we tried understanding. [00:03:00.600] We try to find out, how can I get a solution to this problem? What are the different approaches that are there? We came to know that there are four different inputs that are available. [00:03:12.320] So, as the number of inputs are small, we can definitely try out with the different set of combinations that are involved. [00:03:19.320] And that is how we evolved into understanding that there are totally 16 number of different combination X's. [00:03:28.760] Suppose my input will now change from 4 to 5 in such a way that the number of combinations that I have to search is going to be 32. [00:03:39.730] It is because 2 to the power of 5, 5 different inputs. Suppose if my input also increases by 1, which is going to be 6, 2 to the power of 6, so many number of different combinations that exist. Thus, given any problem, the first and foremost thing that I have to do is try to identify what are the issues that are involved in order to solve a problem. [00:04:05.560] Fine. So, hopefully, what exactly is the problem analysis? Problem analysis is nothing but identifying the issues that are involved while I am trying to solve a problem. Along with knowing the issues, I am also supposed to find out how can I get to a particular solution? What exactly constitutes a solution? Fine. When we talk about solutions, [00:04:32.660] The solutions that you get can be a single solution. There is going to be an exclusively only one solution or there may not be an exactly a solution but somewhere I can come near and then say this is an approximate solution that exists for us. [00:04:47.420] The next one is going to be the best solution. There are n number of approaches that are available out of which I am trying to find out which is the best solution that is going to be there for my particular problem.

EV0000034 · EV0000035 · EV0000036 · EV0000037

Transcript block 2 (source interval 000300-000599 seconds): [00:05:00.950] And the last one, there does not exist only a single solution. There are different solutions that are there. All of them are

best at this point. [00:05:09.440] So, when I have to solve a problem, the first step that I have to follow is analysis. So, what exactly is an analysis? Try to understand what are the issues that are involved. At the same point of time, also try to identify or know what is exactly constitute a solution. [00:05:29.810] There could be a single solution, approximate solution, best solution or it could be all different solutions. [00:05:36.810] So, this is your first step when you try to computationally solve a problem. [00:05:43.980] So, let us try to identify what are the other steps that are there. [00:05:47.980] So, the second step is describe the data and the required algorithms. [00:05:54.230] So, again, we will take a same example because I will be taking the same example in order to explain different steps. [00:06:01.030] Now, in case of your MCGW problem, first thing that I really need to know is, where is the starting point? [00:06:11.510] The starting point basically serves as a current state of the problem. [00:06:17.210] In the sense, I am trying to understand what type of data is available for me in order to solve it. [00:06:24.210] So, I need to understand what is the current state. Whether man and all the items are on the east side or already they are in the west side. So, you need to know the current state. [00:06:34.210] At the same time, when your entire data is available for you, the next thing that you have to do is, you have to solve a problem. In order to solve a problem, you basically need an algorithm. [00:06:51.540] Will you be making use of an existing algorithm or would you like to make use of a different algorithm by writing yourself? Both the options are absolutely valid. [00:07:02.540] So, in the second step of the process of Computation Problem Solving is describing a data along with the algorithm. [00:07:10.540] So, what you need to do? For our MCGW problem, you need to know the what is the current state of the problem. [00:07:21.170] Do you want to use an existing algorithm or you would like to write your own algorithm? So, choice is up to you based on your requirement. [00:07:30.170] So, the third step is the program implementation. The design, decision, everything from the previous step, you basically get the data. [00:07:44.040] You also know what algorithm, how to solve it, everything is ready. At this point, the question that arises is how will you solve? [00:07:55.800] In the sense, how will you make your computer system to solve it for you? You need to communicate with the computer system. So in such way that it is very clear I have to make use of your particular programming language. So which is that programming language that you want to use it? [00:08:11.920] So, this design decision has to be made in the implementation phase. You should know which language you want to make use based on your requirement. So, in case of your program implementation, you have to decide which language you want to implement. So, going ahead. At this point, I have understood my problem. [00:08:39.600] Data is available for me. I also came to know that what is an algorithm that is required. At the same time, I have decided that this is the programming language that I am going to be working with. So, we will understand anyway it is Python. So, I also know that I have to use Python in order to solve a problem. We will also assume that my entire programming is done, everything is ready. What next? [00:09:05.560] Very important step is to find out whether your program works fine or not. That is where your final step comes which is your testing. So what is the last step? The last step is going to be the testing. While testing I will come to know what are the mistakes that I have done. Usually it is a good practice to perform the testing in an incremental fashion. [00:09:37.520] What does that mean? Instead of you writing some thousand lines of code and running your entire code at once, it becomes very difficult for you to find out where the errors are present. [00:09:49.040] So instead, break down your programs in terms of smaller piece of code. Try to perform the testing. That is what we call it by incremental testing. [00:09:59.040] And whenever you are trying to do the

changes, please make sure that what is the reason behind you performing or why are you trying to do the changes.

EV0000038 · EV0000039 · EV0000040 · EV0000041

Transcript block 3 (source interval 000600-000899 seconds): [00:10:10.000] Fine? So, thus, there are, we'll go back to the diagram that I have shown you so that it is easy for us to understand. If you can see, the four different steps that are involved are, first one is called as an analysis, second one is called as a design, third one is implementation, and the final one is called as your testing. So, every time, whenever you are going to be solving the problem, you will have to take care of these four different steps. [00:10:38.130] So, we will go back to where we had stopped. So, this is where it was. So, we have understood what is the process of computational problem solving. Now, the facts, few of the facts. Few of the facts are very clear, errors are common. [00:11:02.390] You cannot avoid errors. If there are no errors in your program, well and good. You are very lucky. It is fine. [00:11:10.320] So, programing errors are persistent in nature. That is why testing is very important part of your software development. [00:11:21.460] So, whenever you want to make the changes to an existing program, either because there is an error or because it does not suit to your requirement, please make sure and you should be very clearly know the reason behind you doing the changes. So, please keep all these three things in mind whenever you want to solve a computational problem. Fine? So, hope the entire process is understandable for you. [00:11:52.820] So, we will go ahead and try understanding the other concepts that are there for today's class. [00:11:59.320] What is this? There are so many questions. What are we going to be doing before we go into the Python or writing our first Python program? [00:12:17.060] These are few questions that we have to answer. So, let's answer these questions and then we will try to go and write our first Python program. [00:12:27.860] So, I want you people also to think about it and answer yourself what is the answer for these questions or what is the solution for these questions. [00:12:39.180] To start with, the first one, what is a programming or what is a program? So, what is a program? [00:12:49.080] Program is nothing but a sequence of instructions or a set of instructions which we use it [00:12:57.970] In order to command to the computer to solve a particular task for us. It's an art. It's a creative activity. Whenever we have something called as a creative, it is always going to be a fun. So that is why programming, we have to think it in terms of fun activity. [00:13:18.710] Hopefully, the first point is clear. So, what is programming? It's an art. It's a creative activity. It's a sequence of instructions, a set of instructions that are given for a computer in order to perform a particular task. They are the commands that you are going to be giving. Command can be in terms of a single command or it could be n number of commands that are present. So, let's go ahead with the next question. [00:13:47.860] Why should we learn program? Any idea why we have to learn program? If you can see, I have given a link. I request you all to copy down this link. I do not want to play it here because it is around a 10 minutes of video that is there. A beautiful video. I request you all to go through this video. Fine? So, you will clearly understand why you should learn program. Fine? Before you go and watch this video, [00:14:19.160] Why should we learn programming? Is it because people think you are an illiterate, you don't know programming? I'm not sure about it. Maybe. Or people will consider me as an illiterate only when I know programming, even I'm not sure about it. But still, we have to learn programming. So, the second point why we have to learn program is? [00:14:44.140] I want to solve a problem computationally. I need a computer. So I have to give a task to a computer, command them, give them instructions. So that is how I'm going to be doing. So I can do it with the help of a program. [00:14:58.140] The

third point is you can think in a different way. You can improve your thinking skills by programming. So that is why you have to know how to write programs.

EV0000042 · EV0000043 · EV0000044 · EV0000045

Transcript block 4 (source interval 000900-001199 seconds): [00:15:15.440] Programming. What is a programming language? What am I trying to do now? I am communicating with you. How am I communicating with you? Through a language called as English. My mode of communication with you is English. Child may have is her own way of communicating with the mother. The differently abled people, the duff and dumb people, they have a different language to communicate. [00:15:48.470] So similarly if I want to communicate with the computer system I need to know the programming language. At this point we are not discussing which programming language but I need to know the programming language because I can make my computer system in order to solve a task for me. [00:16:07.540] The language that a computer knows, as we have already understood, it is zeros and ones. Can I really write programs using zeros and ones? No. Very tough. So, what do I do? I will write it in a different manner. Fine? So, zeros and one is called as a machine language and the language that I will try to write is called as a high-level language. Fine? My immediate question is going to be, [00:16:34.900] How does a computer understand a program in a high-level language? Certainly, it cannot understand for sure. It can understand only a language that is written in terms of zeros and ones. So, high-level language to machine language, how does the conversion happens? How does computer is still able to do the task that we are trying to give? [00:17:00.500] We definitely need a translation models in this case. We have looked into translation models in the previous class. It's called as a compiler. The other one is called as your interpreter. Fine? [00:17:13.050] So, the next question, we are very coming near to what we want to do it in today's class. So, what is Python? So, what is Python? Python is a high-level language. Python is a high-level language. [00:17:33.110] Why it is called as a high level language? It is very close to me. I can write the way in which I perceive. What is the meaning of this statement? We will try to understand when we start writing the Python programs. So, who invented this Python? Python is invented by Guido Van Rossum. It was invented by Van Rossum. [00:17:57.820] Why actually the name Python? He was inspired by a circus which is called as Monty Python's Circus. There is a beautiful video that is available on the YouTube. You can also check with this particular video. So Python is a high level language. It is very powerful where it has been used by both academicians as well as the industry. [00:18:27.510] There are many important features why we have to use Python itself. So what are those features? All these things we will try to discuss in your coming sessions. I won't be taking a separate class to describe only the features. As in when we are coding, at that point itself I will be describing and speaking more about the features of Python. At this point, please try to understand it is more powerful. It is [00:18:56.820] Very easy to learn. So, that's the reason why we are making use of Python. At the same time, Python is being used by both academia as well as the industry. Why Python only? There are so many programming languages on top of which why are we going only for Python? So, this question always arises and it is absolutely a valid question. So, why are we learning Python as [00:19:26.390] You have alternatives. That is, before coming to PES or before joining PES, you would have definitely thought about the other alternatives. If not PES, what else? Before getting into engineering, you would have thought, if I get this ranking, I will go to this, otherwise this. So, everywhere there is an alternatives that are available. Thus, before deciding on Python as your first [00:19:56.140] Programming language that you are going to be learning in PES, the higher levels have tried with the different options that are

there. We tried with Java, we've tried with other programming languages that were there, but finally, it was decided that it is going to be a Python. It is not just by blind we have taken Python. So it was thoroughly thought about and

EV0000046 · EV0000047 · EV0000048 · EV0000049

Transcript block 5 (source interval 001200-001499 seconds): [00:20:23.730] Because of which we have come across to this point where we are going to be making Python as the first programming language for the first year students. So now specifically why are we learning Python? It's simple first point and elegant, easy to master over, very much used by the industry. It's been used all over. [00:20:53.170] In the research area. Recently Stanford has also shifted completely to Python itself. So there are many reasons so as to why we have to learn Python itself. So hopefully you're all convinced. Let's get started and start writing our first Python program. Fine. Hope you're all ready. Let's get started. [00:21:24.400] How do we write a program? You are not aware of anything. So how can you start writing a program? Correct? So this question is always there. So Python programs can be written in two different ways. [00:21:40.630] Please remember, I'm just giving you an introduction to Python programming language. It's just an introduction. In your coming class, we will be exclusively writing Python. I will be speaking about the syntax of the Python and the different features that are available, the constructs that are available in Python. All those things I will be explaining in my next video. So this, at this point, we will just try to identify [00:22:10.020] How do we execute and write our first simple program? So I can write a Python program in two different ways. The first one is called as an interactive mode. And the second one is called as a batch mode. So let me repeat. The first one is called as an interactive mode and the second one is called as your batch mode. [00:22:34.960] Fine. So let's find out how do we work with your interactive mode and the batch mode.

EV0000050 · EV0000051 · EV0000052 · EV0000053

Full transcript: 3277 normalized words, 81 timestamped segments, 1363.285333 seconds duration. Low-confidence segments flagged: 0.

EV0000034 · EV0000035 · EV0000036 · EV0000037

6 Computer Hardware and software

This lesson is indexed from the source video titled Computer Hardware and software. Its complete timestamped transcript is preserved below in segment order. The original video path and content hash are recorded in datasets/lecture-corpus.json. Transcript text is source wording; it has not been silently corrected or rewritten. A candidate slide deck is grouped here according to the current matching score; the exact path and page locators are listed in the source evidence. EV0000054 · EV0000055

Graphs and diagrams

Figure G000088. UE24CS151A PCPS Unit 1 SindhuRPai Class 4 5 Computer Hardware software · source slide page 1.

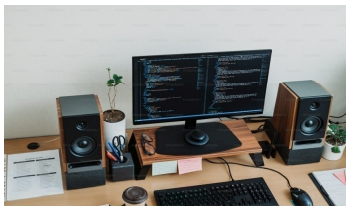
EV0000592

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING
Digital Computer

PES UNIVERSITY

- Computer capable of solving problems by processing information expressed in **discrete form.**
- It operates on data, including magnitudes, letters, and symbols, that are expressed in binary code—i.e., using only the two digits 0 and 1.
- By manipulating combinations of binary digits, it can perform mathematical calculations, organize and analyze data, control industrial and other processes, and simulate dynamic systems such as global weather patterns.



A desktop computer sitting on top of a wooden desk photo – Computer science Image on Unsplash

Figure G000089. UE24CS151A PCPS Unit 1 SindhuRPai Class 4 5 Computer Hardware software · source slide page 2.

EV0000595

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Number System

- Number systems in mathematics are used to express numbers in various forms and are understood by computers.
- Defined as the representation of numbers by using digits or other symbols in a consistent manner.
- A number is a mathematical value used for counting and measuring objects, and for performing arithmetic calculations. Numbers have various categories like natural numbers, whole numbers, rational and irrational numbers, and so on.
- The **value of any digit in number can be determined by a digit, its position in the number, and the base of the number system.**

Figure G000090. UE24CS151A PCPS Unit 1 SindhuRPai Class 4 5 Computer Hardware software · source slide page 3.

EV0000598

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

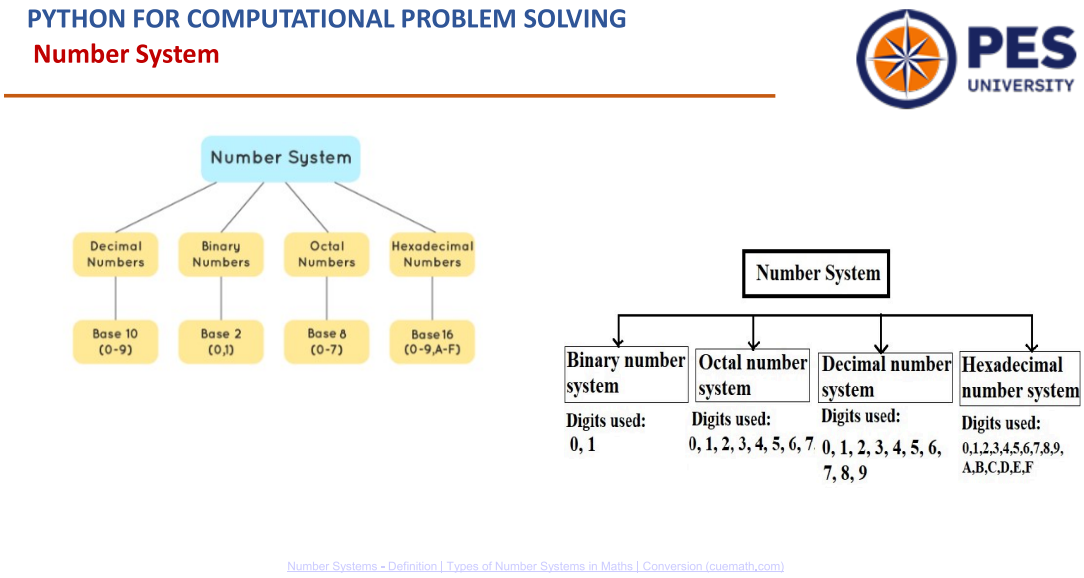


Figure G000091. UE24CS151A PCPS Unit 1 SindhuRPai Class 4 5 Computer Hardware software · source slide page 4.

EV0000601

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

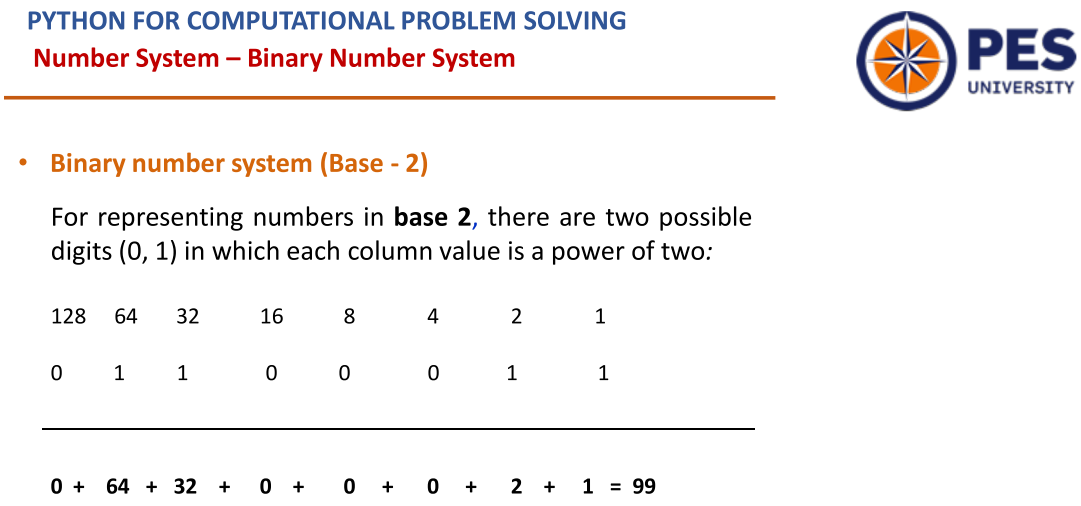


Figure G000092. UE24CS151A PCPS Unit 1 SindhuRPai Class 4 5 Computer Hardware software · source slide page 5.

EV0000604

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Computer Hardware

PES UNIVERSITY

Central processing unit (CPU)

The "brain" of a computer system. **Interprets and executes instructions.**

Main memory

A Memory where **currently executing programs reside**. It is **volatile**, the contents are lost when the power is turned off.

Peripheral components

Input devices, Output devices and Buses

Communication devices

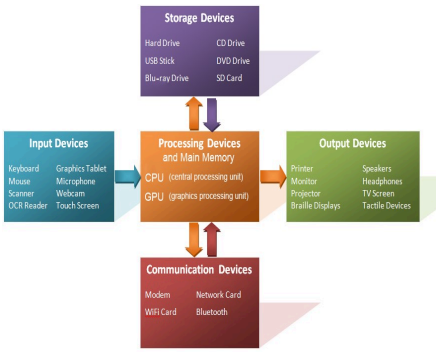


Figure G000093. UE24CS151A PCPS Unit 1 SindhuRPai Class 4 5 Computer Hardware software · source slide page 6.

EV0000607

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING
Computer Hardware

PES UNIVERSITY

Secondary memory

Provides **long-term storage of programs** and data. **Non-volatile**, the contents are retained when power is turned off. Can be magnetic (hard drive), optical (CD or DVD), or flash memory (USB drive).

Figure G000094. UE24CS151A PCPS Unit 1 SindhuRPai Class 4 5 Computer Hardware software · source slide page 7.

EV0000610

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING
Computer Hardware

PES UNIVERSITY

Input Devices – mouse, keyboard, scanner, microphone etc.

Output Devices – monitor, printer, projector, speakers etc

Buses – **Communication system that transfers data between components** inside a computer, or between computers.

- Internal Bus (System Bus: CPU and main memory)
- External Bus (Expansion Bus: printer and the computer)

Communication devices - Modem, WiFi card etc

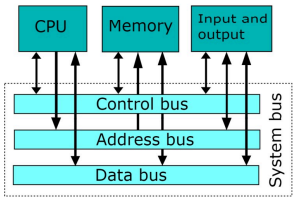


Figure G000095. UE24CS151A PCPS Unit 1 SindhuRPai Class 4 5 Computer Hardware software · source slide page 8.

EV0000613

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING
Computer Software

PES UNIVERSITY

A set of program instructions, including related data and documentation, that can be executed by computer.

- **System software:** Intrinsic to a computer system.

- **Application Software:** Specific purpose software which is used by user for performing specific task.

Figure G000096. UE24CS151A PCPS Unit 1 SindhuRPai Class 4 5 Computer Hardware software · source slide page 9.

EV0000616

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Computer Software

PES UNIVERSITY

- The first computer programs ever written were for a mechanical computer designed by **Charles Babbage** in the mid-1800s.
- **Ada Lovelace** was the person who wrote these programs.
- She is referred to as **"the first computer programmer."**

Ada Lovelace

Figure G000097. UE24CS151A PCPS Unit 1 SindhuRPai Class 4 5 Computer Hardware software · source slide page 10.

EV0000619

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING
Operating System

PES UNIVERSITY

A software that **manages and interacts with the hardware** resources of a computer.

Because an operating system is **intrinsic to the operation** of a computer, it is referred to as **system software**.

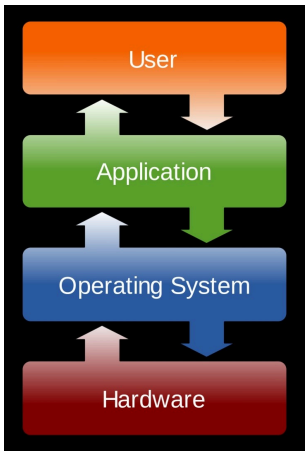


Figure G000098. UE24CS151A PCPS Unit 1 SindhuRPai Class 4 5 Computer Hardware software · source slide page 11.

EV0000622

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

System Software vs Application Software

PES UNIVERSITY

S.No.	System Software	Application Software
1.	System software is used for operating computer hardware.	Application software is used by user to perform a specific task.
2.	System software is installed on the computer when operating system is installed.	Application software is installed according to user's requirements.
3.	In general, the user does not interact with system software because it works in the background.	In general, the user interacts with application software.
4.	System software can run independently. It provides a platform for running application software.	Application software can't run independently. They can't run without the presence of system software.
5.	Some examples of system software are compiler, assembler, debugger, driver, etc.	Some examples of application software are word processor, web browser, media player, etc.

Figure G000099. UE24CS151A PCPS Unit 1 SindhuRPai Class 4 5 Computer Hardware software · source slide page 12.

EV0000625

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

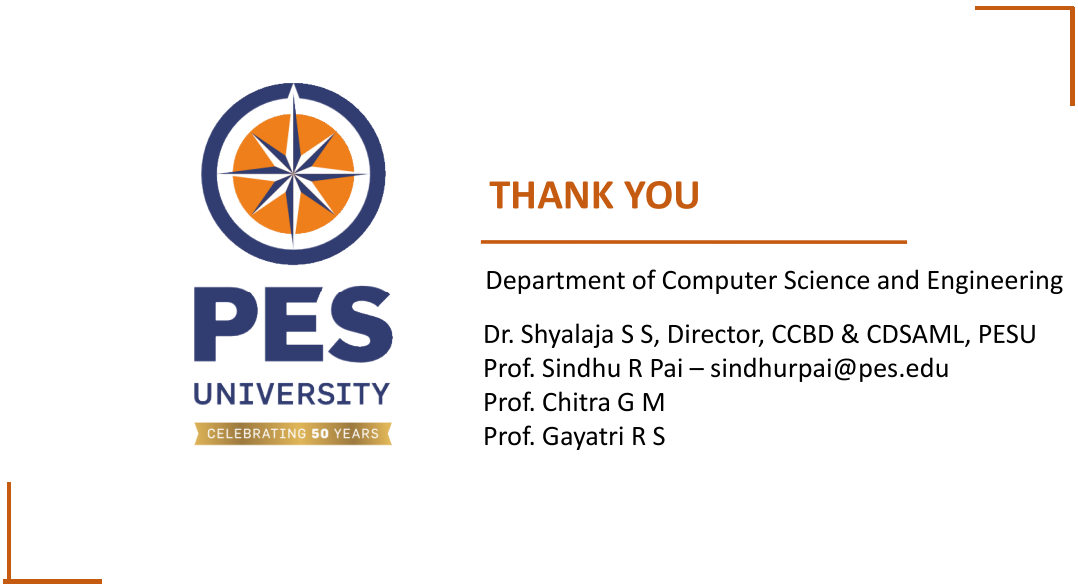


Figure G000100. UE24CS151A PCPS Unit 1 SindhuRPai Class 4 5 Computer Hardware software · source slide page 13.

EV0000628

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Important points and why

1. Source slide page 1 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Hardware and Software Prof. Sindhu R Pai PCPS Theory Anchor - 2024 Department of Computer Science and Engineering EV0000591 · EV0000592 · EV0000593
2. Source slide page 2 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Digital Computer ☒ Computer capable of solving problems by processing information expressed in discrete form. ☒ It operates on data, including magnitudes, letters, and symbols, that are expressed in binary code—i.e., using only the two digits 0 and 1. ☒ By manipulating combinations of binary digits, it can A desktop computer sitting on top of a wooden desk perform mathematical calculations, organize and analyze photo – Computer science Image on Unsplash data, control industrial and other processes, and simulate dynamic systems such as global weather patterns.
   EV0000594 · EV0000595 · EV0000596

3. Source slide page 3 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Number System • Number systems in mathematics are used to express numbers in various forms and are understood by computers. • Defined as the representation of numbers by using digits or other symbols in a consistent manner. • A number is a mathematical value used for counting and measuring objects, and for performing arithmetic calculations. Numbers have various categories like natural numbers, whole numbers, rational and irrational numbers, and so on. • The value of any digit in number can be determined by a digit, its position in the number, and the base of the number system. EV0000597 · EV0000598 · EV0000599
4. Source slide page 4 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Number System Number Systems - Definition | Types of Number Systems in Maths | Conversion (cuemath.com) EV0000600 · EV0000601 · EV0000602
5. Source slide page 5 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Number System – Binary Number System • Binary number system (Base - 2) For representing numbers in base 2, there are two possible digits (0, 1) in which each column value is a power of two: 128 64 32 16 8 4 2 1 0 1 1 0 0 0 1 1 0 + 64 + 32 + 0 + 0 + 0 + 2 + 1 = 99 EV0000603 · EV0000604 · EV0000605
6. Source slide page 6 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Computer Hardware Central processing unit (CPU) The “brain” of a computer system. Interprets and executes instructions. Main memory A Memory where currently executing programs reside. It is volatile, the contents are lost when the power is turned off. Peripheral components Input devices, Output devices and Buses Communication devices EV0000606 · EV0000607 · EV0000608
7. Source slide page 7 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Computer Hardware Secondary memory Provides long-term storage of programs and data. Non-volatile, the contents are retained when power is turned off. Can be magnetic (hard drive), optical (CD or DVD), or flashflash memorymemory (USB(USB drive)drive). EV0000609 · EV0000610 · EV0000611
8. Source slide page 8 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Computer Hardware Input Devices – mouse, keyboard, scanner, microphone etc. Output Devices – monitor, printer, projector, speakers etc Buses – Communication system that transfers data between components inside a computer, or between computers. - Internal Bus (System Bus: CPU and main memory) - External Bus (Expansion Bus: printer and the computer) Communication devices - Modem, WiFi card etc EV0000612 · EV0000613 · EV0000614
9. Source slide page 9 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Computer Software A set of program instructions, including related data and documentation, that can be executed by computer. - System software: Intrinsic to a computer system. - Application Software: Specific purpose software which is used by user for performing specific task. EV0000615 · EV0000616 · EV0000617
10. Source slide page 10 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Computer Software • The first computer programs ever written were for a mechanical computer designed by Charles Babbage in the mid-1800s. • Ada Lovelace was the person who wrote these programs. • She is referred to as “the first computer programmer.” Ada Lovelace EV0000618 · EV0000619 · EV0000620

11. Source slide page 11 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Operating System A software that manages and interacts with the hardware resources of a computer. Because an operating system is intrinsic to the operation of a computer, it is referred to as system software. EV0000621 · EV0000622 · EV0000623
12. Source slide page 12 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING System Software vs Application Software S.No. System Software Application Software 1. System software is used for operating Application software is used by user to perform computer hardware. a specific task. 2. System software is installed on the Application software is installed according to compcomputerter whenhen operatingoperating ssystemstem isis user'sser's reqrequirements.irements installed. 3. In general, the user does not interact In general, the user interacts with application with system software because it software. works in the background. 4. System software can run Application software can't run independently. independently. It provides a platform They can't run without the presence of system for running application software. software. 5. Some examples of system software Some examples of application software are are compiler, assembler, debugger, word processor, web browser, media player, etc. driver, etc. EV0000624 · EV0000625 · EV0000626
13. Source slide page 13 (extracted text): THANK YOU Departmentp of Computerp Science and Engineeringg g Dr. Shyalaja S S, Director, CCBD & CDSAML, PESU Prof. Sindhu R Pai – sindhurpai@pes.edu Prof. Chitra G M Prof. Gayatri R S EV0000627 · EV0000628 · EV0000629

SHORT-NOTE RECAP

Transcript block 1 (source interval 000000-000299 seconds): [00:00:01.668] Hello all. Welcome again. Python for computational problem solving lecture number two. In the previous class, we learned the notion of computational problem solving. We'll also we also looked into an example of river crossing problems. [00:00:19.012] Well solving this problem we basically understood we require two different things. One is called as a representation and the other one is called as an algorithm. So what are we going to be doing in today's class? [00:00:32.580] The topics that I'm going to be covering in today's session are going to be digital computer, computer hardware, operating system, computer software, followed by syntax and semantics, as well as the program translation. I'm pretty sure [00:00:48.900] These are the things that you're already familiar with. So let's not waste time or we'll quickly get into these topics and understand them. So to start with a digital computer or a computer. So what does a computer do? Computer [00:01:07.908] Basically takes input, does some processing, and then it gives us an output. Processing of what? Processing the information. So computer takes an input as an information, does a sum processing and gives me the result. [00:01:27.876] So what exactly is a digital computer? Digital computer does the same thing, it takes an input, does some processing of the information, only thing is when it is doing a processing of the information, it uses the digits as its quantity. [00:01:46.084] And among those digits, it basically works with the binary numbering system. In the coming classes, we will try to work with these binary numbering systems. [00:02:02.244] So what exactly is a computer made up of? What does a computer contain? A computer basically has hardware as well as it also has a software. So we will look into each one of them. Computer hardware. [00:02:20.900] A computer hardware basically represents a physical part of a computer system. It includes CPU, which is your central processing unit, memory, it also has some peripheral devices, such as input device. [00:02:37.572] output device. There are also certain devices which acts both as an input device as well as your output device. [00:02:47.812] So CPU. [00:02:50.980] Lots of debate that is still going on to identify

whether CPU is an heart of a computer or a brain of a computer. So what do you think? [00:03:05.412] Is it an heart or a brain? According to me, it's a brain of a computer system because it interprets the results and executes the instructions for us. Memory. [00:03:20.772] Memory is divided into two different types. One is called as a primary memory, the other one is called as a secondary memory. Primary memory is also called as your main memory. Any program that needs to be executed should be a part of a main memory itself. [00:03:40.324] So, this main memory is a smaller memory, and this memory is very close to the CPU. As it is very close to the CPU, it can access the information present in the main memory at the faster rate. But this memory is volatile in nature. [00:03:59.396] Which means that the contents are lost whenever the power goes off. So you basically lose the content whenever there is no power. In contrast to this, we have something called as a secondary storage. A bigger memory, a larger memory. [00:04:17.124] It's a larger storage device that is available far away from the CPU. That's the reason why if you want to act any access data from the secondary storage, it definitely takes little more amount of time than it takes in order to get the data from your main memory. [00:04:36.196] It is non-volatile in nature, very advantageous. The reason is, even though the power goes off, the information is still retained. [00:04:44.356] An example of your secondary storage could be your pen drives, your CDs, DVD, the now we speak about the external hard disk that is there. So these are the example few of the examples for your secondary memory.

EV0000054 · EV0000055 · EV0000056 · EV0000057

Transcript block 2 (source interval 000300-000599 seconds): [00:05:00.612] But when we were doing our engineering, we were mostly relying on this device as a secondary storage device to store the data. Hopefully you know the name of this. It's called as a floppy disk. [00:05:17.636] At this point we do not see the existence of the floppy disk. But most of us we were relying on the floppy disk itself. So let's continue. As mentioned, computer takes input. [00:05:37.924] So input has to be given through the input device, right? And then it does some processing and gives me an output. So these are a few of the uh devices that have been mentioned, which is input device as well as your output devices. Right? I have an input device. [00:05:58.052] I have an output device, I have a processor, I have a good memory. Now how does the information travel between each of these devices? [00:06:08.196] Fine. I need a communication device which helps me to transfer the data from one device to the other device. So in such case, we need something called as a bus. [00:06:19.632] A bus basically acts as a communication system which is used to transfer the data between the two different computers or within the computer itself. Right? Thus, buses come in two different types. One is called as an internal bus, the other one is called as an external bus. [00:06:40.132] An internal bus is also called as your system bus or a memory bus. An external bus is also called as an expansion bus. The example setup being quoted is for the internal bus is you can use it in order to connect CPU to the main memory because it is within within a computer system. [00:07:00.036] An external bus, it could be between an uh external device to a computer, or it could be between a computer to a computer also. Now that I have a computer and I clearly know what exactly my computer contains. [00:07:18.820] Is it all enough for me to run the computer? How will I manage the different devices that are there? So in such case [00:07:32.644] We basically need an operating system. Fine? So before I get into the operating system, if you could just look in. So an input device, an output device, you have [00:07:46.500] CPU, main memory, then there are storage units. So input device includes the keyboard, mouse, if you can see a touch screen. Please touch screen will also act as an output device. So touch screen could be called as an IO

device. [00:08:02.884] Fine. Along with that your handset is also an um an IO device because it has got both speaker as well as the mic. So let's go ahead. [00:08:15.364] As I was mentioning, we need an operating system because I want to manage and interact with the hardware. To give you a better understanding, let's look at this particular diagram. If you can see we have an hardware user is present here. [00:08:31.012] Right? How will the user interacts with the hardware. So I basically need an operating system in order to interact between an user and an hardware. [00:08:43.108] Fine. So now operating system. Can we really feel the existence of an operating system? No. But without this operating system, my computer is dumb. It cannot do anything. [00:08:55.972] So, as I cannot feel the existence of an operating system, the reason is it's a program. It is some set of instructions that are written. So when I say set of instructions, we basically call it as a program, or it could also be called as a software. [00:09:15.524] So there are two different types of software that exist for us. As I already mentioned, you can see what is a software. Software is nothing but the program instructions. [00:09:27.088] So there are two different types of software. One is called as a system software, and the other one is called as an application software. So we'll try to understand what is the difference between a system software and an application software. System softwares are the ones that are required [00:09:46.916] in order to run my computer system is used to operate the computer hardware. At the same time, application software, I'm gonna be using application software where I need to perform a specific task.

EV0000058 · EV0000059 · EV0000060 · EV0000061

Transcript block 3 (source interval 000600-000899 seconds): [00:10:03.940] such as you want to prepare a document. So in such case, I need a word processor in order to prepare my document. The document that I'm preparing is a customized one. So if your intention is to work with a specific task, then make use of your application software. [00:10:22.468] Coming back again to a system software, you don't need to explicitly uh install any system software that is there, because it comes along with whenever you install your operating systems. So it doesn't need any external [00:10:39.152] installation. But whereas application software needs to be installed according to the user requirements. So going forward, system software works at the background, but whereas application software are required where you're going to be interacting directly. [00:11:00.068] System software are called as independent softwares. They do not need anybody else to support them to work. Whereas application software cannot run independently, they need the existence of the system software. [00:11:16.516] So few of the examples of the system software include compilers, loaders, linkers, assemblers, and so on. Examples of your application software include word processor, as mentioned earlier, followed by web browser, media player, so on. [00:11:38.468] Let's carry forward. [00:11:43.332] When I'm talking to you, I'm basically using English as a language in order to interact. [00:11:52.260] Right. Similarly, if I want to interact with my computer system, I want a specific task to be done. I want to solve a computationally problem. Computational problem. So in such cases, what do I need to do? How will I interact with my computer system? [00:12:09.412] Definitely we are going to be uh talking with respect to the programming language. So similarly, when I'm talking with you, I'm talking in an English language. When I'm talking, I need to be aware what are the uh different letters [00:12:27.652] That makes a part of the alphabets of an English fine. At the same time, I also need to understand, are there any other things that makes up an English language? So if you take English as an example, we have alphabets and then followed by punctuations. [00:12:46.212] Fine. So now let's look into this. I will come back to this particular statement later on. So let's quickly look into this example. So I say

hello there, how are you? [00:13:00.036] So hopefully your answer could be we are fine, we are excellent, and so on. But am I looking for that? No. My intention is different in this case. So what am I looking for? If you can see, I'm trying to say, and I've made a statement here, it is syntactically [00:13:16.900] Correct. Fine? I'm talking about it is syntactically correct. So what does it mean? So it means we are looking at Am I following all the rules and regulations of English that is being defined? [00:13:33.540] What are those rules saying? The rules say it should be a set of characters. Yes, it is a set of characters. At the same time, I'm also talking about are they arranged in a proper order? Are they arranged in a proper order? Yes, they are arranged in a proper order. [00:13:53.540] Fine. Acceptable sequence is what we are trying to say. So in such case, I say this statement as syntactically correct with respect to an English language. Fine. So now let's look into the other example. [00:14:11.172] Fine. So now when you look into this example, you basically come across with the puncations. Yes, it's absolutely valid. Then go ahead with the next word that is there, hello. So it is also absolutely fine. There [00:14:27.300] Yes, but if you come across with this, what does it mean? Maybe it is true or it is correct for some other language, but it is not correct in English language. [00:14:39.108] So thus we are deviating from the uh rules. So if you are deviating from the rules, then we call it as syntactically incorrect. [00:14:49.700] Okay, so thus every language has a syntax. It is a set of characters and the acceptable sequence of those characters. Fine. So moving forward, we understood what is a syntax. Let's quickly go and understand what exactly it means by semantics.

EV0000062 · EV0000063 · EV0000064 · EV0000065

Transcript block 4 (source interval 000900-001199 seconds): [00:15:09.508] Fine. So in order to understand again, we will look into an example. Sorry, we'll look into an example. Fine. So the example that we are going to be looking is, if you can see, I'm talking about how you hello R. [00:15:29.316] So what is the wrong with this? What is the problem with this statement? Looks everything fine. How set of characters are there? [00:15:40.516] Yes it's arrangement is also proper. There is no harm. And if you go to the next word, it's fine. Third word it is fine. Fourth word it is fine. I'm also using a valid punctuation marks. So what is the problem? [00:15:55.556] So in such case, I basically call this statement as it is syntactically correct, but there is no meaning in it. It gives me a wrong meaning. Have you hello are what is the meaning it gives me? [00:16:09.232] It doesn't provide me or required meaning. That's why I call it as it is syntactically correct, but semantically it is wrong or it is incorrect. So what does it mean? So semantics of a language is the meaning that is associated with each [00:16:28.612] correct sequence of syntactic characters. [00:16:33.284] So I'm talking about the meaning. So in this case, everything is fine, valid characters, valid punctuation marks, everything is fine. That's why I'm calling it as a syntactically correct, but it is semantically wrong because it does not provide me the required meaning. [00:16:52.164] Fine. So every language has its own syntax and semantics. So in your coming weeks, when you are whenever you're trying to write a Python program, you need to understand what is the syntax and the semantics of a particular language. We will be doing it in your uh coming sessions. [00:17:12.420] Uh let's go forward. Now I have a computer. I think I'm going to be communicating it with a language. Which language? Definitely Python for sure. But at this point [00:17:27.876] A computer can understand only zeros and ones. That is what we have already described. Zeros and ones we call it as a machine code. [00:17:38.116] Computers are designed, it can interpret and execute only the things that are written in terms of zeros and ones. Fine. So look at this. [00:17:50.212] Zero one zero one zero one it is absolutely fine. But if I ask you to find out what

exactly it is, interpret what it is. Each one of us will try to interpret in a different way. Fine? I may say it is a series of zero cent ones. [00:18:05.252] Fine, let me get my cursor back here. Yes. Okay. So if I'm asking you to interpret in a different way, sorry, interpret you each one of us will interpret in a different manner. I may say it as combination of zeros and ones, or somebody say [00:18:22.592] There are four different lines in this case and each one of them have a different uh decimal values and so on. Now my CPU can execute only those things that are written in terms of zeros and ones. [00:18:41.188] This is called as a low level language, or it is also called as a machine language, where it can understand only zeros and ones. But [00:18:53.284] Is it really easy for me to write any programs using zeros and ones? Absolutely no. It becomes very difficult for me. The moment I start writing zeros and ones, I may end up writing few things, or it may go as an error for me. That is why it is always easy for me to write. [00:19:13.540] programs in an language called as an high level language, which is very close to us. It is very easy for us to write the things. Fine? So now the problem is I can write in a high level language, but a computer can understand only the machine code. [00:19:33.284] What do we do? There should be somewhere where I need to have somebody who is converting an high level language to a [00:19:43.204] machine language. So in such case we have a program called as a translator who is going to be converting a high level language program into a machine code or a machine language. [00:19:57.284] Why? There are basically two different types of translators that exist. One is called a compiler, other one is an interpreter. So let's look into this. If you can see, you can see this is the program that I write, high level language, I can understand.

EV0000066 · EV0000067 · EV0000068 · EV0000069

Transcript block 5 (source interval 001200-001499 seconds): [00:20:16.644] I have a quantity 10, I have a quantity 20, I need to perform addition. What are these we will look later on. I'm also trying to say perform addition here 10 plus 20 and I get a result. Fine. So now it has to be executed. [00:20:34.468] Right. In order to execute this, if you are using a compiler, it will convert an high level language into the machine code. And this is given as an input to your CPU in order to interpret and perform the execution. And thus you obtain a result. [00:20:54.916] This is what your compiler does. So what does an interpreter do for you? The same program that we are doing, but interpreter runs on top of your CPU. [00:21:07.012] It executes the instructions immediately. As and when you give instructions line by line, it executes it immediately. So that's why any program that basically executes an instruction [00:21:24.228] As it is entered, it is referred to as an interactive mode. So in your coming classes, we will also be looking into and then try to understand how do we work with an interactive mode. Hope you understood the concept of compilers as well as the interpreters. [00:21:44.004] So uh before we end the session, we'll quickly uh summarise on what we learned. [00:21:50.500] So we looked into computer hardware, we looked into operating system, what is the need for the operating system, then followed by we also looked into the software, system software, followed by an application software, what is the difference between both of them. [00:22:08.548] And we also looked into what is syntax and semantics, right? Followed by we tried understanding what is a machine code and what is a code that we try to write. And how do we convert from a machine code to a [00:22:25.060] Sorry, how do we convert from a high level code to a machine code in such a way that my computer system can execute it. Hope you understood uh the concepts. Thank you.

EV0000070 · EV0000071 · EV0000072 · EV0000073

Full transcript: 3171 normalized words, 86 timestamped segments, 1359.317333 seconds duration. Low-confidence segments flagged: 0.
EV0000054 · EV0000055 · EV0000056 · EV0000057

7 Computer hardware

This lesson is indexed from the source video titled Computer hardware. Its complete timestamped transcript is preserved below in segment order. The original video path and content hash are recorded in datasets/lecture-corpus.json. Transcript text is source wording; it has not been silently corrected or rewritten. A candidate slide deck is grouped here according to the current matching score; the exact path and page locators are listed in the source evidence. EV0000074 · EV0000075

SHORT-NOTE RECAP

Transcript block 1 (source interval 000000-000299 seconds): [00:00:00.388] Hello everyone, welcome back to the lecture series on Python for computational problem solving. I am Nita Anjakab from the Department of Computer Science and Engineering. [00:00:11.524] In our previous video, we looked at what computational problem solving was, and we now know that a computer or a computing device is essential for computational problem solving. So, in this video, we will look at [00:00:28.112] the computer hardware and how it works internally. [00:00:32.292] So the topics we would cover are the fundamental hardware components, what is digital computing, what is decimal digitization, what is information theory, binary digitization, what is an integrated circuit. [00:00:49.156] We'll look at the binary number system and what are the terms, bits and bytes. [00:00:55.524] So, first let's look at the fundamental hardware components. Now I know most of you know exactly what computers are, what are the components, and so on. So I'm not going to delve deep into those topics, but quickly rush through [00:01:13.060] What the computer hardware components are and how they are connected to each other. So what is a hardware? It is the physical part of a computer system. [00:01:24.004] So it consists of the all-important central processing unit and the memory. It also consists of some peripheral devices like the keyboard, the mouse, the printer, the monitor, etc. [00:01:39.652] So what are the fundamental hardware components? Because we know that not every computer needs to have a printer, right? So the most fundamental hardware components are first the brain of the computer, which is your central processing unit. [00:01:57.636] This unit is what does all the computations that we ask it to do. And the programs that run in order to tell the CPU how to compute it reside on the main memory. So all the programs reside on the main memory. [00:02:15.588] The main memory is very close in proximity in terms of hardware to the CPU, and hence the CPU can quickly access all the information from the main memory. [00:02:28.228] Now the main memory is volatile in nature and what that means is when the power supply is switched off, the memory is wiped clean. So it's more like a short term memory. [00:02:40.624] If you want some information to persist for a longer time, even after your computer is turned off, we need the secondary memory. [00:02:50.628] Which is non-volatile, and the information will remain even though there is no electricity. So examples of it are like your external hard drives, which are your magnetic drives, optical drives like your CD and DVD, and flash drives like your USB. [00:03:11.588] So let's look at the input devices.

[00:03:16.004] And output devices, these are your mouse, keyboard, monitor, printer, and so on. So an input device technically takes input from the user, which is mostly us, and has to send it to the central processing unit. So when it does that [00:03:33.536] How is the information going to be taken from the input device to the central processing unit? It uses something called as a bus. So we've heard the term bus even in our day-to-day lives, right? A bus is a public mode of transport. [00:03:49.456] That takes people from point A to point B. So buses and computers does the same thing. It takes information from point A to point B. [00:04:00.132] So there are two kinds of bus, internal buses and external buses. Internal buses are the ones which take information within the computer system, like your system bus. [00:04:13.348] Similar to your intercity buses. Now like your interstate buses, we have external buses also that take information from outside the computer system, like from your external devices like your printers and so on to the CPU and back. [00:04:31.236] So the layout is the brain of the computer is central, and you have all the other devices in the periphery which communicate with the CPU through these buses. [00:04:45.988] So let's now look at what is digital computing. It basically means we need to now come up with a design that the hardware would work in a way that whatever instruction we give it should be.

EV0000074 · EV0000075 · EV0000076 · EV0000077

Transcript block 2 (source interval 000300-000599 seconds): [00:05:01.092] error free. So we have to ensure that your hardware is reliable. [00:05:06.468] And in order to do so, we have to ensure that if there is no error in the hardware, only then will we get the correct result of our computation. So if there's some error, the results would also be incorrect. So what we have to do. [00:05:23.808] is make sure that the design is simple as possible. [00:05:27.908] So, first we have to look at what is decimal digitization, because that's what the initial design for the hardware was. So we know that the computer takes electricity as input. Now electricity is usually represented as an analog. [00:05:44.996] information. So you can see here this is an analog sine wave, which you must have seen in your school as well. So analog signals are represented by continuous voltage. So if I want to represent information and get [00:06:02.564] Just discrete values is where the decimal digitization came into play, where the values are represented in discrete ways in the base of 10, which is between 0 and 9. So each voltage level represented one decimal value. [00:06:20.644] So, for example, a voltage of 4 volts would be taken as the number 4, 5 as the number 5, and no electricity as the number 0, and so on. Now this design is error prone because if there is a sudden [00:06:36.832] spike of voltage, the information could be misinterpreted. That is when the concept of information theory was introduced by Claude Shannon, who said that all information [00:06:50.384] Can be represented by the use of only two symbols zero and one, which is what is represented or called as binary representation. So your binary digitization is basically where each digit is represented only as zero and one. [00:07:09.188] So we have to use a computer hardware which uses the concept of on or off. So we have an electric switch called transistors that does precisely that. So the voltage levels are now only two levels. [00:07:24.580] Then, if there is a voltage, irrespective of what it is, would be considered as 1, and if there is no voltage, it's considered as 0. [00:07:34.500] Now, a collection of these transistors is what we call an integrated circuit, which is the building blocks of our computer hardware. It comprises of millions or even billions of these transistors on a simple chip. Now, the pioneer of these [00:07:51.684] concept of the integrated circuit was got in E. Moore, who said that the number of transistors that can be placed on the integrated chips would increase by double every two years. And surprisingly it still has not reached its limit. [00:08:10.468] An interesting fact is that they say by 2030 we will have an

integrated circuit or a computer that can perform all the actions that a human brain can. So we have some interesting times to look forward to in the age of computers. [00:08:28.708] So now let's look at how information is represented. Now we are very familiar with the decimal number system. [00:08:36.868] Where we know what is a units place, a tens place, and so on. So when we look at a number, every digit in a number is between 0 and 9, and the position of that digit determines its value. [00:08:50.400] So let's look at how 99 is represented in decimal form. So you can see here, every column represents one power of 10. [00:09:01.732] So tent and in computer systems the counting always starts from zero. So the rightmost column is known as your least significant column and the leftmost is the most significant column. [00:09:16.228] So here you can see 10 to the power of 0 is 1, 10 to the power of 1 is 10, 10 to the power of 2 is 100, which is basically our units place, tens place, 100ths place, and so on. So when I represent the number 99, [00:09:31.344] 9 in the tens place means 9 into 10, 90. 9 in the units place is 9 into 1, 9. You add it, you get 99. So how is it represented in terms of the binary number system where [00:09:45.988] Each digit has or each column is the power of 2. So if you look at it, 2 to the power of 0, 1, 2 to the power of 1, 2, and so on. So 99 is represented as 011011.

EV0000078 · EV0000079 · EV0000080 · EV0000081

Transcript block 3 (source interval 000600-000899 seconds): [00:10:03.236] So each of these numbers or digits are multiplied with the power of 2. So 1 plus 2 plus 0's because these bits are turned off. 1 into 32, 32, 64 as well. So when you add all of this, you get 99. [00:10:21.924] Now you can see that the binary representation of 99 is much larger than the decimal representation, but this is not prone to errors as much as [00:10:34.160] the decimal digitization and gives us such a simple hardware design where you have the transistors that are just turned off or on. Based on the position, the value can be calculated. So what are each of these digits called? [00:10:50.948] That is where we look at the concept of bits and bytes. So every binary digit is called a bit, and a collection of eight binary digits is called a byte. So let me show you one more example of converting a decimal number to a binary number. [00:11:09.572] So when you want to convert from base 1 to base 2, you just have to divide it successively by 2. So let's convert from number 10 to binary representation. So 10 divided by 2 will give you 5 and has no remainder because it's perfectly divisible. [00:11:28.484] 5 divided by 2 will give you 2 with the remainder 1. 2 by 2 [00:11:35.972] Value 1, remainder 0, and 1 by 2 will give you 0 with the remainder 1. So these are your binary digits. And how do we read them? We read them from bottom to top. So the one at the bottom will be the most significant bit. [00:11:52.592] And the 0 at the top will be the least significant bit. So the binary representation of 10 is 1, 0, 1, 0. So 0 into 1 will give you 0. 1 into 2, that's 2 to the power of 1, will give you [00:12:08.964] 2, 0 into 2 to the power of 2 will give you 0. And 1 into 2 to the power of 3, which is 8, will give you 8. 8 plus 2, 10. [00:12:18.980] I hope you've understood how exactly the computer hardware works internally, and in the next video, I will introduce you to the computer software. We will see how the software allows us to communicate with [00:12:32.880] the computer hardware and how this information can be given and stored. So see you in the next lecture. Thank you

EV0000082 · EV0000083 · EV0000084 · EV0000085

Full transcript: 1818 normalized words, 54 timestamped segments, 763.648 seconds duration. Low-confidence segments flagged: 0.

EV0000074 · EV0000075 · EV0000076 · EV0000077

8 Program Structure

This lesson is indexed from the source video titled Program Structure. Its complete timestamped transcript is preserved below in segment order. The original video path and content hash are recorded in datasets/lecture-corpus.json. Transcript text is source wording; it has not been silently corrected or rewritten. A candidate slide deck is grouped here according to the current matching score; the exact path and page locators are listed in the source evidence. EV0000086 · EV0000087

Graphs and diagrams

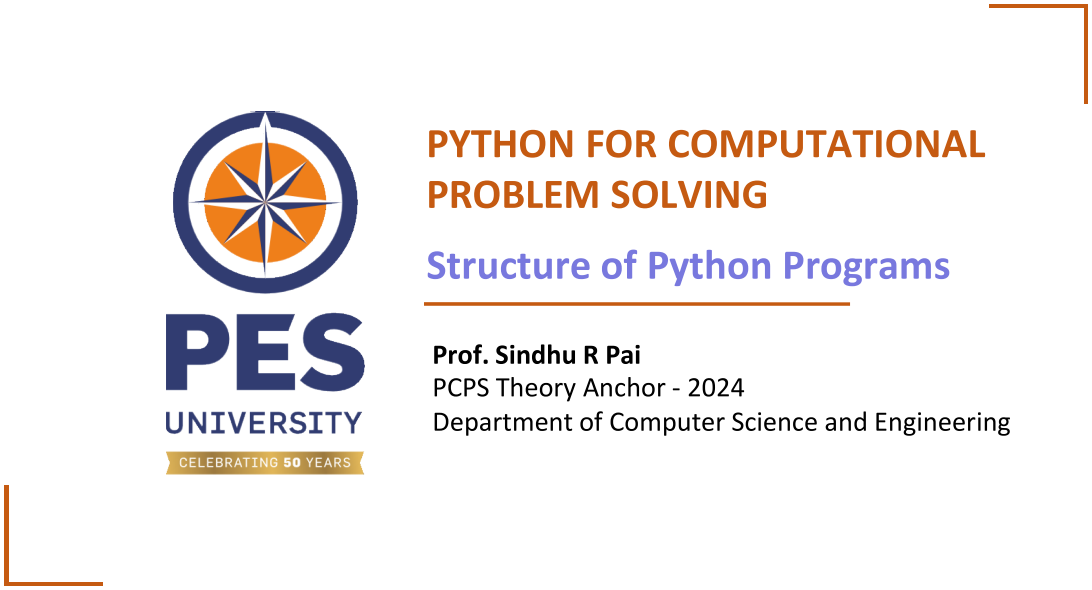


Figure G000123. UE24CS151A PCPS Unit 1 SindhuRPai Class 8 Program structure · source slide page 1.

EV0000697

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

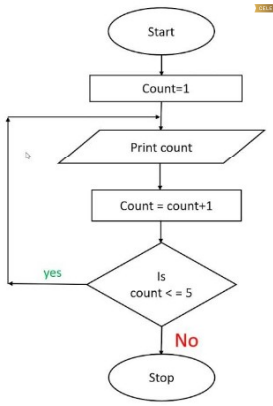
PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Execution Flow

PES UNIVERSITY

Program Execution Flow

- Code is executed **line by line from top to bottom**
- Each statement **executed in sequence unless otherwise specified**, like in the case of constructs like loops and functions



2

Figure G000124. UE24CS151A PCPS Unit 1 SindhuRPai Class 8 Program structure · source slide page 2.

EV0000700

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

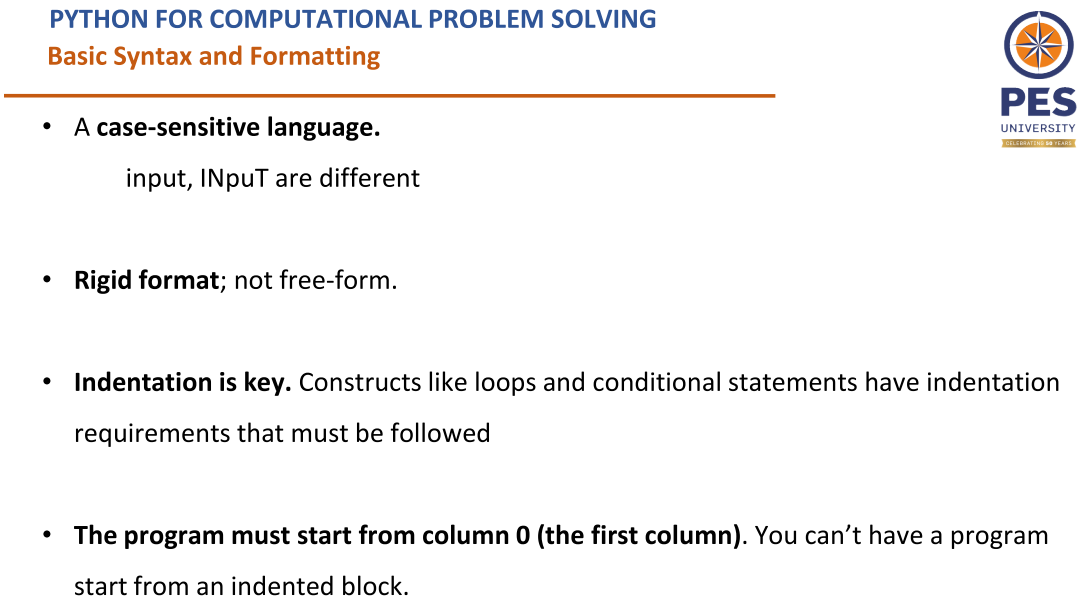


Figure G000125. UE24CS151A PCPS Unit 1 SindhuRPai Class 8 Program structure · source slide page 3.

EV0000703

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

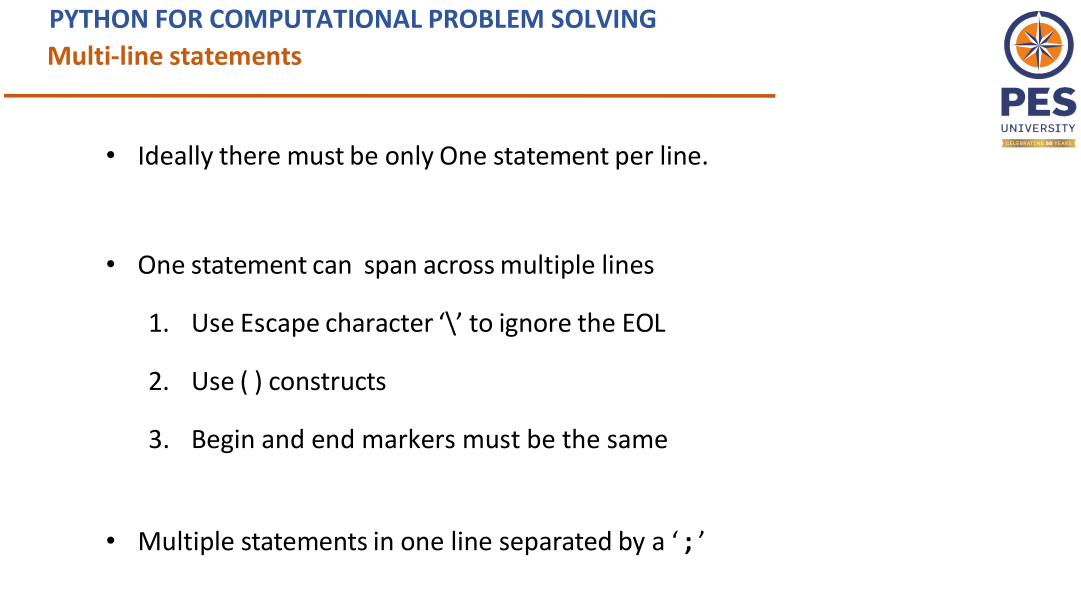


Figure G000126. UE24CS151A PCPS Unit 1 SindhuRPai Class 8 Program structure · source slide page 4.

EV0000706

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Comments

- Single line comments begin with a # symbol
- The documentation string is used for commenting multiple lines which is written using a pair of triple " " " " " " quotes or ' ' ' ' ' '

3

Figure G000127. UE24CS151A PCPS Unit 1 SindhuRPai Class 8 Program structure · source slide page 5.

EV0000709

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

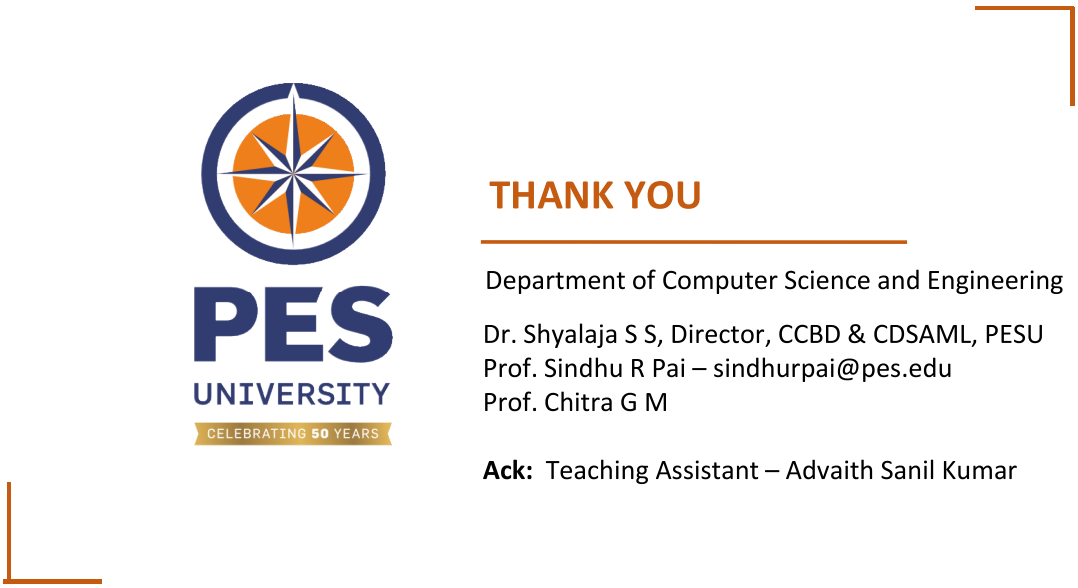


Figure G000128. UE24CS151A PCPS Unit 1 SindhuRPai Class 8 Program structure · source slide page 6.

EV0000712

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Important points and why

1. Source slide page 1 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Structure of Python Programs Prof. Sindhu R Pai PCPS Theory Anchor - 2024 Department of Computer Science and Engineering EV0000696 · EV0000697 · EV0000698
2. Source slide page 2 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Execution Flow Program Execution Flow • Code is executed line by line from top to bottom • EEachh statement executedd iin sequence unlessl otherwise specified, like in the case of constructs like loops and functions 2 EV0000699 · EV0000700 · EV0000701
3. Source slide page 3 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Basic Syntax and Formatting • A case-sensitive language. input, INpuT are different • Rigidg format;; not free-form. • Indentation is key. Constructs like loops and conditional statements have indentation requirements that must be followed • The program must start from column 0 (the first column). You can’t have a program start from an indented block. 3 EV0000702 · EV0000703 · EV0000704

4. Source slide page 4 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Multi-line statements • Ideally there must be only One statement per line. • One statement can span across multiple lines 1. Use Escape character '\' to ignore the EOL 2. Use ( ) constructs 3. Begin and end markers must be the same • Multiple statements in one line separated by a ' ; ' 3 EV0000705 · EV0000706 · EV0000707
5. Source slide page 5 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Comments • Single line comments begin with a # symbol • The documentation string is used for commenting multiple lines which is written using a pair of triple " " " " " " quotes or ' ' ' ' ' ' 3 EV0000708 · EV0000709 · EV0000710
6. Source slide page 6 (extracted text): THANK YOU Departmentp of Computerp Science and Engineeringg g Dr. Shyalaja S S, Director, CCBD & CDSAML, PESU Prof. Sindhu R Pai – sindhurpai@pes.edu Prof. Chitra G M Ack: Teaching Assistant – Advaith Sanil Kumar EV0000711 · EV0000712 · EV0000713

SHORT-NOTE RECAP

Transcript block 1 (source interval 000000-000299 seconds): [00:00:01.700] Last example for today's class. [00:00:07.268] So let me say it as print. I don't do anything. Save your pro So is it a valid one? Yes, it is a valid. The reason is print is basically a functioning. [00:00:23.396] Print is a function name. Anything that is a function, a name of a function has got some value, and anything that has a value, I call it as an expression. Or I can also say it as a statement. [00:00:41.924] So it is still absolutely fine. Okay. But why this function was not called, the reason is until and unless I use the parenthesis, it won't be executed. Okay. Hopefully this concept is clear for you. [00:01:02.532] Okay, so please go through these things. When we are uh writing our programs in your coming classes, we need all these uh details. You enjoyed? Thank you.

EV0000086 · EV0000087 · EV0000088 · EV0000089

Full transcript: 145 normalized words, 5 timestamped segments, 76.650667 seconds duration. Low-confidence segments flagged: 0.

EV0000086 · EV0000087 · EV0000088 · EV0000089

9 Program structure

This lesson is indexed from the source video titled Program structure. Its complete timestamped transcript is preserved below in segment order. The original video path and content hash are recorded in datasets/lecture-corpus.json. Transcript text is source wording; it has not been silently corrected or rewritten. A candidate slide deck is grouped here according to the current matching score; the exact path and page locators are listed in the source evidence. EV0000090 · EV0000091

SHORT-NOTE RECAP

Transcript block 1 (source interval 000000-000299 seconds): [00:00:01.860] Hello all. Welcome back, Python for computational problem solving. What are we going to be doing in today's class? Before we start, let's quickly find out what we have done in a previous class and we will continue our discussion. [00:00:21.348] In the previous session, we looked into the process of computational

problem solving, followed by we also had an introduction to Python language, and then followed by we also wrote our first Python program. [00:00:37.284] Fine. Most of you might have got the doubt we have not installed Python. How do we start working with the Python? Yes, I do agree with that. You can search for any YouTube video. Fine? [00:00:51.876] And then uh follow the process. It's going to take you less than uh five minutes in order to install Python. In case anything required, I will try to show you in your coming sessions. Hope uh you're all uh fine with this. [00:01:13.572] We will continue our discussion for today's. What are we going to be doing? I'll just quickly uh show you one more way of uh uh writing our first program that we have done in the previous class and then we'll continue. Okay. So to start with [00:01:31.632] You can make use of any editor, either Jupyter notebook or your uh G Edit, whatever it is. Fine. Make use of any text editor that is there. So at this point, I'll just use the notepad. Okay, open your notepad. [00:01:52.100] And then you can say print hello or python. Okay. So close this. What exactly is the meaning of this? I will let you know. Do not worry about it. So let me save my program. As I already said. [00:02:12.036] My program is stored in a folder called as Python programs, and this Python program folder is present on your desktop. So please you have to remember the part when you are executing your program. So let me save it as all files. [00:02:30.276] Fine, and I'll say because yesterday we said it as first program, again I will name it as first program itself. And all your Python programs will end with dot PY extension. [00:02:43.812] Dot PY extension. So I'll say it as save. So my program is saved. So now open your command prompt. Fine. I'll just set it as CMD. Open your command prompt. I will minimize it so that [00:03:05.764] You can also see your output simultaneously. Okay? So let me do this. Do you really need to do every time? No. So that you can see the output, I'm trying to do this. Okay. So [00:03:18.884] Uh I also showed you a mistake what was there in yesterday's class. Let me uh repeat it again so that we will not uh commit this mistake every time. The reason is how will you execute your program? I say Python. [00:03:36.612] program name dot p y. So it is f y and I say enter, you can say [00:03:47.620] cannot open a file. It is trying to say that this file that you're trying to open does not exist. The reason is maybe my path is a wrong path that I'm trying to execute. Okay? So what do I need to do? I am very sure that it is stored on the desktop in a Python [00:04:05.312] A programs folder. So let me go. How will I navigate to a folder? So I have to go to your CD. Fine? CD followed by [00:04:18.276] the name of the folder where you have to go, place where you have to go. So now I'm on desktop. Then I say C D Python programs. Fine. And then I'll say this. My terminal is looking messy. Let me clear this. [00:04:38.500] I have cleared CLS is a command to clear this, and then I see Python followed by file name dot pile. Okay. So now when I do this, you can see I'm getting an output which is python. [00:04:58.532] Okay. So hopefully uh you're able to uh know how things are going on. So we wrote our first program.

EV0000090 · EV0000091 · EV0000092 · EV0000093

Transcript block 2 (source interval 000300-000599 seconds): [00:05:09.380] Fine. There are many things, there are many doubts that are going to be there for you. Let all those doubts be there. Do not worry. As and when it is required, I will be clarifying those doubts. Fine. So [00:05:23.460] This is what we have done in uh yesterday's class or your previous class, and what are we going to be doing in today's class. We will still continue our discussion. [00:05:34.596] To your command prompt and execute. If you see it print is getting executed one, and then followed by again I'm executing this line where I'm also getting an output for that. [00:05:47.524] That's fine. So now you had one value, now you are having two different values. These values or whatever you give inside

your print statement or a function, I call it as an argument. [00:06:02.756] Fine. I call it as an arguments. So don't worry, in your unit number three, we will be discussing more about functions and more about arguments. Okay? So as of now you remember you have a print. [00:06:16.832] Which is a function. How will you come to know it is a function? If you have an open parenthesis and closed parenthesis, it means it's a function for us. Now what is inside a function? It is an argument. In this case, I have a single argument, and in this case I have two different arguments. [00:06:34.576] Okay, so now what did your print do? Print basically displayed it. Fine. Where did it display? It displayed it on the terminal. Fine. So print, I was using a print statement in order to display, and it displayed the output on the computer screen. [00:06:54.052] Fine? So whatever you are giving, it is only giving back that and it is not giving any other things for us. Okay? So hopefully is that clear? Fine. So now what did we do? We took a print, it's a function. [00:07:12.068] It takes arguments, an argument can be one or it can be more, and then followed by what does your print will do? It just displays whatever you have given. Okay. So now I'll do one more thing. One more thing here. So I'll say print. [00:07:29.572] And I see one plus two. Okay, so now let's go here. [00:07:37.092] Yes. First statement is one, then second print is giving me one and two. The third statement is giving me one plus two, three. So it also interprets your results. It is also executing and it is giving back what you have given. Okay? So [00:07:57.028] We will hope you have understood this print statement. Let's continue our discussion. Okay. So the program in Python has got number of statements. I say statement one, statement two, and a statement three. [00:08:16.420] Fine. Program in Python has got number of statements. These statements are executed one after the another. [00:08:26.404] Hopefully you're all able to understand. Print or whatever it is. In Python, I'm going to be executing this statement first, go to this statement second, and then you go to this particular statement. So it executes one after the other. [00:08:41.508] Okay, so when it is doing this, we call it as a program is running or program is getting executed. Please always remember that first always the first statement gets executed, followed by second statement, and followed by statement number three, and so on. [00:08:59.908] So is it always in sequence? At this point you understand it is always in sequence. Whenever it is required, we will try to deviate from the statement and we will uh specify the new one. [00:09:12.644] Okay. So hopefully you have understood the first point. The second one. Okay. So what I will do is, instead of me doing it as small p, I will print it as capital P. Okay. So I say print. [00:09:32.196] This is what let me do. I have saved my program. Let me go and so execute the program and if you see you're getting an output. Hopefully it is clear for you. [00:09:49.604] Yes. Let me execute. Let me execute this. [00:09:56.900] See? What does it say? It says, I'm trying to print. Am I deviating from the rule? No, there is no deviation from the rule. The reason is print spelling is correct. It's a function, so we have opened and closed, and I have given an argument. There is no problem, but

EV0000094 · EV0000095 · EV0000096 · EV0000097

Transcript block 3 (source interval 000600-000899 seconds): [00:10:12.708] What is happening? You're getting something called as an indentation error. Please remember this is a common error that you're going to be facing. Fine? So indot also says unexpected indent. [00:10:26.640] It also says unexpected intent. The reason is in Python in Python all statements all the statements unless a special construct [00:10:37.860] Unless a special construct is used, it should always start from the first column only. Fine. So now what was an error here? The error was I was giving some space. So by default, what should happen? It should start from first column only. [00:10:59.332] It has to

start from first column only. There's no option for us. So it has to start from first column. Okay. So hopefully you have understood what was the error. The error was if you can see there is a space here. This is fine. So now what do I do? I just go back here. [00:11:18.468] Fine. There is no space that is given. Make sure there are no spaces. You're going to be starting with the first column itself. Does it mean that you have to follow this process only for this first line of the program? No. You have to follow for all the lines of the program. So let me give this. Let's just check. Okay? So [00:11:39.332] FCLS and then execute and you get the output. So all statements unless you come across with a special construct, everything should start from the first column only. Okay. So executed this program. [00:11:58.308] And we all got the output. If you can see, there's no problem. Fine. If there are no syntax errors in the program, then title will execute the statements one by one. Okay. So I didn't do anything. [00:12:17.348] Fine. We have added one more statement. So it is executing, and also I'm getting it as an output. See, let me execute this. [00:12:31.428] See here, what is happening? It says print first one, I got one, one comma two, I got one comma two. [00:12:39.780] Print 1 plus 2, I'm getting 3, print 5, but when it comes here, it has stopped execution because there is an error in this case. So what is this error? This error is called as a zero division error or division by zero error. If you see [00:12:57.892] This error is called as a semantics error or it is also called as a runtime error. Okay? So runtime errors are those errors which are checked only when [00:13:12.004] A particular point is reached during program execution. Okay but if you had seen previously when I left a space it we did not get any output. Correct? Yes, we did not get any output. So what was the reason? The reason is the syntax is checked first. [00:13:31.140] Okay, if syntax, there are no syntax errors, then if there are any semantic errors or a runtime errors, it is going to be checked only when we arrive at that particular point. Okay? So hopefully is it clear? [00:13:49.988] Is it fine? Okay. So let's execute this. [00:14:00.740] [ASR low confidence; check audio] Secure. [00:14:06.148] So now what happened? I was trying to write two statements in a single line. Fine? Both Python both print statement in a single line, but it says invalid syntax. And if you could see it's called as a syntax error. Yes? So syntax errors are checked first and then [00:14:25.184] If there are no syntax errors, then only I will be looking for your runtime errors. Okay? Now what is the problem with this? [00:14:33.764] So in general, we cannot normally have more than one statement on a single line. Okay? So we cannot have more than one statement on a single line. So not more than single. [00:14:55.236] [ASR low confidence; check audio] Statement. [00:14:58.564] Not more than a single statement in the same line.

EV0000098 · EV0000099 · EV0000100 · EV0000101

Transcript block 4 (source interval 000900-001199 seconds): [00:15:04.964] So that's why we are getting an error. So until I rectify this error, this problem continues. So I have commented on this. Now what is the solution for this? The solution is let me copy paste, instead of me typing it again, I'll just copy paste it. Hope you don't mind doing this. [00:15:24.772] So come back and I'll say control V. So now if I do not want this to be an error, let me do this. You can use C colon in order to separate two different statements if you are trying to write them in a same line. So let's [00:15:43.620] Hopefully what have I done? This is your comment. I have commented. So this is going to be ignored by your Python. It doesn't consider this. So [00:15:55.460] You cannot normally have more than one statement on a single line. If you still want to do it, what you have to do? You have to separate them with a semicolon. If you ask me, avoid doing this. It's not a good practice. So let's try to avoid this practice. So we'll continue. [00:16:14.852] With our discussion. [00:16:24.740] Suppose While writing by mistakenly I write something like this. [00:16:38.180] I'll

try to write something like this. Is this okay? Is this okay is what we are trying to find out. Okay. So now in this case, let me exam and try to do this. I'll say CLS. [00:16:54.244] Execute, this is absolutely fine. Okay. So these are uh few ways if you can also write it in a multiple lines. It's going to add more confusion to you, but by chance if you have to deliberately do it, you can still do it. Okay. So let me execute [00:17:11.268] This is absolutely fine, and if you can see one has been typed here, printed onto your screen. [00:17:18.660] Okay, so anything that is within a double quotes or also within a single quotes, they are called as strings, fine, and anything without that it is called as your integer. Okay? So [00:17:32.580] These are few things that are uh uh clear for you. Okay. So now let me uh show you one more thing before I wind up uh this session. Okay. So I say I open double quote and I write first program something. [00:17:54.532] [ASR low confidence; check audio] See. [00:17:58.596] One which is troubling us a lot. So let me use this only and we will trouble it now. Okay. So what does it mean? So let me uh make sure that uh we don't get uh output in this case. So I have used uh double quotes at the beginning and let me use a single quote. [00:18:17.316] Let me save this. Okay, go to this and I'll say CLS. And then I say execute Python first dot PY. Can you say can you see what exactly it is? Print. [00:18:33.892] Double quotes, and there is a single quote, and it says end of the line while scanning the string literal. I was talking about string. [00:18:42.116] Okay? Anything that is enclosed within a single quotes or it is within a double quotes, it is called as strings. There's something also extra word that is used, literal. Literal means it's a constant. I would be speaking more about literal constant literals or a constants. [00:18:57.156] Okay so now what is the problem? And if you can see I'm not getting an output for any of these. Right? The reason is it's your syntax error. I will first check for the syntax. If there is no syntax, then I will check for your semantics. [00:19:11.456] Or runtime errors. So now what was the problem? The starting marker and the ending markers are wrong. So let me give it correctly. In the sense, you start with a single quote and you end with a double quote, or double quote and with a single quote, it is still going to throw you an error. Okay? So [00:19:31.524] Now you might be thinking, if you have done other programming language, if you have done your other programming language, and in order to write our first program, what we would have done, we would have used something called as a mean. [00:19:48.516] Right. We would use main as a function, then we also say open braces, and we also say closed brace. In between you try to say printf and all those things. Fine. So if you wanted to write a single

EV0000102 · EV0000103 · EV0000104 · EV0000105

Transcript block 5 (source interval 001200-001499 seconds): [00:20:02.852] Uh line program in case of other programming language, especially in the example that I've taken is C. Minimum number of lines of code that I have to write is 1, 2, 3, and 4. Whereas in case of the Python, if you remember, this was only first statement that we used. Is this fine? The reason is. [00:20:22.980] Okay. The other programming language, example C or your C plus plus, Java, they all have a special function which programmer has to write at the beginning, and this function is called as a main function. [00:20:40.868] Python doesn't follow that concept. The execution always starts from the first statements which is present on the top. Fine? So any statement that is present on the star uh on the top will by default becomes the first statement and it starts executing. [00:21:00.420] Okay. So hope it is uh clear for you. And

EV0000106 · EV0000107 · EV0000108 · EV0000109

Full transcript: 2967 normalized words, 82 timestamped segments, 1265.408 seconds duration. Low-confidence segments flagged: 3. Review locators: 00:14:00.740, 00:14:55.236, 00:17:54.532

EV0000090 · EV0000091 · EV0000092 · EV0000093

10 Program structure

This lesson is indexed from the source video titled Program structure. Its complete timestamped transcript is preserved below in segment order. The original video path and content hash are recorded in datasets/lecture-corpus.json. Transcript text is source wording; it has not been silently corrected or rewritten. A candidate slide deck is grouped here according to the current matching score; the exact path and page locators are listed in the source evidence. EV0000110 · EV0000111

SHORT-NOTE RECAP

Transcript block 1 (source interval 000000-000299 seconds): [00:00:00.452] Hello everyone, welcome back to the lecture series on Python for computational problem solving. I am Nika Ann Jacob from the Department of Computer Science and Engineering. And in this video, I will introduce you to the Python programming environment. [00:00:17.764] What is programming? Programming is a means in which we can communicate with the computer. And just like how we have several languages to communicate with each other, we have several languages that we could use to communicate with the computer as well. [00:00:36.260] For this course, we choose the Python programming language. When we want to communicate, we can do so in several modes. We could have a one-on-one discussion where I could say something and you can respond immediately, or we could write an email where I can [00:00:56.064] Put together all the information I want to pass on and email it to you. We could communicate via test text messages, via calls, and so on and so forth. Similarly, the Python programming environment gives us several modes of programming. [00:01:14.372] The first one is the interactive mode, which is similar to our one-on-one discussion, and the next one is the batch mode, which is similar to our email conversations. Now in the interactive mode, [00:01:27.684] All you have to do is open an interface for the interaction, type in the keyword Python, and start typing valid Python commands. Let me show you how to do so with an example. [00:01:43.524] So, to get to the interface, you would first have to open the command prompt in Windows or terminal in Linux. I will show you how to open the command prompt in Windows. So [00:01:58.628] Press the start button, type CND, and it opens the command prompt where you have the whole folder which is currently chosen. [00:02:10.372] Now this is a way in which I can interact with the operating system and give valid commands. So what I want to do is interact with the Python interpreter. So I type the command Python. [00:02:23.364] And what it does is opens an interface through which I can now communicate using Python commands. You see there are three angular brackets followed by a blinking cursor. The blinking cursor simply indicates that it's now ready to receive. [00:02:40.512] any command. So what I'm going to do is write a valid equation or a valid Python command that would add two numbers. So I'm going to use a number, the plus symbol and another number six. [00:02:56.228] Now when I hit enter, it's as though I have finished making one statement and the computer responds with a response. [00:03:06.724] The result here is just the sum of two numbers. I could multiply two numbers by using the asterisk operator. I could divide two numbers, one number by another, using the forward slash symbol, and so on. [00:03:26.340] What you can see here is for every command I type, it's going to give me the result as a response immediately. What if I want to type a collection of

commands and see the result at one go? In order to do that, we have another mode of programming called the batch mode. [00:03:45.860] In batch mode, we will type all the commands in one file [00:03:51.140] and you can run the file at one go itself. All you have to do is open your command prompt, type the command python, followed by the name of the file with a.py extension. [00:04:08.464] to indicate that you're trying to run a Python program. Let's look at how that is done. [00:04:15.076] So to type all the commands in a file, we are going to use GEdit as the software for this course. Now GED is a software that's common for both Windows operating system and the Linux operating system. So first let's create a new document. [00:04:34.212] You could choose to click the Create New Document icon or click on File. [00:04:45.892] And what you have to do is save it so that I can start writing a Python file. As of now, the G editor has just opened a file which is in plain text. So I'm going to save it. [00:04:59.844] And I want to save it in a specific folder so that I organize all my Python programs in one place. I'm going to store it in the D drive, which I can choose from this interface. And I've created a folder called Python PGN.

EV0000110 · EV0000111 · EV0000112 · EV0000113

Transcript block 2 (source interval 000300-000599 seconds): [00:05:15.780] So I've chosen this as the location and I'm going to give a name for my document. Since it's the first program, I'm going to write first and give an extension called dot PY to indicate it's a Python file. [00:05:31.428] So I'm going to choose the Python program folder and click on Save. [00:05:39.236] So you can see here the name of the file is first.py. Now let's type all the commands we typed in the interactive mode. I will type 4 plus 6. I typed 3 star 4 and 4 slash 2. [00:05:57.668] I've put all the commands in one place. I will save the file after the edits I've made. And in order to run the file, you have to use the Python command. But to do so, I first have to navigate to the folder where I have saved this file. [00:06:17.316] Now what I have to do is I've saved it in the D drive under the folder called Python underscore PGM. So to navigate to the D drive, all I have to do is type D and call it. [00:06:31.492] You can see the path change to D drive. Now I have to change the directory or the folder to Python underscore pgm. To do that, I'm just going to use the command cd space. Python underscore pgm. Cd stands for change directory. [00:06:51.620] So when I hit enter, you can see the path has now changed. I want to start fresh and remove all these previous lines. To do so, we have a command to clear the screen called CLS, which is for clear screen. And you can see that we start from the beginning. [00:07:10.148] So to run the first program I have to type the command Python. [00:07:17.412] Followed by the name of the program first dot py and hit the enter. You can see that there is no output, but it also doesn't have errors because all these statements are valid expressions. [00:07:33.124] Now, in batch mode, if I want to see the result of a particular statement, I have to explicitly do so. In order to do that, you have a special function called the print function. A function is something that takes input and gives you an output. [00:07:52.356] Now the output is ideally returned back to the program, but your print function does not return anything back to the program. It just send the value to the output screen. [00:08:04.644] So to give an input, you have to use a function call operator, which is a pair of round brackets. And within that, I can type any valid equation. So I'm just going to type 4 plus 6 within the pair of round brackets. [00:08:23.076] I will save this program and then let's move on and execute it again. [00:08:29.796] So now when I execute or run this program, you can see I have one output here. I have the value 10. Although I have four values or four lines of commands, it only gives me 10. Because in the batch mode, you need to tell the Python interpreter what exactly it must do. [00:08:49.444] This is how you could write programs with multiple commands. Now, to write our first official program and start the

whole exercise of programming, I would like to tell you a couple of standards that we must follow when we write any Python program going forward. [00:09:08.740] Now this is called the program structure. Let's look at how any program that we write should be structured. First we have to understand that the Python interpreter follows a sequential execution. [00:09:23.652] Which simply means that the commands would be executed in a sequential order, which is just the same order that you have written. It will always be executed one after the other. [00:09:38.084] The next thing is that you must always include comments to indicate the intent of the program or the line that you've written in that program. So that anyone who reads your program understands your intention or the aim of that particular program. [00:09:56.772] The execution always starts from the first column, and I will demonstrate that with an example as well. Python is also case sensitive, so when we type the commands, we have to ensure to be careful whether we're using uppercase characters or lowercase characters.

EV0000114 · EV0000115 · EV0000116 · EV0000117

Transcript block 3 (source interval 000600-000899 seconds): [00:10:15.460] We should also understand the syntax of a language. Now what is syntax? Syntax is basically a set of rules. Now you understand what I'm trying to tell you because I I'm adhering to the grammatical rules of the English language. [00:10:31.888] If I want to make a statement say you I write program menu. [00:10:39.076] That makes no sense because I have not adhered to the grammatical rules of the language. Similarly, the Python program also has some set of rules that we must follow that we can have an effective communication with the computer. [00:10:57.028] And I'll also tell you how many lines you should write in a given program or a given line for a program. Let's look at it with examples. So I'm going to create a new file using the icon, going to save it and call it [00:11:15.780] Program structure [00:11:23.396] the extension program structure dot py now the path is already selected the folder is selected because I used it just now I will save it now like I indicated the first line should always have a comment [00:11:39.396] To indicate the intent of the program. So here we're going to look at the program structure. So that's what I would write. Now if my intention can be summarized in one line, I can use a single line comment. [00:11:55.876] So the hash symbol tells the interpreter that anything the hash symbol can be ignored and not interpreted. So these are your single line comments. Now what if I want to give a description about what exactly this program would do? [00:12:15.780] Communicate with the computer, or why did I write this program? It could pan across multiple lines. In order to do that, we have something called as a documentation string, which is a pair of three double cords or a pair of [00:12:35.652] three single quote so this is called your documentation stream [00:12:47.396] use it for my line comments. [00:12:54.468] plan across multiple lines. Both of these are valid documentation streams and if I were to save and run this program the output would be a simple [00:13:10.116] Blank output. So let's clear this. And it's Python Programstructure.py. And when I execute, there is no output because all of these are comments. And comments would be ignored by the Python interpreter. Now [00:13:27.268] Let's write the cliched statement that we would for every programming language, which is your hello world program. So I'm going to write print hello world. [00:13:43.268] So what I've done is passed a string or a sentence to the print function. Now any collection of [00:13:52.484] Alphabets or symbols that you'll find on your keyboard. Keyboard if it's enclosed within a pair of single quotes or double quotes is a valid string. So if I save this program and run it, you see the output as hello world. [00:14:11.172] Now what if I want to just put a space to write the hello world or just say hey this [00:14:23.012] [ASR low confidence; check audio] The size of

space. [00:14:29.092] And you can note that I've used double quotes. This is also a valid string, but [00:14:35.044] If I were to run this program, it says it's an indentation error. It says there's an unexpected indent. An indent basically means a formatted rule. So never use unnecessary space in your program because spaces in Python have a special meaning. [00:14:54.148] So your execution always starts from the first column. So this gives you an error. Now I've also told you that Python is case sensitive.

EV0000118 · EV0000119 · EV0000120 · EV0000121

Transcript block 4 (source interval 000900-001199 seconds): [00:15:06.372] So when I type something in English, I always know a sentence must begin with a capital letter. So I'm going to just write hello here. But I've written the print statement with a capital B. I'm going to save this program. [00:15:25.124] Clear the screen and check the art. I see a new error. It says name error. It prints hello world because it understood the print statement in line 11, but then it says there's a name error, saying print is not defined. [00:15:41.796] We know that print is defined, right? But since Python is case sensitive, [00:15:47.652] Print starting with the lowercase p and print starting with an uppercase p have different meanings. So always ensure that you follow the rules and always write the print statement with the lowercase p. This proves that Python is case sensitive. [00:16:08.100] Now there are some other syntax rules that when I'm writing strings or if I'm using brackets, that I should always match the opening and closing construct. So if I were to write print say word. [00:16:27.460] and close it. You notice that I've written print, open the string with a double quote, written the word world, closed it with a single quote, and I close the function called parameter. Now we clear the screen, run the program. [00:16:46.884] You see another error which says EOL while scanning string literally. EOL stands for end of line. So what happens is the Python interpreter starts [00:17:00.804] looking at the string and keep searching for the string to end and which will only end when it encounters another matching double point. Now [00:17:14.020] When it reaches the end of the line, the Python interpreter always treats the end of the line as the end of a valid statement. So what it does is when it's trying to go to the next line, it cannot find a matching double quote. And that is why it gives you that error. So if I have to [00:17:30.532] Correct this error, all I have to do is match it with a double code. Now the reason I'm commenting all the lines that give you an error is so that I can continue writing the program and not have it stop abruptly. [00:18:04.420] The next thing that I want to explore is what if I want to write my print statement across multiple lines. Now I already told you that the end of the line indicates the end of a statement. So if I want to write print and say new line. [00:18:25.508] What would the output be? Let's check that. You can see that it only prints hello world and world. The print line [00:18:37.220] In line 16 doesn't give me an output, and in line 17 doesn't give me an output. Because here the name of the function has a value which is not displayed, and this is also a valid string. [00:18:54.468] Similar to if I would write 4 plus 6 and not get an output, all of these are considered as valid values. This is just a function because functions in Python have a value. This is just a string within brackets and this is an expression. [00:19:12.004] So if I want to pan my print command across multiple lines, I will use something called as an escape character. An escape character [00:19:27.108] is your backslash symbol which says that anything following that immediately should be ignored. So what I'm going to do is write print [00:19:37.476] Put a backslash character, then when I hit the enter key, that enter key is going to be ignored and I'm going to now write line 22 because it's written on line 22. So when I save this program and run it.

EV0000122 · EV0000123 · EV0000124 · EV0000125

Transcript block 5 (source interval 001200-001499 seconds): [00:20:00.132] You can see it gives hello world, world and line 22. Everything else is ignored because I have now used the escape character to ignore the next line and it will only print the line 22. [00:20:18.340] Now this is how you could pan it, this is one way you can pan it across multiple lines. Another way is to match the constructs. Construct of the bracket. So I could write print. [00:20:35.396] Open the function call bracket. The next line I'm going to write line 25. [00:20:45.156] hit another enter and the closing bracket. So when I do that, what the interpreter does is it says okay this is a function and the function is being activated because it has the bracket. [00:20:58.692] And then I have line 25, and the interpreter will continue searching until it finds a matching closing bracket. So now I'm going to save it and run the program again. You can see that it prints line 25 as well. [00:21:17.988] Because here, even though I've hit enter, when you use the brackets until it finds the matching closing bracket, the interpreter keeps looking for values that it can take as input. [00:21:32.932] So this is your generic program structure. There are obviously many rules which we will explore as we go through the contents of the course. So let's quickly recap. [00:21:44.484] For a good program, always start with comments. Use the hash symbol for single line comments and a pair of triple double quotes or triple single quotes for multi-line comments. [00:21:56.868] It's case sensitive, so when you write a print with an uppercase P or a lowercase P, it would be different. The number of statements that have to be per line ideally should always be restricted to one, so that when someone reads the program, it's easy for them to do so. [00:22:16.068] If you want to pan one statement across multiple lines, you can either use the escape sequence or you could use the brackets as your constructs. [00:22:27.652] And you can have multiple statements in one line separated by a semicolon. Let's look at how multiple statements can be separated by a semicolon. [00:22:41.092] So in line twenty seven I'm going to write print say one [00:22:51.332] And I'm going to write print and two. I've written two statements here, but let's see what's going to happen when I do run this program. [00:23:07.716] Spare screen and run it. It says it's an imbalance syntax because on line 27, when the interpreter reaches this matching bracket, it treats it at the end of a command. Now most programming languages use semicolon or in [00:23:26.948] indicates that a semicolon must always be used to terminate a command. [00:23:32.804] Here in Python, the enter keyword would suffice, the enter key itself. Now, if I want to explicitly have multiple commands per line, all I have to do is use the semi-colid, which will indicate that [00:23:49.316] This part is one valid command, the semicolon ends that command, and this would be another valid command. So let's save that and run it, and you can see it says one and two in different lines. So this is your generic program structure. [00:24:09.124] So always ensure that you remember these basic rules and we will go through many more rules through the course of the subject. Thank you.

EV0000126 · EV0000127 · EV0000128 · EV0000129

Full transcript: 3256 normalized words, 96 timestamped segments, 1460.074667 seconds duration. Low-confidence segments flagged: 1. Review locators: 00:14:23.012

EV0000110 · EV0000111 · EV0000112 · EV0000113

11 print function

This lesson is indexed from the source video titled print function. Its complete timestamped transcript is preserved below in segment order. The original video path and content hash are recorded in datasets/lecture-corpus.json. Transcript text is source wording; it has not been silently corrected or rewritten. A candidate slide deck is grouped here according to the current matching score; the exact path and page locators are listed in the source evidence. EV0000130 · EV0000131

Graphs and diagrams

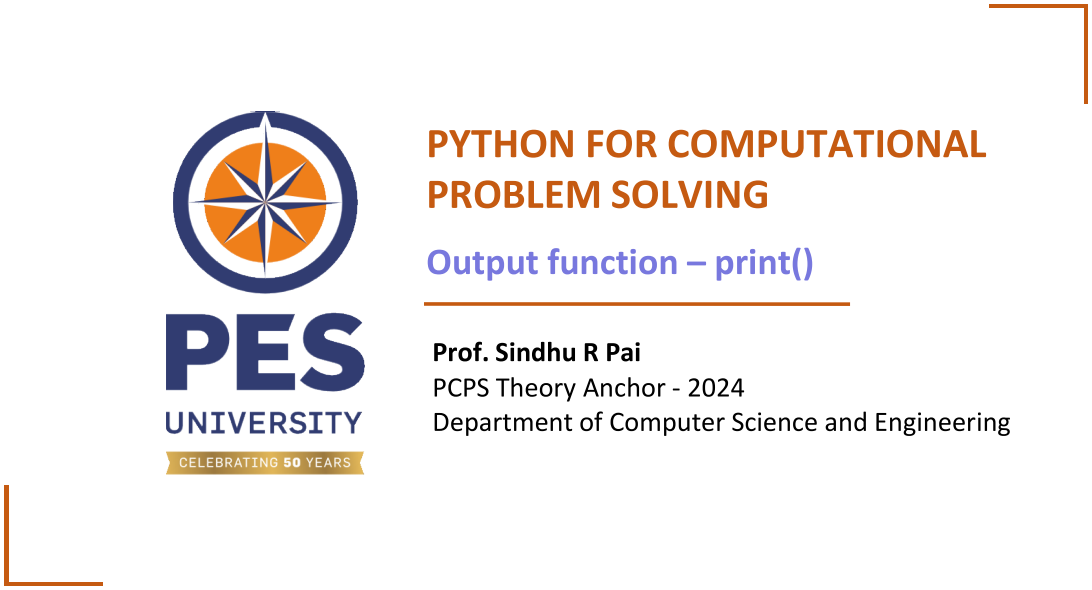


Figure G000156. UE24CS151A PCPS Unit 1 SindhuRPai Class 9 output function print · source slide page 1.

EV0000796

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Output Function - print

- This function takes any argument and as a part of the processing, displays the value of the argument to the output screen.
- Calling is done using a call operator - **()**
- **Characteristics:**

  Can display/print anything in the world.

  Has the capability to evaluate the expression.

  Can take any type of arguments and any number of arguments.

  Formatting is possible to some extent using the keyword separators – sep & end

Figure G000157. UE24CS151A PCPS Unit 1 SindhuRPai Class 9 output function print · source slide page 2.

EV0000799

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

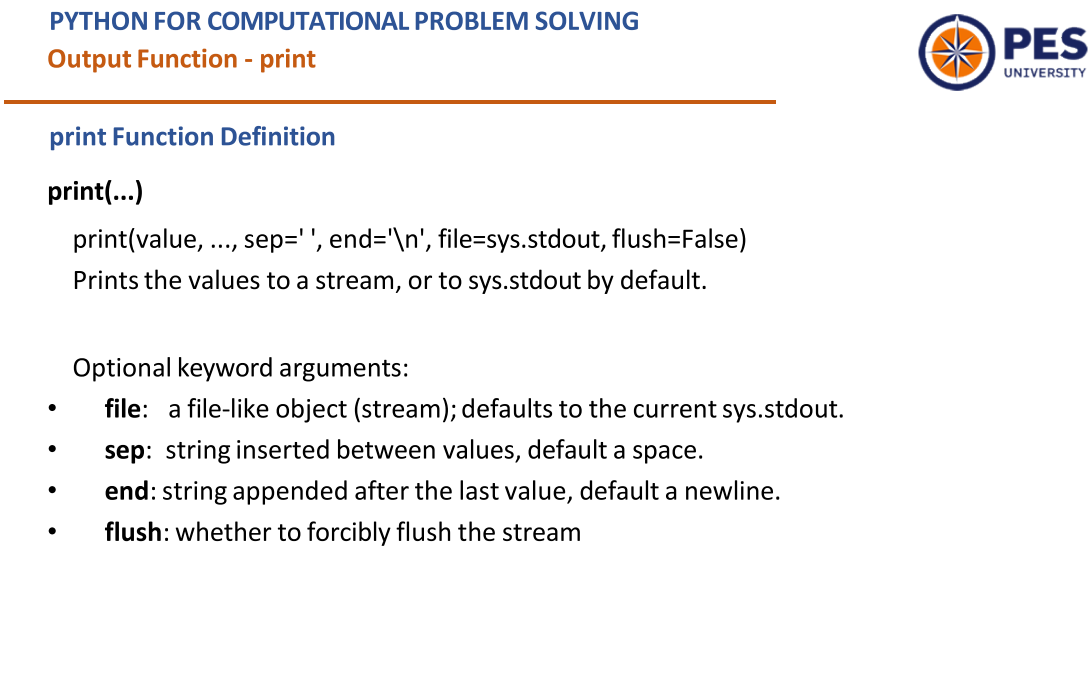


Figure G000158. UE24CS151A PCPS Unit 1 SindhuRPai Class 9 output function print · source slide page 3.

EV0000802

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Output Function - print

PES UNIVERSITY

Configuring the Space between the output fields

- By default, the output field separator is a space character

```
>>> print(10,12,14)
10 12 14
```

- To change the output field separator, use the **sep** argument

```
>>> print(10,12,14, sep = ":")
10:12:14
```

Figure G000159. UE24CS151A PCPS Unit 1 SindhuRPai Class 9 output function print · source slide page 4.

EV0000805

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

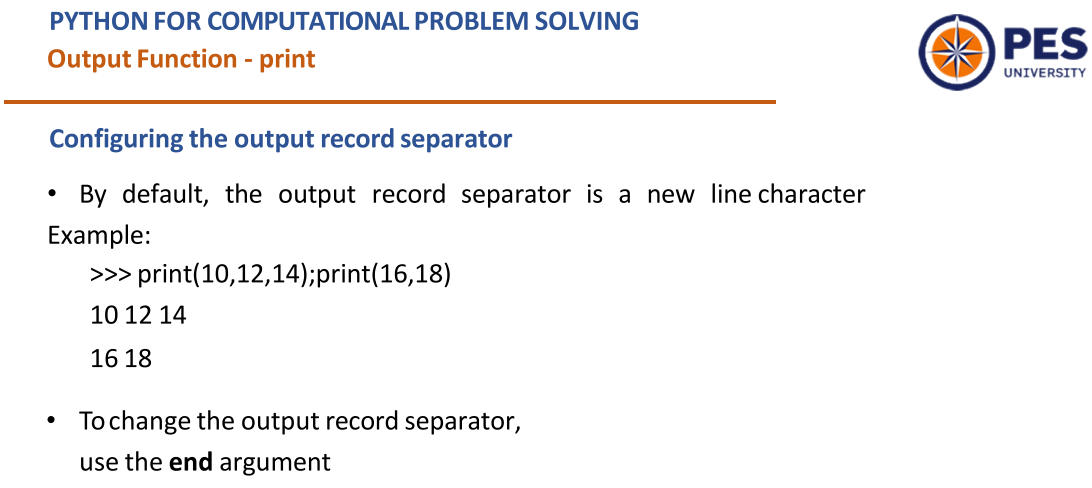


Figure G000160. UE24CS151A PCPS Unit 1 SindhuRPai Class 9 output function print · source slide page 5.

EV0000808

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING
Output Function

PES UNIVERSITY

Demo of print using different examples

Figure G000161. UE24CS151A PCPS Unit 1 SindhuRPai Class 9 output function print · source slide page 6.

EV0000811

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

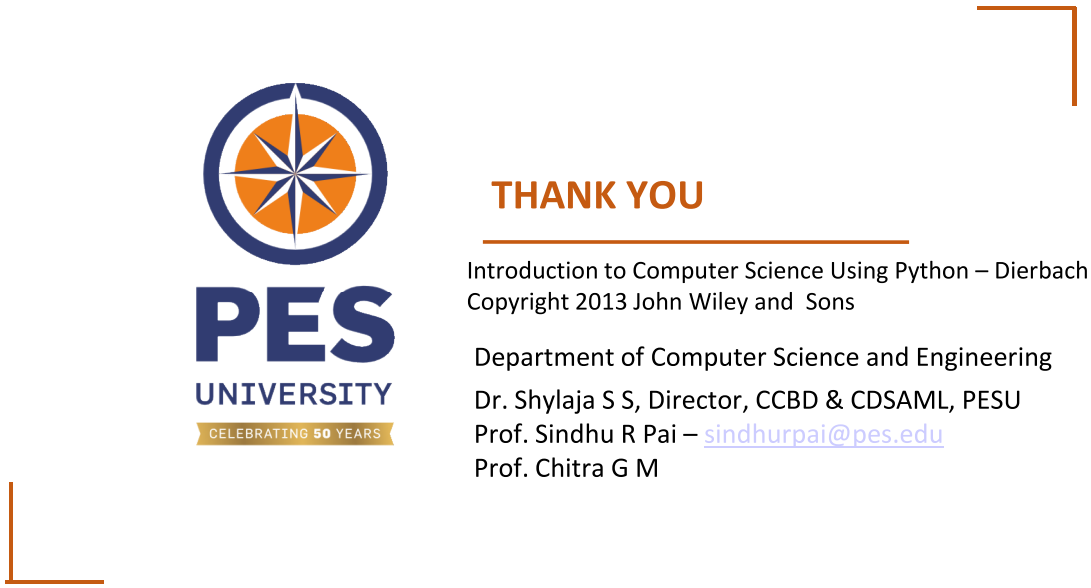


Figure G000162. UE24CS151A PCPS Unit 1 SindhuRPai Class 9 output function print · source slide page 7.

EV0000814

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Important points and why

1. Source slide page 1 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Output function – print() Prof. Sindhu R Pai PCPS Theory Anchor - 2024 Department of Computer Science and Engineering EV0000795 · EV0000796 · EV0000797
2. Source slide page 2 (extracted text): PYTHON FOR COMPUTATIONALPROBLEM SOLVING Output Function - print • This function takes any argument and as a part of the processing, displays the value of the argument to the output screen. • Calling is done using a call operator - () • Characteristics:Characteristics: Can display/print anything in the world. Has the capability to evaluate the expression. Can take any type of arguments and any number of arguments. Formatting is possible to some extent using the keyword separators – sep & end EV0000798 · EV0000799 · EV0000800

3. Source slide page 3 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Output Function - print print Function Definition print(...) print(value, ..., sep=' ', end='\n', file=sys.stdout, flush=False) Prints the values to a stream, or to sys.stdout by default. Optional keyword arguments: • file: a file-like object (stream);defaults to the current sys.stdout. • sep: string inserted between values, default a space. • end: string appended after the last value, default a newline. • flush: whether to forcibly flush the stream EV0000801 · EV0000802 · EV0000803
4. Source slide page 4 (extracted text): PYTHON FOR COMPUTATIONALPROBLEM SOLVING Output Function - print Configuring the Space between the output fields • By default, the output field separator is a space character >>> print(10,12,14) 10 12 14 • Tochange the output field separator, use the sep argument >>> print(10,12,14, sep = ":") 10:12:14 EV0000804 · EV0000805 · EV0000806
5. Source slide page 5 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Output Function - print Configuring the output record separator • By default, the output record separator is a new line character Example: >>> print(10,12,14);print(16,18) 10 12 14 16 18 • Tochange the output record separator, use the end argument >>> print(10,12,14, end = " ");print(16,18) 10 12 14 16 18 EV0000807 · EV0000808 · EV0000809
6. Source slide page 6 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Output Function Demo of printp usingg different examplesp EV0000810 · EV0000811 · EV0000812
7. Source slide page 7 (extracted text): THANK YOU Introduction to Computer Science Using Python – Dierbach Copyrighth 2013 Johnh Wileyl andd Sons Department of Computer Science and Engineering Dr. Shylaja S S, Director, CCBD & CDSAML, PESU Prof. Sindhu R Pai – sindhurpai@pes.edu Prof. Chitra G M EV0000813 · EV0000814 · EV0000815

SHORT-NOTE RECAP

Transcript block 1 (source interval 000000-000299 seconds): [00:00:01.348] Hello, all. Welcome back. In today's session, we will understand the concept of output functions. Hope you're all ready. Let's get started. [00:00:18.468] Fine. So let me start as usual. Let's save our file. So I'll save it as all files. [00:00:32.612] Put as usual dot PY Yes. And the terminal is also ready. Let me give it as Cls. So let's get let's get started. The output function that we are going to be discussing is print. Right. [00:00:51.812] Print is an output function that we are going to be discussing. So let me comment on this. Now the moment I make use of your parentheses, I'm trying to invoke or call a function. [00:01:07.428] Okay, so what is the use of this print statement? Print statement is used to display. [00:01:17.700] So we will try to find out what are the characteristics of this print statement. [00:01:29.732] Are the characteristics. Is the spelling correct? [00:01:39.268] started the first characters take S [00:01:44.996] print statement, print can take [00:01:51.972] Any number of arguments. [00:01:59.428] Take any number of arguments what is that? Suppose I say print one, it's fine. I can also say one, comma two, I can say one, comma two, comma three, four, and so on. So take arguments. [00:02:22.212] So we'll quickly do this. I'll just try to write a simple program to demonstrate it for you. So I'll say print one, and the next I'll say print one comma two. [00:02:38.564] [ASR low confidence; check audio] Yeah. [00:02:42.308] So be one comma two comma three and so. So take any number of archetypes. Execute this for you. And in this case. [00:03:00.260] seen I'm getting the required. This is the first characteristic of the output function. Okay so now what I have to do? I have to comment all of them and then I can do it individually

like this. [00:03:16.580] What if I have a hundred lines and I have to do all the lines individually? Fine, commenting all the lines individually may not be feasible, so what I will do is I'll try to comment all of them together by making use of triple double quotes. [00:03:36.388] Point you understand it is used as a comment. Later on in your coming classes, I would change the statement. Okay so first characteristic is print can take any number of arguments. Let's go ahead. [00:03:56.772] [ASR low confidence; check audio] Take. [00:04:00.228] Any type of argument. It can make any type of [00:04:10.564] You know what is arguments, a function. Print is a function name. The moment I use a parenthesis, I am trying to invoke a function, and anything within this is called as an [00:04:24.068] Now what am I telling? I'm telling it can take any type of argument. What does it mean? It means I can say it's an integer, it's a float, string. [00:04:40.356] Anything in the double quotes is called as a string, so I will write it as a string. I can also specify a complex number. [00:04:50.116] Okay in any order. I can use them in any order. So it takes number of arguments, any number of arguments, and any type of arguments it can take. So let me execute this.

EV0000130 · EV0000131 · EV0000132 · EV0000133

Transcript block 2 (source interval 000300-000599 seconds): [00:05:10.852] This giving us the required up. Suppose if I give this okay. [00:05:27.876] Something? Okay. And let's [00:05:37.124] How many number of arguments are there in this case? Argument one and argument two. Will you know it is one and two? The moment you separate it with the comma, each one of them are treated as a separate argument. So I have an argument one and then followed by an argument two. [00:05:53.924] Okay, so now let's see what will happen to these arguments. Go and execute your program. Okay. [00:06:05.572] See my previous print statement is getting printed at the same time I'm also able to get the output. In this case, each arguments are evaluated separately. [00:06:28.964] Each argument is evaluated separately and it is treated as an expression. So hopefully these two things are clear for you. Let me extend this so that whenever I execute, I will not be executing the same statements. Let me do this. [00:06:49.444] Now going ahead, if you have seen the output, I was trying to give one comma four point five comma something, but where are my commas? [00:07:01.956] Almost doesn't exist. And followed by, if you can see, there is a space between two different arguments. [00:07:11.364] Fine, it's a space between two different arguments. At the same time, the moment I use a print statement and from one print statement to the next print statement, I'm going to the new line. Yes, we are basically going to the new line. So now why is it? [00:07:29.092] Who is doing all these things for us? So in order to understand what we will do is we will use or we will first get into Python terminal. [00:07:41.316] And then we will try to check. I'm interested in finding out for the print statement. So in order to do it, you can specify it as help followed by a print. So in this case, let's see. [00:07:58.276] The syntax for print is print can take any number of values value one, value two, value three, and so on. Fine. Each value is separated by a comma. Then you can see there is a field called a separator field. SEP is a separate field. [00:08:16.836] At the same time, we do have one more field called present end field. By default, separator field is a space and an end field is a new line. Because of which we are getting [00:08:30.896] Okay, so we are getting between two different arguments it's a space and between two different print statements it is a new line. Okay so next question is Can I change this behavior? Certainly, yes, we can change this behavior. [00:08:50.244] One comma two comma three comma four and I'll see separator is equal to oh sh what shall we use as a separator maybe some stars. Okay. This is the first thing that I will do. Okay so now if you can see [00:09:08.164] Between every

argument, I'm getting stars as your separator. So now what I can do is we will also add one more field with this, which is called as an end, and we will try to rewrite its behavior by giving nothing. Okay. [00:09:27.812] Uh in general it was your new line and field was having a new line. So now I'll say this and if you can see I'm getting [00:09:39.524] the terminal that is present on the same line. Okay. So hopefully you understood. Now what I will do is I'll just make use of a print statement here. [00:09:53.732] The moment I use a print statement, you could see that I'm not using anything, there's no arguments, there's nothing, and by default I will come to the new line, which means even though

EV0000134 · EV0000135 · EV0000136 · EV0000137

Transcript block 3 (source interval 000600-000899 seconds): [00:10:07.088] You're doing certain changes here for the separator field and for the end field. It is temporary, okay, it is not permanent in nature. [00:10:18.916] Hope uh you would have understood this particular behavior. There are uh few things that you should be very careful, okay? We want you to check with these things, uh execute them. [00:10:33.136] and then uh try to understand the behavior. Okay? So let me do few things for you. [00:10:44.420] So anyway we have commented. Hopefully you have understood. I'll just write it here. We have understood the separator fields and [00:10:54.276] D and field. We would have understood, okay, with the previous example. So now uh let me uh do few things for you so that quickly will understand. I'll say print. Okay, so I'll say two [00:11:12.356] And followed by and is equal to this. Okay, and I'll see I want to do the same thing, so let me copy paste instead of writing. Okay. [00:11:34.084] I do not want anything. [00:11:40.868] What is the output according to? Okay. Can you quickly let me know what is an output in this case? So let's find out. To come out of this. Let me do this and I'll say pythonoutput.py. [00:12:01.892] Here so what did we do? And we do not want we want it to be um without any space, nothing, so we've set it as an end as this yet is equal to this, and all okay. So this is the behavior of your print statement. [00:12:23.428] [ASR low confidence; check audio] Let me quickly execute a few more things for you. [00:12:30.884] Please trust me, they are tricky. You have to execute them and then find out what is the output. Okay so let me say it as separator is equal to [00:12:50.180] [ASR low confidence; check audio] It has separator. [00:13:03.076] Yeah. [00:13:14.020] [ASR low confidence; check audio] The [00:13:22.212] And I'll see [00:13:41.060] Okay. So what am I trying to do? [00:13:45.860] [ASR low confidence; check audio] Let's see. [00:13:53.892] try doing but something is happening that's absolutely fine. I just want to uh uh say what is this okay separator to [00:14:11.300] What's [00:14:14.724] I'll see print [00:14:19.268] Anyone guess what is the hour that is gonna be there here? Okay, so find out. [00:14:28.804] Okay, so if you see, if you see what is happening here, India Modi because of this statement, I say Namo special, then followed by [00:14:41.764] Separator field is very here, so it is going to be Namu, Very Special, followed by print separator. Our new value, the earlier value of a separator was very, and we were expecting it to be [00:14:56.676] Fine very here. Yes, we wanted the output to be very, but we are getting the output to be something special. Okay. The main reason is in this case when we do, when we do, it does not create

EV0000138 · EV0000139 · EV0000140 · EV0000141

Transcript block 4 (source interval 000900-001199 seconds): [00:15:16.292] Variable separately. [00:15:20.292] Does not create a variable separately called as SEP, but instead it is going to make use of SP that you have given previously. So this SCP is different from this particular SCP. Please always remember this. Okay. This is fine. [00:15:40.964] Let's go ahead. [00:15:47.588] So let me comment on this and let me do the other things for you. Okay. So I will use the same thing. Okay.

So let me copy and paste the same thing and do a simple change here. [00:16:09.348] Okay, what am I trying to do? Okay separator is very so let me remove this I don't need [00:16:29.892] Changes here so that the statement comes fine for us. [00:16:36.900] These things. Okay, so let's find out what does this output refers to. Okay. So now in this case, let me execute any guesses here. [00:16:55.812] the output dot p y and I'll say so what are we getting [00:17:04.804] Fine, so this print statement, there's no doubt about it, it gives us the way in which we were expecting. In the second case, we are seeing print nummo, vary and SCP. What is the value that is assigned for an SAP? In this case it is something special. [00:17:23.748] In this case, this SCP is treated as a variable of a program. Okay, variable of a program. [00:17:36.100] Refers to the variable in this particular program and that is why what is the output that we are getting? We are getting an output to be something special where we have assigned it to something special. [00:17:50.180] Okay, so hopefully these are few things that are uh clear to you before I wind up this particular session, I request you to check with all these things, execute them. [00:18:04.096] do the required changes to find out and understand the behavior of your separator and the end fields. Okay? So the output function that we were discussing was print. It is used to display. [00:18:17.488] The characteristics, any number of arguments it can take, any type of arguments it can take, it evaluates the arguments as an expression separately. The behavior [00:18:32.260] We looked into with the help of separator and end fields. We also manipulated the diff uh the behaviors. Okay. So there are a few more things that you can definitely try out. Hope you understood the concepts. Fine. Happy learning. Thank you.

EV0000142 · EV0000143 · EV0000144 · EV0000145

Full transcript: 2016 normalized words, 81 timestamped segments, 1134.869333 seconds duration. Low-confidence segments flagged: 6. Review locators: 00:02:38.564, 00:03:56.772, 00:12:23.428, 00:12:50.180, 00:13:14.020, 00:13:45.860

EV0000130 · EV0000131 · EV0000132 · EV0000133

12 Variable video 1

This lesson is indexed from the source video titled Variable video 1. Its complete timestamped transcript is preserved below in segment order. The original video path and content hash are recorded in datasets/lecture-corpus.json. Transcript text is source wording; it has not been silently corrected or rewritten. No slide deck is assigned to this video; candidate slide sources remain separate in the unit master. EV0000146 · EV0000147

SHORT-NOTE RECAP

Transcript block 1 (source interval 000000-000299 seconds): [00:00:01.572] Hello, all welcome back. Python for computational problem solving. So what are we going to be doing in today's class? In today's class, we are going to be uh dealing with the concept of variables. Okay so what are we going to be doing today? So it is variables. [00:00:22.852] variables. Fine. Now uh in order to understand the concept of variables there are a few things let's quickly go to them and then uh continue with our discussion. Okay so now I have written an eight cross eight grid here. [00:00:39.504] Okay. So there is an eight cross eight grid. Just bear with me for the diagram. I thought uh this would be useful, so I drew it now. Okay, so now [00:00:50.532] I consider each one

of them as a cell. Okay? So there are how many number of cell? There are totally eight number of cell. So this is eight cross eight cell. What am I trying to represent here? [00:01:04.624] I'm just giving you a notion of something called as your memory. Okay? I'm just talking about a memory here. Okay? So now in this case [00:01:17.828] Yes. Okay. So this is a memory. Now what is a memory? A memory is what memory is made up of, in fact. [00:01:27.108] So now a memory is made up of cells. How many of cells? We will try to find out. At this point I'll say memory is made up of N number of cell, okay? N number of cells, and each cell is capable of holding [00:01:44.528] one bit of information each cell is capable of [00:01:48.752] holding one bit of information. So now what is that bit? In case of your computer system it can understand only zero and one. So I call this bit as zero or it could be one. So it is capable of holding only one bit of information. And this bit is either zero or it is going to be [00:02:06.448] To be one. Okay. So now thus n number of bits put together, I call it as one byte. [00:02:15.748] Okay? So now N number of bits put together, I call it as a byte. What is the value of N at this point? Eight. So in general, if somebody speaks in terms of how many bytes [00:02:28.176] And if it is one byte, you have to consider it as how many number of bits? It is going to be your eight bits. Thus, thus. Okay, so now eight bits, eight bits put together, I get one byte. [00:02:46.148] Okay, so eight bits put together I get one bite. Fine. So now this entire thing is one bite for me. This entire thing is a bite. [00:02:58.960] Fine. Suppose if I combine N number of bytes together. So this is byte one, this is byte two, byte three, byte four, and so on. So now n bytes put together I call it as a word. [00:03:14.884] I call it as a word. Okay? Hopefully it is clear. Bit to byte, bytes to words. And number of bytes put together, we call it as a word. Okay? In in general [00:03:28.036] Fine. What does this n could be? N could be 2, it could be 4, it could be 8, and so on in terms of powers of 2. Okay? So [00:03:39.364] At we uh at this point, let us assume that my word is basically a 4 byte. Okay, let's assume this it's a 4 bytes. So which means what does a 4 byte means? Byte is equal to 8 bits, so it is going to be 4 into 8, which is going to be your 32 bits. [00:03:59.940] Fine. Is this clear? Four into eight, thirty two. So there are totally thirty-two bits. So if I want to get four bytes, then I have to make use of your thirty-two bit. Okay? And this thirty-two bit I call it as a word length. I call it as what? Word length. [00:04:17.540] Hopefully it is clear. It's called as word length. Right? It's called as word length. Now what does this word length basically mean? It means that whenever you want to read [00:04:30.628] Because the basic operation that you can do with the memory is either read or you can write. Okay? Thus. [00:04:38.820] If a word length is thirty-two bits, which basically means that either you perform a reading operation or you perform a writing operation, at the same time you can access these many number of bits at a same time. Is this fine? So you can operate [00:04:56.260] In this case, up to 32 bits at a same time. Okay? So hopefully these things are clear for you. All we want you to understand is bytes, bits, and word.

EV0000146 · EV0000147 · EV0000148 · EV0000149

Transcript block 2 (source interval 000300-000599 seconds): [00:05:11.460] Okay. So with this, let us uh continue uh the discussion with the variables. Okay so let me take up a new sheet here so that uh I can explain the required things for you. Okay, so let me take this [00:05:31.396] Okay. Yes. Hopefully uh you can see. Yes. Let's get started. So what are we talking about? Uh topic is variables. [00:05:43.364] Okay. So you might be thinking why it is a board work or uh writing work. Let's do some sort of writing work and then uh uh we will uh go back to the execution mode and then uh I will demonstrate on all the things that are required. Okay? Let's basically understand the concept first. [00:06:00.900] Fine. So now what is a variable? A variable is a name. A variable is

a name. It is also called as an identifier. [00:06:12.292] It's also called as an identifier. Okay? These identifiers are associated with a value. They are associated with a value. Okay? So a variable is created [00:06:29.956] A variable is created the moment you assign a value to it. Let me repeat, when does a variable gets created? Where does a variable gets created? In the memory itself. Okay? So a variable is created the moment you first assign a value to it. [00:06:48.240] Okay? So let's uh uh try to understand what does this statement mean. So uh suppose my variable name is num is what I will [00:07:00.228] Take. Okay. Variable name is num. Now, for this num, I will associate a value, I will associate a value which is called as 10 in this case. This is a point where my variable is created within the memory. [00:07:19.588] Is this clear now? Okay. So now what can be the name of my identifier? Can I use my name itself as an identifier? Yes. [00:07:29.680] Chitra is equal to 10. Absolutely fine. For example, your name is something X, X is equal to 20. You can use anything. But make sure that you use a valid uh in the sense a a proper uh [00:07:43.440] identify your name because it should uh help you in programming. Chitra one, chitra two, chitra three, four, five. It doesn't make any sense to make uh use of such variable names, but still there is no harm in using them. [00:07:58.208] Okay, so hopefully it is clear for you. Now if you are already um familiar with the other programming language, in case of other programming language, especially C if you're doing, what do I need to do is I have to first say it as int A or int num. [00:08:17.636] Fine. And then I say NAM is equal to 10. Okay. These are the two statements that I will have to write in case of the C program. Now what does this mean? Okay so in this case, the moment the first statement is executed. [00:08:33.476] A location is given to you, right? How many number of bytes? Because we were talking about bytes. How much memory is given? Memory is given in terms of bytes for you. Okay so if it is in this two bytes. [00:08:46.480] Two bytes of memory is given. If it has four bytes, four bytes of memory is given. Fine. And uh I forgot to also tell you one important uh point uh with respect to uh the memory. Fine. So now as I said, eight bits together. But how will I access the bits? [00:09:05.712] Accessing the bits is done with the help of accessing the address. So every memory location is associated with an address. Please remember this. Okay? Every memory location is associated with an address. So I'll write it here so that you can [00:09:22.272] Get to know, okay, every memory location is associated with an address. Okay. Similarly, the moment we do this, we would have been associated with an address here. [00:09:34.212] Okay, it's an address. So working with an address is a difficult task. That's why we are making use of a name or an identifier. So I say it as num. The moment you come across with this statement. [00:09:48.676] Before this, some junk value will be there. What value? Let's not worry about it. Any value that is present is not our value, so it's I'm going to be calling it as a junk value. So the moment I say num is equal to 10, what will happen? The 10 gets associated with that location.

EV0000150 · EV0000151 · EV0000152 · EV0000153

Transcript block 3 (source interval 000600-000899 seconds): [00:10:08.644] Is this fine? So this is what happens in case of the other programming language. But what happens with Python? Fine. These two statements have been replaced by a single statement. Okay? These two statements are replaced by a single statement. [00:10:27.204] Now, the reason is we do not need to specify it as int, float, string, cap, what are they? They are all called as your data types. Okay? They are called as the data types. Okay what are the data types that we were trying to discuss? Integer. [00:10:44.944] float string, okay? There is no separate data type called as a character in case of Python. Both character as well as a strings, that just treated in a was little different way.

EV0000154 · EV0000155 · EV0000156 · EV0000157

Full transcript: 1670 normalized words, 44 timestamped segments, 660.586667 seconds duration. Low-confidence segments flagged: 0.

EV0000146 · EV0000147 · EV0000148 · EV0000149

13 variable and id function

This lesson is indexed from the source video titled variable and id function. Its complete timestamped transcript is preserved below in segment order. The original video path and content hash are recorded in datasets/lecture-corpus.json. Transcript text is source wording; it has not been silently corrected or rewritten. A candidate slide deck is grouped here according to the current matching score; the exact path and page locators are listed in the source evidence. EV0000158 · EV0000159

Graphs and diagrams

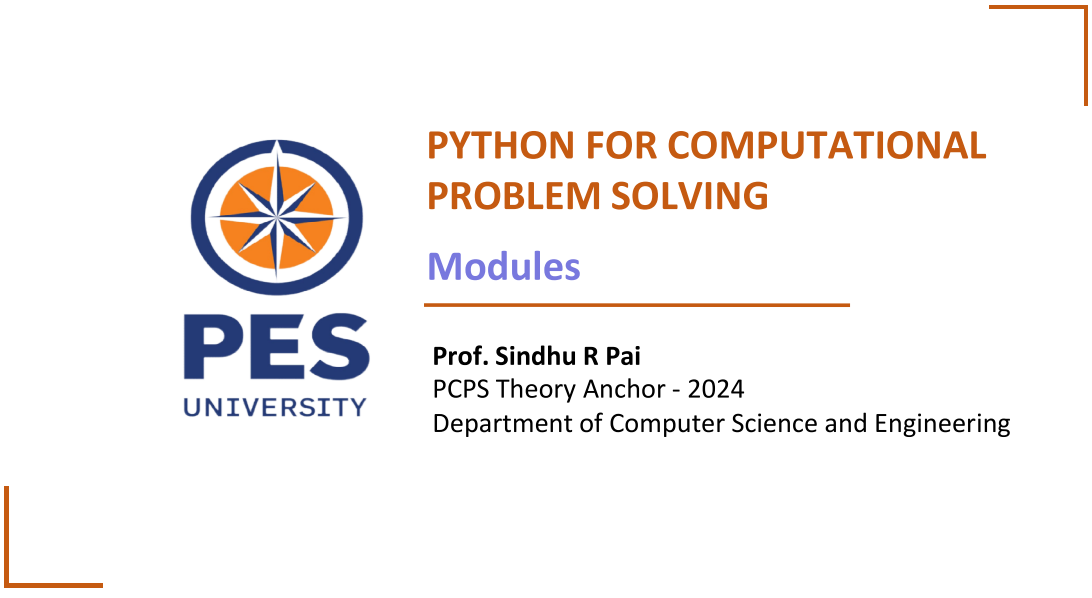


Figure G000049. UE24CS151A PCPS Unit 3 Sindhu Class 14 15 Modules · source slide page 1.

EV0000475

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

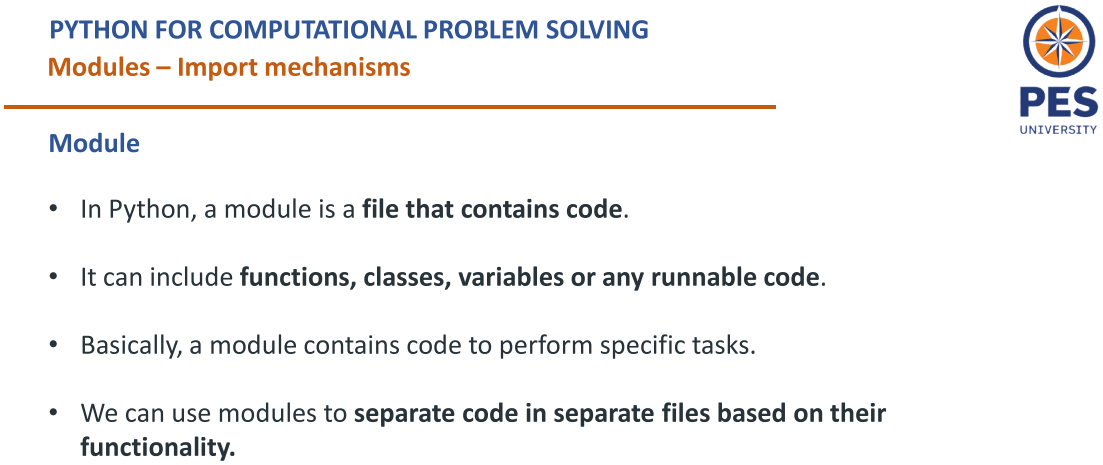


Figure G000050. UE24CS151A PCPS Unit 3 Sindhu Class 14 15 Modules · source slide page 2.

EV0000478

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Modules – Import mechanisms

PES UNIVERSITY

Advantages of Modules

- **Reusability** - makes the code reusable
- **Modularity** – Organizing the code into modules logically
- **Separate scopes** – separate namespace is defined by the module
- **Grouping** - Python modules help us to organize and group content by using files and folders

Figure G000051. UE24CS151A PCPS Unit 3 Sindhu Class 14 15 Modules · source slide page 3.

EV0000481

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Modules – Import mechanisms

PES UNIVERSITY

Packages and Namespace

- Modules in Python can be grouped together into **packages**.
- Packages organize the code into logical groups and provide a namespace to the modules so that they **don't conflict with modules with the same name in other packages**.
- The import statement can also be used to import modules from a package, which allows to access the functions and classes defined in the package's modules.

Figure G000052. UE24CS151A PCPS Unit 3 Sindhu Class 14 15 Modules · source slide page 4.

EV0000484

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Modules – Import mechanisms

PES UNIVERSITY

Packages vs. Modules vs. Libraries

- Modules – contain several functions, variables, classes etc.
- Packages – contain several modules.

  - folder that contains various modules as files.
- Library - a collection of packages and modules used to access built-in functionality

Figure G000053. UE24CS151A PCPS Unit 3 Sindhu Class 14 15 Modules · source slide page 5.

EV0000487

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Modules – Import mechanisms

PES UNIVERSITY

Types of Modules

1. **Built-in Modules**
   - Python's standard library comes bundled with a large number of modules.
   - They are called built-in modules.

2. **User-defined Modules**
   - Any file with .py extension and containing Python code is a module.
   - It can contain definitions of one or more functions, variables, constants, classes etc.

Figure G000054. UE24CS151A PCPS Unit 3 Sindhu Class 14 15 Modules · source slide page 6.

EV0000490

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

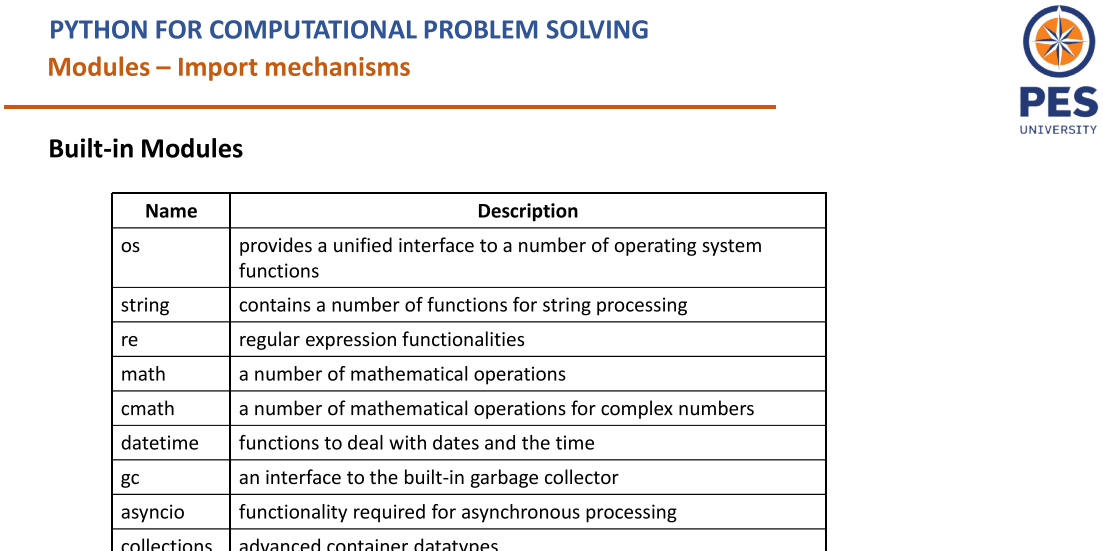


Figure G000055. UE24CS151A PCPS Unit 3 Sindhu Class 14 15 Modules · source slide page 7.

EV0000493

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

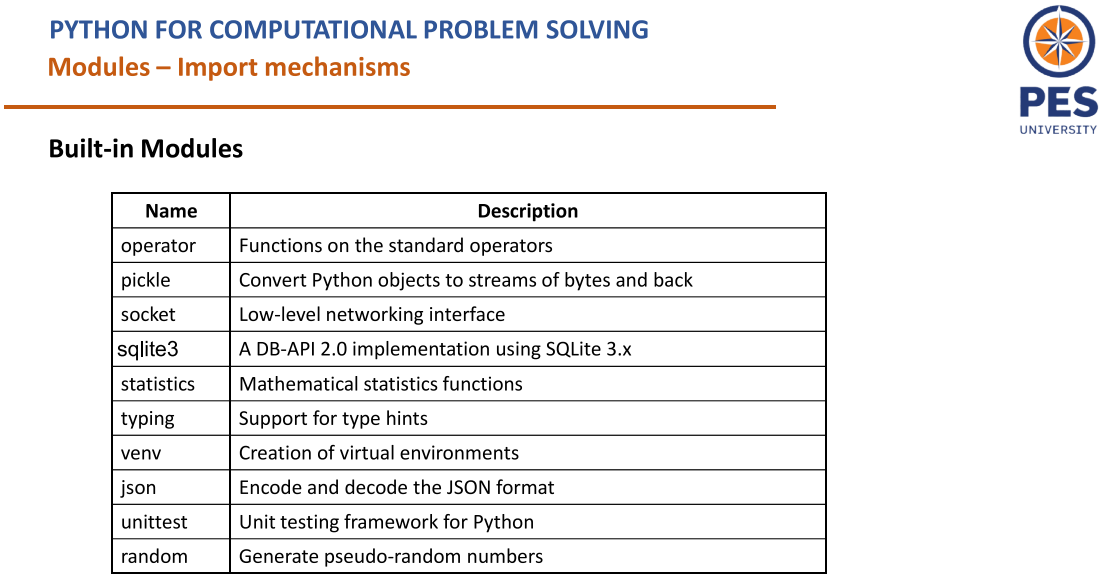


Figure G000056. UE24CS151A PCPS Unit 3 Sindhu Class 14 15 Modules · source slide page 8.

EV0000496

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Modules – Import mechanisms

PES UNIVERSITY

User-defined Modules

- Any file with .py extension and containing Python code is a module.
- It can contain definitions of one or more functions, variables, constants as well as classes.
- Any object from a module can be made available to the interpreter or another Python script by using import statement.

Figure G000057. UE24CS151A PCPS Unit 3 Sindhu Class 14 15 Modules · source slide page 9.

EV0000499

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

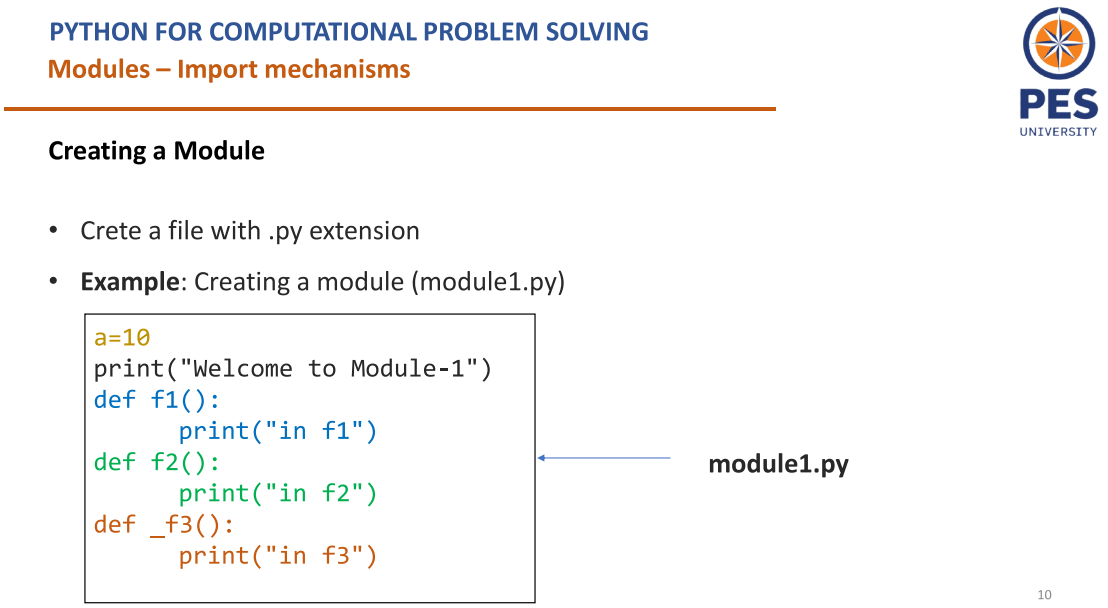


Figure G000058. UE24CS151A PCPS Unit 3 Sindhu Class 14 15 Modules · source slide page 10.

EV0000502

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Modules – Import mechanisms

PES UNIVERSITY

Importing a Module

- When the interpreter encounters an import statement, it imports the module if the module is present. Otherwise, *ModuleNotFoundError* is thrown.
- Syntax - `import module_name`
  import – the keyword used to import the module
  module_name – name of the module to be imported
- To access the functions inside the module the dot(.) operator is used.

Figure G000059. UE24CS151A PCPS Unit 3 Sindhu Class 14 15 Modules · source slide page 11.

EV0000505

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

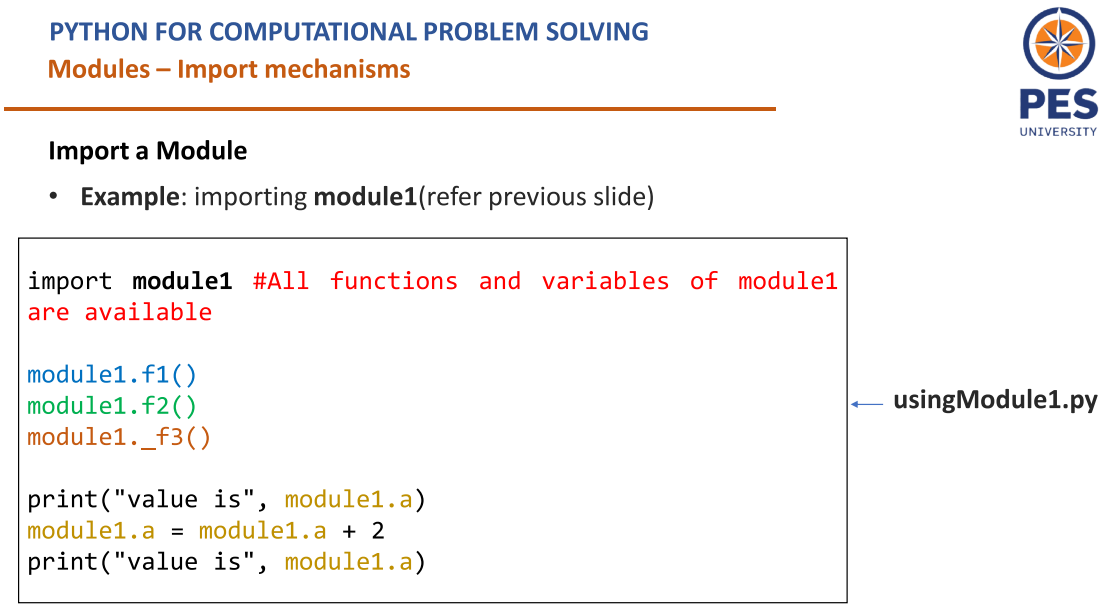


Figure G000060. UE24CS151A PCPS Unit 3 Sindhu Class 14 15 Modules · source slide page 12.

EV0000508

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

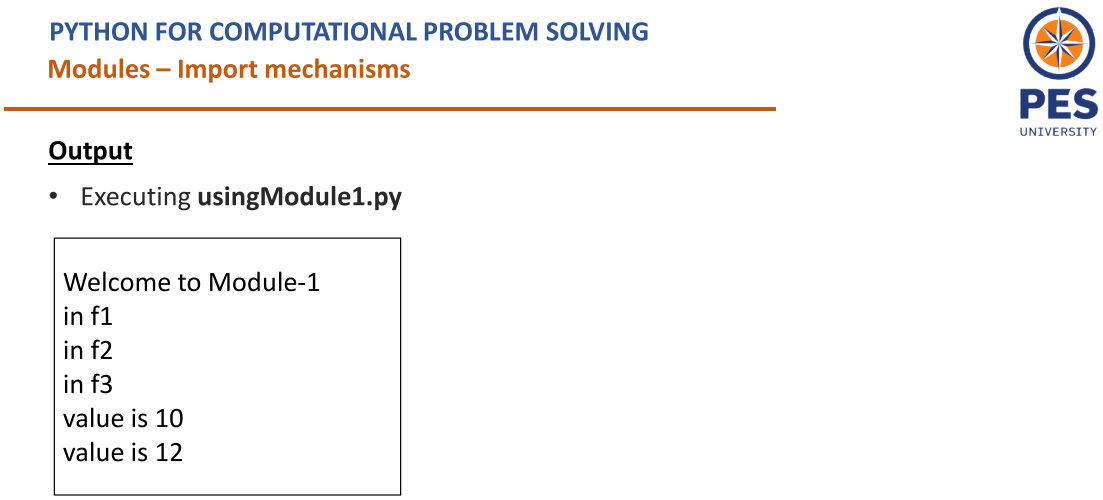


Figure G000061. UE24CS151A PCPS Unit 3 Sindhu Class 14 15 Modules · source slide page 13.

EV0000511

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

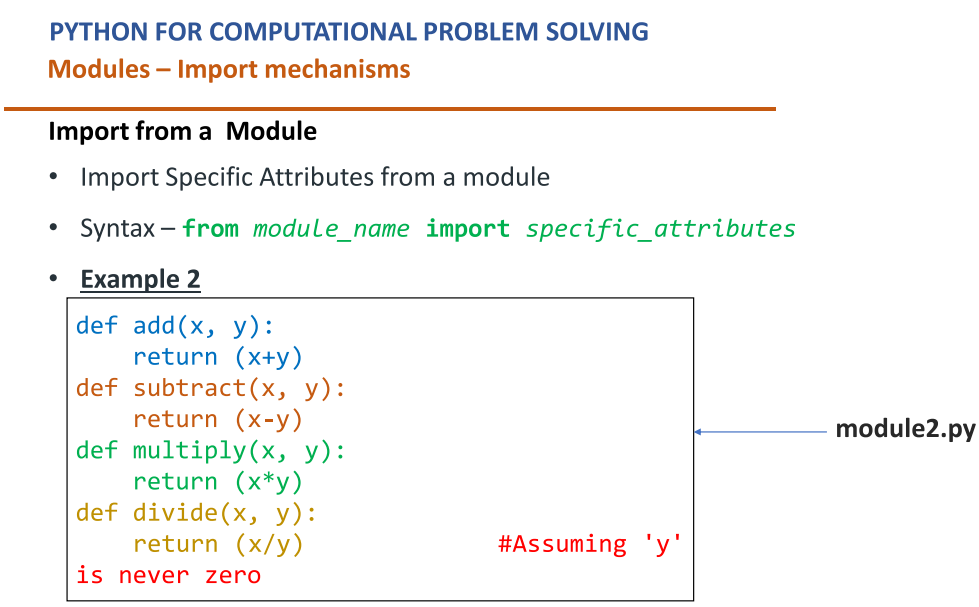


Figure G000062. UE24CS151A PCPS Unit 3 Sindhu Class 14 15 Modules · source slide page 14.

EV0000514

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

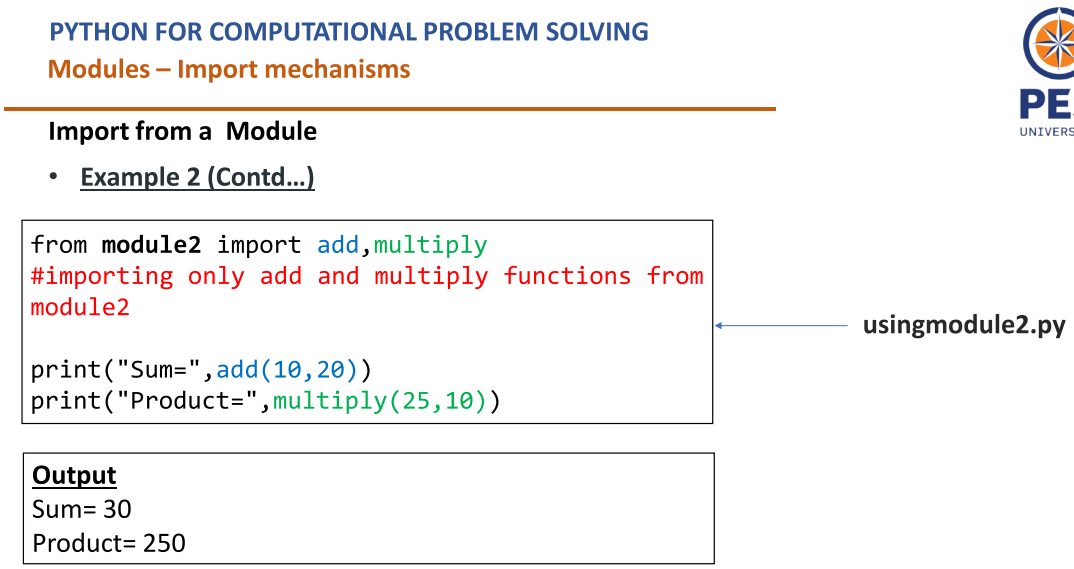


Figure G000063. UE24CS151A PCPS Unit 3 Sindhu Class 14 15 Modules · source slide page 15.

EV0000517

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

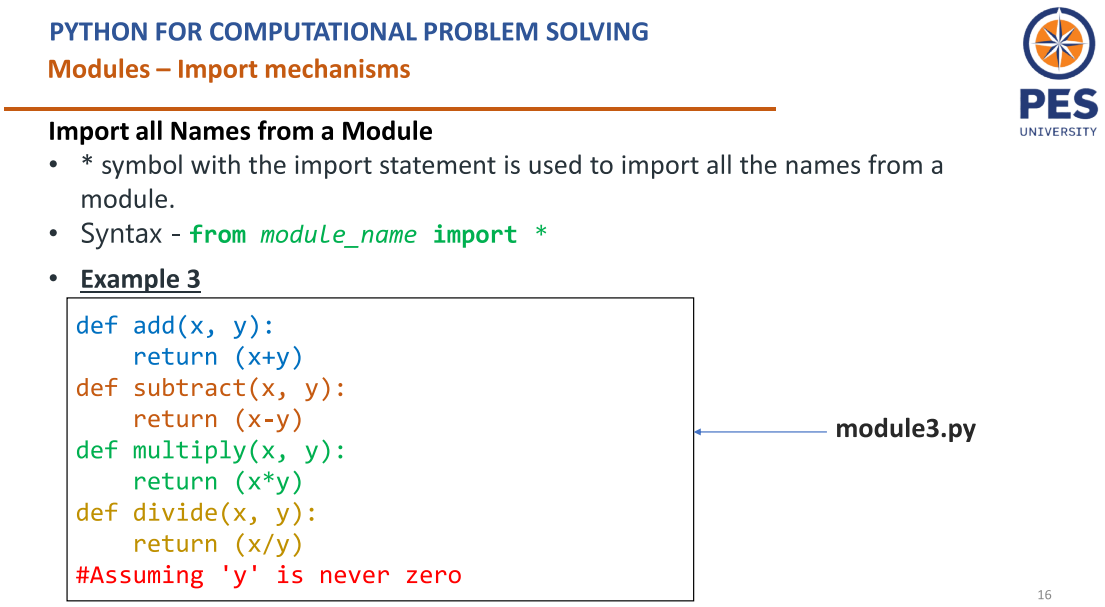


Figure G000064. UE24CS151A PCPS Unit 3 Sindhu Class 14 15 Modules · source slide page 16.

EV0000520

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

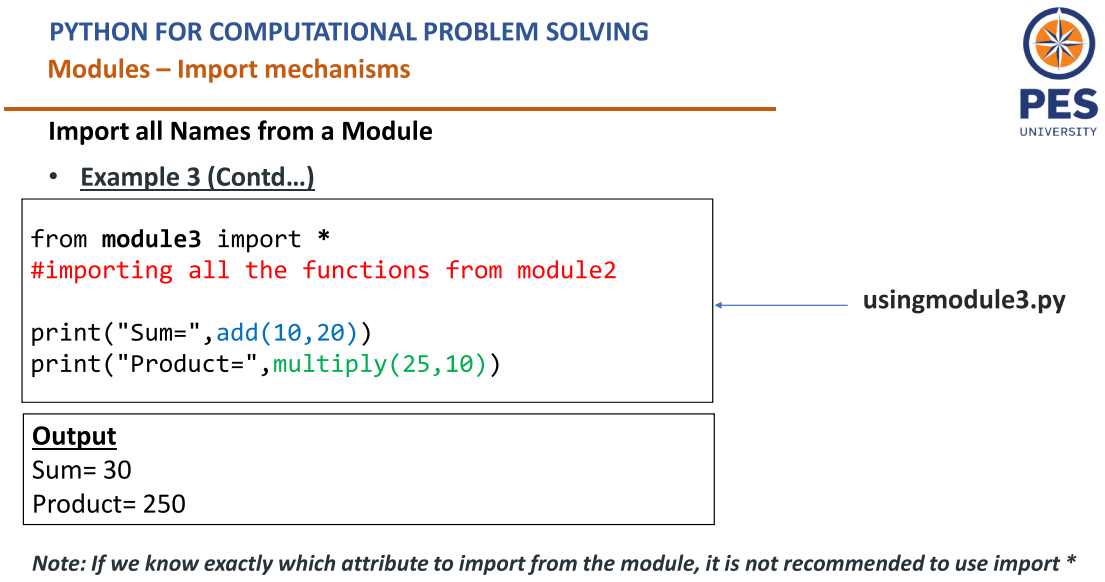


Figure G000065. UE24CS151A PCPS Unit 3 Sindhu Class 14 15 Modules · source slide page 17.

EV0000523

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

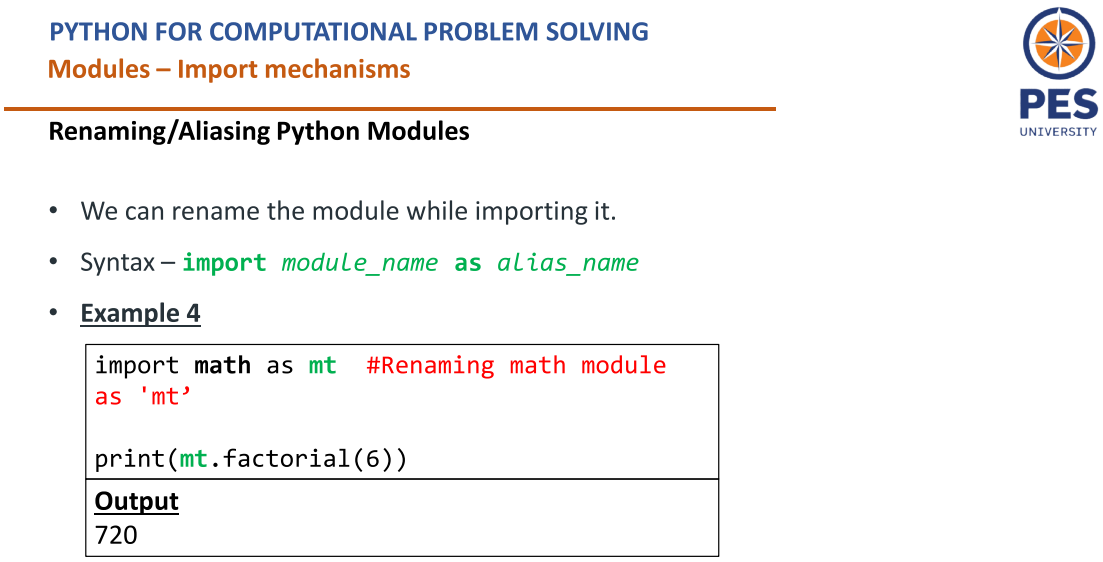


Figure G000066. UE24CS151A PCPS Unit 3 Sindhu Class 14 15 Modules · source slide page 18.

EV0000526

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

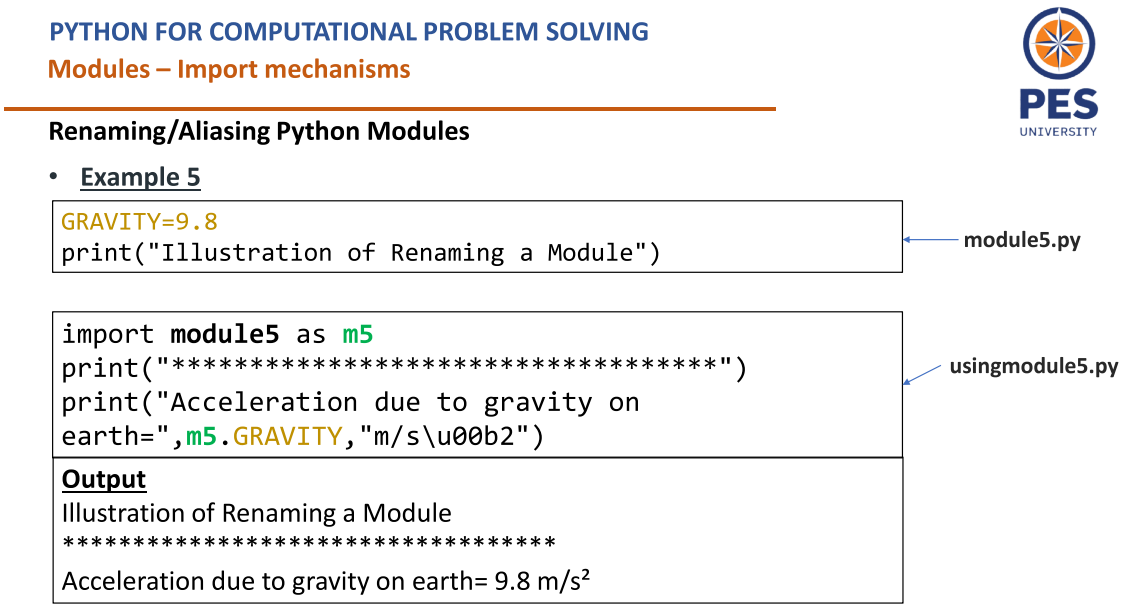


Figure G000067. UE24CS151A PCPS Unit 3 Sindhu Class 14 15 Modules · source slide page 19.

EV0000529

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

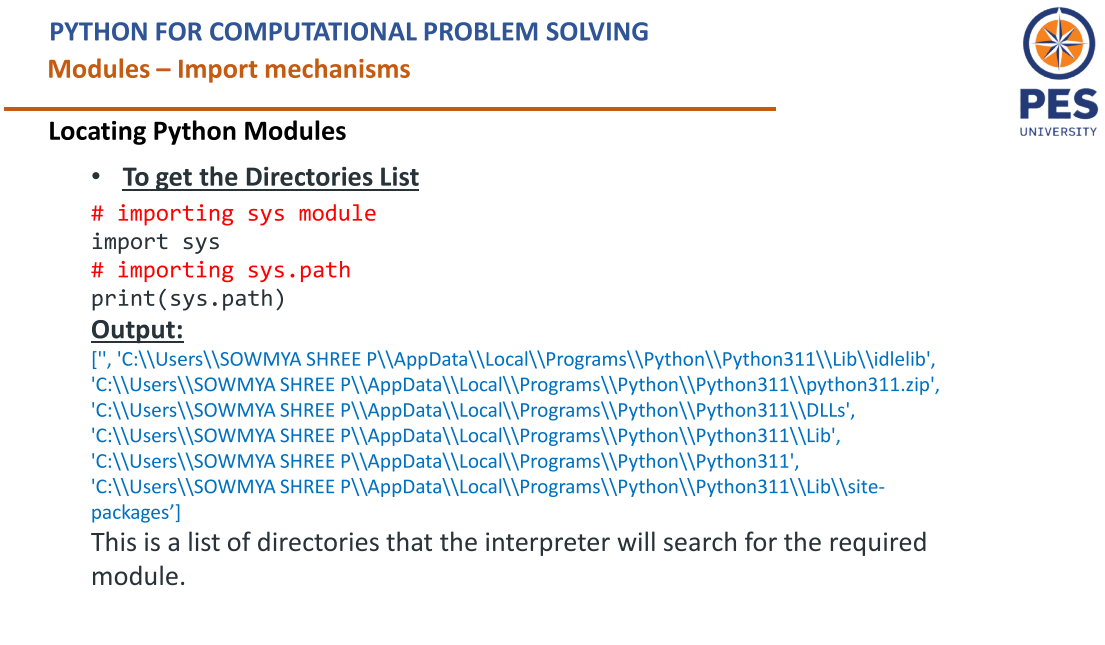


Figure G000068. UE24CS151A PCPS Unit 3 Sindhu Class 14 15 Modules · source slide page 20.

EV0000532

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

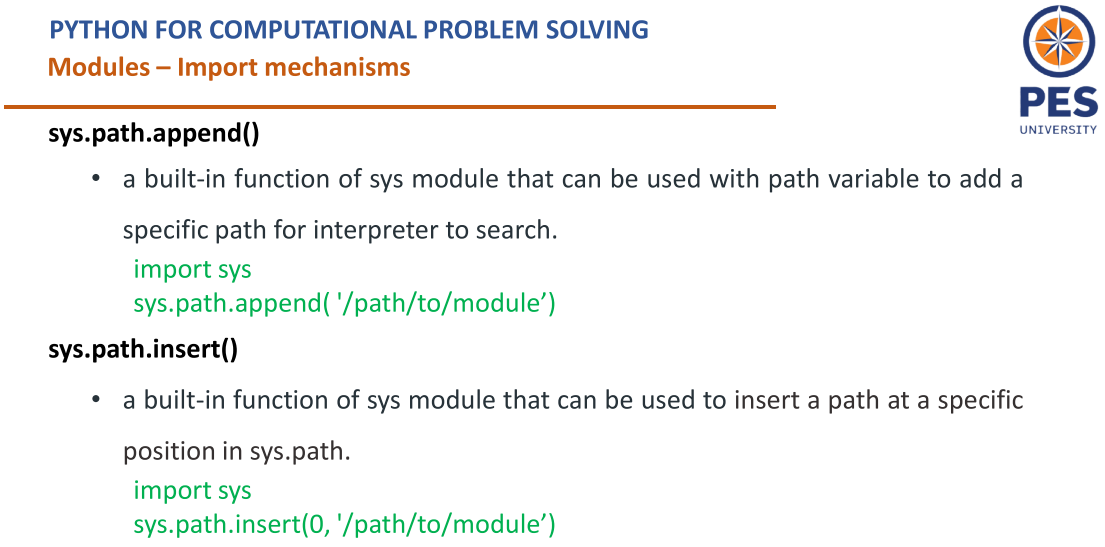


Figure G000069. UE24CS151A PCPS Unit 3 Sindhu Class 14 15 Modules · source slide page 21.

EV0000535

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Modules – Import mechanisms

PES UNIVERSITY

__doc__ variable

- In Python, each object(class, function, variable,..) can be documented using Docstrings.
- Docstrings can be accessed using the __doc__ attribute.

```
def add():
    '''Performing addition of two numbers.'''
    a=10
    b=7
    print(a+b)

print("Using __doc__:")
print(add.__doc__)

print("Using help:")
help(add)
```

Output

```
Using __doc__:
Performing addition of two numbers.
Using help:
Help on function add in module __main__:

add()
    Performing addition of two numbers.
```

Figure G000070. UE24CS151A PCPS Unit 3 Sindhu Class 14 15 Modules · source slide page 22.

EV0000538

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

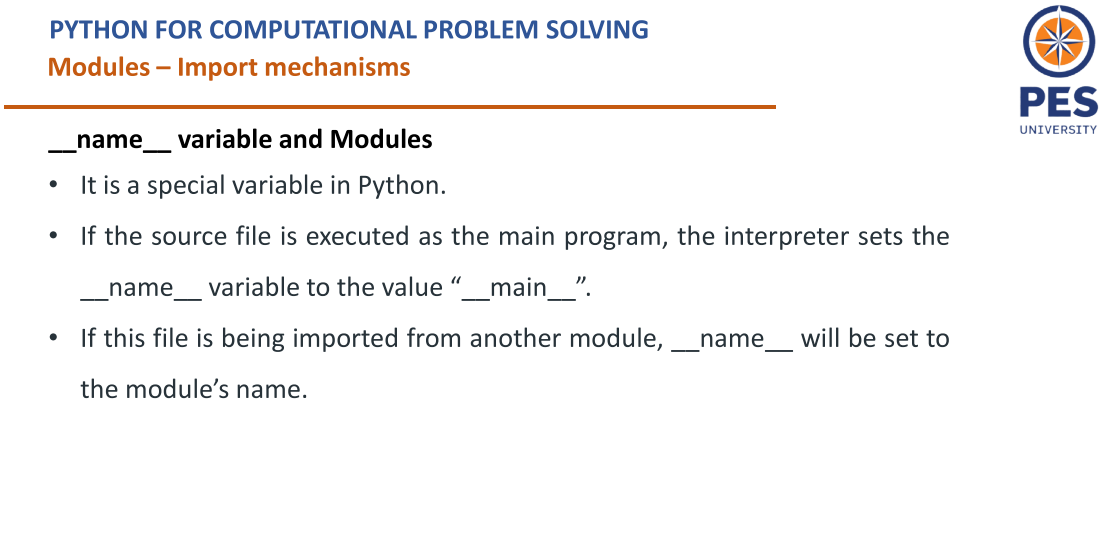


Figure G000071. UE24CS151A PCPS Unit 3 Sindhu Class 14 15 Modules · source slide page 23.

EV0000541

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

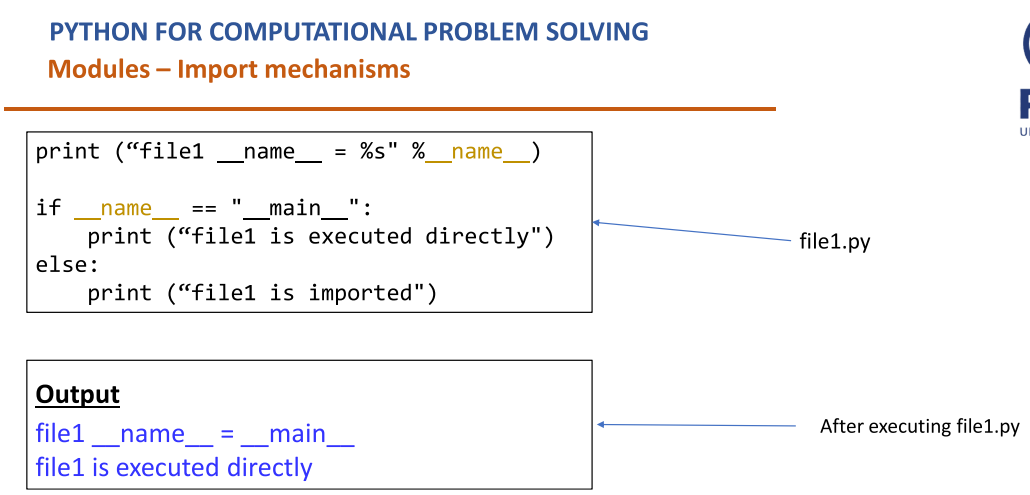


Figure G000072. UE24CS151A PCPS Unit 3 Sindhu Class 14 15 Modules · source slide page 24.

EV0000544

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

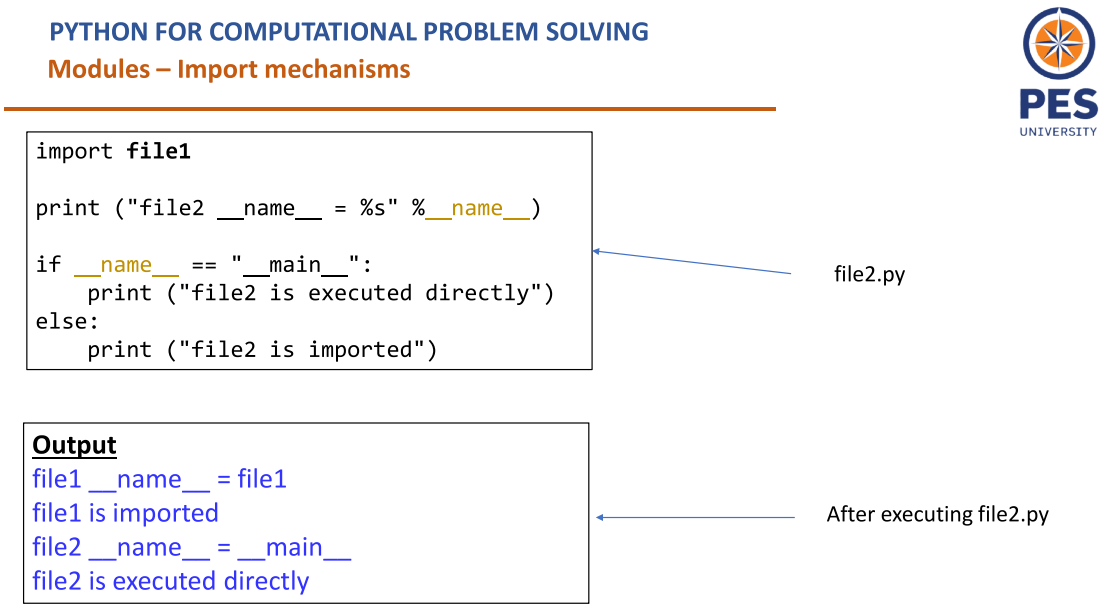


Figure G000073. UE24CS151A PCPS Unit 3 Sindhu Class 14 15 Modules · source slide page 25.

EV0000547

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING
Modules – Import mechanisms

PES UNIVERSITY

Modules - Summary

- A Python module is a script file that can contain variables, functions, and classes.
- Python modules help us organize our code and then referencing them in other classes or python scripts.
- Modular programming is the practice of segmenting a single, complicated coding task into multiple, simpler, easier-to-manage sub-tasks. These sub-tasks are the modules.

Figure G000074. UE24CS151A PCPS Unit 3 Sindhu Class 14 15 Modules · source slide page 26.

EV0000550

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Figure G000075. UE24CS151A PCPS Unit 3 Sindhu Class 14 15 Modules · source slide page 27.

EV0000553

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Important points and why

1. Source slide page 1 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Modules Prof. Sindhu R Pai PCPS Theory Anchor - 2024 Department of Computer Science and Engineering EV0000474 · EV0000475 · EV0000476
2. Source slide page 2 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Modules – Import mechanisms Module • In Python, a module is a file that contains code. • It can include functions, classes, variables or any runnable code. • Basically, a module contains code to perform specific tasks. • We can use modules to separate code in separate files based on their functionality. EV0000477 · EV0000478 · EV0000479
3. Source slide page 3 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Modules – Import mechanisms Advantages of Modules • Reusability - makes the code reusable • Modularity – Organizing the code into modules logically • Separate scopes – separate namespace is defined by the module • Grouping - Python modules help us to organize and group content by using files and folders EV0000480 · EV0000481 · EV0000482

4. Source slide page 4 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Modules – Import mechanisms Packages and Namespace • Modules in Python can be grouped together into packages. • Packages organize the code into logical groups and provide a namespace to the modules so that they don't conflict with modules with the same name in other packages. • The import statement can also be used to import modules from a package, which allows to access the functions and classes defined in the package's modules. EV0000483 · EV0000484 · EV0000485
5. Source slide page 5 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Modules – Import mechanisms Packages vs. Modules vs. Libraries • Modules – contain several functions, variables, classes etc. • Packages – contain several modules. - folder that contains various modules as files. • Library - a collection of packages and modules used to access built-in functionality EV0000486 · EV0000487 · EV0000488
6. Source slide page 6 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Modules – Import mechanisms Types of Modules 1. Built-in Modules • Python's standard library comes bundled with a large number of modules. • They are called built-in modules. 2. User-defined Modules • Any file with .py extension and containing Python code is a module. • It can contain definitions of one or more functions, variables, constants, classes etc. EV0000489 · EV0000490 · EV0000491
7. Source slide page 7 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Modules – Import mechanisms Built-in Modules Name Description os provides a unified interface to a number of operating system functions stringstring containscontains aa numbernumber ofof functionsfunctions forfor stringstring processingprocessing re regular expression functionalities math a number of mathematical operations cmath a number of mathematical operations for complex numbers datetime functions to deal with dates and the time gc an interface to the built-in garbage collector asyncio functionality required for asynchronous processing collections advanced container datatypes functools higher-order functions and operations on callable objects EV0000492 · EV0000493 · EV0000494
8. Source slide page 8 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Modules – Import mechanisms Built-in Modules Name Description operator Functions on the standard operators pickle Convert Python objects to streams of bytes and back socket Low-level networking interface sqlite3 A DB-API 2.0 implementation using SQLite 3.x statistics Mathematical statistics functions typing Support for type hints venv Creation of virtual environments json Encode and decode the JSON format unittest Unit testing framework for Python random Generate pseudo-random numbers EV0000495 · EV0000496 · EV0000497
9. Source slide page 9 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Modules – Import mechanisms User-defined Modules • Any file with .py extension and containing Python code is a module. • It can contain definitions of one or more functions, variables, constants as well as classes. • Any object from a module can be made available to the interpreter or another Python script by using import statement. EV0000498 · EV0000499 · EV0000500
10. Source slide page 10 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Modules – Import mechanisms Creating a Module • Crete a file with .py extension • Example: Creating a module (module1.py) a=10 print("Welcome to Module-1") def f1(): print("in f1") def f2(): module1.py print("in f2") def _f3(): print("in f3") 10 EV0000501 · EV0000502 · EV0000503

11. Source slide page 11 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Modules – Import mechanisms Importing a Module • When the interpreter encounters an import statement, it imports the module if the module is present. Otherwise, ModuleNotFoundError is thrown. • Syntax - import module_name import – the keyword used to import the module module_name – name of the module to be imported • To access the functions inside the module the dot(.) operator is used. EV0000504 · EV0000505 · EV0000506

12. Source slide page 12 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Modules – Import mechanisms Import a Module • Example: importing module1(refer previous slide) import module1 #All functions and variables of module1 are availableil bl module1.f1() usingModule1.pymodule1.f2() module1._f3() print("value is", module1.a) module1.a = module1.a + 2 print("value is", module1.a) EV0000507 · EV0000508 · EV0000509

13. Source slide page 13 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Modules – Import mechanisms Output • Executing usingModule1.py Welcome to Module-1 in f1 in f2 in f3 value is 10 value is 12 EV0000510 · EV0000511 · EV0000512

14. Source slide page 14 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Modules – Import mechanisms Import from a Module • Import Specific Attributes from a module • Syntax – from module_name import specific_attributes • Example 2 def add(x, y): return (x+y) def subtract(x, y): return (x-y) module2.py def multiply(x, y): return (x*y) def divide(x, y): return (x/y) #Assuming 'y' is never zero EV0000513 · EV0000514 · EV0000515

15. Source slide page 15 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Modules – Import mechanisms Import from a Module • Example 2 (Contd...) from module2 import add,multiply #importing only add and multiply functions from module2module2 usingmodule2.py print("Sum=",add(10,20)) print("Product=",multiply(25,10)) Output Sum= 30 Product= 250 EV0000516 · EV0000517 · EV0000518

16. Source slide page 16 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Modules – Import mechanisms Import all Names from a Module • * symbol with the import statement is used to import all the names from a module. • Syntax - from module_name import * • Example 3 def add(x, y): return (x+y) def subtract(x, y): return (x-y) module3.py def multiply(x, y): return (x*y) def divide(x, y): return (x/y) #Assuming 'y' is never zero 16 EV0000519 · EV0000520 · EV0000521

17. Source slide page 17 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Modules – Import mechanisms Import all Names from a Module • Example 3 (Contd...) from module3 import * #importing all the functions from module2 usingmodule3.pyusingmodule3.py print("Sum=",add(10,20)) print("Product=",multiply(25,10)) Output Sum= 30 Product= 250 Note: If we know exactly which attribute to import from the module, it is not recommended to use import * EV0000522 · EV0000523 · EV0000524

18. Source slide page 18 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Modules – Import mechanisms Renaming/Aliasing Python Modules • We can rename the module while importing it. • Syntax – import module_name as alias_name • EExamplel 44 import math as mt #Renaming math module as 'mt' print(mt.factorial(6)) Output 720 EV0000525 · EV0000526 · EV0000527

19. Source slide page 19 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Modules – Import mechanisms Renaming/Aliasing Python Modules • Example 5 GRAVITY=9.8 module5.py print("Illustration of Renaming a Module") import module5 as m5 print("**********") usingmodule5.py print("Acceleration due to gravity on earth=",m5.GRAVITY,"m/s\u00b2") Output Illustration of Renaming a Module ********** Acceleration due to gravity on earth= 9.8 m/s² EV0000528 · EV0000529 · EV0000530

20. Source slide page 20 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Modules – Import mechanisms Locating Python Modules • To get the Directories List # importing sys module import sys # importing sys.path print(sys.path)print(sys.path) Output: ['', 'C:\\Users\\SOWMYA SHREE P\\AppData\\Local\\Programs\\Python\\Python311\\Lib\\idlelib', 'C:\\Users\\SOWMYA SHREE P\\AppData\\Local\\Programs\\Python\\Python311\\python311.zip', 'C:\\Users\\SOWMYA SHREE P\\AppData\\Local\\Programs\\Python\\Python311\\DLLs', 'C:\\Users\\SOWMYA SHREE P\\AppData\\Local\\Programs\\Python\\Python311\\Lib', 'C:\\Users\\SOWMYA SHREE P\\AppData\\Local\\Programs\\Python\\Python311', 'C:\\Users\\SOWMYA SHREE P\\AppData\\Local\\Programs\\Python\\Python311\\Lib\\site-packages'] This is a list of directories that the interpreter will search for the required module. EV0000531 · EV0000532 · EV0000533

21. Source slide page 21 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Modules – Import mechanisms sys.path.append() • a built-in function of sys module that can be used with path variable to add a specific path for interpreter to search. import sys syssys.path.append(path append('/path/to/module')/path/to/module) sys.path.insert() • a built-in function of sys module that can be used to insert a path at a specific position in sys.path. import sys sys.path.insert(0, '/path/to/module') 0 indicates that the path should be inserted at the beginning of sys.path . -1 to insert a path at the end of sys.path. EV0000534 · EV0000535 · EV0000536

22. Source slide page 22 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Modules – Import mechanisms __doc__ variable • In Python, each object(class, function, variable,..) can be documented using Docstrings. • Docstrings can be accessed using the __doc__ attribute. def add():() '''Performing addition of two numbers.''' a=10 Output b=7 Using __doc__: print(a+b) Performing addition of two numbers. Using help: print("Using __doc__:") Help on function add in module __main__: print(add.__doc__) add() print("Using help:") Performing addition of two numbers. help(add) EV0000537 · EV0000538 · EV0000539

23. Source slide page 23 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Modules – Import mechanisms __name__ variable and Modules • It is a special variable in Python. • If the source file is executed as the main program, the interpreter sets the __name__ variable to the value "__main__". • If this file is being imported from another module, __name__ will be set to the module's name. EV0000540 · EV0000541 · EV0000542

24. Source slide page 24 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Modules – Import mechanisms print ("file1 __name__ = %s" %__name__) if __name__ == "__main__": print ("file1 is executed directly") file1.py else: print ("file1 is imported") Output file1 __name__ = __main__ After executing file1.py file1 is executed directly EV0000543 · EV0000544 · EV0000545

25. Source slide page 25 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Modules – Import mechanisms import file1 print ("file2 __name__ = %s" %__name__) if __name__ == "__main__": file2.py print ("file2 is executed directly") else: print ("file2 is imported") Output file1 __name__ = file1 file1 is imported After executing file2.py file2 __name__ = __main__ file2 is executed directly EV0000546 · EV0000547 · EV0000548
26. Source slide page 26 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Modules – Import mechanisms Modules - Summary • A Python module is a script file that can contain variables, functions, and classes. • Python modules help us organize our code and then referencing them in other classes or python scripts. • Modular programming is the practice of segmenting a single, complicated coding task into multiple, simpler, easier-to-manage sub-tasks. These sub-tasks are the modules. EV0000549 · EV0000550 · EV0000551
27. Source slide page 27 (extracted text): THANK YOU Departmentp of Computerp Science and Engineeringg g Dr. Shylaja S S, Director, CDSAML & CCBD, PESU Prof. Sindhu R Pai – sindhurpai@pes.edu Prof. Sowmya Shree P EV0000552 · EV0000553 · EV0000554

SHORT-NOTE RECAP

Transcript block 1 (source interval 000000-000299 seconds): [00:00:02.148] So we were talking about um uh the different data types. Okay. So let's continue. [00:00:11.748] There is no separate data type called as a character in case of Python. Okay. So we can just I can just basically say that as num is equal to 10.5. Now at this point, I'm talking about float, and then I say num is equal to python. [00:00:30.980] Okay, num is equal to Python, now it becomes a string. But how does your Python know what type of data you are trying to give? Inter data, float data, and then string data, how does it come to know? Python has a built-in feature which is basically called as a dynamical. [00:00:50.448] Uh it's basically called as a dynamically typered programming language. Okay. It is so smart enough that the way in which you give the information or the data, it will come to know what is the type of data that is associated with a particular variable. [00:01:05.808] And hence we need not specify something of this kind int is num num is equal to ten. We need not do it in case of Python. You can just make use of a variable name and assign it a particular value. [00:01:20.768] The type of the value depends on the value itself. It comes to know what is the value. But how will I come to know what [00:01:29.252] is the a type of a value that uh Python is able to understand. For example, according to us, it is an integer, but how will I how will the Python come to know? Yes? So in order to do this, there is a function called as a type function [00:01:44.160] There is a function called as a type function, which basically gives me the type of a value. Okay? I just need to mention it like that. So it will give me whether it is an integer, whether it is a float, or whether it is a string. [00:01:59.168] Okay, so now this equal to symbol is called as an assignment operator. Equal to symbol is called as an assignment operator. Please remember assignment operator. [00:02:13.092] Is this fine? Okay. So now uh what did we do? We basically said num is equal to ten, okay? And if I say num is equal to ten plus one, which means num is equal to hopefully yes, num is equal to eleven. [00:02:30.448] Okay, so now uh especially in case of our uh automatic or mathematics, this statement is absolutely an invalid statement. [00:02:40.032] Correct. The reason is we basically try to do ten plus one is equal to eleven. But in case of the programming language, we do it in the other way itself. Okay? So [00:02:53.732] With this example, with this example, hopefully you came to know all the evaluation, yes, okay, right hand side of the assignment is going to be evaluated first and then the value will be assigned to the [00:03:11.008] Now variable on the

left hand side. Okay? Please remember right hand side evaluation will happen and the assignment will happen on the uh left hand side. Okay. Fine. So let me take up the other sheets and then explode. [00:03:31.876] Let's go ahead. Fine. So now the moment I say, the moment I say NAM is equal to 10. Fine. What basically will happen? In case of Python in case of Python. [00:03:50.116] Something of this kind. A value 10 is present in the memory and your variable name gets referenced to this number. To this value. [00:04:02.340] So the moment you say num is equal to 10, if a value already exists in the memory, if a particular value already exists in a memory, I will assume at this point it already exists in the memory, then only your variable gets referenced to this value. [00:04:19.524] If it does not exist, then a new variable or a new value gets created into your memory. Okay so the next statement what I can do is I can also say k is equal to 10. [00:04:32.388] Right? So now I look for 10 in the memory. I in the sense Python looks for 10 in the memory. Is it available? Yes, it is available. Then what do I do? I will also associate [00:04:43.492] This K over this. Fine. At this point, what is the reference count for this value 10? The reference count is 2. Fine. How will I get the reference count? I will also code it to you and show it to you. So do not worry about this at this point.

EV0000158 · EV0000159 · EV0000160 · EV0000161

Transcript block 2 (source interval 000300-000599 seconds): [00:05:00.480] Okay so now what happens in case of your Python referencing will take place. Suppose I say the other way I'll say uh X is equal to K. X is equal to K. Now what is the value of K at this point? [00:05:17.920] Is it already defined? Yes, it is already defined. So now x is equal to k, case value is 10, which means x is equal to 10. So now what will happen? I will look for the value in the memory. Is it there? Yes, it is there. So I will also reference it with x. [00:05:35.632] Fine. What is a reference count for uh ten now? Three is the reference count. [00:05:41.540] Is this fine? Suppose I'll do an extra thing, I'll say num is equal to, suppose num is equal to 10 plus 1, right? Which means I'm saying num is equal to 11. [00:05:56.196] It's just fine. Num is equal to 11. Now what will happen to this num now? Will what and what will happen to K as well as X? Will it change or only num will change? Okay. So to answer to this question, the moment we associate like this. [00:06:13.712] A num will be dereferen, okay? Num will be dereferenced and a new reference for the num gets created here. Again, if value is present, it references to this. Otherwise, a new value also gets created within your memory. [00:06:31.072] Okay, so this is how uh Python reference will take place. Now, when does the reference count of a value become equal to zero? [00:06:40.804] Fine. When does it become equal to zero? Reference count. When there are no more variables names gets associated, fine. When there are no more variable names gets associated. [00:06:53.764] Then at that point, what will happen? The reference count for this 10 becomes a zero. So that which means I can now make use of this location and associate a new value with this. Okay. Hopefully it is clear to you. If there are [00:07:11.376] No other variable references a memory location of the original value, the memory location is deallocated. Is this fine? It can be given to perform some other operation. Is this fine? So now if you see, at this point I will say [00:07:28.736] K I will perform an operation called as delete. Delete K. The moment I do delete K, what will happen? K's reference goes. [00:07:37.264] The moment I say delete X, X reference also goes. At this point, do you have any reference for 10? No, there is no reference for our value 10. What will happen? What will happen? Its reference count becomes equal to 0. Fine. Now [00:07:55.556] Quickly, I'll just give you one more example and then wind up. I say num is equal to 15, X is equal to 15. Fine Okay, so now what will happen? 15, num is associated. [00:08:11.972] Oh hopefully you're not able to see. Okay. Yes. And then X is

also referencing here itself, right? Which means both are pointing to the same memory location. [00:08:24.912] Okay, hopefully it is clear to you. Both of them are pointing to the same memory location. Now, how will we prove that they are pointing to the same memory location? We have a function called as an ID function. [00:08:37.796] an ID function which basically helps us to find out what is the location at which a particular uh value is present. Okay so we will uh try to identify. So ID function [00:08:51.536] It produces an unique number identifying a specific value in the memory. Okay. I will demonstrate all these things. Okay. Along with this, what are the rules for creating the identifiers? [00:09:05.984] And uh what are the uh keywords that are present. The reason is you cannot make use of keywords as an identifiers. So all these things along with the function of uh type [00:09:18.436] Type function and ID function, all those demonstration I will be giving it to you now. Okay? So hope uh you have uh understood uh the overall concepts of variables. Okay.

EV0000162 · EV0000163 · EV0000164 · EV0000165

Full transcript: 1443 normalized words, 39 timestamped segments, 572.48 seconds duration. Low-confidence segments flagged: 0.

EV0000158 · EV0000159 · EV0000160 · EV0000161

14 input function

This lesson is indexed from the source video titled input function. Its complete timestamped transcript is preserved below in segment order. The original video path and content hash are recorded in datasets/lecture-corpus.json. Transcript text is source wording; it has not been silently corrected or rewritten. A candidate slide deck is grouped here according to the current matching score; the exact path and page locators are listed in the source evidence. EV0000166 · EV0000167

Graphs and diagrams

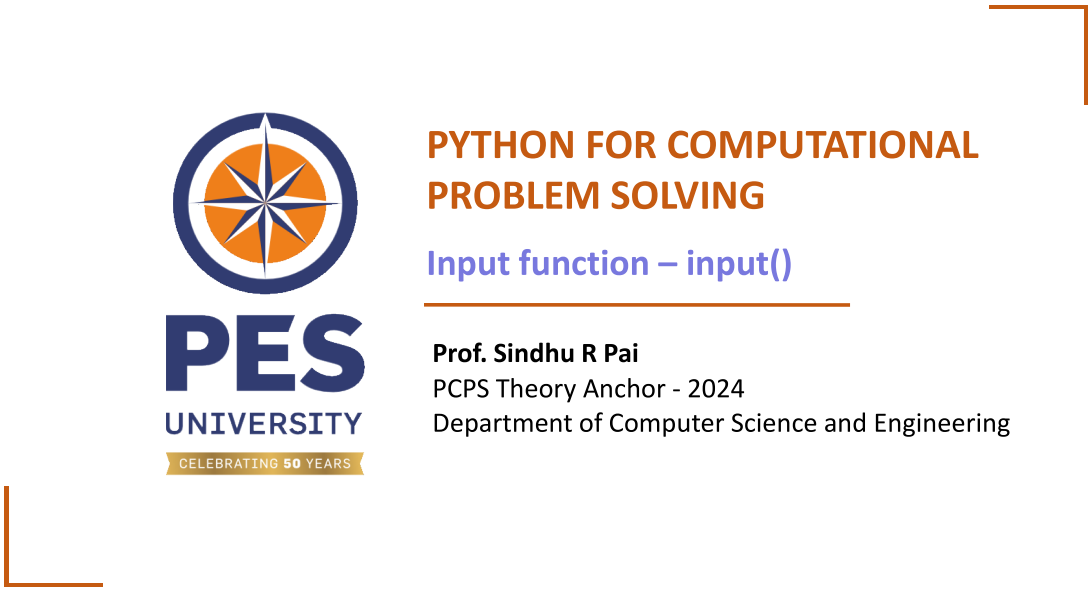


Figure G000175. UE24CS151A PCPS Unit 1 SindhuRPai Class 11 input function · source slide page 1.

EV0000853

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Input Function – input()

- This function is used **to read a string from the user**.
- Can also be used to display the message to the user using the prompt field and then displays the cursor to the user to take some input from the user.

```
input([prompt] =None, /)
    Read a string from standard input.          The trailing newline is stripped.
    The prompt string, if given, is printed to standard output without a
    trailing newline before reading input.
    If the user hits EOF (*nix: Ctrl-D, Windows: Ctrl-Z+Return), raise EOF Error.
```

- Type conversion functions must be used if required. – int(), float(), bool(), complex(), etc
- If multiple values are in the input, use the delimiter while splitting using split()

Figure G000176. UE24CS151A PCPS Unit 1 SindhuRPai Class 11 input function · source slide page 2.

EV0000856

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

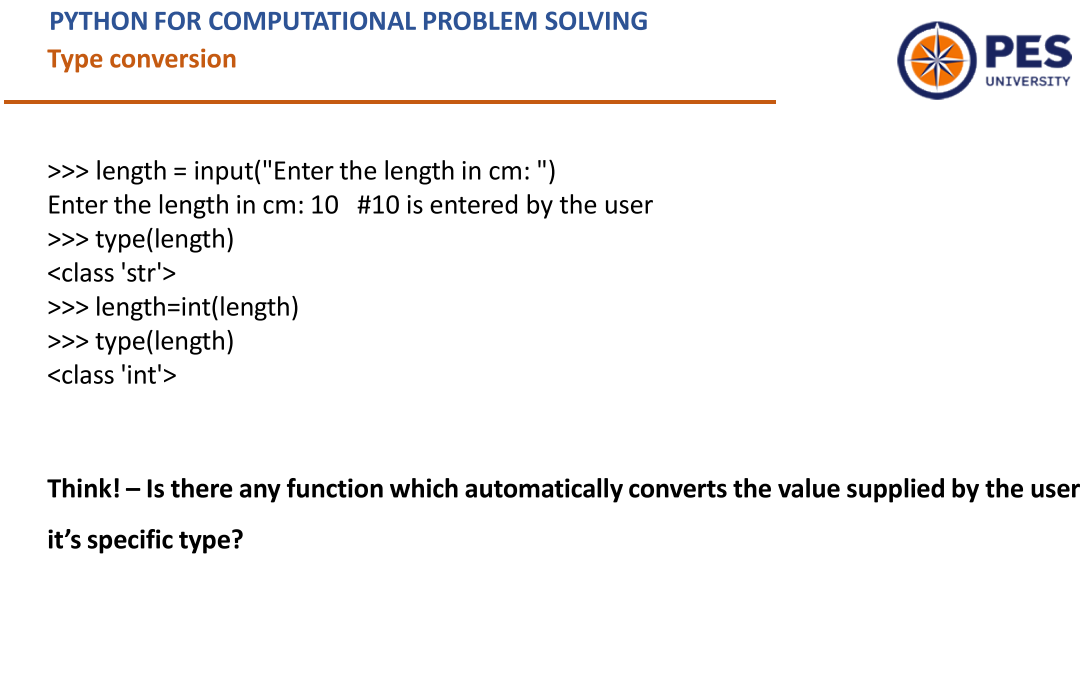


Figure G000177. UE24CS151A PCPS Unit 1 SindhuRPai Class 11 input function · source slide page 3.

EV0000859

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

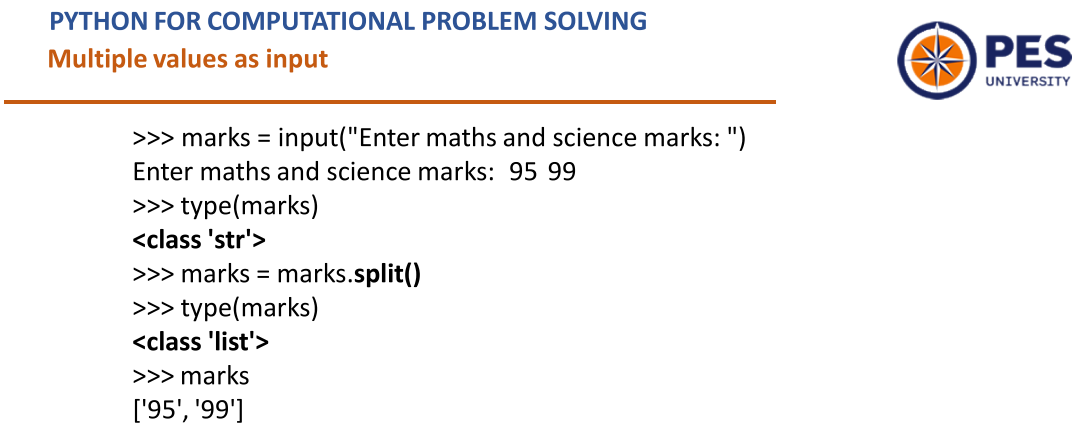


Figure G000178. UE24CS151A PCPS Unit 1 SindhuRPai Class 11 input function · source slide page 4.

EV0000862

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

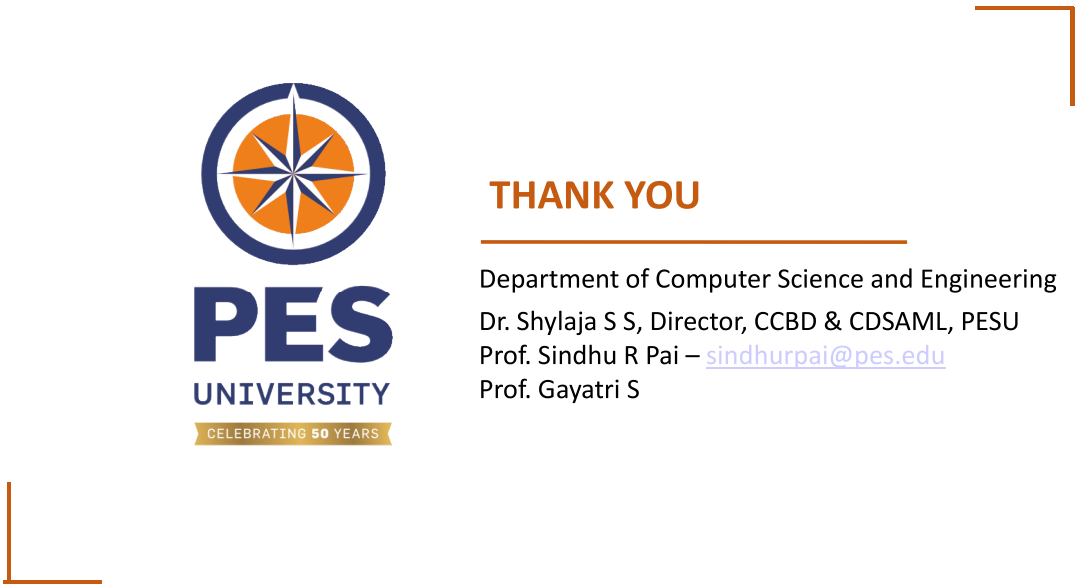


Figure G000179. UE24CS151A PCPS Unit 1 SindhuRPai Class 11 input function · source slide page 5.

EV0000865

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Important points and why

1. Source slide page 1 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Input function – input() Prof. Sindhu R Pai PCPS Theory Anchor - 2024 Department of Computer Science and Engineering EV0000852 · EV0000853 · EV0000854
2. Source slide page 2 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Input Function – input() • This function is used to read a string from the user. • Can also be used to display the message to the user using the prompt field and then displays the cursor to the user to take some input from the user. input([prompt] =None, /) Read a string from standard input. The trailing newline is stripped. The prompt string, if given, is printed to standard output without a trailing newline before reading input. If the user hits EOF (*nix: Ctrl-D, Windows: Ctrl-Z+Return), raise EOF Error. • Type conversion functions must be used if required. – int(), float(), bool(), complex(), etc • If multiple values are in the input, use the delimiter while splitting using split() EV0000855 · EV0000856 · EV0000857

3. Source slide page 3 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Type conversion >>> length = input("Enter the length in cm: ") Enter the length in cm: 10 #10 is entered by the user >>> type(length) <class 'str'> >>> llength=int(length)h i (l h) >>> type(length) <class 'int'> Think! – Is there any function which automatically converts the value supplied by the user to it's specific type? EV0000858 · EV0000859 · EV0000860
4. Source slide page 4 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Multiple values as input >>> marks = input("Enter maths and science marks: ") Enter maths and science marks: 95 99 >>> type(marks) <class 'str'> >>> marks = marks.split() >>> type(marks) (k) <class 'list'> >>> marks ['95', '99'] Think! – Can we use different symbols to separate out the inputs from the user? EV0000861 · EV0000862 · EV0000863
5. Source slide page 5 (extracted text): THANK YOU Departmentp of Computerp Science and Engineeringg g Dr. Shylaja S S, Director, CCBD & CDSAML, PESU Prof. Sindhu R Pai – sindhurpai@pes.edu Prof. Gayatri S EV0000864 · EV0000865 · EV0000866

SHORT-NOTE RECAP

Transcript block 1 (source interval 000000-000299 seconds): [00:00:01.572] Hello all, welcome back. In today's session we will understand the concept of input function. Hope you're all ready. Let's get started. [00:00:17.124] Okay, as usual, let's first process is to save our program. So let me save it as uh import fine as usual dot py [00:00:29.252] And that terminal, let's check with the terminal. Okay. So this terminal is also in place in order to help us to execute. So let's get started. Okay. So let me give you a task. Before that, I will say it as inputs. [00:00:47.748] Okay so now we're talking about uh the concept of input, the function that we use in order to give an input is still an input itself. So that is why we call uh Python as a language which is near to us because um it [00:01:06.512] It is perceived in the way in which I want because I'm trying to say print something onto the console. So we have a print statement, and now I'm saying give an input, so I have an input function. So it's a language that is very close to the way in which we try to speak. Okay. So [00:01:25.572] Yes. And then try to understand. Now what is the syntax? Syntax goes is something like this. Prompt is something that is optional in nature. Okay, we will try to understand. [00:01:43.684] thing that I'll do is I want you people to write a program in order to calculate the area of a rectangle. Okay find the area of a rectangle. [00:02:00.772] So if this is the case, as you know, given any problem, you have to remember the process of computational problem solving. [00:02:08.868] It has four different steps. The first step is to analyze. Understand what constitutes your problem. Understand what you need to do. Okay. So in this case, I clearly know that. I clearly know that I need to calculate the area. [00:02:25.184] Of what? Of a rectangle. And I also know the formula which is the length into red. Okay? So this is what I have understood. Now the next point is [00:02:38.116] What data is readed? With the formula itself, I came to know that I need to make use of two different things, one is length and breadth, and then try to calculate the area. Okay, an algorithm, are you using any new algorithm at this point? No, I'm still making use of an existing algorithm itself. [00:02:58.468] Later on you have to write the program okay and then followed by perform the testing [00:03:04.964] So with this what we will try to do is we will try to write a simple program. So now I'll say length is equal to maybe 10, 5, breadth is equal to 30. That's fine. So calculate area. [00:03:23.300] Followed by the formula is length into breadth and I have calculated print and see area. Okay. So this is my program. Let's execute this. Python. [00:03:43.460] This is input right. Inputs dot p1. And if you see my output is absolutely fine, I'm

getting 300. Can I make it more readable? Let my output be more readable. I say area is [00:04:01.892] And I even say it as comma. So in this case, how does a print statement is behaving? Anything that is given in double quotes gets executed as it is, right? And anything after this, it tries to find out is there any value for this. [00:04:19.940] Previously we have calculated the area, so it gives me value for you. Okay, let's find out. Let me execute area is more readable. Make it a little more [00:04:35.076] What I will do, go mark and execute. So Area is equal to three hundred. Now in this case, what is happening is [00:04:45.092] Every time when I execute the program, I will always get the output to be 300. The reason is my values are hard coded. In this sense, I have already predefined the values for length and breadth. [00:04:59.120] And what if I want to give little more flexibility to the user so that he can give the input of his preference. So in such case, I need to give an option where user can enter the values. So now

EV0000166 · EV0000167 · EV0000168 · EV0000169

Transcript block 2 (source interval 000300-000599 seconds): [00:05:16.032] How will I do it? Okay, so in order to do this we have a function called an input. [00:05:23.780] Okay, we have a function called as an input function. So now suppose user gives 10 again. Where it should be saved? It should be saved in a particular variable. So I will call it as a also say breadth is equal to input. [00:05:43.204] Oh sorry. [00:05:47.428] Okay, and then I will say area itself, let's not change, is equal to length into breadth. Fine. Let's print area is equal to [00:06:09.476] Now. [00:06:14.436] What are we expecting from this? We are expecting as usual I need to get the output which is length into breadth. Okay so before me executing the same set of things, let me comment them as I have already mentioned in my previous video. [00:06:30.596] Okay, triple double quotes. If not, please try to understand. Triple double quotes serves as a multi-line comment. Okay, triple double quotes serve as a multi-line comment. Otherwise, I have to comment each line separately. [00:06:47.008] Okay, let's find out. Will it give us any error or is it fine is what we are trying to find out. So CLS to clear the screen and input. So I'll say this. [00:07:01.124] Okay, so it is asking me to give uh some values because we have said input. Now the cursor is blinking, which means that cur uh the Python interpret uh sorry Python is waiting for you to give some input. So now in this case I'll get chan. [00:07:18.148] It's again waiting for some other inputs, so I'll give 30 and area is supposed to be 300. What is happening? It shows me an error. Okay, and it says you're trying to perform a multiplication. [00:07:34.916] But it is not on an integer, but it is on a string. Okay. So now why exactly is what is the meaning of this error? It basically says you're giving an input. [00:07:50.756] Fine, the moment an input function appears here, we try to give an input through the keyboard. The moment I give an input through the keyboard, it always gets string as its value. [00:08:08.676] as a value. So instead of integer by default we are giving string as a value. So you have a string one, you have a string two, you are trying to multiply two strings because the operation is not defined on the strings. [00:08:26.148] Which are the ones that are defined? I will be dealing and then explaining it to you when we understand the concept of strings. Okay, so it is pretty much clear it is throwing me an error in this case. Okay, so let me uh [00:08:45.060] Comment all of them. Okay. Say this. And what next I will do, instead of me typing it again, okay, or I will type it again because it's first time. So I'll say L is equal to input. [00:09:02.148] This was the statement that was causing a problem for us because whatever we are giving through the keyboard, it is taken as a string for an input function. So now string needs to be converted to an integer. So we will say convert it by type. [00:09:21.156] say int of input. Hope you understood. So similarly do it for the other one also. So I'll say control V and then say breadth is equal to int of

input. So now [00:09:38.148] Both of them are converted to integer. Let's find out. So I'll say area is equal to length into breadth. This anyway, I have the statement. Can I copy paste if you don't have any problem? Control C and then I'll say control.

EV0000170 · EV0000171 · EV0000172 · EV0000173

Transcript block 3 (source interval 000600-000899 seconds): [00:10:01.028] Anyway we have already done it. [00:10:06.628] Input.py still waiting for me to give the inputs, I'll give 10. Cursor is still blinking, I give the value 30, and now you're getting the output as 300. [00:10:19.492] Okay so hopefully uh it is clear for you. The reason is input function always gets a string. Multiplying two strings is an error. What is the thing that we are doing? Convert string to [00:10:36.036] [ASR low confidence; check audio] And [00:10:44.100] How are we doing it is done through type casting of int to string. Okay. [00:10:54.308] This is clear for you? Okay. So [00:11:01.252] Do this. [00:11:07.908] This you can see a star here. The star mark at the top represents it is not saved. Make sure every time you save your program. Okay, so let's continue. Then we saw the output, the cursor was blinking. It was not clear what I have to do. [00:11:24.964] Okay, it was not clear for us. So now can I make it a little more readable? So I say int input. [00:11:38.884] I'll do this paste. [00:11:45.732] I wanted it to make it more uh readable so I will say enter the value [00:12:01.316] Okay, enter the area of length. Okay so similarly I will again copy paste and do it for area of length and breadth again. Okay, so let me do this. [00:12:15.556] And anyway, these are already available. I will copy paste it so as to save the time and let me do this. Okay. So CLS. Okay. Let me execute. C [00:12:34.052] Now it is more readable for me and I clearly know what I have to do. So what is the input function is doing? Input function takes the input. At the same time, whatever you have given as a part of a prompt, it also displays it onto the output screen. [00:12:53.252] The 30 these were the values that we were uh using from the beginning. So we will use the same. So [00:13:05.860] The reason is I'm using B here, but in my program B is not being defined. So go back and define it as B and Yes. So execute the value and enter the [00:13:25.604] I have done a length thirty fine area is three hundred. I'm sorry. So what I'll do is because it's a copy paste error that is happening. Okay so I'll see. [00:13:43.620] This is a problem with copy pasting, but still saves our time, but little we should be little careful in doing so. [00:13:52.708] Sorry for the mistake. Input.py enter the value of length 100, enter the value of breadth 30, and area is 300. So you understood the concept of [00:14:10.660] quickly recollect on what we have done. Inputs is a function. Okay. So this is the syntax prompt is optional. [00:14:20.772] Why? Why we need uh to give input? We want to give more flexibility to the user in a way he can enter the value of his choice. Okay. By default, input gets a string when we type it from the keyboard, because of which we have to do the [00:14:40.580] So you can do the conversion from string to int. You want you can also do it from string to float. Okay. So in such case you will have to mention it as float here. Okay so hope this is clear and hope you have understood.

EV0000174 · EV0000175 · EV0000176 · EV0000177

Transcript block 4 (source interval 000900-001199 seconds): [00:15:01.092] Keep learning, keep programming, okay? Thank you.

EV0000178 · EV0000179 · EV0000180 · EV0000181

Full transcript: 1896 normalized words, 61 timestamped segments, 906.538667 seconds duration. Low-confidence segments flagged: 1. Review locators: 00:10:36.036

EV0000166 · EV0000167 · EV0000168 · EV0000169

15 input function

This lesson is indexed from the source video titled input function. Its complete timestamped transcript is preserved below in segment order. The original video path and content hash are recorded in datasets/lecture-corpus.json. Transcript text is source wording; it has not been silently corrected or rewritten. A candidate slide deck is grouped here according to the current matching score; the exact path and page locators are listed in the source evidence. EV0000182 · EV0000183

SHORT-NOTE RECAP

Transcript block 1 (source interval 000000-000299 seconds): [00:00:00.452] Hello everyone, welcome back to the lecture series on Python for computational problem solving. I am Nita and Jaker from the Department of Computer Science and Engineering, and in this video I will introduce you to the input function. What is an input function and why is it useful? [00:00:20.292] An input function is one which allows the program to interact with the user. It allows the user to input some value based on which the program can process or work further. Let's first look at how exactly it functions. [00:00:39.076] Like any function, it requires some input, which is usually a prompt message that tell the user what kind of an input the program expects. [00:00:51.652] As part of the function processing, it displays this prompt message onto the output screen and waits for the user to input the value. Once the user inputs the value and hits the enter key, [00:01:06.532] It is returned to the function and that function returns the user's input as a result to your program. So what the program gets in turn is the value which was entered by the user. [00:01:25.188] Let's look at how the input function is defined by the Python library. [00:01:31.236] It is a function which takes just one argument which is a prompt message. Now you can see there is a default argument called prompt, which is initialized to none, which means that prompt is an optional argument, and even if you don't enter anything. [00:01:48.800] A blank or nothing is displayed to the output screen. What the input function does is then reads a string from the standard input, whatever the user enters, and when the user hits the enter key, it returns back to the program. [00:02:06.276] So when you do hit an enter key, a chaining new line is added, which is then stripped and only the value is preserved. [00:02:16.292] So the prompt message is displayed and it also takes care of error handling. If the programmer using, say, a Unix system enters control D, it will [00:02:30.148] return an end of file error or if in Windows enters CtrlZ would again return an end of file error. Let's look at the syntax. It's just the name of the function input and all optional arguments are usually indicated within square brackets. [00:02:49.252] So the prompt, if any, is display to the terminal, which is an optional argument, and the input function returns a value. Now, since it does return a value, it only makes sense that it must be stored so that we can use it in the future. [00:03:06.724] So an ideal way of using it would be say I want to enter the length of a rectangle in centimeter. You have length is equal to input and whatever is mentioned here, enter the length in centimeter, would act as a prompt to tell the user what is expected. [00:03:25.348] Let me demonstrate this with an example to show you how exactly an input function works. First, I'm going to write an input function with no prompt and not storing the result value anywhere. So when I do run the program [00:03:44.740] My Python output. Sorry, input. [00:03:51.140] You can see that the program execution is suspended and you have a blinking cursor. This indicates that the program is waiting for user input. [00:04:00.900] Now, since I have given no prompt, it is confusing to the user

to understand what the program requires. Is it a number? Is it a string? Is it a special character? It is not clear. So if I enter 12, it does take the value, but I have not stored it or displayed it. [00:04:20.932] As we know in batch mode, we must always use the print function to display the result of the program. So it's clear now that an input function without a prompt makes no sense. [00:04:37.860] So when I say enter the value, it's still very vague. So what I want to do is ask the user to enter a number. So I will save this program, make no further modifications, and when I run the program it just says enter the pro number and you have a blinking cursor. [00:04:57.828] So now when I enter the number, it still doesn't give us any output because I have not written a print statement. Now I'm going to write print.

EV0000182 · EV0000183 · EV0000184 · EV0000185

Transcript block 2 (source interval 000300-000599 seconds): [00:05:12.036] [ASR low confidence; check audio] The value entered. [00:05:18.436] [ASR low confidence; check audio] Yeah. [00:05:23.364] Now since I have not stored the result of this input function, I don't have another option to display the value. I could use the print the input function as part of the print statement, but ideally what we must do is always store [00:05:40.912] the value returned by the input function in a variable that could be used later on. So I've created an assignment statement here. [00:05:50.468] I've written an input function on the right hand statement of the assignment operator. So enter a number is the prompt that would be displayed. When the user enters a value and hits the enter key, that value is then stored into the variable. [00:06:08.708] Now if I want to see the value, I'm going to use the print statement and simply key in the name of the variable. Let's look at the output now. Enter the number, I'm going to enter 34. I hit enter, you can see the value entered is displayed. [00:06:27.204] This is how you would use an input function. Now what would happen if I don't enter any value and just hit the enter key? It still works as expected, but what the function returns would be an empty string. [00:06:44.900] So we don't see an output because an empty string is what is being displayed. [00:06:50.500] Now I also told you the function is written to take care of error handling. And since I'm using a Windows system, if I type CtrlZ and enter, it gives you an error saying it is an enter file error. [00:07:06.436] Because your endophile character in Windows is the control and the Z key pressed together. So these are some of the variations for your input function. [00:07:18.692] Now next what I want to see is I want to understand what kind of values does the input function take. So for example, when I run this program, it says enter a number, but I'm going to enter my name as an input. Will this give me an alone? [00:07:35.184] No, it doesn't, because your input function's only duty is to prompt the user for a value, wait for the user's input, and return that input whatever it may be back to the program. [00:07:50.816] It does not take care of validating the input. That must be done explicitly by the programmer while writing the program. So let's look at why it doesn't give us an error. So I want to see [00:08:06.180] what the value is, and I also want to see the type of the value. What is the data type the input function returns. To see the type of the value we have already used the type function [00:08:21.316] Now I'm going to check what the type of the value is returned by your input function. [00:08:30.660] So I don't enter a number, you can see the value entered is twelve. But what you see is class string. Again, let's enter my name as a string. It says the value entered is lita and it belongs to the class string. [00:08:47.172] I'm now going to enter a series of special characters. [00:08:54.340] Again, it prints those special characters, but it says it's a string. So with these examples we know that the input function always returns a string. So let's [00:09:09.572] Look at what the input function does. So once it's established that the input function returns a

string, if I want to use that value in different contexts, so for example, if I want to add two numbers, I want it as a number and not a string. [00:09:27.396] So in order to do that, we must use the type conversion functions in order to convert a string into a number, a floating point value, and so on. So let's look at how exactly we could achieve that. Now I want to ensure that this number [00:09:45.668] If a user enters a number, it would be converted to an integer. We've already seen that when we want to convert any number. String save, I'm going to give s is equal to string 12 to a number, all I have to do is

EV0000186 · EV0000187 · EV0000188 · EV0000189

Transcript block 3 (source interval 000600-000899 seconds): [00:10:03.748] N, I'm going to save it as n is equal to int of the variable s. So this would then convert it to a number, and which is in the form of an integer. So if I were to print the value of n [00:10:24.612] And also the type of M. [00:10:33.732] You can see it's 12 and class integer. If I wanted it to be floating point, all I had to do was change this to float. So now that we know how to type cast functions, what I want to do is now [00:10:52.452] Convert this entire value returned by the input into a number. So all I have to do is typecast the return button. I could write say number is equal to int of value or [00:11:12.612] Directly in line five, write int and an argument to the in function could be the input function. So when we have a function call within another function call, the innermost function is first executed. [00:11:29.636] So when I do run, when the program encounters line five, it will first call the input function, sends the prompt to the user, the user enters a value, and this entire highlighted portion is replaced with the string that's entered by the user. [00:11:49.316] That string is given as an argument to int, and it's converted and stored in bad. So let's look at how it would work. [00:12:01.796] So I will run the program, enter 12, and before I use the typecasting function, we saw that 12 was stored as a string. Now when you see 12 is stored as an integer. [00:12:17.860] Now if I were to enter my name as an input, you will get an error saying it is an invalid literal for int, because you cannot give a string as a value, a set of strings which is not between zero and nine as a value to the int function. [00:12:38.084] This is how you could use the input function. Now as seen, you can only enter one value at a time. Let's also apply this to a program to see where exactly is this going to be useful. [00:12:52.740] I have written a program where you have the variable length is equal to twenty. [00:12:58.852] Breadth is equal to thirty and we're calculating the area of a rectangle. We already know the equation, it is length pro into breadth, product of length and breadth, and I'm just going to display this. Now when we run this program [00:13:18.500] It's I think area dot py it will give me 600 centimeters. So every time I run the program it's going to give me the same output, which means this program is only useful for a given set of values. [00:13:34.180] So if I want to change it, I'll have to change the program itself to say four and six, save the program, then run it, and so on. [00:13:47.332] What I want to do is have one standard program where I want the user to enter the values of the length and breadth and calculate the area accordingly. So this is where calculating or using the input function comes in hand. So I provide input. [00:14:07.844] enter the length in centimeter and I could also write same thing for breakfast button I'm just going to copy this [00:14:25.444] So this program now allows the user to enter the values for length and breadth. When I run this program though, this will give me an error. [00:14:37.860] Because I have not converted the length and breadth to int each other. And you cannot multiply a sequence of non-int type of string. So you can't multiply two strings with each other. [00:14:52.996] So what I need to do is convert it to integer as we just saw. So both of them I

will convert to integer in line two and three. And now when I run the program, whatever number I give.

EV0000190 · EV0000191 · EV0000192 · EV0000193

Transcript block 4 (source interval 000900-001199 seconds): [00:15:09.828] will be treated as integers and displayed. [00:15:14.532] Now what I've done here is used an input function every time I need an input. What if I want to use or provide multiple inputs and be used in the program? Is that possible? [00:15:30.660] Well, the input function can only take one input, as long as the user doesn't hit the enter key, you can keep adding as many values as you want, and it would be considered as one single input. [00:15:45.892] What we can do is manipulate the string returned by the input function to [00:15:54.532] Then further split it and use as multiple values. This would be explained in detail when we cover the concept of strings and list in the next module. So [00:16:09.988] You use the input function as a mode of interaction between the program and write programs that can be used in for several inputs rather than for hard-coded values. [00:16:26.468] This brings us to the end of the input function. Thank you.

EV0000194 · EV0000195 · EV0000196 · EV0000197

Full transcript: 2261 normalized words, 67 timestamped segments, 990.677333 seconds duration. Low-confidence segments flagged: 2. Review locators: 00:05:12.036, 00:05:18.436

EV0000182 · EV0000183 · EV0000184 · EV0000185

16 Arithmetic, membership and relational operators

This lesson is indexed from the source video titled Arithmetic, membership and relational operators. Its complete timestamped transcript is preserved below in segment order. The original video path and content hash are recorded in datasets/lecture-corpus.json. Transcript text is source wording; it has not been silently corrected or rewritten. A candidate slide deck is grouped here according to the current matching score; the exact path and page locators are listed in the source evidence. EV0000198 · EV0000199

Graphs and diagrams

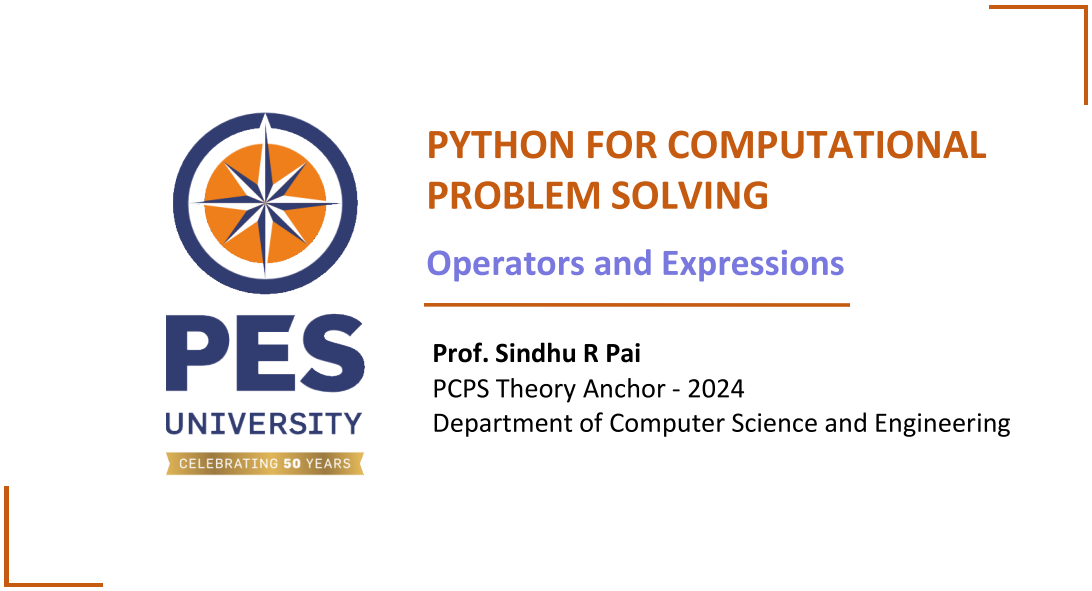


Figure G000180. UE24CS151A PCPS Unit 1 SindhuRPai Class 12 Operators Part1 · source slide page 1.

EV0000868

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING
Introduction

PES UNIVERSITY

- **An operator** is a symbol that represents an operation that may be performed on one or more *operands.*
- An operand is a value that a given operator is applied to.
- An expression is a combination of symbols that evaluates to a value.
- **Expressions,** most commonly, consist of a combination of operators and operands

  4 + (3 * k)
- It can also consist of a single literal or variable. Thus, 4, 3, and k are each expressions
- Expressions that evaluate to a numeric type are called arithmetic expressions

Figure G000181. UE24CS151A PCPS Unit 1 SindhuRPai Class 12 Operators Part1 · source slide page 2.

EV0000871

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

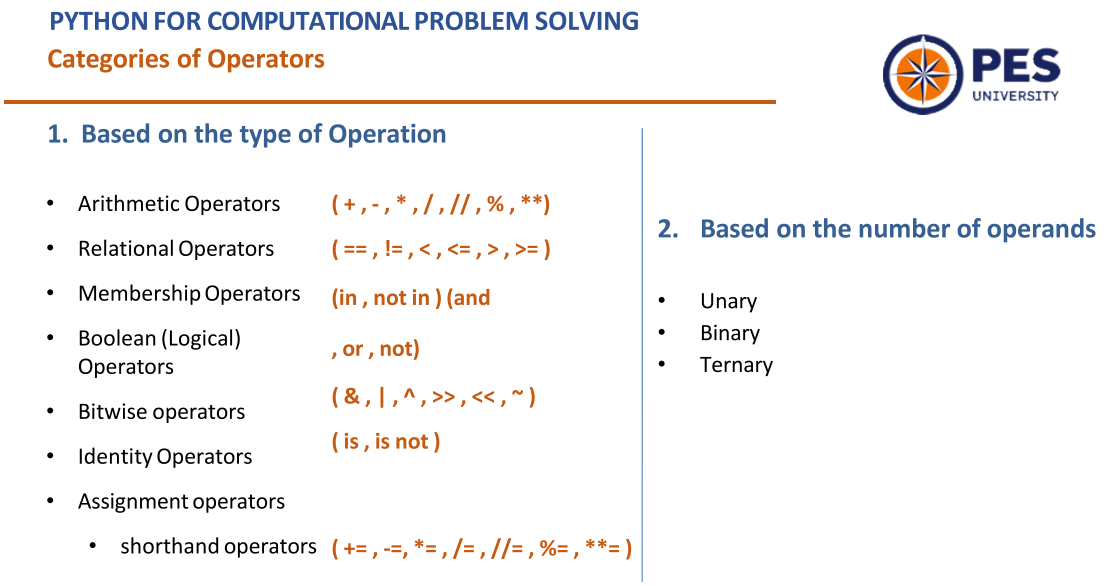


Figure G000182. UE24CS151A PCPS Unit 1 SindhuRPai Class 12 Operators Part1 · source slide page 3.

EV0000874

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

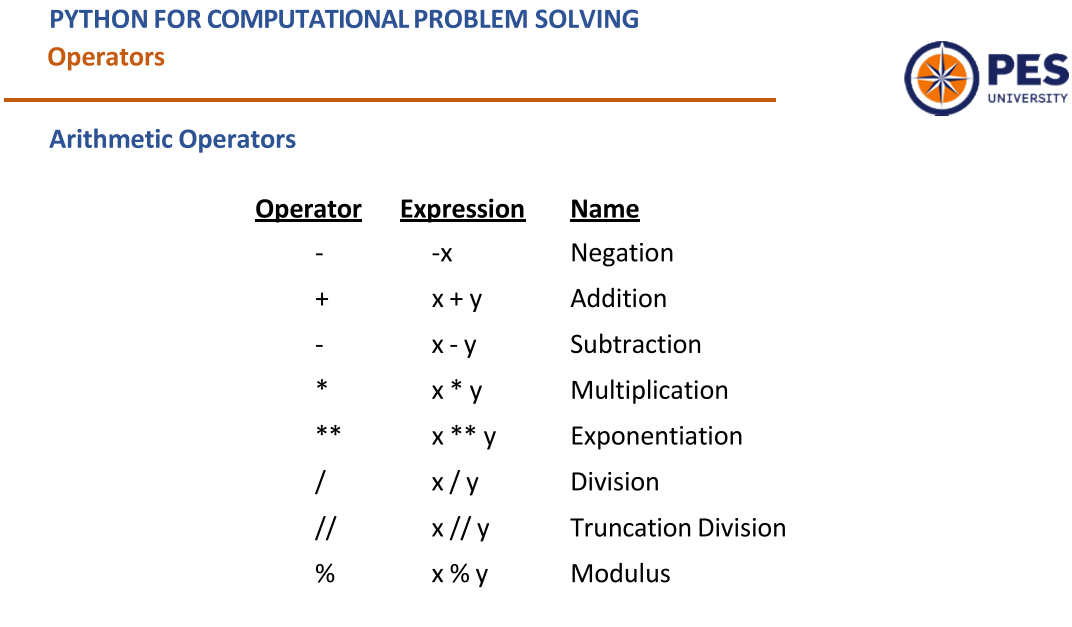


Figure G000183. UE24CS151A PCPS Unit 1 SindhuRPai Class 12 Operators Part1 · source slide page 4.

EV0000877

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

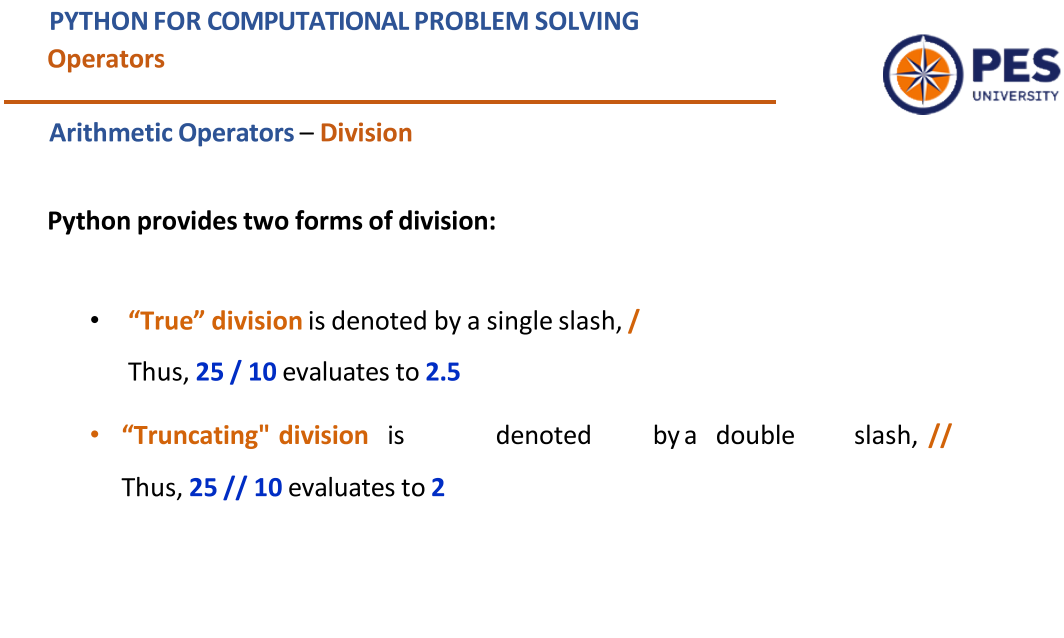


Figure G000184. UE24CS151A PCPS Unit 1 SindhuRPai Class 12 Operators Part1 · source slide page 5.

EV0000880

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Operators

PES UNIVERSITY

Arithmetic Operators – Division

Truncating division provides a truncated result based on the type of operands applied to

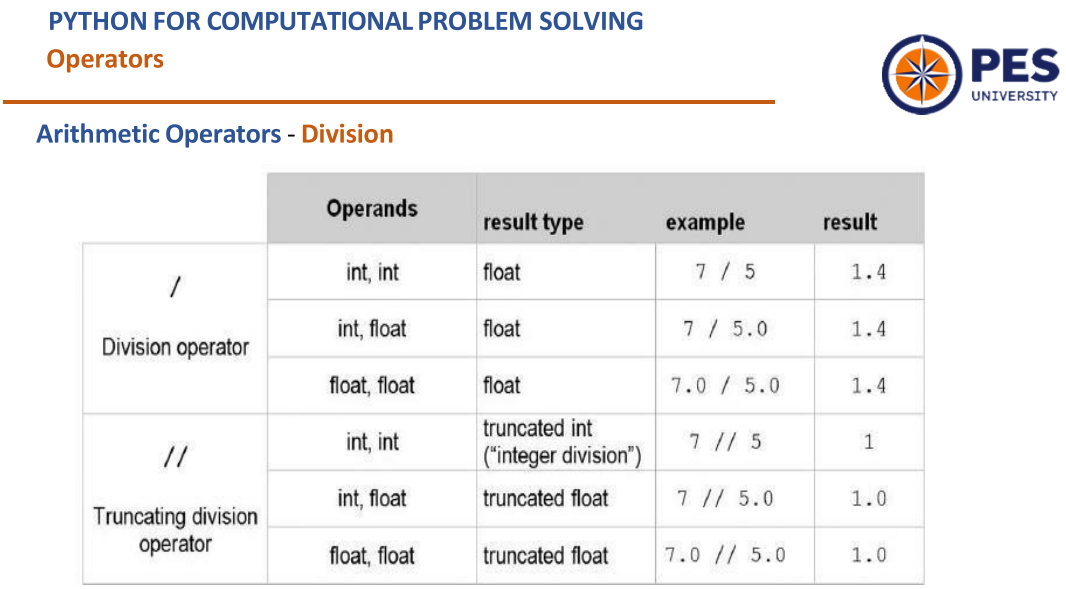
- When both operands are integer values, the result is a truncated integer referred to as integer division.
- When as least one of the operands is a float type, the result is a truncated floating point.

Example:
>>> 5//2
2
>>> 5//2.0
2.0

Figure G000185. UE24CS151A PCPS Unit 1 SindhuRPai Class 12 Operators Part1 · source slide page 6.

EV0000883

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.



	Operands	result type	example	result
/ Division operator	int, int	float	7 / 5	1.4
	int, float	float	7 / 5.0	1.4
	float, float	float	7.0 / 5.0	1.4
// Truncating division operator	int, int	truncated int ("integer division")	7 // 5	1
	int, float	truncated float	7 // 5.0	1.0
	float, float	truncated float	7.0 // 5.0	1.0

Figure G000186. UE24CS151A PCPS Unit 1 SindhuRPai Class 12 Operators Part1 · source slide page 7.

EV0000886

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

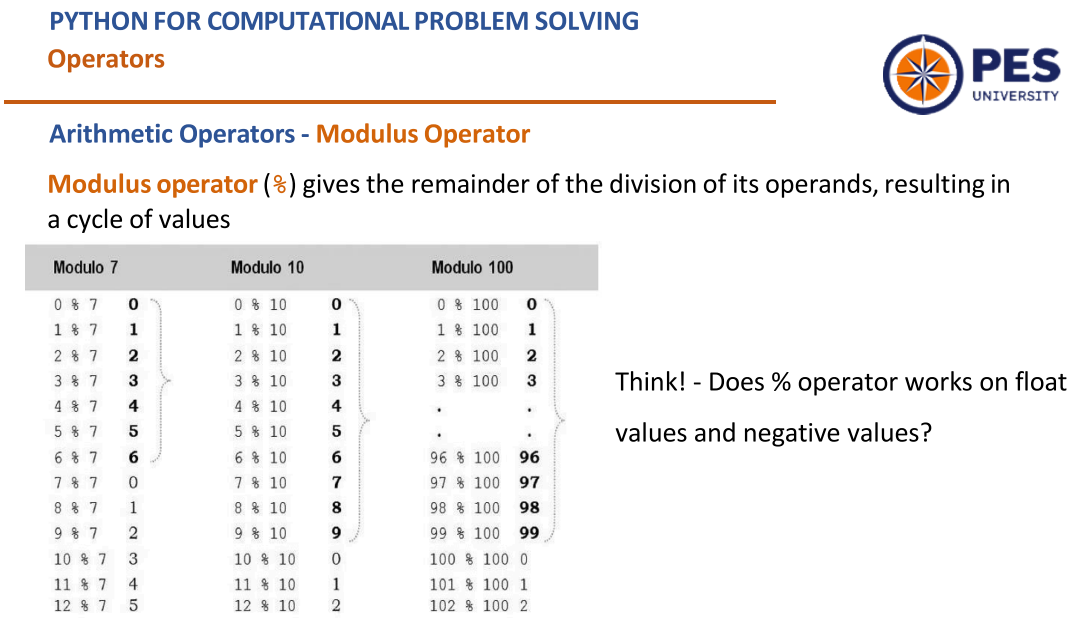


Figure G000187. UE24CS151A PCPS Unit 1 SindhuRPai Class 12 Operators Part1 · source slide page 8.

EV0000889

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

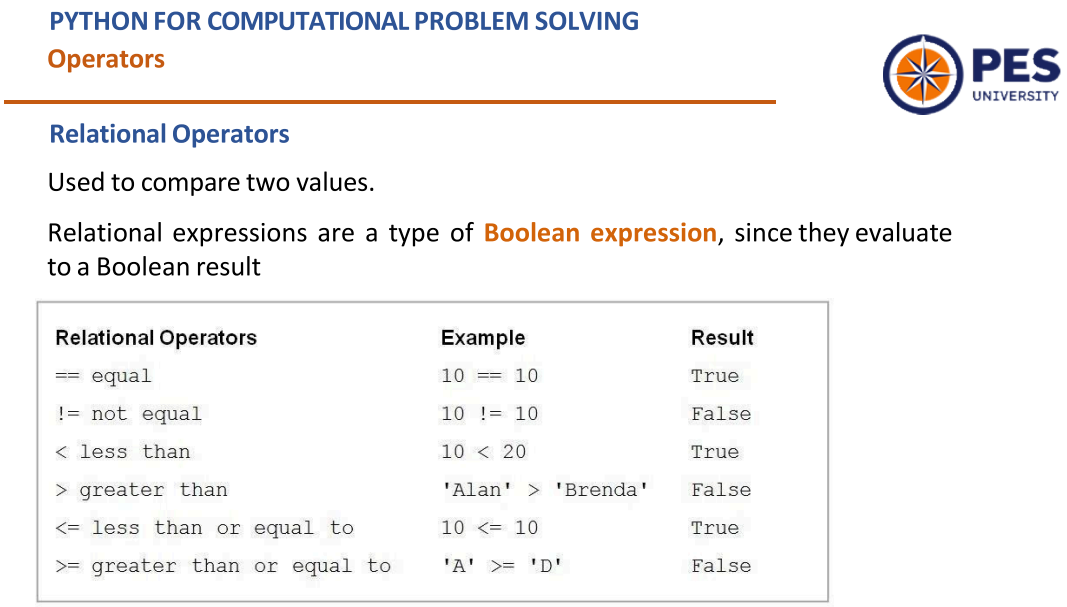


Figure G000188. UE24CS151A PCPS Unit 1 SindhuRPai Class 12 Operators Part1 · source slide page 9.

EV0000892

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

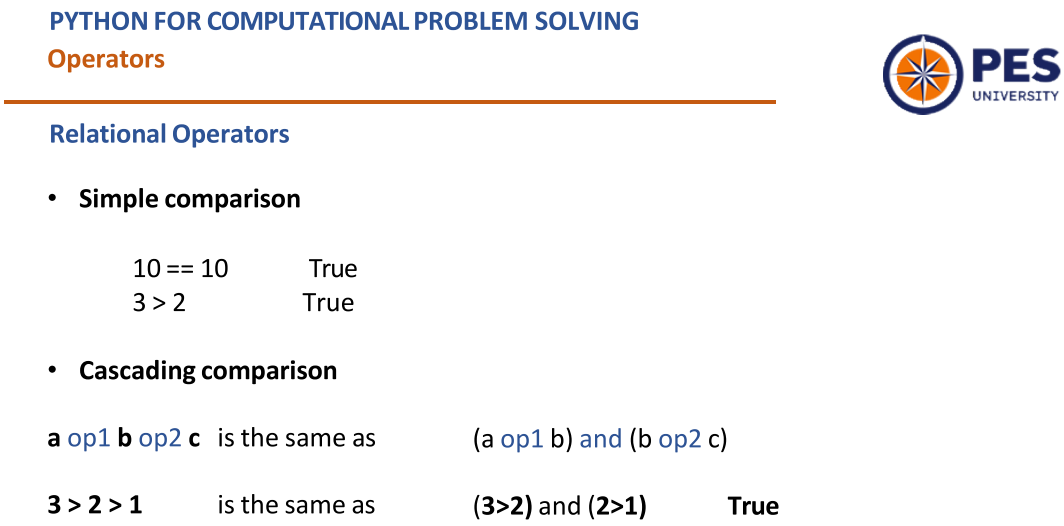


Figure G000189. UE24CS151A PCPS Unit 1 SindhuRPai Class 12 Operators Part1 · source slide page 10.

EV0000895

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

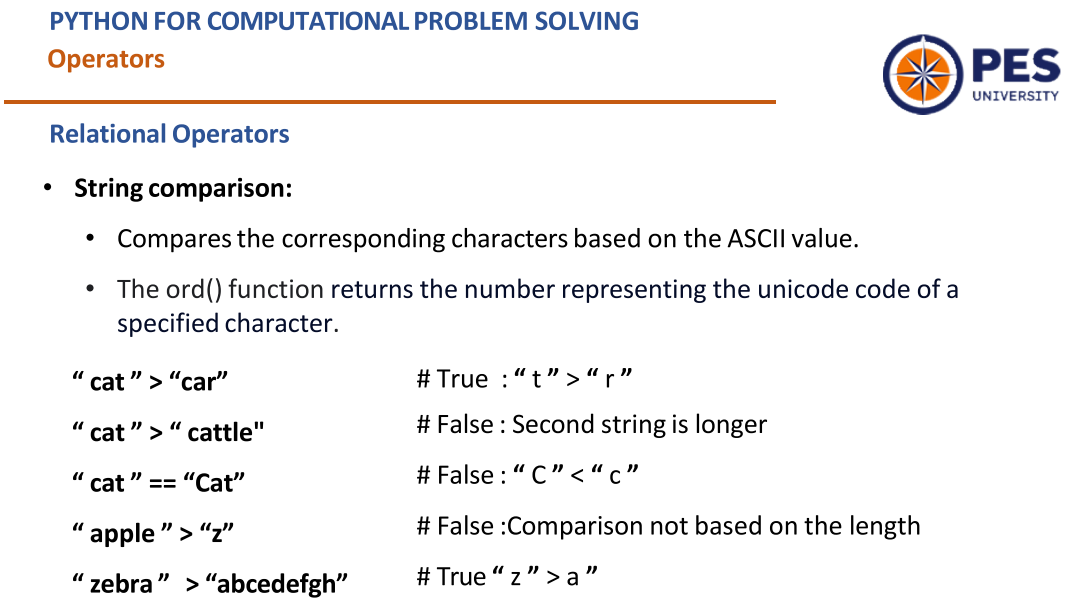


Figure G000190. UE24CS151A PCPS Unit 1 SindhuRPai Class 12 Operators Part1 · source slide page 11.

EV0000898

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Operators

PES UNIVERSITY

Relational Operators

- **List comparison:**

Rules are same as that of string - compare the corresponding elements until a mismatch or one or both ends

[10, 20, 30] > [10, 25] # False 20 > 25 is false

[(10, 20), "abcd"] >[(10, 20), "abcc"] # True d of abcd > last c of abcc

Figure G000191. UE24CS151A PCPS Unit 1 SindhuRPai Class 12 Operators Part1 · source slide page 12.

EV0000901

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

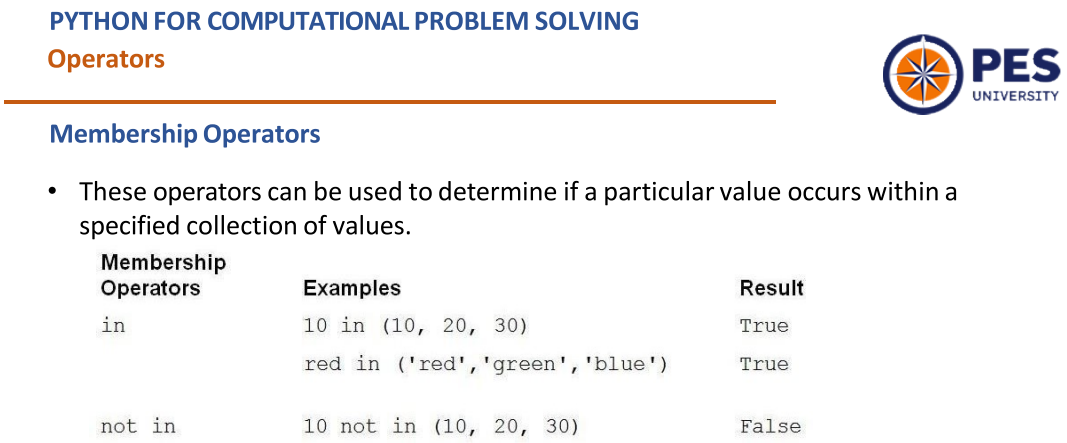
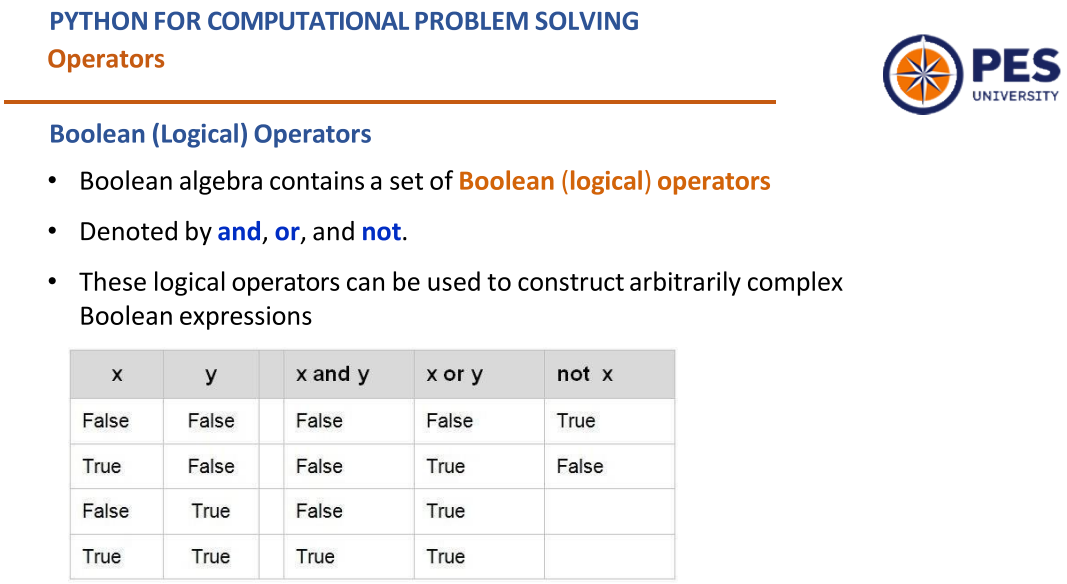


Figure G000192. UE24CS151A PCPS Unit 1 SindhuRPai Class 12 Operators Part1 · source slide page 13.

EV0000904

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.



x	y	x and y	x or y	not x
False	False	False	False	True
True	False	False	True	False
False	True	False	True	
True	True	True	True	

Figure G000193. UE24CS151A PCPS Unit 1 SindhuRPai Class 12 Operators Part1 · source slide page 14.

EV0000907

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

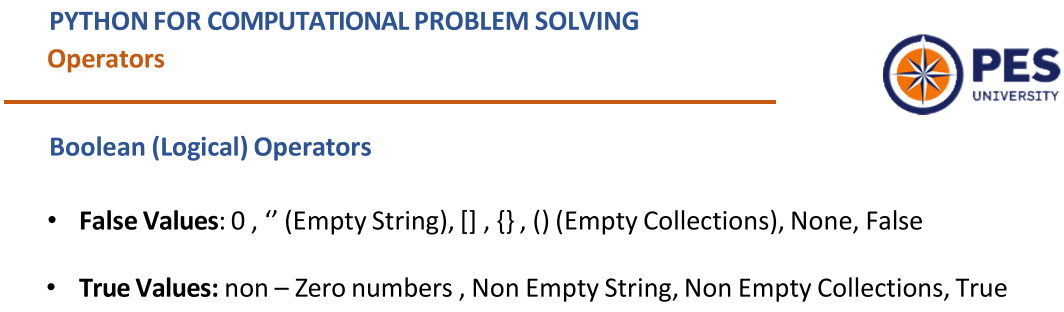


Figure G000194. UE24CS151A PCPS Unit 1 SindhuRPai Class 12 Operators Part1 · source slide page 15.

EV0000910

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Operators

PES UNIVERSITY

Boolean (Logical) Operators

Short Circuit Evaluation

- logical and, if the first operand evaluates to false, then regardless of the value of the second operand, the expression is false
- logical or, if the first operand evaluates to true, regardless of the value of the second operand, the expression is true.
- Python interpreter does not evaluate the second operand when the result is known by the first operand alone
- This is called **short-circuit (lazy) evaluation**

Figure G000195. UE24CS151A PCPS Unit 1 SindhuRPai Class 12 Operators Part1 · source slide page 16.

EV0000913

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Operators

PES UNIVERSITY

Bitwise Operators

Operations are performed at the bit level

Operator	Name	Result
&	AND	result is 1 if the corresponding bits are one
|	OR	result is 1 if even at least one of the bits is one
^	Exclusive OR	result is 1 if and only if one of the bits is 1
<<	LEFT SHIFT	multiply by 2 for each left shift
>>	RIGHT SHIFT	divide by 2 for each right shift
~	ONE'S COMPLIMENT	change 0 to 1 and 1 to 0

Figure G000196. UE24CS151A PCPS Unit 1 SindhuRPai Class 12 Operators Part1 · source slide page 17.

EV0000916

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

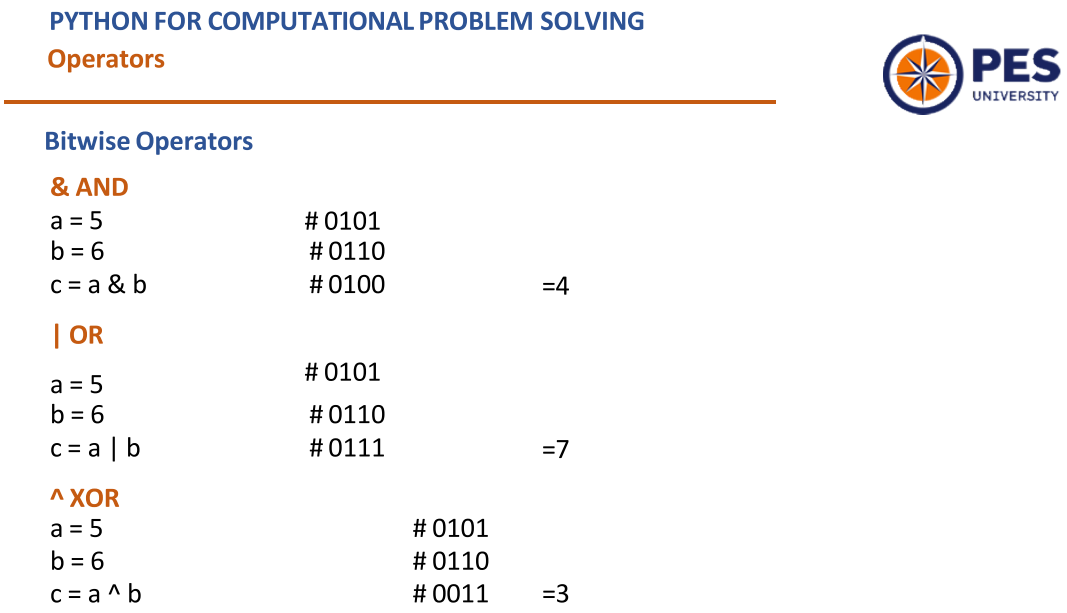


Figure G000197. UE24CS151A PCPS Unit 1 SindhuRPai Class 12 Operators Part1 · source slide page 18.

EV0000919

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

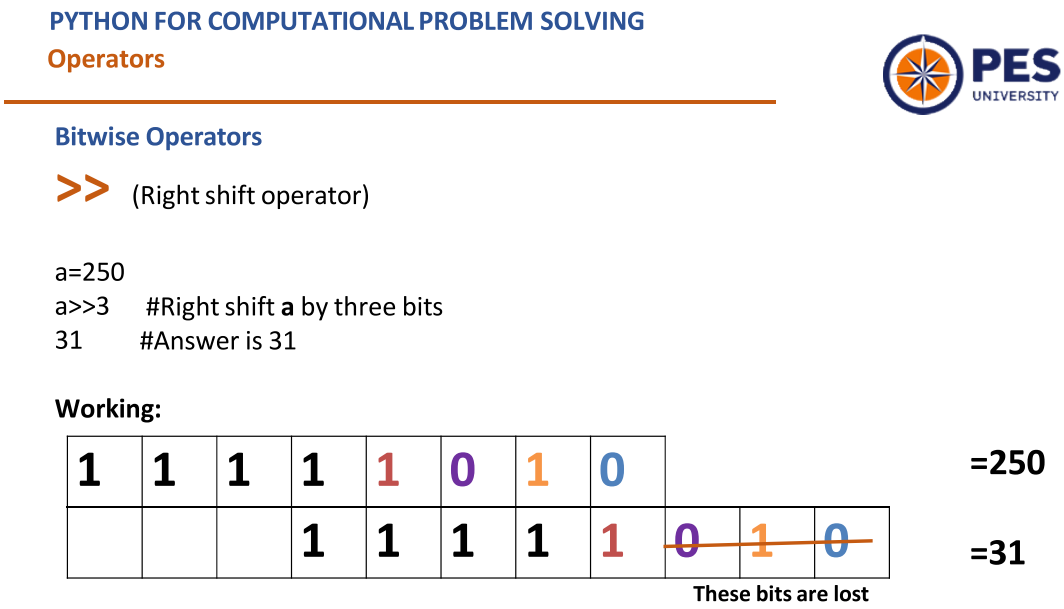


Figure G000198. UE24CS151A PCPS Unit 1 SindhuRPai Class 12 Operators Part1 · source slide page 19.

EV0000922

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

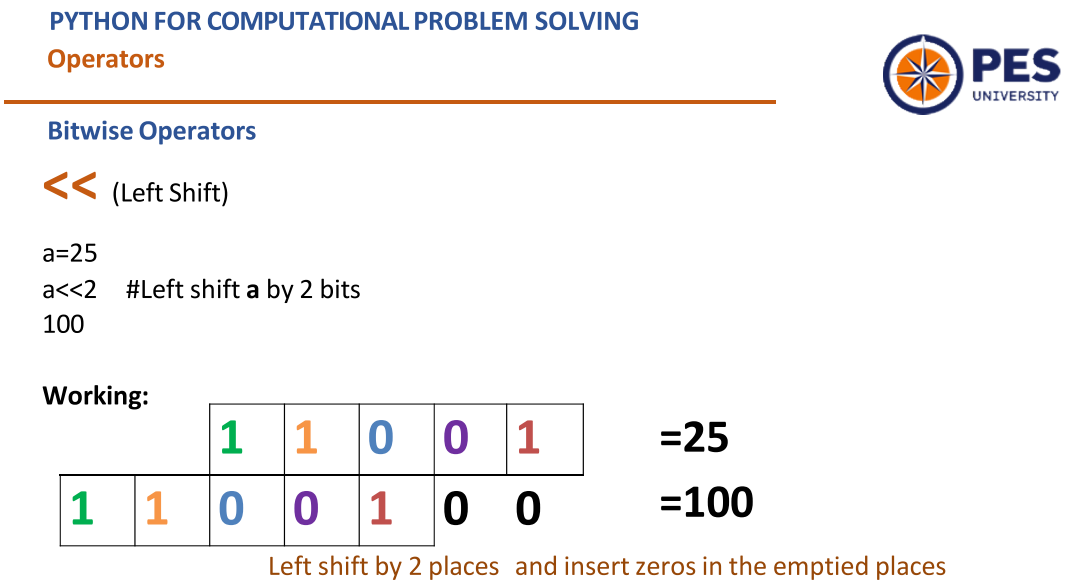


Figure G000199. UE24CS151A PCPS Unit 1 SindhuRPai Class 12 Operators Part1 · source slide page 20.

EV0000925

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

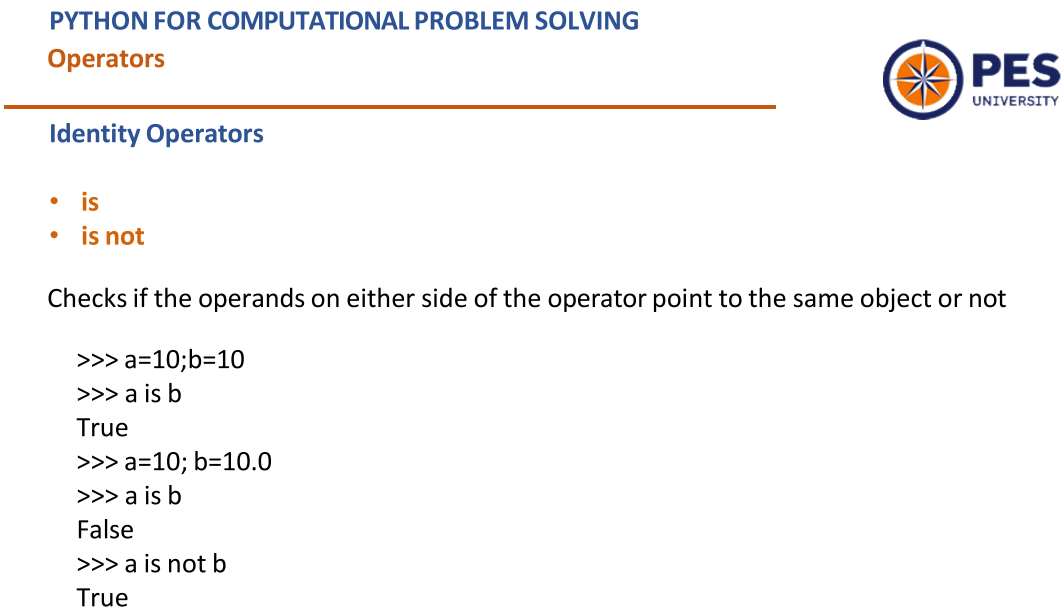


Figure G000200. UE24CS151A PCPS Unit 1 SindhuRPai Class 12 Operators Part1 · source slide page 21.

EV0000928

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Operators

PES UNIVERSITY

Assignment / Shorthand Operators

Combines arithmetic and assignment operators

Operator	Expression	Short Hand
+= (Addition)	a = a + b	a += b
-= (Subtraction)	a = a - b	a -= b
*= (Multiplication)	a = a * b	a *= b
/= (Division)	a = a / b	a /= b
//= (Truncation Division)	a = a // b	a //= b
%= (Modulus)	a = a % b	a %= b
**= (Exponentiation)	a = a ** b	a **= b

Figure G000201. UE24CS151A PCPS Unit 1 SindhuRPai Class 12 Operators Part1 · source slide page 22.

EV0000931

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

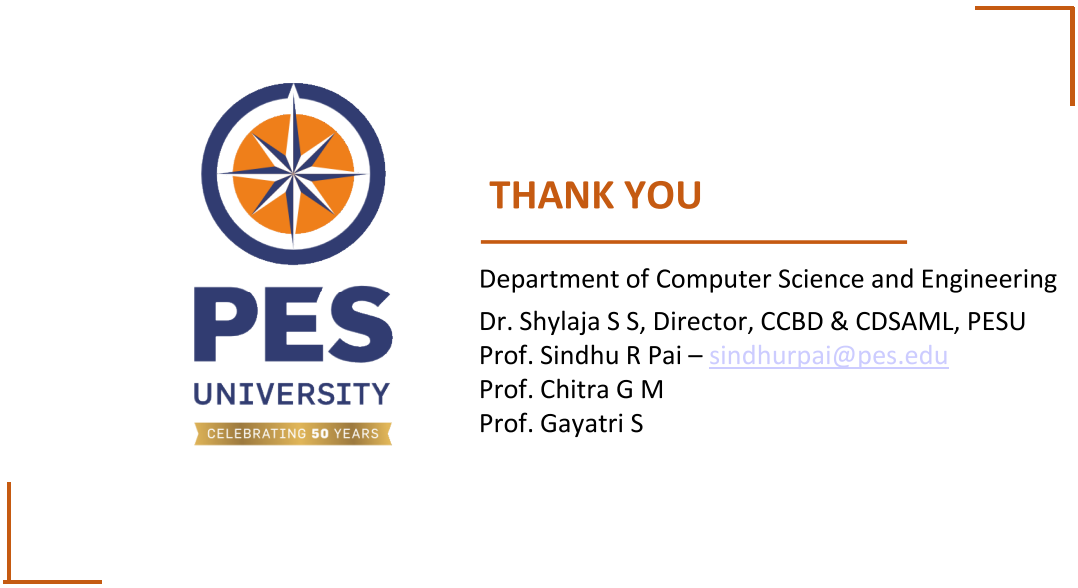


Figure G000202. UE24CS151A PCPS Unit 1 SindhuRPai Class 12 Operators Part1 · source slide page 23.

EV0000934

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Important points and why

1. Source slide page 1 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Operators and Expressions Prof. Sindhu R Pai PCPS Theory Anchor - 2024 Department of Computer Science and Engineering EV0000867 · EV0000868 · EV0000869
2. Source slide page 2 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Introduction • An operator is a symbol that represents an operation that may be performed on one or more operands. • An operand is a value that a given operator is applied to. • An expression is a combination of symbols that evaluates to a value. • Expressions, most commonly, consist of a combination of operators and operands 4 + (3 * k) • It can also consist of a single literal or variable. Thus, 4, 3, and k are each expressions • Expressions that evaluate to a numeric type are called arithmetic expressions EV0000870 · EV0000871 · EV0000872

3. Source slide page 3 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Categories of Operators 1. Based on the type of Operation • Arithmetic Operators (+ , - , * , / , // , % , **) 2. Based on the number of operands • Relational Operators (== , != , < , <= , > , >=) • MembershipOperatorsMembershipOperators (in(in , notnot inin)) (and(and •• UnaryUnary • Boolean (Logical) • Binary , or , not) Operators • Ternary (& , | , ^ , >> , << , ~) • Bitwise operators (is , is not) • IdentityOperators • Assignmentoperators • shorthand operators (+= , -=, *= , /= , //= , %= , **=) EV0000873 · EV0000874 · EV0000875
4. Source slide page 4 (extracted text): PYTHON FOR COMPUTATIONALPROBLEM SOLVING Operators Arithmetic Operators Operator Expression Name - -x Negation ++ xx ++ yy AdditionAddition - x - y Subtraction * x * y Multiplication ** x ** y Exponentiation / x / y Division // x // y Truncation Division % x % y Modulus EV0000876 · EV0000877 · EV0000878
5. Source slide page 5 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Operators Arithmetic Operators – Division Python provides two forms of division: • “True” division is denoted by a single slash, / Thus, 25 / 10 evaluates to 2.5 • “Truncating" division is denoted bya double slash, // Thus, 25 // 10 evaluates to 2 EV0000879 · EV0000880 · EV0000881
6. Source slide page 6 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Operators Arithmetic Operators – Division Truncating division provides a truncated result based on the type of operands applied to • When both operands are integer values, the result is a truncated integer referred to as integerg division. • When as least one of the operands is a float type, the result is a truncated floating point. Example: >>> 5//2 2 >>> 5//2.0 2.0 EV0000882 · EV0000883 · EV0000884
7. Source slide page 7 (extracted text): PYTHON FOR COMPUTATIONALPROBLEM SOLVING Operators Arithmetic Operators - Division EV0000885 · EV0000886 · EV0000887
8. Source slide page 8 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Operators Arithmetic Operators - Modulus Operator Modulus operator (%) gives the remainder of the division of its operands, resulting in a cycle of values Think! - Does % operator works on float values and negative values? EV0000888 · EV0000889 · EV0000890
9. Source slide page 9 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Operators Relational Operators Used to compare two values. Relational expressions are a type of Boolean expression, since they evaluate to a Boolean result EV0000891 · EV0000892 · EV0000893
10. Source slide page 10 (extracted text): PYTHON FOR COMPUTATIONALPROBLEM SOLVING Operators Relational Operators • Simple comparison 10 == 10 True 33 >> 22 TrueTrue • Cascading comparison a op1 b op2 c is the same as (a op1 b) and (b op2 c) 3 > 2 > 1 is the same as (3>2) and (2>1) True EV0000894 · EV0000895 · EV0000896
11. Source slide page 11 (extracted text): PYTHON FOR COMPUTATIONALPROBLEM SOLVING Operators Relational Operators • String comparison: • Compares the corresponding characters based on the ASCII value. • The ord()() function returns the number representingp g the unicode code of a specified character. “ cat ” > “car” # True : “ t ” > “ r ” # False : Second string is longer “ cat ” > “ cattle" # False : “ C ” < “ c ” “ cat ” == “Cat” # False :Comparison not based on the length “ apple ” > “z” # True “ z ” > a ” “ zebra ” > “abcedefgh” EV0000897 · EV0000898 · EV0000899

12. Source slide page 12 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Operators Relational Operators • List comparison: Rules are same as that of string - compare the corresponding elements until aa mismatchmismatch oror oneone oror bothboth endsends [10, 20, 30] > [10, 25] # False 20 > 25 is false [(10, 20), "abcd"] >[(10, 20), "abcc"] # True d of abcd > last c of abcc EV0000900 · EV0000901 · EV0000902

13. Source slide page 13 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Operators Membership Operators • These operators can be used to determine if a particular value occurs within a specified collection of values. • The membership operators can also be used to check if a given string occurs within another string >>> 'E' in 'PES' True EV0000903 · EV0000904 · EV0000905

14. Source slide page 14 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Operators Boolean (Logical) Operators • Boolean algebra contains a set of Boolean (logical) operators • Denoted by and, or, and not. • These logical operators can be used to construct arbitrarily complex Boolean expressions EV0000906 · EV0000907 · EV0000908

15. Source slide page 15 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Operators Boolean (Logical) Operators • False Values: 0 , '' (Empty String), [] , {} , () (Empty Collections), None, False • True Values: non – Zero numbers ,, Non Emptyp y String,g, Non Emptyp y Collections,, True EV0000909 · EV0000910 · EV0000911

16. Source slide page 16 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Operators Boolean (Logical) Operators Short Circuit Evaluation • logical and, if the first operand evaluates to false, then regardless of the value of the secondd operand,d theh expression is ffalsel • logical or, if the first operand evaluates to true, regardless of the value of the second operand, the expression is true. • Python interpreter does not evaluate the second operand when the result is known by the first operand alone • This is called short-circuit (lazy) evaluation EV0000912 · EV0000913 · EV0000914

17. Source slide page 17 (extracted text): PYTHON FOR COMPUTATIONALPROBLEM SOLVING Operators Bitwise Operators Operations are performed at the bit level Operator Name Result && ANDAND resultresult isis 11 ifif thethe correspondingcorresponding bitsbits areare oneone | OR result is 1 if even at least one of the bits is one ^ Exclusive OR result is 1 if and only if one of the bits is 1 << LEFT SHIFT multiply by 2 for each left shift >> RIGHT SHIFT divide by 2 for each right shift ONE'S ~ change 0 to 1 and 1 to 0 COMPLIMENT EV0000915 · EV0000916 · EV0000917

18. Source slide page 18 (extracted text): PYTHON FOR COMPUTATIONALPROBLEM SOLVING Operators Bitwise Operators & AND a = 5 # 0101 b = 6 # 0110 c = a & b # 0100 =4 | OR # 0101a = 5 b = 6 # 0110 c = a | b # 0111 =7 ^ XOR a = 5 # 0101 b = 6 # 0110 c = a ^ b # 0011 =3 EV0000918 · EV0000919 · EV0000920

19. Source slide page 19 (extracted text): PYTHON FOR COMPUTATIONALPROBLEM SOLVING Operators Bitwise Operators (Right shift operator)>> a=250 a>>33 #Ri#Righth shifthif a bby threeh bibits 31 #Answer is 31 Working: 1 1 1 1 1 0 1 0 =250 1 1 1 1 1 1 0 1 0 =31 These bits are lost EV0000921 · EV0000922 · EV0000923

20. Source slide page 20 (extracted text): PYTHON FOR COMPUTATIONALPROBLEM SOLVING Operators Bitwise Operators (Left Shift)<< a=25 a<<2a #Lefte t shifts t aa byby 2 bitsb ts 100 Working: 1 1 0 0 1 =25 1 1 0 0 1 0 0 =100 Left shift by 2 places and insert zeros in the emptied places EV0000924 · EV0000925 · EV0000926

21. Source slide page 21 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Operators Identity Operators • is • is not ChecksChecks ifif thethe operandsoperands onon eithereither sideside ofof thethe operatoroperator pointpoint toto thethe samesame objectobject ororr notnot >>> a=10;b=10 >>> a is b True >>> a=10; b=10.0 >>> a is b False >>> a is not b True EV0000927 · EV0000928 · EV0000929

22. Source slide page 22 (extracted text): PYTHON FOR COMPUTATIONALPROBLEM SOLVING Operators Assignment / Shorthand Operators Combines arithmetic and assignment operators Operator Expression Short Hand ++= (Additi(Addition)) a = a + bb a += bb -= (Subtraction) a = a - b a -= b *= (Multiplication) a = a * b a *= b /= (Division) a = a / b a /= b //= (Truncation Division) a = a // b a //= b %= (Modulus) a = a % b a %= b **= (Exponentiation) a = a ** b a **= b EV0000930 · EV0000931 · EV0000932

23. Source slide page 23 (extracted text): THANK YOU Departmentp of Computerp Science and Engineeringg g Dr. Shylaja S S, Director, CCBD & CDSAML, PESU Prof. Sindhu R Pai – sindhurpai@pes.edu Prof. Chitra G M Prof. Gayatri S EV0000933 · EV0000934 · EV0000935

SHORT-NOTE RECAP

Transcript block 1 (source interval 000000-000299 seconds): [00:00:00.292] Hello, everyone, and welcome back to the lecture series on Python for computational problem solving. I am Nita Ann Jacob from the Department of Computer Science and Engineering. In this video, let us explore the concept of operators. [00:00:16.868] What is an operator? It is a symbol that denotes an operation to be performed on one or more operants. For example, the plus symbol here denotes the operation of addition. [00:00:31.396] And operands are values given to the operators on which the operation is performed on. For example, two and three are operands the expression two plus three. [00:00:45.924] When we talk about operators, we must also understand the concept of Rity or Rank. Rity or rank tells us how many operants can be used with a given operator. [00:00:59.908] So if an operator is a unary operator, it can only be applied to a single operant. As shown in the example, the negation operator minus with the operand 12 together would denote a negative number. [00:01:17.540] A binary operator is one in which two operands can be used together to give a valid expression. So two plus three is an instance of a binary operator plus with two operands, two plus three. [00:01:35.620] So let's look at some of the arithmetic operators. You have the negation operator, as we've already seen, which denotes the sign of a number. By default, the sign of every number is positive unless you explicitly use the negation symbol. [00:01:54.916] Then you have the addition, which can be used in both unary and binary form. When used in unary, it states the obvious that the number is a positive number. You have subtraction, where you subtract the value of y from x. [00:02:12.644] The asterisk symbol is used for multiplication. The double asterisk is used to denote exponentiation, which is basically x to the power of y. A single forward slash for division. [00:02:28.708] Two forward slashes for truncation division, and the percentage symbol for modulus. Let's look at each of these with an example. [00:02:38.020] I am going to use the idle platform to use the Python interpreter in the interactive mode. This is what is available in Windows when you install the Python interpreter. So I'm just going to use the start button in Windows. [00:02:57.124] Choose the idle platform and it gives you the idle interactive mode, the Python interactive mode. So to add two numbers, you can use the plus operator to subtract. [00:03:14.372] One number from the other, let's say 3 minus 2. So I'm subtracting 2 from 3, and what remains is 1. Multiplication is the single asterisk operator. And [00:03:30.276] Double asterisk

is for exponents. So if I give 2 star star 3, it means it's 2 to the power of 3, which is 8. Now before I show you how the division operator works, let me give you some examples or some features of the division operator. [00:03:50.916] Python Poho provides two forms of division. True division, which is denoted by a single slash. So when I do 25 slash 10, it will give you 2.5. Truision always results in a floating point value. [00:04:10.884] The next is called the truncating division, which is denoted by the double slash, and it provides a truncated result. So if I were to give 25 slash 10, the result will be an integer truncated division of the value 2. [00:04:29.636] Now when both operand are integers, it will give you an integer division. If at least one of the operands are floating point, it will give you a truncated floating point. [00:04:44.100] So here are a few combinations. If both of them are integers, then the result is float in terms of the true division. [00:04:54.948] If it's one into one float, it will still be float, and if both are float, then the result is still a float. So you can see here that when we use the true division, the result is always a floating point value.

EV0000198 · EV0000199 · EV0000200 · EV0000201

Transcript block 2 (source interval 000300-000599 seconds): [00:05:08.352] So you see here 7 divided by 5 is 1.4, 7 by 5.0 where you have one integer and one float is 1.4. And when both are floating point numbers, you still get the same result. [00:05:22.532] But in the case of truncating division, if both operand are integers, you can see here 7 and 5, it will give you 1 as the resulting operator. Because 5 when you divide by 7 will give you a decimal value. [00:05:42.756] Int and float, when one object is int and float, it will give you the truncated value in the floating point notation. Similarly, when there are two float values, it will still give you one width. [00:05:58.660] precision pattern. So let's look at that with some examples. I will give it here. So let's see 7 divided by 5, floating point. 7.0 divided by 5, it's still a floating point division. Whereas if I give 7 [00:06:18.692] Truncation division five gives you one because the four will be truncated and if I give seven point zero truncation division five I still get one point zero. So what the truncation division does is it truncates [00:06:35.952] The value after the decimal point. So we do have a loss of precision or loss of data when we use the truncation division. Let's go to the next operator, which is your modulus operator. [00:06:53.444] The modulus operator gives you the remainder of division of the operants and it always results in a cycle of values. So here is a partial listing of the modulus operator. [00:07:06.736] In the first column you have the modulus operator of 7. So you can see here it starts to cycle between 0 to 6 and 0 to 6 and so on. Similarly with the values 10 and 100. [00:07:20.624] So if I were to divide or find the remainder of 9 mod 7, it will have a remainder of 2. And that is what your modulus operator does. Let's look at it with some examples. [00:07:39.844] So in item [00:07:53.444] I'm going to give 10 mod 7. So when I perform a division, I will always get a remainder of 3. And that is what your modulus operator does. Now that we have seen arithmetic operators, let's look at what an expression is. [00:08:12.804] An expression is a combination of symbols that evaluates to a given value. Now expressions most commonly consist of operators and operates. [00:08:26.148] So let's consider the expression 4 plus 3 star k or 3 into k. This is a valid expression because it's a collection of symbols and values or operators and operants. Now [00:08:44.964] Expressions can also consist of a single literal value. We've seen what literals are. They are either numeric literals or string literals. So I have 4, which is a valid expression, 3, which is a valid expression, and k also is a valid expression. [00:09:04.804] Because each of these individual values have some literal meaning. So for example, 4 stands for the numeric literal four. So anything which has a value is an expression. [00:09:20.740] Now this expression 4 plus 3 star k in brackets has a sub expression. [00:09:28.004] So a sub-expression is one, which is a smaller

expression used in the entire context. So 4 is a sub-expression, 3 star k is a sub-expression, where 3 is again [00:09:44.388] An expression and K is also another expression. Now all expressions that amount to a numeric value are called arithmetic expressions. [00:09:57.412] So let's look at what would happen if I have an expression with different operates. It determines the order of evaluation. So we have something called as operator cut.

EV0000202 · EV0000203 · EV0000204 · EV0000205

Transcript block 3 (source interval 000600-000899 seconds): [00:10:15.332] start this slide so we'll have to to cut that part out. [00:10:27.684] Start. Operator precedents will tell us the order of evaluation when we have an expression with multiple operants. So let's consider the expression four plus three star five. [00:10:43.492] There are two possible ways in which I could calculate it. The first one, I could first multiply 3 star 5 and add it with 4, which gives me the value 90, or I could first add 4 plus 3 and then multiply it by 5 and get the value 35. [00:11:02.564] So you can see the order in which I evaluate the expression gives me different results. In order to maintain uniformity, we have the concept of operator precedence. [00:11:15.664] So each programming language has its own rules for the order of preference of precedence, which ideally matches that of mathematics. In this table, we denote the operator precedence for [00:11:31.076] Python programming language. So the highest operator precedents goes to the exponentiation operator. Then comes the unary negation. [00:11:42.244] Then multiplication, division, truncation, division, and modulus. And last but not least, your addition and subtraction. So [00:11:53.668] When we have this expression, Python will always evaluate this with the precedence rules, where three into five is first evaluated, then added with four, which always gives you the value ninety. [00:12:09.156] So, if I want the addition to be performed first, I should ensure to use parentheses to mark the precedents. We have all learned the rules of bord mass in mathematics, brackets of division, multiplication, addition, subtraction. Similarly, in Python, [00:12:28.516] The expression within the parentheses will be first evaluated, and then the rules of precedence would be followed. So here 4 plus 3 would first be applied, and then the multiplication is applied, which will give you the result, 35. [00:12:48.836] Now what if I have an expression which have multiple values to see the operator precedents. [00:13:00.676] I've only seen plus and multiplication. Let's look at division, exponentiation, and so on. So here I have four plus two star star five. [00:13:14.468] Truncation division ten. So as the rules specify, exponentiation is first. So 2 to the power of 5 is first evaluated to give me 32. Then it is divided to give me 3 because it's truncation division. [00:13:30.432] and both are integer values, then added with four to give me the value 7. So operator precedents always guarantees a consistent interpretation of the expressions. [00:13:44.804] So it is good programming practice to always use brackets or parentheses, even when it's not required. [00:13:54.852] So if I want to make sure that my equation or expression is always evaluated in a uniform way, I can ensure to use these parentheses. [00:14:07.300] Now, what if I have an expression with all my operands and operators, where the operators have all the same [00:14:19.716] Cut and we'll redo the slide. Start. Now what if I have an expression where I have a collection of operators all with the same precedents. [00:14:34.852] in order to identify how exactly it should be evaluated, we have the concept of operator associativity. So if we have more than one operator with the same level of precedence, for example, multiplication and division, both have the same [00:14:52.452] order of precedence, how would I know which one should be evaluated first? So for operators that follow the associative law, it doesn't make any difference. For example,

EV0000206 · EV0000207 · EV0000208 · EV0000209

Transcript block 4 (source interval 000900-001199 seconds): [00:15:05.924] The plus operator, if I were to add 2 plus 3 and then 4, it gives me 9. At the same time, if I were to add 3 plus 4 and 2, it gives me the same result, 9. [00:15:18.276] But we have operators like division and subtraction, for example, that do not follow the associative law. [00:15:26.468] So in subtraction, if I were to choose to subtract 4 from 8 and then 2, I will get the result 2. Whereas if I were to subtract 2 from 4 first and then from 8, I will get the value 6. Similarly for division, [00:15:43.908] If I divide the first two operands and then the third, I get one, whereas if I divide the last two operands and then [00:15:55.140] In order to resolve this ambiguity, we have the concept of operator associativity, which will tell you the order in which it should be evaluated. So first the precedence is applied, then the associativity rules are applied. So these are the associativity rules. [00:16:14.564] Exponentiation is right to left, negation, multiplication, division, addition, subtraction all follow left to right. So we have to note that exponentiation [00:16:26.468] is the only one which has a right to left association, whereas every other operator has left to right association. Therefore, if I have an expression that's two exponent three exponent two, it will [00:16:44.932] not give you 64, but rather give you first three to the power of two and then two to the power of nine, which is five twelve. So five twelve is the correct result. [00:17:01.892] Now let's look at the concept of Boolean expressions. What are boolean expressions? A Boolean data type in Python contains just two values, either true or false. So an expression that evaluates to true or false is called a boolean expression. [00:17:22.532] Boolean expressions are usually used for selection or iterated processes. So where exactly would we use the concept of Boolean expressions? [00:17:34.628] We have another operator called the relational operator. A relational operator is used to compare two values and it always results in a true or false values and therefore called a Boolean expression. These are the relational operators that are present in Python. [00:17:54.404] Two equal to symbols or the two assignment operators together denote an equals relationship. You can see here we're trying to see if 10 is equal to 10. Since both the values are equal, it results in true. [00:18:12.996] An exclamation point followed by equals is for not equal. So ten not equal to ten is false, therefore it gives you a false result. So as you can see, all of them result in a boolean button. [00:18:28.752] Which is either true or false, and hence all relational operators are a boolean expression, or form a boolean expression. So less than would check if the first operant is less than the second operand. [00:18:45.380] Greater than the first operant is greater than the second operand and so on. Now you can see here it's not just applied to numbers, it can also be applied to strings and alphabets or collections as well. [00:19:02.724] So let's look at some simple comparison. As already seen, I can compare two numbers, which would result in two. We've seen 10 equal to 10. 3 greater than 2 is true, so it returns true. And we can also use the concept of cascading comparison. [00:19:20.324] Now, since your relational operators are binary operators, I can combine them as well. So I can compare three operands with the two operators. So if I have an operand A [00:19:36.004] operant w operator one, B, Operator 2, C is the same as evaluating the first expression A operator 1B and the second expression. [00:19:49.252] So therefore, if I were to use 3 greater than 2 greater than 1, it's the same as first calculating if 3 is greater than 2 and if 2 is greater than 1 and if both of them are the same.

EV0000210 · EV0000211 · EV0000212 · EV0000213

Transcript block 5 (source interval 001200-001499 seconds): [00:20:04.868] valid expression or results in true, only then will it give you a true value. If the second expression or the first expression was not true, it would result in a false value. [00:20:19.204] Now let's look at how I

could use these relational operators in string comparison. [00:20:25.764] So in string comparison, you can compare the corresponding characters in terms of its ASCII values. So the precedents is given to ASCII values. So if I have cat and car, both of these are [00:20:42.148] Similar in length, that is there are three letters, but we know that the alphabet T has a higher ASCII value than R, so therefore it results in a true result. [00:20:56.708] But what if I have cat and cattle? The first three letters and the first three letters in cattle are both the same. So since the ASCII values are same, the interpreter now looks at [00:21:10.640] What is another criteria based on which I can perform the comparison? So only if the ASCII values do not match will we consider the length. So in this case, since the length of the second operant is greater, it results in the result false. [00:21:31.556] Next one, I have both the same words, but one is in uppercase and the other one is in lowercase. And we know that uppercase characters have a lower ASCII value than lowercase, so therefore it gives you the value false, even though. [00:21:49.604] Semantically, cat and cat are both the same, but syntactically, both of them have different values. Now [00:22:00.548] What about if apple is greater than Z, even though the string 1 is greater in length than the string 2 here, the second operant, the value or the ASCII value of Z [00:22:14.176] is greater than A, so therefore it gives you a false value immediately, and the length is not compared in this case. And the last one I have zebra which is greater than ABCDEFG. [00:22:33.956] And the first operant is shorter in length than the second operand, it still gives you true because the ASCII value of Z is greater than the ASCII value of A. Now what [00:22:49.956] Is done when I have list comparison. The same rules of the string applies that every element of the list is compared till it finds mismatch. So here, even though the first list is larger in size, 10 and 10 are [00:23:09.604] Okay, but 20 is not greater than 25, so therefore it's going to tell you that it is a false value. What about when I have a tuple 1020 and ABCD? Touble 1020, ABC C [00:23:27.780] Now the first values are the same, so the greater than operation doesn't hold any meaning. The second one, it doesn't know how to compare, so it goes to the second one, A, B, C and D, and A, B, C and C. Since D [00:23:44.356] In the string is greater than C in this string here, it will give you a true value. [00:23:55.172] The next operator that we will see is the membership operator. The membership operator is used [00:24:02.436] Check if a particular operant is part of another collection of values. So we have two membership operators, one is in and not in. So it's as simple as it sounds. [00:24:18.768] Is the value 10 in the collection 10, 20, and 13? It is, so it gives you a result true. And is red in the collection of these three values? Yes, it gives you the value true. Note that the red here should be within quotes. [00:24:37.796] Not in will check if a value is not in 10, 20, and 30, which gives you false, since 10 is present in the collection. Let's look at it with some examples.

EV0000214 · EV0000215 · EV0000216 · EV0000217

Transcript block 6 (source interval 001500-001799 seconds): [00:25:05.636] So if I want to give ten in, ten, twenty, and thirty as always, it gives you two. [00:25:14.788] I can give 23 in a list, 12, 22, and 44. It gives you false. I can also use membership operators to check if certain substreams are present in a larger stream. [00:25:32.352] For example, I can check if C E is in chatter. Since it is, it gives me two. What if I give C T in Chatter? [00:25:50.084] This will give me false. Even though C and T are both present in the string, when we give a sequence of characters, it will always look for the existence of this entire string in the same order within. [00:26:08.996] The main string or the main collection of characters. So if I were to give just T in [00:26:16.868] Capital, this will give me true because T does exist. But C and T do not exist next to each other in the stream and that's why it gives you the value false. So these are your membership operators.

EV0000218 · EV0000219 · EV0000220 · EV0000221

Full transcript: 3295 normalized words, 102 timestamped segments, 1592.704 seconds duration. Low-confidence segments flagged: 0.

EV0000198 · EV0000199 · EV0000200 · EV0000201

17 Bitwise operators

This lesson is indexed from the source video titled Bitwise operators. Its complete timestamped transcript is preserved below in segment order. The original video path and content hash are recorded in datasets/lecture-corpus.json. Transcript text is source wording; it has not been silently corrected or rewritten. A candidate slide deck is grouped here according to the current matching score; the exact path and page locators are listed in the source evidence. EV0000222 · EV0000223

SHORT-NOTE RECAP

Transcript block 1 (source interval 000000-000299 seconds): [00:00:00.516] Let's look at the bitwise operators. We know that the computer stores information in terms of binary digits, either 0 or 1, and your bitwise operators are ones that perform operations at the bit level. Let's look at the bitwise operators that are available in Python. [00:00:20.292] The first one is the AND operator, where you use the ampersand symbol to represent the operator, and the result would be one if and only if both the corresponding bits are one, else the value will always amount to zero. [00:00:39.332] The next one is all defined or represented by the pipeline symbol, and the result will be 1 if at least one of the bits is set to 1. If both bits are 0, it will return 0. [00:00:57.988] The next one is the exclusive OR operator, denoted by the caret symbol, where it will result to one only if both of the bits are different. So there should at least be one. [00:01:13.348] 0 bit and another 1 bit. Only then will the result be 1. The next one is your left shift operator, and for every value that you apply the left shift operator to. [00:01:29.380] One shift results in a multiplication by two of the value. And the right shift will for each shift will divide the value by two. [00:01:40.900] All of these operators are binary operators where we will have two operants to be used to get a valid expression. The last one is the complement operator defined by the tilt symbol and [00:01:56.868] What it does is it changes any given bit from 0 to 1 or 1 to 0. So let's look at some of the examples. [00:02:05.988] The first one we will explore is the AND operator. So let's consider two variables a and b. A is 1, and I've mentioned the binary representation as well here. For 5 it is 0101, and for 6 it is 0110. [00:02:25.028] Now when I perform the AND operator, it is applied to every combination of bits in this particular representation. So the rightmost bit combination will result in zero since both the bits are not one. [00:02:43.652] The next column also would result in zero since it's both not one. [00:02:49.924] The third column here would represent or result in one since both the corresponding bits are one. And the last one would be represented as zero. So the output for [00:03:04.228] C is equal to A and B, which is your bitwise operator, would result in 4. So the binary representation of 4 is as C. [00:03:15.460] Now let's look at how we would use the OR operator. I'm using the same values of 5 and 6, and we know the OR operator says that if at least one of the bits are 1, then the result would be 1. So the first three columns we have at least [00:03:33.088] one value or one bit set to one. So therefore the result would be seven, since the first

three columns would all return the value one. [00:03:44.836] The next one is your XOR operator and the rule is that it would return one only if the corresponding bits are different. So the first one it is different, it would return one, second column would also return one. [00:03:59.268] Third and fourth column would return 0 since both the bits are the same. So the result would be 3 because the first two bits would be set to 1. Now let's look at the right shift operator. [00:04:18.724] Like I said, in the right shift operator, what we have to do is shift the bits to the right hand side. So let's look at the working of the right shift operator. The first thing I will [00:04:31.812] right is the binary representation of A, which is five, assign the value five, so it's 011, 01, 01. So [00:04:44.900] I will draw a line here to denote that when a bit is now shifted to the right, anything beyond the least significant bit, which is what I've highlighted here with my highlighter.

EV0000222 · EV0000223 · EV0000224 · EV0000225

Transcript block 2 (source interval 000300-000599 seconds): [00:05:01.060] is the least significant bit. So any value that is beyond the least significant bit does not consider or apply to calculating the value. [00:05:13.092] So what I need to do is right shift it by one first. So what I'm trying to do is here A right shift it by two positions. So to [00:05:24.516] First shift it by one position, you can see the one is shifted out. So therefore the only values which would be considered are 0, 0 and 10. [00:05:36.228] The bit beyond the least significant bit mark is discarded, and now the value of A would be 2. Next, what I have to do is I have to shift it once again. I need to shift zero to the next bit. So therefore, one would also be shifted out again. [00:05:56.772] So when I shift one bit to the right, you can see now 0 and 1 are both beyond the least significant bit marker and are not considered when we calculate the value. And now it is 0, 0, 0, 1. So the value of A is 1, which is what would be stored in the variable P. [00:06:17.412] Left shift works in the exact opposite manner. So I'm going to shift A to the left by two times, by two bit positions. So first I'm going to shift [00:06:31.812] 1 to the left by 1 bit. Now since it's moving closer to the most significant bit, what we would do is add a 0 at the least significant bit. So you can see the value of A is now 10, which is basically a multiplication of 2. [00:06:51.556] The next one, when I shift another bit to the left, it will again be multiplied by 2, and the final value would be 20, which would be stored to B. This is how your left shift and right shift operators work. Let's look at [00:07:14.756] Cut? [00:07:17.892] Start. Let's look at all of this with an example. I have written a program with all of these values that we have just discussed. You have A and B, A or B, A X or B, A right left shift by 2, A right shift by 2, and [00:07:37.472] Complement of A. So the values that we expect are 4, 7, 3, 20, 1, and minus 6. So in the complement you can see that the value of 5 is complemented and everything. [00:07:57.092] in the most significant bit also will be set to 1. So the result that you will get is minus 6 and a rule of thumb every time you say find a complement of a value n is that it will result in minus [00:08:14.148] N plus 1. This is a generic rule that can be followed to find the complement of a number. The extensive calculation can be looked at at a later point of time. So let's run this program. [00:08:31.396] So in order to do that I will type the keyword Python in the path where I've stored the program and the name of my program is bitwise.py. You can see as expected it's 473 201 and minus 6. [00:08:49.060] Let me also show you an example of how these bitwise operators can be useful. [00:08:54.596] Now, we've already seen how we could write a program to swap two numbers with three variables. Now, what I want to do is save the memory and use only two variables so that I can swap it. That can be done with your [00:09:13.924] pitchwise operators so I'm going to take the values of A and B as five and six. So initially when it prints the value of A, it would print five line 9, 8, it would print 6. [00:09:32.260] Now what we're

going to do is first A is equal to AXO B. So when I exall 5 and 6, the value I get would be 3. So at this line, the value of A is going to be 3. [00:09:47.684] Next line, I'm going to calculate A XOR B again and store it back in the variable B. So the value of A is now 3 and the value of B is still 6. So when I perform an XOR operation.

EV0000226 · EV0000227 · EV0000228 · EV0000229

Transcript block 3 (source interval 000600-000899 seconds): [00:10:03.620] The result I get is 5 and I'm going to store that in the variable B. So now you can see the first part of my swapping of values is already done. So B has got the value of A, which was 5. [00:10:20.356] Now A is 3 and the value of B is 5. So to obtain the value 6, I'm again going to use the bitwise operator XOR. So AXOR B, 3, XOR 5. [00:10:35.940] Will give me six. So now the value of A is going to be six. Now this completes the swapping of two values. So when it does print the statements, A will now be six and B will now print. [00:10:55.236] 5. Let's look at the output. When we python, I've stored it as bit by swap.py. When I hit enter, you can see initially it was five and six, and after the swap it is six and five. [00:11:13.444] These are your bitwise operators. [00:11:21.540] The next set of operators are called your identity operators. What it does is checks if either of the operands only cut, start, cut, [00:11:41.412] We will now look at the identity operators. What it does is it checks if the operants on either side of the operator point to the same object or not. And these operators are denoted by the words is and is not. So let's look at how it works with an example. [00:12:01.380] So I'm going to use a variable A is equal to 10 and B is equal to 10. [00:12:08.036] Now we've already seen that in Python values have a memory and not variables. So when I have two variables that point to the same value, they may share the same memory location. And how we see that is by looking at [00:12:24.432] The ID of A and the ID of B. So you can see here that the IDs of both the values are the same. [00:12:37.220] Which means that both A and B point to the same object. And that is what we will explore in your identity operator. I'm going to check if A is B, which means are both the variables pointing to the same value. [00:12:56.036] It's the same as if I would write ten is ten. So you can see that it is true. [00:13:03.140] Now if I have different values, say A is equal to 10 and B is equal to 11, if I check if A is B, if I give you a false value, which [00:13:16.740] Just tells us that both of them point to different objects. I could check if A is not B. So if both the operants point to different memory location, it will give you a true value. [00:13:32.868] These are your identity operators. So ASC is would return true if both of the operants have the same ID and is not would return true if both of the operants have different IDs. [00:13:50.116] The next operator is your assignment or shorthand operator. This is an operator that just combines arithmetic operators and the assignment operator to give a shorthand form of the expression. [00:14:05.220] So let's look at a table with all these values. So plus and equals to in the expression A is equal to X, A plus B just reduces it to A plus equals to B. [00:14:18.948] So both of these expressions are equivalent and perform the same operation. So this could be read as A is equal to A plus B. [00:14:31.364] because the right hand side of your assignment is always evaluated before the left hand side. So similarly for subtraction, multiplication, division, [00:14:43.044] Truncation division modulus and exponent, you can use these assignment operators. Let's look at how we could use them and see if both expressions give us the same value. So I'm going to write A is equal to say 10 and B is equal to 2.

EV0000230 · EV0000231 · EV0000232 · EV0000233

Transcript block 4 (source interval 000900-001199 seconds): [00:15:02.916] So if I were to write A is equal to A plus B and check the value of A, I will get the value 12. So I'm going to reassign the

value of A to 10 and write A plus [00:15:22.628] equals to 10 B and that will now give the same value. So similarly A is equal to 10 and A minus equal to [00:15:40.292] Would return the value 8 and reassign A to 10, and A is equal to A minus B, A will still have the value 8. So it's just an easy way or a shorthand way of writing an arithmetic expression using these assignment. [00:16:00.004] or shorthand operators. So let's quickly recap the operators we have looked at so far. You have arithmetic operators, which are denoted by all these symbols the addition, subtraction, multiplication, true division, [00:16:17.488] truncation division modulus and exponent then you have the relational operator which checks if values are equal or not lesser than lesser than equal to greater than and greater than equal to [00:16:31.204] Logical operators that work on true or false values, which are denoted by and, or and not. Membership operators that check if an operant is part of a collection. Using the words in and not in. [00:16:49.348] Bitwise operators which work at a bit level and you have the representation of ampersand for and pipeline for OR, caret for XOR, two greater than symbols for right shift, two lesser than symbols for left shift, and a tilt symbol for complement. [00:17:09.540] the identity operators that check if the memory location of both operants are the same using is and is not and last [00:17:20.260] One is the assignment or shorthand operators, where you combine the assignment operator with each of these arithmetic operators to give a shorthand representation of your expressions. Thank you.

EV0000234 · EV0000235 · EV0000236 · EV0000237

Full transcript: 2346 normalized words, 68 timestamped segments, 1056.874667 seconds duration. Low-confidence segments flagged: 0.

EV0000222 · EV0000223 · EV0000224 · EV0000225

18 logical operators and short circuit evaluation

This lesson is indexed from the source video titled logical operators and short circuit evaluation. Its complete timestamped transcript is preserved below in segment order. The original video path and content hash are recorded in datasets/lecture-corpus.json. Transcript text is source wording; it has not been silently corrected or rewritten. A candidate slide deck is grouped here according to the current matching score; the exact path and page locators are listed in the source evidence. EV0000238 · EV0000239

SHORT-NOTE RECAP

Transcript block 1 (source interval 000000-000299 seconds): [00:00:00.452] The next set of operators we will look at are your Boolean or logical operators. In Boolean algebra, we have a collection of Boolean operators which are denoted by AND, OR, and NOT. [00:00:15.876] Using these logical operators, we can create arbitrarily complex Boolean expressions. [00:00:23.140] As we've already seen, Boolean expressions are one that are evaluated based on the occurrence true or false value and always result in a true or false value. [00:00:37.156] Let's look at the working using the table below. I have taken two variables x and y and we are going to explore how each of these operators work with a combination of true or false values. [00:00:51.812] Now, as you can see, the AND and OR operator are binary in nature, whereas the not operator is unary. [00:01:02.628] So when you see the column for the AND operator, you can see that it only results in true when both the values of X and Y are true. So we can conclude. [00:01:17.744] that the and operator returns true if both

values are true and false otherwise. [00:01:24.548] Whereas the OR operator returns false only if both the values are false. So even if one of the operings are true, it will always return a true value. [00:01:39.140] And the not operator always gives you the inverse of the value of the variable. So if x were false, it would return true, and if x were true, it would return false. Let's look at this with some examples. [00:01:57.380] I will use the idle platform to use Python in the interactive mode. So let me try true and false. Since the and operator requires both of them to be true, it will give me a false line. So if I do true and true. [00:02:16.452] It gives me true. Let's look at the or so if I give true or false, it will give you true. False or false, will give you false. [00:02:33.188] Unary operators will always precede the operate. So not true will give you false and not false will give you true. Now what I'm going to do is use [00:02:49.252] quotes as an empty string and true but this is giving me an empty string as a result or if I would say true and zero it gives zero as a result. Now we know that Boolean expressions should give zero [00:03:07.344] True or false as the output, and not zeros or empty strings, and so on. Let's look at why this is so. [00:03:16.612] Now, in your Boolean expression, there are true and false equivalent values. We know that true and false denote the Boolean values, but there are other false values as well. [00:03:30.788] A zero number is a false value, an empty string or empty collections are treated as false values as well. [00:03:40.804] And true values are any non-zero numbers, any non-empty string, and any non-empty collections apart from the keyword tree. So let's go back to the idle platform. [00:03:56.548] So instead of giving the Boolean values true and false, why is it giving me an empty string or a zero as a result? This is because of the concept of short circuit evaluation. Now the logical operant. [00:04:13.572] When you see the first operand is false, we know that regardless of what the second operand is, it will always return false. Similarly, in the logical order, if the first operant is true, then it will always result in a true value. [00:04:32.324] Using this concept, what Python does is it does not evaluate the second operant if it knows the result by looking at the first operant. This is called a short circuit or lazy. [00:04:49.008] form of evaluation. So let's look at this with an example and how exactly it does work. So what we're going to look at is we know that when we use 10 divided by 0 it should give me a division by 0. But what I will do is write

EV0000238 · EV0000239 · EV0000240 · EV0000241

Transcript block 2 (source interval 000300-000599 seconds): [00:05:08.036] 3 minus 1, which will return 2, which is a true number, or I'm going to give 2 divided by 0. Now I know that when I use 2 divided by 0, it should give me a 0 division error. [00:05:26.852] But you can see here, it doesn't give me an error, but works just fine. The reason being, since this evaluates to a true equivalent value, and the next operator is an OR operator. [00:05:40.864] It knows that the result is always going to be true, so it returns the value that denotes a true value or a true result. Therefore, it doesn't evaluate the second operant at all. [00:05:55.376] This is called the short circuit or lazy evaluation, so that you can save up on some amount of processing if you already know the result. There is no need to evaluate the rest of the value as well. Similarly, [00:06:10.208] When I know that the first value is false and I have and say 10 divided by 0, it will give me zero and not a zero division error because using the and expression, if the first [00:06:26.372] operant is false, then it knows that the result is already false. And this is how your Boolean expression or your Boolean operators would work. [00:06:38.980] Let's look at the operator precedents and Boolean expressions along with all the other operators we've looked at. So the highest order of precedents still go to all the arithmetic operators, followed by relational operators. As you can see [00:06:57.168] all relational operators have equal precedence and would follow the left to right associativity when

used in an expression. [00:07:07.044] Whereas your Boolean expressions, the unary not takes highest precedence, followed by and and lastly, the all value.

EV0000242 · EV0000243 · EV0000244 · EV0000245

Full transcript: 954 normalized words, 30 timestamped segments, 439.573333 seconds duration. Low-confidence segments flagged: 0.

EV0000238 · EV0000239 · EV0000240 · EV0000241

19 operators and expressions

This lesson is indexed from the source video titled operators and expressions. Its complete timestamped transcript is preserved below in segment order. The original video path and content hash are recorded in datasets/lecture-corpus.json. Transcript text is source wording; it has not been silently corrected or rewritten. A candidate slide deck is grouped here according to the current matching score; the exact path and page locators are listed in the source evidence. EV0000246 · EV0000247

SHORT-NOTE RECAP

Transcript block 1 (source interval 000000-000299 seconds): [00:00:00.804] Hello all. Welcome back, Python for computational problem solving. [00:00:07.780] What are we going to be doing in today's session? In today's session, we are going to be covering the concepts of operators and expressions, precedents and associativity. Hope you're all ready. Let's get started. [00:00:22.820] What I'll be doing is, as and when required, I will be moving out of the presentation mode. I'm gonna be executing uh the things for you so that uh the concepts are uh [00:00:34.336] Uh easy for you to understand. Today I have decided to work with the interactive mode. You can also try working with the batch mode. Only differences as in when we uh it is required for you, make use of the print statement. Along with that, execution is similar. [00:00:51.984] Is it is in a similar way because we have already done it in your previous uh sessions. How will you execute? Save your program with dot per EY extension, name or with dot py extension, go to the terminal. [00:01:07.264] On the batch mode, what do you have to do? Type it as python program name dot p y. Okay, so please uh follow these instructions. Right? So let's get uh started. What is an expression? [00:01:26.404] We've been speaking about an expression all this time, so what is an expression? An expression is a one which has a value. An expression is a one which has a value. [00:01:38.160] Constants are also called as expressions, then a variable is also called as an expression. So please remember an expression can be a constant or it can be [00:01:51.492] a variable also. So in this case if you can see we have uh 123 which is a constant integer constant, we have a string constant, we have a floating point constant. Along with that we also have an identifier which is being referred with the name. [00:02:09.152] A it is also an expression. Okay? So are these the only expressions that we have? No, we have many more. Okay. So [00:02:21.584] What are the uh other ones? So binary operators, unary operators, anything that is enclosed within the parenthesis, they are all called as an expressions. They are all called as an expressions. Okay, so now [00:02:37.604] When we have an expressions, there are few things that we have to understand. Okay? There are few things that we have to keep in mind while working with the expression. So what are they to start with? [00:02:54.564] Fine. An expression always has a value.

[00:02:59.428] And an expression always has a value. There is also something called as a statement. What is the difference between an expression and a statement? I will let you know. Okay so now a statement does not have a value. [00:03:16.580] An expression can also be a statement, but a statement is not an expression. Okay, it need not be an expression. So let me repeat this and then I will explain it to you what exactly it means. Fine. An expression. [00:03:33.424] Has a value, a statement does not have a value. An expression is also a statement, but a statement is not necessarily an expression. So what does this mean? Let me uh give you a quick example. Okay. So I'll go here. [00:03:52.640] Type it as python. Okay so yes, we are into this. So I will also say a is equal to 20. Fine. In this case, what is the value of A? 20. Is this fine? So now A is equal to 20. It's clear for us that it is an expression. Is this fine? So now [00:04:14.084] Okay, so hope uh this is clear for you. Now I'll say print [00:04:19.300] A is equal to 10 is what I will do. Okay. So before this, let me print A, just an A, absolutely fine, because A has been assigned a value called as 20. But now what am I trying to do? I'll do something like this. A is equal to 10. [00:04:36.388] This is fine. So now when I execute, let's find out. Okay. [00:04:44.068] It's an error. Okay? It's giving us an error. The reason is A is equal to 10 as a whole is an assignment statement. This is clear? A is a variable. [00:04:59.008] Fine. A is equal to 10 overall is an assignment statement. That's the reason why it is not an expression. At this point, it is an assignment statement. Hopefully it is clear. Only A is

EV0000246 · EV0000247 · EV0000248 · EV0000249

Transcript block 2 (source interval 000300-000599 seconds): [00:05:16.164] an expression. Is this clear? A is an expression. [00:05:20.964] Clear and this 20 separately is an expression. Okay. When I combine both of them, this becomes an assignment statement for us. Okay. So hope uh this is uh clear for you. Let's [00:05:37.476] continue our discussion. So that is what we meant by this particular statement. Okay let's carry forward and do a few more things. And now operators. Okay. So what is an operator? [00:05:55.044] Operators are the ones which indicate some actions that are resulting in a value. [00:06:02.980] Okay, there are certain actions that take place which results in a value. Okay, they are called as your operators. Whenever we speak with respect to the operators, we basically talk about the three different things. [00:06:17.348] We talk about three different things. One is called as an arity, second is the precedence, and third is the associativity or association. Okay, so what is arity? Arity refers to the number of operand required [00:06:35.104] For an operator. Okay. So in this case, if we say about a plus, plus as an operator, it needs two different operands. So in this case, RT is going to be two. Okay. And um [00:06:50.628] An unary operator. Is this fine? An unary unary operator. So there are binary, unary, and turnary operators. Okay, as and when required, I'm going to be speaking more about it. Fine. So in this case, plus operator, an operator. [00:07:08.516] Plus it requires two operands so its rank is two. Okay. Unary operator requires one, turnary operators requires uh three different uh operands. So in such case we call it as a rank. [00:07:23.844] Okay, so moving uh forward, there's something called as a precedence. When do we use a word called as a precedence? A precedence is a word that is used when which basically represents the order of evaluations. Okay, so [00:07:43.332] How the um operators [00:07:48.144] gets executed. How does the expressions get executed? Okay so that is where the uh precedence comes into picture. Okay? So before going, let me go back and then uh execute few things to you and then I will [00:08:04.656] Come back. Okay. So now what am I trying to speak about? Precedence. Okay. Suppose if I say two, every time two only comes, that's okay. Three star or sorry, four star four. [00:08:19.840] Plus four. Okay. So now in this case, what is the

answer? Because we have both the plus as well as the uh multiplication operator that is being there. So how do we evaluate? [00:08:32.484] Fine. What is the order of evaluation? Whether I have to start like this, or whether I have to go in an opposite direction. So all this um decision should be taken place. Okay. So now [00:08:47.748] Whether it is right to left or whether it is left to right, I will come back later. Okay. So that is called as an association. Now in this case, what is the order of evaluation? Okay. So now precedent says what is the order of evaluation. So if I say [00:09:05.892] for this. So this is basically you have operators [00:09:11.216] Uh multiplication and addition, both of them are orthomatic operators, they are at the same level. Okay? They all belong to orthomatic operators. So in which case whether multiplication executes first or addition executes first. Who says this? [00:09:28.128] Precedency of the operators is set. So in such case, if the operators [00:09:35.044] Okay, if there are two operators, so in such case we have to find out what is the order of evaluation. So first multiplication gets executed and then followed by addition gets executed. Okay, so that's the reason why. [00:09:52.496] The answer is going to be 20 because 4 into 4 is 16. 16 plus 4, 20. Okay. So hope this is clear. What if I just change this? Okay.

EV0000250 · EV0000251 · EV0000252 · EV0000253

Transcript block 3 (source interval 000600-000899 seconds): [00:10:08.932] So what if I change this in this case in order to make it little more um to understand? So I'll do this. So what is the answer in this case? Any guesses? Twenty four. Why? Because [00:10:23.556] Multiplication gets executed first, then followed by plus. So 4 into 5 is 20, 20 plus 4 is 24. Okay, so hope uh this is uh clear for you. Fine. Um clear to this? Okay. So [00:10:43.428] Go back. Fine. This is what we were talking about. Multiplicative operators have a higher precedence compared to additive operators. What does the precedent say? Order of evaluation. Which operator should get evaluated first. Fine. Now, going forward. [00:11:02.372] Going forward, you can see there's something called as an association. If you have more than one operators which are belonging at the same level of precedence, what does it mean? If I'm talking about earthmatic operators only. [00:11:20.228] Fine. Presidency will say. So now in this case, if you can see there are two multiplication operators and they um they have a same [00:11:31.012] level of presidency. Now question arises is whether I have to perform 2 into 3 first or I have to perform 3 into 4 first. [00:11:42.724] Okay, so which means do I need to evaluate in this manner or do I have to evaluate in the other way? So that is where your association comes into picture. Association basically specifies the order of evaluation. Do I need to go from [00:12:00.496] left to right or from right to left. Okay. From left to right it is called as left associativity. From right to left it is called as right associativity. So if you can look at this example I'm trying to [00:12:17.188] Cursor, cursor, yes. Okay. So in this case, because multiplication the order of evaluation, okay. So what is the order? It is left associative. So 2 into 3 is 6, 6 into 4 is. [00:12:38.884] 24. Fine. Six into four is supposed to be twenty-four. Sorry. Um there is a mistake uh from my end. I'm sorry. I would uh quickly rectify this. Six four's twenty-four. [00:12:58.404] For the mistake. [00:13:02.340] Okay, so I hope this is clear six, six into four. [00:13:07.684] is 24. Okay. Similarly, multiplicative operator is left associative, and if you see the exponentiation operator is from right to left. Okay. So basically [00:13:23.088] 3 to the power of 4 will happen and then followed by 2 to the power of 3. Okay, so 2 to the power of this entire um whole answer is going to be that. [00:13:36.768] Clear. So this is what association basically means. Okay. Hope uh this is clear for you. Presidents quickly, Rity mentions the uh rank, presidency, order of evaluation. [00:13:51.044] Okay, followed by association indicates how the evaluation should take place, whether it is left or left associativity, or whether it is right associativity. So these are given by

association. Right? So let's quickly move ahead. [00:14:13.764] Right? So moving ahead, addition [00:14:17.668] Authic operators. I'm talking about orthomatic operators. Hopefully most of them are self explanatory. Addition, subtraction, multiplication. Okay. If you can see this one is a division, the other one is also a division. [00:14:35.344] Okay, let's try to find out what is the difference between this and this. There's also one more called as a now modulus operator. Okay, the result is based on the remainder. Division is basically um [00:14:50.468] Fine, the quotient that we are talking about, uh whereas modulus is the remainder. Okay so let's uh take up a few examples quickly and then uh I would demonstrate it to you so that it is easy for you to understand. Go ahead.

EV0000254 · EV0000255 · EV0000256 · EV0000257

Transcript block 4 (source interval 000900-001199 seconds): [00:15:11.076] So let me give 25 divided by 4 because I don't want to do addition, subtraction, multiplication. There um you can easily understand. Okay. So fine. So let's do with this 25 divided by 4. [00:15:25.936] Answer is six point two five. So similarly I'll say twenty-five divided by again uh four. This is also called division only, fine. So let's see this. [00:15:38.656] You're getting an answer called 6, but whereas the above it gives me 6.25. Okay. The reason is this is called as a true division. [00:15:49.892] And this is called as a trunk division. So true division then followed by trunk division. Hope it is clear for you. Any uh there's no doubts. [00:16:03.044] I hope okay. True division, because I get the value as it is, and here we are truncating the decimal point. So it is trunk division. Okay, so we'll go ahead 24 modulus. [00:16:21.956] And so now in this case we are looking at the answer to be equal to whatever the remainder gives us. Okay, so this is also division, only differences I'm looking at the remainder. So in this case, what is the remainder that we are going to be getting? Zero is the remainder. [00:16:40.964] So my final answer of the modulus is reminder. Okay. So hope this is clear. So shall we take up an example of a floating point modulus operation to check what is going to be the answer. [00:17:01.476] And I'll say this 24.0 modulus 24 uh 4.0 answer is 0.0. Because 24 modulus 4 is 0, and anyway you have used uh the floating point number, so still it's gonna be 0 itself. Okay uh [00:17:20.804] Let me do one more 25 divided by 5. Anyway, we have done, but I'll do it with negative. So minus 5.0. What will happen if I do this? [00:17:38.244] Okay? Because it's a true division. Remember, it's a true division. It's minus. Fine. What if I use a trunk division in this case? So the answer is minus. [00:17:50.180] Okay so hope uh all these operations are uh clear uh for you. There's if there are no doubts, let's go ahead and continue our discussion here. [00:18:06.180] So these are the few things that we have uh already looked into. Okay, let's go ahead. Bitwise operators. So [00:18:20.196] The operators, bitwise operators that are available are ANS Cursor, cursor, cursor. Yes. And operator, it's called as an or, exclusive or left shift, right shift, and once complement. [00:18:38.660] And it's called as a bitwise operators because uh these operators are used to work with each and every bit. Okay? They're used to work with each and every bit. Fine. So now AND operator, it always uh how does the AND operator work? [00:18:58.116] If result is 1 if the corresponding bits are 1. For example, I have 2 bits assume 1 and 1. Here there is 1 and 1. So 1 and 1, result is 1. Or result is 1 if even one of the bits is 0. [00:19:18.116] Then followed by exclusive or result is 1, if and only if one of the bit is 1. Okay. Left shift is nothing but every time you perform the multiplication by multiplication. [00:19:33.316] Okay, sorry, every time you perform a shift operation, it is multiplying a given number by two every time. Okay, I I will demonstrate it to you. Don't worry, I'm going to be doing it for you. And then uh followed by [00:19:48.356] Uh the right shift operator. Every time you perform a right

shift, you are basically dividing a given number by two. Right? And the last one is a once complement where zero.

EV0000258 · EV0000259 · EV0000260 · EV0000261

Transcript block 5 (source interval 001200-001499 seconds): [00:20:03.524] is changed to 1 and 1 gets changed to 0. Okay. So this these are your bitwise operators. Shall we see with an example? Okay. So let's quickly see with an example. So [00:20:21.572] Let me go back here. Yes. Oh, it is not confusing for you. So I'll see A is equal to five. [00:20:34.948] Okay, I'll take up the same uh examples. Okay so in this case give me just a time yes so what we will do is [00:20:48.964] Internally, how these values are stored? Is it next question. The first question for us. [00:20:56.772] And 5, I know it is decimal. Okay. So now how it is internally stored. Internally they are stored in terms of 0s and 1s. Okay. So now how 5 would be stored? I'll say it as 0. [00:21:13.540] 0, 1, 0, 1. Okay, so this is how my 5 is stored. 2 to the power of, please remember, 2 to the power of 0, 2 to the power of 1, 2 power 2, 2 power 3. 2 power 0 is 1. [00:21:30.628] Fine, the value is one in this case. Okay so your values are like this one second, give me 9. 2 to the power of uh 0 is 1, then followed by uh 2 to the power of 1 is 2, 12, then I will have 2 to the power of 24. [00:21:48.324] And then I will have 8. Okay. So this is how the power of 2 are going to be. So if I want 5, what do I need to do? I need to enable this bit 4 and enable this bit 1. The rest other two bits are going to be equal to 0. [00:22:07.908] But internally are they stored like this? They are stored in terms of zeros and ones in case of binary. Only difference is if you are using a 32 bit machine, all your values are going to be stored in 32 bits. [00:22:21.156] If your computer is 64 bit and all the numbers will be stored as 64 bits only. Fine. Then you may ask a question what happens to the rest of the digits? They are all going to be equal to 0. [00:22:34.116] Okay, so we will make them as zero. So there is uh a different way in which uh the negative numbers need to be represented. [00:22:44.048] Uh so what I'm planning to do is I'm going to be uh send you or uh doing uh one more video on this where I'm going to be working on the examples uh with respect to the bitwise um uh operators. Okay, so that I'm going to be doing it shortly and then uh sending [00:23:01.776] It to you so that it is easy for you to understand. If you have understood it's well and good. Otherwise, definitely one I'm going to be sending it from my end. So this is A. [00:23:14.852] And this is my B is equal to 6. How do I save the value? 0, 1, 1, 0. How did I get? In this case, enable one here, enable one, the rest two are going to be 0. Okay. So this is what it is. Now. [00:23:35.588] I say A and B because it's logical and according to us according to us what uh what does an AND operator uh basically do? And [00:23:50.884] It results it result is 1 if the corresponding bits are 1. Okay? So if you look at this, just look at this. 1 and 0, it is 0. 0 and 1, it is still 0. [00:24:05.604] 1 and 1, it is 1, 0 and 0, it is 0. Okay? So only which bit is enabled, the bit number whose value is 4 is enabled. So when I do 5 and 6. [00:24:21.984] My answer is going to be four. Okay. So hope this is clear to you. Let me repeat, given any decimal number, convert it into binary. Okay. When you are converting it to binary in this way, okay. [00:24:36.612] 2 to the uh 4, 1, 2, 4, 8, 16, 32, 64, and so on. For the convenience sake, either you could take it as four digits or better way, take it as an eight digits. [00:24:48.608] Okay, don't try it either 32 bit or 64 bit because uh most of our values are going to be zero, especially when we are uh demonstrating a very simple uh integer numbers. Okay, so hopefully uh this concept is clear for you. Fine. Let me do um a or

EV0000262 · EV0000263 · EV0000264 · EV0000265

Transcript block 6 (source interval 001500-001799 seconds): [00:25:09.188] This is the or operation. Yes, A or B. Fine. So now same values. What does the OR operator basically say? R operator says

if any [00:25:21.700] um one of the bit is one then it results in one. Okay? So now one or zero one zero or [00:25:32.272] or one it is still one or one it is one itself so what is the answer in this case this is also one this results in one and this is also resulting in an one so now [00:25:45.952] What is the final answer is going to be? 4 plus 2 is 6, 6 plus 1, 7. So I'm expecting an answer to be 7. Okay, so we got the value to be 7. [00:25:57.668] Okay I'll quickly do the next one. I say A Xor B. Okay. A Xor B. So what does that XR? Fine? A XR exclusive R operation. The result is one. [00:26:16.356] If and only if one of the bit is one. Okay. The result is one. If and only if one of the bit is one. I want only one bit to be equal to one. [00:26:29.568] Okay. Otherwise what will happen? It's going to be zero. Which means if I have zero and zero, answer is zero. One and one, answer is one. Sorry, zero. [00:26:39.748] Okay. So now in this case we want only one bit to be one. So this one exhaall with zero, answer is one. The reason is if any one bit is one, the answer is going to be one. Okay. Zero exhaled with one, it is still one. Fine. [00:26:58.532] 1 and 1 we do not want because it's 0, 0 and 0, again 0. Okay. So what is the only these two bits are enabled, which means 2 and 1. 2 plus 1, 3. So when I perform A exhort with the B, my answer is 3 in this case. [00:27:25.252] So let me go ahead quickly and uh do the rest of them. So I'll say A [00:27:34.884] Oh which is the one that we have to do? Left shift. Fine. So let's do left shift by one. Okay so what did we say? When you do a left shift, it is multiplying a given number by two every time you perform a shift operation. [00:27:52.740] Fine, so according to this, what is the answer that we are going to be getting? A is fine. When I multiply it by one, what does this one say? How many number of times you want to perform a shift operation? Okay, so in order to make you to understand, I'll just write this zero. [00:28:10.512] 0101 okay so these are the four digits that are there please try doing it okay now when I say perform shift left okay when I say shift left what do I need to do this bit that is there it goes to the next position give me place [00:28:31.556] Okay, so if this is the thing, so now this zero goes to one position ahead, this one comes one position ahead, this one comes one position ahead, and what will happen to this position? It is filled with zero. [00:28:46.820] Okay, so when I do this shift once, shift once, this is shift it once. All of them have to be shifted once because I'm telling the number of times I have to perform a shift operation. So when I do one comes here, okay, so maybe, maybe, maybe. [00:29:05.252] So I will see when I perform a shift operation, one goes to this place, sorry, zero is still there, one has come to the next position. [00:29:18.404] Uh it's quite difficult for me to represent because it looks the uh same. Okay, so 0, 1, I will assume that this place says nothing, so it's going to be [00:29:29.092] Uh filled with 0. Okay. An empty location is going to be filled with 0. So according to this, 2 to the power of 0, 2 power 1, 2 power 2, and 2 power 3. So what is the answer that we are expecting in this case? It should be equal to [00:29:45.796] So 2 to the power 1, 2, 4, 8. 8 plus 2, 10. Okay. So the answer is 10 in this case. So similarly, when I perform the left shift by 2, every what does this 2 represent? 2 represents you want to shift it twice.

EV0000266 · EV0000267 · EV0000268 · EV0000269

Transcript block 7 (source interval 001800-002099 seconds): [00:30:03.888] A should be shifted twice. Every time you perform a shift operation once, it is multiplication by multiplication by two every time. Okay. So once you multiply you get 10. Second time you multiply, you get 20. [00:30:20.324] Hope uh this is clear for you. And similarly, the next one is a right shift once. [00:30:27.780] The moment you're doing it is every time it is division by one. Okay, you're doing a division by one. So one divided, sorry, a divided by two in this case. Okay. So this is multiplication and this is division. So 5 divided by 2. [00:30:47.972] Is what we are expecting. Let's see. Okay.

Answer is two. Uh the reason is when we are performing a shift operation, you always get an integral values. Okay again, when I say a light shift twice. [00:31:05.252] Okay, A right shift twice means divide A every time by two, and how many times you have to do? Two times. So five divided by two, you got two. Again two divided by two, answer is one. [00:31:22.308] So hope this is clear to you. Okay hope this is clear. Right. So let me uh quickly go ahead uh with the last one and I have to perform uh the uh [00:31:37.732] complement of A. Okay. So complement of A is what? Zeros will become equal to one and ones becomes your zeros. [00:31:50.148] Okay, so this is your uh bitwise operators. Hope uh the earthmatic operators and uh bitwise operators are clear for you. Uh there are uh no doubts. Okay, if there are no doubts, let's go ahead. [00:32:08.420] and try to understand the next operators that are there. Okay. So I'll quickly go ahead [00:32:19.748] The next set of operators are called as relational operators. Fine. So these relational operators are used where I can perform the comparison between two different values. [00:32:33.572] The result of a relational operator always results in true or false. Tru or false is called as a boolean type. [00:32:43.072] Okay, true or false is called as a boolean time. What are the relational operators that we are looking into? The relational operators are less than, less than, equal to, greater than, greater than equal to. This is called as equality operator. [00:32:58.960] Single equal is called as an assignment and this is called as equality operator. Not equal to in followed by ease. Okay. So what we will do? Shall we uh quickly uh execute on these things and then we'll come back. Yes. [00:33:20.804] So I'll say ten is equal to ten. [00:33:26.724] 10 is equal to 10. Yes, it is correct. Answer is supposed to be true because relational operators always give me result in terms of true or false. Okay, we said true or false, they belong to a class called Boolean. Fine. So let me say type of true. [00:33:46.724] What is the type of true? So it gives me a boolean. Please remember whenever I'm doing T is capital, otherwise it through an error. Okay. So always remember doing this. Yes. So let's quickly go ahead. Hopefully um 4 greater than 2. [00:34:05.572] Yes, it is. 2 less than 4, yes, it is. 2 less than 2, no it is. They're supposed to be equal. Okay. So that is where you get false. Okay. So hopefully this is clear for you. Suppose if I give [00:34:24.548] Three greater than two greater than one. Okay? If I give this. How does the evaluation take place? We said [00:34:37.284] Python knows marks better than anybody else. Okay so this gets evaluated as 3 greater than 2 followed by and 2 greater than 1. [00:34:55.332] Okay, so this is how the evaluation takes place. And I'm going to be coming back uh quickly and then I'll I'll uh I'll come back to you. Fine. So three greater than two, yes it is true. You have a true here.

EV0000270 · EV0000271 · EV0000272 · EV0000273

Transcript block 8 (source interval 002100-002399 seconds): [00:35:09.700] True, you also have an AND operator. Fine. Again, followed by two greater than one. Yes, it is true. True and true always results in true. Okay. So this type of comparison we call it as a cascading comparison comparison. [00:35:29.924] Called as a cascading comparison. So hopefully uh this is clear for you. So now let's look into this. What is the answer? Suppose if I say, can I compare a strings? That's what we are trying to do. So in this case, I say cat. [00:35:48.644] And then car. Okay. Every time we take up the same example, so I'm also taking up the same example. Okay. So now how does the comparison is going to happen? Comparison is based on the value that is being stored for these characters. [00:36:06.788] So the moment I say the values, what values am I talking about? How will I check the values? So in order to do it, we have a function called as an ORD which represents ordinal. Okay. I am talking about what is the ordinal value of A. [00:36:23.524] Right. So when I do this, if you can see 97. Fine. What is this 97? It's an ASCII value. It's an ASCII value. So A is stored as

97 in the computer memory. [00:36:39.268] Okay, so I will always do a comparison. Suppose if I'm saying A is greater than B. If I say this, how does it know A greater than B? Is it because A comes first and then B comes second? Is it so? No, it is based on. [00:36:58.192] uh the SC value that is present or the way in which it is stored in the computer memory. So how is it? So this is how [00:37:06.724] So when I enter, so it says false because I'm talking about 97 because A is 97 and B is 98. So I'm asking 97 greater than 98? Answer is false in this case. So this is how the comparison. [00:37:24.496] Happens. Hopefully you remember this function ORD ordinal value. And the other way is also true, I can say what is the character that is present at 97. Okay, value 97. [00:37:47.972] uh spelling mistake I'm sorry ch r it is not uh ch ar it is ch r okay what is value that is present so the moment I do you can see that we are getting a value called as A. Is this fine? Clear hopefully um [00:38:06.084] CHR it is not CH AR, it is CHR. Okay. So let's do the comparison. So I say cat is greater than car. Okay. Is cat greater than car. [00:38:25.956] What do I do? I'll start comparison. How do you start doing the comparison? I will um try to do until [00:38:37.956] Any mismatch happens or one of the string ends or when both the strings end. Let me repeat, do the comparison until there is a mismatch. [00:38:47.940] Or one of the string ends, or both the strings ends. So C and C are same, A and A are same, T and R, they are different. It's a mismatch. So now what do I do? I will do is [00:39:03.696] T greater than R is what I will do. Okay. There are strings, don't forget. So this is what I am trying to do. T greater than R. If it is true. [00:39:16.580] It as true, otherwise false. So according to us, T is greater than R S T. So answer is supposed to be true. [00:39:27.556] Right, so let's go ahead. I'll also do this CAT greater than C A T T something, I'll just do this. Right. So in this case, what will happen? Is it true or false? [00:39:48.484] False. The reason is in this case C18, C A T it is same, but this has got the longer. Okay. Second string is longer. That is fine. Answer is false in this case. Okay. So one more let me do C A R.

EV0000274 · EV0000275 · EV0000276 · EV0000277

Transcript block 9 (source interval 002400-002699 seconds): [00:40:08.548] Greater than C A R. So let's find out. Okay. Capital C and small c. Answer is false. The reason is what is the ordinal value of capital C? Ordinal value of capital C is 67. [00:40:28.452] And uh small c is 99. Okay. So we're comparing 67 greater than 99? No, it is not. So it is going to be wrong. Right. So let's it's a string. [00:40:49.860] Up okay, so I'm saying is up greater than some X, sorry this one. So Z I'll do okay, so up greater than Z [00:41:08.772] Okay, so in this case, we may be thinking is false. The reason is comparison never happens based on the length here. It is because it is very clear Z is [00:41:24.420] greater compared to the uh ASCII values so that's the reason why uh the [00:41:32.580] Try with uh the uh different uh examples so that uh you can understand things in a better way. Okay so hopefully uh this is clear to you. [00:41:51.236] Fine. So let me go back. So this these are the things that we have uh learned. [00:41:59.300] Fine. I know this video is going uh a little longer, but uh there's no option. Let's quickly complete these things. Okay. The next one is called as a logical operators. Logical operators are not, and and or [00:42:17.604] Okay, so let me do some examples and then we will come back here. Okay. There's also something called as a short circuit evaluation. Okay. We will find out what exactly it is. So let me go back. [00:42:38.916] Right? So and C A is equal to ten, B is equal to ten, and C [00:42:53.508] So A greater than ten and [00:42:59.876] B greater than 5. Okay. So now what am I trying to check? There are two expressions. I'm trying to check what is the uh result of this. Okay. Now in this case, A is greater than 10. No, it is false. [00:43:15.328] Okay and B is greater than 5? Yes, it is greater than 5. So now I'm getting a false here and I'm getting a true here. False and true. Please remember. [00:43:26.436] True and true

only true. Okay. And operators, we want both of them to be true or to be one, then only the entire result will be one. Otherwise, it is going to be false. Okay. So let's see to this the answer is false in this case. [00:43:45.956] Okay um hopefully this is clear to you. Fine. So let me do one more and then uh help you out. [00:44:00.516] So now I'll see our yes. [00:44:10.244] A less than five, or a greater than or equal to ten. [00:44:23.076] So A for 10 less than 5, no, it is false. Okay? It is false. Because it is an R operation, what is the R requires? Either any one of them are true, right? [00:44:36.224] 10 greater than 10. No, it is not greater than 10. Or is it equal to? Yes, it is equal to 10. So you have false or true. Answer is still true. Okay? This is how your or operator works. [00:44:49.824] Now, in case we call it as a lazy evaluation or it is also called as a short circuit evaluation. So what does this mean? It means suppose look at this uh expression here. I say a greater than 10. A is what 10.

EV0000278 · EV0000279 · EV0000280 · EV0000281

Transcript block 10 (source interval 002700-002999 seconds): [00:45:07.152] 10 greater than 10? No, it is false. So whenever you have an AND operation, I will never go and evaluate this expression. The reason is I already have one false. My AND operation always requires me to have true and true, then only my result is going to be true. [00:45:25.120] Okay, so if you have an AND operation and if the first one results in a false, I will never operate or never evaluate the second expression. [00:45:36.772] But if you have an or any one of them can be true. Okay. Suppose if the first one itself becomes a true, I will never check for the second one because by default it is always a true. So this is called as short circuit evaluation. [00:45:54.352] Okay, so hopefully uh it is uh clear to you. This is called as short circuit evaluation. Okay, so this is what it means. In general, what are true? [00:46:08.388] Any if your result if your answer is true, it is always a true. And positive integer is a true, negative integer, anything other than zero, it is a true. So please, if you are working with an integer, remember anything other than zero, it is always true. Then [00:46:25.892] Okay which are false, zero is false, empty string is false, there's something called as a nun, which I will be explaining uh in the coming uh classes. So these are all the categories that come under false. [00:46:43.076] Okay, so hopefully this is clear. [00:46:46.660] Fine. The last but one. Okay, there are one, there is one more I have to explain. Last but one. Okay. So the operators that we are talking, they behave in a different way. Okay so how do they behave? Plus behaves in a different way. [00:47:03.440] When it is used with the integers or with the numbers, it behaves in a different way when it is used with the strings. [00:47:10.656] Okay, any operators which behave differently, we call them as polymorphic in nature. Okay, so we call them as polymorphic. Let's quickly uh go and understand the ones that are polymorphic. [00:47:26.500] Okay, so let's go one plus two. What are integers? See the answer is three. Suppose I do one plus [00:47:39.940] Because it's a string. Now I'm considering it as a string. They are getting combined, concatenated. Okay, so this is what we meant. So say one star two, answer is two. Suppose if I do one. [00:47:53.904] And I do two, which means one should be repeated twice. So it is called as repetitive operator. Fine. Star behaves as an uh evaluator, it evaluates, and in this case it repeats. [00:48:08.736] Fine. I can also do something like this. Two star one. It's commentative in nature. Okay. So plus [00:48:18.180] And star they are behaving in a different way based on the different uh types that are there. Okay, there's uh one more uh which is called as an in operator, okay. So let me quickly do it on this in operator, right? [00:48:37.188] Now I'll say [00:48:41.924] C [00:48:45.252] [ASR low confidence; check audio] In catch. [00:48:50.116] Trying to check whether C is present in CAT. Is C present? Yes, it is present. True. Okay. In is called as a membership operator. In is called as a membership

operator. Please always remember this. Okay. So I will also say capital C. [00:49:09.444] In that answer is false because capital C is not a part of it. Okay so hopefully all these things are uh clear to you. Hope uh you have uh understood all these concepts. [00:49:26.180] Uh I urge you to try out on all these things. Okay. Happy learning, happy coding. Thank you.

EV0000282 · EV0000283 · EV0000284 · EV0000285

Full transcript: 6643 normalized words, 199 timestamped segments, 2973.610667 seconds duration. Low-confidence segments flagged: 1. Review locators: 00:48:45.252

EV0000246 · EV0000247 · EV0000248 · EV0000249

20 sequential, selection control

This lesson is indexed from the source video titled sequential, selection control. Its complete timestamped transcript is preserved below in segment order. The original video path and content hash are recorded in datasets/lecture-corpus.json. Transcript text is source wording; it has not been silently corrected or rewritten. A candidate slide deck is grouped here according to the current matching score; the exact path and page locators are listed in the source evidence. EV0000286 · EV0000287

Graphs and diagrams

Figure G000216. UE24CS151A PCPS Unit 1 SindhuRPai Class 15 16 Control structures · source slide page 1.

EV0000976

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

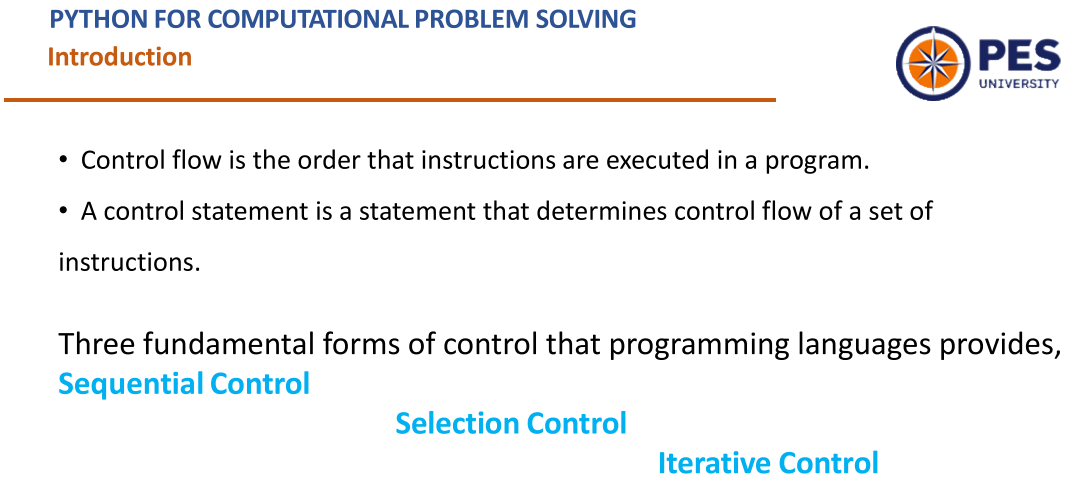


Figure G000217. UE24CS151A PCPS Unit 1 SindhuRPai Class 15 16 Control structures · source slide page 2.

EV0000979

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

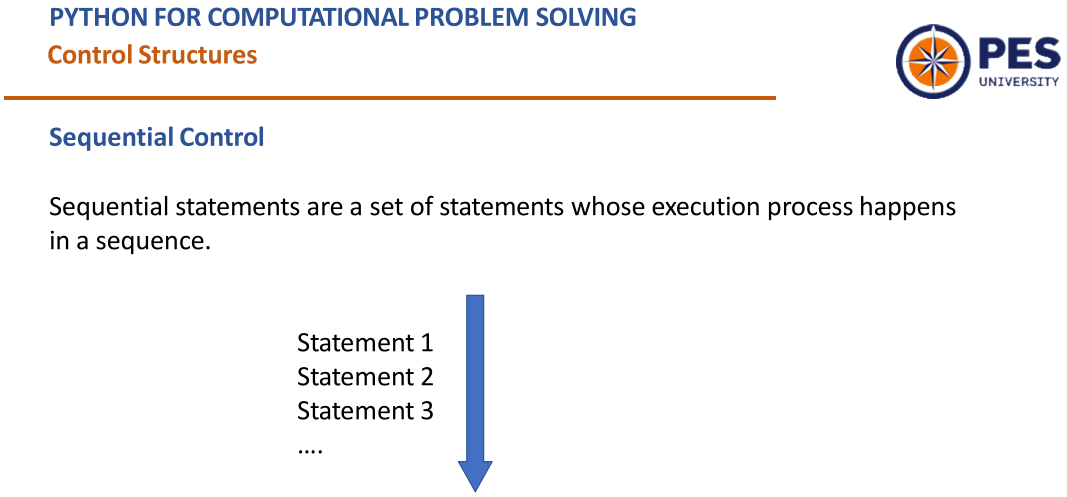


Figure G000218. UE24CS151A PCPS Unit 1 SindhuRPai Class 15 16 Control structures · source slide page 3.

EV0000982

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Control Structures

PES UNIVERSITY

Indentation in Python

- One fairly unique aspect of Python is that the amount of indentation of each program line is significant.
- In most programming languages, indentation has no affect on program logic—it is simply used to align program lines to aid readability.
- In Python, however, indentation is used to associate and group statements.

Figure G000219. UE24CS151A PCPS Unit 1 SindhuRPai Class 15 16 Control structures · source slide page 4.

EV0000985

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

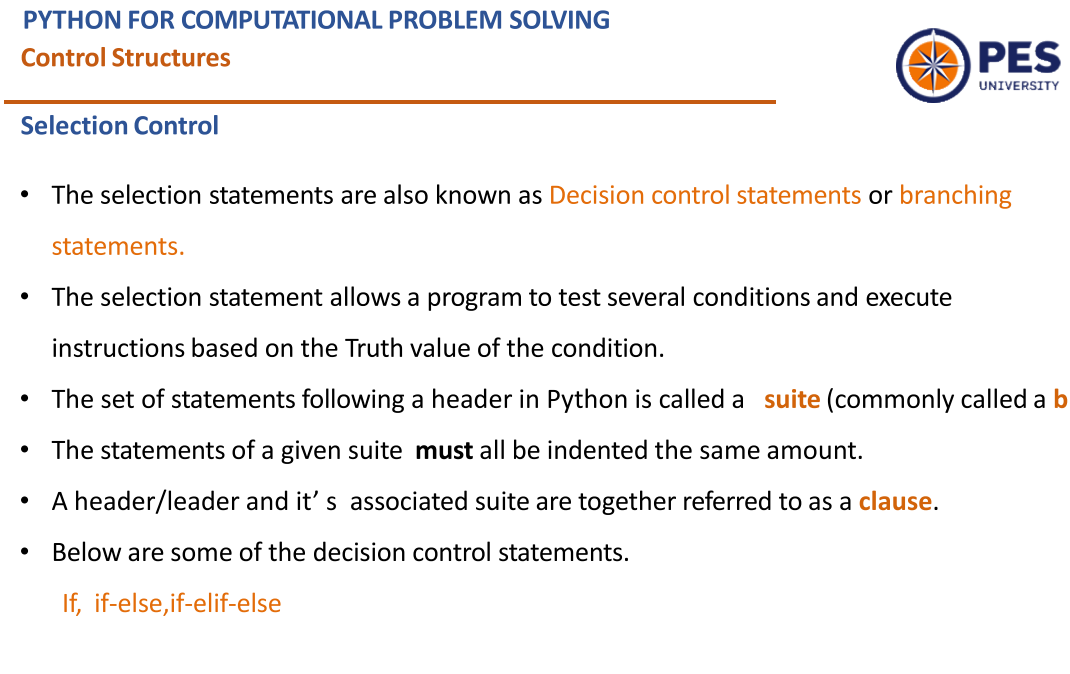


Figure G000220. UE24CS151A PCPS Unit 1 SindhuRPai Class 15 16 Control structures · source slide page 5.

EV0000988

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

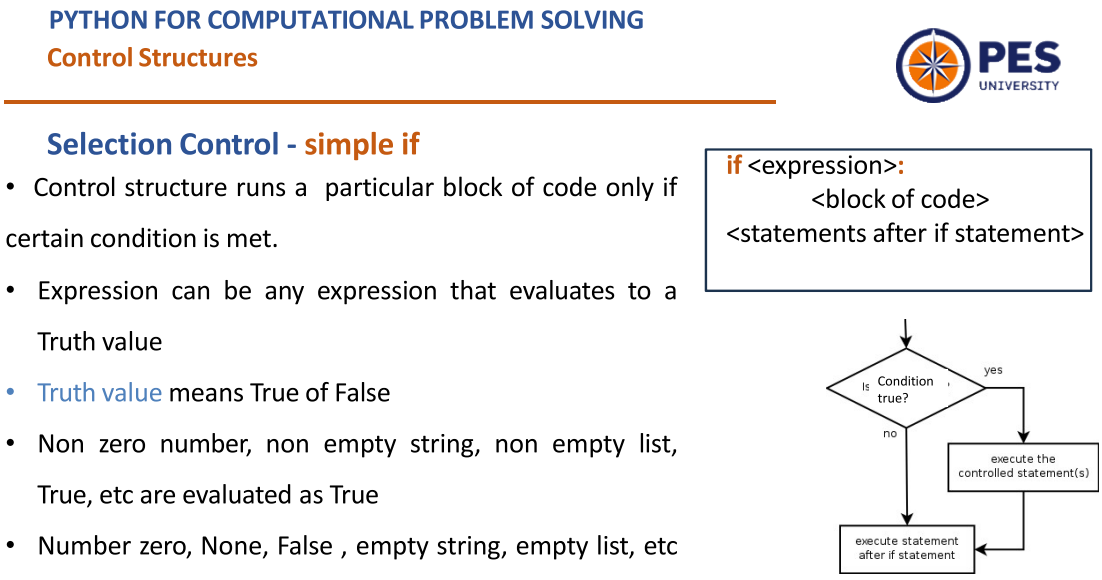


Figure G000221. UE24CS151A PCPS Unit 1 SindhuRPai Class 15 16 Control structures · source slide page 6.

EV0000991

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

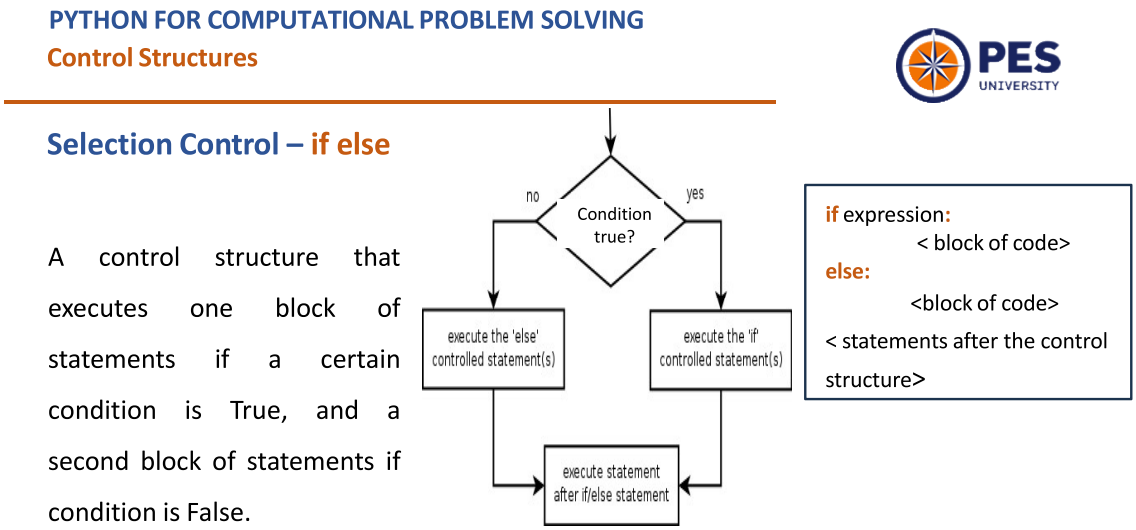


Figure G000222. UE24CS151A PCPS Unit 1 SindhuRPai Class 15 16 Control structures · source slide page 7.

EV0000994

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

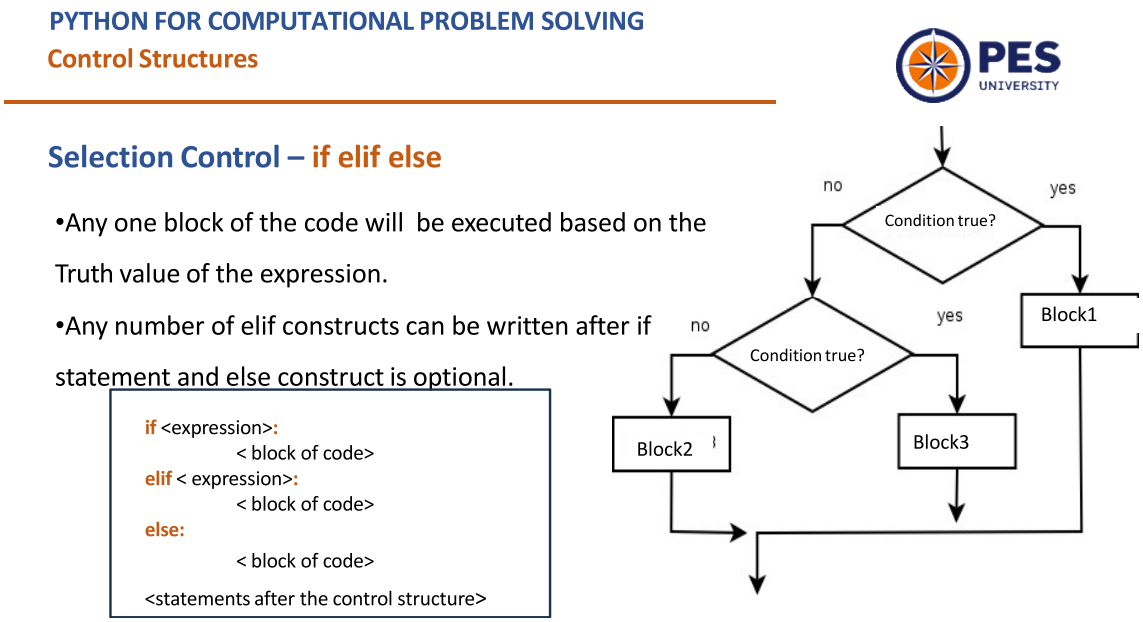


Figure G000223. UE24CS151A PCPS Unit 1 SindhuRPai Class 15 16 Control structures · source slide page 8.

EV0000997

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING
Control Structures

PES UNIVERSITY

Valid and invalid indentation

A **compound statement** in Python may consist of one or more clauses. While four spaces is commonly used for each level of indentation, any number of spaces may be used, as shown below.

Valid indentation		Invalid indentation	

(a)
``if condition:
  statement
  statement
else:
  statement
  statement```
	(b)
	``if condition:
	  statement
	  statement
	else:
	  statement
	  statement```
		(c)
		``if condition:
		  statement
		  statement
		else:
		  statement
		  statement```
			(d)
			``if condition:
			  statement
			  statement
			else:
			  statement
			  statement```

Figure G000224. UE24CS151A PCPS Unit 1 SindhuRPai Class 15 16 Control structures · source slide page 9.

EV0001000

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

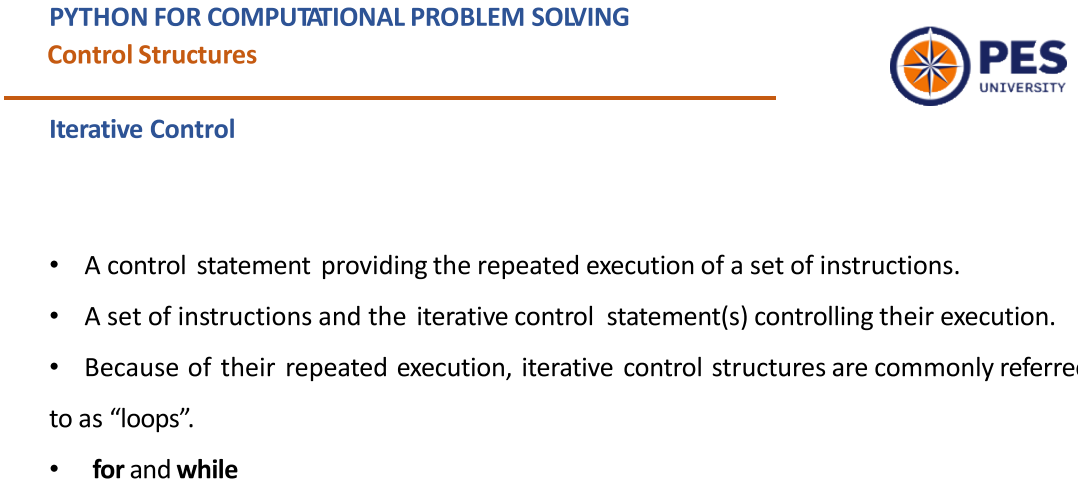


Figure G000225. UE24CS151A PCPS Unit 1 SindhuRPai Class 15 16 Control structures · source slide page 10.

EV0001003

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

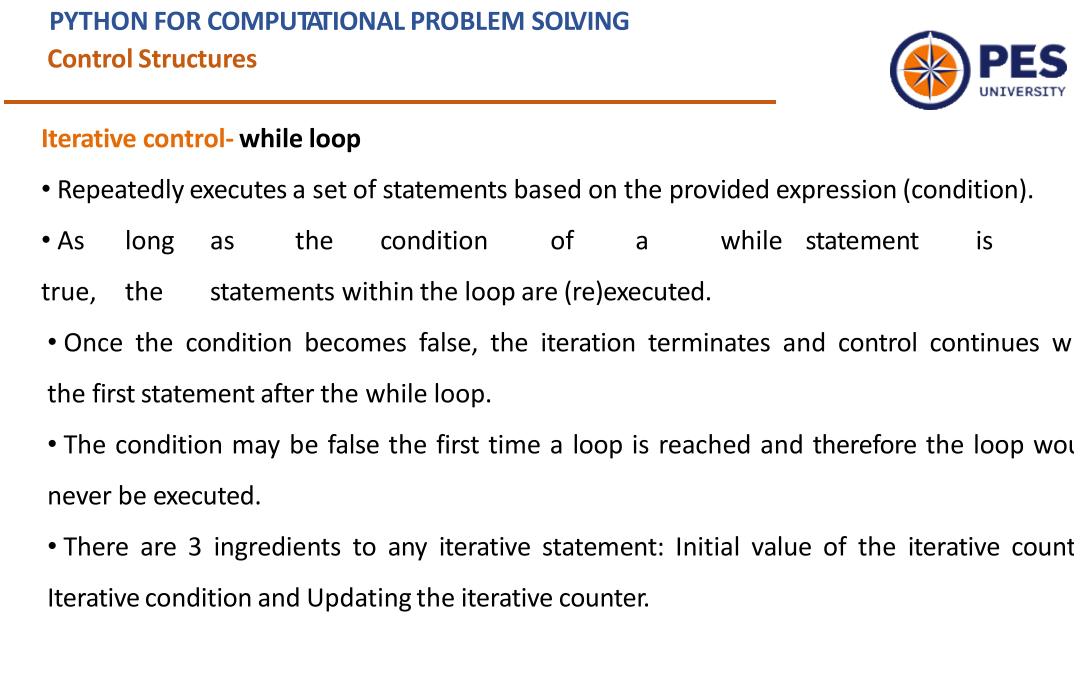


Figure G000226. UE24CS151A PCPS Unit 1 SindhuRPai Class 15 16 Control structures · source slide page 11.

EV0001006

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

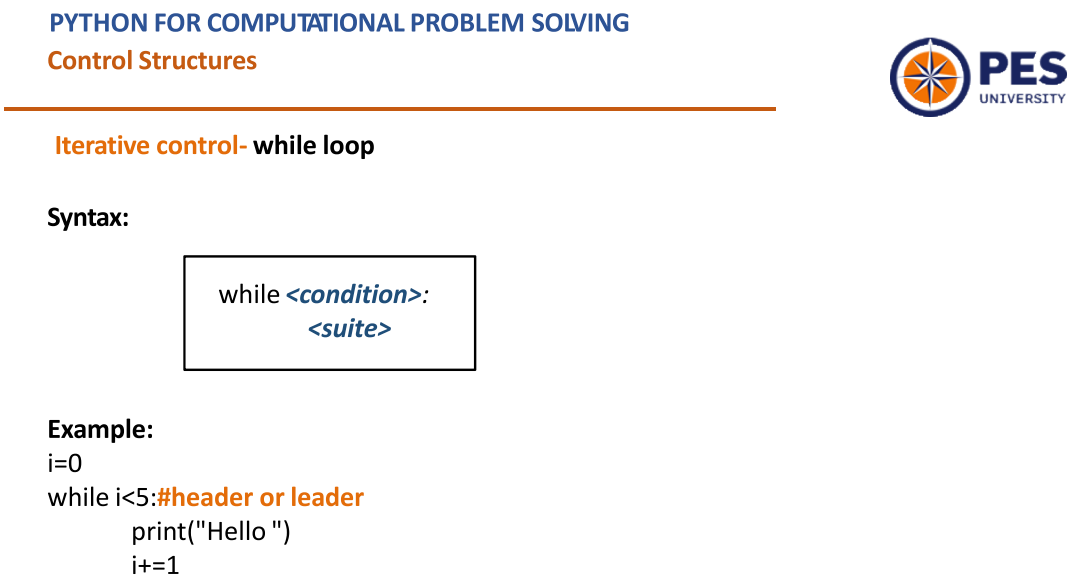


Figure G000227. UE24CS151A PCPS Unit 1 SindhuRPai Class 15 16 Control structures · source slide page 12.

EV0001009

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

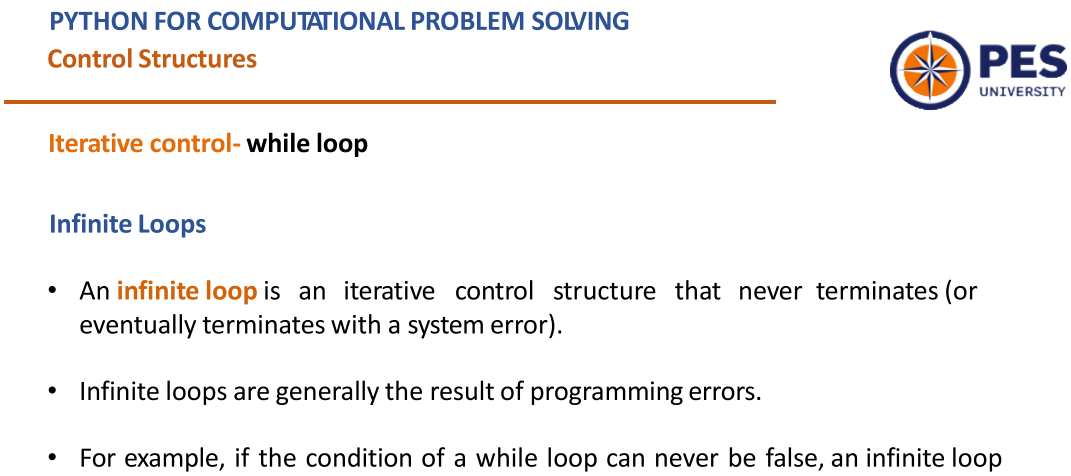


Figure G000228. UE24CS151A PCPS Unit 1 SindhuRPai Class 15 16 Control structures · source slide page 13.

EV0001012

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

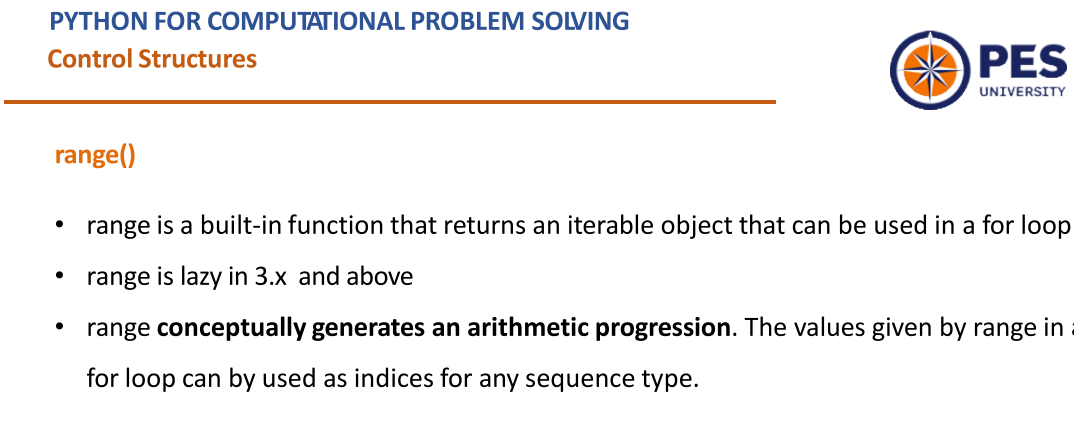


Figure G000229. UE24CS151A PCPS Unit 1 SindhuRPai Class 15 16 Control structures · source slide page 14.

EV0001015

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

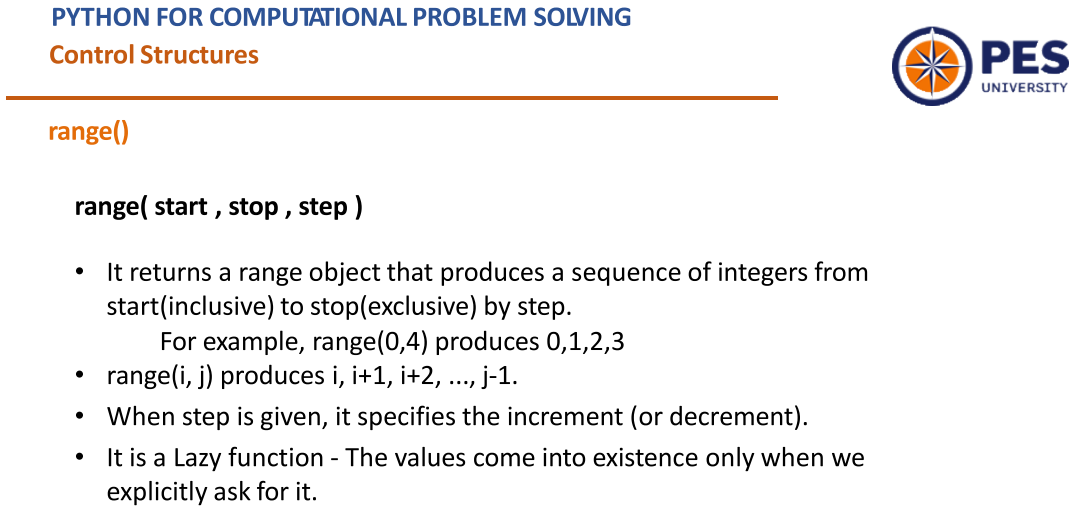


Figure G000230. UE24CS151A PCPS Unit 1 SindhuRPai Class 15 16 Control structures · source slide page 15.

EV0001018

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

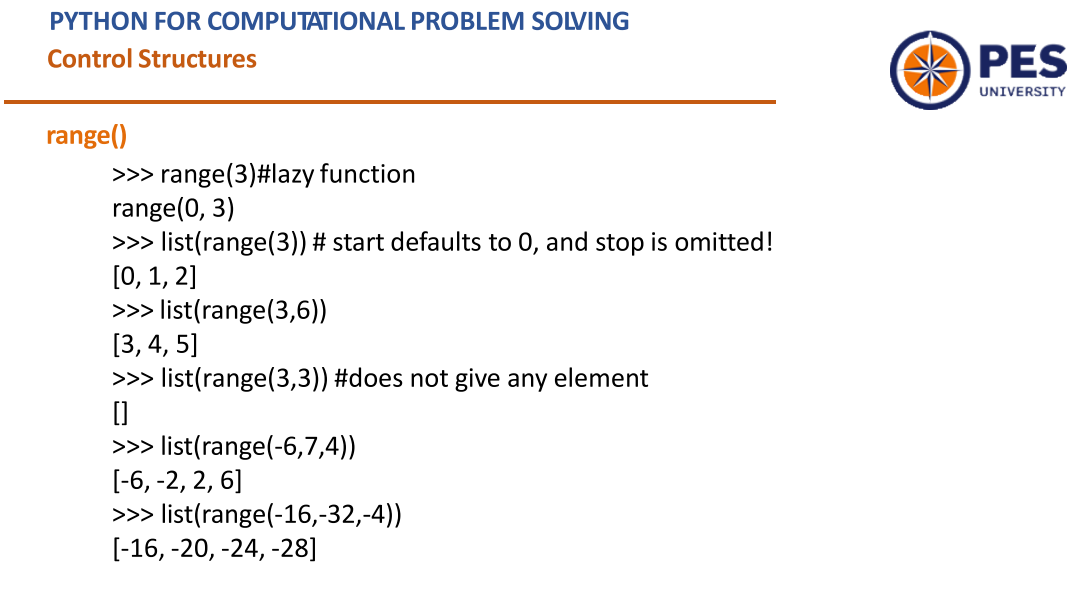


Figure G000231. UE24CS151A PCPS Unit 1 SindhuRPai Class 15 16 Control structures · source slide page 16.

EV0001021

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

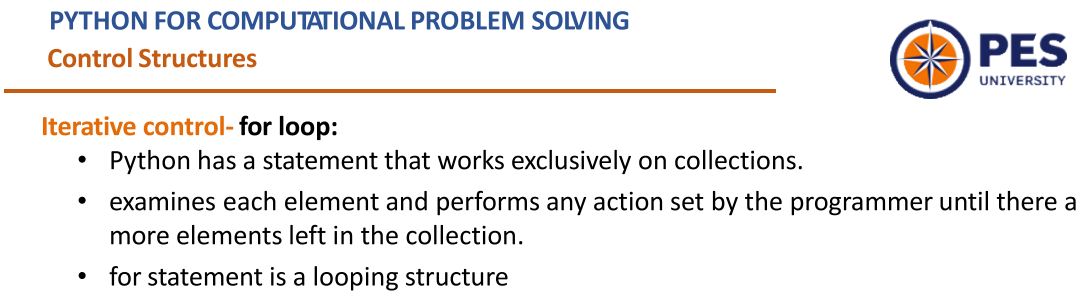


Figure G000232. UE24CS151A PCPS Unit 1 SindhuRPai Class 15 16 Control structures · source slide page 17.

EV0001024

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

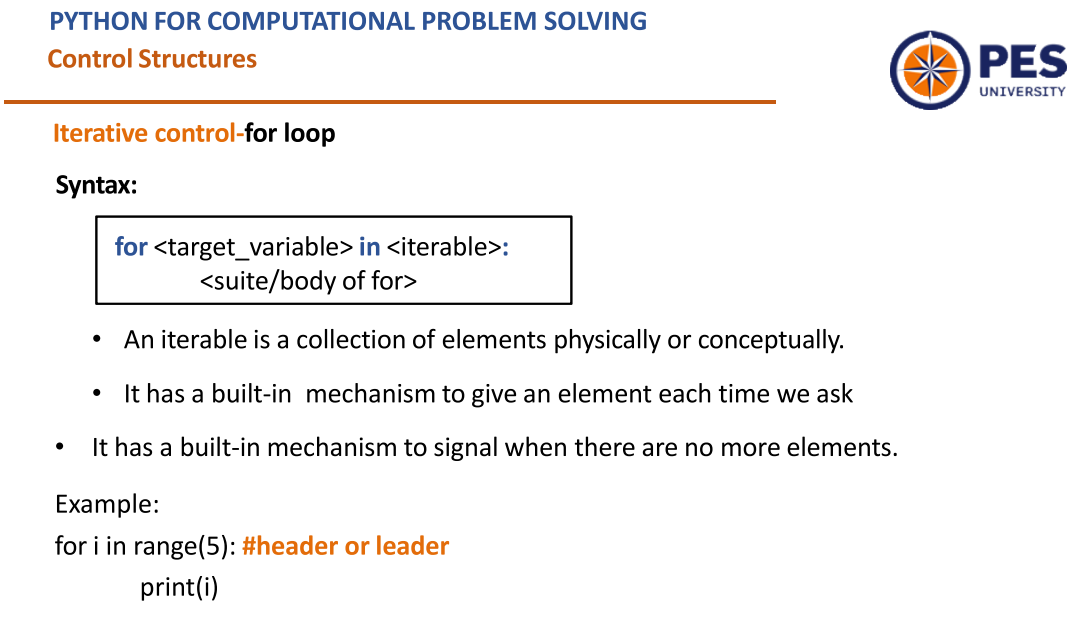


Figure G000233. UE24CS151A PCPS Unit 1 SindhuRPai Class 15 16 Control structures · source slide page 18.

EV0001027

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING
Control Structures

PES UNIVERSITY

Demo of Control Structures

Figure G000234. UE24CS151A PCPS Unit 1 SindhuRPai Class 15 16 Control structures · source slide page 19.

EV0001030

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

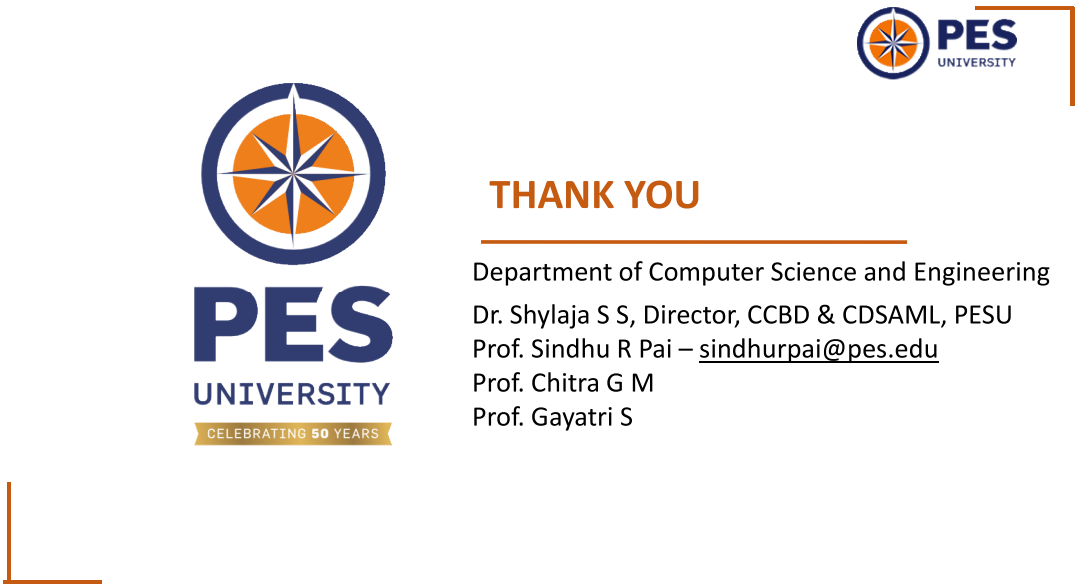


Figure G000235. UE24CS151A PCPS Unit 1 SindhuRPai Class 15 16 Control structures · source slide page 20.

EV0001033

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Important points and why

1. Source slide page 1 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Control Structures Prof. Sindhu R Pai PCPS Theory Anchor - 2024 Department of Computer Science and Engineering EV0000975 · EV0000976 · EV0000977
2. Source slide page 2 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Introduction • Control flow is the order that instructions are executed in a program. • A control statement is a statement that determines control flow of a set of instructions. Three fundamental forms of control that programming languages provides, Sequential Control Selection Control Iterative Control EV0000978 · EV0000979 · EV0000980
3. Source slide page 3 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Control Structures Sequential Control Sequential statements are a set of statements whose execution process happens in a sequence. Statement 1 Statement 2 Statement 3 .... EV0000981 · EV0000982 · EV0000983

4. Source slide page 4 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Control Structures Indentation in Python • One fairly unique aspect of Python is that the amount of indentation of each program line is significant. • In most programming languages, indentation has no affect on program logic—it is simply used to align program lines to aid readability. • In Python, however, indentation is used to associate and group statements. EV0000984 · EV0000985 · EV0000986
5. Source slide page 5 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Control Structures Selection Control • The selection statements are also known as Decision control statements or branching statements. •• TheThe selectionselection statementstatement allowsallows aa programprogram toto testtest severalseveral conditionsconditions andand executeexecute instructions based on the Truth value of the condition. • The set of statements following a header in Python is called a suite (commonly called a block). • The statements of a given suite must all be indented the same amount. • A header/leader and it' s associated suite are together referred to as a clause. • Below are some of the decision control statements. If, if-else,if-elif-else EV0000987 · EV0000988 · EV0000989
6. Source slide page 6 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Control Structures Selection Control - simple if if <expression>: • Control structure runs a particular block of code only if <block of code> <statements after if statement>certain condition is met. • ExpressionExpression cancan bebe anyany expressionexpression thatthat evaluatesevaluates toto aa Truth value • Truth value means True of False Conditiontrue? • Non zero number, non empty string, non empty list, True, etc are evaluated as True • Number zero, None, False , empty string, empty list, etc are evaluated as False. EV0000990 · EV0000991 · EV0000992
7. Source slide page 7 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Control Structures Selection Control – if else Condition if expression: true? < block of code>A control structure that else: executes one blblockk off <bl<blockk off code>d > < statements after the control statements if a certain structure> condition is True, and a second block of statements if condition is False. EV0000993 · EV0000994 · EV0000995
8. Source slide page 8 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Control Structures Selection Control – if elif else Condition true? •Any one block of the code will be executed based on the Truth value of the expression. Block1 •Any number of elif constructs can be written after if Conditiontrue? statement and else construct is optional. if <expression>: Block3 < block of code> Block2 elif < expression>: < block of code> else: < block of code> <statements after the control structure> EV0000996 · EV0000997 · EV0000998
9. Source slide page 9 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Control Structures Valid and invalid indentation A compound statement in Python may consist of one or more clauses. While four spaces is commonly used for each level of indentation, any number of spaces may be used, as shown belowbelow. EV0000999 · EV0001000 · EV0001001
10. Source slide page 10 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Control Structures Iterative Control • A control statement providing the repeated execution of a set of instructions. • A set of instructions and the iterative control statement(s) controlling their execution. • Because of their repeated execution, iterative control structures are commonly referred to as "loops". • for and while EV0001002 · EV0001003 · EV0001004

11. Source slide page 11 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Control Structures Iterative control- while loop • Repeatedly executes a set of statements based on the provided expression (condition). • As long as the condition of a while statement is true,true, thethe statementsstatements withinwithin thethe looploop areare (re)executed.(re)executed. • Once the condition becomes false, the iteration terminates and control continues with the first statement after the while loop. • The condition may be false the first time a loop is reached and therefore the loop would never be executed. • There are 3 ingredients to any iterative statement: Initial value of the iterative counter, Iterative condition and Updating the iterative counter. EV0001005 · EV0001006 · EV0001007

12. Source slide page 12 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Control Structures Iterative control- while loop Syntax: while <condition>: <suite> Example: i=0 while i<5:#header or leader print("Hello ") i+=1 EV0001008 · EV0001009 · EV0001010

13. Source slide page 13 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Control Structures Iterative control- while loop Infinite Loops • AnAn infiniteinfinite looploop isis anan iterativeiterative controlcontrol structurestructure thatthat nevernever terminatesterminates (or(or eventually terminates with a system error). • Infinite loops are generally the result of programming errors. • For example, if the condition of a while loop can never be false, an infinite loop will result when executed. EV0001011 · EV0001012 · EV0001013

14. Source slide page 14 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Control Structures range() • range is a built-in function that returns an iterable object that can be used in a for loop • range is lazy in 3.x and above • range conceptually generates an arithmetic progression. The values given by range in a for loop can by used as indices for any sequence type. EV0001014 · EV0001015 · EV0001016

15. Source slide page 15 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Control Structures range() range(start , stop , step) • It returns a range object that produces a sequence of integers from start(inclusive) to stop(exclusive) by step. For example, range(0,4) produces 0,1,2,3 • range(i, j) produces i, i+1, i+2, ..., j-1. • When step is given, it specifies the increment (or decrement). • It is a Lazy function - The values come into existence only when we explicitly ask for it. EV0001017 · EV0001018 · EV0001019

16. Source slide page 16 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Control Structures range() >>> range(3)#lazy function range(0, 3) >>> list(range(3)) # start defaults to 0, and stop is omitted! [0, 1, 2] >>> list(range(3,6)) [3, 4, 5] >>> list(range(3,3)) #does not give any element [] >>> list(range(-6,7,4)) [-6, -2, 2, 6] >>> list(range(-16,-32,-4)) [-16, -20, -24, -28] EV0001020 · EV0001021 · EV0001022

17. Source slide page 17 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Control Structures Iterative control- for loop: • Python has a statement that works exclusively on collections. • examines each element and performs any action set by the programmer until there are no more elements left in the collection. • for statement is a loopingp g structure • The number of times we execute the body or the suite is normally determinable Semantics of for : 1. Start the iteration(walking through) of the iterable. 2. Get the element to the variable. 3. Execute the suite or the body. 4. Repeat steps 2 and 3 until the iterable signals that it has no more elements. 5. Move to the next statement and exit the for loop. EV0001023 · EV0001024 · EV0001025

18. Source slide page 18 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Control Structures Iterative control-for loop Syntax: for <target_variable> in <iterable>: <suite/body of for> • An iterable is a collection of elements physically or conceptually. • It has a built-in mechanism to give an element each time we ask • It has a built-in mechanism to signal when there are no more elements. Example: for i in range(5): #header or leader print(i) EV0001026 · EV0001027 · EV0001028
19. Source slide page 19 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Control Structures DDemo off CControlt l StStructurest EV0001029 · EV0001030 · EV0001031
20. Source slide page 20 (extracted text): THANK YOU Department of Computer Science and Engineering Dr. Shylaja S S, Director, CCBD & CDSAML, PESU Prof. Sindhu R Pai – sindhurpai@pes.edu Prof. Chitra G M Prof. Gayatri S EV0001032 · EV0001033 · EV0001034

SHORT-NOTE RECAP

Transcript block 1 (source interval 000000-000299 seconds): [00:00:01.284] Hello all, uh welcome back Python for computational problem solving. In today's class, we are going to be learning the concept of control structures. [00:00:20.676] So what exactly is a control structure? Why we need control structures. All those details we will try to learn. So in order to start, let's try to understand what exactly is a control flow first. [00:00:34.308] Right. So a control flow one second, yes, a control flow is the order that the instructions are executed in a program. So it's the order. [00:00:48.228] That the instructions uh get executed in a program. So when I say control flow, I should also have a control statement. [00:01:02.404] So what exactly is a control statement? Let me remove this. Fine, a control statement. [00:01:10.500] Control statement basically uh determines the control flow of a set of instructions. Okay. So this is basically a statement uh which determines the control flow of the set of uh instructions. [00:01:24.612] Instructions. So when we speak about the controls with respect to a Python programming language, we have three different types. The first one is called as, let me say it as three types. The first one is called as sequential. [00:01:47.140] Fine. The second one the second one is called as selection. [00:01:55.748] Third one is called as i trade. So these are the three different uh types of forms, types of forms of controls that a Python uh language provides. Okay, so let me uh quickly explain it to you what exactly is a sequential control of flow. [00:02:15.312] Fine. A sequential control flow is a form of control in which the instructions are executed in the order that they are written. Okay? They follow the order one after other. [00:02:32.740] other. Till now whatever the programs that you have written, they are all your sequential program. For example, if you have uh five different statements, if I say one after um for example you have a second statement, so on. [00:02:49.456] First, the statement one gets executed and then followed by the second statement gets executed, third statement gets executed, fourth statement, and so on. Okay so it is one after the other, each and every instructive. So this is about your sequential. Now selection. [00:03:13.764] Selection control selection statement or a selection control is basically provided by a control statement that selectively executes instructions based on a condition. [00:03:25.476] Right. Let me repeat what exactly is a selection control. It is provided by a control statement. You will have a main control statement, which is basically a condition. If the condition is true, something will happen. If the condition is false, something else will happen. [00:03:41.424] Okay, so this is called as your selection control. Fine. So I will selectively execute. [00:03:54.436] effectively execute. Either this or that. And followed by the last one is iterative. [00:04:04.196] When I say iterative, what I will try to do is set of instructions are executed again

and again. I will try to execute the same set of instructions again and again based on some conditions. So this is called as iterative. This is repeatedly execution. [00:04:36.036] Repeatedly executes. So what we will try to do in today's classes, anyway, you know sequential uh way of writing the program. Today's class we will try to understand how exactly uh to work with selection statements. So let me start with a selection. [00:04:59.012] Let me push all of them here so that it is easy for us to do it. Fine. So in case of selection, what I'll try to do is among uh many alternatives, I will choose one alternative. Okay, choose one alternative.

EV0000286 · EV0000287 · EV0000288 · EV0000289

Transcript block 2 (source interval 000300-000599 seconds): [00:05:26.084] Among n number of alternatives. For example, what I'll say is if it is raining, I'll say if it is raining, I will stay at home, else I will go outside. So I said if it rains. [00:05:46.660] What do I do? What is an action that I will take? The action that I will be taking is stay back home. [00:05:57.316] Home. Right. This is what I will draw. Otherwise, what I'm going to be doing, otherwise, I will go outside and play. Okay. So now what do I do if I have to convert this particular statement in terms of uh a Python language? Okay. So this is where uh [00:06:16.224] Uh I will be using the constructs that are available in Python. Okay. So in order to do this, what I'll do is if okay. [00:06:28.228] quite difficult for me to uh convert this itself because it's an English kind of statements. Okay. So I'll say if the general syntax I would mention if condition [00:06:42.564] Okay? If condition is true, what do I do? Execute some statements. [00:06:52.324] statement or some statements. This is fine. So now in general I call this as an header and this is called as a suite. Okay so this is called as an header. Let me write [00:07:09.828] Okay, and this is called as a sweet. [00:07:14.436] Okay, or I can also call it the body of a particular header. Please remember when I'm trying to write, if you can see, I have left some space here. Okay? Why I did not start with the first column itself? [00:07:28.768] Here. So in such case, please remember I will be speaking something about an indentation. Okay in other programming languages, for example, C, you have an open flower brace and then followed by a closing flow brace to indicate that there are some set of statements which needs to be a part of [00:07:48.400] This particular if. Now Python does not make use of these flower braces. Fine. Instead, it works with the concept of indentation. Worse with the concept of indentation. [00:08:03.076] What is this indentation? Indentation is basically the spaces that you need to say leave so that you say some number of statements belong to this particular header. Okay. So now what exactly is the indentation that I am trying to speak? [00:08:19.012] An indentation that you want to follow could be a single indentation in this case. So let me comment on these things. What I will do is I can say if some condition [00:08:36.036] Okay, I'm talking about different uh uh indentation. I can just leave one space, I can leave two spaces, I can leave three spaces, four spaces, it is up to your convenience. [00:08:50.192] And let me repeat, it is up to your convenience, you can make use of the spaces in order to say that there are n number of statements that needs to be executed if this condition is true. [00:09:02.400] Okay, so now what I will try to do is in general my way of writing is I will always use a tab in order to specify that this is my indentation. Okay so now I'll say statement one. [00:09:18.692] Okay, I'll say give one more tab to say that statement two and so on. Please remember my way of writing indentation is basically working with respect to the tab. Fine. [00:09:33.984] This is clear now. Hope this is clear. You can use one space, two space, three space, four space, any number of spaces. But make sure if you have n number of statements which falls under a particular header, then all of them should have same level of indentation.

[00:09:51.728] It should have the same level of indentation. You cannot have different indentations for a single header, which means

EV0000290 · EV0000291 · EV0000292 · EV0000293

Transcript block 3 (source interval 000600-000899 seconds): [00:10:02.272] In this case, I have left one tap space. If I leave two uh tap spaces here, it through me an error. So I should have a same indentation level for a particular header. [00:10:16.880] Please you have to follow this particular aspect when you're working with the selection with respect to Python. Okay? In general, whenever the indentation needs to be followed, please make sure that you follow same level of indentation for a particular header. [00:10:34.500] Fine. Suppose I have one more header. Second header I have. Okay. So in this case, I can just have one space. It's absolutely fine. Again, if I have, I'll just write it as statement one. And if I have one more statement, I can say statement two. Fine. And I can have uh [00:10:53.568] Uh a single space here because under a particular header I should have a same level of indentation. So you should always remember this. Okay. Uh what is important when you work with selection or with an iterative process? It's basically the indentation. [00:11:11.312] Okay, so hope this is clear to you. Fine. So let me uh remove this. There are uh different um uh if instructs are available. First one is if condition [00:11:30.212] I'll just say it as suit. Body. Okay. The next one, the next way of writing, if condition. [00:11:48.484] Okay. If condition or an expression. Okay. One more important thing I have to say. If you come across with an header, it should end with a colon. This is very important. It should end with a colon. Do not forget to do end with a colon. Okay. So now followed by [00:12:06.180] I can say suits sweep. Sorry. Fine. If expression, do this. Otherwise, how do I mention otherwise? I'll write it as an else. And again, else is also your header. And it with [00:12:26.308] Colon followed by a suit. This is your uh uh second way of writing. Okay, this is first way based on your uh uh [00:12:36.708] program that you are uh trying to solve you have to choose the required construct. Either you want to go with a simple if or you want to go with uh if else, or there is also one more which is called as if condition or an expression. [00:12:55.428] Colon. This is the sweet. Okay. [00:13:03.492] Suppose if I have more condition, more than one condition to be checked. So I'll say else if. Okay? Else if. Please remember there is no else if in Python. Instead, it is el if. [00:13:17.764] Okay, instead it is your elif. Okay. When you have an elif, again you need to uh mention the expression that is required. Again ended with column, then followed by a sweet here. Please remember. [00:13:34.628] Under a particular header, follow the same level of indentation. Fine. After checking all the conditions, you may have uh I will end it with an else statement. So when I have an else statement, I don't need to uh check anything, so I will not have an expression, instead it's going to be your suit. [00:13:55.300] Sorry. And I can have any number of uh el if statements ended with an else statement. And hope uh uh the different uh levels of uh uh or different ways of using an if statement syntax is known to you. [00:14:13.156] So I'll just take up a simple example in order to uh explain it to you. Fine. Very simple example so that it is easy for you to understand. I'll say a is equal to 2. Sorry. I'll say if A [00:14:35.684] A is equal to two. I'll follow one level of indentation which is tab in this case. I'll say print correct. [00:14:48.036] This is what I'll try to do. A simple if. Now we have given A is equal to two. I'm checking two is equal to two. Condition is true. What is the output that I have to get? Correct is the output. So let me execute this program.

EV0000294 · EV0000295 · EV0000296 · EV0000297

Transcript block 4 (source interval 000900-001199 seconds): [00:15:01.956] And if you can see I am getting an output as correct. Suppose if I mention it as 3, what will happen? 3 is equal to 2,

condition is not true. So I will never get into this particular statement. So what gets printed, we will see. [00:15:22.180] It does not print anything because the condition itself is false. If the condition is false, what do I need to do? I am not taking any action. So now in this case, I will say my action is this. Print wrong. [00:15:43.492] Okay, so let me execute. It says wrong. Suppose if I change A is equal to two, what will happen? [00:15:51.908] condition is true, I print two here. Okay? So this is an simple example for you to uh uh understand uh the concept of uh if statements. [00:16:05.700] Fine. So maybe I will take up one more example and uh help you out uh with elif statements. Okay fine. So let me quickly take up one more example. [00:16:25.188] So let me uh find out given a number, for example a is equal to 10, I want to find out whether it is positive, negative, or zero. Okay. So find positive. [00:16:43.172] Negative. [00:16:46.468] Or zero. [00:16:51.300] I will do it. And I'll say if the number A [00:16:57.924] is greater than zero. Okay? I've already uh given the number. Or what you can do is you can allow the user to input a number, which I will change it after this. Okay? I'll say if a number is greater than zero, what it is positive. [00:17:18.404] [ASR low confidence; check audio] L-E-L-SIF. [00:17:24.196] A is less than zero. What it is? Print negative. [00:17:37.476] I have done with positive, I've done with the negative. There are no more conditions that I have to check. So I'll say else. Print. [00:17:50.404] What I will do. Fine. I want to modify this particular statement where I will allow the user to input a value. Fine. Hope you know how to input the values, which is already been uh explained in the previous videos. Please go and check it out. Fine. Input, enter. [00:18:10.244] A number. [00:18:15.044] What I will try to do. So let me execute this program. [00:18:23.940] Okay. So I'm asking the user to input a number. Let me give 10. So what is 10? It checks for the condition. What is 10 here? Positive. So I get an answer called positive. Let me run my program again. I'll give minus 10. [00:18:42.532] Answer is negative. I'll enter a value called a zero. And it has to print me zero. So this is how uh you're going to be using the required if constructs up uh based on your program. [00:19:00.484] Hope you have understood. There are many more things which you're going to be we will be discussing in your regular classes. Thank you.

EV0000298 · EV0000299 · EV0000300 · EV0000301

Full transcript: 2392 normalized words, 75 timestamped segments, 1149.845333 seconds duration. Low-confidence segments flagged: 1. Review locators: 00:17:18.404

EV0000286 · EV0000287 · EV0000288 · EV0000289

21 iterative control structure

This lesson is indexed from the source video titled iterative control structure. Its complete timestamped transcript is preserved below in segment order. The original video path and content hash are recorded in datasets/lecture-corpus.json. Transcript text is source wording; it has not been silently corrected or rewritten. No slide deck is assigned to this video; candidate slide sources remain separate in the unit master. EV0000302 · EV0000303

SHORT-NOTE RECAP

Transcript block 1 (source interval 000000-000299 seconds): [00:00:00.516] Hello everyone and welcome back to the lecture series on Python for computational problem solving. I am Nita and

Jacob from the Department of Computer Science and Engineering, and in this video we will continue our discussion on control structures. [00:00:18.436] So, in the previous video, you have seen what control structures are, and you have explored the selection control statement. In this video, what we will look at is the iterative control statement. So, what do you understand by the term iterative? [00:00:38.532] to iterate something is to go over it multiple times. So it is a control statement that provides the repeated execution of a set of instructions. [00:00:57.412] So your control statement, along with the set of instruction, controlling it is what we call an iterative control structure. Now because of the repeated execution, we give it the name loops. [00:01:12.676] So we'll either call it an iterative process or a looping process or a looping statement. So why exactly would we require something like an iterative process? Let's look at an example of us wanting to go out and it's raining outside. [00:01:32.836] So we've made up our mind to go out for to meet our friends, a movie, or so on. And the condition being that it shouldn't be raining outside. Now we're eager to go out, so what will we do? [00:01:48.196] every ten, fifteen minutes, we'd probably look out the window to check if it's still raining. And given the current situation of Covid, we are all so eager to get out. And what do we do? We keep checking the news to see if there's a slight glimmer of hope that the case cases are reducing. [00:02:07.588] So what we're doing is repeatedly performing the same task of checking whether it stopped raining or checking if the number of cases have come down or if they found a vaccine. So I could write a program where I keep saying [00:02:24.900] using a selection statement saying check if it is raining. If it's true, stay indoors. Again, write the if condition and so on and so forth. Now that program would be extremely lengthy. So what I would require is an iterative control. [00:02:43.172] So what we will look at is a specific statement called the while statement. So how it works is you have statements which are in your sequential control, and when we encounter the while statement, it works on the principle of a Boolean condition. [00:03:02.116] So anything with respect to Boolean results in true or false. So as long as the condition is true, I will execute a set of statements that come under the while statement. So once it executes all the statements within the while loop, [00:03:20.228] It goes back and checks the condition again. If it's true again, these statements are executed once, returns back to the condition. Now this repeated process continues as long as the Boolean condition remains true. [00:03:39.524] So as soon as the condition is false, or in our case we found a vaccine, or the number of cases are reducing, then we can stop the process of waiting and actually exit our homes. Or in this case, exit the while. [00:03:57.252] So let me quickly show you this with an example. So I've created a program called while pgm.py. Now the syntax for the while loop is to use the keyword while followed by a space. [00:04:17.092] the Boolean condition that I need to check [00:04:25.828] for it to be true. And as shown, this would form the leader of my program, or the structure. So the leader is Y followed by the condition. Now a suite has to be indented [00:04:42.436] a little bit and I can say print or any set of statements. It could be any set of valid statements. [00:04:57.348] This is the syntax of the while loop. You need a leader and you need a suite. The suite is a collection of statements. So these valid statements are going to be executed repeatedly until the Boolean condition becomes false.

EV0000302 · EV0000303 · EV0000304 · EV0000305

Transcript block 2 (source interval 000300-000599 seconds): [00:05:16.740] So what I'm going to do is now write the program to emulate our eager wait for the corona vaccine to be discovered and ready for it to be administered after all the human trials. So I'm going to say [00:05:34.052] I have to first define the Boolean condition. So I'm going to say vaccine is equal to [00:05:45.604] Saying

that okay, no vaccine. So there is no vaccine, which is the truth right now. I'm going to use the Boolean value true and see. Why we do not have a vaccine [00:06:02.884] I am going to have to stay under lockdown. So I'm going to say print, continue, self-isolation. Let's look at that. [00:06:19.364] Continuous self-isolation. So I have a valid statement. Okay? So I have my Y keyword, I have a Boolean condition. So as long as no vaccine is true, it will work. Say continuous self isolation. And then [00:06:35.204] As soon as this becomes false, it's going to stop executing these programs and come out and say resume normal [00:06:49.316] Life or life as we once knew. Okay. So I'm going to save this program and I'm going to run it. So in order to run it, we have to navigate to the folder where we've saved it. Type the keyword Python and I've saved it as while program.py. [00:07:08.612] And when I hit enter, you can see that it continues to print this. Right? And for Windows, if I have to stop, I have to use Ctrl C. [00:07:22.084] Now why exactly did it keep saying continue self-isolation and never presume normal life? Well, although it does look like our situation currently, we need to define when this loop needs to stop, right? Which brings me to the fact that [00:07:41.540] A while statement repeatedly executes it based on a provided boolean expression, right? So [00:07:49.732] In order for us to write it successfully and for our program to end without me having to externally stop it using Ctrl D in Windows, there are three key ingredients. [00:08:02.500] First, you have the initial value of the iterative counter or the iterative condition. So in our case, the iterative condition is the no vaccine. [00:08:16.420] Okay. [00:08:19.300] Next, I have to check whether the condition is true or not. So this is what we would call a final condition. [00:08:32.356] So as long as this is true, I keep working. And the final condition is when it becomes false, I have to stop iterating through the loop. And the third one. [00:08:45.348] Is updating the iterative counter or the condition in order for us to reach the final condition. So this right here we can say is the initial condition, this is the final condition, and we need to specify a way in which it has to stop. [00:09:05.860] So that would be our what you call it the uh loop updation statement. [00:09:16.996] Now, since this is what is missing in the program, we got an infinite execution of the loop. So let's look at how exactly we can stop it here. So right now I'm going to say no vaccine is true. And I'm going to say that. [00:09:36.228] I will check this five times, right? So five times is the limit that I'm going to check just for the sake of programming. So I'm going to say that check is a variable. [00:09:55.332] That will be updated plus equal to by one. So which means I have to first initially say what the value of check is. So I'm defining another.

EV0000306 · EV0000307 · EV0000308 · EV0000309

Transcript block 3 (source interval 000600-000899 seconds): [00:10:12.644] Initial condition. So check is initially is equal to one. And I'm going to say as long as there is no vaccine and check is less than or equal to five, print continue self-isolation. [00:10:33.444] Then when you've checked it, if the value of check would increase, it becomes two, goes back, no vaccine is still true, and check is five. [00:10:45.892] And it continues, right? So let's see what would happen if I would now execute the program. I'm going to clear the screen and run the program again. Now you can see it's checked it five times and then says it exits from [00:11:05.156] the normal cycle, which is almost like what we had to do because of the economic crisis the country was facing, right? So we kept checking to see if there was a vaccine. There was no vaccine, and then we're like, okay, let's stop the self isolation, try and resume life again. But we saw what [00:11:24.452] Happened, the number of cases increased, and we had to go back into lockdown. So this is just a simple example, right? To help us understand. So here let me just update it. I have an initial condition that would check for

the vaccine. [00:11:45.316] five times. Okay, so this is now no longer my initial condition. Check is my initial condition and my final condition. Could have been either vaccine or check. I'm using both in this case. Continue self-isolation and update the value. [00:12:02.052] So we saw if I didn't update a value, it would go infinitely, right? So this is just one example. Let's look at other features. So let's quickly iterate while the condition is true, execute the suite. [00:12:20.356] Now I want to look at an example where I can count or find the sum of numbers from 1 to n. So I'm going to have a counter as 1, which is going to basically count the numbers from 1, 2, 3, 4, and 5. Let's stop at 5 for ease of [00:12:39.812] tracing or checking what happens in each group. I want the sum of all numbers from 1 to 5. So my sum will start, I have a variable sum starting at zero. And here, which will be the initial condition here? It would be counter. [00:12:57.156] because I'm going to count the numbers and stop adding the numbers when I reach a particular value. So while count is less than n? So we will write the program where we define what n would be by taking user input. [00:13:11.588] Let's now assume n is 5. So as long as count is lesser than equal to n, which is 5 in our case, sum is equal to sum plus counter. So initially sum is 0 and counter is 1. So I'm going to be 0 plus 1. [00:13:31.172] 1 is going to be stored in sum. Now we know that unless I have a loop updation value, this loop would work infinitely. So I'm going to add counter is equal to counter plus one. I'm incrementing it by one. So now the value of counter is two and sum is one. Goes back to count. [00:13:53.476] Okay, I have made an error here. It should have been counter. Okay, now let's just go back to the program itself and see how we can do this. Okay, I'm going to comment this out and write the program to find the sum of numbers. [00:14:13.572] from one to A. Right? So let's have the initial condition of counter is equal to one. Right, so this is our initial condition [00:14:33.892] Next, I want to calculate the sum of numbers. So sum is equal to zero. And how do we write our y statement? Y followed by as long as the counter is less than or equal to n, I am going to [00:14:53.892] add sum is equal to the previous value of sum plus counter. And you see that sum changes color. We will look at it in module four when we look at some of the inbuilt functions. But right now just know that I'm just using it as a variable.

EV0000310 · EV0000311 · EV0000312 · EV0000313

Transcript block 4 (source interval 000900-001199 seconds): [00:15:13.892] And I have the loop uptation condition where counter is equal to counter plus one. I could have written it using the assignment operator like this as well. [00:15:28.932] I'm just writing this for ease of understanding. So this is my updation. Okay, this is my final condition. [00:15:45.636] Something is missing. I've said till N. So what is N? I'm going to give a user input. Input. Enter [00:16:00.036] [ASR low confidence; check audio] value of [00:16:06.980] It has to be a number, which means I have to convert it to integer, since input always returns a string. So we have a program here, counter is one, sum is zero, counter as long as it's less than or equal to the value entered. [00:16:24.036] keep adding it to sum and just increment the counter so that our program ends. Now this program doesn't display anything so I am going to write the sum [00:16:37.796] of numbers from one to N [00:16:56.804] So let's go back to the execution here. [00:17:02.692] Alright, let's run this program. It's going to say enter the value of n, going to enter 5, and the sum of numbers from 1 to 15. 1 to 5 is 15. So let's see what are the values of sum and counter in each iteration of the loop. Okay. So I'm going to print. [00:17:22.980] The value of sum. [00:17:29.188] is going to be displayed with this print statement. And I'm going to do the same to see what is the value of counter. [00:17:44.004] Let's see how the output would look. Now the reason I've done this is what we call program tracing because I want to see what is happening in each iteration of the loop. So [00:18:00.292] Let's just

write Python my program and okay just to make it a little readable I'm going to add a new line character and a couple [00:18:18.468] lines to just differentiate each iteration of the loop. Alright? So when I write run this program I'm gonna say five. Let's look at how it works. Okay? So [00:18:31.812] The first time the value of sum is one because I've calculated it and then I'm printing sum was initially 0 here, right? And then after calculating, sum becomes 1, the counter is incremented to 2. So is 2 less than 5 or equal to 5? No. So 2 and [00:18:51.940] the value of previous value of sum is going to be added since the right hand side is first executed. So sum becomes three, the counter is also three. Since it's not less, it's it is less than but not equal, not greater than five. [00:19:10.532] It's going to go here. So 3 plus 3 will give you 6. Counter becomes 4. 4 is still less than or equal to 5. [00:19:19.492] So it goes back 6 plus 4 gives you 10, counter becomes 5. So the condition says less than or equal to. So now when the condition, the counter value is 5, the condition is still true. So 5 plus 10 is added, it's displayed. [00:19:38.116] And then counter becomes six. Now you can see after that the loop does not execute again. So the sum of numbers from 1 to 5 is displayed, which is outside the while loop. So this is just a simple example of how the while loop would work. [00:19:57.220] So as long as it's true, it executes. And once the condition becomes false, the iteration terminates. Now what if the condition is false the first time the loop is reached? Then the loop would never be executed.

EV0000314 · EV0000315 · EV0000316 · EV0000317

Transcript block 5 (source interval 001200-001499 seconds): [00:20:15.076] So I'm going to modify this to say counter less than or equal to zero. Okay. So we know that one is not less than or equal to zero. I've just updated this condition here. [00:20:33.412] So let's look at what the value of the program would be. Okay, I'm going to clear the screen and run the program. Entering the value of n. So no matter what value I enter, it says sum of numbers from 1 to 5 is 0. Why is that? Because [00:20:53.380] the condition initially itself is false. So these set of statements are never executed. Just like how we saw in the flow chart, only if the condition is true will the statements in the while loop be executed, else it will never be executed. [00:21:12.900] So quick recap. If we don't have all the three ingredients, especially the loop updation statement, we reach an infinite loop where the control structure never terminates. [00:21:25.604] And infinite loops are generally the result of programming errors, not because of uh interpreter error or a computer error. It's because, as a programmer, we would have forgotten to write the loop updation statement correctly. So if the condition of a while loop can never be false, [00:21:45.028] An infinite loop will result when executed. I hope you have understood the concept of loops and we will look at it extensively in the course of the program of this subject. [00:22:00.292] When we learn other concepts as well. So you can see that we will continue incrementally building it and we will solve some examples where we combine both iterative and selective control statements as well. Thank you.

EV0000318 · EV0000319 · EV0000320 · EV0000321

Full transcript: 2873 normalized words, 81 timestamped segments, 1339.605333 seconds duration. Low-confidence segments flagged: 1. Review locators: 00:16:00.036

EV0000302 · EV0000303 · EV0000304 · EV0000305

22 String Operations

This lesson is indexed from the source video titled String Operations. Its complete timestamped transcript is preserved below in segment order. The original video path and content hash are recorded in datasets/lecture-corpus.json. Transcript text is source wording; it has not been silently corrected or rewritten. A candidate slide deck is grouped here according to the current matching score; the exact path and page locators are listed in the source evidence. EV0000322 · EV0000323

Graphs and diagrams

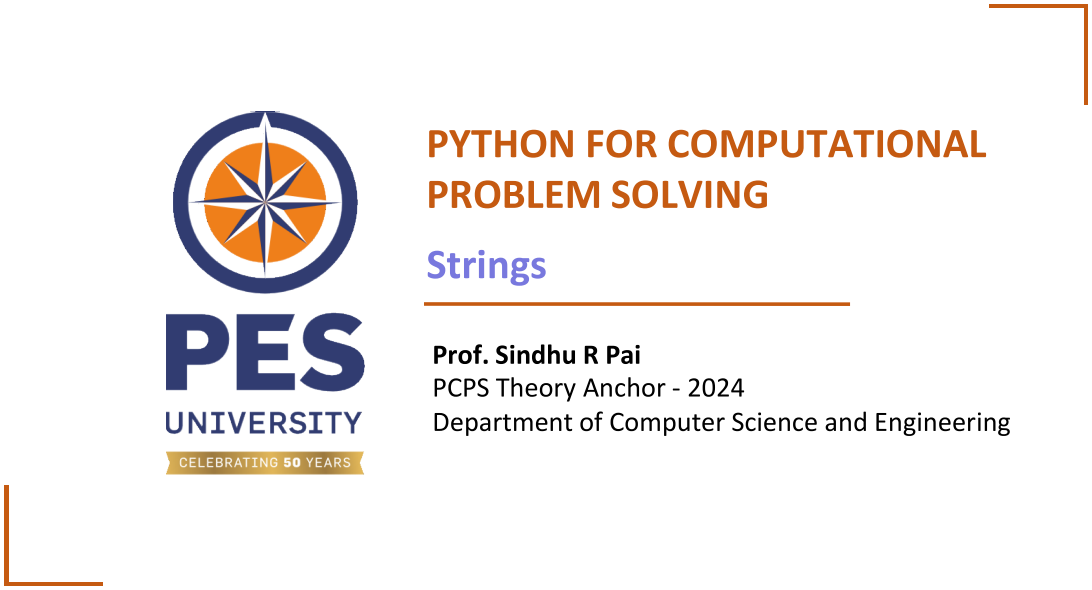


Figure G000243. UE24CS151A PCPS Unit 2 SindhuRPai Class 9 10 Strings types operations · source slide page 1.

EV0001057

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

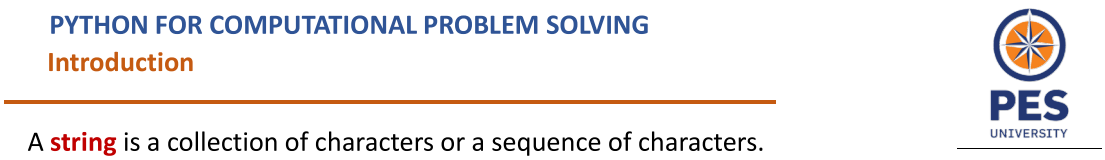


Figure G000244. UE24CS151A PCPS Unit 2 SindhuRPai Class 9 10 Strings types operations · source slide page 2.

EV0001060

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

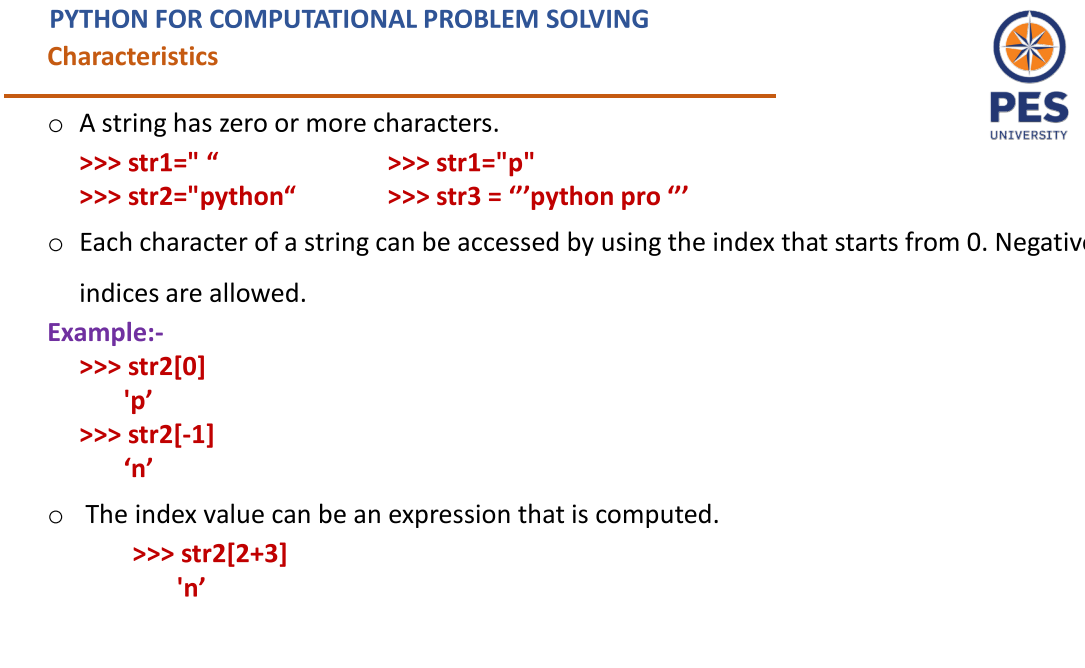


Figure G000245. UE24CS151A PCPS Unit 2 SindhuRPai Class 9 10 Strings types operations · source slide page 3.

EV0001063

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

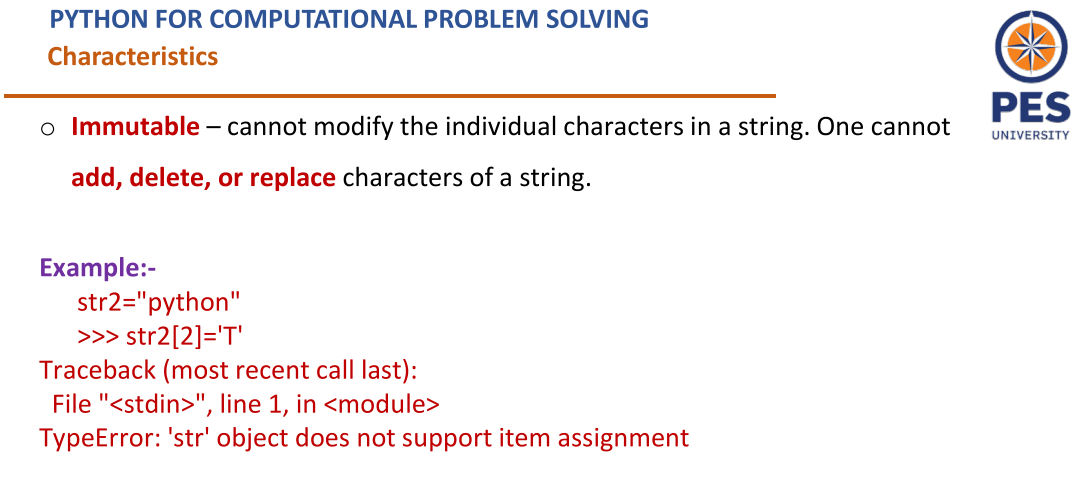


Figure G000246. UE24CS151A PCPS Unit 2 SindhuRPai Class 9 10 Strings types operations · source slide page 4.

EV0001066

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

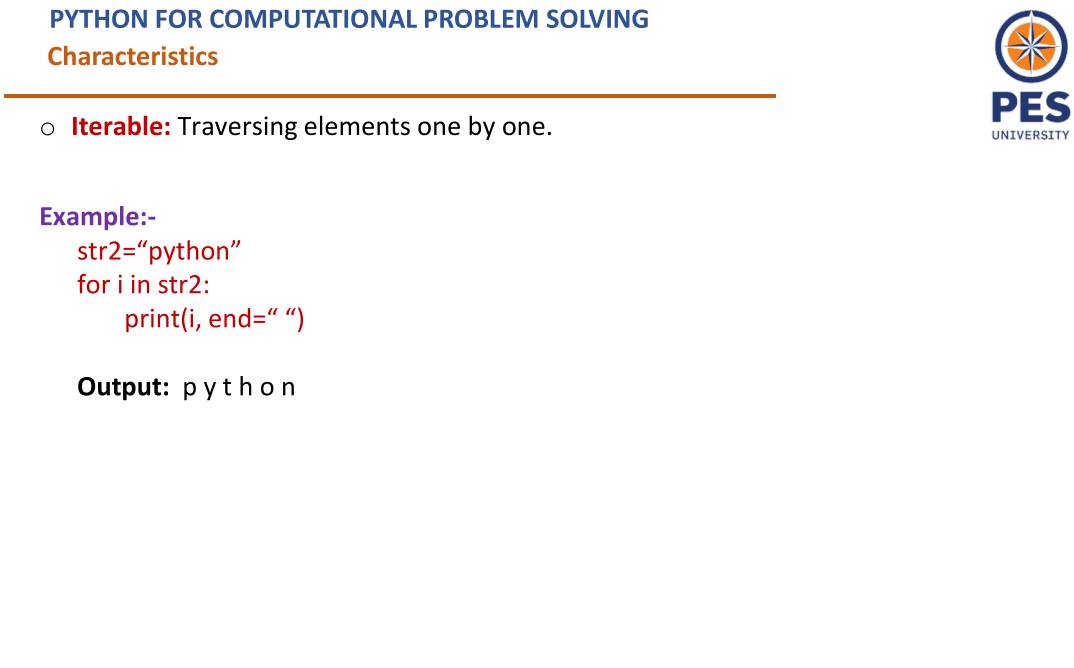
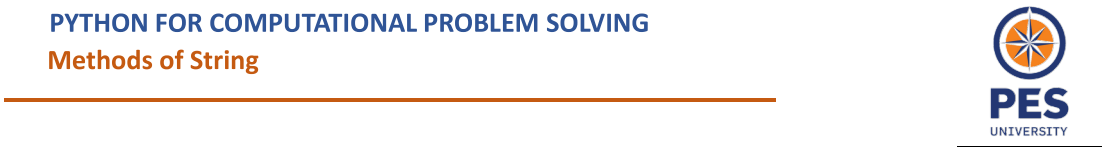


Figure G000247. UE24CS151A PCPS Unit 2 SindhuRPai Class 9 10 Strings types operations · source slide page 5.

EV0001069

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.



All string operations that “modify” a string return a new string that is a modified version of the original string.

Figure G000248. UE24CS151A PCPS Unit 2 SindhuRPai Class 9 10 Strings types operations · source slide page 6.

EV0001072

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

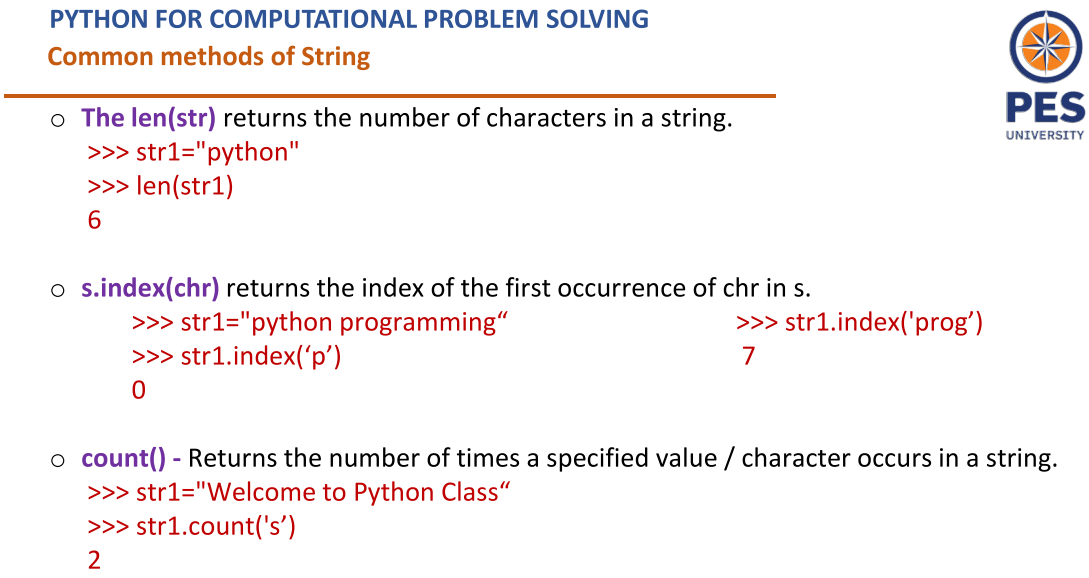


Figure G000249. UE24CS151A PCPS Unit 2 SindhuRPai Class 9 10 Strings types operations · source slide page 7.

EV0001075

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

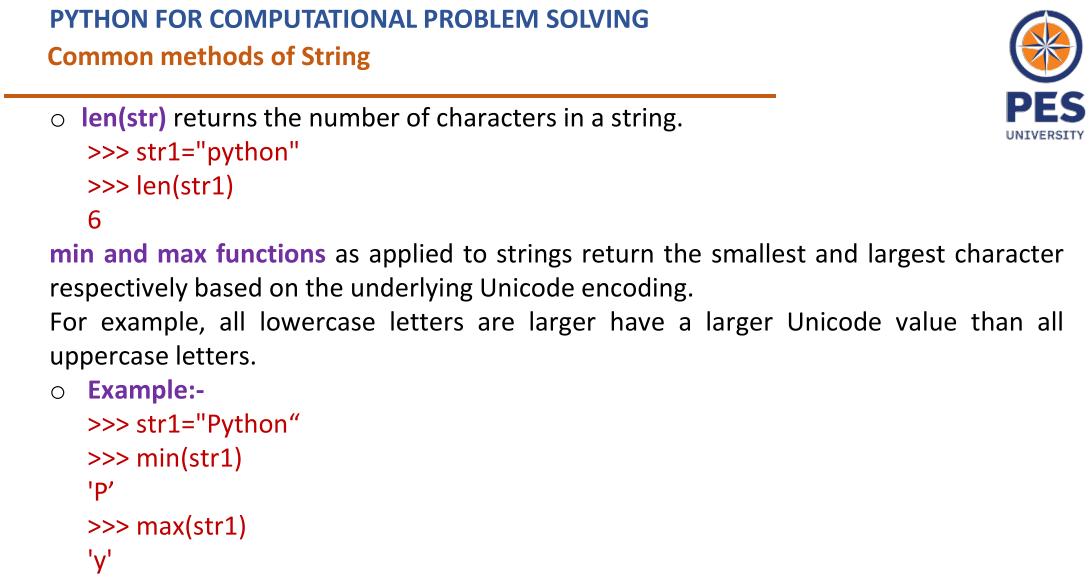


Figure G000250. UE24CS151A PCPS Unit 2 SindhuRPai Class 9 10 Strings types operations · source slide page 8.

EV0001078

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

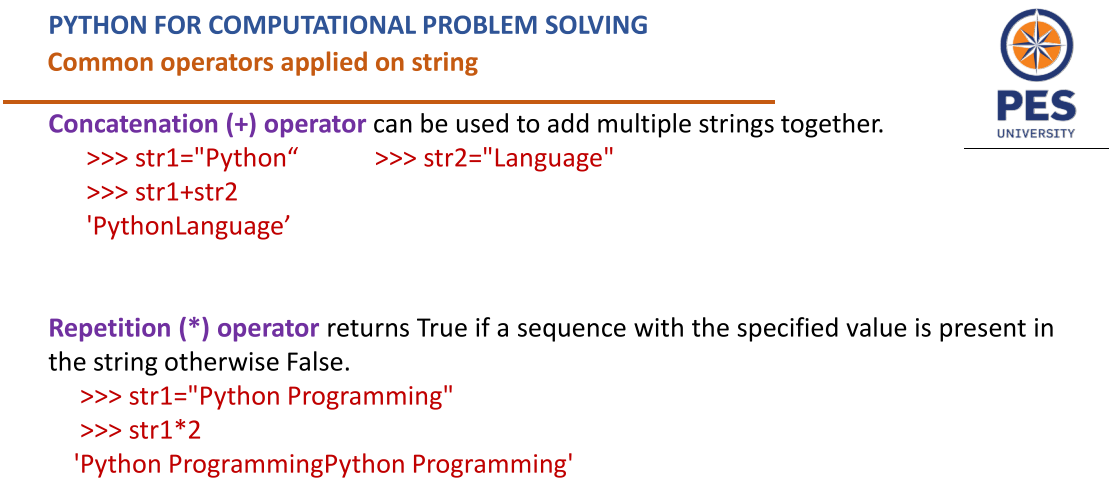


Figure G000251. UE24CS151A PCPS Unit 2 SindhuRPai Class 9 10 Strings types operations · source slide page 9.

EV0001081

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

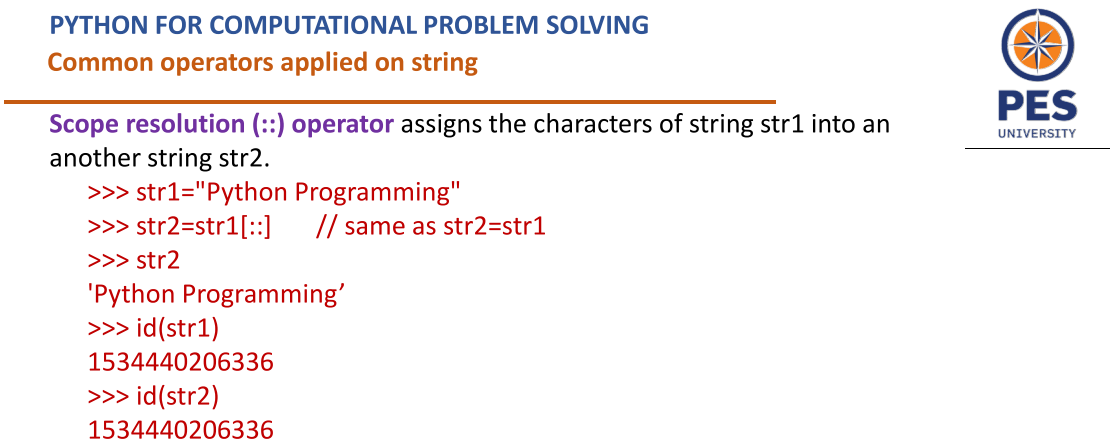


Figure G000252. UE24CS151A PCPS Unit 2 SindhuRPai Class 9 10 Strings types operations · source slide page 10.

EV0001084

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

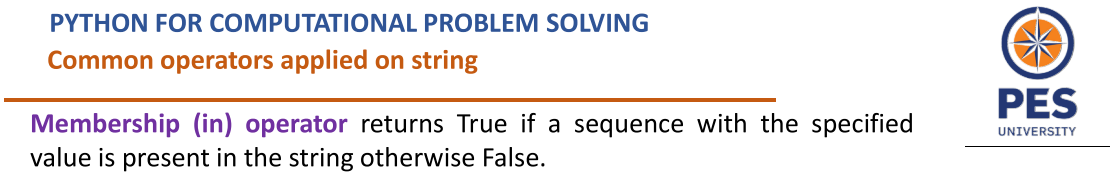


Figure G000253. UE24CS151A PCPS Unit 2 SindhuRPai Class 9 10 Strings types operations · source slide page 11.

EV0001087

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

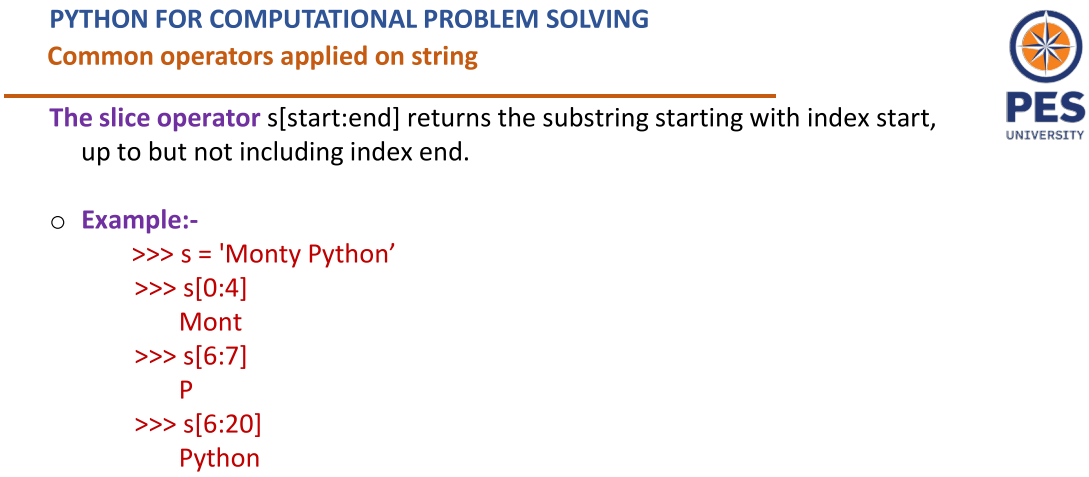


Figure G000254. UE24CS151A PCPS Unit 2 SindhuRPai Class 9 10 Strings types operations · source slide page 12.

EV0001090

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

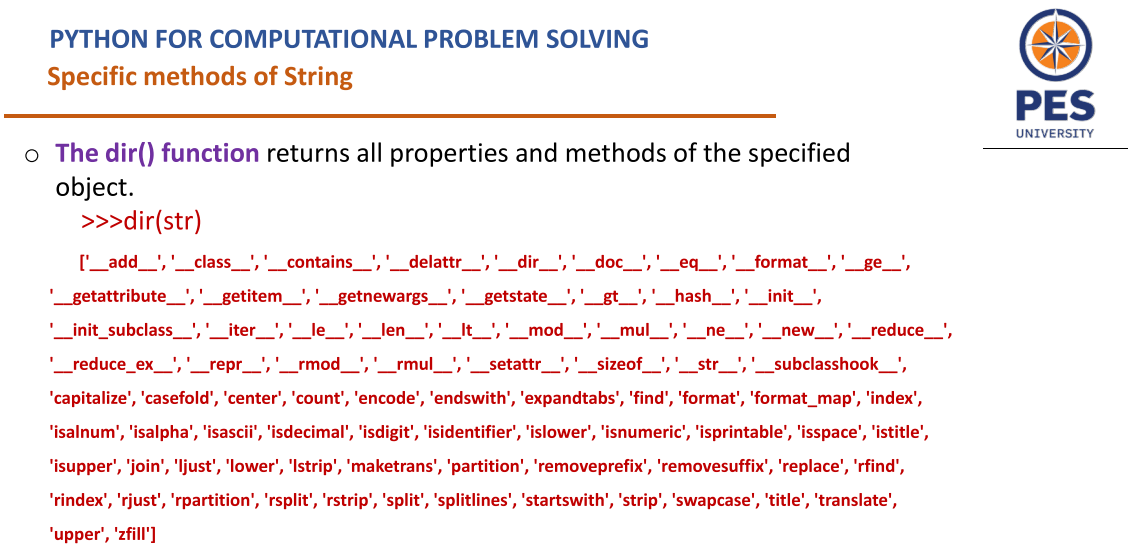


Figure G000255. UE24CS151A PCPS Unit 2 SindhuRPai Class 9 10 Strings types operations · source slide page 13.

EV0001093

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Figure G000256. UE24CS151A PCPS Unit 2 SindhuRPai Class 9 10 Strings types operations · source slide page 14.

EV0001096

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

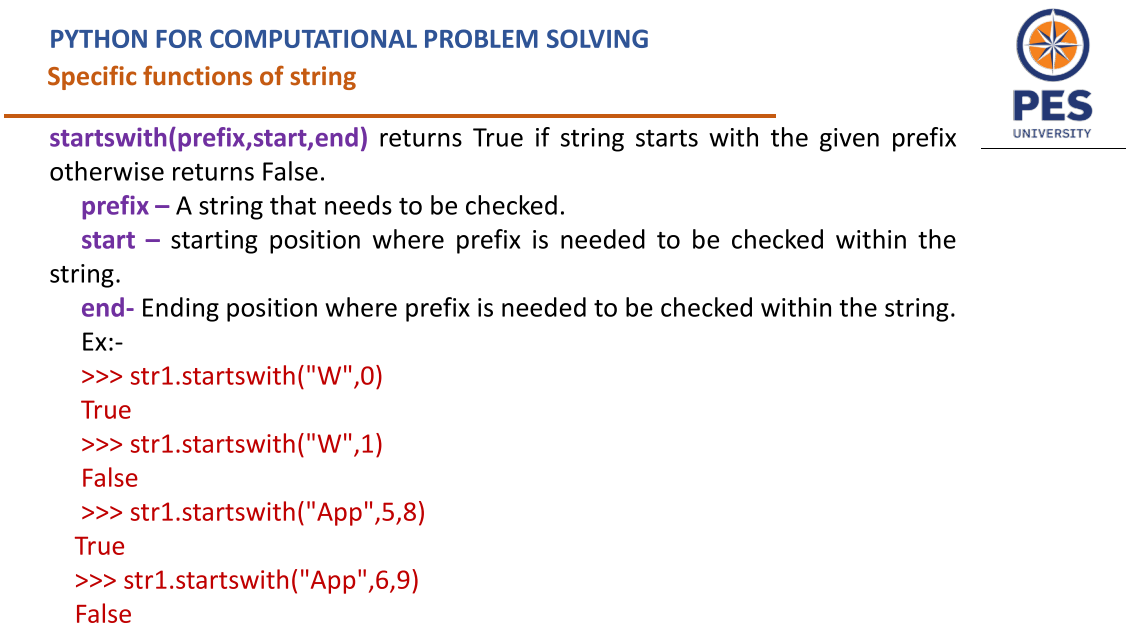


Figure G000257. UE24CS151A PCPS Unit 2 SindhuRPai Class 9 10 Strings types operations · source slide page 15.

EV0001099

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

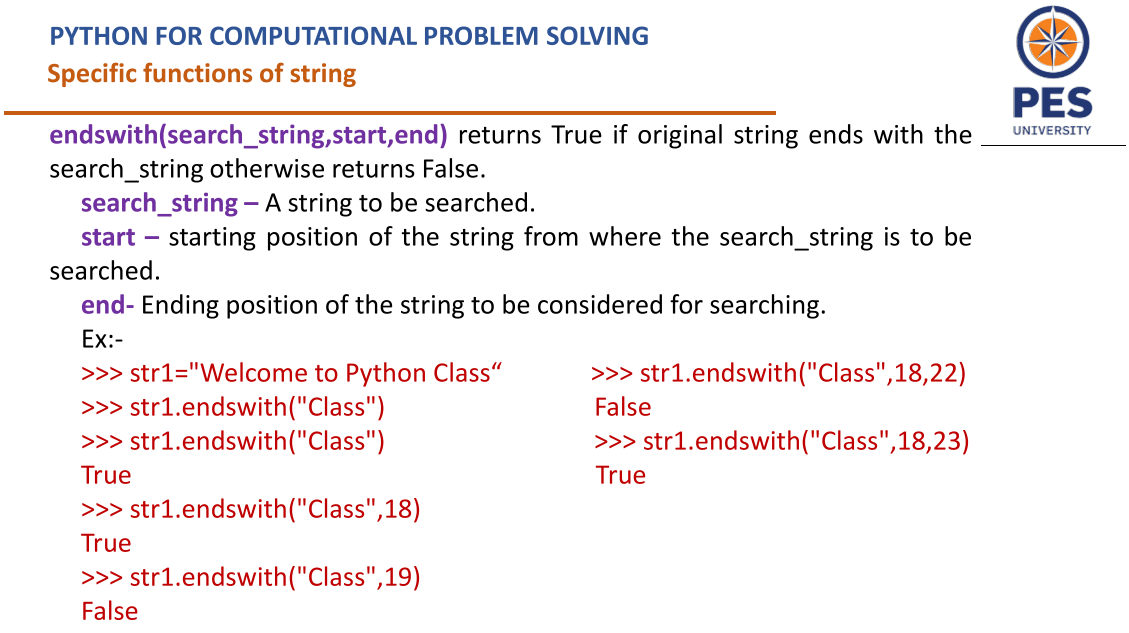


Figure G000258. UE24CS151A PCPS Unit 2 SindhuRPai Class 9 10 Strings types operations · source slide page 16.

EV0001102

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

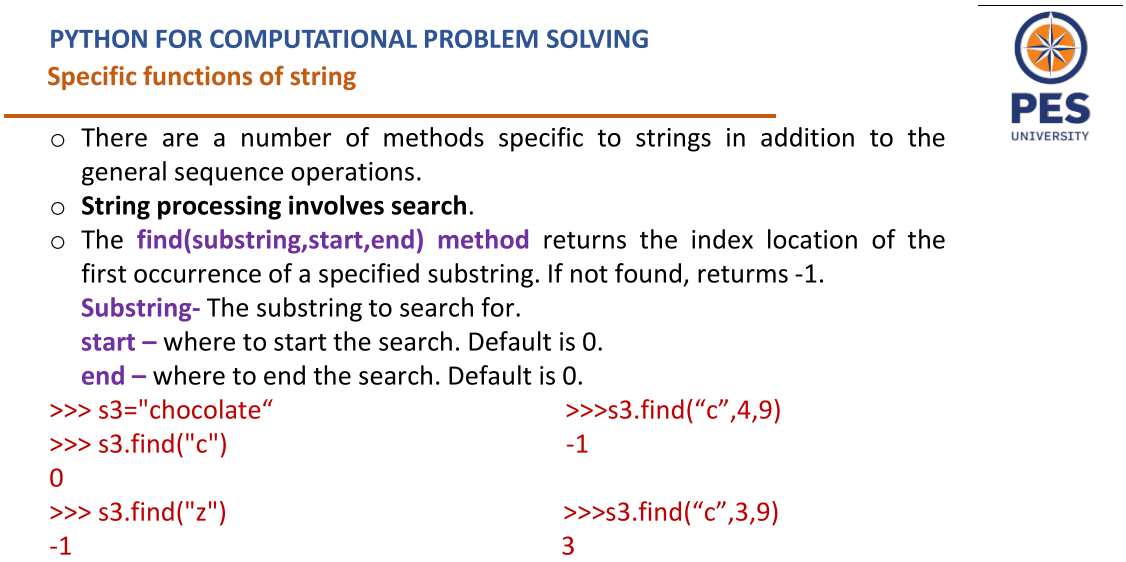


Figure G000259. UE24CS151A PCPS Unit 2 SindhuRPai Class 9 10 Strings types operations · source slide page 17.

EV0001105

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

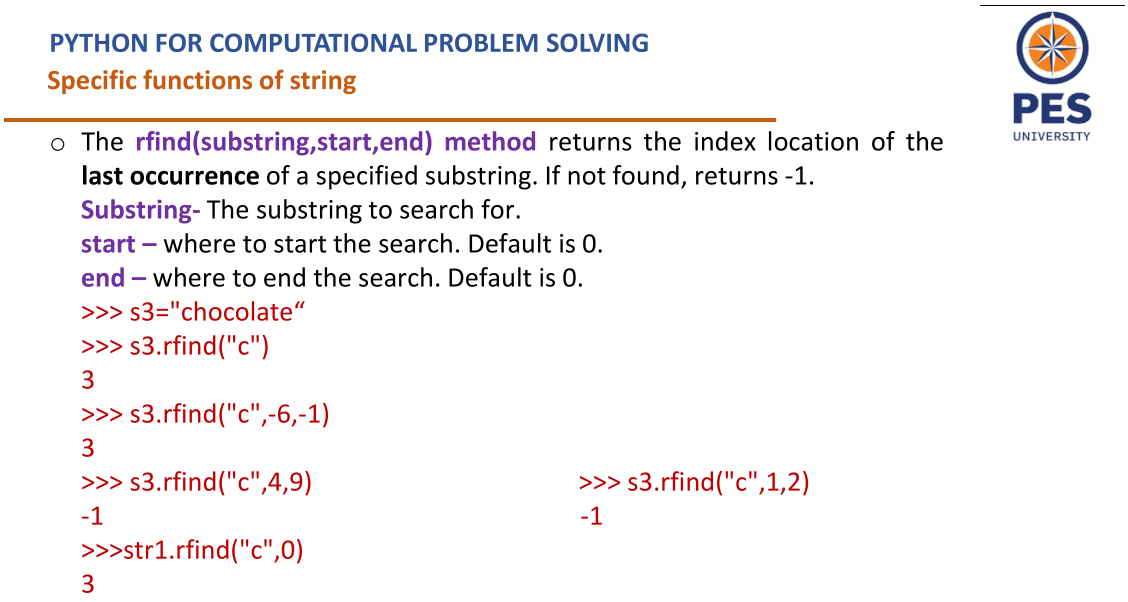


Figure G000260. UE24CS151A PCPS Unit 2 SindhuRPai Class 9 10 Strings types operations · source slide page 18.

EV0001108

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

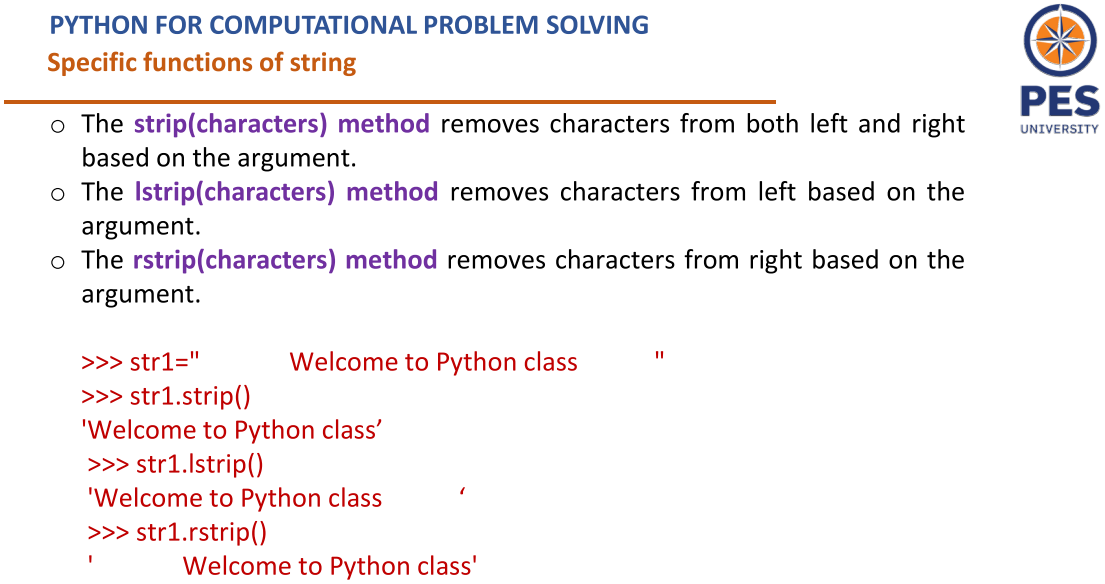


Figure G000261. UE24CS151A PCPS Unit 2 SindhuRPai Class 9 10 Strings types operations · source slide page 19.

EV0001111

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

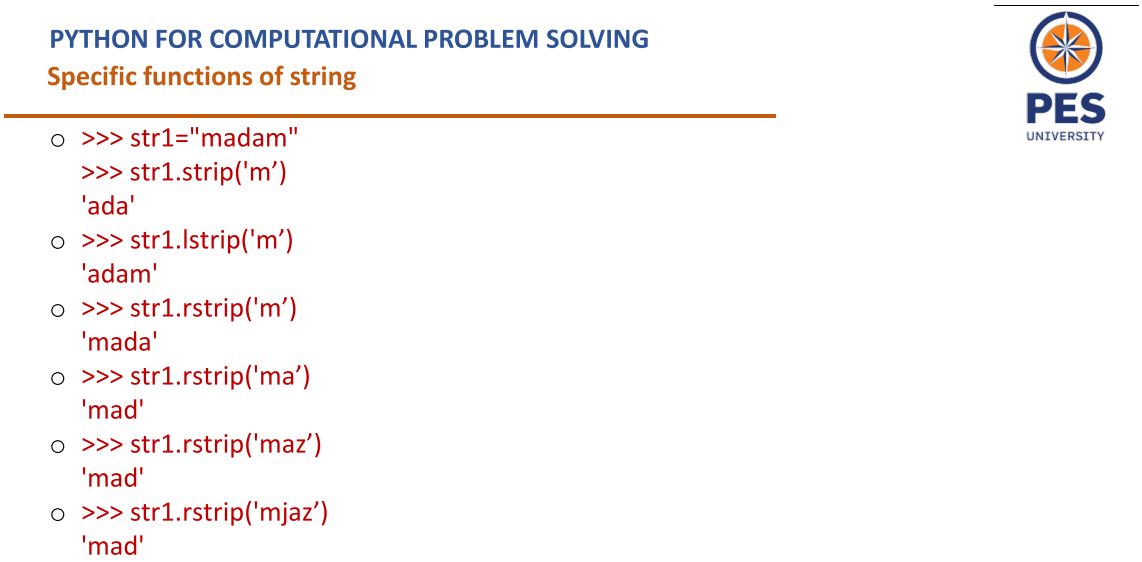


Figure G000262. UE24CS151A PCPS Unit 2 SindhuRPai Class 9 10 Strings types operations · source slide page 20.

EV0001114

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

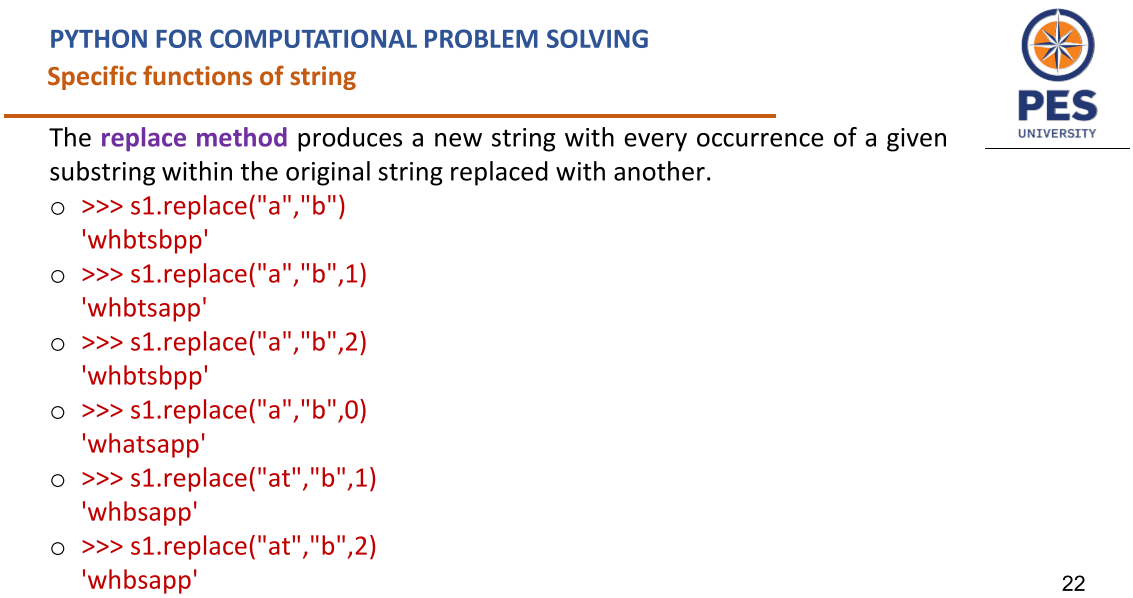


Figure G000263. UE24CS151A PCPS Unit 2 SindhuRPai Class 9 10 Strings types operations · source slide page 21.

EV0001117

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

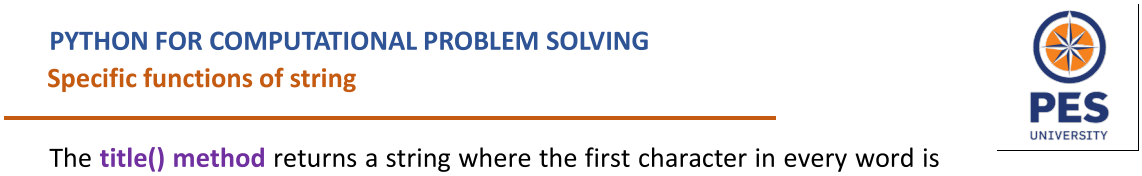


Figure G000264. UE24CS151A PCPS Unit 2 SindhuRPai Class 9 10 Strings types operations · source slide page 22.

EV0001120

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

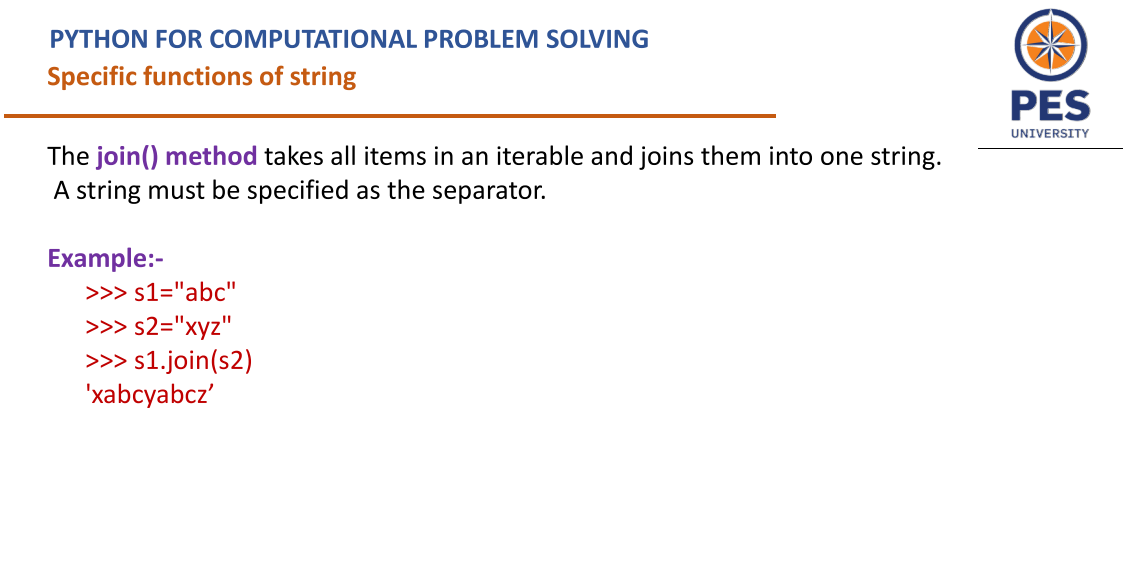


Figure G000265. UE24CS151A PCPS Unit 2 SindhuRPai Class 9 10 Strings types operations · source slide page 23.

EV0001123

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

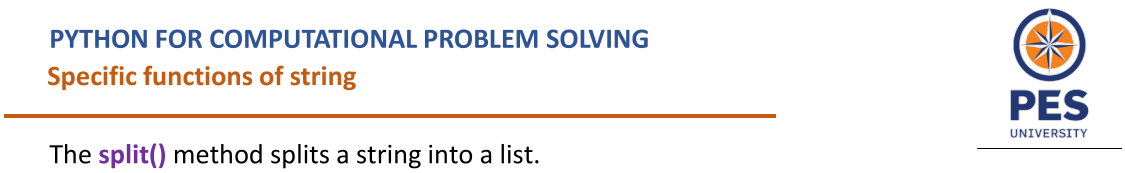


Figure G000266. UE24CS151A PCPS Unit 2 SindhuRPai Class 9 10 Strings types operations · source slide page 24.

EV0001126

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

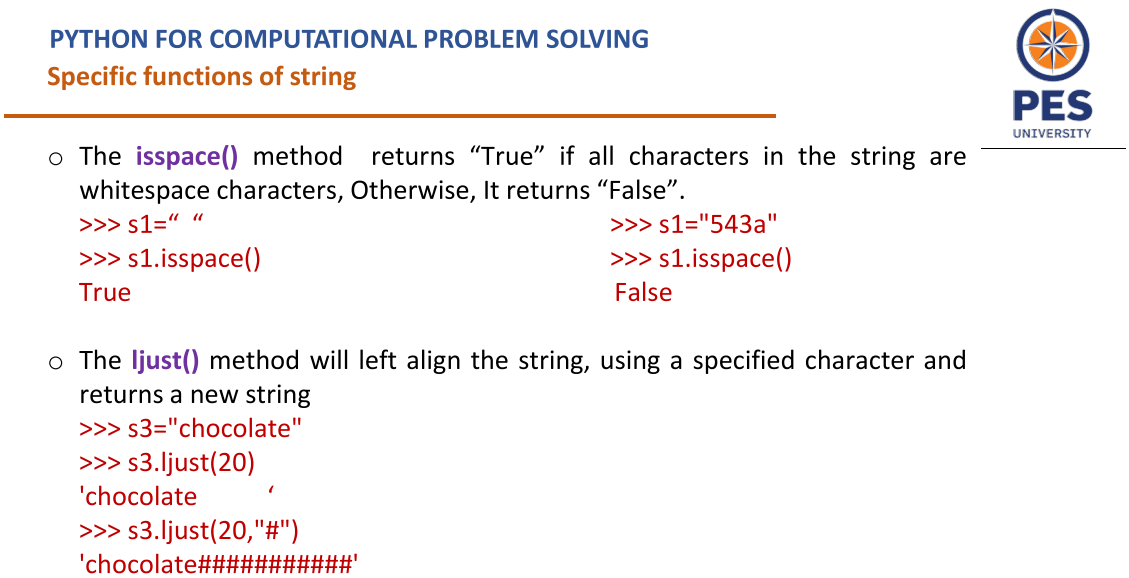


Figure G000267. UE24CS151A PCPS Unit 2 SindhuRPai Class 9 10 Strings types operations · source slide page 25.

EV0001129

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

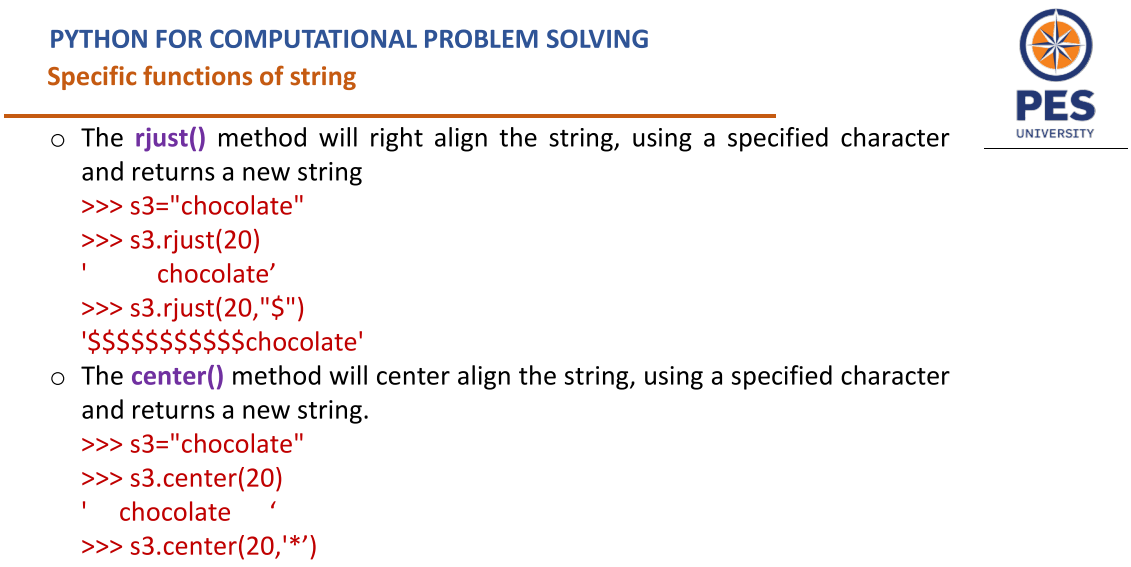


Figure G000268. UE24CS151A PCPS Unit 2 SindhuRPai Class 9 10 Strings types operations · source slide page 26.

EV0001132

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

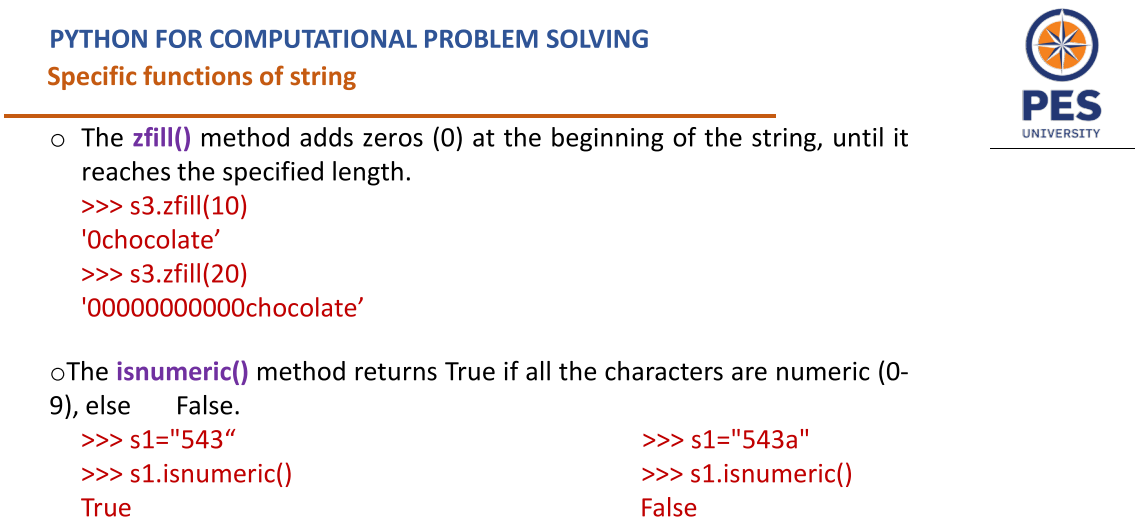


Figure G000269. UE24CS151A PCPS Unit 2 SindhuRPai Class 9 10 Strings types operations · source slide page 27.

EV0001135

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Specific functions of string

PES UNIVERSITY

Checking the Contents of a String			
`str.isalpha()`	Returns True if `str` contains only letters.	`s = 'Hello'`	`s.isalpha() → True`
		`s = 'Hello!'`	`s.isalpha() → False`
`str.isdigit()`	Returns True if `str` contains only digits.	`s = '124'`	`s.isdigit() → True`
		`s = '124A'`	`s.isdigit() → False`
`str.islower()` `str.isupper()`	Returns True if `str` contains only lower (upper) case letters.	`s = 'hello'`	`s.islower() → True`
		`s = 'Hello'`	`s.isupper() → False`
`str.lower()` `str.upper()`	Return lower (upper) case version of `str`.	`s = 'Hello!'`	`s.lower() → 'hello!'`
		`s = 'hello!'`	`s.upper() → 'HELLO!'`
Searching the Contents of a String			
`str.find(w)`	Returns the index of the first occurrence of `w` in `str`. Returns -1 if not found.	`s = 'Hello!'`	`s.find('l') → 2`
		`s = 'Goodbye'`	`s.find('l') → -1`
Replacing the Contents of a String			
`str.replace(w,t)`	Returns a copy of `str` with all occurrences of `w` replaced with `t`.	`s = 'Hello!'`	`s.replace('H', 'J') → 'Jello'`
		`s = 'Hello'`	`s.replace('ll', 'r') → 'Hero'`
Removing the Contents of a String			
`str.strip(w)`	Returns a copy of `str` with all leading and trailing characters that appear in w removed.	`s = ' Hello! '` `s = 'Hello\n'`	`s.strip(' !') → 'Hello'` `s.strip('\n') → 'Hello'`
Splitting a String			
`str.split(w)`	Returns a list containing all strings in `str` delimited by w.	`s = 'Lu, Chao'`	`s.split(',') → ['Lu', 'Chao']`

Figure G000270. UE24CS151A PCPS Unit 2 SindhuRPai Class 9 10 Strings types operations · source slide page 28.

EV0001138

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

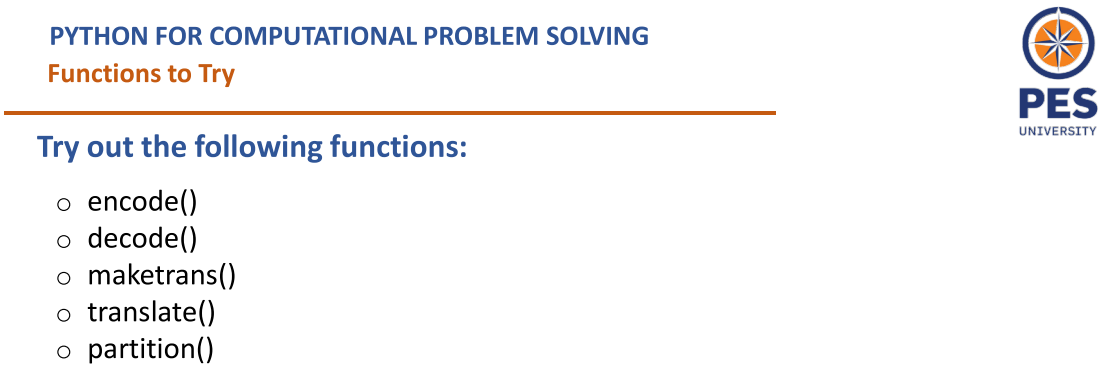


Figure G000271. UE24CS151A PCPS Unit 2 SindhuRPai Class 9 10 Strings types operations · source slide page 29.

EV0001141

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

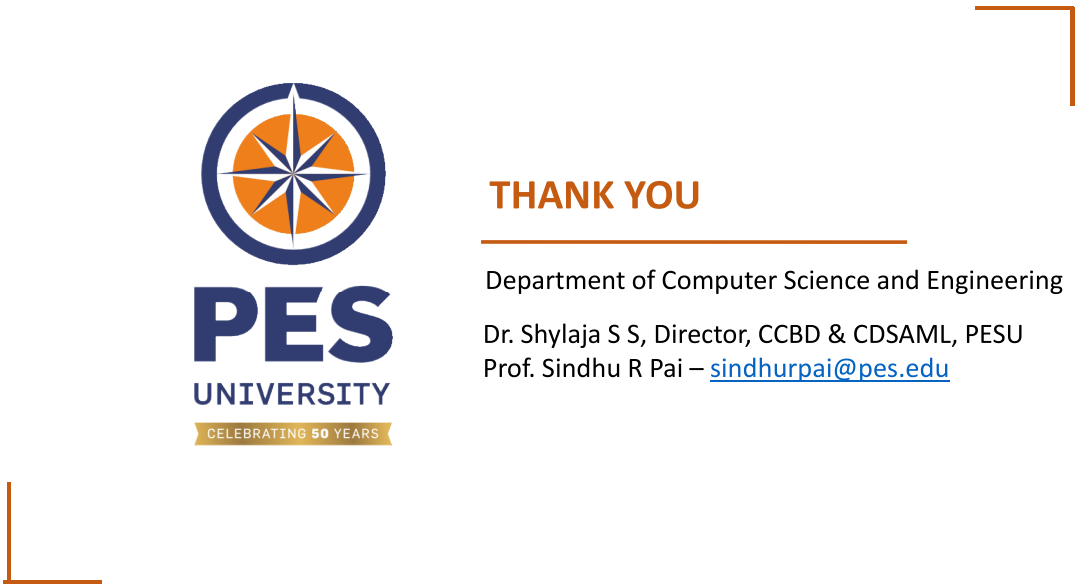


Figure G000272. UE24CS151A PCPS Unit 2 SindhuRPai Class 9 10 Strings types operations · source slide page 30.

EV0001144

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Important points and why

1. Source slide page 1 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Strings Prof. Sindhu R Pai PCPS Theory Anchor - 2024 Department of Computer Science and Engineering EV0001056 · EV0001057 · EV0001058
2. Source slide page 2 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Introduction A string is a collection of characters or a sequence of characters. Creating strings is as straightforward as assigning a value to a variable. Ex: var1 = ““ # empty string. var1 = "python" var1 = “”” this is python””” EV0001059 · EV0001060 · EV0001061
3. Source slide page 3 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Characteristics o A string has zero or more characters. >>> str1=" “ >>> str1="p" >>> str2="python“ >>> str3 = ‘’’python pro ‘’’ o Each character of a string can be accessed by using the index that starts from 0. Negative indicesindices areare allowedallowed. Example:- >>> str2[0] 'p’ >>> str2[-1] ‘n’ o The index value can be an expression that is computed. >>> str2[2+3] 'n’ EV0001062 · EV0001063 · EV0001064

4. Source slide page 4 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Characteristics o Immutable – cannot modify the individual characters in a string. One cannot add, delete, or replace characters of a string. Example:- str2="python"" h " >>> str2[2]='T' Traceback (most recent call last): File "<stdin>", line 1, in <module> TypeError: 'str' object does not support item assignment EV0001065 · EV0001066 · EV0001067
5. Source slide page 5 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Characteristics o Iterable: Traversing elements one by one. Example:- str2="python" forfor ii inin str2:str2: print(i, end=" ") Output: p y t h o n EV0001068 · EV0001069 · EV0001070
6. Source slide page 6 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Methods of String All string operations that "modify" a string return a new stringstring thatthat isis aa modifiedmodified versionersion ofof thethe originaloriginal stringstring. EV0001071 · EV0001072 · EV0001073
7. Source slide page 7 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Common methods of String o The len(str) returns the number of characters in a string. >>> str1="python" >>> len(str1) 6 oo ss.index(chr)index(chr) returnsreturns thethe indexindex ofof thethe firstfirst occurrenceoccurrence ofof chrchr inin ss. >>> str1="python programming" >>> str1.index('prog') >>> str1.index('p') 7 0 o count() - Returns the number of times a specified value / character occurs in a string. >>> str1="Welcome to Python Class" >>> str1.count('s') 2 EV0001074 · EV0001075 · EV0001076
8. Source slide page 8 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Common methods of String o len(str) returns the number of characters in a string. >>> str1="python" >>> len(str1) 6 min and max functions as applied to strings return the smallest and largest character respectivelyrespectively basedbased onon thethe underlyingunderlying UnicodeUnicode encodingencoding. For example, all lowercase letters are larger have a larger Unicode value than all uppercase letters. o Example:- >>> str1="Python" >>> min(str1) 'P' >>> max(str1) 'y' EV0001077 · EV0001078 · EV0001079
9. Source slide page 9 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Common operators applied on string Concatenation (+) operator can be used to add multiple strings together. >>> str1="Python" >>> str2="Language" >>> str1+str2 'PythonLanguage' Repetition (*) operator returns True if a sequence with the specified value is present in the string otherwise False. >>> str1="Python Programming" >>> str1*2 'Python ProgrammingPython Programming' EV0001080 · EV0001081 · EV0001082
10. Source slide page 10 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Common operators applied on string Scope resolution (::) operator assigns the characters of string str1 into an another string str2. >>> str1="Python Programming" >>> str2=str1[::] // same as str2=str1 >>> str2 'PythonPython Programming'Programming >>> id(str1) 1534440206336 >>> id(str2) 1534440206336 EV0001083 · EV0001084 · EV0001085

11. Source slide page 11 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Common operators applied on string Membership (in) operator returns True if a sequence with the specified value is present in the string otherwise False. >>> str2="Language" >>> "Lang" in "Language" True >>>>>> "Png"Png inin "Language"Language False Membership (not in) operator is used to repeat the string to a certain length. >>> str1="Language" >>> "LAN" not in str1 True >>> "Lang" not in str1 False EV0001086 · EV0001087 · EV0001088
12. Source slide page 12 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Common operators applied on string The slice operator s[start:end] returns the substring starting with index start, up to but not including index end. o Example:- >>> s = 'Monty Python' >>>>>> s[0:4]s[0:4] Mont >>> s[6:7] P >>> s[6:20] Python EV0001089 · EV0001090 · EV0001091
13. Source slide page 13 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Specific methods of String o The dir() function returns all properties and methods of the specified object. >>>dir(str) ['__add__', '__class__', '__contains__', '__delattr__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__getattribute__', '__getitem__', '__getnewargs__', '__getstate__', '__gt__', '__hash__', '__init__', '__init_subclass__', '__iter__', '__le__', '__len__', '__lt__', '__mod__', '__mul__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__', '__rmod__', '__rmul__', '__setattr__', '__sizeof__', '__str__', '__subclasshook__', 'capitalize', 'casefold', 'center', 'count', 'encode', 'endswith', 'expandtabs', 'find', 'format', 'format_map', 'index', 'isalnum', 'isalpha', 'isascii', 'isdecimal', 'isdigit', 'isidentifier', 'islower', 'isnumeric', 'isprintable', 'isspace', 'istitle', 'isupper', 'join', 'ljust', 'lower', 'lstrip', 'maketrans', 'partition', 'removeprefix', 'removesuffix', 'replace', 'rfind', 'rindex', 'rjust', 'rpartition', 'rsplit', 'rstrip', 'split', 'splitlines', 'startswith', 'strip', 'swapcase', 'title', 'translate', 'upper', 'zfill'] EV0001092 · EV0001093 · EV0001094
14. Source slide page 14 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Specific methods of String o index(chr) returns the index of the first occurrence of chr in s. >>> str1="python programming“ >>> str1.index('prog') >>> str1.index(‘p’) 0 o count() - Returns the number of times a specified value / character occurs in a string. >>> str1="Welcome to Python Class“ >>> str1.count('s') 2 EV0001095 · EV0001096 · EV0001097
15. Source slide page 15 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Specific functions of string startswith(prefix,start,end) returns True if string starts with the given prefix otherwise returns False. prefix – A string that needs to be checked. start – starting position where prefix is needed to be checked within the string. end-end EndingEnding positionposition wherewhere prefixprefix isis neededneeded totobebe checkedchecked withinwithin thethe stringstring. Ex:- >>> str1.startswith("W",0) True >>> str1.startswith("W",1) False >>> str1.startswith("App",5,8) True >>> str1.startswith("App",6,9) False EV0001098 · EV0001099 · EV0001100

16. Source slide page 16 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Specific functions of string endswith(search_string,start,end) returns True if original string ends with the search_string otherwise returns False. search_string – A string to be searched. start – starting position of the string from where the search_string is to be searched. end-end EndingEnding positionposition ofof thethe stringstring totoo bebe consideredconsidered forfor searchingsearching. Ex:- >>> str1="Welcome to Python Class“ >>> str1.endswith("Class",18,22) >>> str1.endswith("Class") False >>> str1.endswith("Class") >>> str1.endswith("Class",18,23) True True >>> str1.endswith("Class",18) True >>> str1.endswith("Class",19) False EV0001101 · EV0001102 · EV0001103

17. Source slide page 17 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Specific functions of string o There are a number of methods specific to strings in addition to the general sequence operations. o String processing involves search. o The find(substring,start,end) method returns the index location of the first occurrence of a specified substring. If not found, returns -1. Substring-Substring TheThe substringsubstring totoo searchsearch forfor. start – where to start the search. Default is 0. end – where to end the search. Default is 0. >>> s3="chocolate“ >>>s3.find(“c”,4,9) >>> s3.find("c") -1 0 >>> s3.find("z") >>>s3.find(“c”,3,9) -1 3 EV0001104 · EV0001105 · EV0001106

18. Source slide page 18 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Specific functions of string o The rfind(substring,start,end) method returns the index location of the last occurrence of a specified substring. If not found, returns -1. Substring- The substring to search for. start – where to start the search. Default is 0. end – where to end the search. Default is 0. >>>>>> s3="chocolate“s3 chocolate >>> s3.rfind("c") 3 >>> s3.rfind("c",-6,-1) 3 >>> s3.rfind("c",4,9) >>> s3.rfind("c",1,2) -1 -1 >>>str1.rfind("c",0) 3 EV0001107 · EV0001108 · EV0001109

19. Source slide page 19 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Specific functions of string o The strip(characters) method removes characters from both left and right based on the argument. o The lstrip(characters) method removes characters from left based on the argument. o The rstrip(characters) method removes characters from right based on the argumentargument. >>> str1=" Welcome to Python class " >>> str1.strip() 'Welcome to Python class’ >>> str1.lstrip() 'Welcome to Python class ‘ >>> str1.rstrip() ' Welcome to Python class' EV0001110 · EV0001111 · EV0001112

20. Source slide page 20 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Specific functions of string o >>> str1="madam" >>> str1.strip('m’) 'ada' o >>> str1.lstrip('m’) 'adam' oo >>>>>>> str1str1.rstrip(rstrip('m’)m) 'mada' o >>> str1.rstrip('ma’) 'mad' o >>> str1.rstrip('maz’) 'mad' o >>> str1.rstrip('mjaz’) 'mad' EV0001113 · EV0001114 · EV0001115

21. Source slide page 21 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Specific functions of string The replace method produces a new string with every occurrence of a given substring within the original string replaced with another. o >>> s1.replace("a","b") 'whbtsbpp' o >>> s1.replace("a","b",1) 'whbtsapp'whbtsapp o >>> s1.replace("a","b",2) 'whbtsbpp' o >>> s1.replace("a","b",0) 'whatsapp' o >>> s1.replace("at","b",1) 'whbsapp' o >>> s1.replace("at","b",2) 'whbsapp' 22 EV0001116 · EV0001117 · EV0001118

22. Source slide page 22 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Specific functions of string The title() method returns a string where the first character in every word is upper case. o >>> str1="people education society" >>> str1.title() 'People Education Society' The capitalize() method returns a string where the first character is upper case, and the rest is lower case. >>> s1="Python" >>> s1.capitalize() 'Python' EV0001119 · EV0001120 · EV0001121

23. Source slide page 23 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Specific functions of string The join() method takes all items in an iterable and joins them into one string. A string must be specified as the separator. Example:- >>> s1="abc" >>> s2="xyz" >>> s1.join(s2) 'xabcyabcz' EV0001122 · EV0001123 · EV0001124

24. Source slide page 24 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Specific functions of string The split() method splits a string into a list. Example:- o >>> str1="When someone is lost, dare to help them find the way." >>> str1.split() ['When', 'someone', 'is', 'lost,', 'dare', 'to', 'help', 'them', 'find', 'the', 'way.'] o >>> str1.split("i") ['When someone ', 's lost, dare to help them f', 'nd the way.'] o >>> str1.split("i",1) ['When someone ', 's lost, dare to help them find the way.'] o >>> str1.split("i",1,2) Traceback (most recent call last): File "<stdin>", line 1, in <module> TypeError: split() takes at most 2 arguments (3 given) EV0001125 · EV0001126 · EV0001127

25. Source slide page 25 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Specific functions of string o The isspace() method returns “True” if all characters in the string are whitespace characters, Otherwise, It returns “False”. >>> s1=“ “ >>> s1="543a" >>> s1.isspace() >>> s1.isspace() True False o The ljust() method will left align the string, using a specified character and returns a new string >>> s3="chocolate" >>> s3.ljust(20) 'chocolate ‘ >>> s3.ljust(20,"#") 'chocolate##########' EV0001128 · EV0001129 · EV0001130

26. Source slide page 26 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Specific functions of string o The rjust() method will right align the string, using a specified character and returns a new string >>> s3="chocolate" >>> s3.rjust(20) ' chocolate’ >>>>>> s3s3.rjust(20,rjust(20 "$")$) '$$$$$$$$$$chocolate' o The center() method will center align the string, using a specified character and returns a new string. >>> s3="chocolate" >>> s3.center(20) ' chocolate ‘ >>> s3.center(20,'*’) '*****chocolate******' EV0001131 · EV0001132 · EV0001133

27. Source slide page 27 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Specific functions of string o The zfill() method adds zeros (0) at the beginning of the string, until it reaches the specified length. >>> s3.zfill(10) '0chocolate’ >>> s3.zfill(20) '00000000000chocolate’00000000000chocolate oThe isnumeric() method returns True if all the characters are numeric (0- 9), else False. >>> s1="543“ >>> s1="543a" >>> s1.isnumeric() >>> s1.isnumeric() True False EV0001134 · EV0001135 · EV0001136

28. Source slide page 28 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Specific functions of string EV0001137 · EV0001138 · EV0001139

29. Source slide page 29 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Functions to Try Try out the following functions: o encode() o decode() o maketrans() oo translate()translate() o partition() EV0001140 · EV0001141 · EV0001142

30. Source slide page 30 (extracted text): THANK YOU Departmentp of Computerp Science and Engineeringg g Dr. Shylaja S S, Director, CCBD & CDSAML, PESU Prof. Sindhu R Pai – sindhurpai@pes.edu EV0001143 · EV0001144 · EV0001145

SHORT-NOTE RECAP

Transcript block 1 (source interval 000000-000299 seconds): [00:00:01.604] Let's start with the strings functionality. So I will use a string. [00:00:13.188] R one is equal to python. [00:00:18.948] Okay, so let's quickly work on a few of the functions. So I'll say str1.capitalize. [00:00:35.876] So C, capitalize. If we can see our first character in a string is capitalized. So capitalize converts the first character to an uppercase. [00:00:51.908] Converts the first character to an uppercase. It is still temporary. Please remember if you want it to be permanent, then you have to assign a variable to it, in which case it becomes a new variable for us. Okay. So I'll say so we [00:01:11.588] Let's say find out how many times cap small p is present. So it says it is present once. How many times capital P is present? It is present zero times. So it gives this. So next one. [00:01:30.916] Find is a function which uses the index [00:01:36.900] position of a particular character or a subset. Okay. So in this case I can say P is a subset of this particular uh set. Okay? P in the sense of the small P. [00:01:51.812] And so I'm talking about where is the capital P present. If you can see P capital P is not present and we'll see what is the output that it gives us. So it gives us a value called as minus 1. [00:02:05.636] I'll see where exactly my small p is present. My small p is present at the zeroth index, so it gives me a zero. So hopefully it is clear. Next I'll do an index. I'll do an index. [00:02:22.916] What is the difference between an index and a fine? Let's find out. Okay. So index also returns the position of a particular substring in a string. Okay. So it says P is present at the zero location. Ask the next question: Where is my capital B? [00:02:43.364] Is capital P present in this case? No, it is not present. So it throws us an error. Find will not throw an error if a particular substring is not present. Instead, index through an error. [00:03:00.804] So important difference to note required. Use this as and when required for us. So this is my string LC STR1 dot is L num [00:03:21.220] Alarm is alpha numeric. You're trying to ask [00:03:28.516] It has characters either alphabets or numeric return true else return false. Okay, so it has returned true. So let's see when does it return false. So I'll say, and I'm giving a space. I'll say [00:03:46.272] Str2 is L num, so it has to throw me a false because we're talking about only alphabets or only numbers or the combination of alpha and numeric. Okay, nothing else. So it says false because we've used a space where [00:04:04.516] um there's no alphabet or a number. So similarly I'll do this is num is digit. Is it a digit? This requires only digits to be available else it throws a s and [00:04:24.004] false okay so I'll say str three is equal to one two three four only digits so it has to um give us a true okay so STR3 is digit it is true because it contains only digits [00:04:42.692] Right. So similarly if you're talking about e is alpha [00:04:48.772] requires only alpha bits but in this case we have digits let's see so it throws me an error so I will come back here and change it as STR1 which has only alpha bet so it is true

EV0000322 · EV0000323 · EV0000324 · EV0000325

Transcript block 2 (source interval 000300-000599 seconds): [00:05:04.452] it is clear to you what is my str 1 my str 1 is python so I'll say str1 dot upper I'm talking about convert everything to upper okay so it has converted [00:05:20.100] But when I say str1, it is still the same, okay, and if you want the changes to be reflected, maybe you can do something like this. A sign it to some variable and say this. So [00:05:34.992] But the changes that you have done, it has been reflected. So in this

case, STR1 is a separate string and STR5 is a new string for us. So many functionalities of these kinds are there. Please try it out. Okay. I can also ask. [00:05:54.564] You can also ask, is STR5, is it upper? [00:06:03.652] All the characters or the entire substring is in uppercase, so it returns true, else it returns us false. Okay, so Str1, I'll work with SDR1 and say end swift. [00:06:24.292] P does your uh substring ends with P in this case? No, so it is false. Does it end with N? Yes it does, so it returns me true. I can also say Starts with [00:06:43.076] Does it start with N? No it doesn't, so it is false. Does it start with P? Yes, it does, so it returns true. So many functionalities work with all of them. [00:06:58.308] In the uh coming classes or the sessions, we are going to be making use of all the concepts that we are going to be learning in the unit 2, and we will try to solve some exercise problems or an assignment problems in the class itself. Okay. [00:07:16.580] You have understood. Please try out the other functionalities that are available. Okay. Thank you.

EV0000326 · EV0000327 · EV0000328 · EV0000329

Full transcript: 887 normalized words, 29 timestamped segments, 444.970667 seconds duration. Low-confidence segments flagged: 0.

EV0000322 · EV0000323 · EV0000324 · EV0000325

23 Source slide deck: UE23CS151A PCPS Unit 3 Sowmya Shree Sindhu Class 83 84 85 Modules

This entry preserves the complete slide deck titled UE23CS151A PCPS Unit 3 Sowmya Shree Sindhu Class 83 84 85 Modules as searchable extracted page text and page-faithful rendered images. Its association with a particular video is not confirmed; it remains a separate topic under Python for computational Problem Solving, Unit_01, to avoid an unsupported match. EV0000330 · EV0000331

Graphs and diagrams

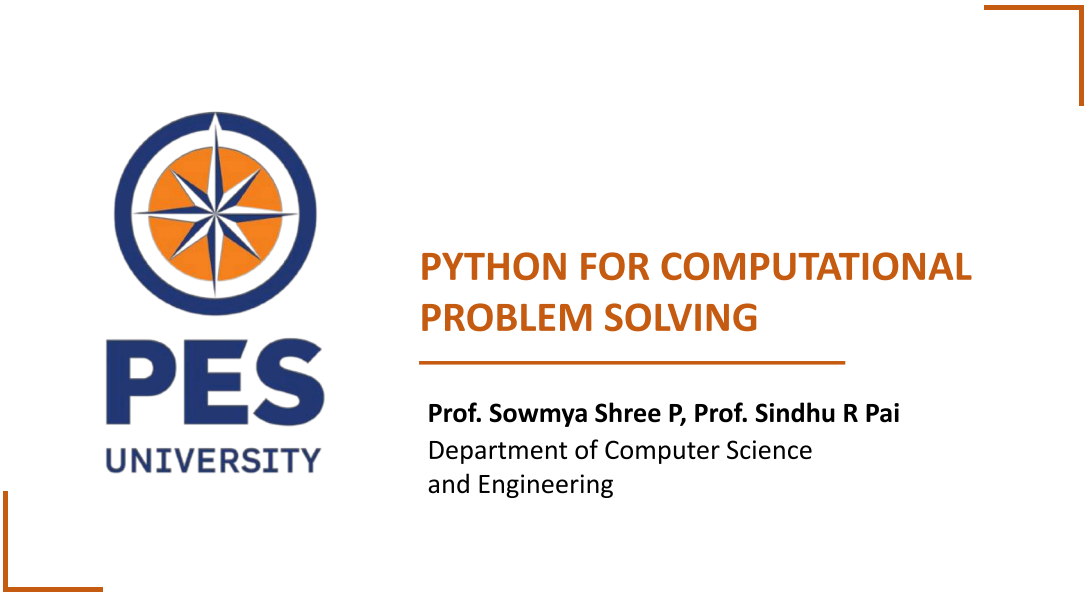


Figure G000001. UE23CS151A PCPS Unit 3 Sowmya Shree Sindhu Class 83 84 85 Modules · source slide page 1.

EV0000331

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Figure G000002. UE23CS151A PCPS Unit 3 Sowmya Shree Sindhu Class 83 84 85 Modules · source slide page 2.

EV0000334

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

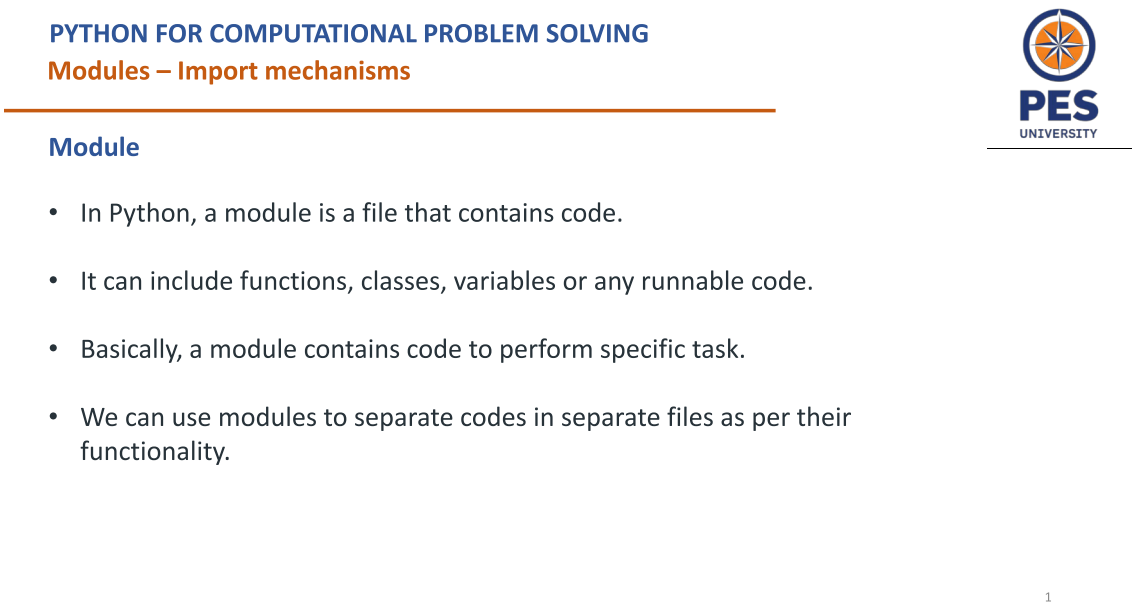


Figure G000003. UE23CS151A PCPS Unit 3 Sowmya Shree Sindhu Class 83 84 85 Modules · source slide page 3.

EV0000337

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

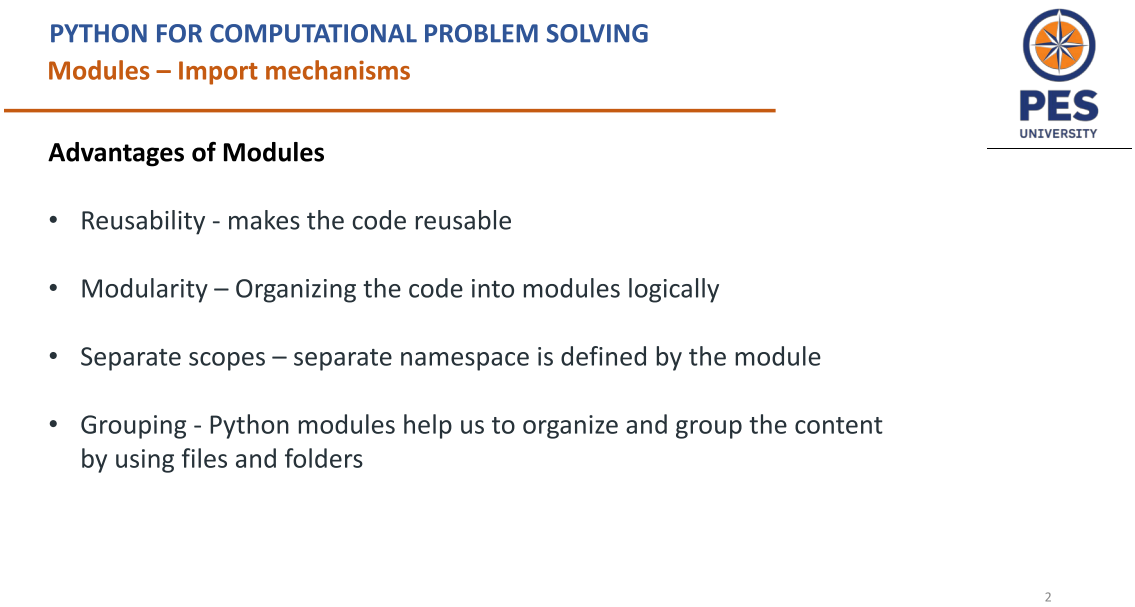


Figure G000004. UE23CS151A PCPS Unit 3 Sowmya Shree Sindhu Class 83 84 85 Modules · source slide page 4.

EV0000340

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Modules – Import mechanisms

PES UNIVERSITY

Need for Modules

- While working on coding, We need to use many classes, variables, functions.
- If we include everything in a single file, the program may become large.
- To reduce size of the code, we can group together some similar functions, classes into a collection.
- That collection is nothing but modules in python.

3

Figure G000005. UE23CS151A PCPS Unit 3 Sowmya Shree Sindhu Class 83 84 85 Modules · source slide page 5.

EV0000343

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

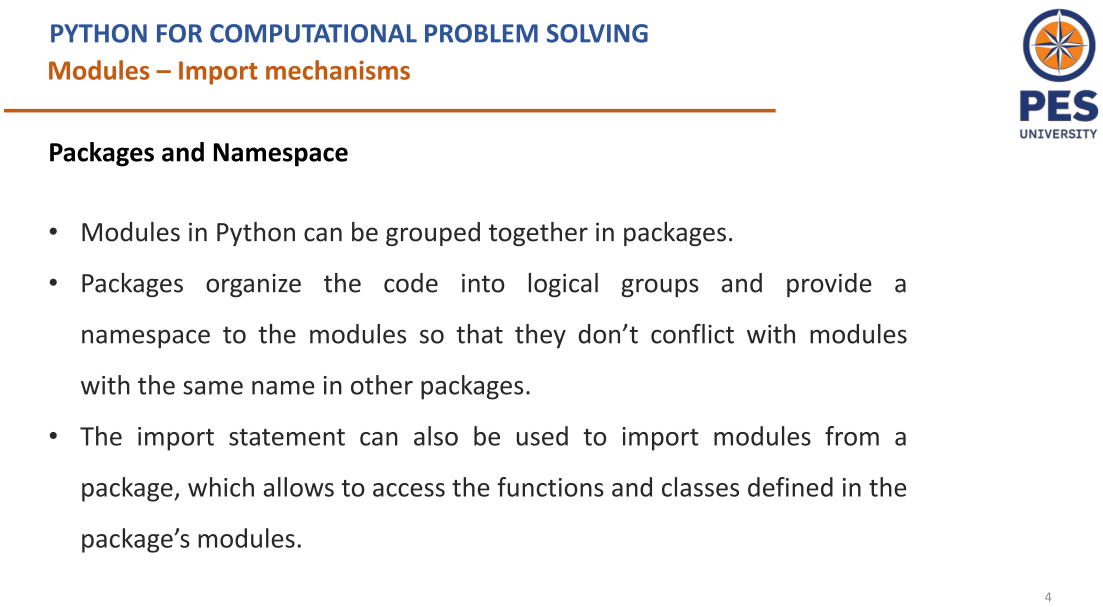


Figure G000006. UE23CS151A PCPS Unit 3 Sowmya Shree Sindhu Class 83 84 85 Modules · source slide page 6.

EV0000346

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

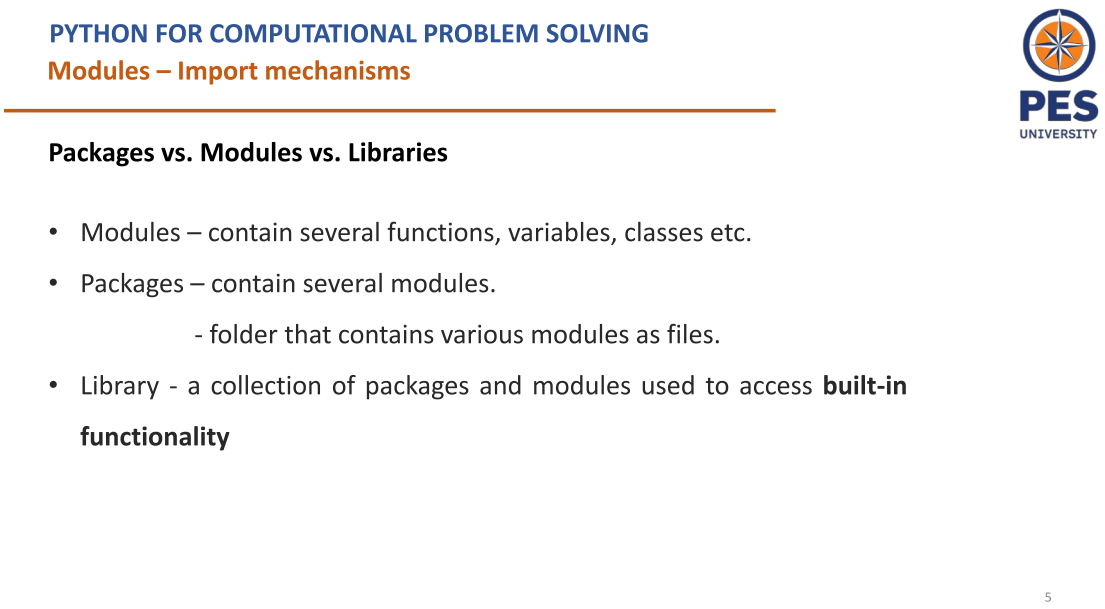


Figure G000007. UE23CS151A PCPS Unit 3 Sowmya Shree Sindhu Class 83 84 85 Modules · source slide page 7.

EV0000349

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

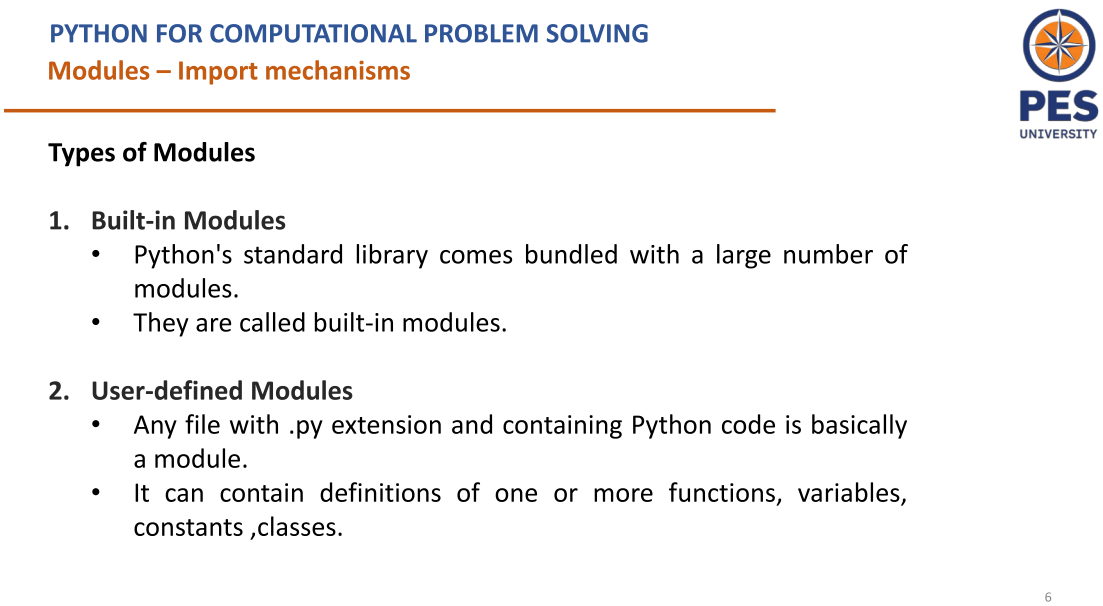


Figure G000008. UE23CS151A PCPS Unit 3 Sowmya Shree Sindhu Class 83 84 85 Modules · source slide page 8.

EV0000352

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

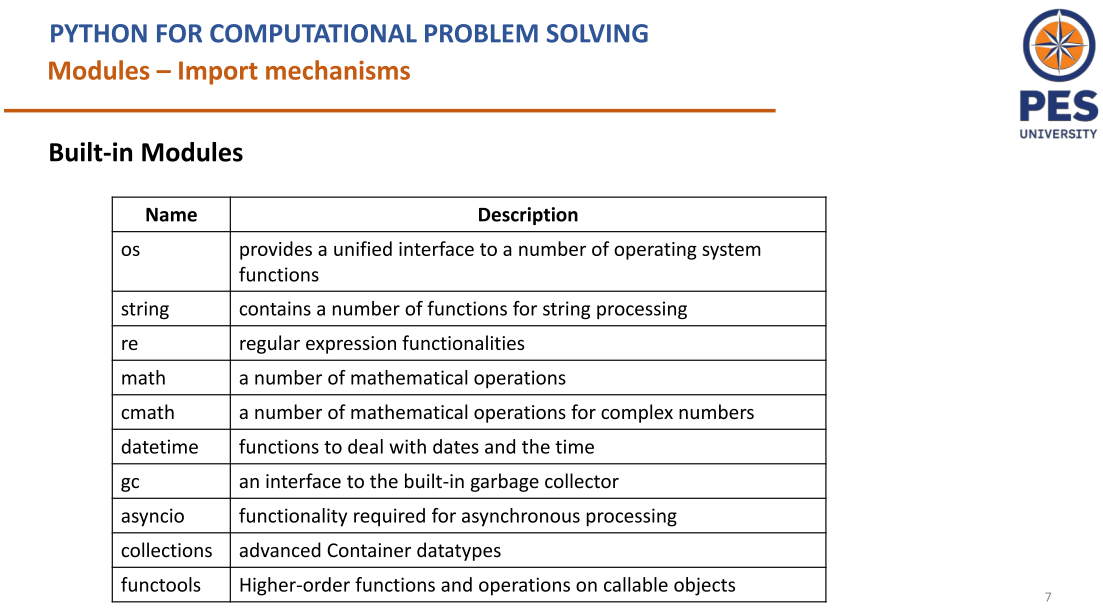


Figure G000009. UE23CS151A PCPS Unit 3 Sowmya Shree Sindhu Class 83 84 85 Modules · source slide page 9.

EV0000355

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

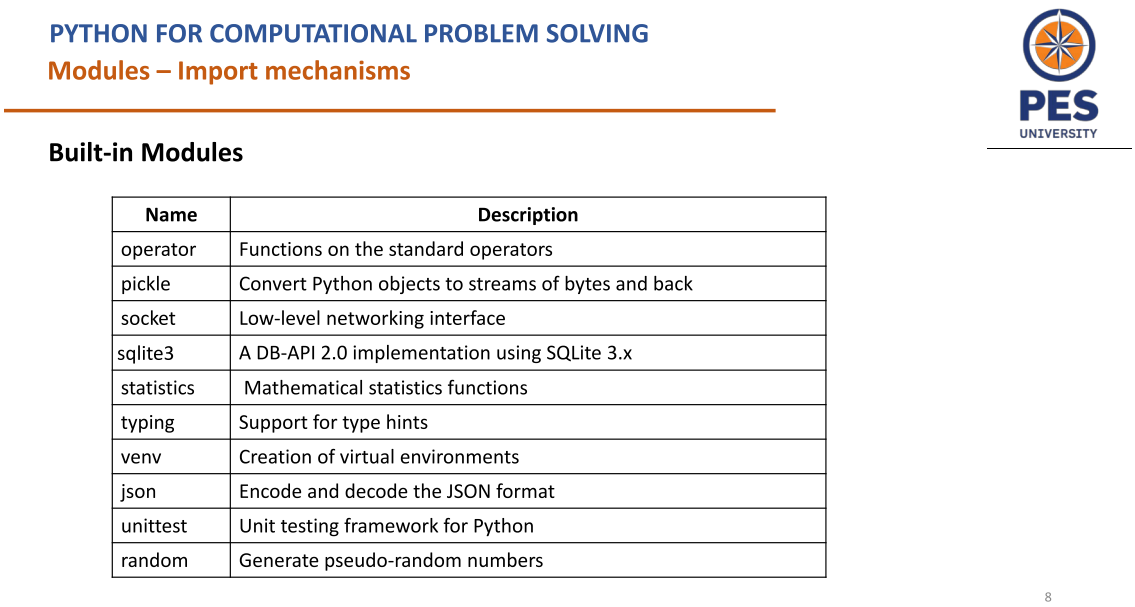


Figure G000010. UE23CS151A PCPS Unit 3 Sowmya Shree Sindhu Class 83 84 85 Modules · source slide page 10.

EV0000358

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Modules – Import mechanisms

PES UNIVERSITY

User-defined Modules

- Any file with .py extension and containing Python code is a module.
- It can contain definitions of one or more functions, variables, constants as well as classes.
- Any object from a module can be made available to interpreter or another Python script by using import statement.

9

Figure G000011. UE23CS151A PCPS Unit 3 Sowmya Shree Sindhu Class 83 84 85 Modules · source slide page 11.

EV0000361

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

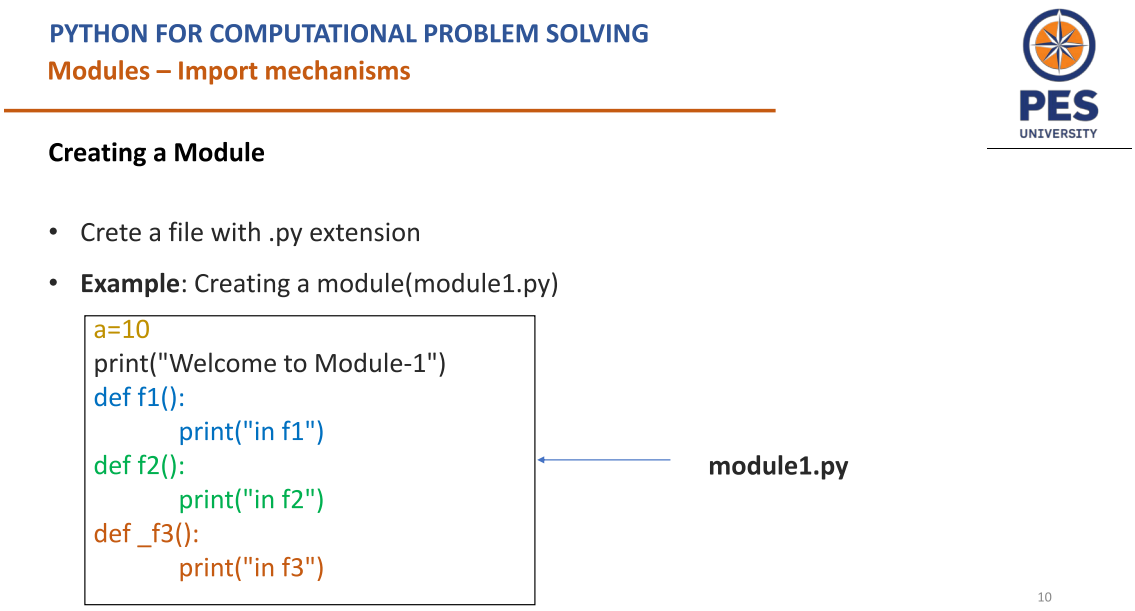


Figure G000012. UE23CS151A PCPS Unit 3 Sowmya Shree Sindhu Class 83 84 85 Modules · source slide page 12.

EV0000364

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Modules – Import mechanisms

PES UNIVERSITY

Import a Module

- import the functions, and classes defined in a module to another module
- When the interpreter encounters an import statement, it imports the module if the module is present. Otherwise, *ModuleNotFoundError* is thrown.
- Syntax - **import** *module_name*
  import – the keyword used to import the module
  module_name – name of the module to be imported
- To access the functions inside the module the dot(.) operator is used.

11

Figure G000013. UE23CS151A PCPS Unit 3 Sowmya Shree Sindhu Class 83 84 85 Modules · source slide page 13.

EV0000367

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

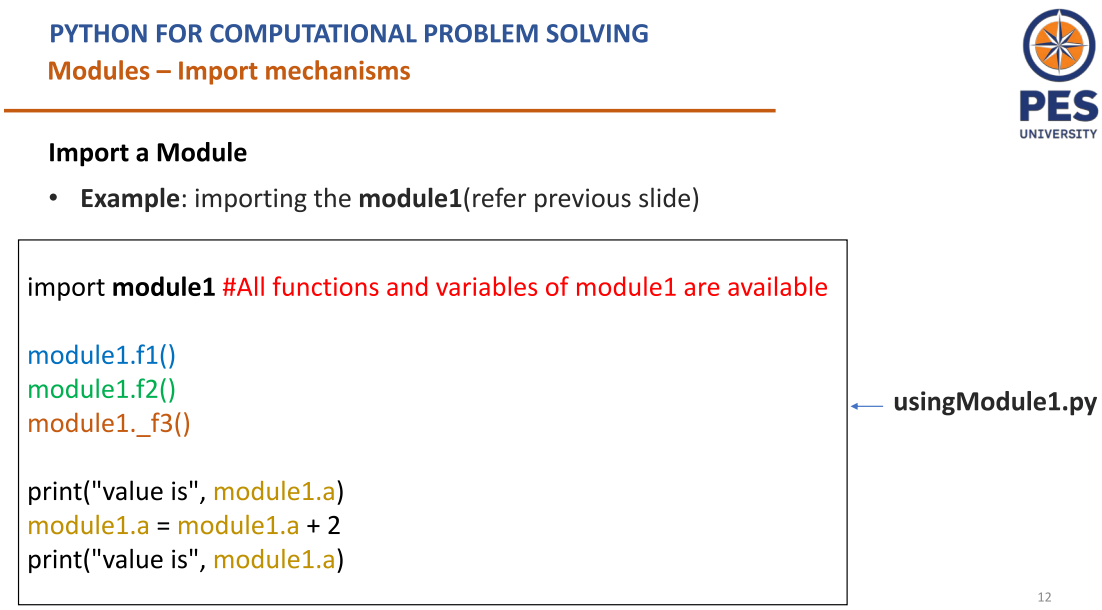


Figure G000014. UE23CS151A PCPS Unit 3 Sowmya Shree Sindhu Class 83 84 85 Modules · source slide page 14.

EV0000370

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

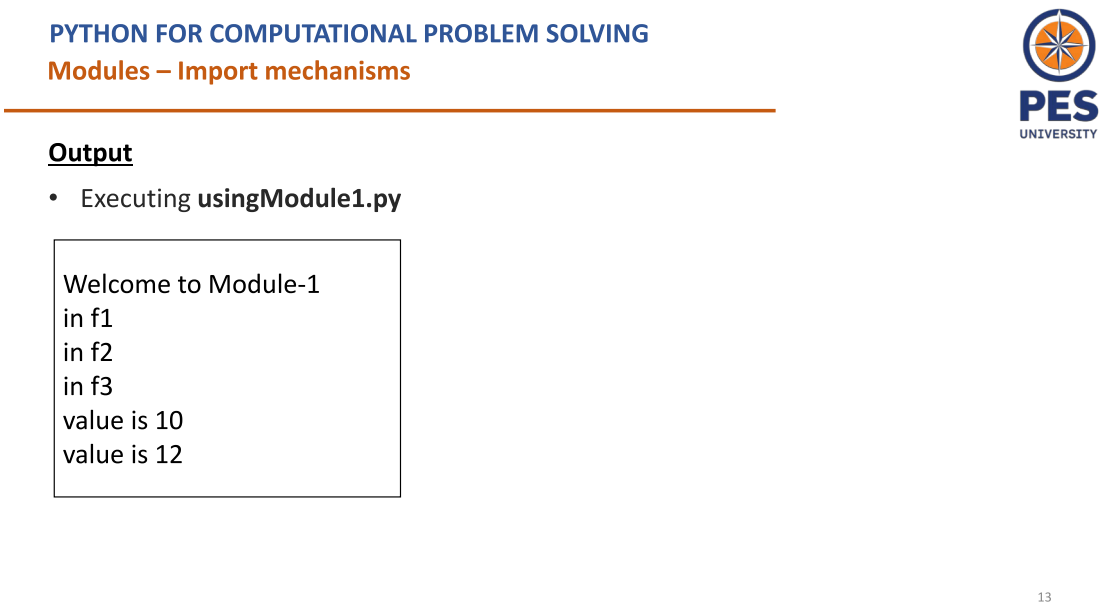


Figure G000015. UE23CS151A PCPS Unit 3 Sowmya Shree Sindhu Class 83 84 85 Modules · source slide page 15.

EV0000373

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

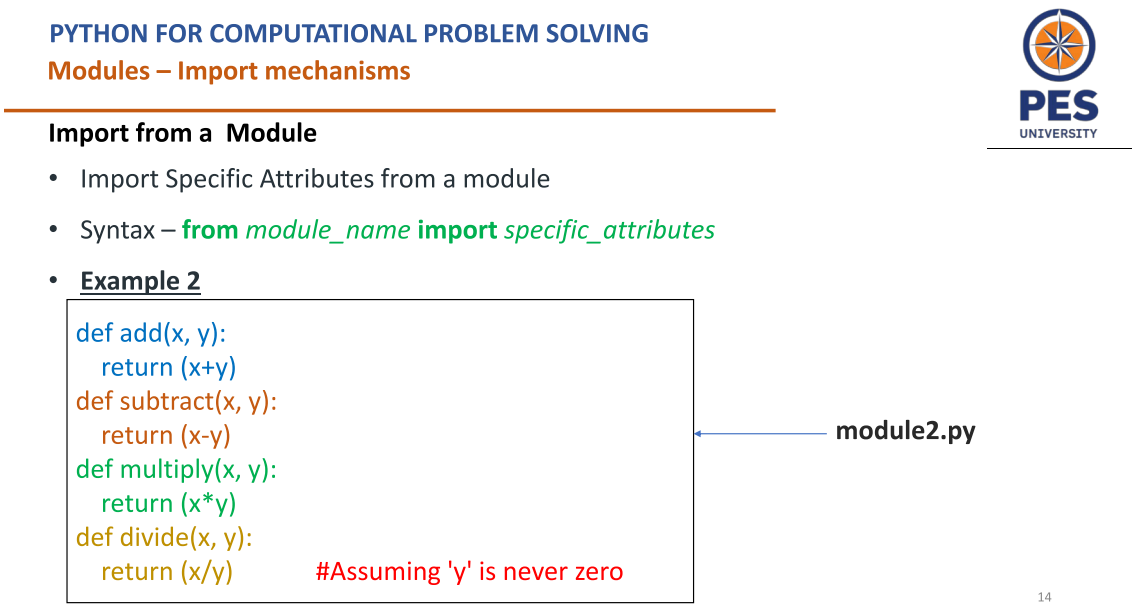


Figure G000016. UE23CS151A PCPS Unit 3 Sowmya Shree Sindhu Class 83 84 85 Modules · source slide page 16.

EV0000376

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

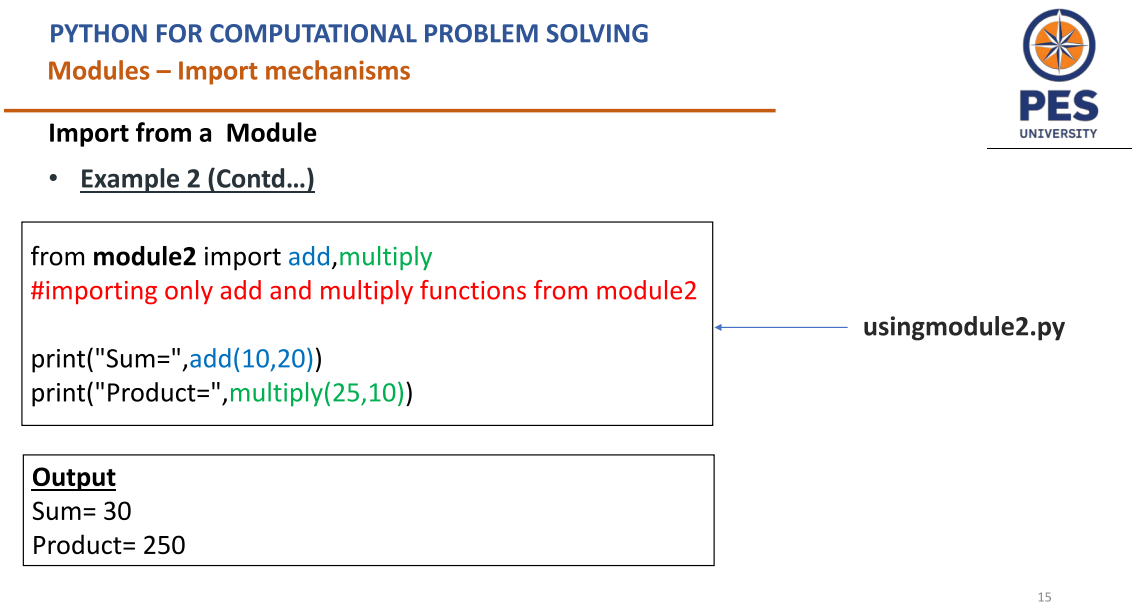


Figure G000017. UE23CS151A PCPS Unit 3 Sowmya Shree Sindhu Class 83 84 85 Modules · source slide page 17.

EV0000379

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

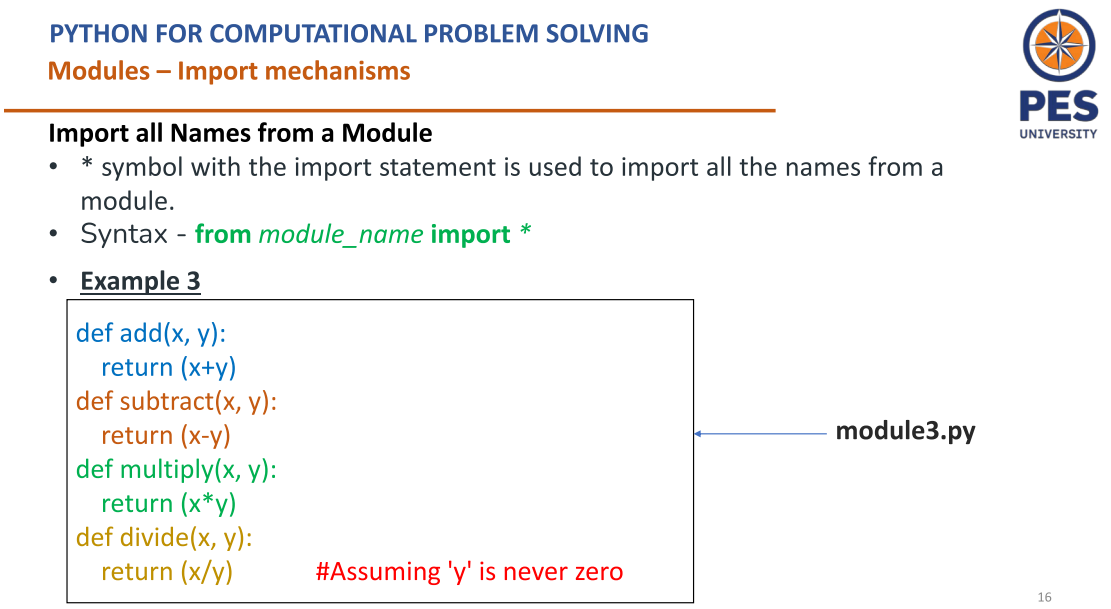


Figure G000018. UE23CS151A PCPS Unit 3 Sowmya Shree Sindhu Class 83 84 85 Modules · source slide page 18.

EV0000382

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

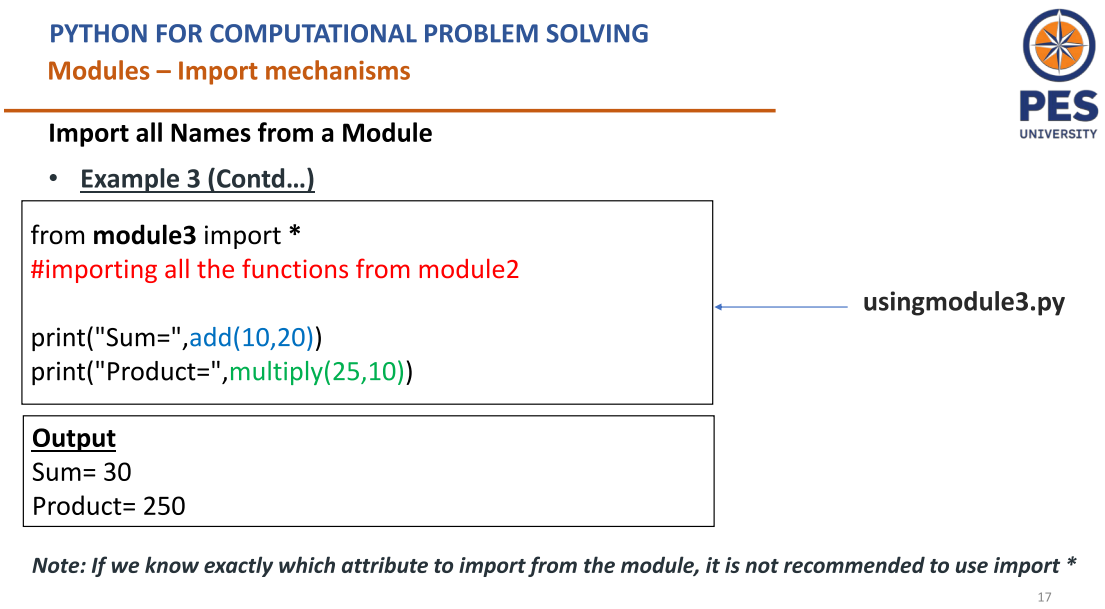


Figure G000019. UE23CS151A PCPS Unit 3 Sowmya Shree Sindhu Class 83 84 85 Modules · source slide page 19.

EV0000385

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

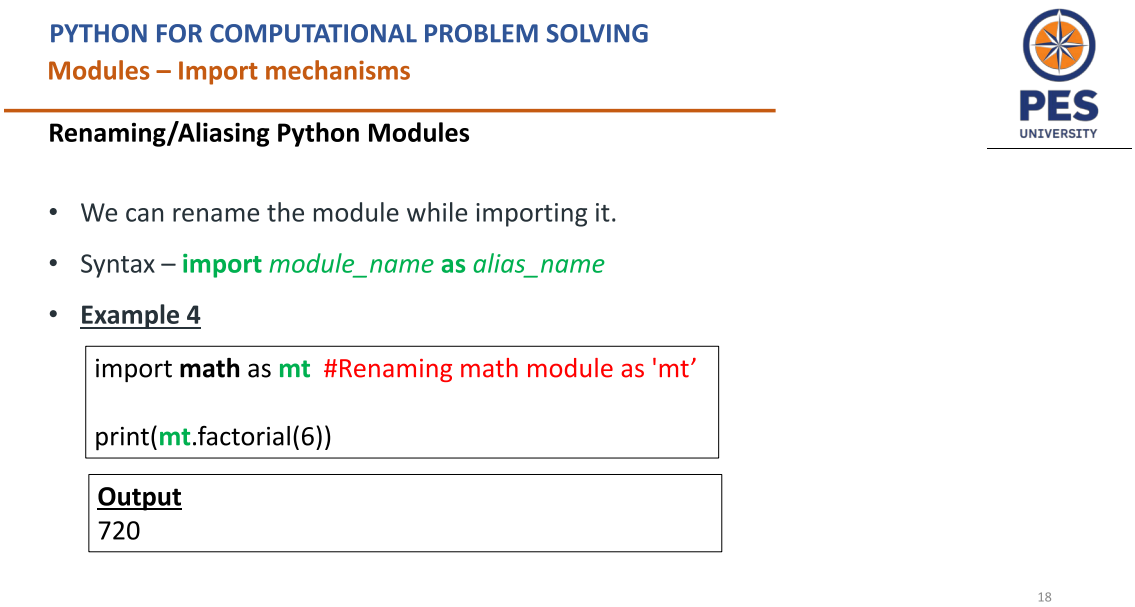


Figure G000020. UE23CS151A PCPS Unit 3 Sowmya Shree Sindhu Class 83 84 85 Modules · source slide page 20.

EV0000388

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

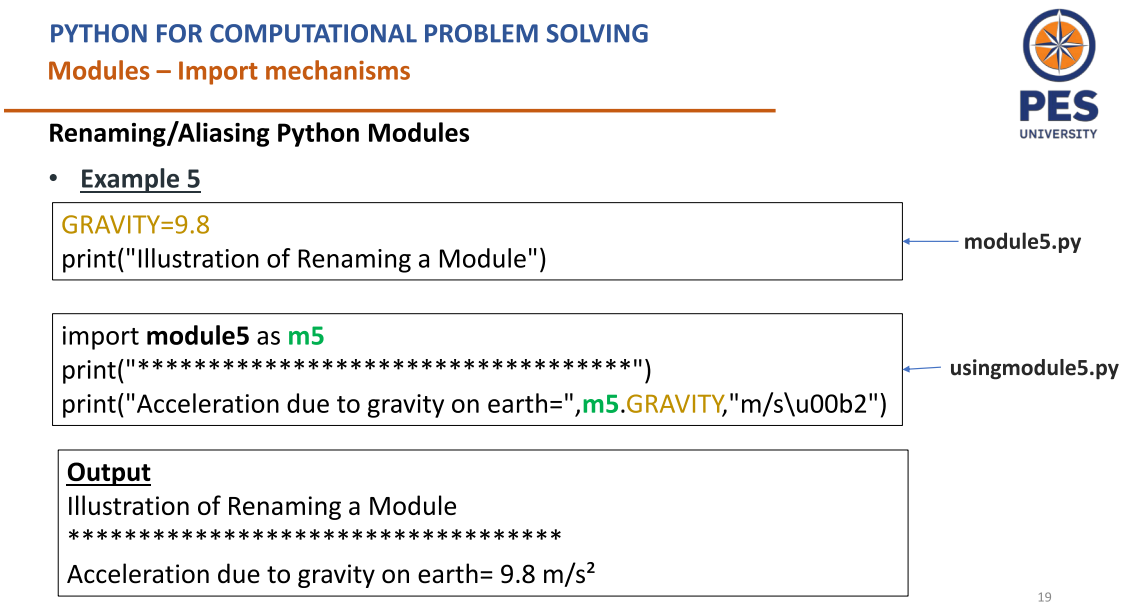


Figure G000021. UE23CS151A PCPS Unit 3 Sowmya Shree Sindhu Class 83 84 85 Modules · source slide page 21.

EV0000391

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

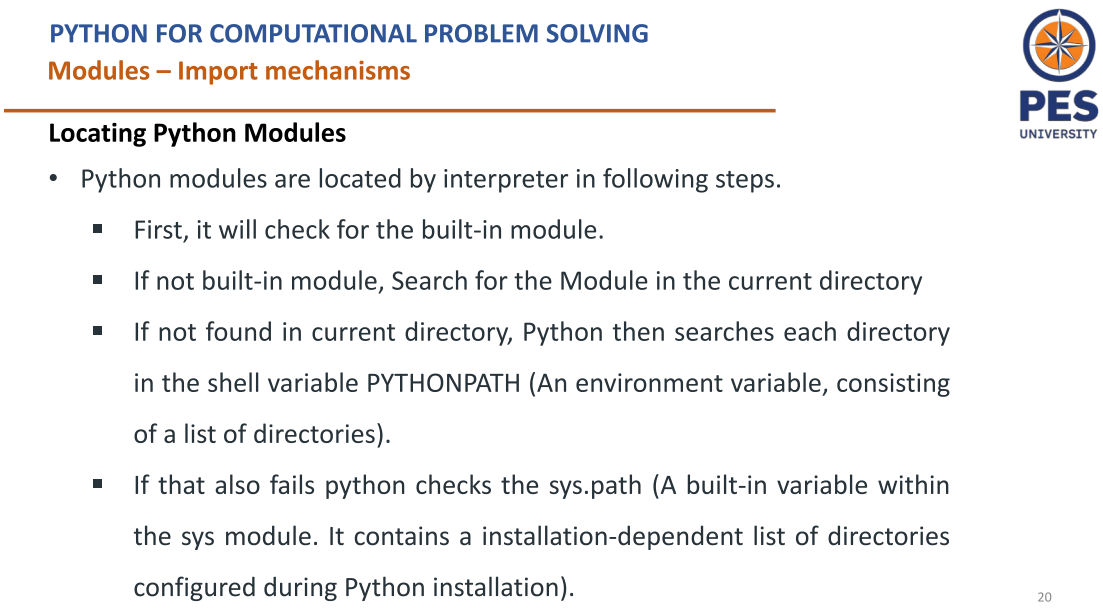


Figure G000022. UE23CS151A PCPS Unit 3 Sowmya Shree Sindhu Class 83 84 85 Modules · source slide page 22.

EV0000394

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

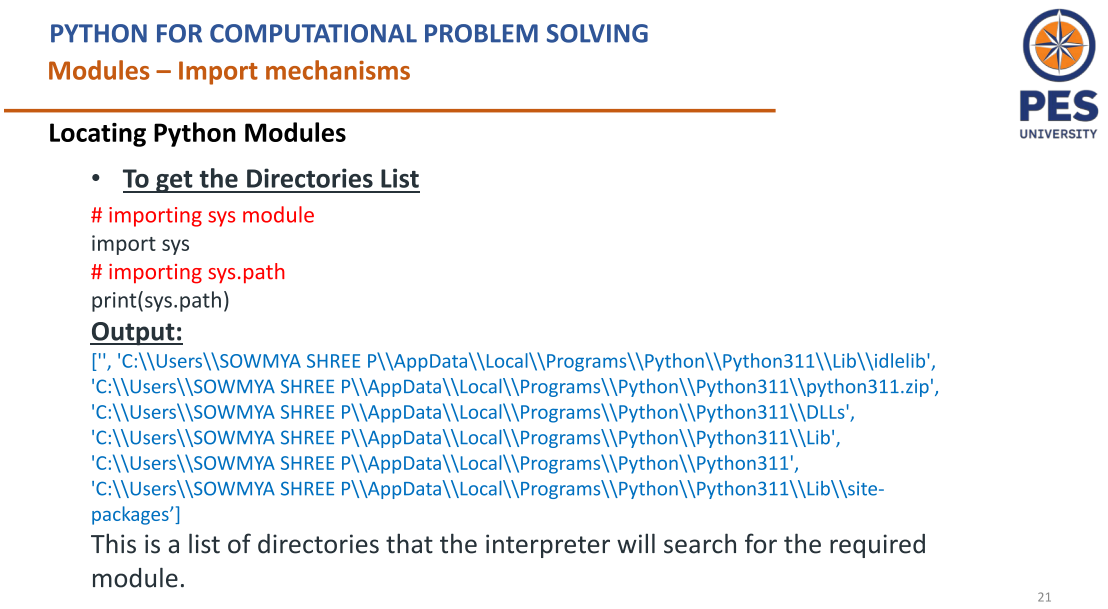


Figure G000023. UE23CS151A PCPS Unit 3 Sowmya Shree Sindhu Class 83 84 85 Modules · source slide page 23.

EV0000397

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

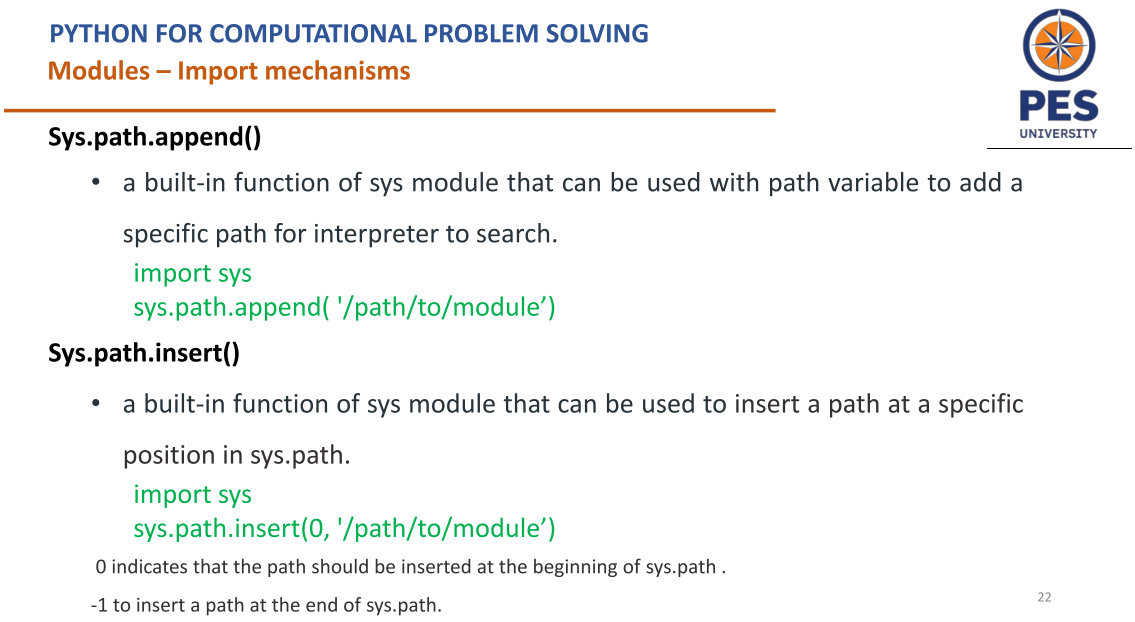


Figure G000024. UE23CS151A PCPS Unit 3 Sowmya Shree Sindhu Class 83 84 85 Modules · source slide page 24.

EV0000400

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

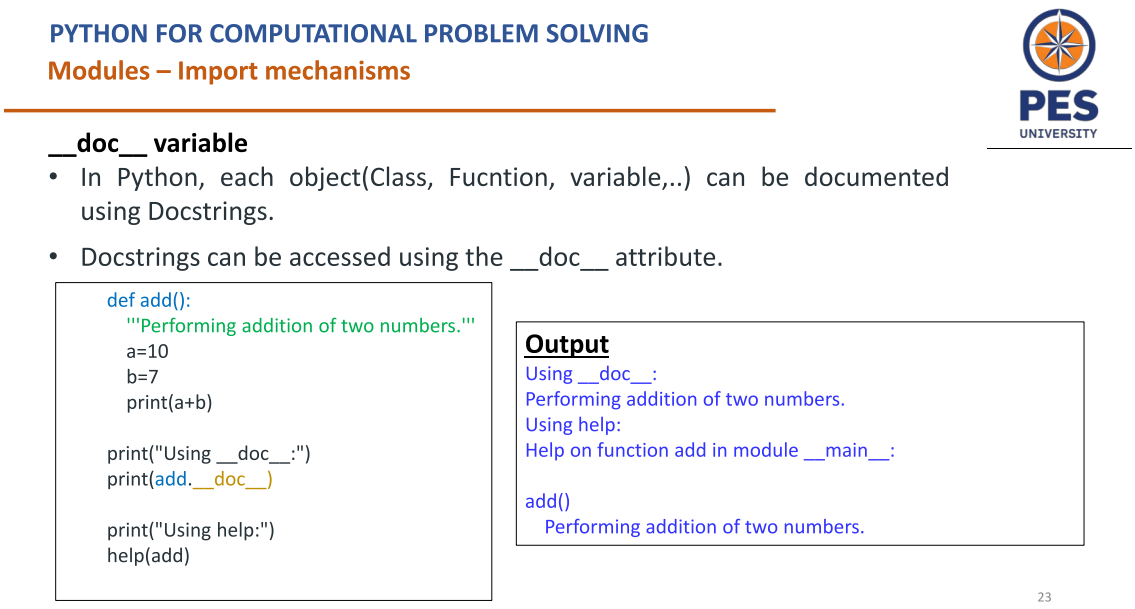


Figure G000025. UE23CS151A PCPS Unit 3 Sowmya Shree Sindhu Class 83 84 85 Modules · source slide page 25.

EV0000403

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

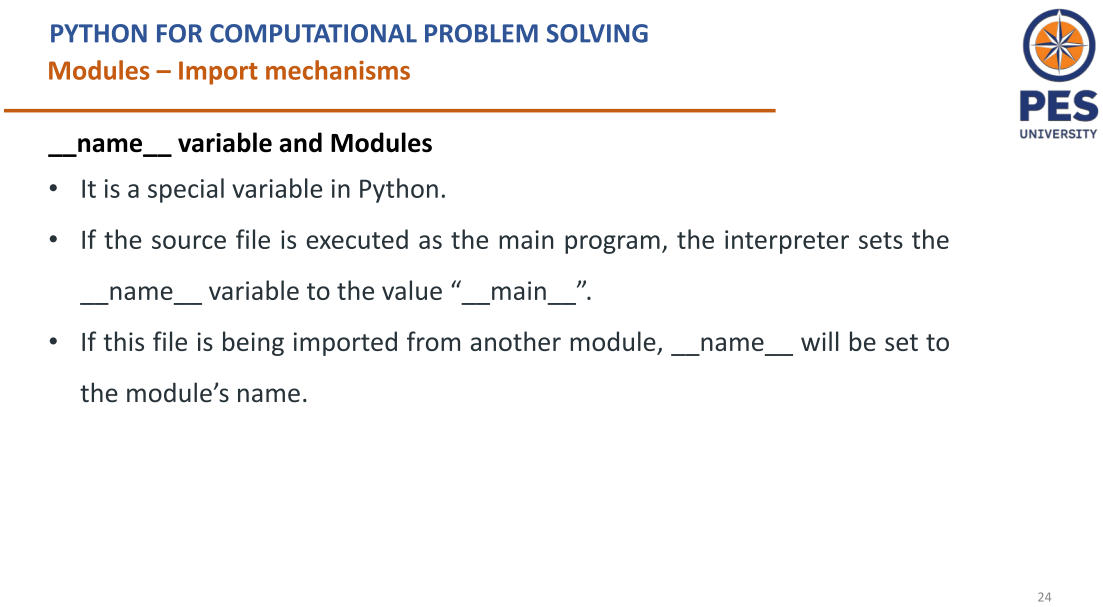


Figure G000026. UE23CS151A PCPS Unit 3 Sowmya Shree Sindhu Class 83 84 85 Modules · source slide page 26.

EV0000406

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

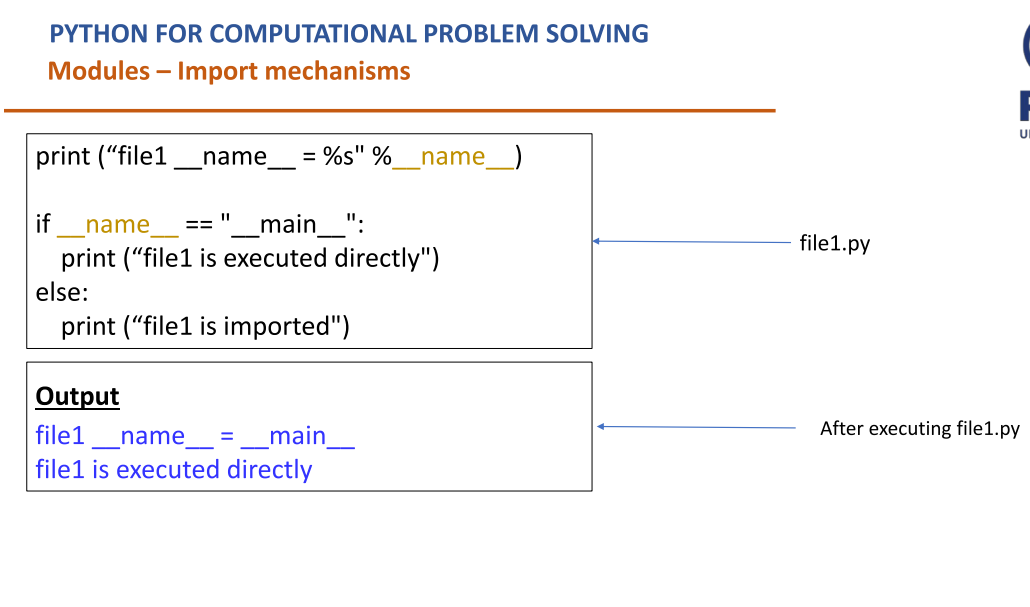


Figure G000027. UE23CS151A PCPS Unit 3 Sowmya Shree Sindhu Class 83 84 85 Modules · source slide page 27.

EV0000409

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

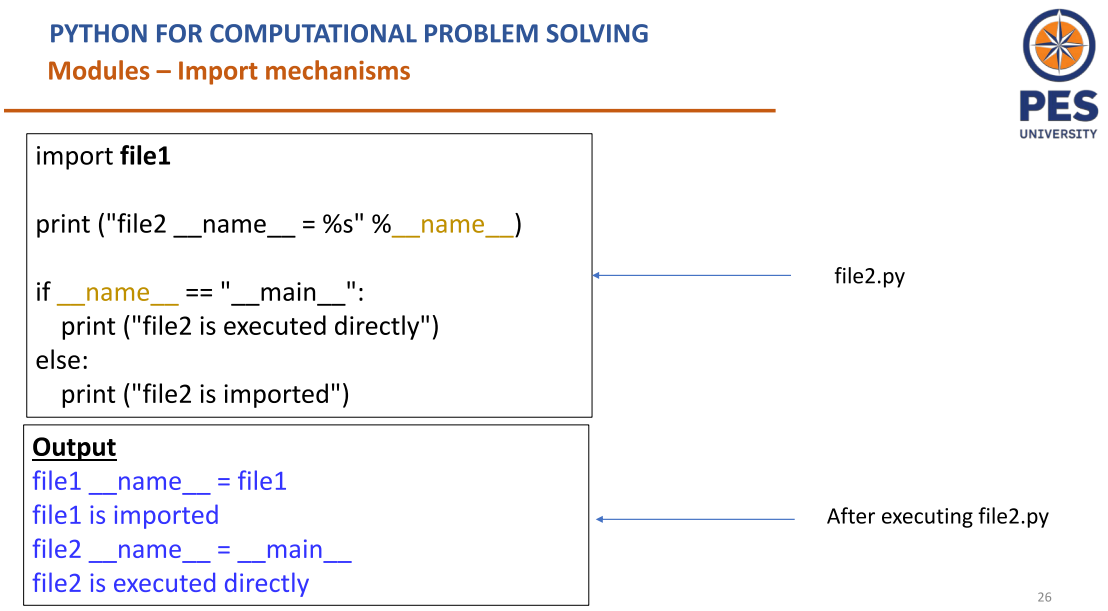


Figure G000028. UE23CS151A PCPS Unit 3 Sowmya Shree Sindhu Class 83 84 85 Modules · source slide page 28.

EV0000412

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

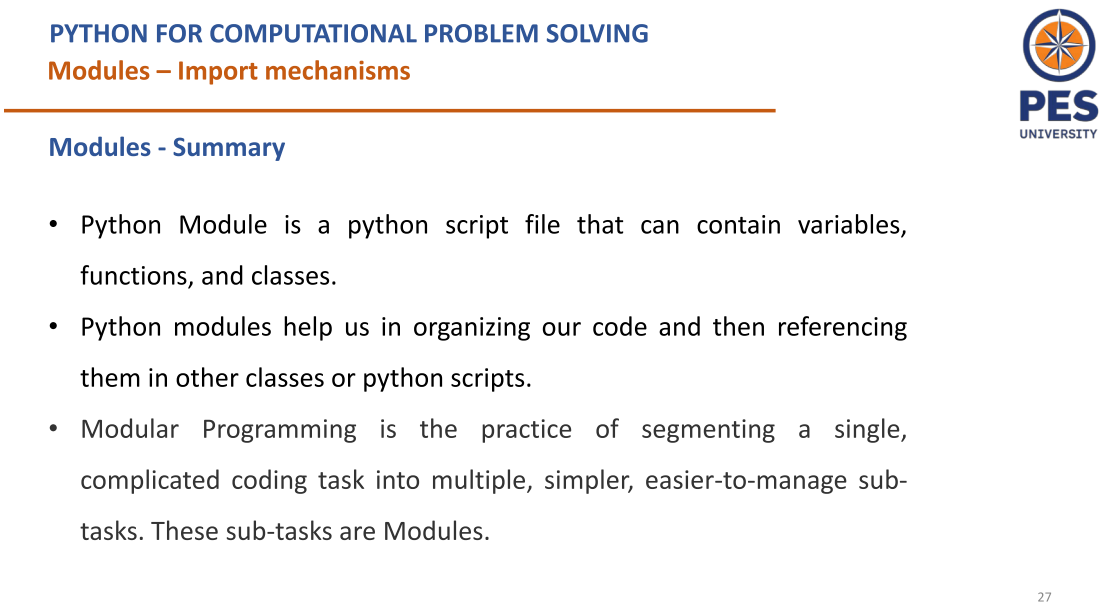


Figure G000029. UE23CS151A PCPS Unit 3 Sowmya Shree Sindhu Class 83 84 85 Modules · source slide page 29.

EV0000415

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Figure G000030. UE23CS151A PCPS Unit 3 Sowmya Shree Sindhu Class 83 84 85 Modules · source slide page 30.

EV0000418

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Important points and why

1. Source slide page 1 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Prof. Sowmya Shree P, Prof. Sindhu R Pai Department of Computer Science and Engineering EV0000330 · EV0000331 · EV0000332
2. Source slide page 2 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Unit - 3: Modules – Import Mechanisms Prof. Sowmya Shree P, Prof. Sindhu R Pai Department of Computer Science and Engineering EV0000333 · EV0000334 · EV0000335
3. Source slide page 3 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Modules – Import mechanisms Module • In Python, a module is a file that contains code. • It can include functions, classes, variables or any runnable code. • Basically, a module contains code to perform specific task. • We can use modules to separate codes in separate files as per their functionality. 1 EV0000336 · EV0000337 · EV0000338
4. Source slide page 4 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Modules – Import mechanisms Advantages of Modules • Reusability - makes the code reusable • Modularity – Organizing the code into modules logically • Separate scopes – separate namespace is defined by the module • Grouping - Python modules help us to organize and group the content by using files and folders 2 EV0000339 · EV0000340 · EV0000341

5. Source slide page 5 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Modules – Import mechanisms Need for Modules • While working on coding, We need to use many classes, variables, functions. • If we include everything in a single file, the program may become large. • To reduce size of the code, we can group together some similar functions, classes into a collection. • That collection is nothing but modules in python. 3 EV0000342 · EV0000343 · EV0000344
6. Source slide page 6 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Modules – Import mechanisms Packages and Namespace • Modules in Python can be grouped together in packages. • Packages organize the code into logical groups and provide a namespace to the modules so that they don't conflict with modules with the same name in other packages. • The import statement can also be used to import modules from a package, which allows to access the functions and classes defined in the package's modules. 4 EV0000345 · EV0000346 · EV0000347
7. Source slide page 7 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Modules – Import mechanisms Packages vs. Modules vs. Libraries • Modules – contain several functions, variables, classes etc. • Packages – contain several modules. - folder that contains various modules as files. • Library - a collection of packages and modules used to access built-in functionality 5 EV0000348 · EV0000349 · EV0000350
8. Source slide page 8 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Modules – Import mechanisms Types of Modules 1. Built-in Modules • Python's standard library comes bundled with a large number of modules. • They are called built-in modules. 2. User-defined Modules • Any file with .py extension and containing Python code is basically a module. • It can contain definitions of one or more functions, variables, constants ,classes. 6 EV0000351 · EV0000352 · EV0000353
9. Source slide page 9 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Modules – Import mechanisms Built-in Modules Name Description os provides a unified interface to a number of operating system functions string contains a number of functions for string processing re regular expression functionalities math a number of mathematical operations cmath a number of mathematical operations for complex numbers datetime functions to deal with dates and the time gc an interface to the built-in garbage collector asyncio functionality required for asynchronous processing collections advanced Container datatypes functools Higher-order functions and operations on callable objects 7 EV0000354 · EV0000355 · EV0000356
10. Source slide page 10 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Modules – Import mechanisms Built-in Modules Name Description operator Functions on the standard operators pickle Convert Python objects to streams of bytes and back socket Low-level networking interface sqlite3 A DB-API 2.0 implementation using SQLite 3.x statistics Mathematical statistics functions typing Support for type hints venv Creation of virtual environments json Encode and decode the JSON format unittest Unit testing framework for Python random Generate pseudo-random numbers 8 EV0000357 · EV0000358 · EV0000359
11. Source slide page 11 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Modules – Import mechanisms User-defined Modules • Any file with .py extension and containing Python code is a module. • It can contain definitions of one or more functions, variables, constants as well as classes. • Any object from a module can be made available to interpreter or another Python script by using import statement. 9 EV0000360 · EV0000361 · EV0000362

12. Source slide page 12 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Modules – Import mechanisms Creating a Module • Crete a file with .py extension • Example: Creating a module(module1.py) a=10 print("Welcome to Module-1") def f1(): print("in f1") def f2(): module1.py print("in f2") def _f3(): print("in f3") 10 EV0000363 · EV0000364 · EV0000365
13. Source slide page 13 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Modules – Import mechanisms Import a Module • import the functions, and classes defined in a module to another module • When the interpreter encounters an import statement, it imports the module if the module is present. Otherwise, ModuleNotFoundError is thrown. • Syntax - import module_name import – the keyword used to import the module module_name – name of the module to be imported • To access the functions inside the module the dot(.) operator is used. 11 EV0000366 · EV0000367 · EV0000368
14. Source slide page 14 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Modules – Import mechanisms Import a Module • Example: importing the module1(refer previous slide) import module1 #All functions and variables of module1 are available module1.f1() module1.f2() usingModule1.py module1._f3() print("value is", module1.a) module1.a = module1.a + 2 print("value is", module1.a) 12 EV0000369 · EV0000370 · EV0000371
15. Source slide page 15 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Modules – Import mechanisms Output • Executing usingModule1.py Welcome to Module-1 in f1 in f2 in f3 value is 10 value is 12 13 EV0000372 · EV0000373 · EV0000374
16. Source slide page 16 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Modules – Import mechanisms Import from a Module • Import Specific Attributes from a module • Syntax – from module_name import specific_attributes • Example 2 def add(x, y): return (x+y) def subtract(x, y): module2.py return (x-y) def multiply(x, y): return (x*y) def divide(x, y): return (x/y) #Assuming 'y' is never zero 14 EV0000375 · EV0000376 · EV0000377
17. Source slide page 17 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Modules – Import mechanisms Import from a Module • Example 2 (Contd...) from module2 import add,multiply #importing only add and multiply functions from module2 usingmodule2.py print("Sum=",add(10,20)) print("Product=",multiply(25,10)) Output Sum= 30 Product= 250 15 EV0000378 · EV0000379 · EV0000380
18. Source slide page 18 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Modules – Import mechanisms Import all Names from a Module • * symbol with the import statement is used to import all the names from a module. • Syntax - from module_name import * • Example 3 def add(x, y): return (x+y) def subtract(x, y): module3.py return (x-y) def multiply(x, y): return (x*y) def divide(x, y): return (x/y) #Assuming 'y' is never zero 16 EV0000381 · EV0000382 · EV0000383
19. Source slide page 19 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Modules – Import mechanisms Import all Names from a Module • Example 3 (Contd...) from module3 import * #importing all the functions from module2 usingmodule3.py print("Sum=",add(10,20)) print("Product=",multiply(25,10)) Output Sum= 30 Product= 250 Note: If we know exactly which attribute to import from the module, it is not recommended to use import * 17 EV0000384 · EV0000385 · EV0000386

20. Source slide page 20 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Modules – Import mechanisms Renaming/Aliasing Python Modules • We can rename the module while importing it. • Syntax – import module_name as alias_name • Example 4 import math as mt #Renaming math module as 'mt' print(mt.factorial(6)) Output 720 18 EV0000387 · EV0000388 · EV0000389

21. Source slide page 21 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Modules – Import mechanisms Renaming/Aliasing Python Modules • Example 5 GRAVITY=9.8 module5.py print("Illustration of Renaming a Module") import module5 as m5 print("**********") usingmodule5.py print("Acceleration due to gravity on earth=",m5.GRAVITY,"m/s\u00b2") Output Illustration of Renaming a Module ********** Acceleration due to gravity on earth= 9.8 m/s² 19 EV0000390 · EV0000391 · EV0000392

22. Source slide page 22 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Modules – Import mechanisms Locating Python Modules • Python modules are located by interpreter in following steps. ■ First, it will check for the built-in module. ■ If not built-in module, Search for the Module in the current directory ■ If not found in current directory, Python then searches each directory in the shell variable PYTHONPATH (An environment variable, consisting of a list of directories). ■ If that also fails python checks the sys.path (A built-in variable within the sys module. It contains a installation-dependent list of directories configured during Python installation). 20 EV0000393 · EV0000394 · EV0000395

23. Source slide page 23 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Modules – Import mechanisms Locating Python Modules • To get the Directories List # importing sys module import sys # importing sys.path print(sys.path) Output: ['', 'C:\\Users\\SOWMYA SHREE P\\AppData\\Local\\Programs\\Python\\Python311\\Lib\\idlelib', 'C:\\Users\\SOWMYA SHREE P\\AppData\\Local\\Programs\\Python\\Python311\\python311.zip', 'C:\\Users\\SOWMYA SHREE P\\AppData\\Local\\Programs\\Python\\Python311\\DLLs', 'C:\\Users\\SOWMYA SHREE P\\AppData\\Local\\Programs\\Python\\Python311\\Lib', 'C:\\Users\\SOWMYA SHREE P\\AppData\\Local\\Programs\\Python\\Python311', 'C:\\Users\\SOWMYA SHREE P\\AppData\\Local\\Programs\\Python\\Python311\\Lib\\site- packages'] This is a list of directories that the interpreter will search for the required module. 21 EV0000396 · EV0000397 · EV0000398

24. Source slide page 24 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Modules – Import mechanisms Sys.path.append() • a built-in function of sys module that can be used with path variable to add a specific path for interpreter to search. import sys sys.path.append('/path/to/module') Sys.path.insert() • a built-in function of sys module that can be used to insert a path at a specific position in sys.path. import sys sys.path.insert(0, '/path/to/module') 0 indicates that the path should be inserted at the beginning of sys.path . 22 -1 to insert a path at the end of sys.path. EV0000399 · EV0000400 · EV0000401

25. Source slide page 25 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Modules – Import mechanisms __doc__ variable • In Python, each object(Class, Fucntion, variable,..) can be documented using Docstrings. • Docstrings can be accessed using the __doc__ attribute. def add(): '''Performing addition of two numbers.''' a=10 Output b=7 Using __doc__: print(a+b) Performing addition of two numbers. Using help: print("Using __doc__:") Help on function add in module __main__: print(add.__doc__) add() print("Using help:") Performing addition of two numbers. help(add) 23 EV0000402 · EV0000403 · EV0000404
26. Source slide page 26 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Modules – Import mechanisms __name__ variable and Modules • It is a special variable in Python. • If the source file is executed as the main program, the interpreter sets the __name__ variable to the value "__main__". • If this file is being imported from another module, __name__ will be set to the module's name. 24 EV0000405 · EV0000406 · EV0000407
27. Source slide page 27 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Modules – Import mechanisms print ("file1 __name__ = %s" %__name__) if __name__ == "__main__": file1.py print ("file1 is executed directly") else: print ("file1 is imported") Output After executing file1.pyfile1 __name__ = __main__ file1 is executed directly 25 EV0000408 · EV0000409 · EV0000410
28. Source slide page 28 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Modules – Import mechanisms import file1 print ("file2 __name__ = %s" %__name__) file2.py if __name__ == "__main__": print ("file2 is executed directly") else: print ("file2 is imported") Output file1 __name__ = file1 file1 is imported After executing file2.py file2 __name__ = __main__ file2 is executed directly 26 EV0000411 · EV0000412 · EV0000413
29. Source slide page 29 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Modules – Import mechanisms Modules - Summary • Python Module is a python script file that can contain variables, functions, and classes. • Python modules help us in organizing our code and then referencing them in other classes or python scripts. • Modular Programming is the practice of segmenting a single, complicated coding task into multiple, simpler, easier-to-manage sub- tasks. These sub-tasks are Modules. 27 EV0000414 · EV0000415 · EV0000416
30. Source slide page 30 (extracted text): THANK YOU Department of Computer Science and Engineering Contact Email ID's: sindhurpai@pes.edu sowmyashree@pes.edu EV0000417 · EV0000418 · EV0000419

SHORT-NOTE RECAP

Source deck preserved: UE23CS151A PCPS Unit 3 Sowmya Shree Sindhu Class 83 84 85 Modules. Its extracted page text is listed as searchable points and all rendered source pages are attached as figures. Match to a specific video is candidate-based; this index does not claim a manual content review.

EV0000330

24 Source slide deck: UE24CS151A PCPS Unit 1 SindhuRPai Class 6 Intro to Programming Languages

This entry preserves the complete slide deck titled UE24CS151A PCPS Unit 1 SindhuRPai Class 6 Intro to Programming Languages as searchable extracted page text and page-faithful rendered images. Its association with a particular video is not confirmed; it remains a separate topic under Python for computational Problem Solving, Unit_01, to avoid an unsupported match. EV0000630 · EV0000631

Graphs and diagrams

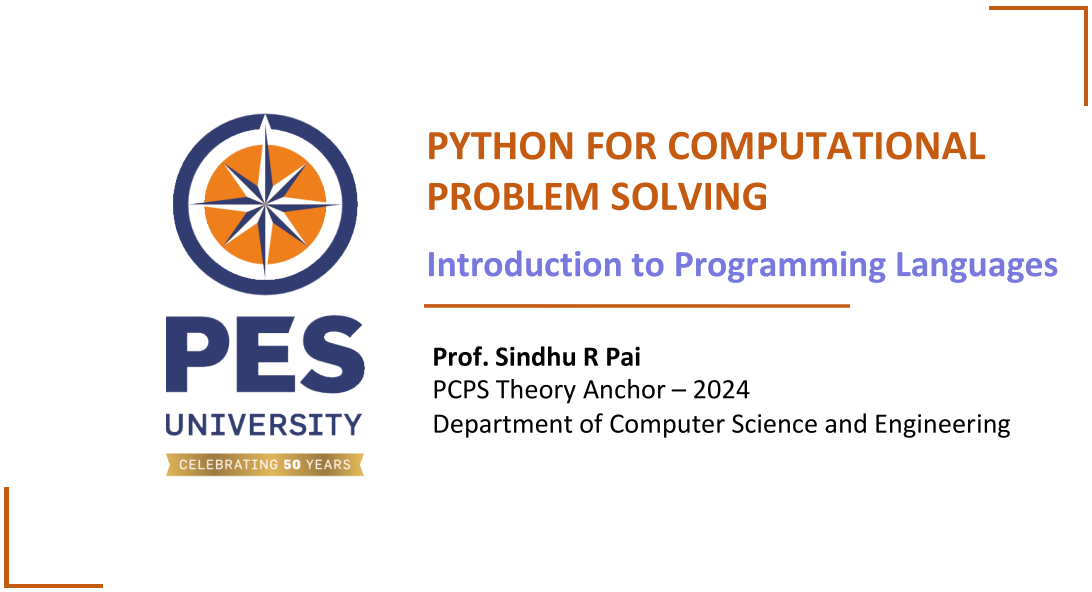


Figure G000101. UE24CS151A PCPS Unit 1 SindhuRPai Class 6 Intro to Programming Languages · source slide page 1.

EV0000631

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

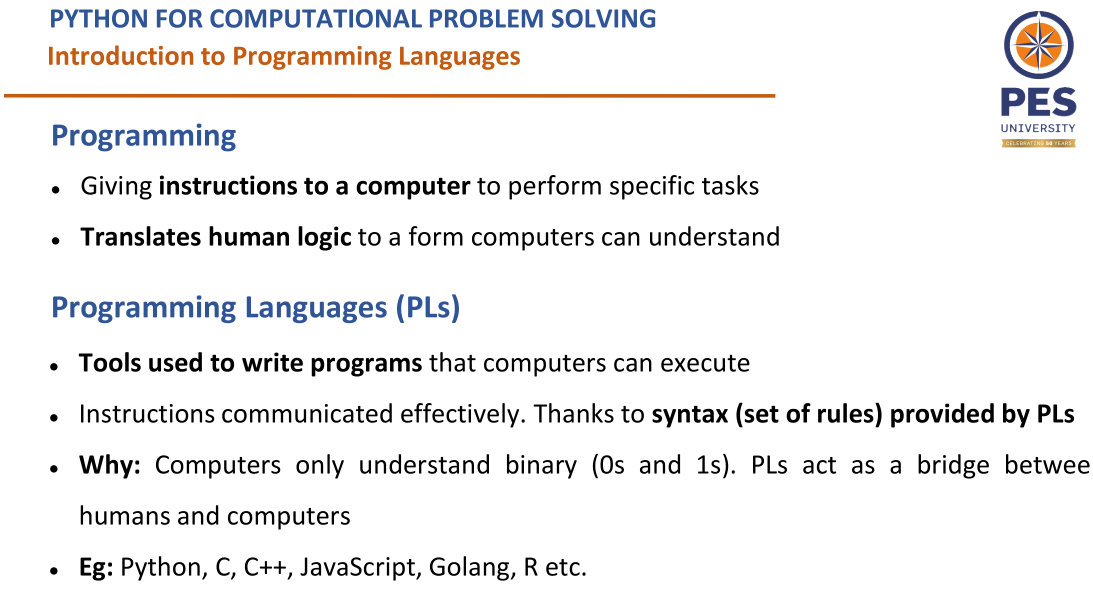


Figure G000102. UE24CS151A PCPS Unit 1 SindhuRPai Class 6 Intro to Programming Languages · source slide page 2.

EV0000634

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING
Introduction to Programming Languages

PES UNIVERSITY

Why so many programming languages?

- **Different Needs:** Each language is **designed to solve specific types of problems** or target different applications (e.g., web development, system programming, data analysis)
- **Evolving Technology:** As technology changes, **new languages emerge** to meet modern requirements, offer better performance, or provide new features.
- **Efficiency and Flexibility:** Some languages are **better suited for certain tasks**. For example, Python is great for rapid development, while C is ideal for performance-critical systems.

Figure G000103. UE24CS151A PCPS Unit 1 SindhuRPai Class 6 Intro to Programming Languages · source slide page 3.

EV0000637

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

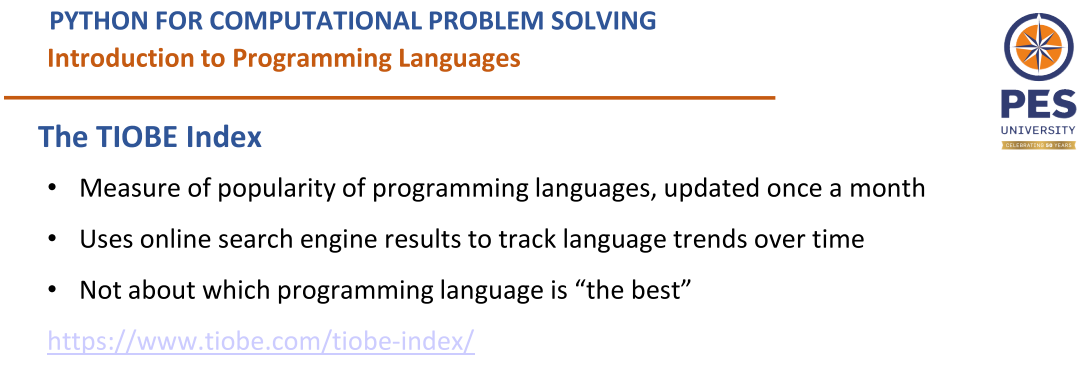


Figure G000104. UE24CS151A PCPS Unit 1 SindhuRPai Class 6 Intro to Programming Languages · source slide page 4.

EV0000640

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Figure G000105. UE24CS151A PCPS Unit 1 SindhuRPai Class 6 Intro to Programming Languages · source slide page 5.

EV0000643

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING
Introduction to Programming Languages

PES UNIVERSITY

Types of Programming Languages

- **Low Level:** Directly understood by machine. Much harder to code with, but has the fastest performance.
Eg: Machine language

- **Middle Level:** Provides some basic data structures and definitions while still maintaining direct interaction with the machine. Still somewhat hard to code with, but has fast performance.
Eg: C, C++

- **High Level:** Focuses on ease of use and provides many programming structures by default. Worse performance but very easy to code and develop with.
Eg: Python, JavaScript

Figure G000106. UE24CS151A PCPS Unit 1 SindhuRPai Class 6 Intro to Programming Languages · source slide page 6.

EV0000646

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

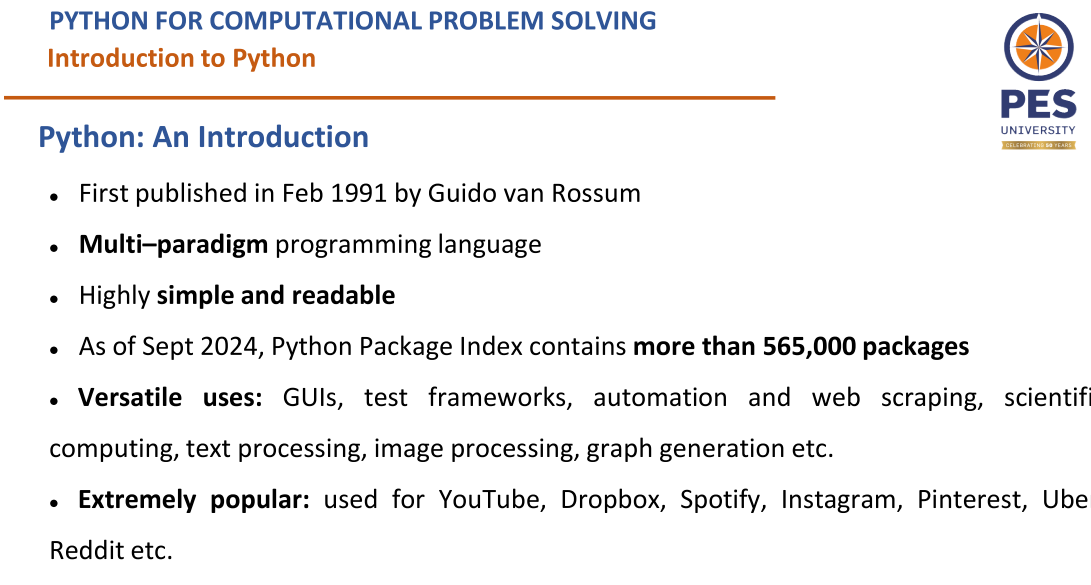


Figure G000107. UE24CS151A PCPS Unit 1 SindhuRPai Class 6 Intro to Programming Languages · source slide page 7.

EV0000649

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

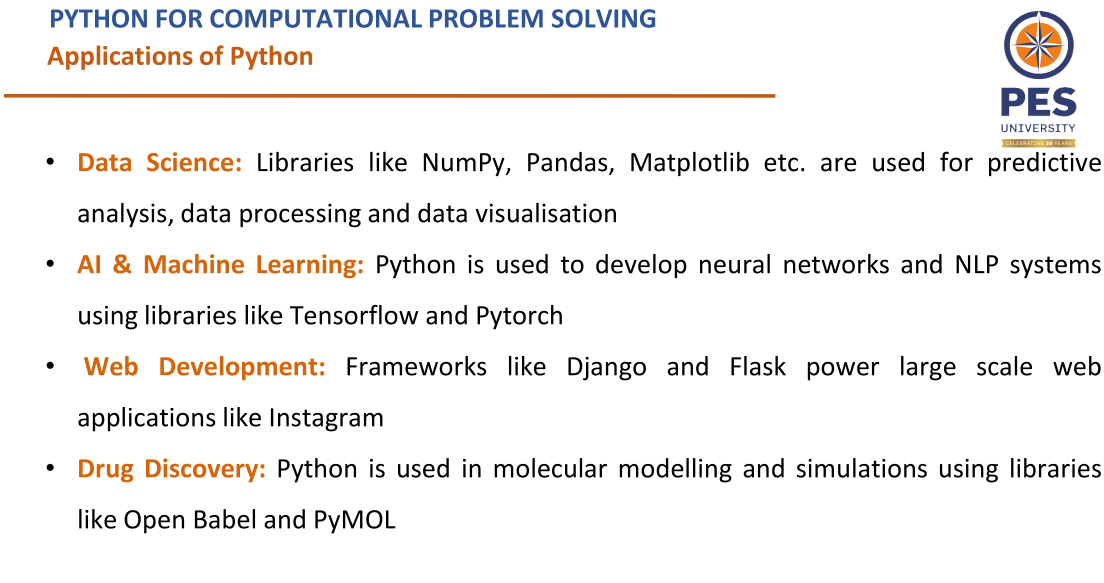


Figure G000108. UE24CS151A PCPS Unit 1 SindhuRPai Class 6 Intro to Programming Languages · source slide page 8.

EV0000652

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING
Applications of Python

PES UNIVERSITY

Applications of Python

- **IoT and Embedded Systems:** Integrations with platforms like Raspberry Pi and Arduino to control hardware devices
- **Circuit Design and Simulation:** Tools like PySpice, SKiDL are used for circuit simulation and PCB design
- **Computational Fluid Dynamics (CFD):** Libraries such as OpenFOAM are used for simulating fluid flow
- **Structural Analysis:** Frameworks like OpenSeesPy are used for structural modelling and earthquake engineering

Figure G000109. UE24CS151A PCPS Unit 1 SindhuRPai Class 6 Intro to Programming Languages · source slide page 9.

EV0000655

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

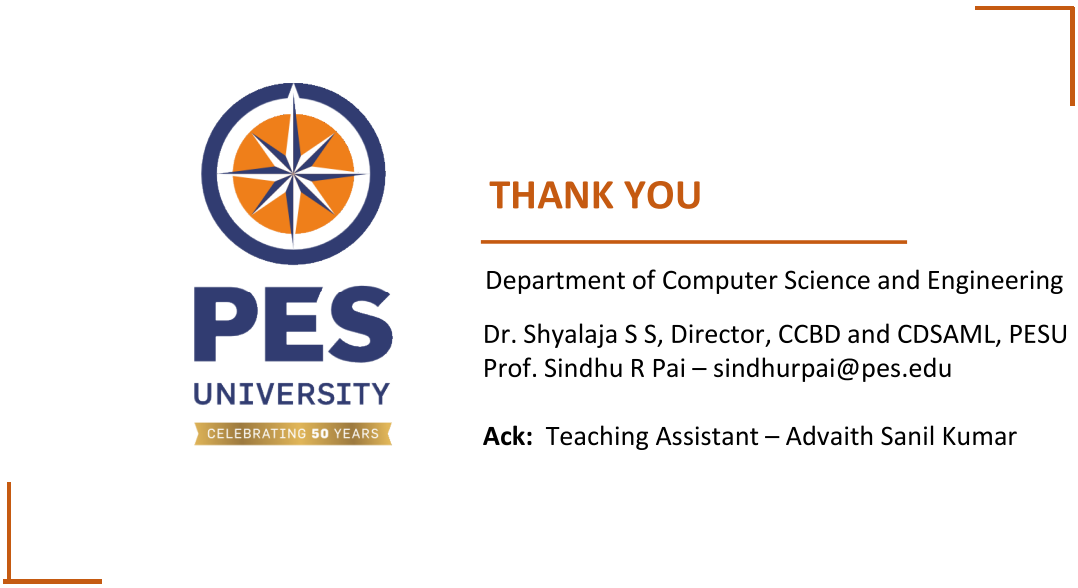


Figure G000110. UE24CS151A PCPS Unit 1 SindhuRPai Class 6 Intro to Programming Languages · source slide page 10.

EV0000658

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Important points and why

1. Source slide page 1 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Introduction to Programming Languages Prof. Sindhu R Pai PCPS Theory Anchor – 2024 Department of Computer Science and Engineering EV0000630 · EV0000631 · EV0000632
2. Source slide page 2 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Introduction to Programming Languages Programming ☒Giving instructions to a computer to perform specific tasks ☒Translates human logic to a form computers can understand Programming Languages (PLs) ☒Tools used to write programs that computers can execute ☒Instructions communicated effectively. Thanks to syntax (set of rules) provided by PLs ☒Why: Computers only understand binary (0s and 1s). PLs act as a bridge between humans and computers ☒Eg: Python, C, C++, JavaScript, Golang, R etc. EV0000633 · EV0000634 · EV0000635

3. Source slide page 3 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Introduction to Programming Languages Why so many programming languages? • Different Needs: Each language is designed to solve specific types of problems or target different applications (e.g., web development, system programming, data analysis) • Evolving Technology: As technology changes, new languages emerge to meet modern requirements, offer better performance, or provide new features. • Efficiency and Flexibility: Some languages are better suited for certain tasks. For example, Python is great for rapid development, while C is ideal for performance-critical systems. EV0000636 · EV0000637 · EV0000638
4. Source slide page 4 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Introduction to Programming Languages The TIOBE Index • Measure of popularity of programming languages, updated once a month • Uses online search engine results to track language trends over time • NotNot aboutabout whichwhich programmingprogramming languagelanguage isis “thethe best”best https://www.tiobe.com/tiobe-index/ EV0000639 · EV0000640 · EV0000641
5. Source slide page 5 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Introduction to Programming Languages The TIOBE Index (as of September 2024) EV0000642 · EV0000643 · EV0000644
6. Source slide page 6 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Introduction to Programming Languages Types of Programming Languages • Low Level: Directly understood by machine. Much harder to code with, but has the fastest performance. Eg: Machine language • Middle Level: Provides some basic data structures and definitions while still maintaining direct interaction with the machine. Still somewhat hard to code with, but has fast performance. Eg: C, C++ • High Level: Focuses on ease of use and provides many programming structures by default. Worse performance but very easy to code and develop with. Eg: Python, JavaScript EV0000645 · EV0000646 · EV0000647
7. Source slide page 7 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Introduction to Python Python: An Introduction ☒First published in Feb 1991 by Guido van Rossum ☒Multi–paradigm programming language ☒HighlyHighly simplesimple andand readablereadable ☒As of Sept 2024, Python Package Index contains more than 565,000 packages ☒Versatile uses: GUIs, test frameworks, automation and web scraping, scientific computing, text processing, image processing, graph generation etc. ☒Extremely popular: used for YouTube, Dropbox, Spotify, Instagram, Pinterest, Uber, Reddit etc. EV0000648 · EV0000649 · EV0000650
8. Source slide page 8 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Applications of Python • Data Science: Libraries like NumPy, Pandas, Matplotlib etc. are used for predictive analysis, data processing and data visualisation • AI & Machine Learning: Python is used to develop neural networks and NLP systems using libraries like Tensorflow and Pytorch • Web Development: Frameworks like Django and Flask power large scale web applications like Instagram • Drug Discovery: Python is used in molecular modelling and simulations using libraries like Open Babel and PyMOL EV0000651 · EV0000652 · EV0000653

9. Source slide page 9 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Applications of Python Applications of Python • IoT and Embedded Systems: Integrations with platforms like Raspberry Pi and Arduino to control hardware devices •• CircuitCircuit DesignDesign andand Simulation:Simulation: ToolsTools likelike PySpicePySpice, SKiDLSKiDL areare usedused forfor circuitcircuit simulationsimulation and PCB design • Computational Fluid Dynamics (CFD): Libraries such as OpenFOAM are used for simulating fluid flow • Structural Analysis: Frameworks like OpenSeesPy are used for structural modelling and earthquake engineering EV0000654 · EV0000655 · EV0000656
10. Source slide page 10 (extracted text): THANK YOU Departmentp of Computerp Science and Engineeringg g Dr. Shyalaja S S, Director, CCBD and CDSAML, PESU Prof. Sindhu R Pai – sindhurpai@pes.edu Ack: Teaching Assistant – Advaith Sanil Kumar EV0000657 · EV0000658 · EV0000659

SHORT-NOTE RECAP

Source deck preserved: UE24CS151A PCPS Unit 1 SindhuRPai Class 6 Intro to Programming Languages. Its extracted page text is listed as searchable points and all rendered source pages are attached as figures. Match to a specific video is candidate-based; this index does not claim a manual content review.

EV0000630

25 Source slide deck: UE24CS151A PCPS Unit 1 SindhuRPai Class 7 Syntax semantics translation models

This entry preserves the complete slide deck titled UE24CS151A PCPS Unit 1 SindhuRPai Class 7 Syntax semantics translation models as searchable extracted page text and page-faithful rendered images. Its association with a particular video is not confirmed; it remains a separate topic under Python for computational Problem Solving, Unit_01, to avoid an unsupported match. EV0000660 · EV0000661

Graphs and diagrams

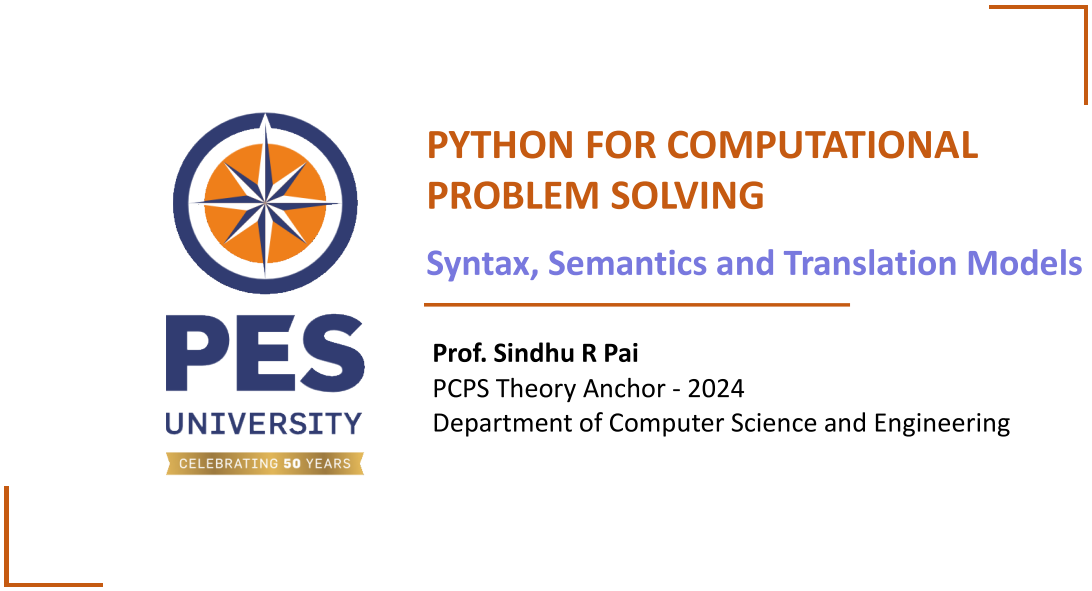


Figure G000111. UE24CS151A PCPS Unit 1 SindhuRPai Class 7 Syntax semantics translation models · source slide page 1.

EV0000661

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

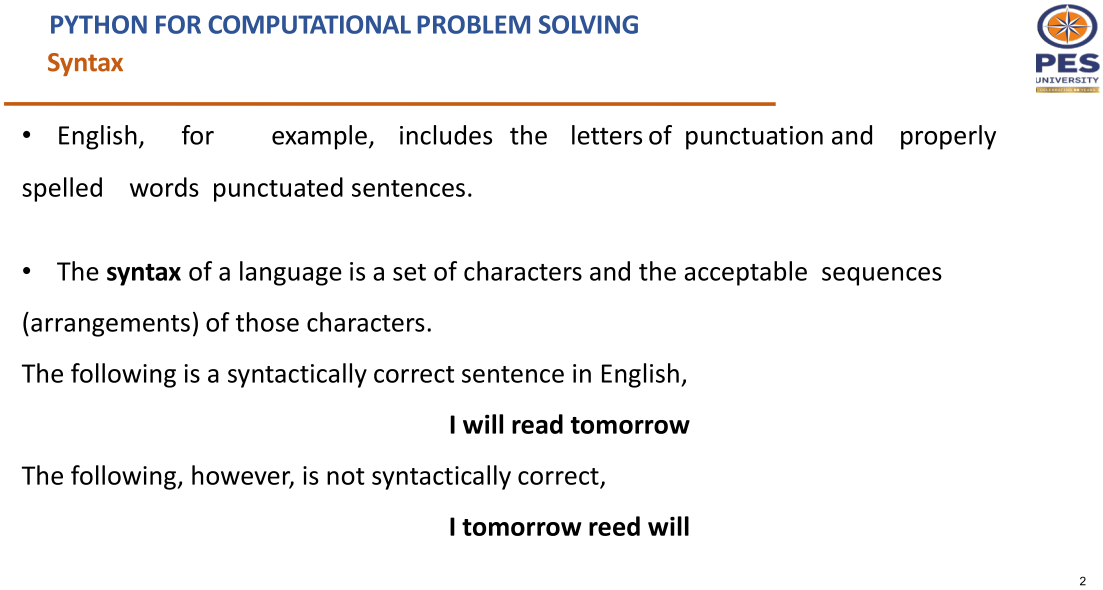


Figure G000112. UE24CS151A PCPS Unit 1 SindhuRPai Class 7 Syntax semantics translation models · source slide page 2.

EV0000664

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

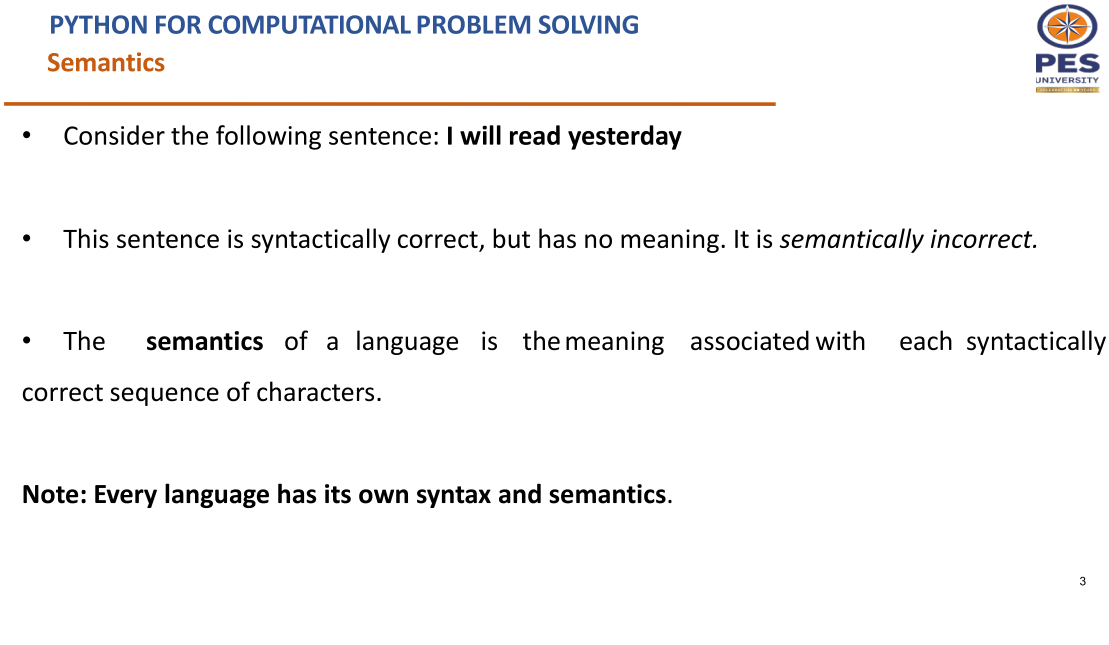


Figure G000113. UE24CS151A PCPS Unit 1 SindhuRPai Class 7 Syntax semantics translation models · source slide page 3.

EV0000667

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Translation

- A programming language provides the necessary constructs to instruct the computer to do the tasks for us.
- Levels – High, medium, low
- Generations – 1 to 5
- Translation models describe the mathematical relationship between two or more languages.
  - **Models of translational equivalence** - Whether expressions in different languages have equivalent meanings.

4

Figure G000114. UE24CS151A PCPS Unit 1 SindhuRPai Class 7 Syntax semantics translation models · source slide page 4.

EV0000670

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Language Translators

- **Compilers, Interpreters, and Assemblers, …**
- Compiler converts entire high-level language programs to machine language at once.
- Interpreter converts high-level language to machine language line by line and execute instructions in place of CPU

5

Figure G000115. UE24CS151A PCPS Unit 1 SindhuRPai Class 7 Syntax semantics translation models · source slide page 5.

EV0000673

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

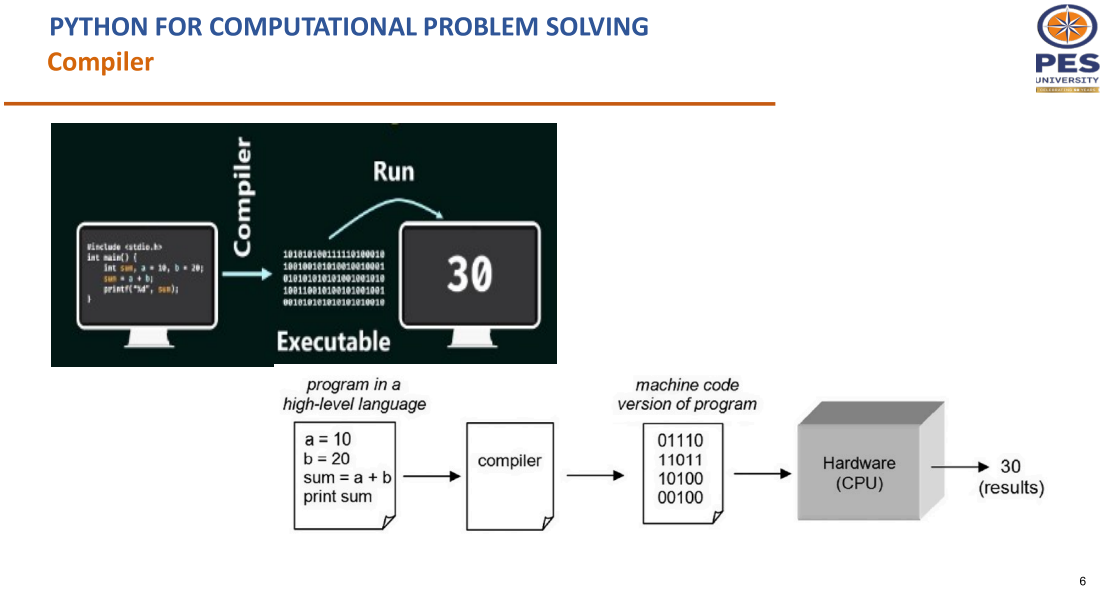


Figure G000116. UE24CS151A PCPS Unit 1 SindhuRPai Class 7 Syntax semantics translation models · source slide page 6.

EV0000676

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

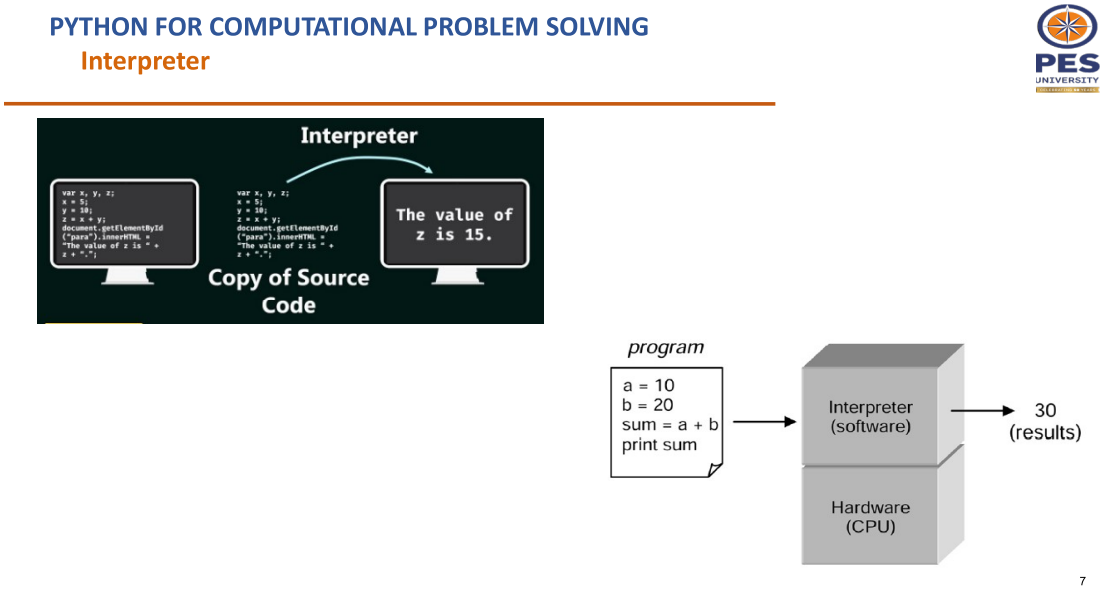


Figure G000117. UE24CS151A PCPS Unit 1 SindhuRPai Class 7 Syntax semantics translation models · source slide page 7.

EV0000679

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

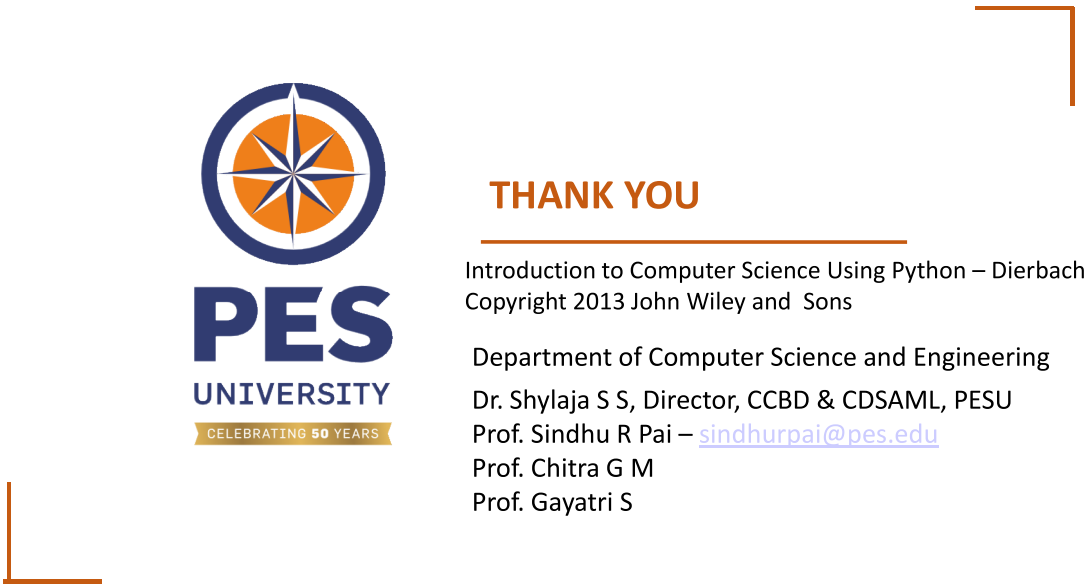


Figure G000118. UE24CS151A PCPS Unit 1 SindhuRPai Class 7 Syntax semantics translation models · source slide page 8.

EV0000682

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Important points and why

1. Source slide page 1 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Syntax, Semantics and Translation Models Prof. Sindhu R Pai PCPS Theory Anchor - 2024 Department of Computer Science and Engineering EV0000660 · EV0000661 · EV0000662
2. Source slide page 2 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Syntax the alphabet,• English, for example, includes the letters of punctuation and properly and properly spelled words punctuated sentences. • The syntax of a language is a set of characters and the acceptable sequences (arrangements) of those characters. The following is a syntactically correct sentence in English, I will read tomorrow The following, however, is not syntactically correct, I tomorrow reed will 2 EV0000663 · EV0000664 · EV0000665
3. Source slide page 3 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Semantics • Consider the following sentence: I will read yesterday • This sentence is syntactically correct, but has no meaning. It is semantically incorrect. • The semantics of a language is themeaning associated with each syntactically correct sequence of characters. Note: Every language has its own syntax and semantics. 3 EV0000666 · EV0000667 · EV0000668

4. Source slide page 4 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Translation ☒A programming language provides the necessary constructs to instruct the computer to do the tasks for us. ☒Levels – High, medium, low ☒Generations – 1 to 5 ☒Translation models describe the mathematical relationship between two or more languages. ☒Models of translational equivalence - Whether expressions in different languages have equivalent meanings. 4 EV0000669 · EV0000670 · EV0000671
5. Source slide page 5 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Language Translators • Compilers, Interpreters, and Assemblers, ... •• CompilerCompiler convertsconverts entireentire highhigh-levellevel languagelanguage programsprograms toto machinemachine languagelanguage atat onceonce. • Interpreter converts high-level language to machine language line by line and execute instructions in place of CPU 5 EV0000672 · EV0000673 · EV0000674
6. Source slide page 6 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Compiler 6 EV0000675 · EV0000676 · EV0000677
7. Source slide page 7 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Interpreter 7 EV0000678 · EV0000679 · EV0000680
8. Source slide page 8 (extracted text): THANK YOU Introduction to Computer Science Using Python – Dierbach Copyrighth 2013 Johnh Wileyl andd Sons Department of Computer Science and Engineering Dr. Shylaja S S, Director, CCBD & CDSAML, PESU Prof. Sindhu R Pai – sindhurpai@pes.edu Prof. Chitra G M Prof. Gayatri S EV0000681 · EV0000682 · EV0000683

SHORT-NOTE RECAP

Source deck preserved: UE24CS151A PCPS Unit 1 SindhuRPai Class 7 Syntax semantics translation models. Its extracted page text is listed as searchable points and all rendered source pages are attached as figures. Match to a specific video is candidate-based; this index does not claim a manual content review.

EV0000660

26 Source slide deck: UE24CS151A PCPS Unit 1 SindhuRPai Class 8 Installation and execution

This entry preserves the complete slide deck titled UE24CS151A PCPS Unit 1 SindhuRPai Class 8 Installation and execution as searchable extracted page text and page-faithful rendered images. Its association with a particular video is not confirmed; it remains a separate topic under Python for computational Problem Solving, Unit_01, to avoid an unsupported match. EV0000684 · EV0000685

Graphs and diagrams

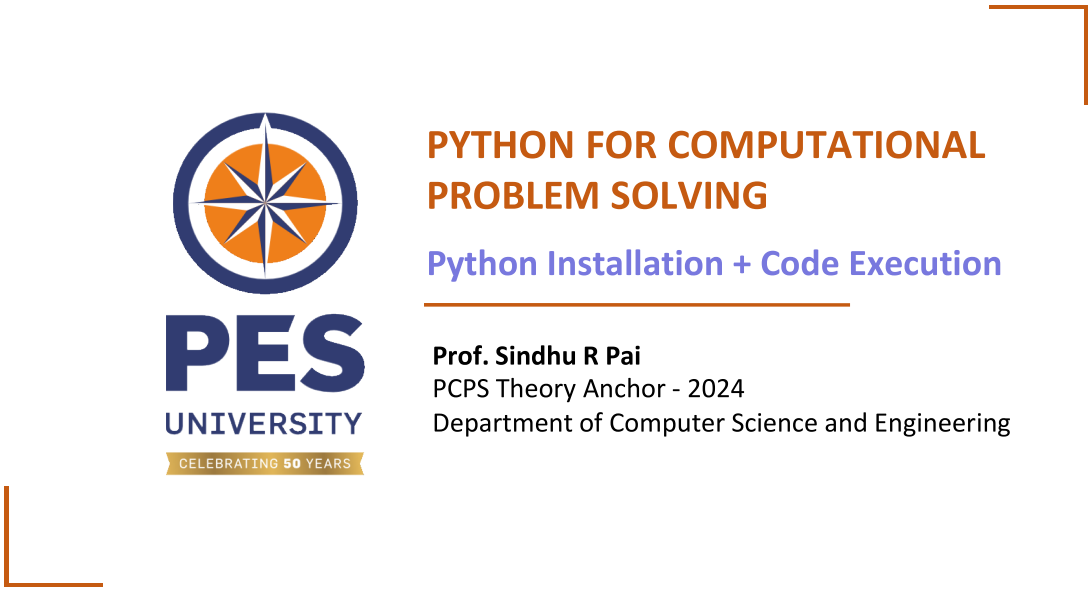


Figure G000119. UE24CS151A PCPS Unit 1 SindhuRPai Class 8 Installation and execution · source slide page 1.

EV0000685

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

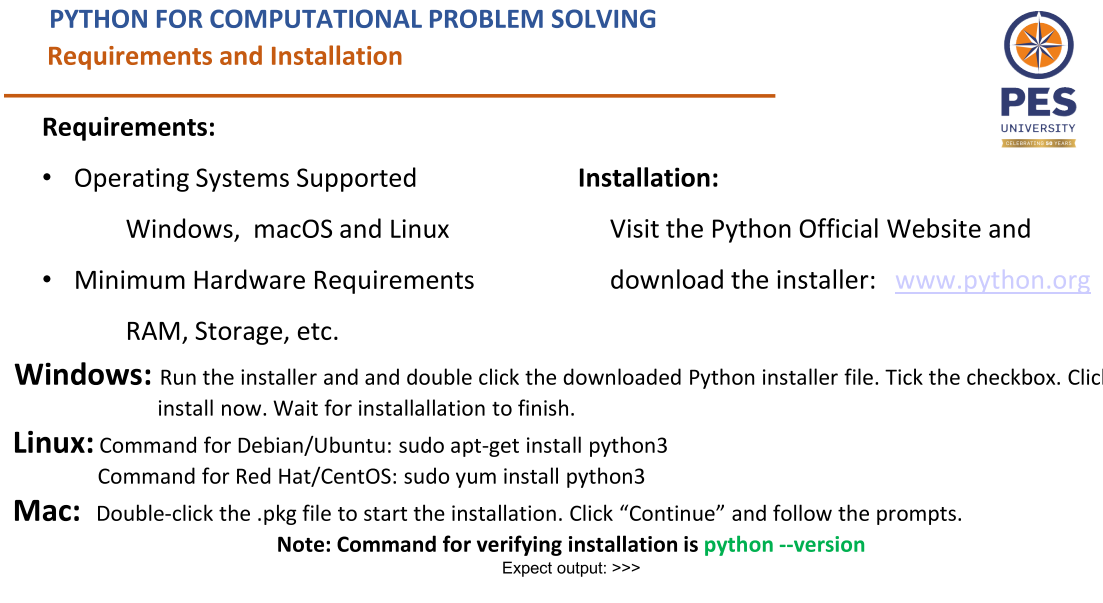


Figure G000120. UE24CS151A PCPS Unit 1 SindhuRPai Class 8 Installation and execution · source slide page 2.

EV0000688

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

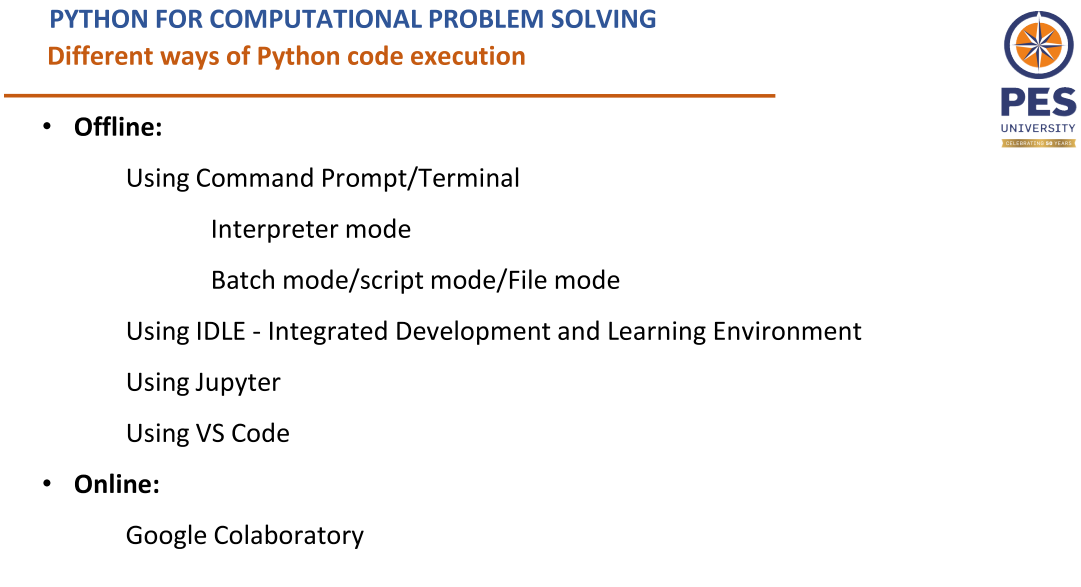


Figure G000121. UE24CS151A PCPS Unit 1 SindhuRPai Class 8 Installation and execution · source slide page 3.

EV0000691

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

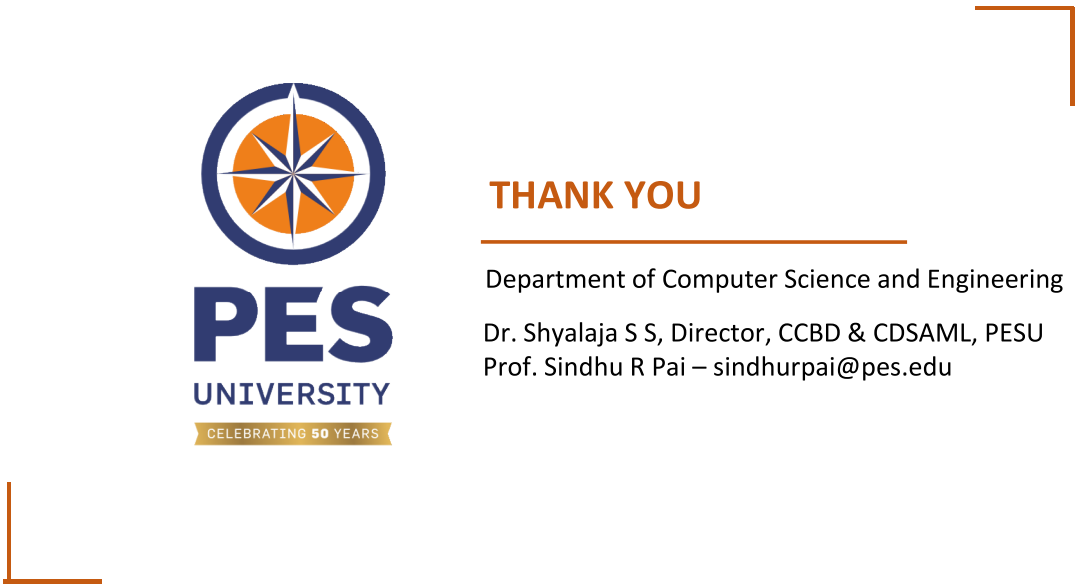


Figure G000122. UE24CS151A PCPS Unit 1 SindhuRPai Class 8 Installation and execution · source slide page 4.

EV0000694

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Important points and why

1. Source slide page 1 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Python Installation + Code Execution Prof. Sindhu R Pai PCPS Theory Anchor - 2024 Department of Computer Science and Engineering EV0000684 · EV0000685 · EV0000686
2. Source slide page 2 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Requirements and Installation Requirements: • Operating Systems Supported Installation: Windows, macOS and Linux Visit the Python Official Website and • Minimum Hardware Requirementsq download the installer: www.python.orgpy g RAM, Storage, etc. Windows: Run the installer and and double click the downloaded Python installer file. Tick the checkbox. Click install now. Wait for installallation to finish. Linux: Command for Debian/Ubuntu: sudo apt-get install python3 Command for Red Hat/CentOS: sudo yum install python3 Mac: Double-click the .pkg file to start the installation. Click “Continue” and follow the prompts. Note: Command for verifying installation is python --version Expect output: >>> 3 EV0000687 · EV0000688 · EV0000689

3. Source slide page 3 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Different ways of Python code execution • Offline: Using Command Prompt/Terminal Interpreter mode Batch mode/script/ p mode/File/ mode Using IDLE - Integrated Development and Learning Environment Using Jupyter Using VS Code • Online: Google Colaboratory 3 EV0000690 · EV0000691 · EV0000692
4. Source slide page 4 (extracted text): THANK YOU Departmentp of Computerp Science and Engineeringg g Dr. Shyalaja S S, Director, CCBD & CDSAML, PESU Prof. Sindhu R Pai – sindhurpai@pes.edu EV0000693 · EV0000694 · EV0000695

SHORT-NOTE RECAP

Source deck preserved: UE24CS151A PCPS Unit 1 SindhuRPai Class 8 Installation and execution. Its extracted page text is listed as searchable points and all rendered source pages are attached as figures. Match to a specific video is candidate-based; this index does not claim a manual content review.

EV0000684

27 Source slide deck: IDLE

This entry preserves the complete slide deck titled IDLE as searchable extracted page text and page-faithful rendered images. Its association with a particular video is not confirmed; it remains a separate topic under Python for computational Problem Solving, Unit_01, to avoid an unsupported match. EV0000714 · EV0000715

Graphs and diagrams

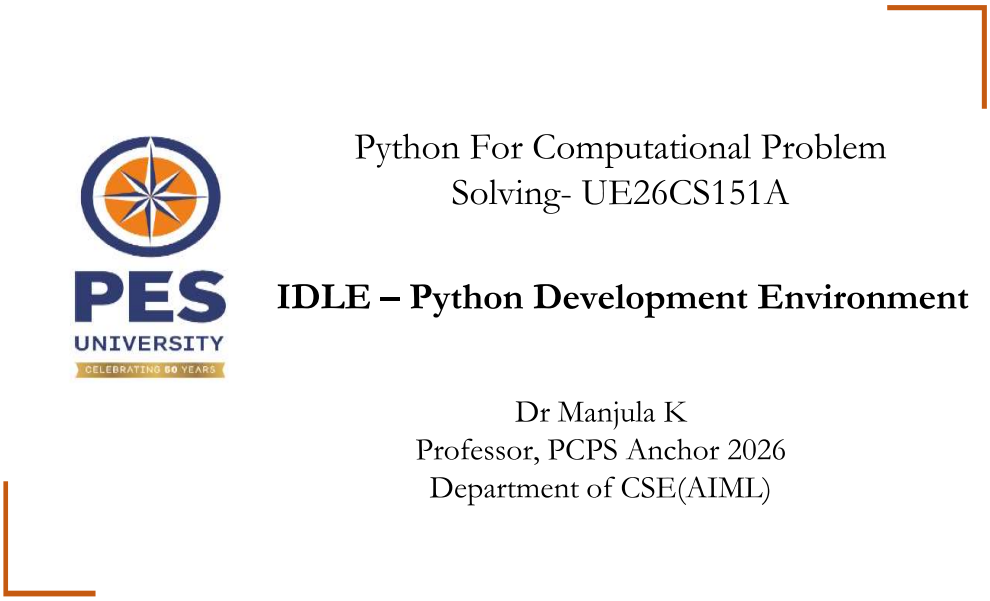


Figure G000129. IDLE · source slide page 1.

EV0000715

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Python For Computational Problem Solving

IDLE – Python Development Environment

What is IDLE?

- IDLE stands for Integrated Development and Learning Environment.
- It is a simple Python development environment that comes with Python.
- It supports interactive programming and script development.
- It helps users write, run, test, and debug Python programs.

Figure G000130. IDLE · source slide page 2.

EV0000718

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Python For Computational Problem Solving

IDLE – Python Development Environment

Why Use IDLE?

- Beginner-friendly interface
- Python Shell for interactive execution
- Code editor for writing programs
- Syntax highlighting
- Automatic indentation
- Built-in debugger
- Easy program execution

Figure G000131. IDLE · source slide page 3.

EV0000721

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Python For Computational Problem Solving

IDLE – Python Development Environment

Installing IDLE

1. Download Python:
   - Go to the official Python Website and click on the download link
   - for Windows.
   - Save the executable file to your computer.
2. Run the Installer:
   - Double-click the downloaded file to start the setup window.
   - At the bottom of the installer window, make sure to check the box
   - that says "Add python.exe to PATH" (if available).
   - Click on "Install Now". This default setting installs Python along
   - with IDLE, pip, and documentation.

Figure G000132. IDLE · source slide page 4.

EV0000724

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Python For Computational Problem Solving

IDLE – Python Development Environment

Installing IDLE

Finish Setup:

Wait for the progress bar to finish until a "Setup was successful" message appears.

How to Open IDLE

1. Press the **Windows Key** on your keyboard to open the Start menu.
2. Type IDLE in the search bar.
3. Click on **IDLE (Python 3.x)** to launch the application

Figure G000133. IDLE · source slide page 5.

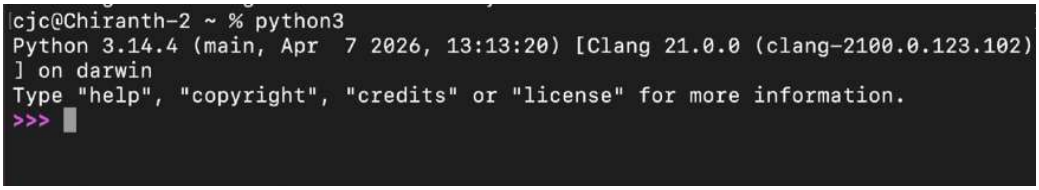
EV0000727

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Python For Computational Problem Solving

IDLE – Python Development Environment

Starting IDLE



When IDLE opens, the Python Shell is displayed.
The Shell provides the >>> prompt for entering Python statements.

Figure G000134. IDLE · source slide page 6.

EV0000730

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

IDLE Interface

- Menu Bar
- Python Shell
- Interactive prompt (>>>)
- Editor Window
- Output area
- Debugger and other development tools

Figure G000135. IDLE · source slide page 7.

EV0000733

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Python For Computational Problem Solving

IDLE – Python Development Environment

Python Shell

The Python Shell executes Python statements interactively.

Figure G000136. IDLE · source slide page 8.

EV0000736

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Python For Computational Problem Solving

IDLE – Python Development Environment

Interactive Mode

- Executes one statement at a time.
- Results appear immediately.
- Useful for learning, calculations, testing small code fragments, and experimenting with Python.

Figure G000137. IDLE · source slide page 9.

EV0000739

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Python For Computational Problem Solving

IDLE – Python Development Environment

Script Mode

- Used to write complete Python programs.
- Programs are normally saved with the .py extension.
- Example: hello.py
- A complete program can be edited, saved, and executed repeatedly.

Figure G000138. IDLE · source slide page 10.

EV0000742

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Python For Computational Problem Solving

IDLE – Python Development Environment

Creating a Python Program

- File → New File
- Write the Python program in the Editor window.
- File → Save
- Save using a meaningful .py filename.

Figure G000139. IDLE · source slide page 11.

EV0000745

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Python For Computational Problem Solving

IDLE – Python Development Environment

Running a Program

- Use Run → Run Module.
- Keyboard shortcut: F5
- IDLE executes the current module.
- Output is displayed in the Python Shell.

Figure G000140. IDLE · source slide page 12.

EV0000748

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Python For Computational Problem Solving

IDLE – Python Development Environment

Interactive Mode vs Script Mode

Interactive Mode	Script Mode
Executes one line at a time	Executes complete program
Immediate output	Saved as .py file
Good for testing	Good for projects

Figure G000141. IDLE · source slide page 13.

EV0000751

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Python For Computational Problem Solving

IDLE – Python Development Environment

Script Mode Features

1: Syntax Highlighting

Syntax highlighting is a feature in IDLE that displays different parts of Python code in different colors, making the code easier to read and understand.

Benefits

- Highlights **keywords**, **strings**, **comments**, and other code elements.
- Improves code readability.
- Makes it easier to identify syntax mistakes.
- Helps beginners understand Python code more easily.

Figure G000142. IDLE · source slide page 14.

EV0000754

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Python For Computational Problem Solving

IDLE – Python Development Environment

PES UNIVERSITY

2: Automatic Indentation

- Python uses indentation to define blocks.
- IDLE assists with automatic indentation.
- Indentation is part of Python syntax.

Figure G000143. IDLE · source slide page 15.

EV0000757

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Python For Computational Problem Solving

IDLE – Python Development Environment

Script Mode: Saving Python Programs

- Python source files use the .py extension.
- Examples: calculator.py, student.py, area.py
- Use File → Save or Ctrl + S.
- Use meaningful filenames.

Figure G000144. IDLE · source slide page 16.

EV0000760

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Python For Computational Problem Solving

IDLE – Python Development Environment

Script Mode: Opening an Existing Program

- File → Open
- Select the required .py file.
- The file opens in the IDLE Editor Window.
- You can edit, save, and run it using F5.

Figure G000145. IDLE · source slide page 17.

EV0000763

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Python For Computational Problem Solving

IDLE – Python Development Environment

Errors in IDLE

IDLE displays error messages when problems occur.

Error	Description	Example
SyntaxError	Invalid Python syntax	print(“Hello"
NameError	Variable not defined	print(x)
TypeError	Unsupported operation between data types	"5" + 5
ValueError	Invalid value passed to a function	int("abc")
IndentationError	Incorrect indentation in code	Missing spaces after if satement

Figure G000146. IDLE · source slide page 18.

EV0000766

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Python For Computational Problem Solving

IDLE – Python Development Environment

Debugging in IDLE

Features of IDLE Debugger

- Built-in debugger included with IDLE.
- Executes programs step by step.
- Allows you to inspect variable values during execution.
- Helps identify logical and runtime errors.

Benefits

- Makes it easier to understand program flow.
- Simplifies locating and fixing bugs.
- Useful for testing and improving code.

Figure G000147. IDLE · source slide page 19.

EV0000769

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Python For Computational Problem Solving

IDLE – Python Development Environment

Breakpoints

A breakpoint is a marker that temporarily pauses program execution at a selected line of code.

Why Use Breakpoints?

- Pause execution at a specific line.
- Examine the values of variables.
- Observe the flow of the program step by step.
- Help locate and fix logical errors.

Benefits

- Makes debugging easier.
- Improves understanding of program execution.
- Saves time when troubleshooting complex code.

Figure G000148. IDLE · source slide page 20.

EV0000772

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Python For Computational Problem Solving

IDLE – Python Development Environment

Useful IDLE Shortcuts

Action	Windows	macOS
New File	Ctrl + N	⌘ + N
Open File	Ctrl + O	⌘ + O
Save File	Ctrl + S	⌘ + S
Run Module	F5	Fn + F5
Undo	Ctrl + Z	⌘ + Z
Find	Ctrl + F	⌘ + F
Select All	Ctrl + A	⌘ + A

Figure G000149. IDLE · source slide page 21.

EV0000775

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Python For Computational Problem Solving

IDLE – Python Development Environment

Example: Area of a Circle

```
radius = float(input("Enter radius: "))
area = 3.14159 * radius * radius
print("Area =", area)
```

Example output: Enter radius: 5 → Area = 78.53975

Figure G000150. IDLE · source slide page 22.

EV0000778

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Python For Computational Problem Solving

IDLE – Python Development Environment

IDLE Workflow

Create / Open File

↓

Write Program

↓

Save .py

↓

Run using F5

↓

View Output in Python Shell

Figure G000151. IDLE · source slide page 23.

EV0000781

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Python For Computational Problem Solving

IDLE – Python Development Environment

Advantages of IDLE

- Comes with Python
- Easy to use
- Beginner-friendly
- Interactive Shell
- Built-in editor
- Syntax highlighting
- Automatic indentation
- Debugging support

Figure G000152. IDLE · source slide page 24.

EV0000784

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Python For Computational Problem Solving

IDLE – Python Development Environment

Limitations of IDLE

- Basic interface
- Fewer advanced development features
- Limited project management
- Less suitable for very large projects
- Advanced users may prefer feature-rich IDEs such as VS Code or PyCharm.

Figure G000153. IDLE · source slide page 25.

EV0000787

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Python For Computational Problem Solving

IDLE – Python Development Environment

Key Takeaways

- IDLE stands for Integrated Development and Learning Environment.
- It provides a Python Shell for interactive execution.
- The Editor Window is used to write and run Python scripts.
- Python programs are saved with the .py extension.
- Press F5 to run the current program.
- IDLE includes basic debugging tools to help find and fix errors.

Figure G000154. IDLE · source slide page 26.

EV0000790

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

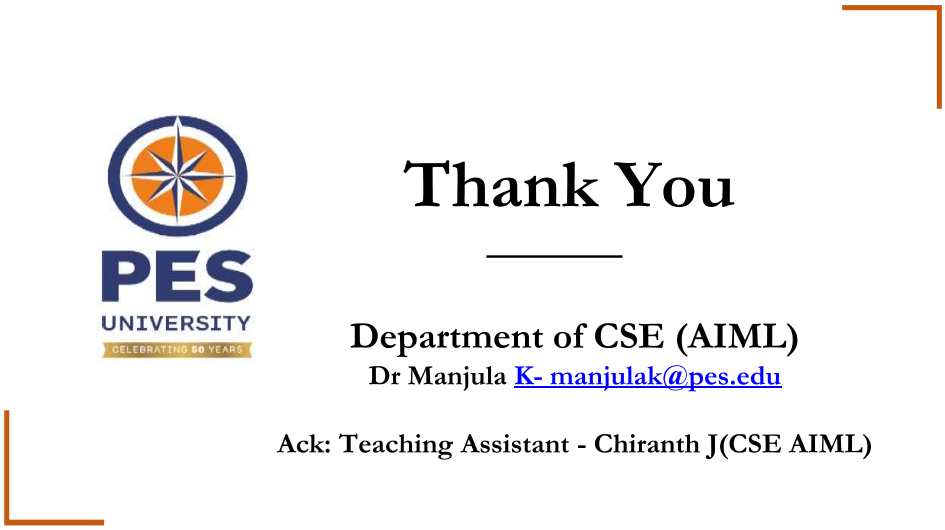


Figure G000155. IDLE · source slide page 27.

EV0000793

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Important points and why

1. Source slide page 1 (extracted text): Python For Computational Problem Solving- UE26CS151A IDLE – Python Development Environment Dr Manjula K Professor, PCPS Anchor 2026 Department of CSE(AIML) EV0000714 · EV0000715 · EV0000716
2. Source slide page 2 (extracted text): Python For Computational Problem Solving IDLE – Python Development Environment What is IDLE? • IDLE stands for Integrated Development and Learning Environment. • It is a simple Python development environment that comes with Python. • It supports interactive programming and script development. • It helps users write, run, test, and debug Python programs. EV0000717 · EV0000718 · EV0000719
3. Source slide page 3 (extracted text): Python For Computational Problem Solving IDLE – Python Development Environment Why Use IDLE? • Beginner-friendly interface • Python Shell for interactive execution • Code editor for writing programs • Syntax highlighting • Automatic indentation • Built-in debugger • Easy program execution EV0000720 · EV0000721 · EV0000722

4. Source slide page 4 (extracted text): Python For Computational Problem Solving IDLE – Python Development Environment Installing IDLE 1. Download Python: ◦ Go to the official Python Website and click on the download link ◦ for Windows. ◦ Save the executable file to your computer. 2. Run the Installer: ◦ Double-click the downloaded file to start the setup window. ◦ At the bottom of the installer window, make sure to check the box ◦ that says "Add python.exe to PATH" (if available). ◦ Click on "Install Now". This default setting installs Python along ◦ with IDLE, pip, and documentation. EV0000723 · EV0000724 · EV0000725
5. Source slide page 5 (extracted text): Python For Computational Problem Solving IDLE – Python Development Environment Installing IDLE Finish Setup: Wait for the progress bar to finish until a "Setup was successful" message appears. How to Open IDLE 1. Press the Windows Key on your keyboard to open the Start menu. 2. Type IDLE in the search bar. 3. Click on IDLE (Python 3.x) to launch the application EV0000726 · EV0000727 · EV0000728
6. Source slide page 6 (extracted text): Python For Computational Problem Solving IDLE – Python Development Environment Starting IDLE When IDLE opens, the Python Shell is displayed. The Shell provides the >>> prompt for entering Python statements. EV0000729 · EV0000730 · EV0000731
7. Source slide page 7 (extracted text): Python For Computational Problem Solving IDLE – Python Development Environment IDLE Interface • Menu Bar • Python Shell • Interactive prompt (>>>) • Editor Window • Output area • Debugger and other development tools EV0000732 · EV0000733 · EV0000734
8. Source slide page 8 (extracted text): Python For Computational Problem Solving IDLE – Python Development Environment Python Shell The Python Shell executes Python statements interactively. EV0000735 · EV0000736 · EV0000737
9. Source slide page 9 (extracted text): Python For Computational Problem Solving IDLE – Python Development Environment Interactive Mode • Executes one statement at a time. • Results appear immediately. • Useful for learning, calculations, testing small code fragments, and experimenting with Python. EV0000738 · EV0000739 · EV0000740
10. Source slide page 10 (extracted text): Python For Computational Problem Solving IDLE – Python Development Environment Script Mode • Used to write complete Python programs. • Programs are normally saved with the .py extension. • Example: hello.py • A complete program can be edited, saved, and executed repeatedly. EV0000741 · EV0000742 · EV0000743
11. Source slide page 11 (extracted text): Python For Computational Problem Solving IDLE – Python Development Environment Creating a Python Program • File → New File • Write the Python program in the Editor window. • File → Save • Save using a meaningful .py filename. EV0000744 · EV0000745 · EV0000746
12. Source slide page 12 (extracted text): Python For Computational Problem Solving IDLE – Python Development Environment Running a Program • Use Run → Run Module. • Keyboard shortcut: F5 • IDLE executes the current module. • Output is displayed in the Python Shell. EV0000747 · EV0000748 · EV0000749
13. Source slide page 13 (extracted text): Python For Computational Problem Solving IDLE – Python Development Environment Interactive Mode vs Script Mode Interactive Mode Script Mode Executes one line at a time Executes complete program Immediate output Saved as .py file Good for testing Good for projects EV0000750 · EV0000751 · EV0000752

14. Source slide page 14 (extracted text): Python For Computational Problem Solving IDLE – Python Development Environment Script Mode Features 1: Syntax Highlighting Syntax highlighting is a feature in IDLE that displays different parts of Python code in different colors, making the code easier to read and understand. Benefits • Highlights keywords, strings, comments, and other code elements. • Improves code readability. • Makes it easier to identify syntax mistakes. • Helps beginners understand Python code more easily. EV0000753 · EV0000754 · EV0000755

15. Source slide page 15 (extracted text): Python For Computational Problem Solving IDLE – Python Development Environment 2: Automatic Indentation • Python uses indentation to define blocks. • IDLE assists with automatic indentation. • Indentation is part of Python syntax. EV0000756 · EV0000757 · EV0000758

16. Source slide page 16 (extracted text): Python For Computational Problem Solving IDLE – Python Development Environment Script Mode: Saving Python Programs • Python source files use the .py extension. • Examples: calculator.py, student.py, area.py • Use File → Save or Ctrl + S. • Use meaningful filenames. EV0000759 · EV0000760 · EV0000761

17. Source slide page 17 (extracted text): Python For Computational Problem Solving IDLE – Python Development Environment Script Mode: Opening an Existing Program • File → Open • Select the required .py file. • The file opens in the IDLE Editor Window. • You can edit, save, and run it using F5. EV0000762 · EV0000763 · EV0000764

18. Source slide page 18 (extracted text): Python For Computational Problem Solving IDLE – Python Development Environment Errors in IDLE IDLE displays error messages when problems occur. Error Description Example SyntaxError Invalid Python syntax print(“Hello" NameError Variable not defined print(x) Unsupported operation TypeError "5" + 5 between data types Invalid value passed to a ValueError int("abc") function IndentationErr Incorrect indentation in Missing spaces after if or code satement EV0000765 · EV0000766 · EV0000767

19. Source slide page 19 (extracted text): Python For Computational Problem Solving IDLE – Python Development Environment Debugging in IDLE Features of IDLE Debugger • Built-in debugger included with IDLE. • Executes programs step by step. • Allows you to inspect variable values during execution. • Helps identify logical and runtime errors. Benefits • Makes it easier to understand program flow. • Simplifies locating and fixing bugs. • Useful for testing and improving code. EV0000768 · EV0000769 · EV0000770

20. Source slide page 20 (extracted text): Python For Computational Problem Solving IDLE – Python Development Environment Breakpoints A breakpoint is a marker that temporarily pauses program execution at a selected line of code. Why Use Breakpoints? • Pause execution at a specific line. • Examine the values of variables. • Observe the flow of the program step by step. • Help locate and fix logical errors. Benefits • Makes debugging easier. • Improves understanding of program execution. • Saves time when troubleshooting complex code. EV0000771 · EV0000772 · EV0000773

21. Source slide page 21 (extracted text): Python For Computational Problem Solving IDLE – Python Development Environment Useful IDLE Shortcuts Action Windows macOS New File Ctrl + N ⌘+ N Open File Ctrl + O ⌘+ O Save File Ctrl + S ⌘+ S Run Module F5 Fn + F5 Undo Ctrl + Z ⌘+ Z Find Ctrl + F ⌘+ F Select All Ctrl + A ⌘+ A EV0000774 · EV0000775 · EV0000776

22. Source slide page 22 (extracted text): Python For Computational Problem Solving IDLE – Python Development Environment Example: Area of a Circle radius = float(input("Enter radius: ")) area = 3.14159 * radius * radius print("Area =", area) Example output: Enter radius: 5 → Area = 78.53975 EV0000777 · EV0000778 · EV0000779
23. Source slide page 23 (extracted text): Python For Computational Problem Solving IDLE – Python Development Environment IDLE Workflow EV0000780 · EV0000781 · EV0000782
24. Source slide page 24 (extracted text): Python For Computational Problem Solving IDLE – Python Development Environment Advantages of IDLE • Comes with Python • Easy to use • Beginner-friendly • Interactive Shell • Built-in editor • Syntax highlighting • Automatic indentation • Debugging support EV0000783 · EV0000784 · EV0000785
25. Source slide page 25 (extracted text): Python For Computational Problem Solving IDLE – Python Development Environment Limitations of IDLE • Basic interface • Fewer advanced development features • Limited project management • Less suitable for very large projects • Advanced users may prefer feature-rich IDEs such as VS Code or PyCharm. EV0000786 · EV0000787 · EV0000788
26. Source slide page 26 (extracted text): Python For Computational Problem Solving IDLE – Python Development Environment Key Takeaways • IDLE stands for Integrated Development and Learning Environment. • It provides a Python Shell for interactive execution. • The Editor Window is used to write and run Python scripts. • Python programs are saved with the .py extension. • Press F5 to run the current program. • IDLE includes basic debugging tools to help find and fix errors. EV0000789 · EV0000790 · EV0000791
27. Source slide page 27 (extracted text): Thank You Department of CSE (AIML) Dr Manjula K-manjulak@pes.edu Ack: Teaching Assistant - Chiranth J(CSE AIML) EV0000792 · EV0000793 · EV0000794

SHORT-NOTE RECAP

Source deck preserved: IDLE. Its extracted page text is listed as searchable points and all rendered source pages are attached as figures. Match to a specific video is candidate-based; this index does not claim a manual content review.

EV0000714

28 Source slide deck: UE24CS151A PCPS Unit 1 SindhuRPai Class 10 variable id type

This entry preserves the complete slide deck titled UE24CS151A PCPS Unit 1 SindhuRPai Class 10 variable id type as searchable extracted page text and page-faithful rendered images. Its association with a particular video is not confirmed; it remains a separate topic under Python for computational Problem Solving, Unit_01, to avoid an unsupported match. EV0000816 · EV0000817

Graphs and diagrams

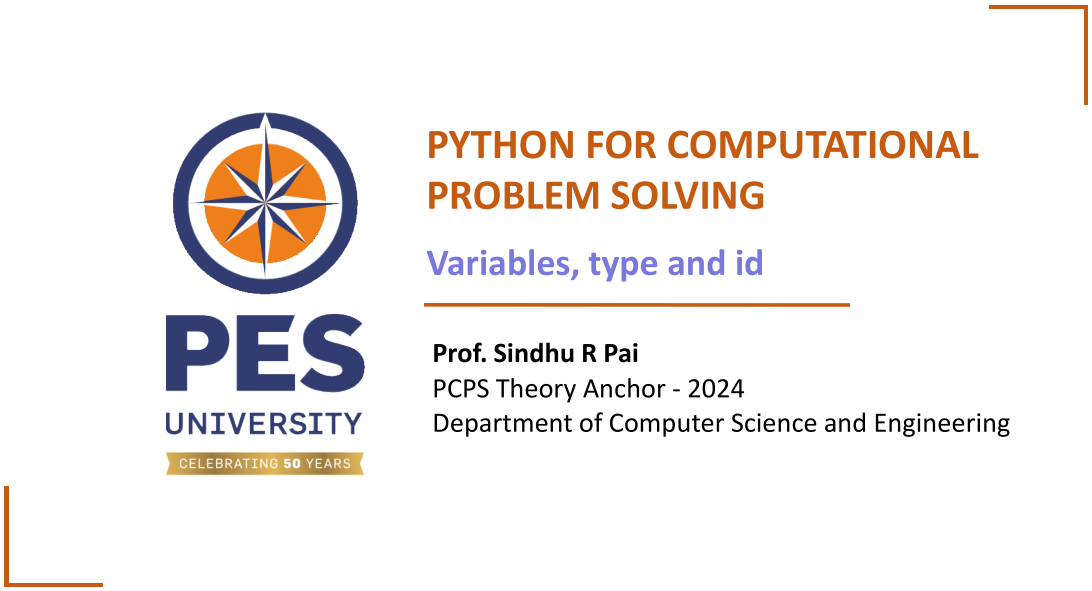


Figure G000163. UE24CS151A PCPS Unit 1 SindhuRPai Class 10 variable id type · source slide page 1.

EV0000817

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

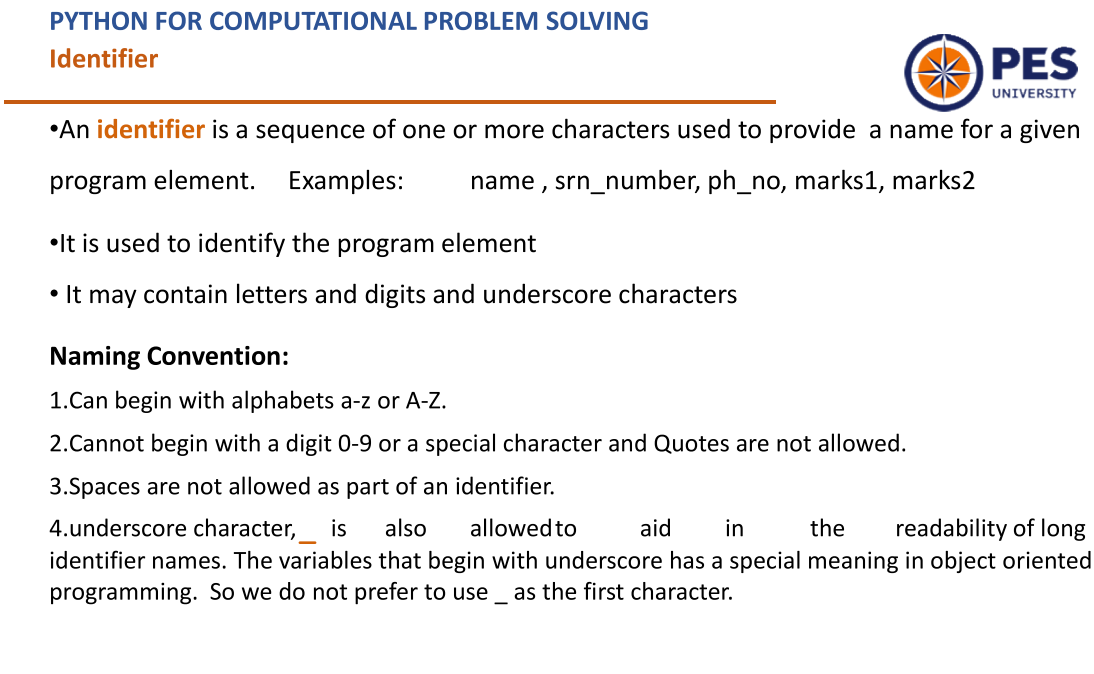


Figure G000164. UE24CS151A PCPS Unit 1 SindhuRPai Class 10 variable id type · source slide page 2.

EV0000820

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Valid and invalid Identifiers

PES UNIVERSITY

Valid Identifiers	Invalid Identifiers	Reason Invalid
totalSales	'totalSales'	quotes not allowed
totalsales	total sales	spaces not allowed
salesFor2010	2010Sales	cannot begin with a digit

Figure G000165. UE24CS151A PCPS Unit 1 SindhuRPai Class 10 variable id type · source slide page 3.

EV0000823

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING
Keywords

PES UNIVERSITY

- Keywords are **reserved** words that have a predefined meaning.
- To know the keywords, type help() in the python prompt and in the help prompt, type keywords

```
help> keywords

Here is a list of the Python keywords.  Enter any keyword to get more help.

False               class               from                or
None                continue            global              pass
True                def                 if                  raise
and                 del                 import              return
as                  elif                in                  try
assert              else                is                  while
async               except              lambda              with
await               finally             nonlocal            yield
break               for                 not
```

Figure G000166. UE24CS151A PCPS Unit 1 SindhuRPai Class 10 variable id type · source slide page 4.

EV0000826

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

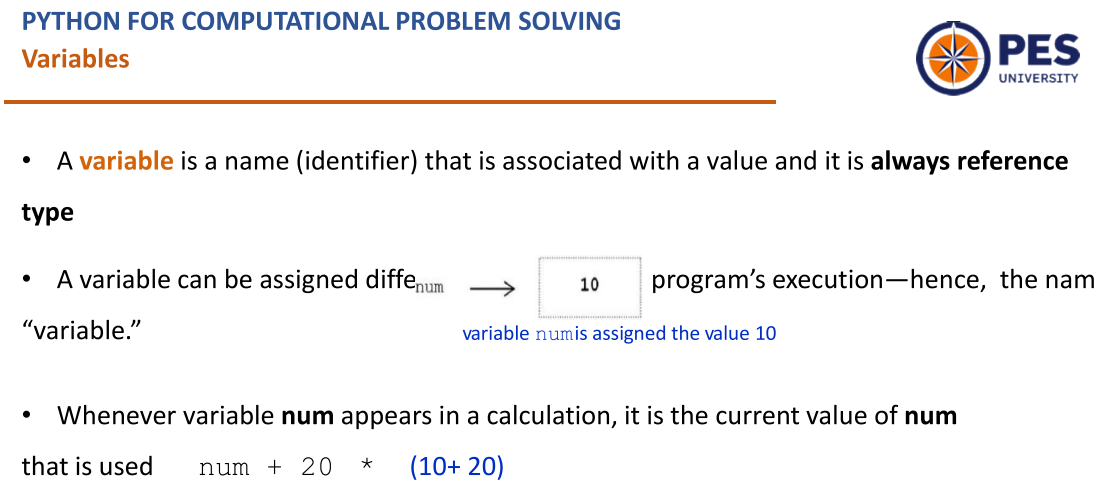


Figure G000167. UE24CS151A PCPS Unit 1 SindhuRPai Class 10 variable id type · source slide page 5.

EV0000829

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

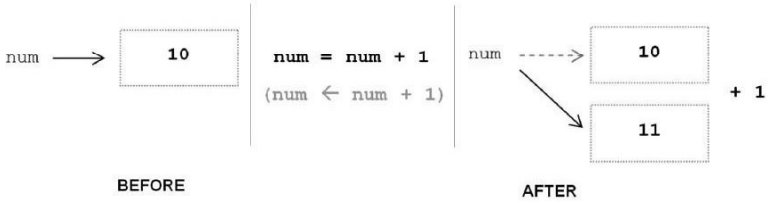
PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Diagrammatic Representation

PES UNIVERSITY

- Variables are assigned values by use of the **assignment operator , =**

num = 10



Note: The right side of an assignment is evaluated first, then the result is assigned to the variable on the left.

Figure G000168. UE24CS151A PCPS Unit 1 SindhuRPai Class 10 variable id type · source slide page 6.

EV0000832

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

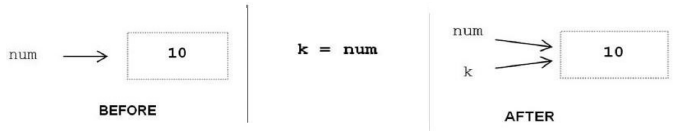
PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Diagrammatic Representation

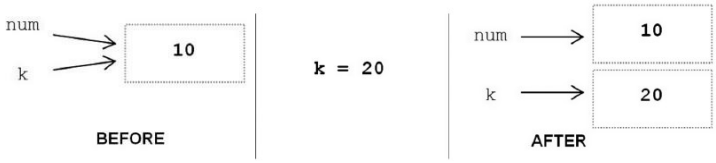
PES UNIVERSITY

- Variables may also be assigned to the value of another variable.

num = 10



Note: Variables `num` and `k` are both associated with the same literal value 10 in memory. You can check with id()



Note: If no other variable references the memory location of the original value, the memory location is deallocated (that is, it is made available for reuse).

Figure G000169. UE24CS151A PCPS Unit 1 SindhuRPai Class 10 variable id type · source slide page 7.

EV0000835

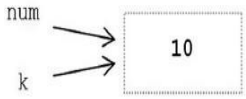
How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Diagrammatic Representation

PES UNIVERSITY

- If the value of `num` changed, would variable `k` change along with it?



- Here variables refer to integer values, and integer values are *immutable.*
- An **immutable value** is a value that cannot be changed.
- Thus, both will continue to refer to the same value until one (or both) of them is reassigned

Figure G000170. UE24CS151A PCPS Unit 1 SindhuRPai Class 10 variable id type · source slide page 8.

EV0000838

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

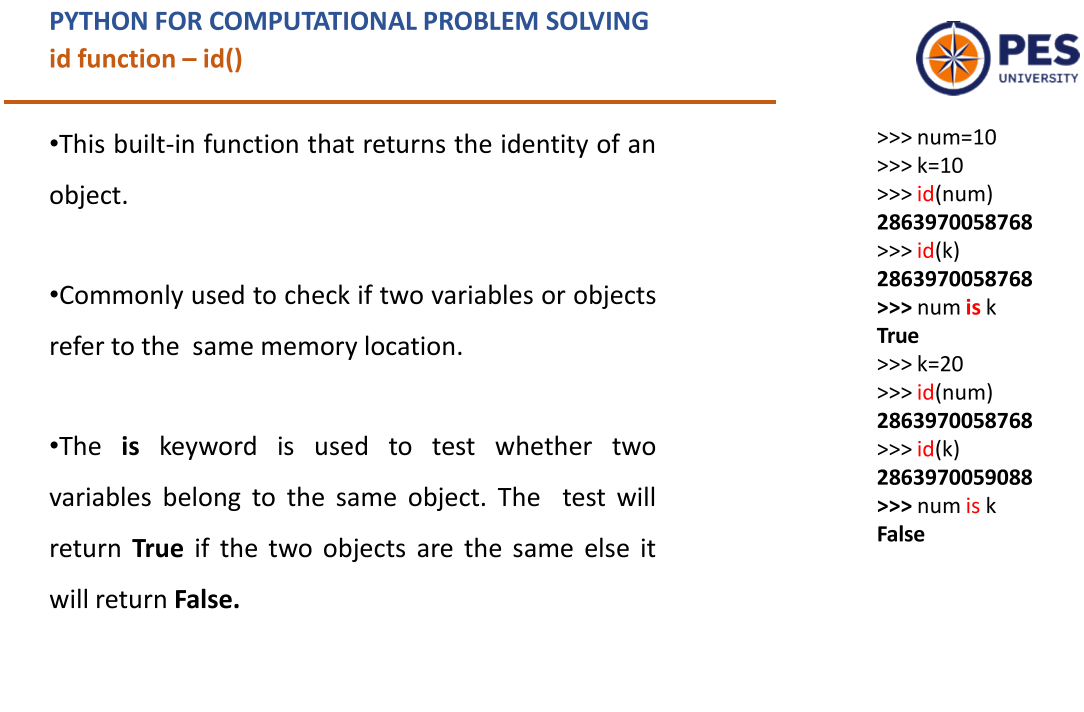


Figure G000171. UE24CS151A PCPS Unit 1 SindhuRPai Class 10 variable id type · source slide page 9.

EV0000841

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

- Datatype refers to the type of value a variable refers to.
- Significance of data type:
  - Memory associated with it
  - Operations that can be performed on it.
  - Range of values allowed in it
- Types:
  - Scalar - Integers, floats, boolean, complex
  - Reference - List, tuple, set, dict

Figure G000172. UE24CS151A PCPS Unit 1 SindhuRPai Class 10 variable id type · source slide page 10.

EV0000844

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

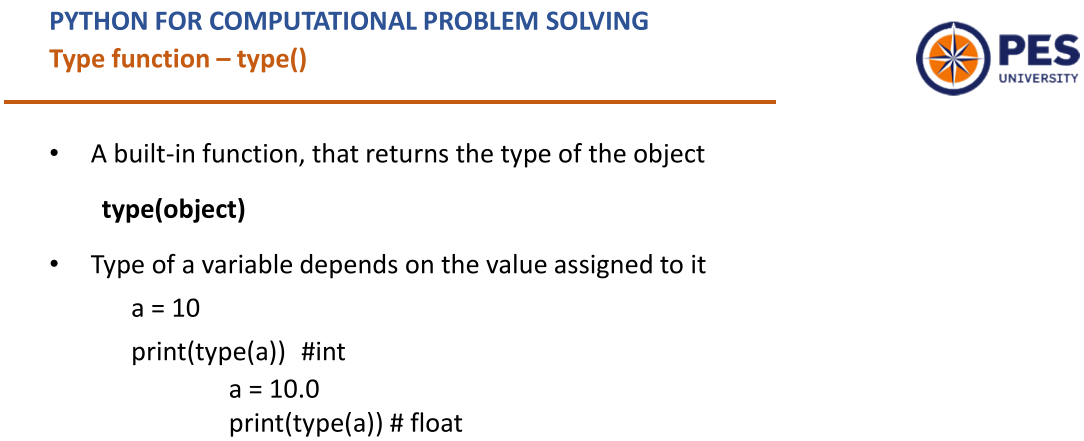


Figure G000173. UE24CS151A PCPS Unit 1 SindhuRPai Class 10 variable id type · source slide page 11.

EV0000847

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

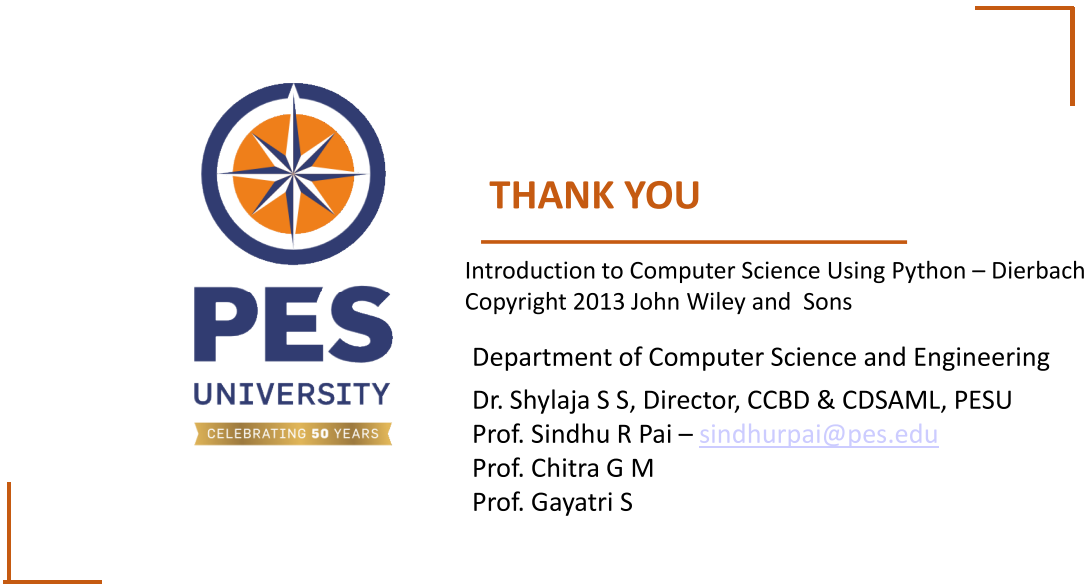


Figure G000174. UE24CS151A PCPS Unit 1 SindhuRPai Class 10 variable id type · source slide page 12.

EV0000850

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Important points and why

1. Source slide page 1 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Variables, type and id Prof. Sindhu R Pai PCPS Theory Anchor - 2024 Department of Computer Science and Engineering EV0000816 · EV0000817 · EV0000818
2. Source slide page 2 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Identifier •An identifier is a sequence of one or more characters used to provide a name for a given program element. Examples: name , srn_number, ph_no, marks1, marks2 •It is used to identify the program element • ItIt maymay containcontain lettersletters andand digitsdigits andand underscoreunderscore characterscharacters Naming Convention: 1.Can begin with alphabets a-z or A-Z. 2.Cannot begin with a digit 0-9 or a special character and Quotes are not allowed. 3.Spaces are not allowed as part of an identifier. 4.underscore character,_ is also allowedto aid in the readability of long identifier names. The variables that begin with underscore has a special meaning in object oriented programming. So we do not prefer to use _ as the first character. EV0000819 · EV0000820 · EV0000821

3. Source slide page 3 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Valid and invalid Identifiers EV0000822 · EV0000823 · EV0000824
4. Source slide page 4 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Keywords • Keywords are reserved words that have a predefined meaning. • To know the keywords, type help() in the python prompt and in the help prompt, type keywords EV0000825 · EV0000826 · EV0000827
5. Source slide page 5 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Variables • A variable is a name (identifier) that is associated with a value and it is always reference type • A variable can be assigned different values during a program's execution—hence, the name "variable." variable numis assigned the value 10 • Whenever variable num appears in a calculation, it is the current value of num that is used num + 20 * (10+ 20) • If variable num is assigned a new value, then the same expression will produce a different num = 5 result. num + 20 (5 + 20) EV0000828 · EV0000829 · EV0000830
6. Source slide page 6 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Diagrammatic Representation • Variables are assigned values by use of the assignment operator , = num = 10 Note: The right side of an assignment is evaluated first, then the result is assigned to the variable on the left. EV0000831 · EV0000832 · EV0000833
7. Source slide page 7 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Diagrammatic Representation • Variables may also be assigned to the value of another variable. num = 10 Note: Variables num and k are both associated with the same literal value 10 in memory. You can check with id() Note: If no other variable references the memory location of the original value, the memory location is deallocated (that is, it is made available for reuse). EV0000834 · EV0000835 · EV0000836
8. Source slide page 8 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Diagrammatic Representation • If the value of num changed, would variable k change along with it? • Here variables refer to integer values, and integer values are immutable. • An immutable value is a value that cannot be changed. • Thus, both will continue to refer to the same value until one (or both) of them is reassigned EV0000837 · EV0000838 · EV0000839
9. Source slide page 9 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING id function – id() •This built-in function that returns the identity of an >>> num=10 >>> k=10 object. >>> id(num) 2863970058768 >>> id(k) 2863970058768 •CommonlyCommonly usedused toto checkcheck ifif twotwo variablesvariables ororobjectsobjects >>> num is k Truerefer to the same memory location. >>> k=20 >>> id(num) 2863970058768 •The is keyword is used to test whether two >>> id(k) 2863970059088 variables belong to the same object. The test will >>> num is k Falsereturn True if the two objects are the same else it will return False. EV0000840 · EV0000841 · EV0000842
10. Source slide page 10 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Data Types • Datatype refers to the type of value a variable refers to. • Significance of data type: • Memory associated with it • Operations that can be performed on it. • Range of values allowed in it • Types: • Scalar - Integers, floats, boolean, complex • Reference - List, tuple, set, dict EV0000843 · EV0000844 · EV0000845

11. Source slide page 11 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Type function – type() • A built-in function, that returns the type of the object type(object) • Type of a variable depends on the value assigned to it a = 10 print(type(a)) #int a = 10.0 print(type(a)) # float EV0000846 · EV0000847 · EV0000848
12. Source slide page 12 (extracted text): THANK YOU Introduction to Computer Science Using Python – Dierbach Copyrighth 2013 Johnh Wileyl andd Sons Department of Computer Science and Engineering Dr. Shylaja S S, Director, CCBD & CDSAML, PESU Prof. Sindhu R Pai – sindhurpai@pes.edu Prof. Chitra G M Prof. Gayatri S EV0000849 · EV0000850 · EV0000851

SHORT-NOTE RECAP

Source deck preserved: UE24CS151A PCPS Unit 1 SindhuRPai Class 10 variable id type. Its extracted page text is listed as searchable points and all rendered source pages are attached as figures. Match to a specific video is candidate-based; this index does not claim a manual content review.

EV0000816

29 Source slide deck: Operator Polymorphism

This entry preserves the complete slide deck titled Operator Polymorphism as searchable extracted page text and page-faithful rendered images. Its association with a particular video is not confirmed; it remains a separate topic under Python for computational Problem Solving, Unit_01, to avoid an unsupported match. EV0000936 · EV0000937

Graphs and diagrams

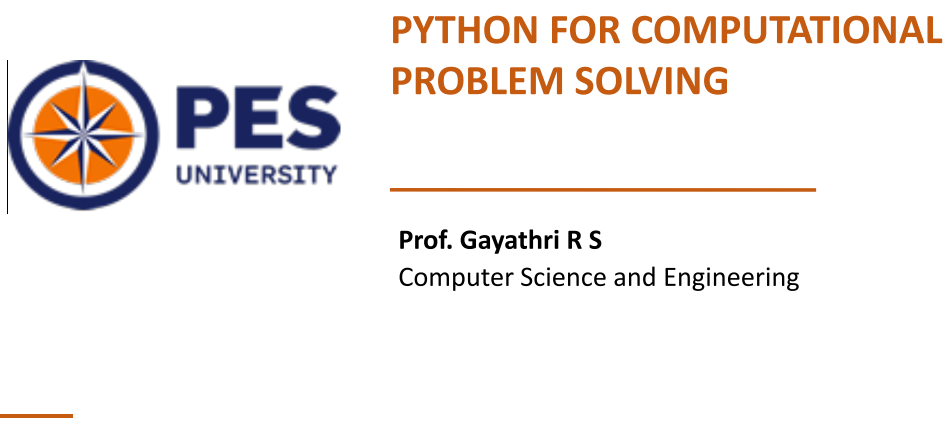


Figure G000203. Operator Polymorphism · source slide page 1.

EV0000937

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

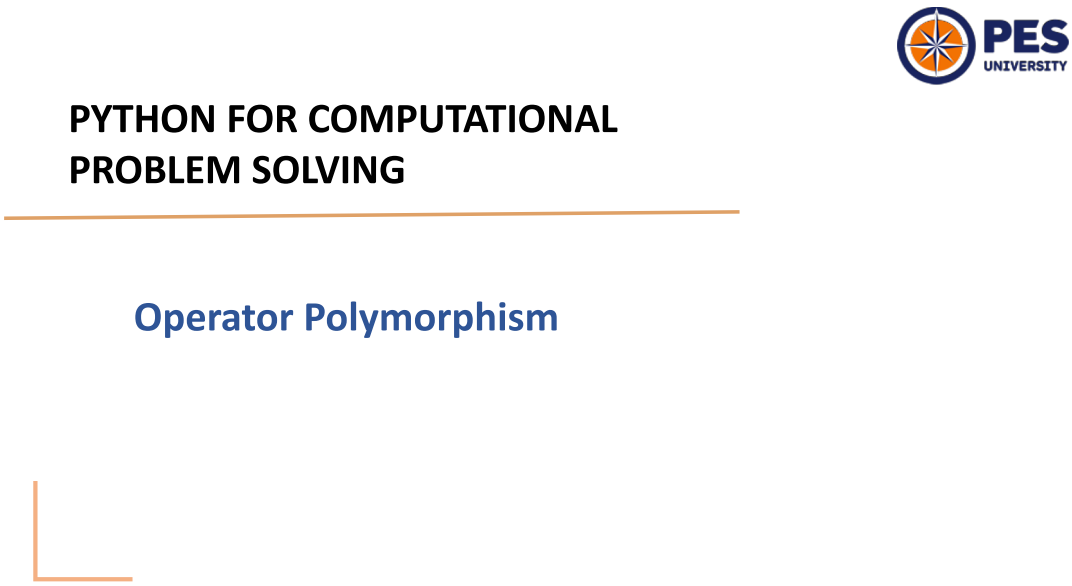


Figure G000204. Operator Polymorphism · source slide page 2.

EV0000940

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

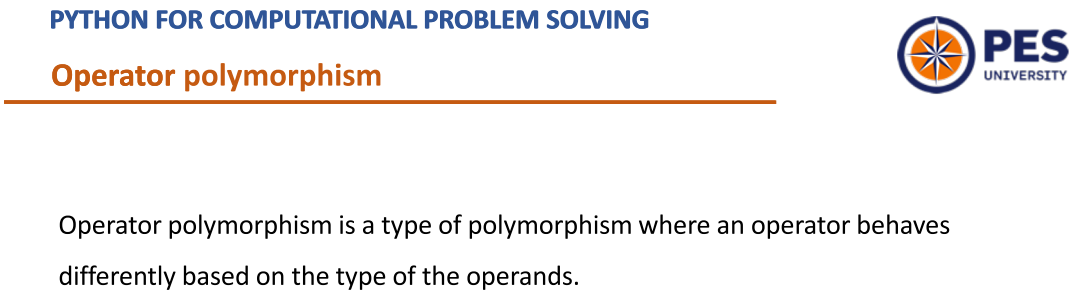


Figure G000205. Operator Polymorphism · source slide page 3.

EV0000943

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

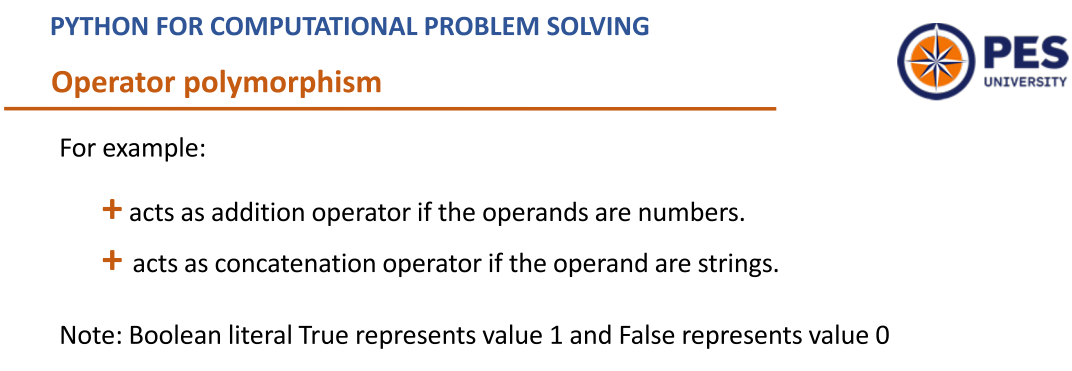


Figure G000206. Operator Polymorphism · source slide page 4.

EV0000946

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

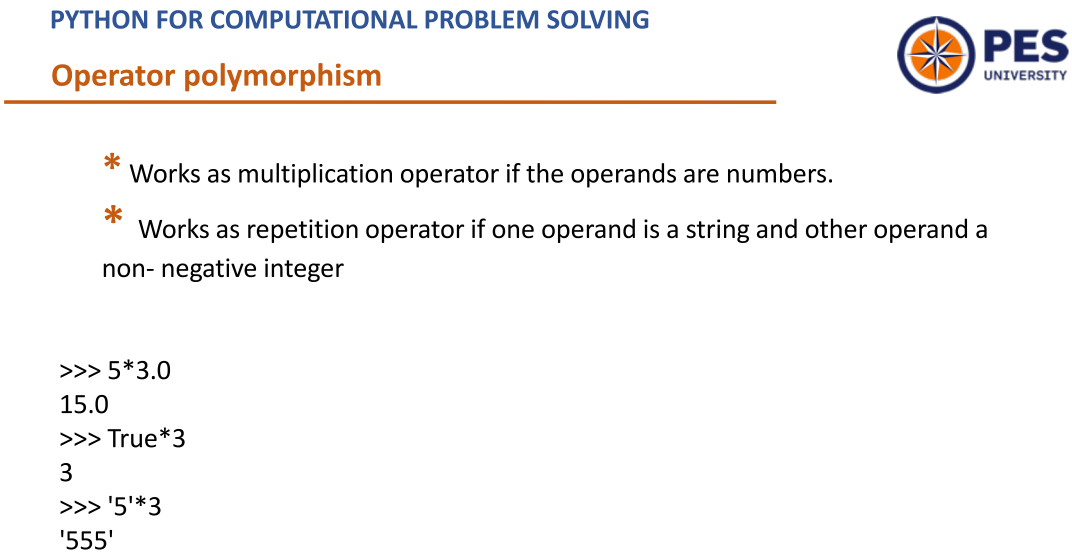


Figure G000207. Operator Polymorphism · source slide page 5.

EV0000949

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Figure G000208. Operator Polymorphism · source slide page 6.

EV0000952

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Important points and why

1. Source slide page 1 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Prof. Gayathri R S Computer Science and Engineering EV0000936 · EV0000937 · EV0000938
2. Source slide page 2 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Operator Polymorphism EV0000939 · EV0000940 · EV0000941
3. Source slide page 3 (extracted text): PYTHONPYTHON FORFOR COMPUTATIONALCOMPUTATIONAL PROBLEMPROBLEM SOLVINGSOLVING OperatorOperator polymorphism Operator polymorphism is a type of polymorphism where an operator behaves differently based on the type of the operands. EV0000942 · EV0000943 · EV0000944
4. Source slide page 4 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Operator polymorphism For example: + acts as addition operator if the operands are numbers. + acts as concatenation operator if the operand are strings. Note: Boolean literal True represents value 1 and False represents value 0 >>> 4+5.0 9.0 >>> True+True 2 >>> '4'+'5.0' '45.0' EV0000945 · EV0000946 · EV0000947
5. Source slide page 5 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Operator polymorphism * Works as multiplication operator if the operands are numbers. Works as repetition operator if one operand is a string and other operand a * non- negative integer >>> 5*3.0 15.0 >>> True*3 3 >>> '5'*3 '555' EV0000948 · EV0000949 · EV0000950
6. Source slide page 6 (extracted text): THANK YOU gayathrirs@pes.edu Department of Computer Science and Engineering EV0000951 · EV0000952 · EV0000953

SHORT-NOTE RECAP

Source deck preserved: Operator Polymorphism. Its extracted page text is listed as searchable points and all rendered source pages are attached as figures. Match to a specific video is candidate-based; this index does not claim a manual content review.

EV0000936

30 Source slide deck: Operator precedence and associativity

This entry preserves the complete slide deck titled Operator precedence and associativity as searchable extracted page text and page-faithful rendered images. Its association with a particular video is not confirmed; it remains a separate topic under Python for computational Problem Solving, Unit_01, to avoid an unsupported match. EV0000954 · EV0000955

Graphs and diagrams

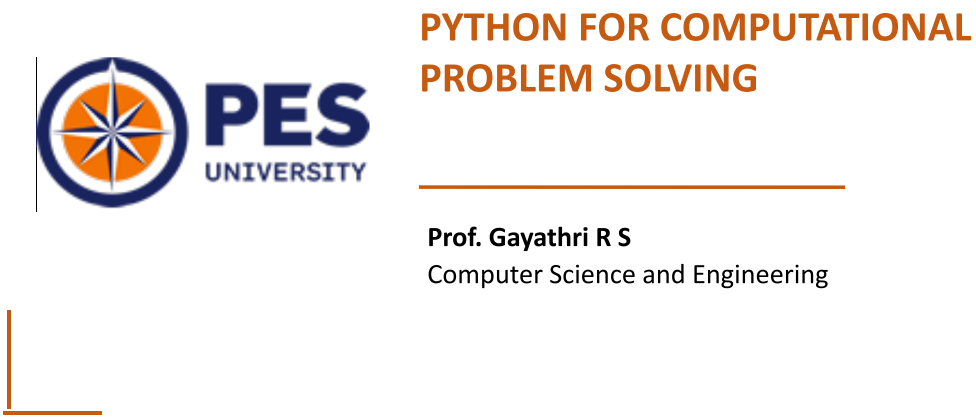


Figure G000209. Operator precedence and associativity · source slide page 1.

EV0000955

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

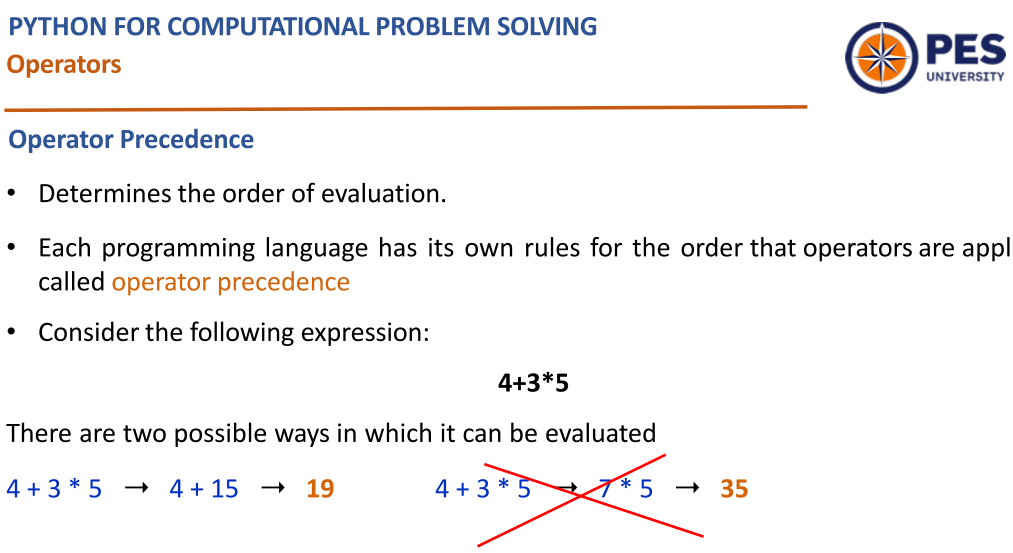


Figure G000210. Operator precedence and associativity · source slide page 2.

EV0000958

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

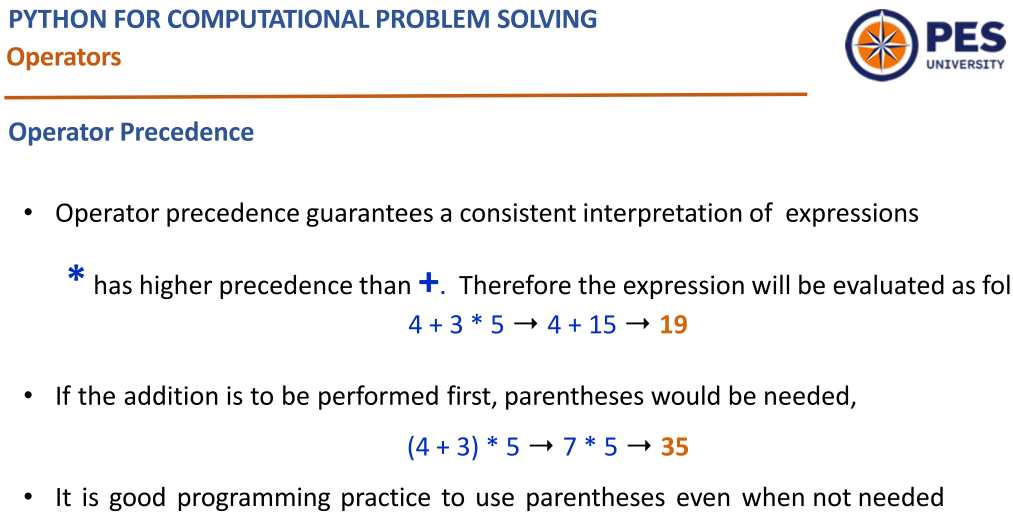


Figure G000211. Operator precedence and associativity · source slide page 3.

EV0000961

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

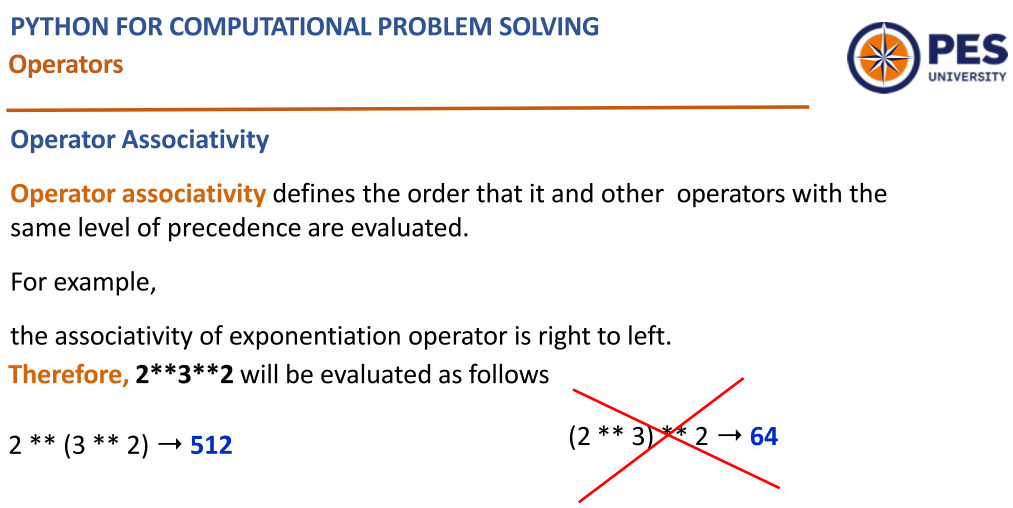


Figure G000212. Operator precedence and associativity · source slide page 4.

EV0000964

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

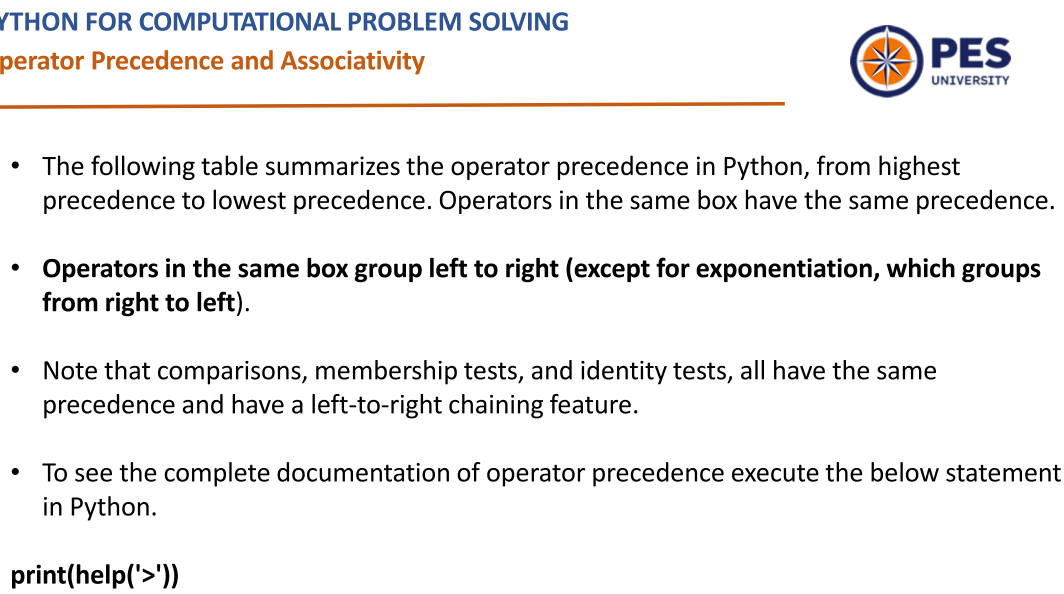
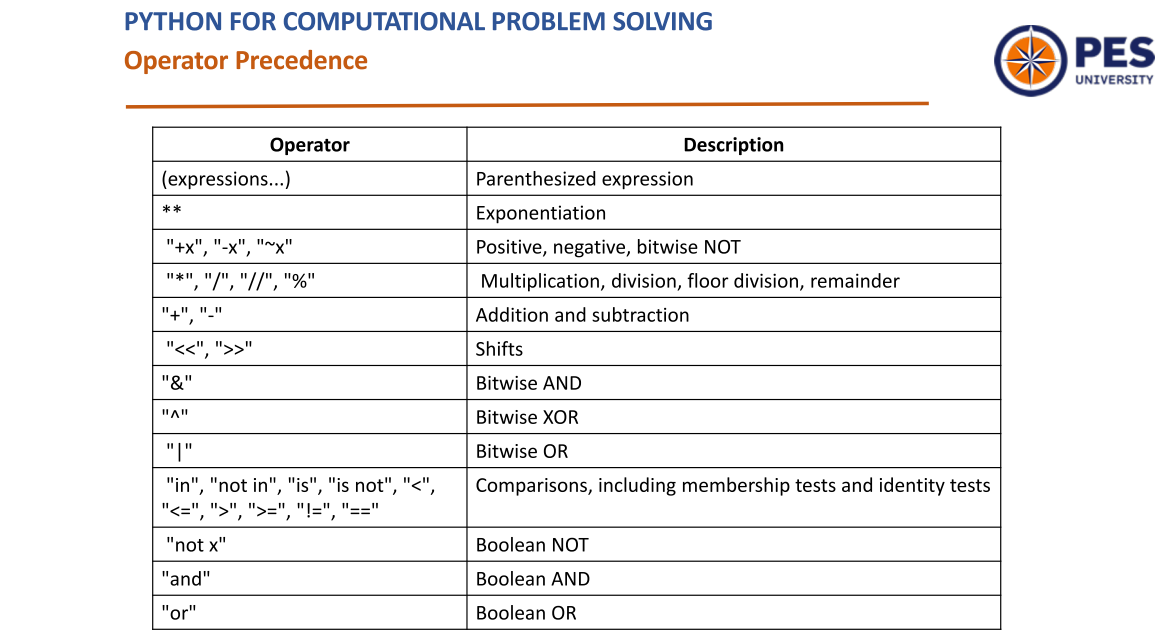


Figure G000213. Operator precedence and associativity · source slide page 5.

EV0000967

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.



Operator	Description
(expressions...)	Parenthesized expression
**	Exponentiation
"+x", "-x", "~x"	Positive, negative, bitwise NOT
"*", "/", "//", "%"	Multiplication, division, floor division, remainder
"+", "-"	Addition and subtraction
"<<", ">>"	Shifts
"&"	Bitwise AND
"^"	Bitwise XOR
"|"	Bitwise OR
"in", "not in", "is", "is not", "<", "<=", ">", ">=", "!=", "=="	Comparisons, including membership tests and identity tests
"not x"	Boolean NOT
"and"	Boolean AND
"or"	Boolean OR

Figure G000214. Operator precedence and associativity · source slide page 6.

EV0000970

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Figure G000215. Operator precedence and associativity · source slide page 7.

EV0000973

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Important points and why

1. Source slide page 1 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Prof. Gayathri R S Computer Science and Engineering EV0000954 · EV0000955 · EV0000956
2. Source slide page 2 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Operators Operator Precedence • Determines the order of evaluation. • Each programming language has its own rules for the order that operators are applied, called operator precedence • Consider the following expression: 4+3*5 There are two possible ways in which it can be evaluated 4 + 3 * 5 → 4 + 15 → 19 4 + 3 * 5 → 7 * 5 → 35 EV0000957 · EV0000958 · EV0000959
3. Source slide page 3 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Operators Operator Precedence • Operator precedence guarantees a consistent interpretation of expressions * has higher precedence than+. Therefore the expression will be evaluated as follows. 4 + 3 * 5 →4 + 15 →19 • If the addition is to be performed first, parentheses would be needed, (4 + 3) * 5 → 7 * 5 → 35 • It is good programming practice to use parentheses even when not needed EV0000960 · EV0000961 · EV0000962
4. Source slide page 4 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Operators Operator Associativity Operator associativity defines the order that it and other operators with the same level of precedence are evaluated. For example, the associativity of exponentiation operator is right to left. Therefore, 2**3**2 will be evaluated as follows (2 ** 3) ** 2 → 642 ** (3 ** 2) → 512 EV0000963 · EV0000964 · EV0000965
5. Source slide page 5 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Operator Precedence and Associativity • The following table summarizes the operator precedence in Python, from highest precedence to lowest precedence. Operators in the same box have the same precedence. • Operators in the same box group left to right (except for exponentiation, which groups from right to left). • Note that comparisons, membership tests, and identity tests, all have the same precedence and have a left-to-right chaining feature. • To see the complete documentation of operator precedence execute the below statement in Python. print(help('>')) EV0000966 · EV0000967 · EV0000968
6. Source slide page 6 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Operator Precedence Operator Description (expressions...) Parenthesized expression ** Exponentiation "+x", "-x", "~x" Positive, negative, bitwise NOT "*", "/", "//", "%" Multiplication, division, floor division, remainder "+", "-" Addition and subtraction "<<", ">>" Shifts "&" Bitwise AND "^" Bitwise XOR "|" Bitwise OR "in", "not in", "is", "is not", "<", Comparisons, including membership tests and identity tests "<=", ">", ">=", "!=", "==" "not x" Boolean NOT "and" Boolean AND "or" Boolean OR EV0000969 · EV0000970 · EV0000971
7. Source slide page 7 (extracted text): THANK YOU Prof. Gayathri R S gayathrirs@pes.edu Department of Computer Science and Engineering EV0000972 · EV0000973 · EV0000974

SHORT-NOTE RECAP

Source deck preserved: Operator precedence and associativity. Its extracted page text is listed as searchable points and all rendered source pages are attached as figures. Match to a specific video is candidate-based; this index does not claim a manual content review.

EV0000954

31 Source slide deck: UE24CS151A PCPS Unit 2 SindhuRPai Class 1 General intro to data structures in python

This entry preserves the complete slide deck titled UE24CS151A PCPS Unit 2 SindhuRPai Class 1 General intro to data structures in python as searchable extracted page text and page-faithful rendered images. Its association with a particular video is not confirmed; it remains a separate topic under Python for computational Problem Solving, Unit_01, to avoid an unsupported match. EV0001035 · EV0001036

Graphs and diagrams

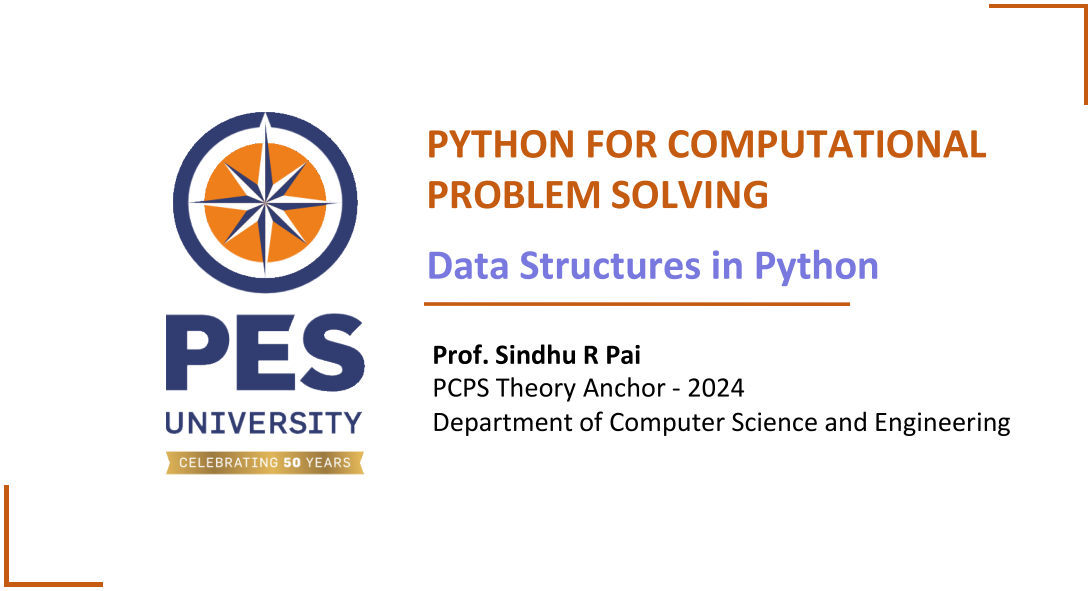


Figure G000236. UE24CS151A PCPS Unit 2 SindhuRPai Class 1 General intro to data structures in python · source slide page 1.

EV0001036

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING
Introduction

PES UNIVERSITY

- **A way of organizing and storing data** so that they can be accessed and worked with efficiently.
- **Becomes more significant with increase in the volume of data** to be stored and retrieved.
- Real life examples:
  - Organization of books in library
  - Organization of clothes
  - File system on a computer

Figure G000237. UE24CS151A PCPS Unit 2 SindhuRPai Class 1 General intro to data structures in python · source slide page 2.

EV0001039

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

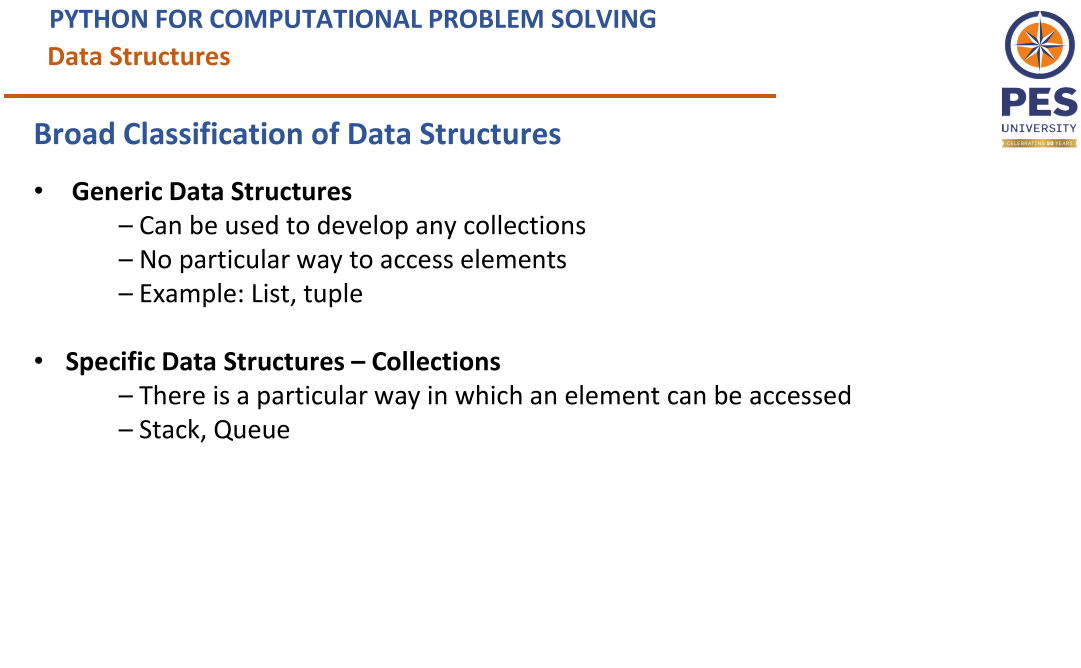


Figure G000238. UE24CS151A PCPS Unit 2 SindhuRPai Class 1 General intro to data structures in python · source slide page 3.

EV0001042

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING
Data Structures

PES UNIVERSITY

Classification Based On Memory Usage

- **Sequence Type Data Structures**
  - Data stored in contiguous manner
  - Elements can be accessed through indexes/subscript notation
  - Enables random access of elements
  - Can store homogeneous or heterogeneous data

- **Non-Sequence Type Data Structures**
  - Data stored in non-contiguous manner
  - No direct indexing
  - No random access of elements
  - Typically stores homogeneous data

Figure G000239. UE24CS151A PCPS Unit 2 SindhuRPai Class 1 General intro to data structures in python · source slide page 4.

EV0001045

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

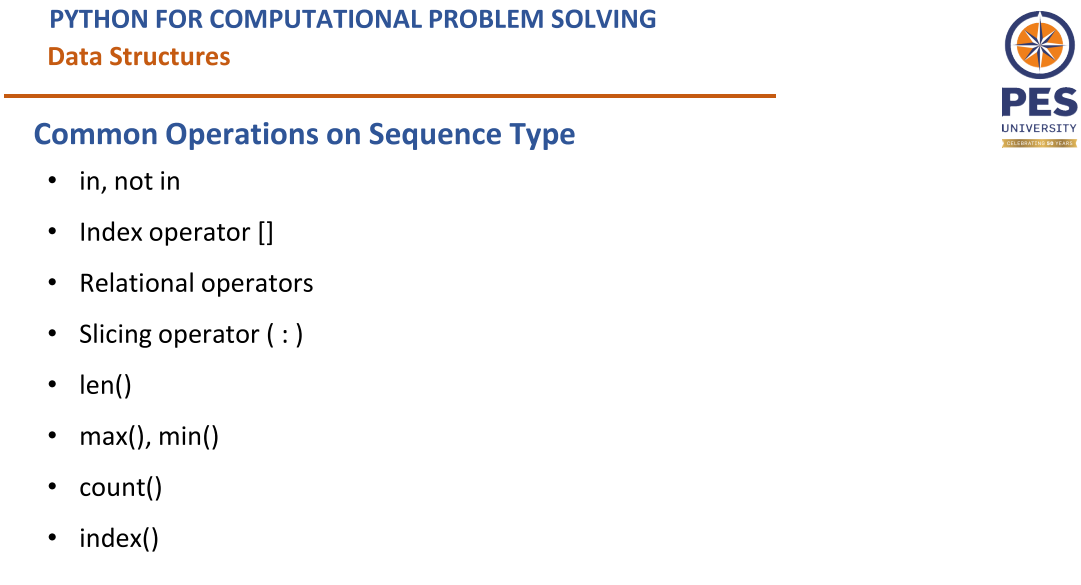


Figure G000240. UE24CS151A PCPS Unit 2 SindhuRPai Class 1 General intro to data structures in python · source slide page 5.

EV0001048

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

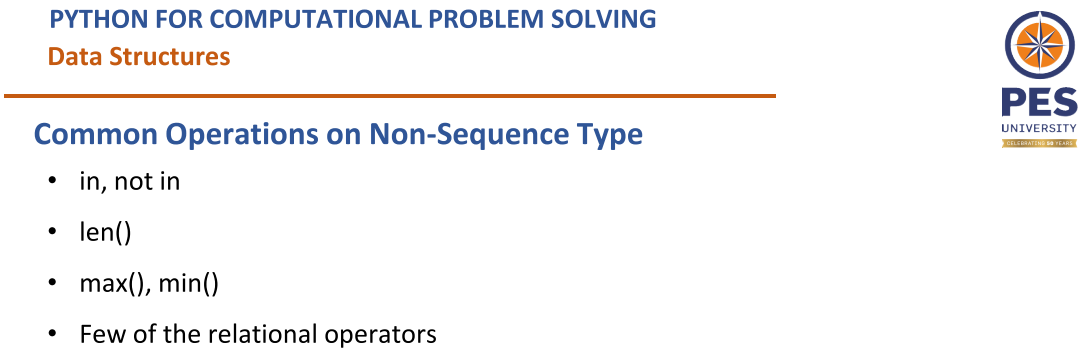


Figure G000241. UE24CS151A PCPS Unit 2 SindhuRPai Class 1 General intro to data structures in python · source slide page 6.

EV0001051

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

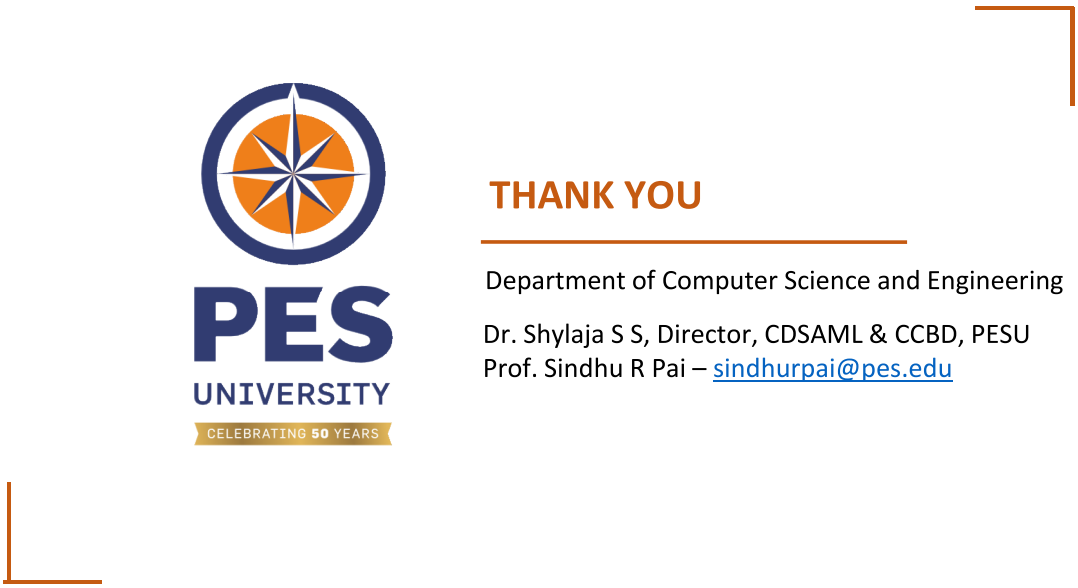


Figure G000242. UE24CS151A PCPS Unit 2 SindhuRPai Class 1 General intro to data structures in python · source slide page 7.

EV0001054

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Important points and why

1. Source slide page 1 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Data Structures in Python Prof. Sindhu R Pai PCPS Theory Anchor - 2024 Department of Computer Science and Engineering EV0001035 · EV0001036 · EV0001037
2. Source slide page 2 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Introduction • A way of organizing and storing data so that they can be accessed and worked with efficiently. •• BecomesBecomes moremore significantsignificant withwith increaseincrease inin thethe volumevolume ofof datadata toto bebe storedstored andand retrieved. • Real life examples: – Organization of books in library – Organization of clothes – File system on a computer EV0001038 · EV0001039 · EV0001040
3. Source slide page 3 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Data Structures Broad Classification of Data Structures • Generic Data Structures – Can be used to develop any collections – No particular way to access elements – Example:Example: ListList, tupletuple • Specific Data Structures – Collections – There is a particular way in which an element can be accessed – Stack, Queue EV0001041 · EV0001042 · EV0001043

4. Source slide page 4 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Data Structures Classification Based On Memory Usage • Sequence Type Data Structures – Data stored in contiguous manner – Elements can be accessed through indexes/subscript notation – EnablesEnables randomrandom accessaccess ofof elementselements – Can store homogeneous or heterogeneous data • Non-Sequence Type Data Structures – Data stored in non-contiguous manner – No direct indexing – No random access of elements – Typically stores homogeneous data EV0001044 · EV0001045 · EV0001046
5. Source slide page 5 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Data Structures Common Operations on Sequence Type • in, not in • Index operator [] • Relational operatorsp • Slicing operator (:) • len() • max(), min() • count() • index() EV0001047 · EV0001048 · EV0001049
6. Source slide page 6 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Data Structures Common Operations on Non-Sequence Type • in, not in • len() • max(),(), min()() • Few of the relational operators EV0001050 · EV0001051 · EV0001052
7. Source slide page 7 (extracted text): THANK YOU Departmentp of Computerp Science and Engineeringg g Dr. Shylaja S S, Director, CDSAML & CCBD, PESU Prof. Sindhu R Pai – sindhurpai@pes.edu EV0001053 · EV0001054 · EV0001055

SHORT-NOTE RECAP

Source deck preserved: UE24CS151A PCPS Unit 2 SindhuRPai Class 1 General intro to data structures in python. Its extracted page text is listed as searchable points and all rendered source pages are attached as figures. Match to a specific video is candidate-based; this index does not claim a manual content review.

EV0001035

32 Source slide deck: UE24CS151A PCPS Unit 2 SindhuRPai Class 2 lists

This entry preserves the complete slide deck titled UE24CS151A PCPS Unit 2 SindhuRPai Class 2 lists as searchable extracted page text and page-faithful rendered images. Its association with a particular video is not confirmed; it remains a separate topic under Python for computational Problem Solving, Unit_01, to avoid an unsupported match. EV0001146 · EV0001147

Graphs and diagrams

Figure G000273. UE24CS151A PCPS Unit 2 SindhuRPai Class 2 lists · source slide page 1.

EV0001147

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

List

- **A list is a collection** that allows us to put many things / values under a single variable.
- Values in the list are called **elements / items**.
- List is an **ordered sequence of items.**
- List is a **linear data structure** where elements have linear ordering.

Figure G000274. UE24CS151A PCPS Unit 2 SindhuRPai Class 2 lists · source slide page 2.

EV0001150

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

List

PES UNIVERSITY

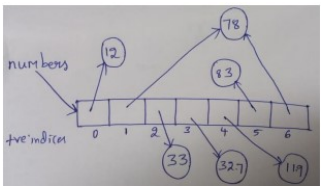
List Characteristics

- Elements in the list can be **heterogeneous** (different datatypes) or **homogeneous** (same datatype).

  heterogeneous = [1,"rahul",3.5,{1,2,3}]

  homogeneous = ["tendulkar","bolt"," federer","messi"]

- Elements are **accessed using indexing operation** (also called **subscript notation**).




```
>>> numbers = [12, 78, 33, 32.7, 11.9, 83, 78]
>>> print (numbers[1])
    78
```

List index always starts with 0, which is called **zero based indexing.**

Figure G000275. UE24CS151A PCPS Unit 2 SindhuRPai Class 2 lists · source slide page 3.

EV0001153

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

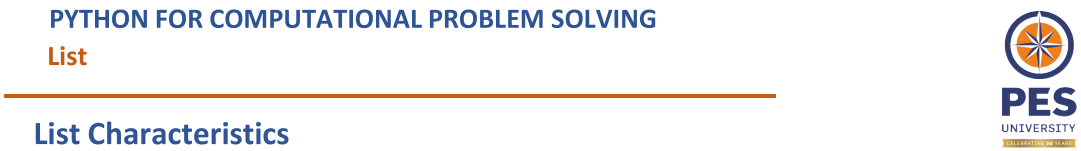


Figure G000276. UE24CS151A PCPS Unit 2 SindhuRPai Class 2 lists · source slide page 4.

EV0001156

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

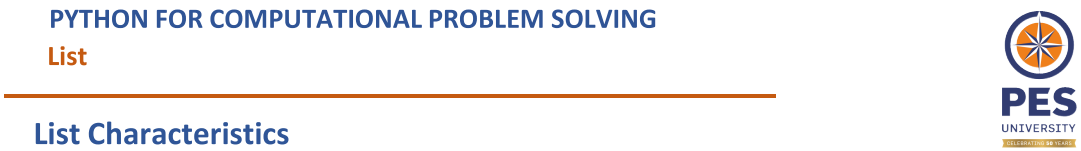


Figure G000277. UE24CS151A PCPS Unit 2 SindhuRPai Class 2 lists · source slide page 5.

EV0001159

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

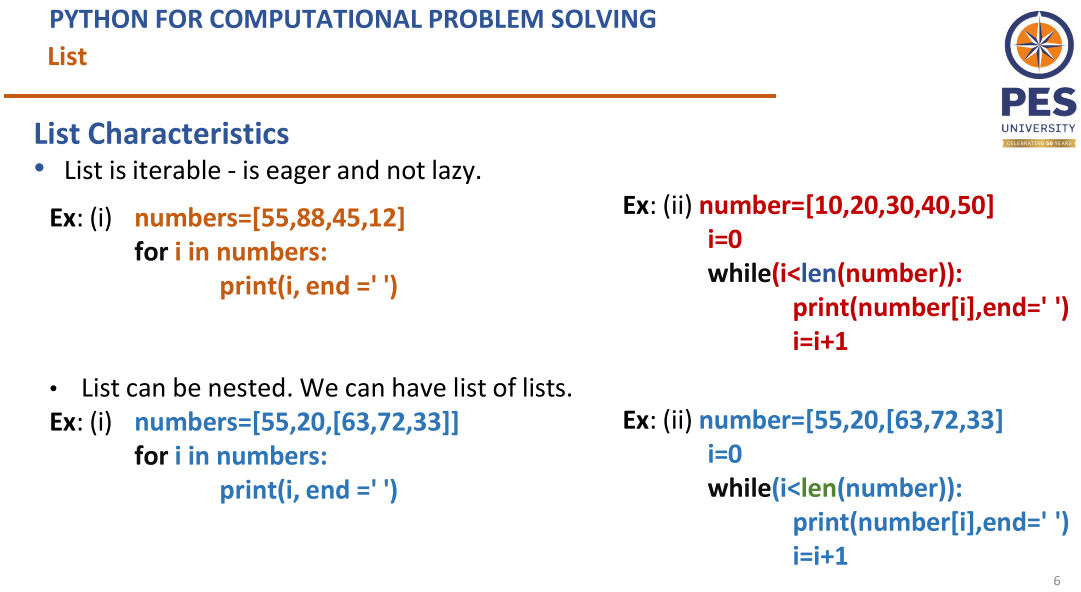


Figure G000278. UE24CS151A PCPS Unit 2 SindhuRPai Class 2 lists · source slide page 6.

EV0001162

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

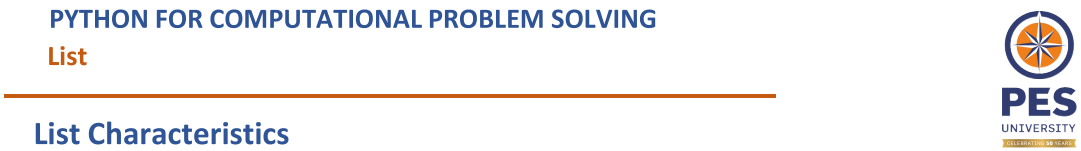


Figure G000279. UE24CS151A PCPS Unit 2 SindhuRPai Class 2 lists · source slide page 7.

EV0001165

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

List

List Characteristics

- List can be sliced. This creates a new (sub)list.

Given lst1 = [12,44,55,89,11,24]

```
>>> lst2 = lst1[::] # creates a copy of lst1. Not same as lst2 = lst1

>>> print(id(lst1))
    2894635511936

>>> print(id(lst2)) #These two are different values
    2894635298304
```

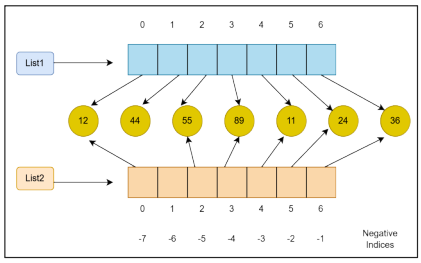


Figure G000280. UE24CS151A PCPS Unit 2 SindhuRPai Class 2 lists · source slide page 8.

EV0001168

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

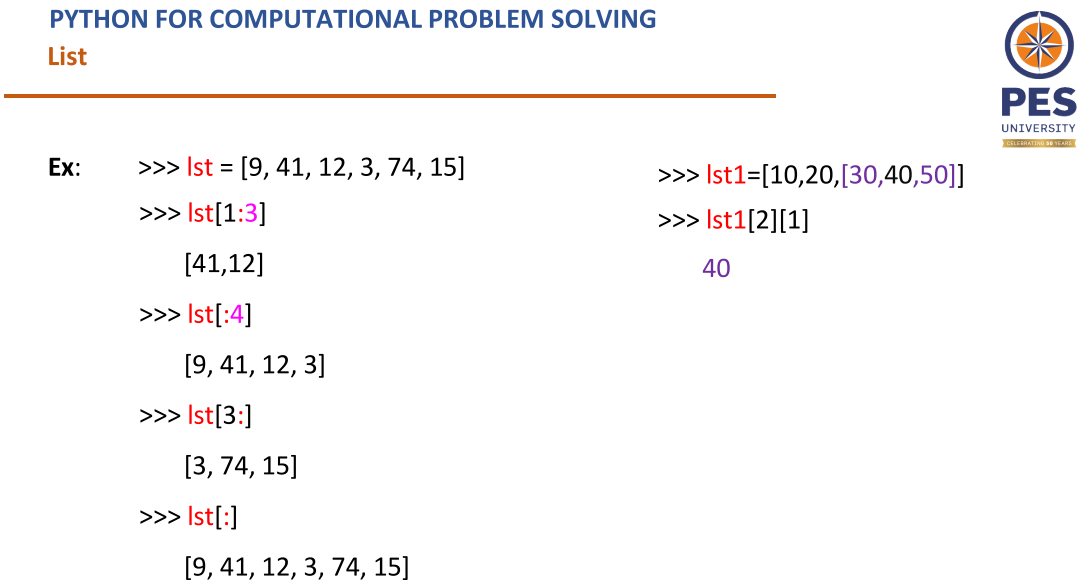


Figure G000281. UE24CS151A PCPS Unit 2 SindhuRPai Class 2 lists · source slide page 9.

EV0001171

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

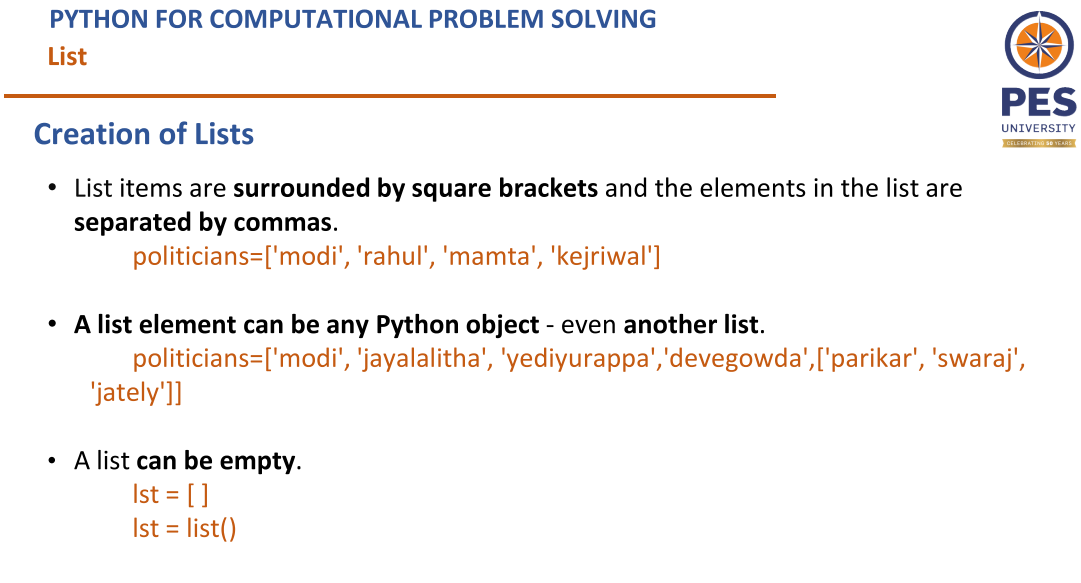


Figure G000282. UE24CS151A PCPS Unit 2 SindhuRPai Class 2 lists · source slide page 10.

EV0001174

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

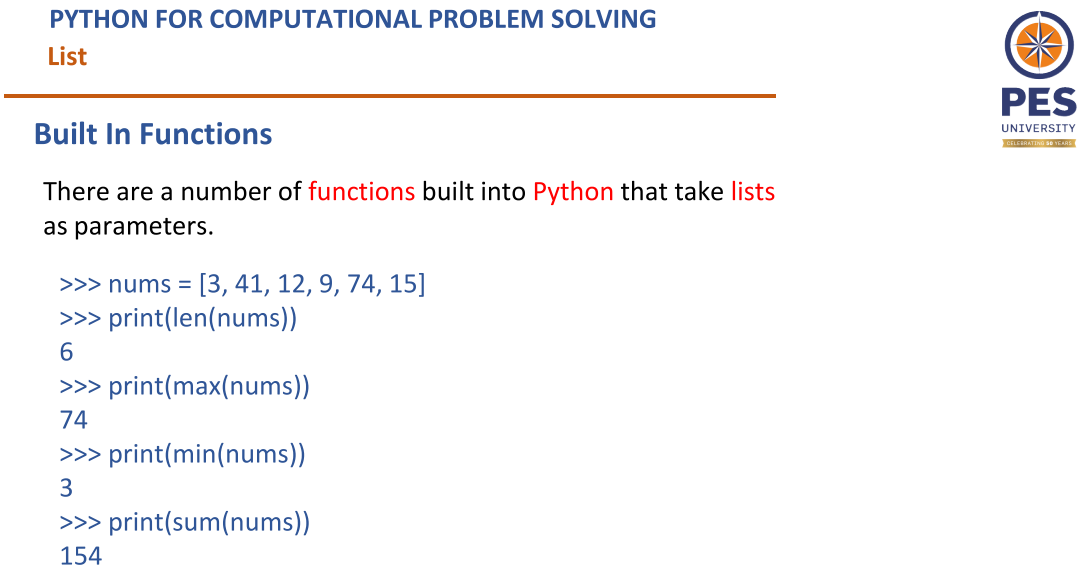


Figure G000283. UE24CS151A PCPS Unit 2 SindhuRPai Class 2 lists · source slide page 11.

EV0001177

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

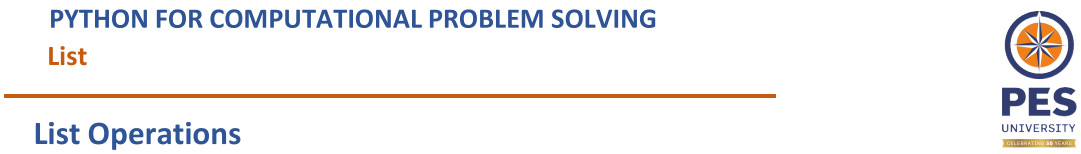


Figure G000284. UE24CS151A PCPS Unit 2 SindhuRPai Class 2 lists · source slide page 12.

EV0001180

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

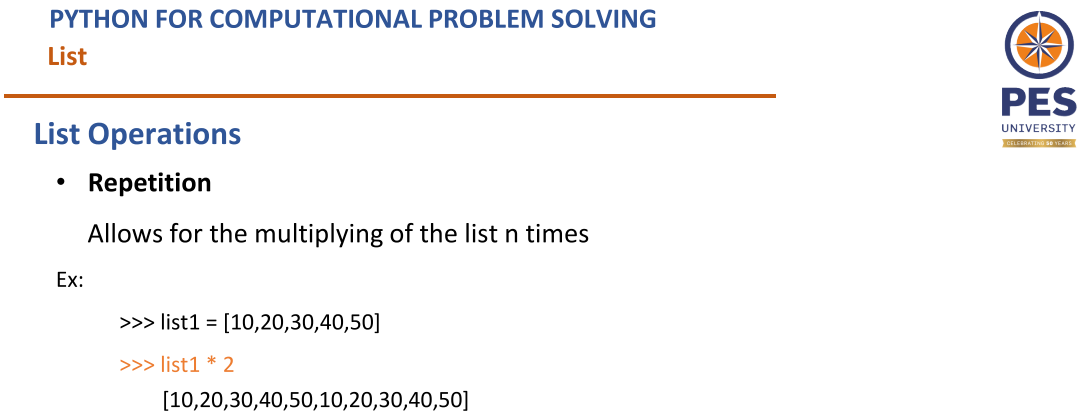


Figure G000285. UE24CS151A PCPS Unit 2 SindhuRPai Class 2 lists · source slide page 13.

EV0001183

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

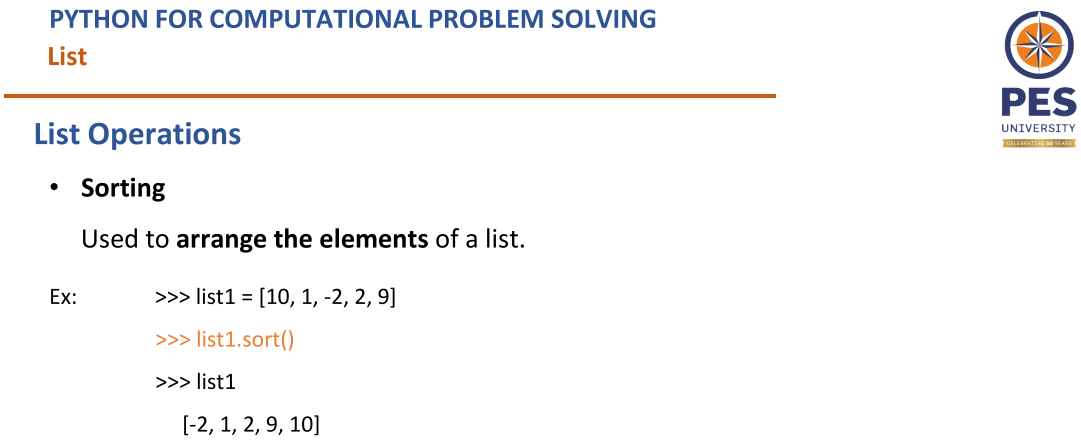


Figure G000286. UE24CS151A PCPS Unit 2 SindhuRPai Class 2 lists · source slide page 14.

EV0001186

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

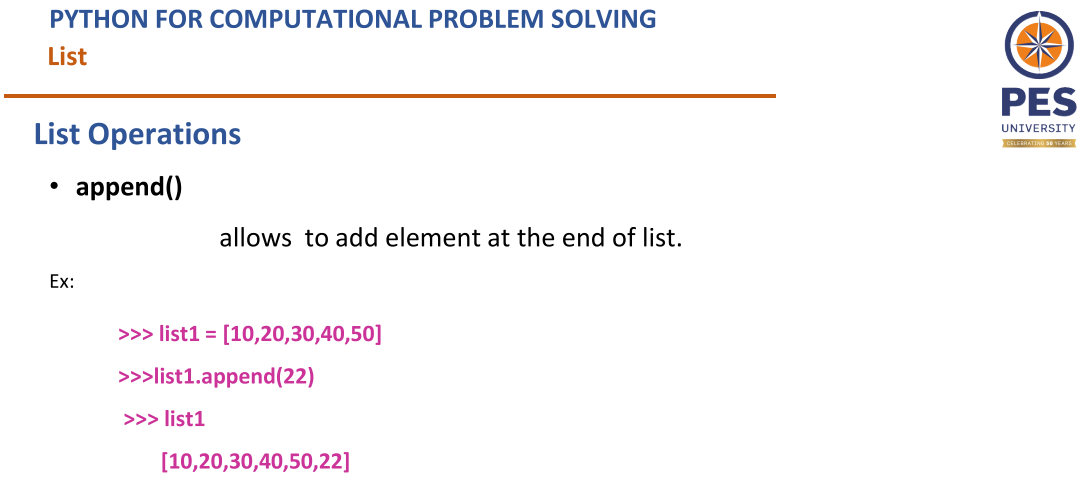


Figure G000287. UE24CS151A PCPS Unit 2 SindhuRPai Class 2 lists · source slide page 15.

EV0001189

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

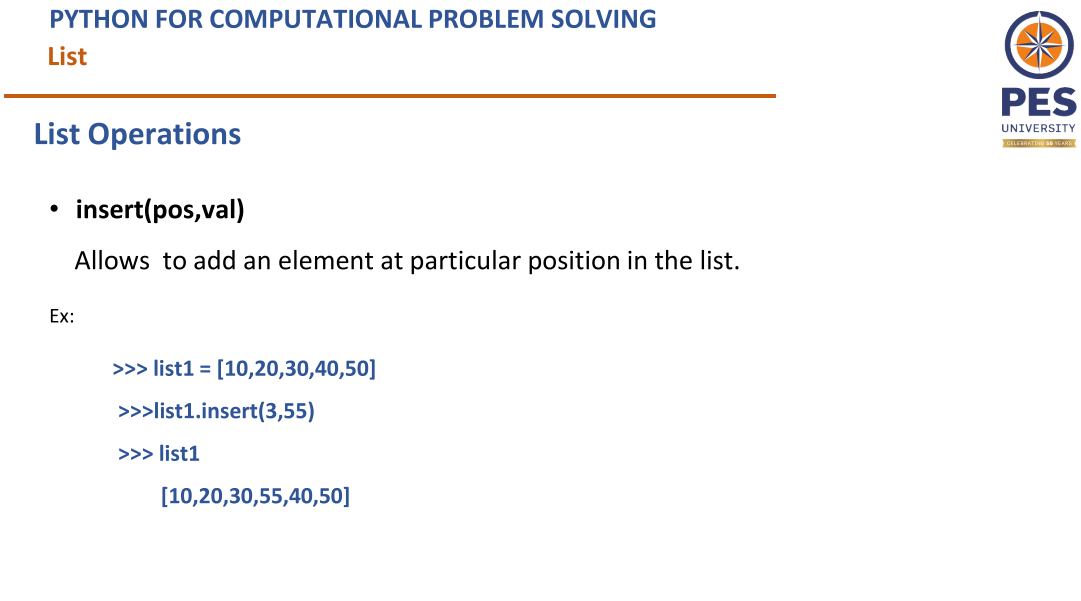


Figure G000288. UE24CS151A PCPS Unit 2 SindhuRPai Class 2 lists · source slide page 16.

EV0001192

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

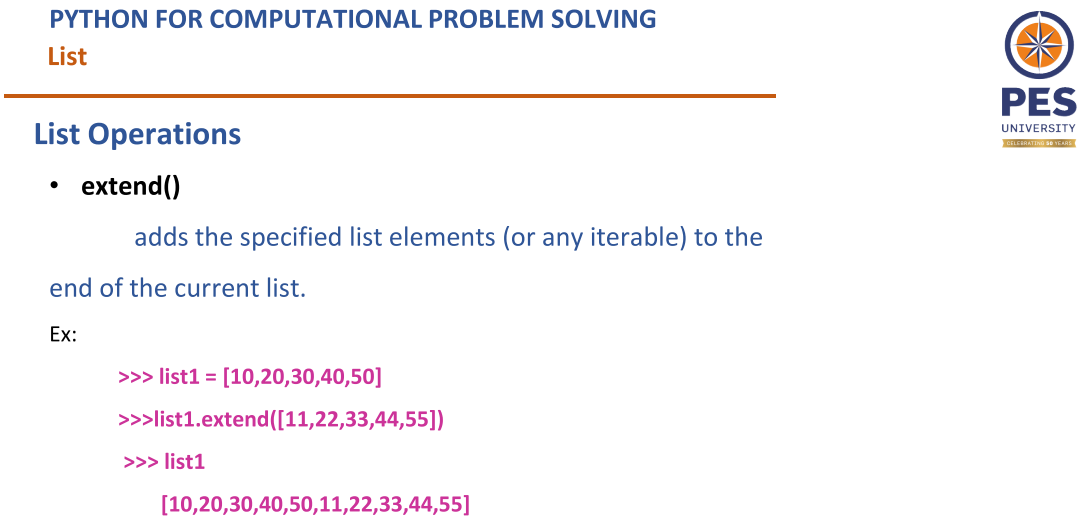


Figure G000289. UE24CS151A PCPS Unit 2 SindhuRPai Class 2 lists · source slide page 17.

EV0001195

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

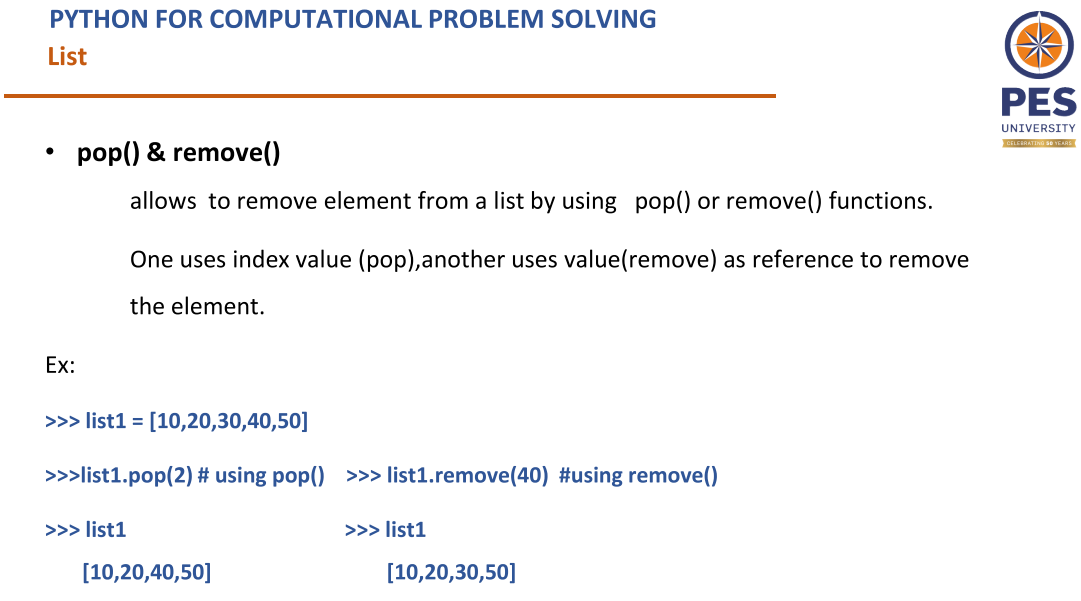


Figure G000290. UE24CS151A PCPS Unit 2 SindhuRPai Class 2 lists · source slide page 18.

EV0001198

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

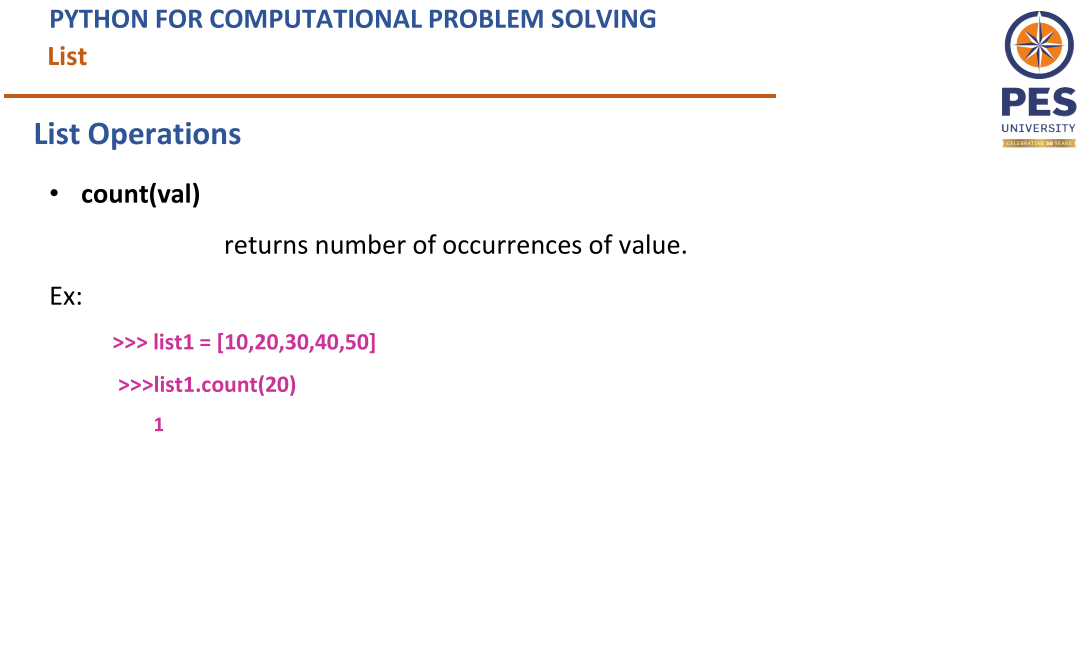


Figure G000291. UE24CS151A PCPS Unit 2 SindhuRPai Class 2 lists · source slide page 19.

EV0001201

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

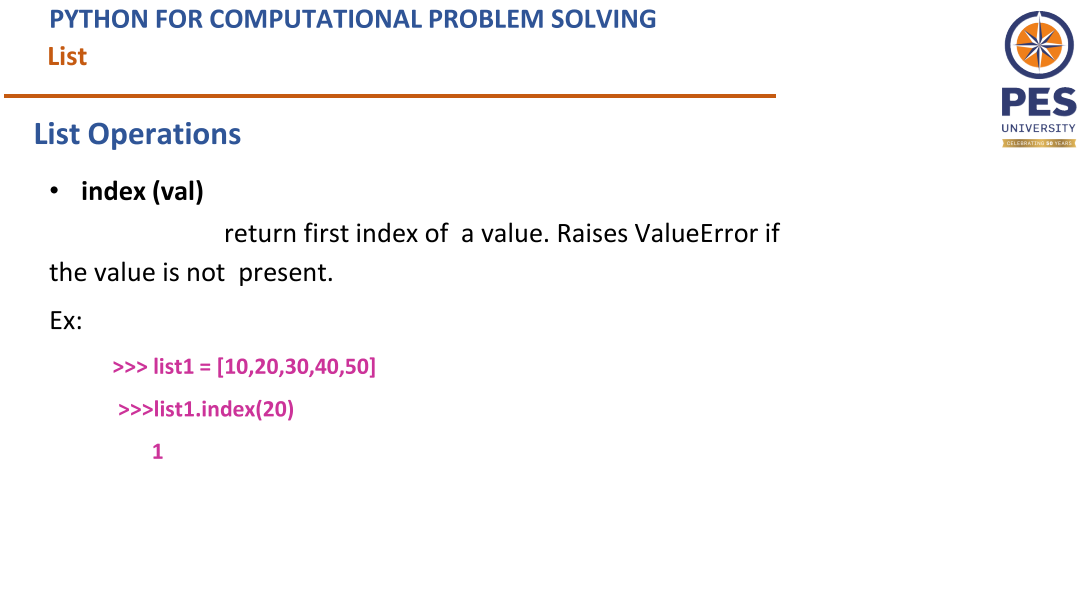


Figure G000292. UE24CS151A PCPS Unit 2 SindhuRPai Class 2 lists · source slide page 20.

EV0001204

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

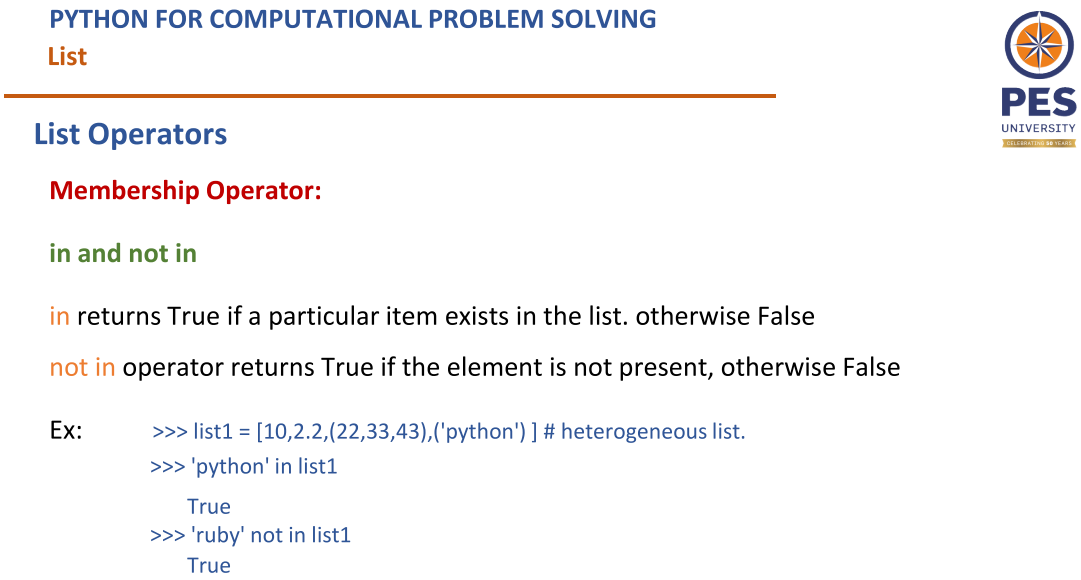


Figure G000293. UE24CS151A PCPS Unit 2 SindhuRPai Class 2 lists · source slide page 21.

EV0001207

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

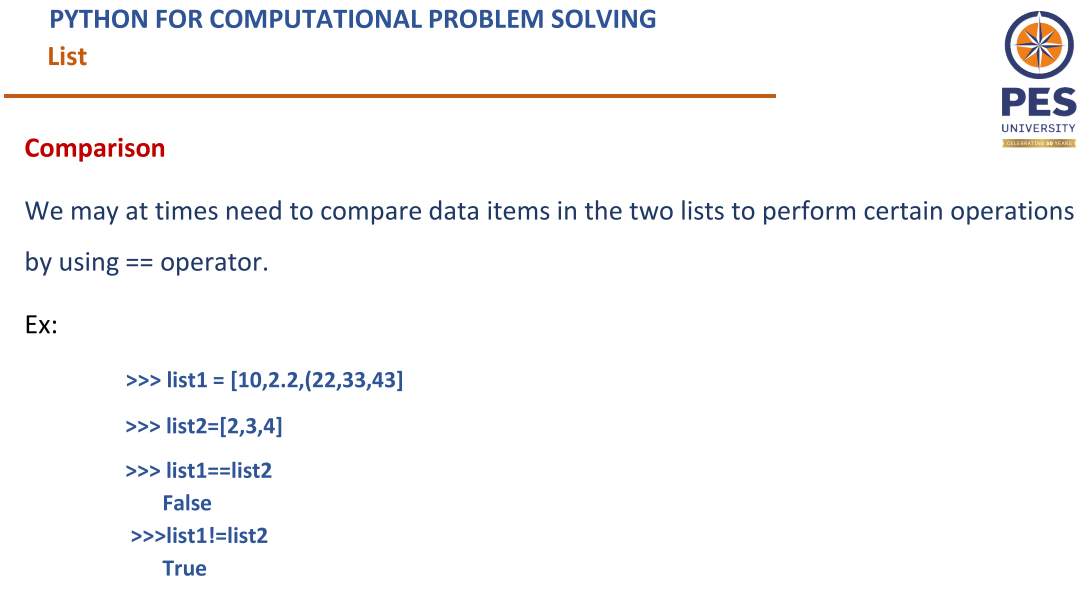


Figure G000294. UE24CS151A PCPS Unit 2 SindhuRPai Class 2 lists · source slide page 22.

EV0001210

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

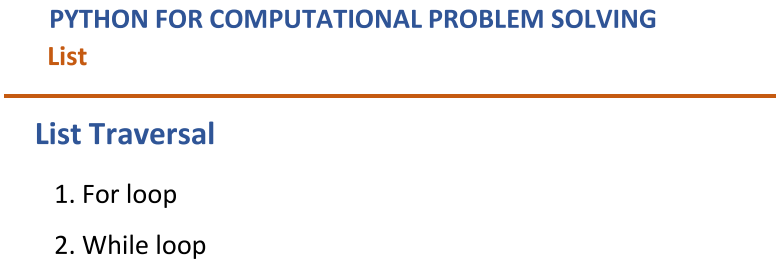
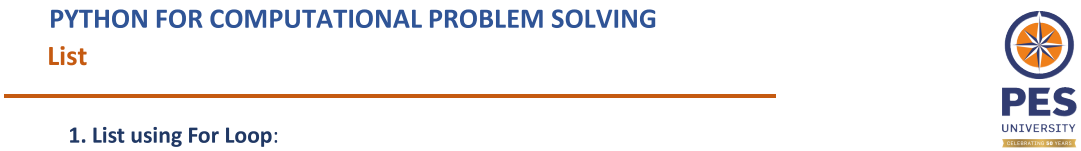


Figure G000295. UE24CS151A PCPS Unit 2 SindhuRPai Class 2 lists · source slide page 23.

EV0001213

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.



1. List using For Loop:

- The for loop in Python is used to iterate over a sequence (list, tuple, string) or other iterable objects.
- Iterating over a sequence is called traversal.
- Loop continues until we reach the last item in the sequence
- The body of for loop is separated from the rest of the code using indentation.

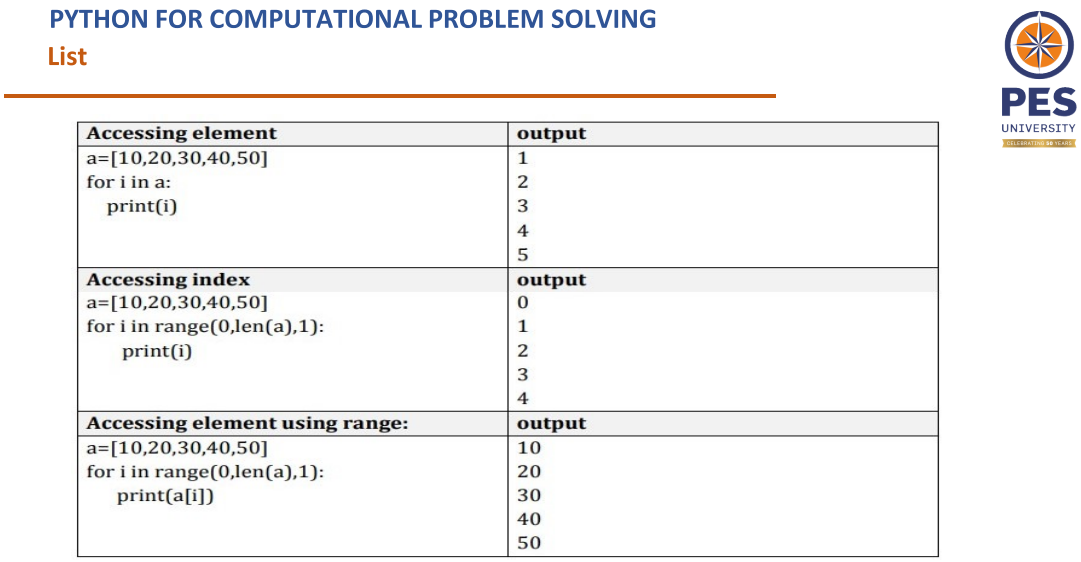
Syntax:

for val in sequence:

Figure G000296. UE24CS151A PCPS Unit 2 SindhuRPai Class 2 lists · source slide page 24.

EV0001216

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.



Accessing element	output
a=[10,20,30,40,50] for i in a: print(i)	1 2 3 4 5
Accessing index	**output**
a=[10,20,30,40,50] for i in range(0,len(a),1): print(i)	0 1 2 3 4
Accessing element using range:	**output**
a=[10,20,30,40,50] for i in range(0,len(a),1): print(a[i])	10 20 30 40 50

Figure G000297. UE24CS151A PCPS Unit 2 SindhuRPai Class 2 lists · source slide page 25.

EV0001219

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

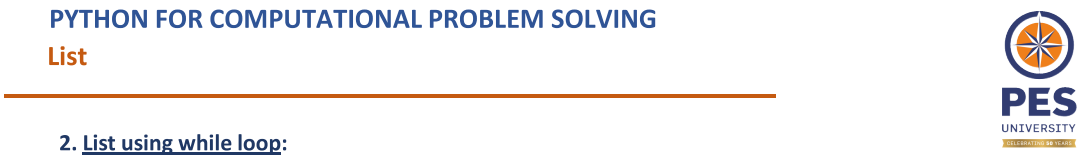


Figure G000298. UE24CS151A PCPS Unit 2 SindhuRPai Class 2 lists · source slide page 26.

EV0001222

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

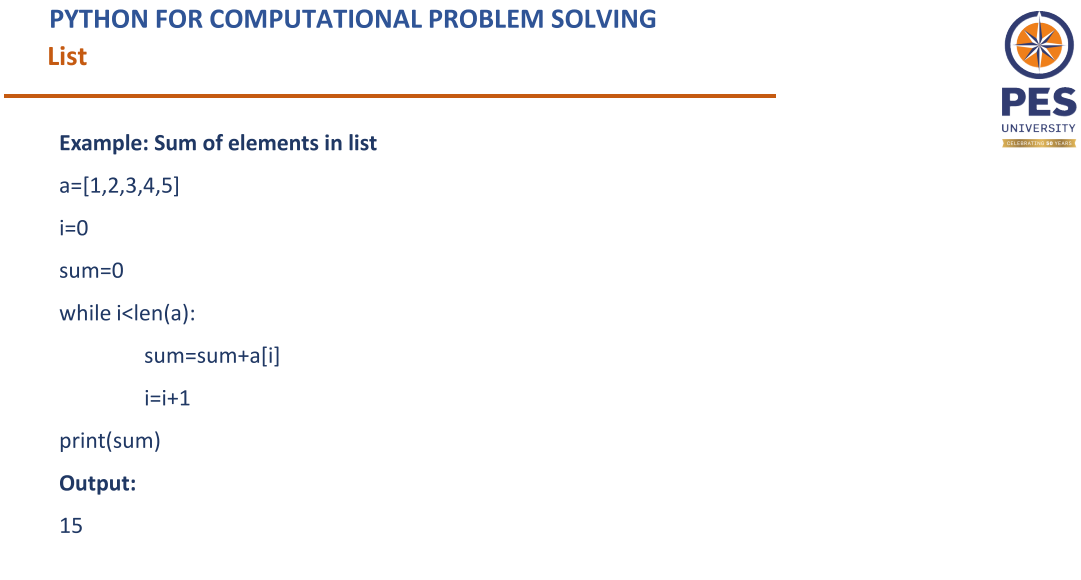


Figure G000299. UE24CS151A PCPS Unit 2 SindhuRPai Class 2 lists · source slide page 27.

EV0001225

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

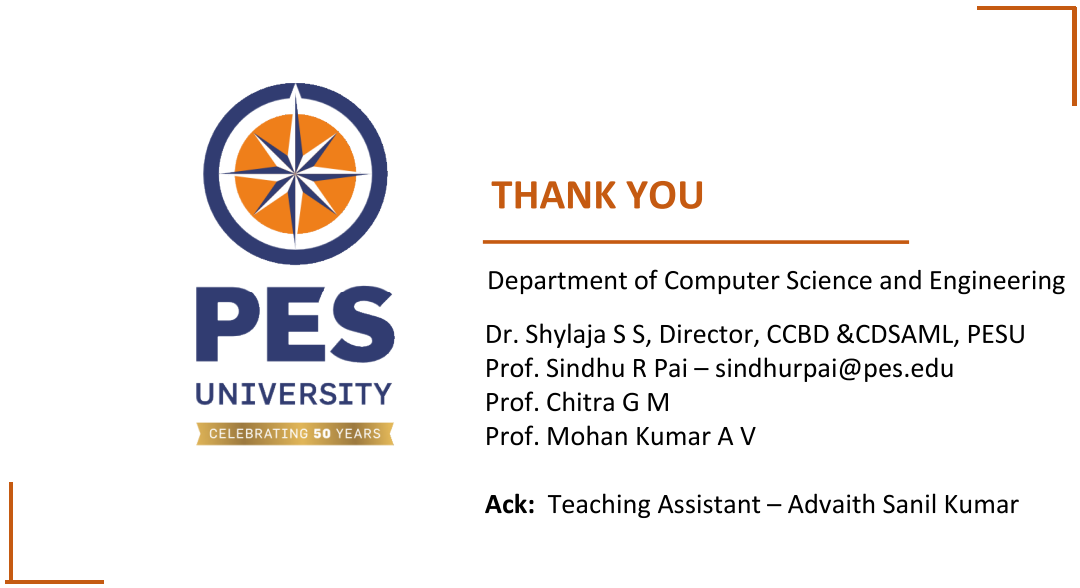


Figure G000300. UE24CS151A PCPS Unit 2 SindhuRPai Class 2 lists · source slide page 28.

EV0001228

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Important points and why

1. Source slide page 1 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING List in python Prof. Sindhu R Pai PCPS Theory Anchor - 2024 Department of Computer Science and Engineering EV0001146 · EV0001147 · EV0001148
2. Source slide page 2 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING List • A list is a collection that allows us to put many things / values under a single variable. • Values in the list are called elements / items. • List is an ordered sequence of items. • List is a linear data structure where elements have linear ordering. EV0001149 · EV0001150 · EV0001151
3. Source slide page 3 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING List List Characteristics • Elements in the list can be heterogeneous (different datatypes) or homogeneous (same datatype). heterogeneous = [1,"rahul",3.5,{1,2,3}] homogeneoushomogeneous == ["tendulkar"[tendulkar , "bolt"bolt ," federer"federer , "messi"]messi] • Elements are accessed using indexing operation (also called subscript notation). >>> numbers = [12, 78, 33, 32.7, 11.9, 83, 78] List index always starts >>> print (numbers[1]) with 0, which is called 78 zero based indexing. EV0001152 · EV0001153 · EV0001154

4. Source slide page 4 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING List List Characteristics • Lists are mutable, as it can grow and shrink • List is iterable - is eager and not lazy. •• AssignmentAssignment ofof oneone listlist totoanotheranother causescauses bothboth totoreferrefer totothethe samesame listlist. • List can be sliced. This creates a new (sub)list. EV0001155 · EV0001156 · EV0001157
5. Source slide page 5 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING List List Characteristics • Lists are mutable, as list can grow or shrink . o We can change an element of a list. Ex: >>> numbers=[55,88,45,12] >>> numbers[0]=10 # index operation is used. >>> numbers [10, 88, 45, 12] EV0001158 · EV0001159 · EV0001160
6. Source slide page 6 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING List List Characteristics • List is iterable - is eager and not lazy. Ex: (ii) number=[10,20,30,40,50] Ex: (i) numbers=[55,88,45,12] i=0 for i in numbers: while(i<len(number)): print(i,print(i, endend =' ')) print(number[i],end=' ') i=i+1 • List can be nested. We can have list of lists. Ex: (i) numbers= [55,20,[63,72,33]] Ex: (ii) number=[55,20,[63,72,33] for i in numbers: i=0 print(i, end =' ') while(i<len(number)): print(number[i],end=' ') i=i+1 6 EV0001161 · EV0001162 · EV0001163
7. Source slide page 7 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING List List Characteristics • Assignment of one list to another causes both to refer to the same list. Ex: >>> list1=[12,44,55,89,11,24] >>> list2=list1 >>> print(id(list1)) 2894590353408 >>> print(id(list2)) 2894590353408 Note: In Python, the id() function is a built-in function that returns the unique identifier of an object. EV0001164 · EV0001165 · EV0001166
8. Source slide page 8 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING List List Characteristics • List can be sliced. This creates a new (sub)list. Given lst1 = [12,44,55,89,11,24] >>> llst22 = llst1[::]1[ ] ## creates a copy off llst1.1 NNot same as llst22 = llst11 >>> print(id(lst1)) 2894635511936 >>> print(id(lst2)) #These two are different values 2894635298304 EV0001167 · EV0001168 · EV0001169
9. Source slide page 9 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING List Ex: >>> lst = [9, 41, 12, 3, 74, 15] >>> lst1=[10,20,[30,40,50]] >>> lst[1:3] >>> lst1[2][1] [41,12] 40 >>> lst[:4] [9, 41, 12, 3] >>> lst[3:] [3, 74, 15] >>> lst[:] [9, 41, 12, 3, 74, 15] EV0001170 · EV0001171 · EV0001172
10. Source slide page 10 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING List Creation of Lists • List items are surrounded by square brackets and the elements in the list are separated by commas. politicians=['modi', 'rahul', 'mamta', 'kejriwal'] • A list element can be any Python object - even another list. politicians=['modi', 'jayalalitha', 'yediyurappa','devegowda', ['parikar', 'swaraj', 'jately']] • A list can be empty. lst = [ ] lst = list() EV0001173 · EV0001174 · EV0001175
11. Source slide page 11 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING List Built In Functions There are a number of functions built into Python that take lists as parameters. >>> numsnums = [3,[3, 41,41, 12,12, 9,9, 74,74, 15]15] >>> print(len(nums)) 6 >>> print(max(nums)) 74 >>> print(min(nums)) 3 >>> print(sum(nums)) 154 EV0001176 · EV0001177 · EV0001178

12. Source slide page 12 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING List List Operations • Concatenation We can create a new list by adding two existing lists together. Ex:Ex: >>> list1 = [10,20,30,40,50] >>> list2 =[100,200,300,400,500] >>> list1 + list2 #concatenates two lists [10,20,30,40,50, 100,200,300,400,500] EV0001179 · EV0001180 · EV0001181
13. Source slide page 13 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING List List Operations • Repetition Allows for the multiplying of the list n times Ex: >>> list1 = [10,20,30,40,50] >>> list1 * 2 [10,20,30,40,50,10,20,30,40,50] EV0001182 · EV0001183 · EV0001184
14. Source slide page 14 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING List List Operations • Sorting Used to arrange the elements of a list. EEx: >>>>>> lilist1t1 = [10[10, 11, -2,2 22, 9]9] >>> list1.sort() >>> list1 [-2, 1, 2, 9, 10] EV0001185 · EV0001186 · EV0001187
15. Source slide page 15 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING List List Operations • append() allows to add element at the end of list. Ex: >>> list1 = [10,20,30,40,50] >>>list1.append(22) >>> list1 [10,20,30,40,50,22] EV0001188 · EV0001189 · EV0001190
16. Source slide page 16 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING List List Operations • insert(pos,val) Allows to add an element at particular position in the list. Ex: >>> list1 = [10,20,30,40,50] >>>list1.insert(3,55) >>> list1 [10,20,30,55,40,50] EV0001191 · EV0001192 · EV0001193
17. Source slide page 17 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING List List Operations • extend() adds the specified list elements (or any iterable) to the endend ofof thethe currentcurrent listlist. Ex: >>> list1 = [10,20,30,40,50] >>>list1.extend([11,22,33,44,55]) >>> list1 [10,20,30,40,50,11,22,33,44,55] EV0001194 · EV0001195 · EV0001196
18. Source slide page 18 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING List • pop() & remove() allows to remove element from a list by using pop() or remove() functions. One uses index value (pop),another uses value(remove) as reference to remove the element. Ex: >>> list1 = [10,20,30,40,50] >>>list1.pop(2) # using pop() >>> list1.remove(40) #using remove() >>> list1 >>> list1 [10,20,40,50] [10,20,30,50] EV0001197 · EV0001198 · EV0001199
19. Source slide page 19 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING List List Operations • count(val) returns number of occurrences of value. Ex:Ex: >>> list1 = [10,20,30,40,50] >>>list1.count(20) 1 EV0001200 · EV0001201 · EV0001202
20. Source slide page 20 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING List List Operations • index (val) return first index of a value. Raises ValueError if the value is not present. Ex: >>> list1 = [10,20,30,40,50] >>>list1.index(20) 1 EV0001203 · EV0001204 · EV0001205
21. Source slide page 21 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING List List Operators Membership Operator: in and not in in returns True if a particular item exists in the list. otherwise False not in operator returns True if the element is not present, otherwise False Ex: >>> list1 = [10,2.2,(22,33,43),('python')] # heterogeneous list. >>> 'python' in list1 True >>> 'ruby' not in list1 True EV0001206 · EV0001207 · EV0001208
22. Source slide page 22 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING List Comparison We may at times need to compare data items in the two lists to perform certain operations by using == operator. Ex: >>> list1 = [10,2.2,(22,33,43] >>> list2=[2,3,4] >>> list1==list2 False >>>list1!=list2 True EV0001209 · EV0001210 · EV0001211

23. Source slide page 23 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING List List Traversal 1. For loop 2. While loop EV0001212 · EV0001213 · EV0001214
24. Source slide page 24 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING List 1. List using For Loop: ☒ The for loop in Python is used to iterate over a sequence (list, tuple, string) or other iterable objects. ☒ Iterating over a sequence is called traversal. ☒ Loop continues until we reach the last item in the sequence ☒ The body of for loop is separated from the rest of the code using indentation. Syntax: for val in sequence: EV0001215 · EV0001216 · EV0001217
25. Source slide page 25 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING List EV0001218 · EV0001219 · EV0001220
26. Source slide page 26 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING List 2. List using while loop: The while loop in Python is used to iterate over a block of code as long as the test expression (condition) is true. When the condition is tested and the result is false, the loop body will be skipped and the first statement after the while loop will be executed. Syntax: while (condition): body of while EV0001221 · EV0001222 · EV0001223
27. Source slide page 27 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING List Example: Sum of elements in list a=[1,2,3,4,5] i=0 sum=0 while i<len(a): sum=sum+a[i] i=i+1 print(sum) Output: 15 EV0001224 · EV0001225 · EV0001226
28. Source slide page 28 (extracted text): THANK YOU Departmentp of Computerp Science and Engineeringg g Dr. Shylaja S S, Director, CCBD &CDSAML, PESU Prof. Sindhu R Pai – sindhurpai@pes.edu Prof. Chitra G M Prof. Mohan Kumar A V Ack: Teaching Assistant – Advaith Sanil Kumar EV0001227 · EV0001228 · EV0001229

SHORT-NOTE RECAP

Source deck preserved: UE24CS151A PCPS Unit 2 SindhuRPai Class 2 lists. Its extracted page text is listed as searchable points and all rendered source pages are attached as figures. Match to a specific video is candidate-based; this index does not claim a manual content review.
EV0001146

33 Historical question-source pages · unit mapping candidates

This section preserves historical paper/question-bank pages whose extracted terms matched Python for computational Problem Solving, Unit_01 (Unit 1: Introduction) above the dataset's candidate threshold. The association has not been manually confirmed, source text/OCR may be incomplete, and the pages are not rewritten or presented as solved questions. EV0001230 · EV0001231

Important points and why

1. Archive page 1 from Dec__1aef5kbQ-b.pdf; unit match candidate score 0.75; source text/OCR may be partial. 11/8/24, 12:59 PM ESA - DEC 2023 -UE22CS151A about:blank 1/8 (5.0 Marks) (5.0 Marks) (5.0 Marks) PES University, Bengaluru (Established under Karnataka Act 16 of 2013) END SEMESTER ASSESSMENT (ESA) - DEC 2023 UE22CS151A - Python for Computational Problem Solving Total Marks : 100.0 1.a. Classify & group the following devices into Input/ Output/ Processing/ Storage/ Communication devices. Central Processing Unit, Modem, USB Stick, Microphone, Blue-Ray device, Speakers, Graphics Processing Unit, Touch screen, Projectors, Blue-tooth. 1.b. Evaluate the following expressions: i) a='rama'>'ravana'; print(a) ii) print("1" * 2, 2 * "1") iii) a = 20; b = 5; print(a&b) iv) 5(2 + 3 * 4) v) import math; print(math.sqrt(-25)) 1.c. Write a Python program that prompts the user to enter positive integer values, and continues to prompt the user until a value of -1 is entered, adding up all the value entered and displaying the result. Input = Microphone, Microphone, Output = Blue-ray, Projector, Speakers Processing = CPU, GPU, Storage = USB stick Comms = Modem, Blue-tooth False 11 11 4 TypeError: 'int' object is not callable or 70 ValueError: math domain error EV0001230
2. Archive page 2 from Dec__1aef5kbQ-b.pdf; unit match candidate score 1.0; source text/OCR may be partial. 11/8/24, 12:59 PM ESA - DEC 2023 -UE22CS151A about:blank 2/8 (5.0 Marks) (5.0 Marks) (5.0 Marks) 1.d. Explain the following: Algorithm Compiler Interpreter Semantic error Application software 2.a. Mention the properties of the list data structure. 2.b. Given s="python programming". What is the output of the following statements i) print (s[:]) ii) print (len(s)) iii) print (s[2:8]) iv) s[0]='T' print (s) v) k=s[0:6]+"not"+s[7:] print (k) 1 A sequence of unambiguous instructions for solving a problem 2 Compiler converts entire high-level language programs to machine language at once. 3 Interpreter converts high-level language to machine language line by line and execute instructions in place of CPU 4 a type of error that occurs when a program's meaning is incorrect, even though it runs successfully basically Logic error 5 Application software is used by user to perform a specific task. Mutable Indexing & slicing works ordered Iterable uses [ ] 1 python programming 2 18 3 thon p 4 they forgot ; ig. string immutable so TypeError 5 they forgot ; ig. pythonnotprogramming EV0001231
3. Archive page 3 from Dec__1aef5kbQ-b.pdf; unit match candidate score 0.5; source text/OCR may be partial. 11/8/24, 12:59 PM ESA - DEC 2023 -UE22CS151A about:blank 3/8 (5.0 Marks) (5.0 Marks) (5.0 Marks) 2.c. Given a dictionary of student marks, create another dictionary that has the total marks of each student. Given, students= { 'Raja':[90,89,85,90], 'Ram':[89,87,80,91], 'Mohan': [91,88,88,92] } Sample output: Total_marks= { 'Raja': 354, 'Ram': 347, 'Mohan':359 } 2.d. Explain the following file functions. read() read(5) readlines() readline() write() 3.a. Write the function cumulative_sum which takes a list of numbers and returns a list of numbers obtained by adding successive elements. If the input list has n elements, the resulting list also have n elements. The first element is obtained by adding 0 to the first element of the input list. l1 = [10, 15, 30, 50] l2 = cumulative_sum(l1) print(l2) # should print [10, 25, 55, 105] 1 reads the entire file and returns a string 2 reads 5 characters/bytes and returns a string 3 reads the entire file and returns a list with each element: a string of each line 4 reads a single line each time and return a string (each time called moves to next line) 5 write data to a file EV0001232

4. Archive page 1 from July__17cJZWr8dZ.pdf; unit match candidate score 0.8333; source text/OCR may be partial. (4.0 Marks) (7.0 Marks) (4.0 Marks) PES University, Bengaluru (Established under Karnataka Act 16 of 2013) END SEMESTER ASSESSMENT (ESA) - JULY - 2023 UE22CS151A - Python for Computational Problem Solving Total Marks : 100.0 1.a. Explain the stages in the computational problem-solving process. 1.b. Answer the following: i) _____ is a built-in function that is used to determine the type of a Python object. ii) print(len(["ChatGPT", "Bard", 10, 2, 7.3, 20,77.0])). iii) print("My", "last", "Exam ", sep="*") iv) print(5 + (2 ** 5) // 10) v) The _____ operator is used for integer division. vi) Which function returns a random integer in range [a, b], including both end points in random module. (a and b are the start and end values for the range). vii) 7+5i. (State if the decimal literal is valid or invalid in python.) 1.c. Given the start and end numbers, write a python program to display all even numbers between the range. Sample Output: Enter the start of range: 6 Enter the end of range: 12 6 8 10 12 7 type() 8 My*last*Exam // import random random.randint Invalid EV0001233

5. Archive page 2 from July__17cJZWr8dZ.pdf; unit match candidate score 0.5; source text/OCR may be partial. (5.0 Marks) (5.0 Marks) (4.0 Marks) 1.d. A student's marks for five distinct subjects are taken using user input. Calculate and display the mark percentage. 2.a. Write a python program to read a string of 4 characters in lowercase from the user. Print every character of a string along with its uppercase letter and its ASCII/Unicode. Sample Output: Enter the string of 4 characters in lowercase: abcd a A 65 b B 66 c C 67 d D 68 2.b. Write a python program that takes the file name from the user and reads the contents(lines) of the file in the reverse order (from bottom). File1.txt contents: The laughter filled the room, infectious and joyous. It echoed and echoed, spreading happiness all around. It danced on the airwaves, filling hearts with warmth and light. Sample output: Enter file name: File1.txt It danced on the airwaves, filling hearts with warmth and light. It echoed and echoed, spreading happiness all around. The laughter filled the room, infectious and joyous. EV0001234

6. Archive page 3 from July__17cJZWr8dZ.pdf; unit match candidate score 0.5; source text/OCR may be partial. (6.0 Marks) (5.0 Marks) (5.0 Marks) 2.c. Find the output of the following code snippets. i) t1 = (10,20,30,40,50) t2 = t1 + (60) t1 ii) t1=(100,200) t1*2 iii) d1=dict([(1,"Python"), (2,"Exam"),(3,"Over")]) print(d1,type(d1)) 2.d. Give any 5 built in functions of list in a tabular form with its description and code snippets 3.a. Write a function that accepts a string and calculate the number of upper-case letters and lower case letters in it. TypeError as (60) is not tuple its jut int. correct should be (60,) nothing gets printed but like it gives (100,200,100,200) pop,remove,clear,count,index EV0001235

7. Archive page 6 from July__17cJZWr8dZ.pdf; unit match candidate score 0.5; source text/OCR may be partial. (5.0 Marks) (6.0 Marks) (3.0 Marks) 5.b. List the keywords in python with respect to Exception handling. 5.c. Find the output for the following python code. class Car: make = "TATA" def __init__(self,name,mileage): self.name = name self.mil = mileage c1 = Car("Harrier",18) c2 = Car("Nexon",25) print(c1.name,c1.mil,c1.make) c2.name = "i20" c2.make = "Hyundai" print(c2.name,c2.mil,c2.make) 5.d. Create a super class named A and a subclass named B. B class inherits the properties of the A class. The super class has an attribute called list L, which is homogeneous and contains integers. List may contain duplicate elements. Write a program such that the subclass removes the duplicate elements from the list (if any). Sample input: L= [10,20,30,30,20,90,10,80] Expected output: L = [10,20,30,90,80] try, finally, else, except, raise Harrier 18 TATA i20 25 Hyundai EV0001236

8. Archive page 1 from May__1SdbvafL_8.pdf; unit match candidate score 0.6667; source text/OCR may be partial. (5.0 Marks) (6.0 Marks) (5.0 Marks) PES University, Bengaluru (Established under Karnataka Act 16 of 2013) END SEMESTER ASSESSMENT (ESA) - Jan - May 2024 UE22CS151A - Python for Computational Problem Solving Total Marks : 100.0 1.a. Print the output of each statement when executed separately. i) a = 40; print("a") ii) print(c == 0) iii) a = 4; b = a; print(id(a) == id(b)) iv) print(2 * 3 * 2) v) print(False and True) 1.b. 1) Provide the binary equivalent of a given decimal number - 53 2) Define Translator. Name any two types of translators. 1.c. Write a program to print the following pattern. A B C D B C D C D D 1 a 2 NameError c not defined 3 True 4 12 5 False 110101 Translator translates between two or more languages compilers, interpreters EV0001237

9. Archive page 2 from May__1SdbvafL_8.pdf; unit match candidate score 0.5833; source text/OCR may be partial. (4.0 Marks) (5.0 Marks) (6.0 Marks) (5.0 Marks) 1.d. Write a neat diagram to explain the phases involved in the process of computational problem solving. 2.a. What is a user defined function? Give an example code. 2.b. Write a program to read the length and breadth of a rectangle. Find the perimeter and the area of the rectangle. 2.c. Consider a text file (t.txt) with the following contents. Write a program to display all the words that starts with a capital letter. PCPS final exam date is announced. We thought ESA paper will be easy. It was good. user defined func is defined by user. syntax is def funcname(): EV0001238

10. Archive page 3 from May__1SdbvafL_8.pdf; unit match candidate score 0.75; source text/OCR may be partial. (4.0 Marks) (5.0 Marks) (6.0 Marks) (5.0 Marks) 2.d. Describe briefly the characteristics of a string data structure. 3.a. Write a python code to demo this - Returning multiple elements in the same return statement from a function. 3.b. What is recursion? Which data structure is used while storing the recursive calls? Fill the implementation of a recursive function to find the sum of first n natural numbers for a given client code. def find_sum(n): #Fill this function suite Client code is as below: number = 7 print("The sum of first", number,"numbers is", find_sum(number)) 3.c. Find the output when executed. def f1(x,y,*arg,**kwarg): print(x+y) print(arg) print(kwarg) f1(2,3,56,33,34,4,6,a=21,b=31,c=61) immutable, slicing, indexing, concatenation, iterable recursion - function calling itself over and over until it solves the problem using base case . ||| uses stack 5 [56,33,34,4,6] {'a': 21, 'b': 31, 'c': 61} EV0001239

11. Archive page 5 from May__1SdbvafL_8.pdf; unit match candidate score 0.5833; source text/OCR may be partial. (4.0 Marks) (5.0 Marks) (5.0 Marks) 4.d. Define Generator in python. Explain with an example code to create generator objects in python. 5.a. Define Inheritance. Mention its types 5.b. Fill the implementation of a new type – Chocolate to get the output as shown for the given client code. class Chocolate: #Fill this class #client code c1 = Chocolate("sweet", "imli") print("Number of chocolates:", Chocolate.count) c2 = Chocolate("sugary", "salty") c3 = Chocolate("imly", "sweety") print("Number of chocolates:", Chocolate.count) print(c2) del c1 print("Number of chocolates:", Chocolate.count) #expected output Number of chocolates: 1 Number of chocolates: 3 sugary salty Number of chocolates: 2 Generator is an iterable object inheritance - child take stuff of parent 'is-a' & 'has-a' relation multi-level, single-level, multiple, hierarchical, hybrid EV0001240

12. Archive page 1 from Dec__1enz7ur0jb.pdf; unit match candidate score 0.75; source text/OCR may be partial. 11/8/24, 12:59 PM ESA - DEC 2023 -UE22CS151A about:blank 1/8 (5.0 Marks) (5.0 Marks) (5.0 Marks) PES University, Bengaluru (Established under Karnataka Act 16 of 2013) END SEMESTER ASSESSMENT (ESA) - DEC 2023 UE22CS151A - Python for Computational Problem Solving Total Marks : 100.0 1.a. Classify & group the following devices into Input/ Output/ Processing/ Storage/ Communication devices. Central Processing Unit, Modem, USB Stick, Microphone, Blue-Ray device, Speakers, Graphics Processing Unit, Touch screen, Projectors, Blue-tooth. 1.b. Evaluate the following expressions: i) a='rama'>'ravana'; print(a) ii) print("1" * 2, 2 * "1") iii) a = 20; b = 5; print(a&b) iv) 5(2 + 3 * 4) v) import math; print(math.sqrt(-25)) 1.c. Write a Python program that prompts the user to enter positive integer values, and continues to prompt the user until a value of -1 is entered, adding up all the value entered and displaying the result. EV0001241

13. Archive page 2 from Dec__1enz7ur0jb.pdf; unit match candidate score 0.5833; source text/OCR may be partial. 11/8/24, 12:59 PM ESA - DEC 2023 -UE22CS151A about:blank 2/8 (5.0 Marks) (5.0 Marks) (5.0 Marks) 1.d. Explain the following: Algorithm Compiler Interpreter Semantic error Application software 2.a. Mention the properties of the list data structure. 2.b. Given s="python programming". What is the output of the following statements i) print (s[:]) ii) print (len(s)) iii) print (s[2:8]) iv) s[0]='T' print (s) v) k=s[0:6]+"not"+s[7:] print (k) EV0001242

14. Archive page 1 from July__1e0jrB6Yh8.pdf; unit match candidate score 0.8333; source text/OCR may be partial. (4.0 Marks) (7.0 Marks) (4.0 Marks) PES University, Bengaluru (Established under Karnataka Act 16 of 2013) END SEMESTER ASSESSMENT (ESA) - JULY - 2023 UE22CS151A - Python for Computational Problem Solving Total Marks : 100.0 1.a. Explain the stages in the computational problem-solving process. 1.b. Answer the following: i) _____ is a built-in function that is used to determine the type of a Python object. ii) print(len(["ChatGPT", "Bard", 10, 2, 7.3, 20,77.0])). iii) print("My", "last", "Exam ", sep="*") iv) print(5 + (2 ** 5) // 10) v) The _____ operator is used for integer division. vi) Which function returns a random integer in range [a, b], including both end points in random module. (a and b are the start and end values for the range). vii) 7+5i. (State if the decimal literal is valid or invalid in python.) 1.c. Given the start and end numbers, write a python program to display all even numbers between the range. Sample Output: Enter the start of range: 6 Enter the end of range: 12 6 8 10 12 EV0001243

15. Archive page 2 from July__1e0jrB6Yh8.pdf; unit match candidate score 0.5; source text/OCR may be partial. (5.0 Marks) (5.0 Marks) (4.0 Marks) 1.d. A student's marks for five distinct subjects are taken using user input. Calculate and display the mark percentage. 2.a. Write a python program to read a string of 4 characters in lowercase from the user. Print every character of a string along with its uppercase letter and its ASCII/Unicode. Sample Output: Enter the string of 4 characters in lowercase: abcd a A 65 b B 66 c C 67 d D 68 2.b. Write a python program that takes the file name from the user and reads the contents(lines) of the file in the reverse order (from bottom). File1.txt contents: The laughter filled the room, infectious and joyous. It echoed and echoed, spreading happiness all around. It danced on the airwaves, filling hearts with warmth and light. Sample output: Enter file name: File1.txt It danced on the airwaves, filling hearts with warmth and light. It echoed and echoed, spreading happiness all around. The laughter filled the room, infectious and joyous. EV0001244

16. Archive page 3 from July__1e0jrB6Yh8.pdf; unit match candidate score 0.5; source text/OCR may be partial. (6.0 Marks) (5.0 Marks) (5.0 Marks) 2.c. Find the output of the following code snippets. i) t1 = (10,20,30,40,50) t2 = t1 + (60) t1 ii) t1=(100,200) t1*2 iii) d1=dict([(1,"Python"), (2,"Exam"),(3,"Over")]) print(d1,type(d1)) 2.d. Give any 5 built in functions of list in a tabular form with its description and code snippets 3.a. Write a function that accepts a string and calculate the number of upper-case letters and lower case letters in it. EV0001245

17. Archive page 6 from July__1e0jrB6Yh8.pdf; unit match candidate score 0.5; source text/OCR may be partial. (5.0 Marks) (6.0 Marks) (3.0 Marks) 5.b. List the keywords in python with respect to Exception handling. 5.c. Find the output for the following python code. class Car: make = "TATA" def __init__(self,name,mileage): self.name = name self.mil = mileage c1 = Car("Harrier",18) c2 = Car("Nexon",25) print(c1.name,c1.mil,c1.make) c2.name = "i20" c2.make = "Hyundai" print(c2.name,c2.mil,c2.make) 5.d. Create a super class named A and a subclass named B. B class inherits the properties of the A class. The super class has an attribute called list L, which is homogeneous and contains integers. List may contain duplicate elements. Write a program such that the subclass removes the duplicate elements from the list (if any). Sample input: L= [10,20,30,30,20,90,10,80] Expected output: L = [10,20,30,90,80] EV0001246

18. Archive page 1 from May__1rBeUIlse2.pdf; unit match candidate score 0.5833; source text/OCR may be partial. (5.0 Marks) (6.0 Marks) (5.0 Marks) PES University, Bengaluru (Established under Karnataka Act 16 of 2013) END SEMESTER ASSESSMENT (ESA) - Jan - May 2024 UE22CS151A - Python for Computational Problem Solving Total Marks : 100.0 1.a. Print the output of each statement when executed separately. i) a = 40; print("a") ii) print(c == 0) iii) a = 4; b = a; print(id(a) == id(b)) iv) print(2 * 3 * 2) v) print(False and True) 1.b. 1) Provide the binary equivalent of a given decimal number - 53 2) Define Translator. Name any two types of translators. 1.c. Write a program to print the following pattern. A B C D B C D C D D EV0001247

19. Archive page 2 from May__1rBeUIlse2.pdf; unit match candidate score 0.5; source text/OCR may be partial. (4.0 Marks) (5.0 Marks) (6.0 Marks) (5.0 Marks) 1.d. Write a neat diagram to explain the phases involved in the process of computational problem solving. 2.a. What is a user defined function? Give an example code. 2.b. Write a program to read the length and breadth of a rectangle. Find the perimeter and the area of the rectangle. 2.c. Consider a text file (t.txt) with the following contents. Write a program to display all the words that starts with a capital letter. PCPS final exam date is announced. We thought ESA paper will be easy. It was good. EV0001248

20. Archive page 3 from May__1rBeUIlse2.pdf; unit match candidate score 0.6667; source text/OCR may be partial. (4.0 Marks) (5.0 Marks) (6.0 Marks) (5.0 Marks) 2.d. Describe briefly the characteristics of a string data structure. 3.a. Write a python code to demo this - Returning multiple elements in the same return statement from a function. 3.b. What is recursion? Which data structure is used while storing the recursive calls? Fill the implementation of a recursive function to find the sum of first n natural numbers for a given client code. def find_sum(n): #Fill this function suite Client code is as below: number = 7 print("The sum of first", number,"numbers is", find_sum(number)) 3.c. Find the output when executed. def f1(x,y,*arg,**kwarg): print(x+y) print(arg) print(kwarg) f1(2,3,56,33,34,4,6,a=21,b=31,c=61) EV0001249

21. Archive page 5 from May__1rBeUIlse2.pdf; unit match candidate score 0.5833; source text/OCR may be partial. (4.0 Marks) (5.0 Marks) (5.0 Marks) 4.d. Define Generator in python. Explain with an example code to create generator objects in python. 5.a. Define Inheritance. Mention its types 5.b. Fill the implementation of a new type – Chocolate to get the output as shown for the given client code. class Chocolate: #Fill this class #client code c1 = Chocolate("sweet", "imli") print("Number of chocolates:", Chocolate.count) c2 = Chocolate("sugary", "salty") c3 = Chocolate("imly", "sweety") print("Number of chocolates:", Chocolate.count) print(c2) del c1 print("Number of chocolates:", Chocolate.count) #expected output Number of chocolates: 1 Number of chocolates: 3 sugary salty Number of chocolates: 2 EV0001250

22. Archive page 1 from Jan - (Set-1)__1S8XvO2bSr.pdf; unit match candidate score 1.0; source text/OCR may be partial. 11/11/24, 11:33 AM ESA -JAN 2024 - UE23CS151A (set- 1) about:blank 1/7 (5.0 Marks) (5.0 Marks) (6.0 Marks) PES University, Bengaluru (Established under Karnataka Act 16 of 2013) END SEMESTER ASSESSMENT (ESA) - JAN 2023 UE23CS151A - Python for Computational Problem Solving Total Marks : 100.0 1.a. Print the output of the following expressions when executed each statement separately using the python 3.x i) print(print(0),end = "") ii) a = 3; b = a; print(a is b) iii) a = (3,4,5); print(type(a)) iv) a = 5; if a =10: print("It is ten") v) print(range(3,10,2)) 1.b. Say True or False. i) The function print can take any type of arguments. ii) The built in function, which returns the unique identity of the given object in python is ID() iii) Compiler executes one line of code at a time. iv) The function input() returns a string always. v) The syntax of a language is nothing but the set of rules to be followed in a language. 1.c. Take a whole number from the user. Display whether the entered number is perfect number or not. Expected output #1: Enter the number: 28 28 is a perfect number Expected output #2: Enter the number: 25 25 is not a perfect number Hint: A perfect number is a positive integer that is equal to the sum of its positive divisors, excluding the number itself. 1 0 None 2 True 3 tuple 4 syntax error 5 range(3, 10, 2) True Flase it's id() False it's interpreter True True EV0001251

23. Archive page 2 from Jan - (Set-1)__1S8XvO2bSr.pdf; unit match candidate score 0.75; source text/OCR may be partial. 11/11/24, 11:33 AM ESA -JAN 2024 - UE23CS151A (set- 1) about:blank 2/7 (4.0 Marks) (5.0 Marks) (5.0 Marks) 1.d. Separate the keyword/s and variable/s from a given collection of identifiers. i) and ii) print iii) PriNt iv) input 1.e. With a neat diagram, explain the phases involved in the process of computational problem solving. 2.a. Mention any five characteristics of dictionary type in python. key: value, uses {key: value}, mutable, iterable, key should be immutable, sorted using hashing, doesnt support concatenation keyword and variable PriNt, print, input EV0001252

24. Archive page 3 from Jan - (Set-1)__1S8XvO2bSr.pdf; unit match candidate score 0.6667; source text/OCR may be partial. 11/11/24, 11:33 AM ESA -JAN 2024 - UE23CS151A (set- 1) about:blank 3/7 (5.0 Marks) (6.0 Marks) (4.0 Marks) 2.b. Perform the given operations separately on a sample list. li = [23, "yesterday", 44.6, 78.5, 900, "today", "tomorrow"] i) Create a new list with only strings from the given list. ii) Add a tuple t = (3,4,5) to the list li at the end to increment the length of the list by 1. Print the length of the list. 2.c. Develop a program to read the contents from one text file. Choose the words which are starting with a letter P and write only these words to another file line by line. Source file content is as below: Python is a Professional language which is the part of syllabus in PES university Output file content is as below: Python Professional PES 2.d. What happens when the below code is run? Modify the code to get the expected output. x=10 def f1(): x=x+1 print("Inside",x) f1() print("Outside",x) Expected output: Inside 11 Outside 11 You get error as you can't directly change variables of outer scope. UnboundLocalError: cannot access local variable 'x' where it is not associated with a value EV0001253

25. Archive page 4 from Jan - (Set-1)__1S8XvO2bSr.pdf; unit match candidate score 0.75; source text/OCR may be partial. 11/11/24, 11:33 AM ESA -JAN 2024 - UE23CS151A (set- 1) about:blank 4/7 (5.0 Marks) (5.0 Marks) (5.0 Marks) 2.e. Find the output of below mentioned code when executed using python 3.x. def my_function(p, q = 2, *arg, **kw): print(p); print(q); print(arg[1]) print(kw['b']); return arg print(my_function(2,3,10,14, a = 4,b = 5,c = 6)) 3.a. Recursion is implemented using _____ to store the activation record of function calls in Last In First Out order. Fill the implementation of a recursive function to find the length of the string for a given client code. def length(s): #Fill this function suite #client code print(length("pesu-python")) 3.b. What is the type(m) in the below code? What gets printed when this code is run? def my_code1(): yield 6 yield 12 yield 18 m = my_code1() print(next(m), end = " ") print(next(m), end = " ") print(next(m)) print(next(m), end = " ") 2 3 14 5 (10,14) Stack class generator 6 12 18 error stopiteration EV0001254

26. Archive page 6 from Jan - (Set-1)__1S8XvO2bSr.pdf; unit match candidate score 0.4167; source text/OCR may be partial. 11/11/24, 11:33 AM ESA -JAN 2024 - UE23CS151A (set- 1) about:blank 6/7 (5.0 Marks) (5.0 Marks) 4.a. Given list1 = [10,34,32,21,370,101]. Use list comprehension to do the following. i) Create a list of tuples where each tuple having an element from the given list and its square root. ii) Create a list of only even elements 4.b. #---------- class My_Exception(Exception): pass try: a = int(input()) b = int(input()) c = a/b if b == 5: raise My_Exception("Divisor cannot be 5") except ZeroDivisionError: print("Division by zero") except ValueError as v: print("Input must be an integer") except Exception: print("It is a problem if divisor is 5") except: print("error") else: print(c) i) If user enters 10 and 5 in two different lines, what gets printed? ii) If user enters 10 and 2 in two different lines, what gets printed? iii) If user enters 10 and five in two different lines, what gets printed? iv) If user enters 10 and 0 in two different lines, what gets printed? v) If user enters ten and press enters, what happens? It is a problem if divisor is 5 5 Division by zero Input must be an integer Input must be an integer EV0001255

27. Archive page 1 from Jan - (Set-2)__1HZjDaXNmS.pdf; unit match candidate score 1.0833; source text/OCR may be partial. 11/11/24, 11:32 AM ESA -JAN 2024 - UE23CS151A (set- 2) about:blank 1/8 (5.0 Marks) (5.0 Marks) PES University, Bengaluru (Established under Karnataka Act 16 of 2013) END SEMESTER ASSESSMENT (ESA) - JAN 2023 UE23CS151A - Python for Computational Problem Solving Total Marks : 100.0 1.a. Print the output of the following expressions when executed each statement separately using python 3.x i) print(print("choco",end = "")) ii) a = 3; b = 10; print(a and b) iii) a = {3,4,5}; print(type(a)) iv) a = 6; if a != 5: print("It is five") v) print(list(range(3,6))) 1.b. Say True or False. i) The word sep is used as keyword argument in input function. ii) The variable can have only digits to represent a value. iii) Interpreter executes one line of code at a time. iv) The membership operator returns a boolean value. v) a = 2 . This is an expression in python. chocoNone 10 set It is five [3, 4, 5] False False True True False Expression must give a result. like 2+3,etc ig. I am not sure for this question . because they tell even just 2 is an expression EV0001256

28. Archive page 2 from Jan - (Set-2)__1HZjDaXNmS.pdf; unit match candidate score 0.8333; source text/OCR may be partial. 11/11/24, 11:32 AM ESA -JAN 2024 - UE23CS151A (set- 2) about:blank 2/8 (6.0 Marks) (4.0 Marks) (5.0 Marks) 1.c. Take a whole number from the user. Display whether the entered number is an Armstrong number or not. Expected output #1: Enter the number: 153 153 is an armstrong number Expected output #2: Enter the number: 154 154 is not an armstrong number Hint: Armstrong number is a positive number, if the sum of its own digits raised to the power of number of digits in a number gives the number itself. 1.d. Separate the keywords and variables from a given collection of identifiers. i) for ii) sum iii) Input iv) def 1.e. List the two essences and three phases involved in the process of computational problem solving. keywords:for def variables:Input sum Representation, Algorithm Analysis, Design, Implementation, Testing EV0001257

29. Archive page 3 from Jan - (Set-2)__1HZjDaXNmS.pdf; unit match candidate score 0.5833; source text/OCR may be partial. 11/11/24, 11:32 AM ESA -JAN 2024 - UE23CS151A (set- 2) about:blank 3/8 (5.0 Marks) (5.0 Marks) (6.0 Marks) 2.a. Mention any five characteristics of set type in python 2.b. Perform the given operations separately on a sample dictionary d. d = {1: 111, 5: 555, 4:444, 7:777} i) Create a new list with only keys from the given dictionary. ii) Add a tuple t = (3,4,5) as key to the dictionary d with a value 345. iii)Pop any one item from the dictionary without specifying the key and print the popped item. 2.c. Develop a program to read only first line from a text file. Choose the words which are starting with a capital letter and write only these words to another file line by line. Source file content is as below: Python is a Professional course in PES university It is one of the Popular Language. Output file: Python Professional PES mutable, unordered, no duplicates iterable, uses {a,b,..} pop - needs key popitem - pops random and gives output as tuple (key, value) EV0001258

30. Archive page 4 from Jan - (Set-2)__1HZjDaXNmS.pdf; unit match candidate score 0.5; source text/OCR may be partial. 11/11/24, 11:32 AM ESA -JAN 2024 - UE23CS151A (set- 2) about:blank 4/8 (4.0 Marks) (5.0 Marks) (5.0 Marks) 2.d. What happens when the below code is run? Modify the code to get the expected output. f1(2) f1(33) f1(12,[3,4]) f1(25) def f1(a,b=[]): b.append(a) print(b) Expected output: [2] [2, 33] [3, 4, 12] [2, 33, 25] 2.e. Find the output when the below code is executed using python 3.x. def my_function(p, q, *arg, **kw): print(p); print(q); print(arg[0]); print(kw['a']); return list(kw.values()) print(my_function(2,3,10,14, a = 400,b = 500,c = 600)) 3.a. What is recursion? Fill the implementation of a recursive function to find the sum of first n natural numbers for a given client code. def find_sum(n): #Fill this function suite #Client code number = 10 print("The sum of first", number,"numbers is", find_sum(number)) error as f1 is not defined when the function call is done. first should def f1 then should be func calling 2 3 10 400 [400,500,600] Repeat EV0001259
31. Archive page 8 from Jan - (Set-2)__1HZjDaXNmS.pdf; unit match candidate score 0.6667; source text/OCR may be partial. 11/11/24, 11:32 AM ESA -JAN 2024 - UE23CS151A (set- 2) about:blank 8/8 (4.0 Marks) (5.0 Marks) 4.d. List any four types of is – a relationship supported by Python. 4.e. Fill the implementation of a new type – Chocolate to get the output as expected for the given client code. class Chocolate: #Fill this class #client code c1 = Chocolate("sweet", "imli") print("Number of chocolates:", Chocolate.count) c2 = Chocolate("sugary", "salty") c3 = Chocolate("imly", "sweety") print("Number of chocolates:", Chocolate.count) print(c2) del c1 print("Number of chocolates:", Chocolate.count) #expected output Number of chocolates: 1 Number of chocolates: 3 sugary salty Number of chocolates: 2 single-level, multi-level, heirarchial, hybrid, multiple repeat EV0001260
32. Archive page 1 from Jan - (Set-1)__1kzIfeDFc9.pdf; unit match candidate score 1.0; source text/OCR may be partial. 11/11/24, 11:33 AM ESA -JAN 2024 - UE23CS151A (set- 1) about:blank 1/7 (5.0 Marks) (5.0 Marks) (6.0 Marks) PES University, Bengaluru (Established under Karnataka Act 16 of 2013) END SEMESTER ASSESSMENT (ESA) - JAN 2023 UE23CS151A - Python for Computational Problem Solving Total Marks : 100.0 1.a. Print the output of the following expressions when executed each statement separately using the python 3.x i) print(print(0),end = "") ii) a = 3; b = a; print(a is b) iii) a = (3,4,5); print(type(a)) iv) a = 5; if a =10: print("It is ten") v) print(range(3,10,2)) 1.b. Say True or False. i) The function print can take any type of arguments. ii) The built in function, which returns the unique identity of the given object in python is ID() iii) Compiler executes one line of code at a time. iv) The function input() returns a string always. v) The syntax of a language is nothing but the set of rules to be followed in a language. 1.c. Take a whole number from the user. Display whether the entered number is perfect number or not. Expected output #1: Enter the number: 28 28 is a perfect number Expected output #2: Enter the number: 25 25 is not a perfect number Hint: A perfect number is a positive integer that is equal to the sum of its positive divisors, excluding the number itself. EV0001261
33. Archive page 2 from Jan - (Set-1)__1kzIfeDFc9.pdf; unit match candidate score 0.75; source text/OCR may be partial. 11/11/24, 11:33 AM ESA -JAN 2024 - UE23CS151A (set- 1) about:blank 2/7 (4.0 Marks) (5.0 Marks) (5.0 Marks) 1.d. Separate the keyword/s and variable/s from a given collection of identifiers. i) and ii) print iii) PriNt iv) input 1.e. With a neat diagram, explain the phases involved in the process of computational problem solving. 2.a. Mention any five characteristics of dictionary type in python. EV0001262

34. Archive page 3 from Jan - (Set-1)__1kzIfeDFc9.pdf; unit match candidate score 0.5833; source text/OCR may be partial. 11/11/24, 11:33 AM ESA -JAN 2024 - UE23CS151A (set- 1) about:blank 3/7 (5.0 Marks) (6.0 Marks) (4.0 Marks) 2.b. Perform the given operations separately on a sample list. li = [23, "yesterday", 44.6, 78.5, 900, "today", "tomorrow"] i) Create a new list with only strings from the given list. ii) Add a tuple t = (3,4,5) to the list li at the end to increment the length of the list by 1. Print the length of the list. 2.c. Develop a program to read the contents from one text file. Choose the words which are starting with a letter P and write only these words to another file line by line. Source file content is as below: Python is a Professional language which is the part of syllabus in PES university Output file content is as below: Python Professional PES 2.d. What happens when the below code is run? Modify the code to get the expected output. x=10 def f1(): x=x+1 print("Inside",x) f1() print("Outside",x) Expected output: Inside 11 Outside 11 EV0001263

35. Archive page 4 from Jan - (Set-1)__1kzIfeDFc9.pdf; unit match candidate score 0.6667; source text/OCR may be partial. 11/11/24, 11:33 AM ESA -JAN 2024 - UE23CS151A (set- 1) about:blank 4/7 (5.0 Marks) (5.0 Marks) (5.0 Marks) 2.e. Find the output of below mentioned code when executed using python 3.x. def my_function(p, q = 2, *arg, **kw): print(p); print(q); print(arg[1]) print(kw['b']); return arg print(my_function(2,3,10,14, a = 4,b = 5,c = 6)) 3.a. Recursion is implemented using _____ to store the activation record of function calls in Last In First Out order. Fill the implementation of a recursive function to find the length of the string for a given client code. def length(s): #Fill this function suite #client code print(length("pesu-python")) 3.b. What is the type(m) in the below code? What gets printed when this code is run? def my_code1(): yield 6 yield 12 yield 18 m = my_code1() print(next(m), end = " ") print(next(m), end = " ") print(next(m)) print(next(m), end = " ") EV0001264

36. Archive page 6 from Jan - (Set-1)__1kzIfeDFc9.pdf; unit match candidate score 0.4167; source text/OCR may be partial. 11/11/24, 11:33 AM ESA -JAN 2024 - UE23CS151A (set- 1) about:blank 6/7 (5.0 Marks) (5.0 Marks) 4.a. Given list1 = [10,34,32,21,370,101]. Use list comprehension to do the following. i) Create a list of tuples where each tuple having an element from the given list and its square root. ii) Create a list of only even elements 4.b. #---------- class My_Exception(Exception): pass try: a = int(input()) b = int(input()) c = a/b if b == 5: raise My_Exception("Divisor cannot be 5") except ZeroDivisionError: print("Division by zero") except ValueError as v: print("Input must be an integer") except Exception: print("It is a problem if divisor is 5") except: print("error") else: print(c) i) If user enters 10 and 5 in two different lines, what gets printed? ii) If user enters 10 and 2 in two different lines, what gets printed? iii) If user enters 10 and five in two different lines, what gets printed? iv) If user enters 10 and 0 in two different lines, what gets printed? v) If user enters ten and press enters, what happens? EV0001265

37. Archive page 1 from Jan - (Set-2)__1YxZdNR8Ws.pdf; unit match candidate score 1.0833; source text/OCR may be partial. 11/11/24, 11:32 AM ESA -JAN 2024 - UE23CS151A (set- 2) about:blank 1/8 (5.0 Marks) (5.0 Marks) PES University, Bengaluru (Established under Karnataka Act 16 of 2013) END SEMESTER ASSESSMENT (ESA) - JAN 2023 UE23CS151A - Python for Computational Problem Solving Total Marks : 100.0 1.a. Print the output of the following expressions when executed each statement separately using python 3.x i) print(print("choco",end = "")) ii) a = 3; b = 10; print(a and b) iii) a = {3,4,5}; print(type(a)) iv) a = 6; if a != 5: print("It is five") v) print(list(range(3,6))) 1.b. Say True or False. i) The word sep is used as keyword argument in input function. ii) The variable can have only digits to represent a value. iii) Interpreter executes one line of code at a time. iv) The membership operator returns a boolean value. v) a = 2 . This is an expression in python. EV0001266

38. Archive page 2 from Jan - (Set-2)__1YxZdNR8Ws.pdf; unit match candidate score 0.5; source text/OCR may be partial. 11/11/24, 11:32 AM ESA -JAN 2024 - UE23CS151A (set- 2) about:blank 2/8 (6.0 Marks) (4.0 Marks) (5.0 Marks) 1.c. Take a whole number from the user. Display whether the entered number is an Armstrong number or not. Expected output #1: Enter the number: 153 153 is an armstrong number Expected output #2: Enter the number: 154 154 is not an armstrong number Hint: Armstrong number is a positive number, if the sum of its own digits raised to the power of number of digits in a number gives the number itself. 1.d. Separate the keywords and variables from a given collection of identifiers. i) for ii) sum iii) Input iv) def 1.e. List the two essences and three phases involved in the process of computational problem solving. EV0001267

39. Archive page 3 from Jan - (Set-2)__1YxZdNR8Ws.pdf; unit match candidate score 0.5833; source text/OCR may be partial. 11/11/24, 11:32 AM ESA -JAN 2024 - UE23CS151A (set- 2) about:blank 3/8 (5.0 Marks) (5.0 Marks) (6.0 Marks) 2.a. Mention any five characteristics of set type in python 2.b. Perform the given operations separately on a sample dictionary d. d = {1: 111, 5: 555, 4:444, 7:777} i) Create a new list with only keys from the given dictionary. ii) Add a tuple t = (3,4,5) as key to the dictionary d with a value 345. iii)Pop any one item from the dictionary without specifying the key and print the popped item. 2.c. Develop a program to read only first line from a text file. Choose the words which are starting with a capital letter and write only these words to another file line by line. Source file content is as below: Python is a Professional course in PES university It is one of the Popular Language. Output file: Python Professional PES EV0001268

40. Archive page 4 from Jan - (Set-2)__1YxZdNR8Ws.pdf; unit match candidate score 0.5; source text/OCR may be partial. 11/11/24, 11:32 AM ESA -JAN 2024 - UE23CS151A (set- 2) about:blank 4/8 (4.0 Marks) (5.0 Marks) (5.0 Marks) 2.d. What happens when the below code is run? Modify the code to get the expected output. f1(2) f1(33) f1(12,[3,4]) f1(25) def f1(a,b=[]): b.append(a) print(b) Expected output: [2] [2, 33] [3, 4, 12] [2, 33, 25] 2.e. Find the output when the below code is executed using python 3.x. def my_function(p, q, *arg, **kw): print(p); print(q); print(arg[0]); print(kw['a']); return list(kw.values()) print(my_function(2,3,10,14, a = 400,b = 500,c = 600)) 3.a. What is recursion? Fill the implementation of a recursive function to find the sum of first n natural numbers for a given client code. def find_sum(n): #Fill this function suite #Client code number = 10 print("The sum of first", number,"numbers is", find_sum(number)) EV0001269

41. Archive page 8 from Jan - (Set-2)__1YxZdNR8Ws.pdf; unit match candidate score 0.6667; source text/OCR may be partial. 11/11/24, 11:32 AM ESA -JAN 2024 - UE23CS151A (set- 2) about:blank 8/8 (4.0 Marks) (5.0 Marks) 4.d. List any four types of is – a relationship supported by Python. 4.e. Fill the implementation of a new type – Chocolate to get the output as expected for the given client code. class Chocolate: #Fill this class #client code c1 = Chocolate("sweet", "imli") print("Number of chocolates:", Chocolate.count) c2 = Chocolate("sugary", "salty") c3 = Chocolate("imly", "sweety") print("Number of chocolates:", Chocolate.count) print(c2) del c1 print("Number of chocolates:", Chocolate.count) #expected output Number of chocolates: 1 Number of chocolates: 3 sugary salty Number of chocolates: 2 EV0001270

42. Archive page 1 from Jan - (Set-3)__1ghwvCbUda.pdf; unit match candidate score 0.75; source text/OCR may be partial. 11/11/24, 11:33 AM ESA -JAN 2024 - UE23CS151A (set- 3) about:blank 1/7 (5.0 Marks) (6.0 Marks) PES University, Bengaluru (Established under Karnataka Act 16 of 2013) END SEMESTER ASSESSMENT (ESA) - JAN 2023 UE23CS151A - Python for Computational Problem Solving Total Marks : 100.0 1.a. i) How do compilers and interpreters differ in their approach to translating code? (2 marks) ii) List any 3 input devices. (3 marks) 1.b. Answer the following. i) _____ is a built-in function that is used to determine the type of a python object. ii) print("My", "last", "Exam ", sep="*") iii) The _____ operator is used for integer division. iv) print(15 & 3) v) 7+5i. (State if the literal is valid or invalid in python.) vi) print(True == 1) EV0001271

43. Archive page 2 from Jan - (Set-3)__1ghwvCbUda.pdf; unit match candidate score 0.9167; source text/OCR may be partial. 11/11/24, 11:33 AM ESA -JAN 2024 - UE23CS151A (set- 3) about:blank 2/7 (5.0 Marks) (5.0 Marks) (4.0 Marks) 1.c. Given the below python code, x = 10 while x > 5: x -= 2 print(x) i) Find the output of the code – 1 mark ii) If you change the initial value of "x" to 6, what will be the output? -- 2 marks iii) What happens if “x -= 2” line is removed from the code. -- 2 marks 1.d. i) Find the output - 2 marks name = "PESU" age = 50 is_active = True if name == "PESIT" or (name == "PESU" and age <= 30 and is_active): print("Welcome") else: print("Access denied") ii) Brief the concept of short-circuit evaluation wrt Python's Logical operators. - 3 marks 1.e. Write a program that takes the number of rows 'n' as input from the user and prints a diamond pattern using asterisks (*) as shown in the sample output. Note that input ‘n’ will be an odd integer. The pattern should have a diamond shape with 'n' rows in the middle. If n is 5, the expected pattern is as below. * *** ***** *** * EV0001272

44. Archive page 3 from Jan - (Set-3)__1ghwvCbUda.pdf; unit match candidate score 0.5; source text/OCR may be partial. 11/11/24, 11:33 AM ESA -JAN 2024 - UE23CS151A (set- 3) about:blank 3/7 (4.0 Marks) (5.0 Marks) (6.0 Marks) 2.a. List any four characteristics of a set. 2.b. Given the multiline string student_scores = """pes001 98 78 67 pes002 99 44 79 pes006 44 55 33 pes007 55 88 40 pes009 34 78 65""" Each row has details of a student (srn, phy_marks, chem_marks, math_marks) separated by a space n between. Calculate the total marks of a particular student. Display the srn as the key and the total marks as the value. Expected Output: {'pes001': 243, 'pes002': 222, 'pes006': 132, 'pes007': 183, 'pes009': 177} 2.c. Write a python program that takes a sentence as input and converts each alphabet in a given sentence to the next letter in the alphabet, while preserving the case of the letters. For example, a is converted to b, b to c, so on and z to a. Expected Output: Enter a sentence: hello how are you Converted sentence: ifmmp ipx bsf zpv EV0001273

45. Archive page 4 from Jan - (Set-3)__1ghwvCbUda.pdf; unit match candidate score 0.5833; source text/OCR may be partial. 11/11/24, 11:33 AM ESA -JAN 2024 - UE23CS151A (set- 3) about:blank 4/7 (5.0 Marks) (5.0 Marks) (6.0 Marks) 2.d. Write a python program to open a file, iterate through its contents line by line, print each line along with its corresponding line number, and then close the file. 2.e. Find the output def example(a, b=10, *args, **kwargs): print(a,b) print(args) print(kwargs) print(type(args),type(kwargs)) example(1, 2, 3, "Python", name="PESU", city="Blr") 3.a. Define Recursion. Write a recursive function to find the gcd of two whole numbers. These two whole numbers are sent from the client code to this function. EV0001274

46. Archive page 5 from Jan - (Set-3)__1ghwvCbUda.pdf; unit match candidate score 0.5; source text/OCR may be partial. 11/11/24, 11:33 AM ESA -JAN 2024 - UE23CS151A (set- 3) about:blank 5/7 (5.0 Marks) (5.0 Marks) (4.0 Marks) 3.b. Given a list1 = ['23','909','44','23','66','720', '9'] Perform sorting of these elements in a list using ascending order based on the last digit of every element. Expected output: ['720', '23', '23', '44', '66', '909', '9'] 3.c. i) Mention any two characteristics of closures. (2 marks) ii) Create a generator function that yields cubes of numbers from 1 to n. Accept n from the user. Test this generator function. (3 marks) 3.d. i)To install a third-party module in Python, the preferred command is _____. ii) _____ is the built in global variable which contains the document string of the function specified or the module itself. iii) _____ is the module which allows writing test cases as a part of document sting and runs those test cases using the function in that module iv) testadd is a valid method name which pytest can recognize. State True or False EV0001275

47. Archive page 7 from Jan - (Set-3)__1ghwvCbUda.pdf; unit match candidate score 0.5; source text/OCR may be partial. 11/11/24, 11:33 AM ESA -JAN 2024 - UE23CS151A (set- 3) about:blank 7/7 (6.0 Marks) (2.0 Marks) (5.0 Marks) 4.c. i) Explain constructors with an example code. (2 marks) ii) Create a super class named X and a subclass named Y. Y class inherits the properties of the X class. Create a constructor to initialize two lists in X. Write a method in the subclass that concatenates both the lists and displays the concatenated list. (4 marks) Sample input: list1= [10,20,30] list2=[40,50,60] Expected output: [10, 20, 30, 40, 50, 60] 4.d. Find the output list1 = [10,20,30] list2 = ['a', 'b', 'c', 'd'] list3 = [100,200] result = list(zip(list1, list2, list3)) print(result) 4.e. What is an exception? Mention it's categories. List any four built-in Exceptions. EV0001276

48. Archive page 1 from Jan - (Set-4)__1cbShhBdIG.pdf; unit match candidate score 0.75; source text/OCR may be partial. 11/11/24, 11:33 AM ESA -JAN 2024 - UE23CS151A (set- 4) about:blank 1/7 (5.0 Marks) (6.0 Marks) PES University, Bengaluru (Established under Karnataka Act 16 of 2013) END SEMESTER ASSESSMENT (ESA) - JAN 2023 UE23CS151A - Python for Computational Problem Solving Total Marks : 100.0 1.a. i) How do the functions of main memory and secondary memory differ in a computer system ? (2 marks) ii) List any 3 output devices. (3 marks) 1.b. Answer the following: i) What is the output of the expression type(3.5+2) ii) print("Pesu", "is", "the", "Best ", sep="#") iii) The _____ operator is used for shifting bits right. iv) print(15 | 0) v) 4+6j. (State if the decimal literal is valid or invalid in python.) vi) print(False == 0) EV0001277

49. Archive page 2 from Jan - (Set-4)__1cbShhBdIG.pdf; unit match candidate score 0.5; source text/OCR may be partial. 11/11/24, 11:33 AM ESA -JAN 2024 - UE23CS151A (set- 4) about:blank 2/7 (5.0 Marks) (5.0 Marks) (4.0 Marks) 1.c. Given the below python code i = 1 while i <= 5: print(i * i) i += 2 i) Find the output of the code – 2 marks ii) If you change the initial value of "i" to 4 in the beginning, what will be the output? -- 2 marks iii)What happens if “i += 2” line is removed from the code. -- 1 mark 1.d. i) How does Python handle nested if statements? Provide an example. (3 marks) ii) Find the output of the below code. (2 marks) x = 5 y = 10 z = 15 if x < y and y < z: print("Good Luck") else: print("Best Wishes") if x == y and y != z: print("All the best") else: print("Best of Luck") 1.e. Write a program that takes the number of rows ‘n’ as input and prints the following pattern. For n=4, Output is as below. 1 10 101 1010 EV0001278

50. Archive page 3 from Jan - (Set-4)__1cbShhBdIG.pdf; unit match candidate score 0.5; source text/OCR may be partial. 11/11/24, 11:33 AM ESA -JAN 2024 - UE23CS151A (set- 4) about:blank 3/7 (5.0 Marks) (6.0 Marks) (4.0 Marks) 2.a. Given the multiline string student_scores= """pes001 98 78 67 pes002 99 44 79 pes001 44 55 33 pes003 55 88 40 pes002 33 78 51 pes005 34 28 65 pes006 88 61 53""" Each row has details of a student (srn, phy_marks, chem_marks, math_marks) separated by a space in between. Remove the duplicate srn’s in the string, display the unique srn and the count of it. Expected Output: {'pes007', 'pes001', 'pes009', 'pes006', 'pes002'} The count is: 5 2.b. Write a python function that accepts two strings and tests if two strings are anagrams of each other. Note: An anagram is a word or phrase formed by rearranging the letters of another word Sample output: 1 Enter the first word: ink Enter the second word: kin True Sample output: 2 Enter the first word: pac Enter the second word: rac False 2.c. List any four characteristics of a list in python. EV0001279

51. Archive page 4 from Jan - (Set-4)__1cbShhBdIG.pdf; unit match candidate score 0.6667; source text/OCR may be partial. 11/11/24, 11:33 AM ESA -JAN 2024 - UE23CS151A (set- 4) about:blank 4/7 (5.0 Marks) (5.0 Marks) (6.0 Marks) 2.d. Write a python program that takes the file name from the user and reads all the contents of the file. Display these contents on the terminal from the last line of the file to the first line. 2.e. Find the output def test(a, b=10, *args, **kwargs): print(a + b) print(type(b),type(args)) print(args) print(kwargs) test("Good","Day", 3, "Python",5+4j, city="Blr", state="KA") 3.a. Mention the two criteria of recursion. Write a recursive function to find the the length of the string and test this function. EV0001280

52. Archive page 5 from Jan - (Set-4)__1cbShhBdIG.pdf; unit match candidate score 0.5; source text/OCR may be partial. 11/11/24, 11:33 AM ESA -JAN 2024 - UE23CS151A (set- 4) about:blank 5/7 (5.0 Marks) (5.0 Marks) (4.0 Marks) 3.b. The list has a mixture of integers and strings as shown here. list1 = [23,909,'54','19','66','720', '93'] Create a new sorted list in ascending order based on the second digit of every element in the list. Expected output: [909, '720', 23, '93', '54', '66', '19'] 3.c. i) Define Generators. (2 marks) ii) Write a program to generate odd numbers using closure. (3 marks) 3.d. i)The keyword used to include an entire module into a python code is _____. ii) _____ is the built in global variable which contains the name of the module. iii)testone.py is a valid pytest filename. State True or False. iv)_____ means a successful test case when you run the file which is recognized by the pytest module EV0001281

53. Archive page 6 from Jan - (Set-4)__1cbShhBdIG.pdf; unit match candidate score 0.9167; source text/OCR may be partial. 11/11/24, 11:33 AM ESA -JAN 2024 - UE23CS151A (set- 4) about:blank 6/7 (5.0 Marks) (6.0 Marks) (6.0 Marks) 3.e. Define the following along with their syntax in the tkinter module. i) Entry() (2 marks) ii) Label() (2 marks) iii) mainloop() (1 mark) 4.a. A. Given the list, perform the operations using functional programming constructs list1 = ['Java', 'Python', 'Csharp', 'Javascript', 'Go', 'Swift'] i) Convert the strings in a list to lowercase if the length of it less than 4. Then display it. (2 marks) ii) Display all the strings in the list which has the letter o in it. (2 marks) B. What is the purpose of reduce function from Python's functools module? (2 marks) 4.b. i) Find the output: (2 marks) numbers = [1, 2, 3, 4, 5] L = [x**2 for x in numbers if x % 2 != 0] print(L, type(L)) ii) Find the output: (2 marks) words = ["tiger", "leopard", "elephant", "lion"] count = [sum(1 for letter in word if letter not in 'aeiou') for word in words] print(count) iii) Write a function using lambda that returns the reverse of a string passed to it. (2 marks) EV0001282

54. Archive page 7 from Jan - (Set-4)__1cbShhBdIG.pdf; unit match candidate score 0.6667; source text/OCR may be partial. 11/11/24, 11:33 AM ESA -JAN 2024 - UE23CS151A (set- 4) about:blank 7/7 (6.0 Marks) (2.0 Marks) (5.0 Marks) 4.c. i) Brief about destructors with an example code. (2 marks) ii)Create a super class named A and a subclass named B. B class inherits the properties of the A class. The super class has an attribute called list L, which is homogeneous and contains integers. List may contain duplicate elements. Write a program such that the subclass removes the duplicate elements from the list nu adding a function the subclass(if any). (4 marks) Sample input: L=[10,20,30,30,20,90,10,80] Expected output: L = [10,20,30,90,80] 4.d. Find the output list1 = [1, 2, 3, 4] list2 = ['a', 'b', 'c', 'd'] list3 = ['x', 'y'] result = list(zip(list1, list2, list3)) print(result) 4.e. List the keywords of python used in Exception handling. EV0001283

55. Archive page 1 from June__1ba82WHmaJ.pdf; unit match candidate score 0.9167; source text/OCR may be partial. (6.0 Marks) (4.0 Marks) (5.0 Marks) PES University, Bengaluru (Established under Karnataka Act 16 of 2013) END SEMESTER ASSESSMENT (ESA) - Jan - June 2024 UE23CS151A - Python for Computational Problem Solving Total Marks : 100.0 1.a. Mention the output of the following expressions when executed each statement separately using the python 3.x. i) print((1),end = "") ii) a = 3; b = 0; print(a is not b) iii) a = {3,4,5}; print(type(a)) iv) a = 5; if a == 10: print("It is ten") else: print("It is not ten") v) print(list(range(2,15,3))) vi) print(p = 2) 1.b. Say True or False.
i) The print() function in python can take any number of arguments. ii) Python supports Object Oriented Programming paradigm. iii) Compiler checks for syntactical errors in the code. iv) The function print() returns a string always. 1.c. Take a whole number from the user. Display appropriate message by checking whether the entered number is a multiple of 10 or not. Expected output #1: Enter the number: 28 28 is not a multiple of 10 Expected output #2: Enter the number: 250 28 is a multiple of 10 Hint: A whole number is a positive integer starting from 0. EV0001284

56. Archive page 2 from June__1ba82WHmaJ.pdf; unit match candidate score 0.6667; source text/OCR may be partial. (5.0 Marks) (5.0 Marks) (5.0 Marks) (5.0 Marks) 1.d. Separate the keywords and variables from a given collection of identifiers. i) or ii) If iii) choco iv) pass v) break 1.e. List any three phases involved in the process of computational problem solving. Explain any one in detail. 2.a. Mention any five characteristics of list type in python. 2.b. Perform the given operations separately on a sample list. Sample_li = ["23", "yesterday", "44.6", "78.5", "900", "today", "tomorrow"] i) Create a new list with only first letter from the given list of strings and print the new list. #3 MARKS ii) Append the elements of the tuple t = (3,4,5) to the Sample_li using a single statement and print the Sample_li. #2 MARKS EV0001285

57. Archive page 3 from June__1ba82WHmaJ.pdf; unit match candidate score 0.4167; source text/OCR may be partial. (6.0 Marks) (4.0 Marks) (5.0 Marks) 2.c. Write a program to read all the contents from one text file and count the number of times the word 'sea' is repeated? Print the count to a new file. Source file content is as below: A sailor went to sea, sea, sea To see what he could see, see, see But all that he could see, see, see Was the bottom of the deep blue sea, sea, sea! Output file: 6 2.d. Find the output of below code. i) x=10 def f1(): x=x+1 print("Inside",x) f1() print("Outside",x) ii) x=10 def f1(): global x x=x+1 print("Inside",x) f1() print("Outside",x) 2.e. Find the output when the below code is executed using python 3.x. def my_func(p, q = 20, **kw): print(p); print(q); print(kw['a']); return type(kw) print(my_func(2, a = 4,b = 5,c = 6)) EV0001286

58. Archive page 6 from June__1ba82WHmaJ.pdf; unit match candidate score 0.5833; source text/OCR may be partial. (4.0 Marks) (5.0 Marks) 4.d. List the features of Object oriented Programming languages. 4.e. Fill the implementation of a new type – Chocolate to get the output as shown for the given client code. class Chocolate: #Fill this class #client code c1 = Chocolate("sweet", "imli") print("Number of chocolates:", Chocolate.count) c2 = Chocolate("sugary", "salty") c3 = Chocolate("imly", "sweety") print("Number of chocolates:", Chocolate.count) print(c2) del c1 print("Number of chocolates:", Chocolate.count) #expected output Number of chocolates: 1 Number of chocolates: 3 sugary salty Number of chocolates: 2 EV0001287

SHORT-NOTE RECAP

58 historical archive pages are included as question-source references. Inspect the original paper before relying on OCR or unit placement; no answers are newly authored here.

EV0001230

A Symbol glossary

Symbol	Name	Meaning	Appears in

B Sources, coverage and gaps

Source	Path	Use
Official unit catalogue	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/units.json	Official course/unit title and index.
Introduction transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/001_Introduction__ca07fd2e558c8a7c.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Introduction transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/001_Introduction__ca07fd2e558c8a7c.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Introduction transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/001_Introduction__ca07fd2e558c8a7c.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9996; only the JSON segmentation supplies the displayed timestamps.
Introduction transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/001_Introduction__ca07fd2e558c8a7c.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Man Cabbage Goat Wolf Problem transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/001_Man_Cabbage_Goat_Wolf_Problem__1ef4a632cd03282d.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Man Cabbage Goat Wolf Problem transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/001_Man_Cabbage_Goat_Wolf_Problem__1ef4a632cd03282d.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.

Source	Path	Use
Man Cabbage Goat Wolf Problem transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/001_Man_Cabbage_Goat_Wolf_Problem__1ef4a632cd03282d.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9988; only the JSON segmentation supplies the displayed timestamps.
Man Cabbage Goat Wolf Problem transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/001_Man_Cabbage_Goat_Wolf_Problem__1ef4a632cd03282d.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Process of Computation Problem Solving - Analysis, Design, Implementation, Testing transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/003_Process_of_Computation_Problem_Solving_-_Analysis_Design_Implementation_Testing__2849d807580cb7ae.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Process of Computation Problem Solving - Analysis, Design, Implementation, Testing transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/003_Process_of_Computation_Problem_Solving_-_Analysis_Design_Implementation_Testing__2849d807580cb7ae.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Process of Computation Problem Solving - Analysis, Design, Implementation, Testing transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/003_Process_of_Computation_Problem_Solving_-_Analysis_Design_Implementation_Testing__2849d807580cb7ae.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9971; only the JSON segmentation supplies the displayed timestamps.

Source	Path	Use
Process of Computation Problem Solving - Analysis, Design, Implementation, Testing transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/003_Process_of_Computation_Problem_Solving_-_Analysis_Design_Implementation_Testing__2849d807580cb7ae.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Process of Computation Problem Solving - Analysis, Design, Implementation, Testing transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/003_Process_of_Computation_Problem_Solving_-_Analysis_Design_Implementation_Testing__c5451c431739faf6.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Process of Computation Problem Solving - Analysis, Design, Implementation, Testing transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/003_Process_of_Computation_Problem_Solving_-_Analysis_Design_Implementation_Testing__c5451c431739faf6.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Process of Computation Problem Solving - Analysis, Design, Implementation, Testing transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/003_Process_of_Computation_Problem_Solving_-_Analysis_Design_Implementation_Testing__c5451c431739faf6.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9986; only the JSON segmentation supplies the displayed timestamps.
Process of Computation Problem Solving - Analysis, Design, Implementation, Testing transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/003_Process_of_Computation_Problem_Solving_-_Analysis_Design_Implementation_Testing__c5451c431739faf6.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.

Source	Path	Use
Computer Hardware and software transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/004_Computer_Hardware_and_software__d0cc7ac4df04ef9f.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Computer Hardware and software transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/004_Computer_Hardware_and_software__d0cc7ac4df04ef9f.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Computer Hardware and software transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/004_Computer_Hardware_and_software__d0cc7ac4df04ef9f.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9994; only the JSON segmentation supplies the displayed timestamps.
Computer Hardware and software transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/004_Computer_Hardware_and_software__d0cc7ac4df04ef9f.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Computer hardware transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/004_Computer_hardware__845ddeaeeec94ce6.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Computer hardware transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/004_Computer_hardware__845ddeaeeec94ce6.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.

Source	Path	Use
Computer hardware transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/004_Computer_hardware__845ddeaeeec94ce6.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9995; only the JSON segmentation supplies the displayed timestamps.
Computer hardware transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/004_Computer_hardware__845ddeaeeec94ce6.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Program Structure transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/008_Program_Structure__c33bde9f45f13d1b.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Program Structure transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/008_Program_Structure__c33bde9f45f13d1b.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Program Structure transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/008_Program_Structure__c33bde9f45f13d1b.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9932; only the JSON segmentation supplies the displayed timestamps.
Program Structure transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/008_Program_Structure__c33bde9f45f13d1b.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Program structure transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/008_Program_structure__141e953573b7fab3.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.

Source	Path	Use
Program structure transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/008_Program_structure__141e953573b7fab3.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Program structure transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/008_Program_structure__141e953573b7fab3.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9997; only the JSON segmentation supplies the displayed timestamps.
Program structure transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/008_Program_structure__141e953573b7fab3.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Program structure transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/008_Program_structure__c191b948fb0cbd98.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Program structure transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/008_Program_structure__c191b948fb0cbd98.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Program structure transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/008_Program_structure__c191b948fb0cbd98.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9997; only the JSON segmentation supplies the displayed timestamps.
Program structure transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/008_Program_structure__c191b948fb0cbd98.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.

Source	Path	Use
print function transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/009_print_function__fa29381f8beff681.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
print function transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/009_print_function__fa29381f8beff681.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
print function transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/009_print_function__fa29381f8beff681.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9995; only the JSON segmentation supplies the displayed timestamps.
print function transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/009_print_function__fa29381f8beff681.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Variable video 1 transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/010_Variable_video_1__6a17ee49c6f79e16.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Variable video 1 transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/010_Variable_video_1__6a17ee49c6f79e16.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Variable video 1 transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/010_Variable_video_1__6a17ee49c6f79e16.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9991; only the JSON segmentation supplies the displayed timestamps.

Source	Path	Use
Variable video 1 transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/010_Variable_video_1__6a17ee49c6f79e16.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
variable and id function transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/010_variable_and_id_function__92e3df0e8b11255d.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
variable and id function transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/010_variable_and_id_function__92e3df0e8b11255d.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
variable and id function transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/010_variable_and_id_function__92e3df0e8b11255d.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9986; only the JSON segmentation supplies the displayed timestamps.
variable and id function transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/010_variable_and_id_function__92e3df0e8b11255d.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
input function transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/011_input_function__729751eddd0f266f.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.

Source	Path	Use
input function transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/011_input_function__729751eddd0f266f.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
input function transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/011_input_function__729751eddd0f266f.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9995; only the JSON segmentation supplies the displayed timestamps.
input function transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/011_input_function__729751eddd0f266f.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
input function transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/011_input_function__c33256eff090b5ec.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
input function transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/011_input_function__c33256eff090b5ec.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
input function transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/011_input_function__c33256eff090b5ec.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9996; only the JSON segmentation supplies the displayed timestamps.
input function transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/011_input_function__c33256eff090b5ec.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.

Source	Path	Use
Arithmetic, membership and relational operators transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/012_Arithmetic_membership_and_relational_operators__ff38db0dde601b27.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Arithmetic, membership and relational operators transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/012_Arithmetic_membership_and_relational_operators__ff38db0dde601b27.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Arithmetic, membership and relational operators transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/012_Arithmetic_membership_and_relational_operators__ff38db0dde601b27.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9992; only the JSON segmentation supplies the displayed timestamps.
Arithmetic, membership and relational operators transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/012_Arithmetic_membership_and_relational_operators__ff38db0dde601b27.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Bitwise operators transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/012_Bitwise_operators__45d6586e889881f1.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Bitwise operators transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/012_Bitwise_operators__45d6586e889881f1.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.

Source	Path	Use
Bitwise operators transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/012_Bitwise_operators__45d6586e889881f1.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9996; only the JSON segmentation supplies the displayed timestamps.
Bitwise operators transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/012_Bitwise_operators__45d6586e889881f1.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
logical operators and short circuit evaluation transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/012_logical_operators_and_short_circuit_evaluation__7ee47e67d3dc874f.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
logical operators and short circuit evaluation transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/012_logical_operators_and_short_circuit_evaluation__7ee47e67d3dc874f.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
logical operators and short circuit evaluation transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/012_logical_operators_and_short_circuit_evaluation__7ee47e67d3dc874f.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9969; only the JSON segmentation supplies the displayed timestamps.
logical operators and short circuit evaluation transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/012_logical_operators_and_short_circuit_evaluation__7ee47e67d3dc874f.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.

Source	Path	Use
operators and expressions transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/012_operators_and_expressions__c32527d44167ebb9.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
operators and expressions transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/012_operators_and_expressions__c32527d44167ebb9.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
operators and expressions transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/012_operators_and_expressions__c32527d44167ebb9.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9998; only the JSON segmentation supplies the displayed timestamps.
operators and expressions transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/012_operators_and_expressions__c32527d44167ebb9.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
sequential, selection control transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/015_sequential_selection_control__3c385369aff92ed0.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
sequential, selection control transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/015_sequential_selection_control__3c385369aff92ed0.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.

Source	Path	Use
sequential, selection control transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/015_sequential_selection_control__3c385369aff92ed0.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9994; only the JSON segmentation supplies the displayed timestamps.
sequential, selection control transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/015_sequential_selection_control__3c385369aff92ed0.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
iterative control structure transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/016_iterative_control_structure__f5f1a1d31c8dff99.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
iterative control structure transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/016_iterative_control_structure__f5f1a1d31c8dff99.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
iterative control structure transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/016_iterative_control_structure__f5f1a1d31c8dff99.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9995; only the JSON segmentation supplies the displayed timestamps.
iterative control structure transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/016_iterative_control_structure__f5f1a1d31c8dff99.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.

Source	Path	Use
String Operations transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/041_String_Operations__8206ac9c8ddf207b.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
String Operations transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/041_String_Operations__8206ac9c8ddf207b.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
String Operations transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/041_String_Operations__8206ac9c8ddf207b.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9989; only the JSON segmentation supplies the displayed timestamps.
String Operations transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/041_String_Operations__8206ac9c8ddf207b.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Slide page text: UE23CS151A PCPS Unit 3 Sowmya Shree Sindhu Class 83 84 85 Modules	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE23CS151A PCPS Unit 3 Sowmya Shree Sindhu Class 83 84 85 Modules	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.

Source	Path	Use
Page-render source for UE23CS151A PCPS Unit 3 Sowmya Shree Sindhu Class 83 84 85 Modules (converted PDF; original hash d6dc37db35a480942e7cde93c54bf6cad030db9b925d2ee71ebc02d5091c4f25)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: UE24CS151A PCPS Unit 1 SindhuRPai Class 1 2 Intro to title	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__1__2_Intro_to_title__37baecfa2540966d.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 1 SindhuRPai Class 1 2 Intro to title	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__1__2_Intro_to_title__37baecfa2540966d.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for UE24CS151A PCPS Unit 1 SindhuRPai Class 1 2 Intro to title (converted PDF; original hash 7eaea6a237d5be0a996eadc48813107306d0efdca313cf9fd092a18231688315)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__1__2_Intro_to_title__37baecfa2540966d.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: UE24CS151A PCPS Unit 3 Sindhu Class 14 15 Modules	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.

Source	Path	Use
Alternate extracted slide page text: UE24CS151A PCPS Unit 3 Sindhu Class 14 15 Modules	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for UE24CS151A PCPS Unit 3 Sindhu Class 14 15 Modules (converted PDF; original hash 5f23fb8257ff88f8822fddae23f377b282ffb7c2b945a66003f796669d7078e3)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: UE24CS151A PCPS Unit 1 SindhuRPai Class 3 Process of CPS	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/003_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__3_Process_of_CPS__3f3254779c6a452a.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 1 SindhuRPai Class 3 Process of CPS	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/003_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__3_Process_of_CPS__3f3254779c6a452a.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for UE24CS151A PCPS Unit 1 SindhuRPai Class 3 Process of CPS (converted PDF; original hash 5bb4abd3e907609962af7a67d252efb563a03696068503cf2b4a2344063789fbb)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/003_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__3_Process_of_CPS__3f3254779c6a452a.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.

Source	Path	Use
Slide page text: UE24CS151A PCPS Unit 1 SindhuRPai Class 4 5 Computer Hardware software	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/004_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__4_5_Computer_Hardware_software__57579a8e9ad25cbb.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 1 SindhuRPai Class 4 5 Computer Hardware software	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/004_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__4_5_Computer_Hardware_software__57579a8e9ad25cbb.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for UE24CS151A PCPS Unit 1 SindhuRPai Class 4 5 Computer Hardware software (converted PDF; original hash 19e83e41e1f18720140876bf601405c14698c9efe95c3f4f43d1f9d0316ea332)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/004_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__4_5_Computer_Hardware_software__57579a8e9ad25cbb.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: UE24CS151A PCPS Unit 1 SindhuRPai Class 6 Intro to Programming Languages	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/006_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__6_Intro_to_Programming_Languages__0cbeda79209c1dbf.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 1 SindhuRPai Class 6 Intro to Programming Languages	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/006_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__6_Intro_to_Programming_Languages__0cbeda79209c1dbf.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.

Source	Path	Use
Page-render source for UE24CS151A PCPS Unit 1 SindhuRPai Class 6 Intro to Programming Languages (converted PDF; original hash 4818457e456ef5ddbd6e665e8631502f553418ef564bf0718f5f50501ef3f818)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/006_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__6_Intro_to_Programming_Languages__0cbeda79209c1dbf.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: UE24CS151A PCPS Unit 1 SindhuRPai Class 7 Syntax semantics translation models	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/007_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__7_Syntax_semantics_translation_models__656e0afe2d8d3282.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 1 SindhuRPai Class 7 Syntax semantics translation models	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/007_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__7_Syntax_semantics_translation_models__656e0afe2d8d3282.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for UE24CS151A PCPS Unit 1 SindhuRPai Class 7 Syntax semantics translation models (converted PDF; original hash bd5b4881571bd02e47afa988e5e24e089bbf620bc9bd823b27c88a762252fb02)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/007_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__7_Syntax_semantics_translation_models__656e0afe2d8d3282.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: UE24CS151A PCPS Unit 1 SindhuRPai Class 8 Installation and execution	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/008_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__8_Installation_and_execution__2fbbee57dfe57679.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.

Source	Path	Use
Alternate extracted slide page text: UE24CS151A PCPS Unit 1 SindhuRPai Class 8 Installation and execution	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/008_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__8_Installation_and_execution__2fbbee57dfe57679.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for UE24CS151A PCPS Unit 1 SindhuRPai Class 8 Installation and execution (converted PDF; original hash ae7555a2800a32932355a1e1b7e7fcc203aac277b798052079a43edb274bc3c8)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/008_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__8_Installation_and_execution__2fbbee57dfe57679.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: UE24CS151A PCPS Unit 1 SindhuRPai Class 8 Program structure	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/008_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__8_Program_structure__ed4abb0db838da93.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 1 SindhuRPai Class 8 Program structure	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/008_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__8_Program_structure__ed4abb0db838da93.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for UE24CS151A PCPS Unit 1 SindhuRPai Class 8 Program structure (converted PDF; original hash c4f94ab07e4c657e4877f4613ecf7a2fe96eed53a554092a38d82844b5c3cb73)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/008_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__8_Program_structure__ed4abb0db838da93.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.

Source	Path	Use
Slide page text: IDLE	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: IDLE	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for IDLE (converted PDF; original hash 7dc0bdb848c62ce25230b2a2552529bce68cd6ed9140c11fa060db5ad16d9e90)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: UE24CS151A PCPS Unit 1 SindhuRPai Class 9 output function print	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__9_output_function_print__1ca49294268909b9.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 1 SindhuRPai Class 9 output function print	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__9_output_function_print__1ca49294268909b9.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for UE24CS151A PCPS Unit 1 SindhuRPai Class 9 output function print (converted PDF; original hash aa9c058d97b323eb3f1bce810006ad283b52dad346b0bdde16d9c5b7fbea7bb2)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__9_output_function_print__1ca49294268909b9.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.

Source	Path	Use
Slide page text: UE24CS151A PCPS Unit 1 SindhuRPai Class 10 variable id type	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/010_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__10_variable_id_type__b024fe65bb8bef33.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 1 SindhuRPai Class 10 variable id type	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/010_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__10_variable_id_type__b024fe65bb8bef33.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for UE24CS151A PCPS Unit 1 SindhuRPai Class 10 variable id type (converted PDF; original hash a2054b399002f95eb5351ec8f47676d24c7180c4f0618cae39011724cc9ea66f)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/010_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__10_variable_id_type__b024fe65bb8bef33.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: UE24CS151A PCPS Unit 1 SindhuRPai Class 11 input function	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/011_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__11_input_function__fe2e37a1fa374eaf.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 1 SindhuRPai Class 11 input function	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/011_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__11_input_function__fe2e37a1fa374eaf.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.

Source	Path	Use
Page-render source for UE24CS151A PCPS Unit 1 SindhuRPai Class 11 input function (converted PDF; original hash 789fa22f6e8813727f06aa92ca42305fa9663a997a14017bf8a1291cddd60160)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/011_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__11_input_function__fe2e37a1fa374eaf.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: UE24CS151A PCPS Unit 1 SindhuRPai Class 12 Operators Part1	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/012_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__12_Operators_Part1__b53129837a8bbfbd.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 1 SindhuRPai Class 12 Operators Part1	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/012_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__12_Operators_Part1__b53129837a8bbfbd.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for UE24CS151A PCPS Unit 1 SindhuRPai Class 12 Operators Part1 (converted PDF; original hash 9c50d1cabe8737f8b6ce34cd242f300e169cdffbee523d31ac671b45fe50b1f2)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/012_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__12_Operators_Part1__b53129837a8bbfbd.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: Operator Polymorphism	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/014_Operator_Polymorphism__32ad541949df1cab.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: Operator Polymorphism	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/014_Operator_Polymorphism__32ad541949df1cab.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.

Source	Path	Use
Page-render source for Operator Polymorphism (converted PDF; original hash f73f31e80dd965d432bfa3e587c4f6296d49489a6ee16b86f6c64eb959b0faf4)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/014_Operator_Polymorphism__32ad541949df1cab.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: Operator precedence and associativity	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/014_Operator_precedence_and_associativity__d91f176414ae90ad.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: Operator precedence and associativity	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/014_Operator_precedence_and_associativity__d91f176414ae90ad.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for Operator precedence and associativity (converted PDF; original hash 25fedd35c1a45740981f8e60315c4ba28620516f88089827b68da2057257ea75)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/014_Operator_precedence_and_associativity__d91f176414ae90ad.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: UE24CS151A PCPS Unit 1 SindhuRPai Class 15 16 Control structures	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/015_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__15_16_Control_structures__bcb5c5877df41eba.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 1 SindhuRPai Class 15 16 Control structures	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/015_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__15_16_Control_structures__bcb5c5877df41eba.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.

Source	Path	Use
Page-render source for UE24CS151A PCPS Unit 1 SindhuRPai Class 15 16 Control structures (converted PDF; original hash 9f9dc7b6245dd3f58cf0c0e4592bf57ecda60736fb392763d5a1e8495f4bfd44)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/015_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__15_16_Control_structures__bcb5c5877df41eba.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: UE24CS151A PCPS Unit 2 SindhuRPai Class 1 General intro to data structures in python	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/027_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__1_General_intro_to_data_structures_in_python__a531eb93c7c504ed.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 2 SindhuRPai Class 1 General intro to data structures in python	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/027_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__1_General_intro_to_data_structures_in_python__a531eb93c7c504ed.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for UE24CS151A PCPS Unit 2 SindhuRPai Class 1 General intro to data structures in python (converted PDF; original hash f9c2774ed4c06b020b330361d708d4c9dbddf28f7eb5e28c3e2e2687130ef91d)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/027_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__1_General_intro_to_data_structures_in_python__a531eb93c7c504ed.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: UE24CS151A PCPS Unit 2 SindhuRPai Class 9 10 Strings types operations	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.

Source	Path	Use
Alternate extracted slide page text: UE24CS151A PCPS Unit 2 SindhuRPai Class 9 10 Strings types operations	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for UE24CS151A PCPS Unit 2 SindhuRPai Class 9 10 Strings types operations (converted PDF; original hash c38d2f28c14f3eae41c162f2a5c12da5e0ae9dd217dd3bc58f9bbb806f156513)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: UE24CS151A PCPS Unit 2 SindhuRPai Class 2 lists	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 2 SindhuRPai Class 2 lists	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for UE24CS151A PCPS Unit 2 SindhuRPai Class 2 lists (converted PDF; original hash db25dac784ce6be4ce8de74c6ed9167a461035fdd661176317408e6595a1ed55)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Extracted historical pages: Dec__1aef5kbQ-b.pdf	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/First_Year_8enterprise/Files/Python/Papers/ESA Papers/UE22CS151A/Answers/Dec__1aef5kbQ-b.pdf	Candidate-matched page text excerpt; original PDF bytes, hash and path remain in datasets/question-archive.json.

Source	Path	Use
Extracted historical pages: July__17cJZWr8dZ.pdf	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/First_Year_8enterprise/Files/Python/Papers/ESA Papers/UE22CS151A/Answers/July__17cJZWr8dZ.pdf	Candidate-matched page text excerpt; original PDF bytes, hash and path remain in datasets/question-archive.json.
Extracted historical pages: May__1SdbvafL_8.pdf	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/First_Year_8enterprise/Files/Python/Papers/ESA Papers/UE22CS151A/Answers/May__1SdbvafL_8.pdf	Candidate-matched page text excerpt; original PDF bytes, hash and path remain in datasets/question-archive.json.
Extracted historical pages: Dec__1enz7ur0jb.pdf	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/First_Year_8enterprise/Files/Python/Papers/ESA Papers/UE22CS151A/Dec__1enz7ur0jb.pdf	Candidate-matched page text excerpt; original PDF bytes, hash and path remain in datasets/question-archive.json.
Extracted historical pages: July__1e0jrB6Yh8.pdf	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/First_Year_8enterprise/Files/Python/Papers/ESA Papers/UE22CS151A/July__1e0jrB6Yh8.pdf	Candidate-matched page text excerpt; original PDF bytes, hash and path remain in datasets/question-archive.json.
Extracted historical pages: May__1rBeUIlse2.pdf	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/First_Year_8enterprise/Files/Python/Papers/ESA Papers/UE22CS151A/May__1rBeUIlse2.pdf	Candidate-matched page text excerpt; original PDF bytes, hash and path remain in datasets/question-archive.json.
Extracted historical pages: Jan - (Set-1)__1S8XvO2bSr.pdf	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/First_Year_8enterprise/Files/Python/Papers/ESA Papers/UE23CS151A/Answers/Jan - (Set-1)__1S8XvO2bSr.pdf	Candidate-matched page text excerpt; original PDF bytes, hash and path remain in datasets/question-archive.json.
Extracted historical pages: Jan - (Set-2)__1HZjDaXNmS.pdf	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/First_Year_8enterprise/Files/Python/Papers/ESA Papers/UE23CS151A/Answers/Jan - (Set-2)__1HZjDaXNmS.pdf	Candidate-matched page text excerpt; original PDF bytes, hash and path remain in datasets/question-archive.json.
Extracted historical pages: Jan - (Set-1)__1kzIfeDFc9.pdf	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/First_Year_8enterprise/Files/Python/Papers/ESA Papers/UE23CS151A/Jan - (Set-1)__1kzIfeDFc9.pdf	Candidate-matched page text excerpt; original PDF bytes, hash and path remain in datasets/question-archive.json.
Extracted historical pages: Jan - (Set-2)__1YxZdNR8Ws.pdf	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/First_Year_8enterprise/Files/Python/Papers/ESA Papers/UE23CS151A/Jan - (Set-2)__1YxZdNR8Ws.pdf	Candidate-matched page text excerpt; original PDF bytes, hash and path remain in datasets/question-archive.json.

Source	Path	Use
Extracted historical pages: Jan - (Set-3)__1ghwvCbUda.pdf	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/First_Year_8enterprise/Files/Python/Papers/ESA Papers/UE23CS151A/Jan - (Set-3)__1ghwvCbUda.pdf	Candidate-matched page text excerpt; original PDF bytes, hash and path remain in datasets/question-archive.json.
Extracted historical pages: Jan - (Set-4)__1cbShhBdIG.pdf	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/First_Year_8enterprise/Files/Python/Papers/ESA Papers/UE23CS151A/Jan - (Set-4)__1cbShhBdIG.pdf	Candidate-matched page text excerpt; original PDF bytes, hash and path remain in datasets/question-archive.json.
Extracted historical pages: June__1ba82WHmaJ.pdf	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/First_Year_8enterprise/Files/Python/Papers/ESA Papers/UE23CS151A/June__1ba82WHmaJ.pdf	Candidate-matched page text excerpt; original PDF bytes, hash and path remain in datasets/question-archive.json.

Historical question-source excerpts are shown only when a page has a strong text-based unit match. They are source references, not new solutions or exam predictions.

Known gaps, conflicts and corrections

1. 2 video-to-slide associations were not promoted to a match. Candidate rankings are listed in the transcript corpus and video-slide-matches.csv; the slide decks appear separately.
2. 16 transcript segments have low ASR confidence; they are flagged with timestamps in the transcript blocks and indexed in the corpus.
3. This is a visibly labelled source-preservation draft. Human page-level semantic and visual review is not complete; no unsupported subject summary or formula correction is claimed.
4. The 19.26 GiB MP4 files remain hosted on Hugging Face; the full video path/hash and transcript coverage are indexed. The full audio/video was not watched end-to-end.
5. Formula candidates are machine-extracted text with source locators; use the adjacent original slide page and the formula index before treating a candidate as an authoritative equation.

Coverage

Every inventoried source unit has an explicit coverage decision. Covered means reviewed and mapped to content; exclusions and gaps remain visible.

Evidence	Source unit	Status	Decision
EV0000001	SRC00001/catalogue	covered	Unit title and index copied from the official units.json catalogue; human content review is pending.

Evidence	Source unit	Status	Decision
EV00000002	SRC00002/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0035. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000003	SRC00003/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0035. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000004	SRC00004/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0035. Sidecar variants compare at similarity 0.9996; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000005	SRC00005/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0035. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000006	SRC00002/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0036 to s0067. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000007	SRC00003/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0036 to s0067. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000008	SRC00004/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0036 to s0067. Sidecar variants compare at similarity 0.9996; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV0000009	SRC00005/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0036 to s0067. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000010	SRC00006/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0023. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000011	SRC00007/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0023. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000012	SRC00008/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0023. Sidecar variants compare at similarity 0.9988; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000013	SRC00009/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0023. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000014	SRC00006/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0024 to s0047. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000015	SRC00007/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0024 to s0047. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV0000016	SRC00008/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0024 to s0047. Sidecar variants compare at similarity 0.9988; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000017	SRC00009/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0024 to s0047. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000018	SRC00006/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0048 to s0066. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000019	SRC00007/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0048 to s0066. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000020	SRC00008/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0048 to s0066. Sidecar variants compare at similarity 0.9988; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000021	SRC00009/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0048 to s0066. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000022	SRC00010/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0026. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV0000023	SRC00011/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0026. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000024	SRC00012/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0026. Sidecar variants compare at similarity 0.9971; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000025	SRC00013/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0026. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000026	SRC00010/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0027 to s0050. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000027	SRC00011/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0027 to s0050. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000028	SRC00012/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0027 to s0050. Sidecar variants compare at similarity 0.9971; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000029	SRC00013/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0027 to s0050. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV0000030	SRC00010/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0051 to s0056. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000031	SRC00011/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0051 to s0056. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000032	SRC00012/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0051 to s0056. Sidecar variants compare at similarity 0.9971; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000033	SRC00013/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0051 to s0056. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000034	SRC00014/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000035	SRC00015/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000036	SRC00016/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare at similarity 0.9986; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV00000037	SRC00017/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000038	SRC00014/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0022 to s0046. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000039	SRC00015/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0022 to s0046. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000040	SRC00016/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0022 to s0046. Sidecar variants compare at similarity 0.9986; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000041	SRC00017/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0022 to s0046. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000042	SRC00014/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0047 to s0063. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000043	SRC00015/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0047 to s0063. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV0000044	SRC00016/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0047 to s0063. Sidecar variants compare at similarity 0.9986; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000045	SRC00017/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0047 to s0063. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000046	SRC00014/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0064 to s0075. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000047	SRC00015/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0064 to s0075. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000048	SRC00016/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0064 to s0075. Sidecar variants compare at similarity 0.9986; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000049	SRC00017/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0064 to s0075. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000050	SRC00014/t001200-001499	covered	Timestamp interval t001200-001499; canonical segment IDs s0076 to s0081. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV0000051	SRC00015/t001200-001499	covered	Timestamp interval t001200-001499; canonical segment IDs s0076 to s0081. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000052	SRC00016/t001200-001499	covered	Timestamp interval t001200-001499; canonical segment IDs s0076 to s0081. Sidecar variants compare at similarity 0.9986; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000053	SRC00017/t001200-001499	covered	Timestamp interval t001200-001499; canonical segment IDs s0076 to s0081. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000054	SRC00018/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0019. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000055	SRC00019/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0019. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000056	SRC00020/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0019. Sidecar variants compare at similarity 0.9994; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000057	SRC00021/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0019. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV00000058	SRC00018/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0020 to s0038. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000059	SRC00019/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0020 to s0038. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000060	SRC00020/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0020 to s0038. Sidecar variants compare at similarity 0.9994; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000061	SRC00021/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0020 to s0038. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000062	SRC00018/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0039 to s0057. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000063	SRC00019/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0039 to s0057. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000064	SRC00020/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0039 to s0057. Sidecar variants compare at similarity 0.9994; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV0000065	SRC00021/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0039 to s0057. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000066	SRC00018/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0058 to s0077. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000067	SRC00019/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0058 to s0077. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000068	SRC00020/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0058 to s0077. Sidecar variants compare at similarity 0.9994; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000069	SRC00021/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0058 to s0077. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000070	SRC00018/t001200-001499	covered	Timestamp interval t001200-001499; canonical segment IDs s0078 to s0086. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000071	SRC00019/t001200-001499	covered	Timestamp interval t001200-001499; canonical segment IDs s0078 to s0086. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV00000072	SRC00020/t001200-001499	covered	Timestamp interval t001200-001499; canonical segment IDs s0078 to s0086. Sidecar variants compare at similarity 0.9994; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000073	SRC00021/t001200-001499	covered	Timestamp interval t001200-001499; canonical segment IDs s0078 to s0086. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000074	SRC00022/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0022. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000075	SRC00023/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0022. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000076	SRC00024/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0022. Sidecar variants compare at similarity 0.9995; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000077	SRC00025/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0022. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000078	SRC00022/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0023 to s0043. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV00000079	SRC00023/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0023 to s0043. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000080	SRC00024/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0023 to s0043. Sidecar variants compare at similarity 0.9995; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000081	SRC00025/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0023 to s0043. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000082	SRC00022/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0044 to s0054. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000083	SRC00023/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0044 to s0054. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000084	SRC00024/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0044 to s0054. Sidecar variants compare at similarity 0.9995; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000085	SRC00025/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0044 to s0054. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV00000086	SRC00026/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0005. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000087	SRC00027/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0005. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000088	SRC00028/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0005. Sidecar variants compare at similarity 0.9932; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000089	SRC00029/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0005. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000090	SRC00030/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0018. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000091	SRC00031/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0018. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000092	SRC00032/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0018. Sidecar variants compare at similarity 0.9997; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV0000093	SRC00033/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0018. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000094	SRC00030/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0019 to s0038. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000095	SRC00031/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0019 to s0038. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000096	SRC00032/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0019 to s0038. Sidecar variants compare at similarity 0.9997; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000097	SRC00033/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0019 to s0038. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000098	SRC00030/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0039 to s0058. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000099	SRC00031/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0039 to s0058. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV0000100	SRC00032/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0039 to s0058. Sidecar variants compare at similarity 0.9997; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000101	SRC00033/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0039 to s0058. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000102	SRC00030/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0059 to s0078. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000103	SRC00031/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0059 to s0078. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000104	SRC00032/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0059 to s0078. Sidecar variants compare at similarity 0.9997; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000105	SRC00033/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0059 to s0078. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000106	SRC00030/t001200-001499	covered	Timestamp interval t001200-001499; canonical segment IDs s0079 to s0082. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV0000107	SRC00031/t001200-001499	covered	Timestamp interval t001200-001499; canonical segment IDs s0079 to s0082. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000108	SRC00032/t001200-001499	covered	Timestamp interval t001200-001499; canonical segment IDs s0079 to s0082. Sidecar variants compare at similarity 0.9997; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000109	SRC00033/t001200-001499	covered	Timestamp interval t001200-001499; canonical segment IDs s0079 to s0082. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000110	SRC00034/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000111	SRC00035/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000112	SRC00036/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare at similarity 0.9997; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000113	SRC00037/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV0000114	SRC00034/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0022 to s0040. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000115	SRC00035/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0022 to s0040. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000116	SRC00036/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0022 to s0040. Sidecar variants compare at similarity 0.9997; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000117	SRC00037/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0022 to s0040. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000118	SRC00034/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0041 to s0061. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000119	SRC00035/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0041 to s0061. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000120	SRC00036/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0041 to s0061. Sidecar variants compare at similarity 0.9997; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV0000121	SRC00037/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0041 to s0061. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000122	SRC00034/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0062 to s0078. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000123	SRC00035/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0062 to s0078. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000124	SRC00036/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0062 to s0078. Sidecar variants compare at similarity 0.9997; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000125	SRC00037/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0062 to s0078. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000126	SRC00034/t001200-001499	covered	Timestamp interval t001200-001499; canonical segment IDs s0079 to s0096. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000127	SRC00035/t001200-001499	covered	Timestamp interval t001200-001499; canonical segment IDs s0079 to s0096. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV0000128	SRC00036/t001200-001499	covered	Timestamp interval t001200-001499; canonical segment IDs s0079 to s0096. Sidecar variants compare at similarity 0.9997; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000129	SRC00037/t001200-001499	covered	Timestamp interval t001200-001499; canonical segment IDs s0079 to s0096. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000130	SRC00038/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0023. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000131	SRC00039/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0023. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000132	SRC00040/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0023. Sidecar variants compare at similarity 0.9995; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000133	SRC00041/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0023. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000134	SRC00038/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0024 to s0042. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV0000135	SRC00039/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0024 to s0042. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000136	SRC00040/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0024 to s0042. Sidecar variants compare at similarity 0.9995; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000137	SRC00041/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0024 to s0042. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000138	SRC00038/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0043 to s0066. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000139	SRC00039/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0043 to s0066. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000140	SRC00040/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0043 to s0066. Sidecar variants compare at similarity 0.9995; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000141	SRC00041/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0043 to s0066. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV0000142	SRC00038/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0067 to s0081. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000143	SRC00039/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0067 to s0081. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000144	SRC00040/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0067 to s0081. Sidecar variants compare at similarity 0.9995; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000145	SRC00041/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0067 to s0081. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000146	SRC00042/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0022. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000147	SRC00043/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0022. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000148	SRC00044/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0022. Sidecar variants compare at similarity 0.9991; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV0000149	SRC00045/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0022. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000150	SRC00042/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0023 to s0041. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000151	SRC00043/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0023 to s0041. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000152	SRC00044/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0023 to s0041. Sidecar variants compare at similarity 0.9991; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000153	SRC00045/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0023 to s0041. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000154	SRC00042/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0042 to s0044. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000155	SRC00043/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0042 to s0044. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV0000156	SRC00044/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0042 to s0044. Sidecar variants compare at similarity 0.9991; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000157	SRC00045/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0042 to s0044. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000158	SRC00046/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000159	SRC00047/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000160	SRC00048/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare at similarity 0.9986; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000161	SRC00049/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000162	SRC00046/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0021 to s0039. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV0000163	SRC00047/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0021 to s0039. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000164	SRC00048/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0021 to s0039. Sidecar variants compare at similarity 0.9986; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000165	SRC00049/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0021 to s0039. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000166	SRC00050/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000167	SRC00051/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000168	SRC00052/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare at similarity 0.9995; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000169	SRC00053/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV0000170	SRC00050/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0021 to s0038. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000171	SRC00051/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0021 to s0038. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000172	SRC00052/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0021 to s0038. Sidecar variants compare at similarity 0.9995; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000173	SRC00053/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0021 to s0038. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000174	SRC00050/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0039 to s0060. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000175	SRC00051/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0039 to s0060. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000176	SRC00052/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0039 to s0060. Sidecar variants compare at similarity 0.9995; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV0000177	SRC00053/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0039 to s0060. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000178	SRC00050/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0061 to s0061. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000179	SRC00051/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0061 to s0061. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000180	SRC00052/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0061 to s0061. Sidecar variants compare at similarity 0.9995; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000181	SRC00053/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0061 to s0061. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000182	SRC00054/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000183	SRC00055/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV0000184	SRC00056/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare at similarity 0.9996; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000185	SRC00057/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000186	SRC00054/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0021 to s0041. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000187	SRC00055/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0021 to s0041. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000188	SRC00056/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0021 to s0041. Sidecar variants compare at similarity 0.9996; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000189	SRC00057/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0021 to s0041. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000190	SRC00054/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0042 to s0060. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV0000191	SRC00055/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0042 to s0060. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000192	SRC00056/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0042 to s0060. Sidecar variants compare at similarity 0.9996; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000193	SRC00057/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0042 to s0060. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000194	SRC00054/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0061 to s0067. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000195	SRC00055/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0061 to s0067. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000196	SRC00056/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0061 to s0067. Sidecar variants compare at similarity 0.9996; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000197	SRC00057/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0061 to s0067. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV00001 98	SRC00058/t00000 0-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0019. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00001 99	SRC00059/t00000 0-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0019. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00002 00	SRC00060/t00000 0-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0019. Sidecar variants compare at similarity 0.9992; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00002 01	SRC00061/t00000 0-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0019. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00002 02	SRC00058/t00030 0-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0020 to s0038. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00002 03	SRC00059/t00030 0-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0020 to s0038. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00002 04	SRC00060/t00030 0-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0020 to s0038. Sidecar variants compare at similarity 0.9992; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV0000205	SRC00061/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0020 to s0038. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000206	SRC00058/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0039 to s0058. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000207	SRC00059/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0039 to s0058. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000208	SRC00060/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0039 to s0058. Sidecar variants compare at similarity 0.9992; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000209	SRC00061/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0039 to s0058. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000210	SRC00058/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0059 to s0077. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000211	SRC00059/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0059 to s0077. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV0000212	SRC00060/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0059 to s0077. Sidecar variants compare at similarity 0.9992; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000213	SRC00061/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0059 to s0077. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000214	SRC00058/t001200-001499	covered	Timestamp interval t001200-001499; canonical segment IDs s0078 to s0096. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000215	SRC00059/t001200-001499	covered	Timestamp interval t001200-001499; canonical segment IDs s0078 to s0096. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000216	SRC00060/t001200-001499	covered	Timestamp interval t001200-001499; canonical segment IDs s0078 to s0096. Sidecar variants compare at similarity 0.9992; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000217	SRC00061/t001200-001499	covered	Timestamp interval t001200-001499; canonical segment IDs s0078 to s0096. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000218	SRC00058/t001500-001799	covered	Timestamp interval t001500-001799; canonical segment IDs s0097 to s0102. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV0000219	SRC00059/t001500-001799	covered	Timestamp interval t001500-001799; canonical segment IDs s0097 to s0102. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000220	SRC00060/t001500-001799	covered	Timestamp interval t001500-001799; canonical segment IDs s0097 to s0102. Sidecar variants compare at similarity 0.9992; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000221	SRC00061/t001500-001799	covered	Timestamp interval t001500-001799; canonical segment IDs s0097 to s0102. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000222	SRC00062/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000223	SRC00063/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000224	SRC00064/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare at similarity 0.9996; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000225	SRC00065/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV00002 26	SRC00062/t00030 0-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0021 to s0039. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00002 27	SRC00063/t00030 0-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0021 to s0039. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00002 28	SRC00064/t00030 0-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0021 to s0039. Sidecar variants compare at similarity 0.9996; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00002 29	SRC00065/t00030 0-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0021 to s0039. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00002 30	SRC00062/t00060 0-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0040 to s0059. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00002 31	SRC00063/t00060 0-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0040 to s0059. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00002 32	SRC00064/t00060 0-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0040 to s0059. Sidecar variants compare at similarity 0.9996; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV0000233	SRC00065/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0040 to s0059. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000234	SRC00062/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0060 to s0068. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000235	SRC00063/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0060 to s0068. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000236	SRC00064/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0060 to s0068. Sidecar variants compare at similarity 0.9996; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000237	SRC00065/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0060 to s0068. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000238	SRC00066/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000239	SRC00067/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV0000240	SRC00068/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare at similarity 0.9969; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000241	SRC00069/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000242	SRC00066/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0022 to s0030. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000243	SRC00067/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0022 to s0030. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000244	SRC00068/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0022 to s0030. Sidecar variants compare at similarity 0.9969; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000245	SRC00069/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0022 to s0030. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000246	SRC00070/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0022. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV0000247	SRC00071/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0022. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000248	SRC00072/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0022. Sidecar variants compare at similarity 0.9998; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000249	SRC00073/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0022. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000250	SRC00070/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0023 to s0043. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000251	SRC00071/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0023 to s0043. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000252	SRC00072/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0023 to s0043. Sidecar variants compare at similarity 0.9998; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000253	SRC00073/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0023 to s0043. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV0000254	SRC00070/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0044 to s0063. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000255	SRC00071/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0044 to s0063. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000256	SRC00072/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0044 to s0063. Sidecar variants compare at similarity 0.9998; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000257	SRC00073/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0044 to s0063. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000258	SRC00070/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0064 to s0081. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000259	SRC00071/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0064 to s0081. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000260	SRC00072/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0064 to s0081. Sidecar variants compare at similarity 0.9998; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV00002 61	SRC00073/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0064 to s0081. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00002 62	SRC00070/t001200-001499	covered	Timestamp interval t001200-001499; canonical segment IDs s0082 to s0101. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00002 63	SRC00071/t001200-001499	covered	Timestamp interval t001200-001499; canonical segment IDs s0082 to s0101. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00002 64	SRC00072/t001200-001499	covered	Timestamp interval t001200-001499; canonical segment IDs s0082 to s0101. Sidecar variants compare at similarity 0.9998; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00002 65	SRC00073/t001200-001499	covered	Timestamp interval t001200-001499; canonical segment IDs s0082 to s0101. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00002 66	SRC00070/t001500-001799	covered	Timestamp interval t001500-001799; canonical segment IDs s0102 to s0120. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00002 67	SRC00071/t001500-001799	covered	Timestamp interval t001500-001799; canonical segment IDs s0102 to s0120. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV00002 68	SRC00072/t00150 0-001799	covered	Timestamp interval t001500-001799; canonical segment IDs s0102 to s0120. Sidecar variants compare at similarity 0.9998; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00002 69	SRC00073/t00150 0-001799	covered	Timestamp interval t001500-001799; canonical segment IDs s0102 to s0120. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00002 70	SRC00070/t00180 0-002099	covered	Timestamp interval t001800-002099; canonical segment IDs s0121 to s0140. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00002 71	SRC00071/t00180 0-002099	covered	Timestamp interval t001800-002099; canonical segment IDs s0121 to s0140. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00002 72	SRC00072/t00180 0-002099	covered	Timestamp interval t001800-002099; canonical segment IDs s0121 to s0140. Sidecar variants compare at similarity 0.9998; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00002 73	SRC00073/t00180 0-002099	covered	Timestamp interval t001800-002099; canonical segment IDs s0121 to s0140. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00002 74	SRC00070/t00210 0-002399	covered	Timestamp interval t002100-002399; canonical segment IDs s0141 to s0158. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV00002 75	SRC00071/t00210 0-002399	covered	Timestamp interval t002100-002399; canonical segment IDs s0141 to s0158. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00002 76	SRC00072/t00210 0-002399	covered	Timestamp interval t002100-002399; canonical segment IDs s0141 to s0158. Sidecar variants compare at similarity 0.9998; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00002 77	SRC00073/t00210 0-002399	covered	Timestamp interval t002100-002399; canonical segment IDs s0141 to s0158. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00002 78	SRC00070/t00240 0-002699	covered	Timestamp interval t002400-002699; canonical segment IDs s0159 to s0178. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00002 79	SRC00071/t00240 0-002699	covered	Timestamp interval t002400-002699; canonical segment IDs s0159 to s0178. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00002 80	SRC00072/t00240 0-002699	covered	Timestamp interval t002400-002699; canonical segment IDs s0159 to s0178. Sidecar variants compare at similarity 0.9998; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00002 81	SRC00073/t00240 0-002699	covered	Timestamp interval t002400-002699; canonical segment IDs s0159 to s0178. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV0000282	SRC00070/t002700-002999	covered	Timestamp interval t002700-002999; canonical segment IDs s0179 to s0199. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000283	SRC00071/t002700-002999	covered	Timestamp interval t002700-002999; canonical segment IDs s0179 to s0199. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000284	SRC00072/t002700-002999	covered	Timestamp interval t002700-002999; canonical segment IDs s0179 to s0199. Sidecar variants compare at similarity 0.9998; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000285	SRC00073/t002700-002999	covered	Timestamp interval t002700-002999; canonical segment IDs s0179 to s0199. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000286	SRC00074/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0019. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000287	SRC00075/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0019. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000288	SRC00076/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0019. Sidecar variants compare at similarity 0.9994; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV0000289	SRC00077/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0019. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000290	SRC00074/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0020 to s0038. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000291	SRC00075/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0020 to s0038. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000292	SRC00076/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0020 to s0038. Sidecar variants compare at similarity 0.9994; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000293	SRC00077/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0020 to s0038. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000294	SRC00074/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0039 to s0056. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000295	SRC00075/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0039 to s0056. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV00002 96	SRC00076/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0039 to s0056. Sidecar variants compare at similarity 0.9994; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00002 97	SRC00077/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0039 to s0056. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00002 98	SRC00074/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0057 to s0075. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00002 99	SRC00075/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0057 to s0075. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00003 00	SRC00076/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0057 to s0075. Sidecar variants compare at similarity 0.9994; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00003 01	SRC00077/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0057 to s0075. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00003 02	SRC00078/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0018. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV00003 03	SRC00079/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0018. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00003 04	SRC00080/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0018. Sidecar variants compare at similarity 0.9995; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00003 05	SRC00081/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0018. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00003 06	SRC00078/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0019 to s0038. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00003 07	SRC00079/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0019 to s0038. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00003 08	SRC00080/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0019 to s0038. Sidecar variants compare at similarity 0.9995; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00003 09	SRC00081/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0019 to s0038. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV0000310	SRC00078/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0039 to s0054. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000311	SRC00079/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0039 to s0054. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000312	SRC00080/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0039 to s0054. Sidecar variants compare at similarity 0.9995; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000313	SRC00081/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0039 to s0054. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000314	SRC00078/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0055 to s0074. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000315	SRC00079/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0055 to s0074. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000316	SRC00080/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0055 to s0074. Sidecar variants compare at similarity 0.9995; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV00003317	SRC00081/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0055 to s0074. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00003318	SRC00078/t001200-001499	covered	Timestamp interval t001200-001499; canonical segment IDs s0075 to s0081. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00003319	SRC00079/t001200-001499	covered	Timestamp interval t001200-001499; canonical segment IDs s0075 to s0081. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00003320	SRC00080/t001200-001499	covered	Timestamp interval t001200-001499; canonical segment IDs s0075 to s0081. Sidecar variants compare at similarity 0.9995; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00003321	SRC00081/t001200-001499	covered	Timestamp interval t001200-001499; canonical segment IDs s0075 to s0081. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00003322	SRC00082/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00003323	SRC00083/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV00003 24	SRC00084/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare at similarity 0.9989; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00003 25	SRC00085/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00003 26	SRC00082/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0021 to s0029. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00003 27	SRC00083/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0021 to s0029. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00003 28	SRC00084/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0021 to s0029. Sidecar variants compare at similarity 0.9989; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00003 29	SRC00085/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0021 to s0029. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00003 30	SRC00086/p1	covered	Extracted slide page 1 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000331	SRC00088/p1	covered	Original slide page 1 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000332	SRC00087/p1	covered	Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human comparison is pending.
EV0000333	SRC00086/p2	covered	Extracted slide page 2 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000334	SRC00088/p2	covered	Original slide page 2 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000335	SRC00087/p2	covered	Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human comparison is pending.
EV0000336	SRC00086/p3	covered	Extracted slide page 3 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000337	SRC00088/p3	covered	Original slide page 3 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000338	SRC00087/p3	covered	Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human comparison is pending.
EV0000339	SRC00086/p4	covered	Extracted slide page 4 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000340	SRC00088/p4	covered	Original slide page 4 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000341	SRC00087/p4	covered	Alternate extracted slide text page 4; normalized text matches the .slides.md page. Human comparison is pending.
EV0000342	SRC00086/p5	covered	Extracted slide page 5 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000343	SRC00088/p5	covered	Original slide page 5 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000344	SRC00087/p5	covered	Alternate extracted slide text page 5; normalized text matches the .slides.md page. Human comparison is pending.
EV0000345	SRC00086/p6	covered	Extracted slide page 6 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000346	SRC00088/p6	covered	Original slide page 6 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000347	SRC00087/p6	covered	Alternate extracted slide text page 6; normalized text matches the .slides.md page. Human comparison is pending.
EV0000348	SRC00086/p7	covered	Extracted slide page 7 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000349	SRC00088/p7	covered	Original slide page 7 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000350	SRC00087/p7	covered	Alternate extracted slide text page 7; normalized text matches the .slides.md page. Human comparison is pending.
EV0000351	SRC00086/p8	covered	Extracted slide page 8 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000352	SRC00088/p8	covered	Original slide page 8 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000353	SRC00087/p8	covered	Alternate extracted slide text page 8; normalized text matches the .slides.md page. Human comparison is pending.
EV0000354	SRC00086/p9	covered	Extracted slide page 9 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000355	SRC00088/p9	covered	Original slide page 9 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000356	SRC00087/p9	covered	Alternate extracted slide text page 9; normalized text matches the .slides.md page. Human comparison is pending.
EV0000357	SRC00086/p10	covered	Extracted slide page 10 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000358	SRC00088/p10	covered	Original slide page 10 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000359	SRC00087/p10	covered	Alternate extracted slide text page 10; normalized text matches the .slides.md page. Human comparison is pending.
EV0000360	SRC00086/p11	covered	Extracted slide page 11 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000361	SRC00088/p11	covered	Original slide page 11 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000362	SRC00087/p11	covered	Alternate extracted slide text page 11; normalized text matches the .slides.md page. Human comparison is pending.
EV0000363	SRC00086/p12	covered	Extracted slide page 12 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000364	SRC00088/p12	covered	Original slide page 12 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000365	SRC00087/p12	covered	Alternate extracted slide text page 12; normalized text matches the .slides.md page. Human comparison is pending.
EV0000366	SRC00086/p13	covered	Extracted slide page 13 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000367	SRC00088/p13	covered	Original slide page 13 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000368	SRC00087/p13	covered	Alternate extracted slide text page 13; normalized text matches the .slides.md page. Human comparison is pending.
EV0000369	SRC00086/p14	covered	Extracted slide page 14 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000370	SRC00088/p14	covered	Original slide page 14 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000371	SRC00087/p14	covered	Alternate extracted slide text page 14; normalized text matches the .slides.md page. Human comparison is pending.
EV0000372	SRC00086/p15	covered	Extracted slide page 15 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000373	SRC00088/p15	covered	Original slide page 15 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000374	SRC00087/p15	covered	Alternate extracted slide text page 15; normalized text matches the .slides.md page. Human comparison is pending.
EV0000375	SRC00086/p16	covered	Extracted slide page 16 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000376	SRC00088/p16	covered	Original slide page 16 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000377	SRC00087/p16	covered	Alternate extracted slide text page 16; normalized text matches the .slides.md page. Human comparison is pending.
EV0000378	SRC00086/p17	covered	Extracted slide page 17 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV00003379	SRC00088/p17	covered	Original slide page 17 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.pdf; visual page included as vector figure. Human layout/content review is pending.
EV00003380	SRC00087/p17	covered	Alternate extracted slide text page 17; normalized text matches the .slides.md page. Human comparison is pending.
EV00003381	SRC00086/p18	covered	Extracted slide page 18 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV00003382	SRC00088/p18	covered	Original slide page 18 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.pdf; visual page included as vector figure. Human layout/content review is pending.
EV00003383	SRC00087/p18	covered	Alternate extracted slide text page 18; normalized text matches the .slides.md page. Human comparison is pending.
EV00003384	SRC00086/p19	covered	Extracted slide page 19 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV00003 85	SRC00088/p19	covered	Original slide page 19 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.pdf; visual page included as vector figure. Human layout/content review is pending.
EV00003 86	SRC00087/p19	covered	Alternate extracted slide text page 19; normalized text matches the .slides.md page. Human comparison is pending.
EV00003 87	SRC00086/p20	covered	Extracted slide page 20 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV00003 88	SRC00088/p20	covered	Original slide page 20 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.pdf; visual page included as vector figure. Human layout/content review is pending.
EV00003 89	SRC00087/p20	covered	Alternate extracted slide text page 20; normalized text matches the .slides.md page. Human comparison is pending.
EV00003 90	SRC00086/p21	covered	Extracted slide page 21 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000391	SRC00088/p21	covered	Original slide page 21 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000392	SRC00087/p21	covered	Alternate extracted slide text page 21; normalized text matches the .slides.md page. Human comparison is pending.
EV0000393	SRC00086/p22	covered	Extracted slide page 22 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000394	SRC00088/p22	covered	Original slide page 22 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000395	SRC00087/p22	covered	Alternate extracted slide text page 22; normalized text matches the .slides.md page. Human comparison is pending.
EV0000396	SRC00086/p23	covered	Extracted slide page 23 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000397	SRC00088/p23	covered	Original slide page 23 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000398	SRC00087/p23	covered	Alternate extracted slide text page 23; normalized text matches the .slides.md page. Human comparison is pending.
EV0000399	SRC00086/p24	covered	Extracted slide page 24 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000400	SRC00088/p24	covered	Original slide page 24 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000401	SRC00087/p24	covered	Alternate extracted slide text page 24; normalized text matches the .slides.md page. Human comparison is pending.
EV0000402	SRC00086/p25	covered	Extracted slide page 25 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000403	SRC00088/p25	covered	Original slide page 25 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000404	SRC00087/p25	covered	Alternate extracted slide text page 25; normalized text matches the .slides.md page. Human comparison is pending.
EV0000405	SRC00086/p26	covered	Extracted slide page 26 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000406	SRC00088/p26	covered	Original slide page 26 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000407	SRC00087/p26	covered	Alternate extracted slide text page 26; normalized text matches the .slides.md page. Human comparison is pending.
EV0000408	SRC00086/p27	covered	Extracted slide page 27 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000409	SRC00088/p27	covered	Original slide page 27 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000410	SRC00087/p27	covered	Alternate extracted slide text page 27; normalized text matches the .slides.md page. Human comparison is pending.
EV0000411	SRC00086/p28	covered	Extracted slide page 28 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000412	SRC00088/p28	covered	Original slide page 28 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000413	SRC00087/p28	covered	Alternate extracted slide text page 28; normalized text matches the .slides.md page. Human comparison is pending.
EV0000414	SRC00086/p29	covered	Extracted slide page 29 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000415	SRC00088/p29	covered	Original slide page 29 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000416	SRC00087/p29	covered	Alternate extracted slide text page 29; normalized text matches the .slides.md page. Human comparison is pending.
EV0000417	SRC00086/p30	covered	Extracted slide page 30 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000418	SRC00088/p30	covered	Original slide page 30 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000419	SRC00087/p30	covered	Alternate extracted slide text page 30; normalized text matches the .slides.md page. Human comparison is pending.
EV0000420	SRC00089/p1	covered	Extracted slide page 1 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__1__2_Intro_to_title__37baecfa2540966d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000421	SRC00091/p1	covered	Original slide page 1 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__1__2_Intro_to_title__37baecfa2540966d.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000422	SRC00090/p1	covered	Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human comparison is pending.
EV0000423	SRC00089/p2	covered	Extracted slide page 2 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__1__2_Intro_to_title__37baecfa2540966d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000424	SRC00091/p2	covered	Original slide page 2 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__1__2_Intro_to_title__37baecfa2540966d.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000425	SRC00090/p2	covered	Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human comparison is pending.
EV0000426	SRC00089/p3	covered	Extracted slide page 3 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__1__2_Intro_to_title__37baecfa2540966d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000427	SRC00091/p3	covered	Original slide page 3 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__1__2_Intro_to_title__37baecfa2540966d.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000428	SRC00090/p3	covered	Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human comparison is pending.
EV0000429	SRC00089/p4	covered	Extracted slide page 4 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__1__2_Intro_to_title__37baecfa2540966d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000430	SRC00091/p4	covered	Original slide page 4 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__1__2_Intro_to_title__37baecfa2540966d.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000431	SRC00090/p4	covered	Alternate extracted slide text page 4; normalized text matches the .slides.md page. Human comparison is pending.
EV0000432	SRC00089/p5	covered	Extracted slide page 5 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__1__2_Intro_to_title__37baecfa2540966d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000433	SRC00091/p5	covered	Original slide page 5 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__1__2_Intro_to_title__37baecfa2540966d.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000434	SRC00090/p5	covered	Alternate extracted slide text page 5; normalized text matches the .slides.md page. Human comparison is pending.
EV0000435	SRC00089/p6	covered	Extracted slide page 6 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__1__2_Intro_to_title__37baecfa2540966d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000436	SRC00091/p6	covered	Original slide page 6 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__1__2_Intro_to_title__37baecfa2540966d.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000437	SRC00090/p6	covered	Alternate extracted slide text page 6; normalized text matches the .slides.md page. Human comparison is pending.
EV0000438	SRC00089/p7	covered	Extracted slide page 7 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__1__2_Intro_to_title__37baecfa2540966d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000439	SRC00091/p7	covered	Original slide page 7 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__1__2_Intro_to_title__37baecfa2540966d.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000440	SRC00090/p7	covered	Alternate extracted slide text page 7; normalized text matches the .slides.md page. Human comparison is pending.
EV0000441	SRC00089/p8	covered	Extracted slide page 8 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__1__2_Intro_to_title__37baecfa2540966d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000442	SRC00091/p8	covered	Original slide page 8 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__1__2_Intro_to_title__37baecfa2540966d.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000443	SRC00090/p8	covered	Alternate extracted slide text page 8; normalized text matches the .slides.md page. Human comparison is pending.
EV0000444	SRC00089/p9	covered	Extracted slide page 9 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__1__2_Intro_to_title__37baecfa2540966d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000445	SRC00091/p9	covered	Original slide page 9 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__1__2_Intro_to_title__37baecfa2540966d.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000446	SRC00090/p9	covered	Alternate extracted slide text page 9; normalized text matches the .slides.md page. Human comparison is pending.
EV0000447	SRC00089/p10	covered	Extracted slide page 10 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__1__2_Intro_to_title__37baecfa2540966d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000448	SRC00091/p10	covered	Original slide page 10 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__1__2_Intro_to_title__37baecfa2540966d.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000449	SRC00090/p10	covered	Alternate extracted slide text page 10; normalized text matches the .slides.md page. Human comparison is pending.
EV0000450	SRC00089/p11	covered	Extracted slide page 11 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__1__2_Intro_to_title__37baecfa2540966d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000451	SRC00091/p11	covered	Original slide page 11 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__1__2_Intro_to_title__37baecfa2540966d.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000452	SRC00090/p11	covered	Alternate extracted slide text page 11; normalized text matches the .slides.md page. Human comparison is pending.
EV0000453	SRC00089/p12	covered	Extracted slide page 12 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__1__2_Intro_to_title__37baecfa2540966d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000454	SRC00091/p12	covered	Original slide page 12 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__1__2_Intro_to_title__37baecfa2540966d.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000455	SRC00090/p12	covered	Alternate extracted slide text page 12; normalized text matches the .slides.md page. Human comparison is pending.
EV0000456	SRC00089/p13	covered	Extracted slide page 13 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__1__2_Intro_to_title__37baecfa2540966d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000457	SRC00091/p13	covered	Original slide page 13 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__1__2_Intro_to_title__37baecfa2540966d.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000458	SRC00090/p13	covered	Alternate extracted slide text page 13; normalized text matches the .slides.md page. Human comparison is pending.
EV0000459	SRC00089/p14	covered	Extracted slide page 14 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__1__2_Intro_to_title__37baecfa2540966d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000460	SRC00091/p14	covered	Original slide page 14 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__1__2_Intro_to_title__37baecfa2540966d.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000461	SRC00090/p14	covered	Alternate extracted slide text page 14; normalized text matches the .slides.md page. Human comparison is pending.
EV0000462	SRC00089/p15	covered	Extracted slide page 15 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__1__2_Intro_to_title__37baecfa2540966d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000463	SRC00091/p15	covered	Original slide page 15 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__1__2_Intro_to_title__37baecfa2540966d.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000464	SRC00090/p15	covered	Alternate extracted slide text page 15; normalized text matches the .slides.md page. Human comparison is pending.
EV0000465	SRC00089/p16	covered	Extracted slide page 16 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__1__2_Intro_to_title__37baecfa2540966d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000466	SRC00091/p16	covered	Original slide page 16 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__1__2_Intro_to_title__37baecfa2540966d.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000467	SRC00090/p16	covered	Alternate extracted slide text page 16; normalized text matches the .slides.md page. Human comparison is pending.
EV0000468	SRC00089/p17	covered	Extracted slide page 17 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__1__2_Intro_to_title__37baecfa2540966d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000469	SRC00091/p17	covered	Original slide page 17 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__1__2_Intro_to_title__37baecfa2540966d.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000470	SRC00090/p17	covered	Alternate extracted slide text page 17; normalized text matches the .slides.md page. Human comparison is pending.
EV0000471	SRC00089/p18	covered	Extracted slide page 18 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__1__2_Intro_to_title__37baecfa2540966d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000472	SRC00091/p18	covered	Original slide page 18 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__1__2_Intro_to_title__37baecfa2540966d.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000473	SRC00090/p18	covered	Alternate extracted slide text page 18; normalized text matches the .slides.md page. Human comparison is pending.
EV0000474	SRC00092/p1	covered	Extracted slide page 1 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000475	SRC00094/p1	covered	Original slide page 1 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000476	SRC00093/p1	covered	Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human comparison is pending.
EV0000477	SRC00092/p2	covered	Extracted slide page 2 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000478	SRC00094/p2	covered	Original slide page 2 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000479	SRC00093/p2	covered	Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human comparison is pending.
EV0000480	SRC00092/p3	covered	Extracted slide page 3 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV00004 81	SRC00094/p3	covered	Original slide page 3 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.pdf; visual page included as vector figure. Human layout/content review is pending.
EV00004 82	SRC00093/p3	covered	Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human comparison is pending.
EV00004 83	SRC00092/p4	covered	Extracted slide page 4 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV00004 84	SRC00094/p4	covered	Original slide page 4 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.pdf; visual page included as vector figure. Human layout/content review is pending.
EV00004 85	SRC00093/p4	covered	Alternate extracted slide text page 4; normalized text matches the .slides.md page. Human comparison is pending.
EV00004 86	SRC00092/p5	covered	Extracted slide page 5 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000487	SRC00094/p5	covered	Original slide page 5 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000488	SRC00093/p5	covered	Alternate extracted slide text page 5; normalized text matches the .slides.md page. Human comparison is pending.
EV0000489	SRC00092/p6	covered	Extracted slide page 6 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000490	SRC00094/p6	covered	Original slide page 6 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000491	SRC00093/p6	covered	Alternate extracted slide text page 6; normalized text matches the .slides.md page. Human comparison is pending.
EV0000492	SRC00092/p7	covered	Extracted slide page 7 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000493	SRC00094/p7	covered	Original slide page 7 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000494	SRC00093/p7	covered	Alternate extracted slide text page 7; normalized text matches the .slides.md page. Human comparison is pending.
EV0000495	SRC00092/p8	covered	Extracted slide page 8 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000496	SRC00094/p8	covered	Original slide page 8 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000497	SRC00093/p8	covered	Alternate extracted slide text page 8; normalized text matches the .slides.md page. Human comparison is pending.
EV0000498	SRC00092/p9	covered	Extracted slide page 9 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000499	SRC00094/p9	covered	Original slide page 9 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000500	SRC00093/p9	covered	Alternate extracted slide text page 9; normalized text matches the .slides.md page. Human comparison is pending.
EV0000501	SRC00092/p10	covered	Extracted slide page 10 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000502	SRC00094/p10	covered	Original slide page 10 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000503	SRC00093/p10	covered	Alternate extracted slide text page 10; normalized text matches the .slides.md page. Human comparison is pending.
EV0000504	SRC00092/p11	covered	Extracted slide page 11 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000505	SRC00094/p11	covered	Original slide page 11 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000506	SRC00093/p11	covered	Alternate extracted slide text page 11; normalized text matches the .slides.md page. Human comparison is pending.
EV0000507	SRC00092/p12	covered	Extracted slide page 12 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000508	SRC00094/p12	covered	Original slide page 12 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000509	SRC00093/p12	covered	Alternate extracted slide text page 12; normalized text matches the .slides.md page. Human comparison is pending.
EV0000510	SRC00092/p13	covered	Extracted slide page 13 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000511	SRC00094/p13	covered	Original slide page 13 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000512	SRC00093/p13	covered	Alternate extracted slide text page 13; normalized text matches the .slides.md page. Human comparison is pending.
EV0000513	SRC00092/p14	covered	Extracted slide page 14 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000514	SRC00094/p14	covered	Original slide page 14 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000515	SRC00093/p14	covered	Alternate extracted slide text page 14; normalized text matches the .slides.md page. Human comparison is pending.
EV0000516	SRC00092/p15	covered	Extracted slide page 15 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000517	SRC00094/p15	covered	Original slide page 15 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000518	SRC00093/p15	covered	Alternate extracted slide text page 15; normalized text matches the .slides.md page. Human comparison is pending.
EV0000519	SRC00092/p16	covered	Extracted slide page 16 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000520	SRC00094/p16	covered	Original slide page 16 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000521	SRC00093/p16	covered	Alternate extracted slide text page 16; normalized text matches the .slides.md page. Human comparison is pending.
EV0000522	SRC00092/p17	covered	Extracted slide page 17 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000523	SRC00094/p17	covered	Original slide page 17 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000524	SRC00093/p17	covered	Alternate extracted slide text page 17; normalized text matches the .slides.md page. Human comparison is pending.
EV0000525	SRC00092/p18	covered	Extracted slide page 18 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000526	SRC00094/p18	covered	Original slide page 18 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000527	SRC00093/p18	covered	Alternate extracted slide text page 18; normalized text matches the .slides.md page. Human comparison is pending.
EV0000528	SRC00092/p19	covered	Extracted slide page 19 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000529	SRC00094/p19	covered	Original slide page 19 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000530	SRC00093/p19	covered	Alternate extracted slide text page 19; normalized text matches the .slides.md page. Human comparison is pending.
EV0000531	SRC00092/p20	covered	Extracted slide page 20 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000532	SRC00094/p20	covered	Original slide page 20 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000533	SRC00093/p20	covered	Alternate extracted slide text page 20; normalized text matches the .slides.md page. Human comparison is pending.
EV0000534	SRC00092/p21	covered	Extracted slide page 21 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000535	SRC00094/p21	covered	Original slide page 21 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000536	SRC00093/p21	covered	Alternate extracted slide text page 21; normalized text matches the .slides.md page. Human comparison is pending.
EV0000537	SRC00092/p22	covered	Extracted slide page 22 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000538	SRC00094/p22	covered	Original slide page 22 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000539	SRC00093/p22	covered	Alternate extracted slide text page 22; normalized text matches the .slides.md page. Human comparison is pending.
EV0000540	SRC00092/p23	covered	Extracted slide page 23 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000541	SRC00094/p23	covered	Original slide page 23 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000542	SRC00093/p23	covered	Alternate extracted slide text page 23; normalized text matches the .slides.md page. Human comparison is pending.
EV0000543	SRC00092/p24	covered	Extracted slide page 24 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000544	SRC00094/p24	covered	Original slide page 24 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000545	SRC00093/p24	covered	Alternate extracted slide text page 24; normalized text matches the .slides.md page. Human comparison is pending.
EV0000546	SRC00092/p25	covered	Extracted slide page 25 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000547	SRC00094/p25	covered	Original slide page 25 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000548	SRC00093/p25	covered	Alternate extracted slide text page 25; normalized text matches the .slides.md page. Human comparison is pending.
EV0000549	SRC00092/p26	covered	Extracted slide page 26 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000550	SRC00094/p26	covered	Original slide page 26 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000551	SRC00093/p26	covered	Alternate extracted slide text page 26; normalized text matches the .slides.md page. Human comparison is pending.
EV0000552	SRC00092/p27	covered	Extracted slide page 27 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000553	SRC00094/p27	covered	Original slide page 27 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000554	SRC00093/p27	covered	Alternate extracted slide text page 27; normalized text matches the .slides.md page. Human comparison is pending.
EV0000555	SRC00095/p1	covered	Extracted slide page 1 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/003_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__3_Process_of_CPS__3f3254779c6a452a.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000556	SRC00097/p1	covered	Original slide page 1 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/003_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__3_Process_of_CPS__3f3254779c6a452a.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000557	SRC00096/p1	covered	Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human comparison is pending.
EV0000558	SRC00095/p2	covered	Extracted slide page 2 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/003_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__3_Process_of_CPS__3f3254779c6a452a.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000559	SRC00097/p2	covered	Original slide page 2 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/003_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__3_Process_of_CPS__3f3254779c6a452a.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000560	SRC00096/p2	covered	Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human comparison is pending.
EV0000561	SRC00095/p3	covered	Extracted slide page 3 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/003_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__3_Process_of_CPS__3f3254779c6a452a.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000562	SRC00097/p3	covered	Original slide page 3 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/003_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__3_Process_of_CPS__3f3254779c6a452a.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000563	SRC00096/p3	covered	Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human comparison is pending.
EV0000564	SRC00095/p4	covered	Extracted slide page 4 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/003_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__3_Process_of_CPS__3f3254779c6a452a.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000565	SRC00097/p4	covered	Original slide page 4 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/003_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__3_Process_of_CPS__3f3254779c6a452a.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000566	SRC00096/p4	covered	Alternate extracted slide text page 4; normalized text matches the .slides.md page. Human comparison is pending.
EV0000567	SRC00095/p5	covered	Extracted slide page 5 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/003_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__3_Process_of_CPS__3f3254779c6a452a.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000568	SRC00097/p5	covered	Original slide page 5 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/003_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__3_Process_of_CPS__3f3254779c6a452a.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000569	SRC00096/p5	covered	Alternate extracted slide text page 5; normalized text matches the .slides.md page. Human comparison is pending.
EV0000570	SRC00095/p6	covered	Extracted slide page 6 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/003_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__3_Process_of_CPS__3f3254779c6a452a.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000571	SRC00097/p6	covered	Original slide page 6 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/003_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__3_Process_of_CPS__3f3254779c6a452a.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000572	SRC00096/p6	covered	Alternate extracted slide text page 6; normalized text matches the .slides.md page. Human comparison is pending.
EV0000573	SRC00095/p7	covered	Extracted slide page 7 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/003_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__3_Process_of_CPS__3f3254779c6a452a.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000574	SRC00097/p7	covered	Original slide page 7 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/003_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__3_Process_of_CPS__3f3254779c6a452a.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000575	SRC00096/p7	covered	Alternate extracted slide text page 7; normalized text matches the .slides.md page. Human comparison is pending.
EV0000576	SRC00095/p8	covered	Extracted slide page 8 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/003_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__3_Process_of_CPS__3f3254779c6a452a.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000577	SRC00097/p8	covered	Original slide page 8 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/003_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__3_Process_of_CPS__3f3254779c6a452a.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000578	SRC00096/p8	covered	Alternate extracted slide text page 8; normalized text matches the .slides.md page. Human comparison is pending.
EV0000579	SRC00095/p9	covered	Extracted slide page 9 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/003_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__3_Process_of_CPS__3f3254779c6a452a.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000580	SRC00097/p9	covered	Original slide page 9 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/003_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__3_Process_of_CPS__3f3254779c6a452a.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000581	SRC00096/p9	covered	Alternate extracted slide text page 9; normalized text matches the .slides.md page. Human comparison is pending.
EV0000582	SRC00095/p10	covered	Extracted slide page 10 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/003_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__3_Process_of_CPS__3f3254779c6a452a.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000583	SRC00097/p10	covered	Original slide page 10 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/003_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__3_Process_of_CPS__3f3254779c6a452a.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000584	SRC00096/p10	covered	Alternate extracted slide text page 10; normalized text matches the .slides.md page. Human comparison is pending.
EV0000585	SRC00095/p11	covered	Extracted slide page 11 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/003_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__3_Process_of_CPS__3f3254779c6a452a.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000586	SRC00097/p11	covered	Original slide page 11 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/003_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__3_Process_of_CPS__3f3254779c6a452a.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000587	SRC00096/p11	covered	Alternate extracted slide text page 11; normalized text matches the .slides.md page. Human comparison is pending.
EV0000588	SRC00095/p12	covered	Extracted slide page 12 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/003_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__3_Process_of_CPS__3f3254779c6a452a.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000589	SRC00097/p12	covered	Original slide page 12 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/003_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__3_Process_of_CPS__3f3254779c6a452a.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000590	SRC00096/p12	covered	Alternate extracted slide text page 12; normalized text matches the .slides.md page. Human comparison is pending.
EV0000591	SRC00098/p1	covered	Extracted slide page 1 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/004_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__4_5_Computer_Hardware_software__57579a8e9ad25cbb.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000592	SRC00100/p1	covered	Original slide page 1 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/004_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__4_5_Computer_Hardware_software__57579a8e9ad25cbb.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000593	SRC00099/p1	covered	Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human comparison is pending.
EV0000594	SRC00098/p2	covered	Extracted slide page 2 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/004_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__4_5_Computer_Hardware_software__57579a8e9ad25cbb.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000595	SRC00100/p2	covered	Original slide page 2 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/004_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__4_5_Computer_Hardware_software__57579a8e9ad25cbb.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000596	SRC00099/p2	covered	Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human comparison is pending.
EV0000597	SRC00098/p3	covered	Extracted slide page 3 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/004_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__4_5_Computer_Hardware_software__57579a8e9ad25cbb.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000598	SRC00100/p3	covered	Original slide page 3 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/004_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__4_5_Computer_Hardware_software__57579a8e9ad25cbb.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000599	SRC00099/p3	covered	Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human comparison is pending.
EV0000600	SRC00098/p4	covered	Extracted slide page 4 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/004_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__4_5_Computer_Hardware_software__57579a8e9ad25cbb.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000601	SRC00100/p4	covered	Original slide page 4 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/004_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__4_5_Computer_Hardware_software__57579a8e9ad25cbb.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000602	SRC00099/p4	covered	Alternate extracted slide text page 4; normalized text matches the .slides.md page. Human comparison is pending.
EV0000603	SRC00098/p5	covered	Extracted slide page 5 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/004_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__4_5_Computer_Hardware_software__57579a8e9ad25cbb.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000604	SRC00100/p5	covered	Original slide page 5 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/004_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__4_5_Computer_Hardware_software__57579a8e9ad25cbb.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000605	SRC00099/p5	covered	Alternate extracted slide text page 5; normalized text matches the .slides.md page. Human comparison is pending.
EV0000606	SRC00098/p6	covered	Extracted slide page 6 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/004_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__4_5_Computer_Hardware_software__57579a8e9ad25cbb.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000607	SRC00100/p6	covered	Original slide page 6 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/004_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__4_5_Computer_Hardware_software__57579a8e9ad25cbb.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000608	SRC00099/p6	covered	Alternate extracted slide text page 6; normalized text matches the .slides.md page. Human comparison is pending.
EV0000609	SRC00098/p7	covered	Extracted slide page 7 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/004_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__4_5_Computer_Hardware_software__57579a8e9ad25cbb.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000610	SRC00100/p7	covered	Original slide page 7 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/004_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__4_5_Computer_Hardware_software__57579a8e9ad25cbb.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000611	SRC00099/p7	covered	Alternate extracted slide text page 7; normalized text matches the .slides.md page. Human comparison is pending.
EV0000612	SRC00098/p8	covered	Extracted slide page 8 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/004_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__4_5_Computer_Hardware_software__57579a8e9ad25cbb.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000613	SRC00100/p8	covered	Original slide page 8 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/004_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__4_5_Computer_Hardware_software__57579a8e9ad25cbb.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000614	SRC00099/p8	covered	Alternate extracted slide text page 8; normalized text matches the .slides.md page. Human comparison is pending.
EV0000615	SRC00098/p9	covered	Extracted slide page 9 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/004_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__4_5_Computer_Hardware_software__57579a8e9ad25cbb.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000616	SRC00100/p9	covered	Original slide page 9 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/004_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__4_5_Computer_Hardware_software__57579a8e9ad25cbb.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000617	SRC00099/p9	covered	Alternate extracted slide text page 9; normalized text matches the .slides.md page. Human comparison is pending.
EV0000618	SRC00098/p10	covered	Extracted slide page 10 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/004_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__4_5_Computer_Hardware_software__57579a8e9ad25cbb.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000619	SRC00100/p10	covered	Original slide page 10 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/004_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__4_5_Computer_Hardware_software__57579a8e9ad25cbb.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000620	SRC00099/p10	covered	Alternate extracted slide text page 10; normalized text matches the .slides.md page. Human comparison is pending.
EV0000621	SRC00098/p11	covered	Extracted slide page 11 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/004_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__4_5_Computer_Hardware_software__57579a8e9ad25cbb.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000622	SRC00100/p11	covered	Original slide page 11 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/004_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__4_5_Computer_Hardware_software__57579a8e9ad25cbb.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000623	SRC00099/p11	covered	Alternate extracted slide text page 11; normalized text matches the .slides.md page. Human comparison is pending.
EV0000624	SRC00098/p12	covered	Extracted slide page 12 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/004_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__4_5_Computer_Hardware_software__57579a8e9ad25cbb.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000625	SRC00100/p12	covered	Original slide page 12 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/004_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__4_5_Computer_Hardware_software__57579a8e9ad25cbb.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000626	SRC00099/p12	covered	Alternate extracted slide text page 12; normalized text matches the .slides.md page. Human comparison is pending.
EV0000627	SRC00098/p13	covered	Extracted slide page 13 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/004_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__4_5_Computer_Hardware_software__57579a8e9ad25cbb.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000628	SRC00100/p13	covered	Original slide page 13 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/004_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__4_5_Computer_Hardware_software__57579a8e9ad25cbb.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000629	SRC00099/p13	covered	Alternate extracted slide text page 13; normalized text matches the .slides.md page. Human comparison is pending.
EV0000630	SRC00101/p1	covered	Extracted slide page 1 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/006_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__6_Intro_to_Programming_Languages__0cbeda79209c1dbf.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000631	SRC00103/p1	covered	Original slide page 1 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/006_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__6_Intro_to_Programming_Languages__0cbeda79209c1dbf.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000632	SRC00102/p1	covered	Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human comparison is pending.
EV0000633	SRC00101/p2	covered	Extracted slide page 2 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/006_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__6_Intro_to_Programming_Languages__0cbeda79209c1dbf.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000634	SRC00103/p2	covered	Original slide page 2 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/006_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__6_Intro_to_Programming_Languages__0cbeda79209c1dbf.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000635	SRC00102/p2	covered	Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human comparison is pending.
EV0000636	SRC00101/p3	covered	Extracted slide page 3 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/006_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__6_Intro_to_Programming_Languages__0cbeda79209c1dbf.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000637	SRC00103/p3	covered	Original slide page 3 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/006_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__6_Intro_to_Programming_Languages__0cbeda79209c1dbf.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000638	SRC00102/p3	covered	Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human comparison is pending.
EV0000639	SRC00101/p4	covered	Extracted slide page 4 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/006_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__6_Intro_to_Programming_Languages__0cbeda79209c1dbf.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000640	SRC00103/p4	covered	Original slide page 4 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/006_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__6_Intro_to_Programming_Languages__0cbeda79209c1dbf.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000641	SRC00102/p4	covered	Alternate extracted slide text page 4; normalized text matches the .slides.md page. Human comparison is pending.
EV0000642	SRC00101/p5	covered	Extracted slide page 5 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/006_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__6_Intro_to_Programming_Languages__0cbeda79209c1dbf.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000643	SRC00103/p5	covered	Original slide page 5 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/006_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__6_Intro_to_Programming_Languages__0cbeda79209c1dbf.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000644	SRC00102/p5	covered	Alternate extracted slide text page 5; normalized text matches the .slides.md page. Human comparison is pending.
EV0000645	SRC00101/p6	covered	Extracted slide page 6 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/006_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__6_Intro_to_Programming_Languages__0cbeda79209c1dbf.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000646	SRC00103/p6	covered	Original slide page 6 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/006_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__6_Intro_to_Programming_Languages__0cbeda79209c1dbf.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000647	SRC00102/p6	covered	Alternate extracted slide text page 6; normalized text matches the .slides.md page. Human comparison is pending.
EV0000648	SRC00101/p7	covered	Extracted slide page 7 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/006_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__6_Intro_to_Programming_Languages__0cbeda79209c1dbf.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000649	SRC00103/p7	covered	Original slide page 7 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/006_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__6_Intro_to_Programming_Languages__0cbeda79209c1dbf.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000650	SRC00102/p7	covered	Alternate extracted slide text page 7; normalized text matches the .slides.md page. Human comparison is pending.
EV0000651	SRC00101/p8	covered	Extracted slide page 8 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/006_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__6_Intro_to_Programming_Languages__0cbeda79209c1dbf.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000652	SRC00103/p8	covered	Original slide page 8 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/006_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__6_Intro_to_Programming_Languages__0cbeda79209c1dbf.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000653	SRC00102/p8	covered	Alternate extracted slide text page 8; normalized text matches the .slides.md page. Human comparison is pending.
EV0000654	SRC00101/p9	covered	Extracted slide page 9 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/006_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__6_Intro_to_Programming_Languages__0cbeda79209c1dbf.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000655	SRC00103/p9	covered	Original slide page 9 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/006_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__6_Intro_to_Programming_Languages__0cbeda79209c1dbf.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000656	SRC00102/p9	covered	Alternate extracted slide text page 9; normalized text matches the .slides.md page. Human comparison is pending.
EV0000657	SRC00101/p10	covered	Extracted slide page 10 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/006_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__6_Intro_to_Programming_Languages__0cbeda79209c1dbf.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000658	SRC00103/p10	covered	Original slide page 10 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/006_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__6_Intro_to_Programming_Languages__0cbeda79209c1dbf.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000659	SRC00102/p10	covered	Alternate extracted slide text page 10; normalized text matches the .slides.md page. Human comparison is pending.
EV0000660	SRC00104/p1	covered	Extracted slide page 1 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/007_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__7_Syntax_semantics_translation_models__656e0afe2d8d3282.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000661	SRC00106/p1	covered	Original slide page 1 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/007_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__7_Syntax_semantics_translation_models__656e0afe2d8d3282.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000662	SRC00105/p1	covered	Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human comparison is pending.
EV0000663	SRC00104/p2	covered	Extracted slide page 2 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/007_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__7_Syntax_semantics_translation_models__656e0afe2d8d3282.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000664	SRC00106/p2	covered	Original slide page 2 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/007_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__7_Syntax_semantics_translation_models__656e0afe2d8d3282.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000665	SRC00105/p2	covered	Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human comparison is pending.
EV0000666	SRC00104/p3	covered	Extracted slide page 3 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/007_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__7_Syntax_semantics_translation_models__656e0afe2d8d3282.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000667	SRC00106/p3	covered	Original slide page 3 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/007_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__7_Syntax_semantics_translation_models__656e0afe2d8d3282.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000668	SRC00105/p3	covered	Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human comparison is pending.
EV0000669	SRC00104/p4	covered	Extracted slide page 4 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/007_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__7_Syntax_semantics_translation_models__656e0afe2d8d3282.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000670	SRC00106/p4	covered	Original slide page 4 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/007_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__7_Syntax_semantics_translation_models__656e0afe2d8d3282.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000671	SRC00105/p4	covered	Alternate extracted slide text page 4; normalized text matches the .slides.md page. Human comparison is pending.
EV0000672	SRC00104/p5	covered	Extracted slide page 5 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/007_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__7_Syntax_semantics_translation_models__656e0afe2d8d3282.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000673	SRC00106/p5	covered	Original slide page 5 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/007_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__7_Syntax_semantics_translation_models__656e0afe2d8d3282.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000674	SRC00105/p5	covered	Alternate extracted slide text page 5; normalized text matches the .slides.md page. Human comparison is pending.
EV0000675	SRC00104/p6	covered	Extracted slide page 6 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/007_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__7_Syntax_semantics_translation_models__656e0afe2d8d3282.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000676	SRC00106/p6	covered	Original slide page 6 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/007_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__7_Syntax_semantics_translation_models__656e0afe2d8d3282.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000677	SRC00105/p6	covered	Alternate extracted slide text page 6; normalized text matches the .slides.md page. Human comparison is pending.
EV0000678	SRC00104/p7	covered	Extracted slide page 7 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/007_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__7_Syntax_semantics_translation_models__656e0afe2d8d3282.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000679	SRC00106/p7	covered	Original slide page 7 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/007_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__7_Syntax_semantics_translation_models__656e0afe2d8d3282.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000680	SRC00105/p7	covered	Alternate extracted slide text page 7; normalized text matches the .slides.md page. Human comparison is pending.
EV0000681	SRC00104/p8	covered	Extracted slide page 8 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/007_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__7_Syntax_semantics_translation_models__656e0afe2d8d3282.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000682	SRC00106/p8	covered	Original slide page 8 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/007_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__7_Syntax_semantics_translation_models__656e0afe2d8d3282.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000683	SRC00105/p8	covered	Alternate extracted slide text page 8; normalized text matches the .slides.md page. Human comparison is pending.
EV0000684	SRC00107/p1	covered	Extracted slide page 1 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/008_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__8_Installation_and_execution__2fbbee57dfe57679.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000685	SRC00109/p1	covered	Original slide page 1 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/008_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__8_Installation_and_execution__2fbbee57dfe57679.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000686	SRC00108/p1	covered	Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human comparison is pending.
EV0000687	SRC00107/p2	covered	Extracted slide page 2 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/008_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__8_Installation_and_execution__2fbbee57dfe57679.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000688	SRC00109/p2	covered	Original slide page 2 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/008_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__8_Installation_and_execution__2fbbee57dfe57679.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000689	SRC00108/p2	covered	Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human comparison is pending.
EV0000690	SRC00107/p3	covered	Extracted slide page 3 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/008_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__8_Installation_and_execution__2fbbee57dfe57679.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000691	SRC00109/p3	covered	Original slide page 3 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/008_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__8_Installation_and_execution__2fbbee57dfe57679.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000692	SRC00108/p3	covered	Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human comparison is pending.
EV0000693	SRC00107/p4	covered	Extracted slide page 4 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/008_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__8_Installation_and_execution__2fbbee57dfe57679.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000694	SRC00109/p4	covered	Original slide page 4 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/008_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__8_Installation_and_execution__2fbbee57dfe57679.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000695	SRC00108/p4	covered	Alternate extracted slide text page 4; normalized text matches the .slides.md page. Human comparison is pending.
EV0000696	SRC00110/p1	covered	Extracted slide page 1 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/008_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__8_Program_structure__ed4abb0db838da93.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000697	SRC00112/p1	covered	Original slide page 1 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/008_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__8_Program_structure__ed4abb0db838da93.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000698	SRC00111/p1	covered	Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human comparison is pending.
EV0000699	SRC00110/p2	covered	Extracted slide page 2 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/008_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__8_Program_structure__ed4abb0db838da93.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000700	SRC00112/p2	covered	Original slide page 2 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/008_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__8_Program_structure__ed4abb0db838da93.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000701	SRC00111/p2	covered	Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human comparison is pending.
EV0000702	SRC00110/p3	covered	Extracted slide page 3 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/008_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__8_Program_structure__ed4abb0db838da93.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000703	SRC00112/p3	covered	Original slide page 3 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/008_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__8_Program_structure__ed4abb0db838da93.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000704	SRC00111/p3	covered	Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human comparison is pending.
EV0000705	SRC00110/p4	covered	Extracted slide page 4 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/008_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__8_Program_structure__ed4abb0db838da93.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000706	SRC00112/p4	covered	Original slide page 4 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/008_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__8_Program_structure__ed4abb0db838da93.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000707	SRC00111/p4	covered	Alternate extracted slide text page 4; normalized text matches the .slides.md page. Human comparison is pending.
EV0000708	SRC00110/p5	covered	Extracted slide page 5 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/008_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__8_Program_structure__ed4abb0db838da93.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000709	SRC00112/p5	covered	Original slide page 5 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/008_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__8_Program_structure__ed4abb0db838da93.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000710	SRC00111/p5	covered	Alternate extracted slide text page 5; normalized text matches the .slides.md page. Human comparison is pending.
EV0000711	SRC00110/p6	covered	Extracted slide page 6 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/008_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__8_Program_structure__ed4abb0db838da93.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000712	SRC00112/p6	covered	Original slide page 6 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/008_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__8_Program_structure__ed4abb0db838da93.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000713	SRC00111/p6	covered	Alternate extracted slide text page 6; normalized text matches the .slides.md page. Human comparison is pending.
EV0000714	SRC00113/p1	covered	Extracted slide page 1 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000715	SRC00115/p1	covered	Original slide page 1 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000716	SRC00114/p1	covered	Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human comparison is pending.
EV0000717	SRC00113/p2	covered	Extracted slide page 2 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000718	SRC00115/p2	covered	Original slide page 2 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000719	SRC00114/p2	covered	Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human comparison is pending.
EV0000720	SRC00113/p3	covered	Extracted slide page 3 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000721	SRC00115/p3	covered	Original slide page 3 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.pdf; visual page included as vector figure. Human layout/content review is pending.

Evidence	Source unit	Status	Decision
EV0000722	SRC00114/p3	covered	Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human comparison is pending.
EV0000723	SRC00113/p4	covered	Extracted slide page 4 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000724	SRC00115/p4	covered	Original slide page 4 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000725	SRC00114/p4	covered	Alternate extracted slide text page 4; normalized text matches the .slides.md page. Human comparison is pending.
EV0000726	SRC00113/p5	covered	Extracted slide page 5 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000727	SRC00115/p5	covered	Original slide page 5 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000728	SRC00114/p5	covered	Alternate extracted slide text page 5; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000729	SRC00113/p6	covered	Extracted slide page 6 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000730	SRC00115/p6	covered	Original slide page 6 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000731	SRC00114/p6	covered	Alternate extracted slide text page 6; normalized text matches the .slides.md page. Human comparison is pending.
EV0000732	SRC00113/p7	covered	Extracted slide page 7 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000733	SRC00115/p7	covered	Original slide page 7 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000734	SRC00114/p7	covered	Alternate extracted slide text page 7; normalized text matches the .slides.md page. Human comparison is pending.
EV0000735	SRC00113/p8	covered	Extracted slide page 8 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000736	SRC00115/p8	covered	Original slide page 8 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000737	SRC00114/p8	covered	Alternate extracted slide text page 8; normalized text matches the .slides.md page. Human comparison is pending.
EV0000738	SRC00113/p9	covered	Extracted slide page 9 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000739	SRC00115/p9	covered	Original slide page 9 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000740	SRC00114/p9	covered	Alternate extracted slide text page 9; normalized text matches the .slides.md page. Human comparison is pending.
EV0000741	SRC00113/p10	covered	Extracted slide page 10 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000742	SRC00115/p10	covered	Original slide page 10 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.pdf; visual page included as vector figure. Human layout/content review is pending.

Evidence	Source unit	Status	Decision
EV0000743	SRC00114/p10	covered	Alternate extracted slide text page 10; normalized text matches the .slides.md page. Human comparison is pending.
EV0000744	SRC00113/p11	covered	Extracted slide page 11 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000745	SRC00115/p11	covered	Original slide page 11 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000746	SRC00114/p11	covered	Alternate extracted slide text page 11; normalized text matches the .slides.md page. Human comparison is pending.
EV0000747	SRC00113/p12	covered	Extracted slide page 12 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000748	SRC00115/p12	covered	Original slide page 12 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000749	SRC00114/p12	covered	Alternate extracted slide text page 12; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000750	SRC00113/p13	covered	Extracted slide page 13 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000751	SRC00115/p13	covered	Original slide page 13 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000752	SRC00114/p13	covered	Alternate extracted slide text page 13; normalized text matches the .slides.md page. Human comparison is pending.
EV0000753	SRC00113/p14	covered	Extracted slide page 14 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000754	SRC00115/p14	covered	Original slide page 14 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000755	SRC00114/p14	covered	Alternate extracted slide text page 14; normalized text matches the .slides.md page. Human comparison is pending.
EV0000756	SRC00113/p15	covered	Extracted slide page 15 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000757	SRC00115/p15	covered	Original slide page 15 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000758	SRC00114/p15	covered	Alternate extracted slide text page 15; normalized text matches the .slides.md page. Human comparison is pending.
EV0000759	SRC00113/p16	covered	Extracted slide page 16 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000760	SRC00115/p16	covered	Original slide page 16 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000761	SRC00114/p16	covered	Alternate extracted slide text page 16; normalized text matches the .slides.md page. Human comparison is pending.
EV0000762	SRC00113/p17	covered	Extracted slide page 17 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000763	SRC00115/p17	covered	Original slide page 17 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.pdf; visual page included as vector figure. Human layout/content review is pending.

Evidence	Source unit	Status	Decision
EV0000764	SRC00114/p17	covered	Alternate extracted slide text page 17; normalized text matches the .slides.md page. Human comparison is pending.
EV0000765	SRC00113/p18	covered	Extracted slide page 18 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000766	SRC00115/p18	covered	Original slide page 18 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000767	SRC00114/p18	covered	Alternate extracted slide text page 18; normalized text matches the .slides.md page. Human comparison is pending.
EV0000768	SRC00113/p19	covered	Extracted slide page 19 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000769	SRC00115/p19	covered	Original slide page 19 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000770	SRC00114/p19	covered	Alternate extracted slide text page 19; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000771	SRC00113/p20	covered	Extracted slide page 20 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000772	SRC00115/p20	covered	Original slide page 20 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000773	SRC00114/p20	covered	Alternate extracted slide text page 20; normalized text matches the .slides.md page. Human comparison is pending.
EV0000774	SRC00113/p21	covered	Extracted slide page 21 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000775	SRC00115/p21	covered	Original slide page 21 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000776	SRC00114/p21	covered	Alternate extracted slide text page 21; normalized text matches the .slides.md page. Human comparison is pending.
EV0000777	SRC00113/p22	covered	Extracted slide page 22 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.

Evidence	Source unit	Status	Decision
EV0000778	SRC00115/p22	covered	Original slide page 22 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000779	SRC00114/p22	covered	Alternate extracted slide text page 22; normalized text matches the .slides.md page. Human comparison is pending.
EV0000780	SRC00113/p23	covered	Extracted slide page 23 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000781	SRC00115/p23	covered	Original slide page 23 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000782	SRC00114/p23	covered	Alternate extracted slide text page 23; normalized text matches the .slides.md page. Human comparison is pending.
EV0000783	SRC00113/p24	covered	Extracted slide page 24 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000784	SRC00115/p24	covered	Original slide page 24 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.pdf; visual page included as vector figure. Human layout/content review is pending.

Evidence	Source unit	Status	Decision
EV0000785	SRC00114/p24	covered	Alternate extracted slide text page 24; normalized text matches the .slides.md page. Human comparison is pending.
EV0000786	SRC00113/p25	covered	Extracted slide page 25 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000787	SRC00115/p25	covered	Original slide page 25 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000788	SRC00114/p25	covered	Alternate extracted slide text page 25; normalized text matches the .slides.md page. Human comparison is pending.
EV0000789	SRC00113/p26	covered	Extracted slide page 26 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000790	SRC00115/p26	covered	Original slide page 26 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000791	SRC00114/p26	covered	Alternate extracted slide text page 26; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000792	SRC00113/p27	covered	Extracted slide page 27 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000793	SRC00115/p27	covered	Original slide page 27 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000794	SRC00114/p27	covered	Alternate extracted slide text page 27; normalized text matches the .slides.md page. Human comparison is pending.
EV0000795	SRC00116/p1	covered	Extracted slide page 1 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__9_output_function_print__1ca49294268909b9.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000796	SRC00118/p1	covered	Original slide page 1 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__9_output_function_print__1ca49294268909b9.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000797	SRC00117/p1	covered	Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000798	SRC00116/p2	covered	Extracted slide page 2 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__9_output_function_print__1ca49294268909b9.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000799	SRC00118/p2	covered	Original slide page 2 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__9_output_function_print__1ca49294268909b9.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000800	SRC00117/p2	covered	Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human comparison is pending.
EV0000801	SRC00116/p3	covered	Extracted slide page 3 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__9_output_function_print__1ca49294268909b9.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000802	SRC00118/p3	covered	Original slide page 3 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__9_output_function_print__1ca49294268909b9.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000803	SRC00117/p3	covered	Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000804	SRC00116/p4	covered	Extracted slide page 4 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__9_output_function_print__1ca49294268909b9.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000805	SRC00118/p4	covered	Original slide page 4 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__9_output_function_print__1ca49294268909b9.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000806	SRC00117/p4	covered	Alternate extracted slide text page 4; normalized text matches the .slides.md page. Human comparison is pending.
EV0000807	SRC00116/p5	covered	Extracted slide page 5 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__9_output_function_print__1ca49294268909b9.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000808	SRC00118/p5	covered	Original slide page 5 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__9_output_function_print__1ca49294268909b9.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000809	SRC00117/p5	covered	Alternate extracted slide text page 5; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000810	SRC00116/p6	covered	Extracted slide page 6 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__9_output_function_print__1ca49294268909b9.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000811	SRC00118/p6	covered	Original slide page 6 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__9_output_function_print__1ca49294268909b9.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000812	SRC00117/p6	covered	Alternate extracted slide text page 6; normalized text matches the .slides.md page. Human comparison is pending.
EV0000813	SRC00116/p7	covered	Extracted slide page 7 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__9_output_function_print__1ca49294268909b9.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000814	SRC00118/p7	covered	Original slide page 7 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__9_output_function_print__1ca49294268909b9.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000815	SRC00117/p7	covered	Alternate extracted slide text page 7; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000816	SRC00119/p1	covered	Extracted slide page 1 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/010_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__10_variable_id_type__b024fe65bb8bef33.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000817	SRC00121/p1	covered	Original slide page 1 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/010_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__10_variable_id_type__b024fe65bb8bef33.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000818	SRC00120/p1	covered	Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human comparison is pending.
EV0000819	SRC00119/p2	covered	Extracted slide page 2 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/010_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__10_variable_id_type__b024fe65bb8bef33.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000820	SRC00121/p2	covered	Original slide page 2 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/010_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__10_variable_id_type__b024fe65bb8bef33.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000821	SRC00120/p2	covered	Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000822	SRC00119/p3	covered	Extracted slide page 3 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/010_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__10_variable_id_type__b024fe65bb8bef33.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000823	SRC00121/p3	covered	Original slide page 3 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/010_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__10_variable_id_type__b024fe65bb8bef33.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000824	SRC00120/p3	covered	Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human comparison is pending.
EV0000825	SRC00119/p4	covered	Extracted slide page 4 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/010_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__10_variable_id_type__b024fe65bb8bef33.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000826	SRC00121/p4	covered	Original slide page 4 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/010_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__10_variable_id_type__b024fe65bb8bef33.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000827	SRC00120/p4	covered	Alternate extracted slide text page 4; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000828	SRC00119/p5	covered	Extracted slide page 5 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/010_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__10_variable_id_type__b024fe65bb8bef33.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000829	SRC00121/p5	covered	Original slide page 5 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/010_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__10_variable_id_type__b024fe65bb8bef33.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000830	SRC00120/p5	covered	Alternate extracted slide text page 5; normalized text matches the .slides.md page. Human comparison is pending.
EV0000831	SRC00119/p6	covered	Extracted slide page 6 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/010_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__10_variable_id_type__b024fe65bb8bef33.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000832	SRC00121/p6	covered	Original slide page 6 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/010_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__10_variable_id_type__b024fe65bb8bef33.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000833	SRC00120/p6	covered	Alternate extracted slide text page 6; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000834	SRC00119/p7	covered	Extracted slide page 7 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/010_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__10_variable_id_type__b024fe65bb8bef33.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000835	SRC00121/p7	covered	Original slide page 7 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/010_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__10_variable_id_type__b024fe65bb8bef33.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000836	SRC00120/p7	covered	Alternate extracted slide text page 7; normalized text matches the .slides.md page. Human comparison is pending.
EV0000837	SRC00119/p8	covered	Extracted slide page 8 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/010_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__10_variable_id_type__b024fe65bb8bef33.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000838	SRC00121/p8	covered	Original slide page 8 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/010_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__10_variable_id_type__b024fe65bb8bef33.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000839	SRC00120/p8	covered	Alternate extracted slide text page 8; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000840	SRC00119/p9	covered	Extracted slide page 9 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/010_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__10_variable_id_type__b024fe65bb8bef33.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000841	SRC00121/p9	covered	Original slide page 9 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/010_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__10_variable_id_type__b024fe65bb8bef33.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000842	SRC00120/p9	covered	Alternate extracted slide text page 9; normalized text matches the .slides.md page. Human comparison is pending.
EV0000843	SRC00119/p10	covered	Extracted slide page 10 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/010_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__10_variable_id_type__b024fe65bb8bef33.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000844	SRC00121/p10	covered	Original slide page 10 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/010_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__10_variable_id_type__b024fe65bb8bef33.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000845	SRC00120/p10	covered	Alternate extracted slide text page 10; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV00008046	SRC00119/p11	covered	Extracted slide page 11 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/010_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__10_variable_id_type__b024fe65bb8bef33.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV00008047	SRC00121/p11	covered	Original slide page 11 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/010_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__10_variable_id_type__b024fe65bb8bef33.pdf; visual page included as vector figure. Human layout/content review is pending.
EV00008048	SRC00120/p11	covered	Alternate extracted slide text page 11; normalized text matches the .slides.md page. Human comparison is pending.
EV00008049	SRC00119/p12	covered	Extracted slide page 12 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/010_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__10_variable_id_type__b024fe65bb8bef33.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV00008050	SRC00121/p12	covered	Original slide page 12 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/010_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__10_variable_id_type__b024fe65bb8bef33.pdf; visual page included as vector figure. Human layout/content review is pending.
EV00008051	SRC00120/p12	covered	Alternate extracted slide text page 12; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000852	SRC00122/p1	covered	Extracted slide page 1 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/011_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__11_input_function__fe2e37a1fa374eaf.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000853	SRC00124/p1	covered	Original slide page 1 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/011_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__11_input_function__fe2e37a1fa374eaf.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000854	SRC00123/p1	covered	Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human comparison is pending.
EV0000855	SRC00122/p2	covered	Extracted slide page 2 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/011_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__11_input_function__fe2e37a1fa374eaf.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000856	SRC00124/p2	covered	Original slide page 2 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/011_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__11_input_function__fe2e37a1fa374eaf.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000857	SRC00123/p2	covered	Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000858	SRC00122/p3	covered	Extracted slide page 3 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/011_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__11_input_function__fe2e37a1fa374eaf.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000859	SRC00124/p3	covered	Original slide page 3 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/011_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__11_input_function__fe2e37a1fa374eaf.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000860	SRC00123/p3	covered	Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human comparison is pending.
EV0000861	SRC00122/p4	covered	Extracted slide page 4 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/011_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__11_input_function__fe2e37a1fa374eaf.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000862	SRC00124/p4	covered	Original slide page 4 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/011_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__11_input_function__fe2e37a1fa374eaf.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000863	SRC00123/p4	covered	Alternate extracted slide text page 4; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000864	SRC00122/p5	covered	Extracted slide page 5 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/011_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__11_input_function__fe2e37a1fa374eaf.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000865	SRC00124/p5	covered	Original slide page 5 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/011_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__11_input_function__fe2e37a1fa374eaf.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000866	SRC00123/p5	covered	Alternate extracted slide text page 5; normalized text matches the .slides.md page. Human comparison is pending.
EV0000867	SRC00125/p1	covered	Extracted slide page 1 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/012_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__12_Operators_Part1__b53129837a8bbfbd.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000868	SRC00127/p1	covered	Original slide page 1 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/012_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__12_Operators_Part1__b53129837a8bbfbd.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000869	SRC00126/p1	covered	Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000870	SRC00125/p2	covered	Extracted slide page 2 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/012_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__12_Operators_Part1__b53129837a8bbfbd.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000871	SRC00127/p2	covered	Original slide page 2 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/012_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__12_Operators_Part1__b53129837a8bbfbd.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000872	SRC00126/p2	covered	Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human comparison is pending.
EV0000873	SRC00125/p3	covered	Extracted slide page 3 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/012_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__12_Operators_Part1__b53129837a8bbfbd.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000874	SRC00127/p3	covered	Original slide page 3 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/012_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__12_Operators_Part1__b53129837a8bbfbd.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000875	SRC00126/p3	covered	Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV00008076	SRC00125/p4	covered	Extracted slide page 4 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/012_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__12_Operators_Part1__b53129837a8bbfbd.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV00008077	SRC00127/p4	covered	Original slide page 4 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/012_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__12_Operators_Part1__b53129837a8bbfbd.pdf; visual page included as vector figure. Human layout/content review is pending.
EV00008078	SRC00126/p4	covered	Alternate extracted slide text page 4; normalized text matches the .slides.md page. Human comparison is pending.
EV00008079	SRC00125/p5	covered	Extracted slide page 5 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/012_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__12_Operators_Part1__b53129837a8bbfbd.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV00008080	SRC00127/p5	covered	Original slide page 5 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/012_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__12_Operators_Part1__b53129837a8bbfbd.pdf; visual page included as vector figure. Human layout/content review is pending.
EV00008081	SRC00126/p5	covered	Alternate extracted slide text page 5; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000882	SRC00125/p6	covered	Extracted slide page 6 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/012_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__12_Operators_Part1__b53129837a8bbfbd.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000883	SRC00127/p6	covered	Original slide page 6 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/012_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__12_Operators_Part1__b53129837a8bbfbd.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000884	SRC00126/p6	covered	Alternate extracted slide text page 6; normalized text matches the .slides.md page. Human comparison is pending.
EV0000885	SRC00125/p7	covered	Extracted slide page 7 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/012_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__12_Operators_Part1__b53129837a8bbfbd.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000886	SRC00127/p7	covered	Original slide page 7 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/012_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__12_Operators_Part1__b53129837a8bbfbd.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000887	SRC00126/p7	covered	Alternate extracted slide text page 7; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000888	SRC00125/p8	covered	Extracted slide page 8 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/012_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__12_Operators_Part1__b53129837a8bbfbd.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000889	SRC00127/p8	covered	Original slide page 8 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/012_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__12_Operators_Part1__b53129837a8bbfbd.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000890	SRC00126/p8	covered	Alternate extracted slide text page 8; normalized text matches the .slides.md page. Human comparison is pending.
EV0000891	SRC00125/p9	covered	Extracted slide page 9 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/012_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__12_Operators_Part1__b53129837a8bbfbd.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000892	SRC00127/p9	covered	Original slide page 9 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/012_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__12_Operators_Part1__b53129837a8bbfbd.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000893	SRC00126/p9	covered	Alternate extracted slide text page 9; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000894	SRC00125/p10	covered	Extracted slide page 10 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/012_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__12_Operators_Part1__b53129837a8bbfbd.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000895	SRC00127/p10	covered	Original slide page 10 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/012_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__12_Operators_Part1__b53129837a8bbfbd.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000896	SRC00126/p10	covered	Alternate extracted slide text page 10; normalized text matches the .slides.md page. Human comparison is pending.
EV0000897	SRC00125/p11	covered	Extracted slide page 11 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/012_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__12_Operators_Part1__b53129837a8bbfbd.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000898	SRC00127/p11	covered	Original slide page 11 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/012_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__12_Operators_Part1__b53129837a8bbfbd.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000899	SRC00126/p11	covered	Alternate extracted slide text page 11; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000900	SRC00125/p12	covered	Extracted slide page 12 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/012_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__12_Operators_Part1__b53129837a8bbfbd.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000901	SRC00127/p12	covered	Original slide page 12 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/012_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__12_Operators_Part1__b53129837a8bbfbd.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000902	SRC00126/p12	covered	Alternate extracted slide text page 12; normalized text matches the .slides.md page. Human comparison is pending.
EV0000903	SRC00125/p13	covered	Extracted slide page 13 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/012_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__12_Operators_Part1__b53129837a8bbfbd.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000904	SRC00127/p13	covered	Original slide page 13 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/012_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__12_Operators_Part1__b53129837a8bbfbd.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000905	SRC00126/p13	covered	Alternate extracted slide text page 13; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000906	SRC00125/p14	covered	Extracted slide page 14 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/012_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__12_Operators_Part1__b53129837a8bbfbd.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000907	SRC00127/p14	covered	Original slide page 14 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/012_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__12_Operators_Part1__b53129837a8bbfbd.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000908	SRC00126/p14	covered	Alternate extracted slide text page 14; normalized text matches the .slides.md page. Human comparison is pending.
EV0000909	SRC00125/p15	covered	Extracted slide page 15 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/012_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__12_Operators_Part1__b53129837a8bbfbd.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000910	SRC00127/p15	covered	Original slide page 15 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/012_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__12_Operators_Part1__b53129837a8bbfbd.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000911	SRC00126/p15	covered	Alternate extracted slide text page 15; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000912	SRC00125/p16	covered	Extracted slide page 16 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/012_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__12_Operators_Part1__b53129837a8bbfbd.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000913	SRC00127/p16	covered	Original slide page 16 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/012_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__12_Operators_Part1__b53129837a8bbfbd.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000914	SRC00126/p16	covered	Alternate extracted slide text page 16; normalized text matches the .slides.md page. Human comparison is pending.
EV0000915	SRC00125/p17	covered	Extracted slide page 17 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/012_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__12_Operators_Part1__b53129837a8bbfbd.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000916	SRC00127/p17	covered	Original slide page 17 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/012_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__12_Operators_Part1__b53129837a8bbfbd.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000917	SRC00126/p17	covered	Alternate extracted slide text page 17; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000918	SRC00125/p18	covered	Extracted slide page 18 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/012_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__12_Operators_Part1__b53129837a8bbfbd.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000919	SRC00127/p18	covered	Original slide page 18 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/012_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__12_Operators_Part1__b53129837a8bbfbd.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000920	SRC00126/p18	covered	Alternate extracted slide text page 18; normalized text matches the .slides.md page. Human comparison is pending.
EV0000921	SRC00125/p19	covered	Extracted slide page 19 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/012_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__12_Operators_Part1__b53129837a8bbfbd.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000922	SRC00127/p19	covered	Original slide page 19 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/012_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__12_Operators_Part1__b53129837a8bbfbd.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000923	SRC00126/p19	covered	Alternate extracted slide text page 19; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000924	SRC00125/p20	covered	Extracted slide page 20 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/012_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__12_Operators_Part1__b53129837a8bbfbd.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000925	SRC00127/p20	covered	Original slide page 20 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/012_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__12_Operators_Part1__b53129837a8bbfbd.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000926	SRC00126/p20	covered	Alternate extracted slide text page 20; normalized text matches the .slides.md page. Human comparison is pending.
EV0000927	SRC00125/p21	covered	Extracted slide page 21 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/012_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__12_Operators_Part1__b53129837a8bbfbd.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000928	SRC00127/p21	covered	Original slide page 21 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/012_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__12_Operators_Part1__b53129837a8bbfbd.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000929	SRC00126/p21	covered	Alternate extracted slide text page 21; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV00009330	SRC00125/p22	covered	Extracted slide page 22 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/012_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__12_Operators_Part1__b53129837a8bbfbd.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV00009331	SRC00127/p22	covered	Original slide page 22 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/012_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__12_Operators_Part1__b53129837a8bbfbd.pdf; visual page included as vector figure. Human layout/content review is pending.
EV00009332	SRC00126/p22	covered	Alternate extracted slide text page 22; normalized text matches the .slides.md page. Human comparison is pending.
EV00009333	SRC00125/p23	covered	Extracted slide page 23 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/012_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__12_Operators_Part1__b53129837a8bbfbd.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV00009334	SRC00127/p23	covered	Original slide page 23 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/012_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__12_Operators_Part1__b53129837a8bbfbd.pdf; visual page included as vector figure. Human layout/content review is pending.
EV00009335	SRC00126/p23	covered	Alternate extracted slide text page 23; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000936	SRC00128/p1	covered	Extracted slide page 1 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/014_Operator_Polymorphism__32ad541949df1cab.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000937	SRC00130/p1	covered	Original slide page 1 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/014_Operator_Polymorphism__32ad541949df1cab.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000938	SRC00129/p1	covered	Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human comparison is pending.
EV0000939	SRC00128/p2	covered	Extracted slide page 2 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/014_Operator_Polymorphism__32ad541949df1cab.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000940	SRC00130/p2	covered	Original slide page 2 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/014_Operator_Polymorphism__32ad541949df1cab.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000941	SRC00129/p2	covered	Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000942	SRC00128/p3	covered	Extracted slide page 3 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/014_Operator_Polymorphism__32ad541949df1cab.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000943	SRC00130/p3	covered	Original slide page 3 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/014_Operator_Polymorphism__32ad541949df1cab.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000944	SRC00129/p3	covered	Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human comparison is pending.
EV0000945	SRC00128/p4	covered	Extracted slide page 4 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/014_Operator_Polymorphism__32ad541949df1cab.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000946	SRC00130/p4	covered	Original slide page 4 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/014_Operator_Polymorphism__32ad541949df1cab.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000947	SRC00129/p4	covered	Alternate extracted slide text page 4; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV00009948	SRC00128/p5	covered	Extracted slide page 5 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/014_Operator_Polymorphism__32ad541949df1cab.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV00009949	SRC00130/p5	covered	Original slide page 5 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/014_Operator_Polymorphism__32ad541949df1cab.pdf; visual page included as vector figure. Human layout/content review is pending.
EV00009950	SRC00129/p5	covered	Alternate extracted slide text page 5; normalized text matches the .slides.md page. Human comparison is pending.
EV00009951	SRC00128/p6	covered	Extracted slide page 6 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/014_Operator_Polymorphism__32ad541949df1cab.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV00009952	SRC00130/p6	covered	Original slide page 6 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/014_Operator_Polymorphism__32ad541949df1cab.pdf; visual page included as vector figure. Human layout/content review is pending.
EV00009953	SRC00129/p6	covered	Alternate extracted slide text page 6; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000954	SRC00131/p1	covered	Extracted slide page 1 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/014_Operator_precedence_and_associativity__d91f176414ae90ad.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000955	SRC00133/p1	covered	Original slide page 1 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/014_Operator_precedence_and_associativity__d91f176414ae90ad.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000956	SRC00132/p1	covered	Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human comparison is pending.
EV0000957	SRC00131/p2	covered	Extracted slide page 2 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/014_Operator_precedence_and_associativity__d91f176414ae90ad.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000958	SRC00133/p2	covered	Original slide page 2 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/014_Operator_precedence_and_associativity__d91f176414ae90ad.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000959	SRC00132/p2	covered	Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000960	SRC00131/p3	covered	Extracted slide page 3 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/014_Operator_precedence_and_associativity__d91f176414ae90ad.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000961	SRC00133/p3	covered	Original slide page 3 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/014_Operator_precedence_and_associativity__d91f176414ae90ad.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000962	SRC00132/p3	covered	Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human comparison is pending.
EV0000963	SRC00131/p4	covered	Extracted slide page 4 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/014_Operator_precedence_and_associativity__d91f176414ae90ad.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000964	SRC00133/p4	covered	Original slide page 4 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/014_Operator_precedence_and_associativity__d91f176414ae90ad.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000965	SRC00132/p4	covered	Alternate extracted slide text page 4; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000966	SRC00131/p5	covered	Extracted slide page 5 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/014_Operator_precedence_and_associativity__d91f176414ae90ad.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000967	SRC00133/p5	covered	Original slide page 5 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/014_Operator_precedence_and_associativity__d91f176414ae90ad.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000968	SRC00132/p5	covered	Alternate extracted slide text page 5; normalized text matches the .slides.md page. Human comparison is pending.
EV0000969	SRC00131/p6	covered	Extracted slide page 6 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/014_Operator_precedence_and_associativity__d91f176414ae90ad.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000970	SRC00133/p6	covered	Original slide page 6 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/014_Operator_precedence_and_associativity__d91f176414ae90ad.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000971	SRC00132/p6	covered	Alternate extracted slide text page 6; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000972	SRC00131/p7	covered	Extracted slide page 7 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/014_Operator_precedence_and_associativity__d91f176414ae90ad.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000973	SRC00133/p7	covered	Original slide page 7 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/014_Operator_precedence_and_associativity__d91f176414ae90ad.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000974	SRC00132/p7	covered	Alternate extracted slide text page 7; normalized text matches the .slides.md page. Human comparison is pending.
EV0000975	SRC00134/p1	covered	Extracted slide page 1 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/015_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__15_16_Control_structures__bcb5c5877df41eba.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000976	SRC00136/p1	covered	Original slide page 1 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/015_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__15_16_Control_structures__bcb5c5877df41eba.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000977	SRC00135/p1	covered	Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV00009978	SRC00134/p2	covered	Extracted slide page 2 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/015_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__15_16_Control_structures__bcb5c5877df41eba.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV00009979	SRC00136/p2	covered	Original slide page 2 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/015_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__15_16_Control_structures__bcb5c5877df41eba.pdf; visual page included as vector figure. Human layout/content review is pending.
EV00009980	SRC00135/p2	covered	Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human comparison is pending.
EV00009981	SRC00134/p3	covered	Extracted slide page 3 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/015_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__15_16_Control_structures__bcb5c5877df41eba.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV00009982	SRC00136/p3	covered	Original slide page 3 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/015_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__15_16_Control_structures__bcb5c5877df41eba.pdf; visual page included as vector figure. Human layout/content review is pending.
EV00009983	SRC00135/p3	covered	Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000984	SRC00134/p4	covered	Extracted slide page 4 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/015_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__15_16_Control_structures__bcb5c5877df41eba.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000985	SRC00136/p4	covered	Original slide page 4 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/015_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__15_16_Control_structures__bcb5c5877df41eba.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000986	SRC00135/p4	covered	Alternate extracted slide text page 4; normalized text matches the .slides.md page. Human comparison is pending.
EV0000987	SRC00134/p5	covered	Extracted slide page 5 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/015_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__15_16_Control_structures__bcb5c5877df41eba.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000988	SRC00136/p5	covered	Original slide page 5 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/015_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__15_16_Control_structures__bcb5c5877df41eba.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000989	SRC00135/p5	covered	Alternate extracted slide text page 5; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000990	SRC00134/p6	covered	Extracted slide page 6 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/015_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__15_16_Control_structures__bcb5c5877df41eba.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000991	SRC00136/p6	covered	Original slide page 6 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/015_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__15_16_Control_structures__bcb5c5877df41eba.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000992	SRC00135/p6	covered	Alternate extracted slide text page 6; normalized text matches the .slides.md page. Human comparison is pending.
EV0000993	SRC00134/p7	covered	Extracted slide page 7 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/015_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__15_16_Control_structures__bcb5c5877df41eba.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000994	SRC00136/p7	covered	Original slide page 7 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/015_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__15_16_Control_structures__bcb5c5877df41eba.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000995	SRC00135/p7	covered	Alternate extracted slide text page 7; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000996	SRC00134/p8	covered	Extracted slide page 8 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/015_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__15_16_Control_structures__bcb5c5877df41eba.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000997	SRC00136/p8	covered	Original slide page 8 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/015_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__15_16_Control_structures__bcb5c5877df41eba.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000998	SRC00135/p8	covered	Alternate extracted slide text page 8; normalized text matches the .slides.md page. Human comparison is pending.
EV0000999	SRC00134/p9	covered	Extracted slide page 9 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/015_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__15_16_Control_structures__bcb5c5877df41eba.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001000	SRC00136/p9	covered	Original slide page 9 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/015_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__15_16_Control_structures__bcb5c5877df41eba.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001001	SRC00135/p9	covered	Alternate extracted slide text page 9; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0001002	SRC00134/p10	covered	Extracted slide page 10 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/015_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__15_16_Control_structures__bcb5c5877df41eba.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001003	SRC00136/p10	covered	Original slide page 10 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/015_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__15_16_Control_structures__bcb5c5877df41eba.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001004	SRC00135/p10	covered	Alternate extracted slide text page 10; normalized text matches the .slides.md page. Human comparison is pending.
EV0001005	SRC00134/p11	covered	Extracted slide page 11 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/015_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__15_16_Control_structures__bcb5c5877df41eba.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001006	SRC00136/p11	covered	Original slide page 11 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/015_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__15_16_Control_structures__bcb5c5877df41eba.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001007	SRC00135/p11	covered	Alternate extracted slide text page 11; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0001008	SRC00134/p12	covered	Extracted slide page 12 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/015_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__15_16_Control_structures__bcb5c5877df41eba.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001009	SRC00136/p12	covered	Original slide page 12 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/015_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__15_16_Control_structures__bcb5c5877df41eba.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001010	SRC00135/p12	covered	Alternate extracted slide text page 12; normalized text matches the .slides.md page. Human comparison is pending.
EV0001011	SRC00134/p13	covered	Extracted slide page 13 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/015_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__15_16_Control_structures__bcb5c5877df41eba.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001012	SRC00136/p13	covered	Original slide page 13 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/015_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__15_16_Control_structures__bcb5c5877df41eba.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001013	SRC00135/p13	covered	Alternate extracted slide text page 13; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0001014	SRC00134/p14	covered	Extracted slide page 14 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/015_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__15_16_Control_structures__bcb5c5877df41eba.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001015	SRC00136/p14	covered	Original slide page 14 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/015_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__15_16_Control_structures__bcb5c5877df41eba.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001016	SRC00135/p14	covered	Alternate extracted slide text page 14; normalized text matches the .slides.md page. Human comparison is pending.
EV0001017	SRC00134/p15	covered	Extracted slide page 15 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/015_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__15_16_Control_structures__bcb5c5877df41eba.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001018	SRC00136/p15	covered	Original slide page 15 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/015_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__15_16_Control_structures__bcb5c5877df41eba.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001019	SRC00135/p15	covered	Alternate extracted slide text page 15; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0001020	SRC00134/p16	covered	Extracted slide page 16 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/015_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__15_16_Control_structures__bcb5c5877df41eba.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001021	SRC00136/p16	covered	Original slide page 16 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/015_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__15_16_Control_structures__bcb5c5877df41eba.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001022	SRC00135/p16	covered	Alternate extracted slide text page 16; normalized text matches the .slides.md page. Human comparison is pending.
EV0001023	SRC00134/p17	covered	Extracted slide page 17 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/015_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__15_16_Control_structures__bcb5c5877df41eba.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001024	SRC00136/p17	covered	Original slide page 17 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/015_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__15_16_Control_structures__bcb5c5877df41eba.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001025	SRC00135/p17	covered	Alternate extracted slide text page 17; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV00010026	SRC00134/p18	covered	Extracted slide page 18 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/015_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__15_16_Control_structures__bcb5c5877df41eba.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV00010027	SRC00136/p18	covered	Original slide page 18 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/015_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__15_16_Control_structures__bcb5c5877df41eba.pdf; visual page included as vector figure. Human layout/content review is pending.
EV00010028	SRC00135/p18	covered	Alternate extracted slide text page 18; normalized text matches the .slides.md page. Human comparison is pending.
EV00010029	SRC00134/p19	covered	Extracted slide page 19 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/015_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__15_16_Control_structures__bcb5c5877df41eba.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV00010030	SRC00136/p19	covered	Original slide page 19 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/015_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__15_16_Control_structures__bcb5c5877df41eba.pdf; visual page included as vector figure. Human layout/content review is pending.
EV00010031	SRC00135/p19	covered	Alternate extracted slide text page 19; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0001032	SRC00134/p20	covered	Extracted slide page 20 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/015_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__15_16_Control_structures__bcb5c5877df41eba.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001033	SRC00136/p20	covered	Original slide page 20 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/015_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__15_16_Control_structures__bcb5c5877df41eba.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001034	SRC00135/p20	covered	Alternate extracted slide text page 20; normalized text matches the .slides.md page. Human comparison is pending.
EV0001035	SRC00137/p1	covered	Extracted slide page 1 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/027_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__1_General_intro_to_data_structures_in_python__a531eb93c7c504ed.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001036	SRC00139/p1	covered	Original slide page 1 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/027_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__1_General_intro_to_data_structures_in_python__a531eb93c7c504ed.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001037	SRC00138/p1	covered	Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0001038	SRC00137/p2	covered	Extracted slide page 2 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/027_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__1_General_intro_to_data_structures_in_python__a531eb93c7c504ed.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001039	SRC00139/p2	covered	Original slide page 2 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/027_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__1_General_intro_to_data_structures_in_python__a531eb93c7c504ed.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001040	SRC00138/p2	covered	Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human comparison is pending.
EV0001041	SRC00137/p3	covered	Extracted slide page 3 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/027_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__1_General_intro_to_data_structures_in_python__a531eb93c7c504ed.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001042	SRC00139/p3	covered	Original slide page 3 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/027_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__1_General_intro_to_data_structures_in_python__a531eb93c7c504ed.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001043	SRC00138/p3	covered	Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0001044	SRC00137/p4	covered	Extracted slide page 4 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/027_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__1_General_intro_to_data_structures_in_python__a531eb93c7c504ed.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001045	SRC00139/p4	covered	Original slide page 4 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/027_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__1_General_intro_to_data_structures_in_python__a531eb93c7c504ed.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001046	SRC00138/p4	covered	Alternate extracted slide text page 4; normalized text matches the .slides.md page. Human comparison is pending.
EV0001047	SRC00137/p5	covered	Extracted slide page 5 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/027_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__1_General_intro_to_data_structures_in_python__a531eb93c7c504ed.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001048	SRC00139/p5	covered	Original slide page 5 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/027_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__1_General_intro_to_data_structures_in_python__a531eb93c7c504ed.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001049	SRC00138/p5	covered	Alternate extracted slide text page 5; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0001050	SRC00137/p6	covered	Extracted slide page 6 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/027_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__1_General_intro_to_data_structures_in_python__a531eb93c7c504ed.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001051	SRC00139/p6	covered	Original slide page 6 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/027_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__1_General_intro_to_data_structures_in_python__a531eb93c7c504ed.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001052	SRC00138/p6	covered	Alternate extracted slide text page 6; normalized text matches the .slides.md page. Human comparison is pending.
EV0001053	SRC00137/p7	covered	Extracted slide page 7 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/027_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__1_General_intro_to_data_structures_in_python__a531eb93c7c504ed.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001054	SRC00139/p7	covered	Original slide page 7 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/027_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__1_General_intro_to_data_structures_in_python__a531eb93c7c504ed.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001055	SRC00138/p7	covered	Alternate extracted slide text page 7; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0001056	SRC00140/p1	covered	Extracted slide page 1 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001057	SRC00142/p1	covered	Original slide page 1 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001058	SRC00141/p1	covered	Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human comparison is pending.
EV0001059	SRC00140/p2	covered	Extracted slide page 2 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001060	SRC00142/p2	covered	Original slide page 2 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001061	SRC00141/p2	covered	Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0001062	SRC00140/p3	covered	Extracted slide page 3 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001063	SRC00142/p3	covered	Original slide page 3 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001064	SRC00141/p3	covered	Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human comparison is pending.
EV0001065	SRC00140/p4	covered	Extracted slide page 4 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001066	SRC00142/p4	covered	Original slide page 4 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001067	SRC00141/p4	covered	Alternate extracted slide text page 4; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV00010068	SRC00140/p5	covered	Extracted slide page 5 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV00010069	SRC00142/p5	covered	Original slide page 5 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.pdf; visual page included as vector figure. Human layout/content review is pending.
EV00010070	SRC00141/p5	covered	Alternate extracted slide text page 5; normalized text matches the .slides.md page. Human comparison is pending.
EV00010071	SRC00140/p6	covered	Extracted slide page 6 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV00010072	SRC00142/p6	covered	Original slide page 6 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.pdf; visual page included as vector figure. Human layout/content review is pending.
EV00010073	SRC00141/p6	covered	Alternate extracted slide text page 6; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0001074	SRC00140/p7	covered	Extracted slide page 7 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001075	SRC00142/p7	covered	Original slide page 7 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001076	SRC00141/p7	covered	Alternate extracted slide text page 7; normalized text matches the .slides.md page. Human comparison is pending.
EV0001077	SRC00140/p8	covered	Extracted slide page 8 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001078	SRC00142/p8	covered	Original slide page 8 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001079	SRC00141/p8	covered	Alternate extracted slide text page 8; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0001080	SRC00140/p9	covered	Extracted slide page 9 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001081	SRC00142/p9	covered	Original slide page 9 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001082	SRC00141/p9	covered	Alternate extracted slide text page 9; normalized text matches the .slides.md page. Human comparison is pending.
EV0001083	SRC00140/p10	covered	Extracted slide page 10 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001084	SRC00142/p10	covered	Original slide page 10 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001085	SRC00141/p10	covered	Alternate extracted slide text page 10; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV00010086	SRC00140/p11	covered	Extracted slide page 11 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV00010087	SRC00142/p11	covered	Original slide page 11 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.pdf; visual page included as vector figure. Human layout/content review is pending.
EV00010088	SRC00141/p11	covered	Alternate extracted slide text page 11; normalized text matches the .slides.md page. Human comparison is pending.
EV00010089	SRC00140/p12	covered	Extracted slide page 12 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV00010090	SRC00142/p12	covered	Original slide page 12 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.pdf; visual page included as vector figure. Human layout/content review is pending.
EV00010091	SRC00141/p12	covered	Alternate extracted slide text page 12; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0001092	SRC00140/p13	covered	Extracted slide page 13 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001093	SRC00142/p13	covered	Original slide page 13 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001094	SRC00141/p13	covered	Alternate extracted slide text page 13; normalized text matches the .slides.md page. Human comparison is pending.
EV0001095	SRC00140/p14	covered	Extracted slide page 14 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001096	SRC00142/p14	covered	Original slide page 14 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001097	SRC00141/p14	covered	Alternate extracted slide text page 14; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0001098	SRC00140/p15	covered	Extracted slide page 15 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001099	SRC00142/p15	covered	Original slide page 15 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001100	SRC00141/p15	covered	Alternate extracted slide text page 15; normalized text matches the .slides.md page. Human comparison is pending.
EV0001101	SRC00140/p16	covered	Extracted slide page 16 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001102	SRC00142/p16	covered	Original slide page 16 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001103	SRC00141/p16	covered	Alternate extracted slide text page 16; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0001104	SRC00140/p17	covered	Extracted slide page 17 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001105	SRC00142/p17	covered	Original slide page 17 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001106	SRC00141/p17	covered	Alternate extracted slide text page 17; normalized text matches the .slides.md page. Human comparison is pending.
EV0001107	SRC00140/p18	covered	Extracted slide page 18 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001108	SRC00142/p18	covered	Original slide page 18 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001109	SRC00141/p18	covered	Alternate extracted slide text page 18; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV00011 10	SRC00140/p19	covered	Extracted slide page 19 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV00011 11	SRC00142/p19	covered	Original slide page 19 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.pdf; visual page included as vector figure. Human layout/content review is pending.
EV00011 12	SRC00141/p19	covered	Alternate extracted slide text page 19; normalized text matches the .slides.md page. Human comparison is pending.
EV00011 13	SRC00140/p20	covered	Extracted slide page 20 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV00011 14	SRC00142/p20	covered	Original slide page 20 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.pdf; visual page included as vector figure. Human layout/content review is pending.
EV00011 15	SRC00141/p20	covered	Alternate extracted slide text page 20; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0001116	SRC00140/p21	covered	Extracted slide page 21 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001117	SRC00142/p21	covered	Original slide page 21 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001118	SRC00141/p21	covered	Alternate extracted slide text page 21; normalized text matches the .slides.md page. Human comparison is pending.
EV0001119	SRC00140/p22	covered	Extracted slide page 22 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001120	SRC00142/p22	covered	Original slide page 22 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001121	SRC00141/p22	covered	Alternate extracted slide text page 22; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV00011122	SRC00140/p23	covered	Extracted slide page 23 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV00011123	SRC00142/p23	covered	Original slide page 23 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.pdf; visual page included as vector figure. Human layout/content review is pending.
EV00011124	SRC00141/p23	covered	Alternate extracted slide text page 23; normalized text matches the .slides.md page. Human comparison is pending.
EV00011125	SRC00140/p24	covered	Extracted slide page 24 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV00011126	SRC00142/p24	covered	Original slide page 24 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.pdf; visual page included as vector figure. Human layout/content review is pending.
EV00011127	SRC00141/p24	covered	Alternate extracted slide text page 24; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0001128	SRC00140/p25	covered	Extracted slide page 25 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001129	SRC00142/p25	covered	Original slide page 25 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001130	SRC00141/p25	covered	Alternate extracted slide text page 25; normalized text matches the .slides.md page. Human comparison is pending.
EV0001131	SRC00140/p26	covered	Extracted slide page 26 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001132	SRC00142/p26	covered	Original slide page 26 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001133	SRC00141/p26	covered	Alternate extracted slide text page 26; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0001134	SRC00140/p27	covered	Extracted slide page 27 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001135	SRC00142/p27	covered	Original slide page 27 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001136	SRC00141/p27	covered	Alternate extracted slide text page 27; normalized text matches the .slides.md page. Human comparison is pending.
EV0001137	SRC00140/p28	covered	Extracted slide page 28 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001138	SRC00142/p28	covered	Original slide page 28 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001139	SRC00141/p28	covered	Alternate extracted slide text page 28; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0001140	SRC00140/p29	covered	Extracted slide page 29 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001141	SRC00142/p29	covered	Original slide page 29 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001142	SRC00141/p29	covered	Alternate extracted slide text page 29; normalized text matches the .slides.md page. Human comparison is pending.
EV0001143	SRC00140/p30	covered	Extracted slide page 30 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001144	SRC00142/p30	covered	Original slide page 30 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001145	SRC00141/p30	covered	Alternate extracted slide text page 30; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV00011146	SRC00143/p1	covered	Extracted slide page 1 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV00011147	SRC00145/p1	covered	Original slide page 1 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.pdf; visual page included as vector figure. Human layout/content review is pending.
EV00011148	SRC00144/p1	covered	Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human comparison is pending.
EV00011149	SRC00143/p2	covered	Extracted slide page 2 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV00011150	SRC00145/p2	covered	Original slide page 2 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.pdf; visual page included as vector figure. Human layout/content review is pending.
EV00011151	SRC00144/p2	covered	Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0001152	SRC00143/p3	covered	Extracted slide page 3 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001153	SRC00145/p3	covered	Original slide page 3 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001154	SRC00144/p3	covered	Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human comparison is pending.
EV0001155	SRC00143/p4	covered	Extracted slide page 4 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001156	SRC00145/p4	covered	Original slide page 4 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001157	SRC00144/p4	covered	Alternate extracted slide text page 4; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0001158	SRC00143/p5	covered	Extracted slide page 5 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001159	SRC00145/p5	covered	Original slide page 5 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001160	SRC00144/p5	covered	Alternate extracted slide text page 5; normalized text matches the .slides.md page. Human comparison is pending.
EV0001161	SRC00143/p6	covered	Extracted slide page 6 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001162	SRC00145/p6	covered	Original slide page 6 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001163	SRC00144/p6	covered	Alternate extracted slide text page 6; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV00011 64	SRC00143/p7	covered	Extracted slide page 7 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV00011 65	SRC00145/p7	covered	Original slide page 7 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.pdf; visual page included as vector figure. Human layout/content review is pending.
EV00011 66	SRC00144/p7	covered	Alternate extracted slide text page 7; normalized text matches the .slides.md page. Human comparison is pending.
EV00011 67	SRC00143/p8	covered	Extracted slide page 8 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV00011 68	SRC00145/p8	covered	Original slide page 8 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.pdf; visual page included as vector figure. Human layout/content review is pending.
EV00011 69	SRC00144/p8	covered	Alternate extracted slide text page 8; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0001170	SRC00143/p9	covered	Extracted slide page 9 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001171	SRC00145/p9	covered	Original slide page 9 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001172	SRC00144/p9	covered	Alternate extracted slide text page 9; normalized text matches the .slides.md page. Human comparison is pending.
EV0001173	SRC00143/p10	covered	Extracted slide page 10 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001174	SRC00145/p10	covered	Original slide page 10 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001175	SRC00144/p10	covered	Alternate extracted slide text page 10; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV00011776	SRC00143/p11	covered	Extracted slide page 11 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV00011777	SRC00145/p11	covered	Original slide page 11 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.pdf; visual page included as vector figure. Human layout/content review is pending.
EV00011778	SRC00144/p11	covered	Alternate extracted slide text page 11; normalized text matches the .slides.md page. Human comparison is pending.
EV00011779	SRC00143/p12	covered	Extracted slide page 12 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV00011780	SRC00145/p12	covered	Original slide page 12 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.pdf; visual page included as vector figure. Human layout/content review is pending.
EV00011781	SRC00144/p12	covered	Alternate extracted slide text page 12; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV00011182	SRC00143/p13	covered	Extracted slide page 13 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV00011183	SRC00145/p13	covered	Original slide page 13 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.pdf; visual page included as vector figure. Human layout/content review is pending.
EV00011184	SRC00144/p13	covered	Alternate extracted slide text page 13; normalized text matches the .slides.md page. Human comparison is pending.
EV00011185	SRC00143/p14	covered	Extracted slide page 14 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV00011186	SRC00145/p14	covered	Original slide page 14 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.pdf; visual page included as vector figure. Human layout/content review is pending.
EV00011187	SRC00144/p14	covered	Alternate extracted slide text page 14; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0001188	SRC00143/p15	covered	Extracted slide page 15 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001189	SRC00145/p15	covered	Original slide page 15 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001190	SRC00144/p15	covered	Alternate extracted slide text page 15; normalized text matches the .slides.md page. Human comparison is pending.
EV0001191	SRC00143/p16	covered	Extracted slide page 16 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001192	SRC00145/p16	covered	Original slide page 16 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001193	SRC00144/p16	covered	Alternate extracted slide text page 16; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0001194	SRC00143/p17	covered	Extracted slide page 17 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001195	SRC00145/p17	covered	Original slide page 17 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001196	SRC00144/p17	covered	Alternate extracted slide text page 17; normalized text matches the .slides.md page. Human comparison is pending.
EV0001197	SRC00143/p18	covered	Extracted slide page 18 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001198	SRC00145/p18	covered	Original slide page 18 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001199	SRC00144/p18	covered	Alternate extracted slide text page 18; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0001200	SRC00143/p19	covered	Extracted slide page 19 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001201	SRC00145/p19	covered	Original slide page 19 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001202	SRC00144/p19	covered	Alternate extracted slide text page 19; normalized text matches the .slides.md page. Human comparison is pending.
EV0001203	SRC00143/p20	covered	Extracted slide page 20 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001204	SRC00145/p20	covered	Original slide page 20 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001205	SRC00144/p20	covered	Alternate extracted slide text page 20; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0001206	SRC00143/p21	covered	Extracted slide page 21 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001207	SRC00145/p21	covered	Original slide page 21 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001208	SRC00144/p21	covered	Alternate extracted slide text page 21; normalized text matches the .slides.md page. Human comparison is pending.
EV0001209	SRC00143/p22	covered	Extracted slide page 22 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001210	SRC00145/p22	covered	Original slide page 22 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001211	SRC00144/p22	covered	Alternate extracted slide text page 22; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0001212	SRC00143/p23	covered	Extracted slide page 23 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001213	SRC00145/p23	covered	Original slide page 23 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001214	SRC00144/p23	covered	Alternate extracted slide text page 23; normalized text matches the .slides.md page. Human comparison is pending.
EV0001215	SRC00143/p24	covered	Extracted slide page 24 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001216	SRC00145/p24	covered	Original slide page 24 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001217	SRC00144/p24	covered	Alternate extracted slide text page 24; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0001218	SRC00143/p25	covered	Extracted slide page 25 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001219	SRC00145/p25	covered	Original slide page 25 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001220	SRC00144/p25	covered	Alternate extracted slide text page 25; normalized text matches the .slides.md page. Human comparison is pending.
EV0001221	SRC00143/p26	covered	Extracted slide page 26 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001222	SRC00145/p26	covered	Original slide page 26 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001223	SRC00144/p26	covered	Alternate extracted slide text page 26; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0001224	SRC00143/p27	covered	Extracted slide page 27 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001225	SRC00145/p27	covered	Original slide page 27 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001226	SRC00144/p27	covered	Alternate extracted slide text page 27; normalized text matches the .slides.md page. Human comparison is pending.
EV0001227	SRC00143/p28	covered	Extracted slide page 28 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0001228	SRC00145/p28	covered	Original slide page 28 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0001229	SRC00144/p28	covered	Alternate extracted slide text page 28; normalized text matches the .slides.md page. Human comparison is pending.
EV0001230	SRC00146/q1	covered	Original archive PDF page 1; candidate unit association is text-based and unconfirmed.

Evidence	Source unit	Status	Decision
EV00012231	SRC00146/q2	covered	Original archive PDF page 2; candidate unit association is text-based and unconfirmed.
EV00012232	SRC00146/q3	covered	Original archive PDF page 3; candidate unit association is text-based and unconfirmed.
EV00012233	SRC00147/q1	covered	Original archive PDF page 1; candidate unit association is text-based and unconfirmed.
EV00012234	SRC00147/q2	covered	Original archive PDF page 2; candidate unit association is text-based and unconfirmed.
EV00012235	SRC00147/q3	covered	Original archive PDF page 3; candidate unit association is text-based and unconfirmed.
EV00012236	SRC00147/q6	covered	Original archive PDF page 6; candidate unit association is text-based and unconfirmed.
EV00012237	SRC00148/q1	covered	Original archive PDF page 1; candidate unit association is text-based and unconfirmed.
EV00012238	SRC00148/q2	covered	Original archive PDF page 2; candidate unit association is text-based and unconfirmed.
EV00012239	SRC00148/q3	covered	Original archive PDF page 3; candidate unit association is text-based and unconfirmed.
EV00012240	SRC00148/q5	covered	Original archive PDF page 5; candidate unit association is text-based and unconfirmed.
EV00012241	SRC00149/q1	covered	Original archive PDF page 1; candidate unit association is text-based and unconfirmed.
EV00012242	SRC00149/q2	covered	Original archive PDF page 2; candidate unit association is text-based and unconfirmed.
EV00012243	SRC00150/q1	covered	Original archive PDF page 1; candidate unit association is text-based and unconfirmed.
EV00012244	SRC00150/q2	covered	Original archive PDF page 2; candidate unit association is text-based and unconfirmed.
EV00012245	SRC00150/q3	covered	Original archive PDF page 3; candidate unit association is text-based and unconfirmed.
EV00012246	SRC00150/q6	covered	Original archive PDF page 6; candidate unit association is text-based and unconfirmed.

Evidence	Source unit	Status	Decision
EV00012147	SRC00151/q1	covered	Original archive PDF page 1; candidate unit association is text-based and unconfirmed.
EV00012148	SRC00151/q2	covered	Original archive PDF page 2; candidate unit association is text-based and unconfirmed.
EV00012149	SRC00151/q3	covered	Original archive PDF page 3; candidate unit association is text-based and unconfirmed.
EV00012150	SRC00151/q5	covered	Original archive PDF page 5; candidate unit association is text-based and unconfirmed.
EV00012151	SRC00152/q1	covered	Original archive PDF page 1; candidate unit association is text-based and unconfirmed.
EV00012152	SRC00152/q2	covered	Original archive PDF page 2; candidate unit association is text-based and unconfirmed.
EV00012153	SRC00152/q3	covered	Original archive PDF page 3; candidate unit association is text-based and unconfirmed.
EV00012154	SRC00152/q4	covered	Original archive PDF page 4; candidate unit association is text-based and unconfirmed.
EV00012155	SRC00152/q6	covered	Original archive PDF page 6; candidate unit association is text-based and unconfirmed.
EV00012156	SRC00153/q1	covered	Original archive PDF page 1; candidate unit association is text-based and unconfirmed.
EV00012157	SRC00153/q2	covered	Original archive PDF page 2; candidate unit association is text-based and unconfirmed.
EV00012158	SRC00153/q3	covered	Original archive PDF page 3; candidate unit association is text-based and unconfirmed.
EV00012159	SRC00153/q4	covered	Original archive PDF page 4; candidate unit association is text-based and unconfirmed.
EV00012160	SRC00153/q8	covered	Original archive PDF page 8; candidate unit association is text-based and unconfirmed.
EV00012161	SRC00154/q1	covered	Original archive PDF page 1; candidate unit association is text-based and unconfirmed.
EV00012162	SRC00154/q2	covered	Original archive PDF page 2; candidate unit association is text-based and unconfirmed.

Evidence	Source unit	Status	Decision
EV00012763	SRC00154/q3	covered	Original archive PDF page 3; candidate unit association is text-based and unconfirmed.
EV00012764	SRC00154/q4	covered	Original archive PDF page 4; candidate unit association is text-based and unconfirmed.
EV00012765	SRC00154/q6	covered	Original archive PDF page 6; candidate unit association is text-based and unconfirmed.
EV00012766	SRC00155/q1	covered	Original archive PDF page 1; candidate unit association is text-based and unconfirmed.
EV00012767	SRC00155/q2	covered	Original archive PDF page 2; candidate unit association is text-based and unconfirmed.
EV00012768	SRC00155/q3	covered	Original archive PDF page 3; candidate unit association is text-based and unconfirmed.
EV00012769	SRC00155/q4	covered	Original archive PDF page 4; candidate unit association is text-based and unconfirmed.
EV00012770	SRC00155/q8	covered	Original archive PDF page 8; candidate unit association is text-based and unconfirmed.
EV00012771	SRC00156/q1	covered	Original archive PDF page 1; candidate unit association is text-based and unconfirmed.
EV00012772	SRC00156/q2	covered	Original archive PDF page 2; candidate unit association is text-based and unconfirmed.
EV00012773	SRC00156/q3	covered	Original archive PDF page 3; candidate unit association is text-based and unconfirmed.
EV00012774	SRC00156/q4	covered	Original archive PDF page 4; candidate unit association is text-based and unconfirmed.
EV00012775	SRC00156/q5	covered	Original archive PDF page 5; candidate unit association is text-based and unconfirmed.
EV00012776	SRC00156/q7	covered	Original archive PDF page 7; candidate unit association is text-based and unconfirmed.
EV00012777	SRC00157/q1	covered	Original archive PDF page 1; candidate unit association is text-based and unconfirmed.
EV00012778	SRC00157/q2	covered	Original archive PDF page 2; candidate unit association is text-based and unconfirmed.

Evidence	Source unit	Status	Decision
EV00012 79	SRC00157/q3	covered	Original archive PDF page 3; candidate unit association is text-based and unconfirmed.
EV00012 80	SRC00157/q4	covered	Original archive PDF page 4; candidate unit association is text-based and unconfirmed.
EV00012 81	SRC00157/q5	covered	Original archive PDF page 5; candidate unit association is text-based and unconfirmed.
EV00012 82	SRC00157/q6	covered	Original archive PDF page 6; candidate unit association is text-based and unconfirmed.
EV00012 83	SRC00157/q7	covered	Original archive PDF page 7; candidate unit association is text-based and unconfirmed.
EV00012 84	SRC00158/q1	covered	Original archive PDF page 1; candidate unit association is text-based and unconfirmed.
EV00012 85	SRC00158/q2	covered	Original archive PDF page 2; candidate unit association is text-based and unconfirmed.
EV00012 86	SRC00158/q3	covered	Original archive PDF page 3; candidate unit association is text-based and unconfirmed.
EV00012 87	SRC00158/q6	covered	Original archive PDF page 6; candidate unit association is text-based and unconfirmed.

C Revision and retrieval practice